

# THUẬT TOÁN VÀ MÔ HÌNH

---



## TÓM TẮT

Phần C đi qua bảy họ thuật toán và mô hình, mỗi họ đọc như một chuỗi kỹ thuật ra đời đúng theo thứ tự để gỡ nút thắt của nhau: cơ chế huấn luyện, các họ backbone, ba bài toán thị giác lõi, thị giác chuỗi và video, thị giác 3D, mô hình sinh, và pretraining. Đọc xong bạn đoán được một kiến trúc lạ sẽ hồng ở đâu mà không cần chạy thử. Nếu chỉ nhớ một điều: đừng đọc phần này như danh mục kiến trúc — hãy đọc như chuỗi vấn đề, vì chuỗi vấn đề còn giá trị lâu sau khi tên mô hình đã cũ.

Đây là phần dài nhất của cây, và cũng là phần dễ đọc sai nhất. Đọc sai nghĩa là đọc nó như một danh mục: một danh sách kiến trúc kèm năm ra đời và điểm số trên bảng xếp hạng. Đọc như vậy thì kiến thức hết hạn cùng lúc với bảng xếp hạng. Cách đọc đúng là coi mỗi họ mô hình như *một chuỗi kỹ thuật ra đời theo đúng thứ tự để gỡ nút thắt của nhau*: mỗi bước giải quyết một khiếm khuyết cụ thể của bước trước và gần như luôn tạo ra một vấn đề mới, và chính chuỗi vấn đề đó mới là thứ còn giá trị sau mười năm.

Bảy chương được sắp theo thứ tự phụ thuộc chứ không theo độ hấp dẫn. Chương 12 là cơ chế huấn luyện, đi từ vi mô tới vĩ mô, và là chương quyết định nhiều nhất tới năng lực thực hành: nếu không hiểu vì sao gradient chết và vì sao learning rate là siêu tham số quan trọng nhất thì mọi kiến trúc phía sau chỉ là hộp đen. Chương 13 là các họ backbone, đọc như ba mạch song song — dòng CNN, dòng chuỗi tới attention, và ViT cùng bài học của nó về inductive bias. Chương 14 và 15 áp các backbone đó vào các lớp bài toán thật, trước là ảnh tĩnh rồi tới chuỗi và video, nơi thời gian thêm vào một chiều ràng buộc mới. Chương 16 là thị giác 3D, nơi hình học cổ điển và học sâu gặp nhau trên cùng một bài toán qua bốn thể hệ. Chương 17 là các mô hình sinh và low-level vision, chứa một trong vài kết quả *có tính lý thuyết* hiếm hoi của cả cây là trade-off giữa perceptual và distortion. Chương 18 khép lại bằng câu chuyện thống nhất phía sau mười năm qua: nguồn giám sát ngày càng rẻ, từ nhãn người gán tới chính bức ảnh, tới văn bản đi kèm, tới việc ghép thị giác vào một mô hình ngôn ngữ đã biết suy luận.

Xuyên suốt bảy chương, hãy giữ hai câu hỏi trong đầu ở mỗi kỹ thuật mới: *nó giả định gì*, và *độ khó đã bị đẩy đi đâu*. Khi một phương pháp trông như bữa trưa miễn phí thì gần như chắc chắn là bạn chưa tìm ra chỗ trả giá.

**Bảng 1.** Bản đồ Phần C: các chương, nội dung, và mức đầu tư đề xuất.

| Ch. | Thư mục                 | Nội dung                                                                                         | Mức                   |
|-----|-------------------------|--------------------------------------------------------------------------------------------------|-----------------------|
| 12  | 12-co-che-huan-luyen    | Bốn mục: cơ học của gradient; làm cho training chạy được; tối ưu hoá; tổng quát hoá và chẩn đoán | <b>cốt lõi</b>        |
| 13  | 13-kien-truc-backbone   | Ba mạch: dòng CNN; từ chuỗi tới attention; ViT và bài học inductive bias                         | <b>cốt lõi</b>        |
| 14  | 14-bai-toan-cv-loi      | Bốn mục: detection hai giai đoạn, một giai đoạn, segmentation, các bài toán lân cận              | <b>cốt lõi</b>        |
| 15  | 15-thi-giac-khong-thoi- | Ba mục: đầu ra độ dài thay đổi; biểu diễn video; flow, tracking và ràng buộc streaming           | <b>cốt lõi</b>        |
| 16  | 16-thi-giac-3d          | Ba mục: bốn thể hệ 3D; biểu diễn và depth và SLAM; NeRF, 3DGS, hợp nhất cảm biến                 | <b>cốt lõi</b>        |
| 17  | 17-mo-hinh-sinh         | Bốn mục: bốn họ sinh; tối ưu trên pixel; diffusion; low-level vision                             | tham khảo có chọn lọc |
| 18  | 18-pretraining-va-found | Hai mục: nguồn giám sát ngày càng rẻ; kinh tế học của pretraining                                | <b>cốt lõi</b>        |

Chương 12 trước hết và đọc kỹ nhất. Chương 13 là nền cho 14 tới 16. Chương 17 đọc theo nhu cầu. Chương 18 nên đọc sớm hơn vị trí của nó nếu bạn đang phải chọn giữa train từ đầu và dùng mô hình có sẵn.

# Cơ chế huấn luyện, từ vi mô tới vĩ mô (Training Mechanics, from Micro to Macro)

**PHỤ THUỘC** A/01-toan-toi-thieu (quy tắc chuỗi, phương sai, số điều kiện); B/09-thiet-ke-ham-mat-mat (biết mất mát là gì trước khi hỏi cách tối thiểu hoá nó). Chương này là tiền đề cho toàn bộ Phần C: mọi kiến trúc ở các chương sau đều được huấn luyện bằng bộ máy mô tả ở đây.

## TÓM TẮT

Cả chương là câu trả lời cho *một* câu hỏi: làm sao đưa **gradient** đi qua một mạng sâu mà không làm nó chết hoặc nổ. Gần như mọi thủ thuật trong một file cấu hình huấn luyện — từ cách khởi tạo, chọn hàm kích hoạt, đặt normalization, tới warmup và **mixed precision** — tồn tại vì lý do đó. Đọc xong bốn mục, bạn tự viết được lượt ngược cho một tầng mới và đặt được siêu tham số có căn cứ. Bạn cũng chẩn đoán được một lần chạy hỏng trong vài phút, và đưa được mô hình lên nhiều GPU đúng thứ tự. Nếu chỉ nhớ một điều: đây là chương duy nhất trong cây mà kỹ năng chỉ hình thành qua tay — đọc hiểu hết mà chưa từng cố ý làm hỏng một lần chạy thì coi như chưa học.

## KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình (model)** — một hàm có tham số, nhận đầu vào và trả về dự đoán; “huấn luyện” nghĩa là đi tìm bộ tham số làm dự đoán sát nhất.
- **tham số (parameter, weight)** — các con số bên trong mô hình mà huấn luyện được phép thay đổi; một mạng thị giác cỡ vừa có hàng chục triệu con số như vậy.
- **hàm mất mát (loss function)** — hàm biến dự đoán và nhãn thành một số duy nhất đo mức sai; toàn bộ huấn luyện là bài toán làm giảm số đó.
- **gradient** — vectơ đạo hàm riêng của hàm mất mát theo từng tham số; nó chỉ hướng làm mất mát TĂNG nhanh nhất, nên ta đi ngược lại.
- **batch và epoch** — “batch” là nhóm mẫu đi qua mạng cùng lúc; một “epoch” là khi mô hình đã đi hết tập huấn luyện một lượt. Một epoch gồm nhiều bước cập nhật, mỗi bước một batch.
- **learning rate** — độ dài bước mà thuật toán tối ưu đi theo hướng ngược gradient ở mỗi lần cập nhật; đây là siêu tham số quan trọng nhất trong cả chương.
- **kích hoạt và hàm kích hoạt** — “kích hoạt” là giá trị đầu ra của một tầng giữa mạng; “hàm kích hoạt” là phép phi tuyến sinh ra nó, ví dụ ReLU. Tên gần giống nhưng là hai thứ khác nhau.
- **normalization** — phép chuẩn hoá lại phân bố của kích hoạt ở giữa mạng về trung bình 0 độ lệch chuẩn 1; nó tồn tại để giữ tín hiệu không trôi khi đi qua nhiều tầng.
- **overfitting** — mô hình khớp rất tốt dữ liệu huấn luyện nhưng kém trên dữ liệu mới; đây là chủ đề của *mục 4*.

## CÂU HỎI MỞ ĐẦU

- Q1. Vì sao một mạng  $10^9$  tham số huấn luyện được trong vài ngày, trong khi vi phân thuận cần  $10^9$  lượt chạy để có cùng lượng thông tin?
- Q2. Nếu gradient chỉ là quy tắc chuỗi — thứ đã có từ thế kỷ XVII — vì sao suốt hai mươi năm người ta không huấn luyện nổi mạng sâu?

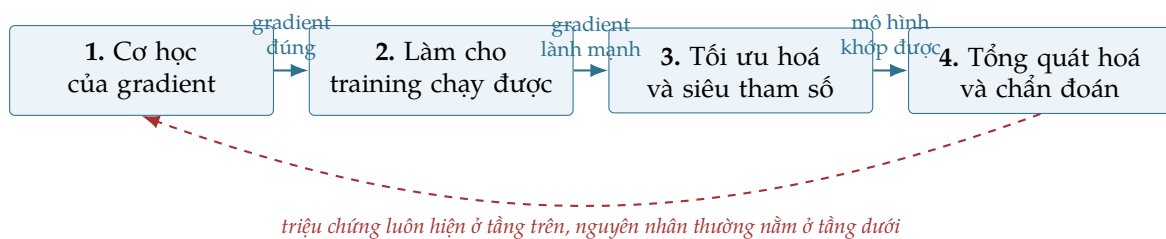
- Q3.** Bầy *optimizer* quen thuộc thường được dạy như bầy lựa chọn trong một menu. Vì sao thực ra chúng là bầy lần vá liên tiếp, và cái nào sinh ra chỉ để sửa lỗi do chính cái trước tạo ra?
- Q4.** Một mạng đủ sức học thuộc cả một tập ảnh với nhãn xáo ngẫu nhiên — vậy vì sao nó vẫn tổng quát hoá tốt khi nhãn là chân trị?
- Q5.** *bf16* có dải động rộng đúng bằng *fp32*. Vậy vì sao huấn luyện ở *bf16* vẫn buộc phải giữ một bản trọng số *fp32*?

## 1.1 Vì sao chương này chia làm bốn mục

Bốn mục dưới đây không phải bốn chủ đề song song. Chúng là bốn tầng của cùng một câu hỏi, xếp từ vi mô tới vĩ mô. Và *mỗi tầng chỉ có nghĩa khi tầng dưới nó đã đúng*. Đó là lý do thứ tự đọc ở đây là bắt buộc chứ không phải gợi ý.

- 1. Cơ học của gradient** hỏi: gradient được tính thế nào, và vì sao cách tính đó rẻ. Đây là tầng vi mô nhất — một phép toán, một tầng, một đồ thị. Không nắm tầng này thì ba tầng sau chỉ là các quy tắc phải học thuộc.
- 2. Làm cho training chạy được** hỏi: gradient đúng rồi thì làm sao nó sống sót qua năm mươi tầng. Ở đây xuất hiện đại lượng trung tâm của cả chương — hệ số nhân của tích các Jacobian — và ba hướng tấn công vào nó.
- 3. Tối ưu hoá và siêu tham số** giả định gradient đã lành mạnh và hỏi tiếp: nên đi bước dài bao nhiêu, theo hướng nào, ở độ chính xác số nào. Đây là tầng mà bầy *optimizer* quen thuộc hiện ra như một chuỗi vá lỗi chứ không như một menu.
- 4. Tổng quát hoá và chẩn đoán** là tầng vĩ mô nhất: mô hình chạy tốt trên dữ liệu huấn luyện rồi, còn ngoài đời thì sao, và khi mọi thứ không như kỳ vọng thì tìm nguyên nhân theo trình tự nào.

Hình 1.1 cho thấy vì sao thứ tự này không đảo được. Mỗi tầng tiêu thụ đúng thứ mà tầng dưới sản xuất ra. Một lỗi ở tầng dưới lại luôn hiện ra thành triệu chứng ở tầng trên. Đó chính là lý do người ta hay tune learning rate để chữa một bug ở lượt ngược.



**Hình 1.1.** Bốn tầng của chương, và chiều mà lỗi lan truyền. Mũi tên liền là quan hệ “tầng sau giả định tầng trước đã đúng”; mũi tên đứt là điều khiến việc gỡ lỗi khó: một bug ở tầng 1 hầu như luôn biểu hiện thành một đường cong xấu ở tầng 4.

## Bản đồ chương 12 — Cơ chế huấn luyện

**TÓM TẮT**

Chương 12 gồm bốn mục và chúng *không* song song: đây là bốn tầng của cùng một câu hỏi, xếp từ vi mô tới vĩ mô, mỗi tầng giả định tầng dưới đã đúng. Đọc theo đúng thứ tự 1 → 2 → 3 → 4; đọc nhảy cóc thì mục 3 và mục 4 biến thành danh sách quy tắc phải học thuộc. Nếu chỉ nhớ một điều: câu hỏi trung tâm của cả chương là *làm sao đưa gradient đi qua một mạng sâu mà không làm nó chết hoặc nổ*. Mọi kỹ thuật trong chương là một câu trả lời cho đúng câu hỏi đó.

**KIẾN THỨC CẦN CÓ TRƯỚC**

- **gradient** — vectơ đạo hàm riêng của hàm mất mát theo từng tham số; nó chỉ hướng làm mất mát TĂNG nhanh nhất, nên ta đi ngược lại.
- **lan truyền ngược (backpropagation)** — thuật toán tính gradient bằng cách đi ngược đồ thị phép tính một lượt duy nhất; nội dung của mục 1.
- **siêu tham số (hyperparameter)** — con số do người đặt trước khi huấn luyện chứ không học từ dữ liệu — learning rate, batch size, cường độ weight decay; nội dung của mục 3.
- **tổng quát hoá (generalization)** — khả năng mô hình chạy tốt trên dữ liệu chưa từng thấy, chứ không chỉ trên dữ liệu đã huấn luyện; nội dung của mục 4.

**Có gì trong chương này**

**Bảng 1.1.** Bốn mục của chương 12: câu hỏi trung tâm, phụ thuộc, và mức độ đầu tư.

| Mục                           | Câu hỏi trung tâm                                                             | Phụ thuộc   | Mức độ       |
|-------------------------------|-------------------------------------------------------------------------------|-------------|--------------|
| 1. Cơ học của gradient        | Gradient được tính thế nào, vì sao cách đó rẻ, và nó tồn bộ nhớ ở đâu?        | A/01        | Cốt lõi [T1] |
| 2. Làm cho training chạy được | Gradient đúng rồi thì làm sao nó sống sót qua năm mươi tầng?                  | Mục 1       | Cốt lõi [T1] |
| 3. Tối ưu hoá và siêu tham số | Đi bước dài bao nhiêu, theo hướng nào, ở độ chính xác số nào?                 | Mục 2       | Cốt lõi [T2] |
| 4. Tổng quát hoá và chẩn đoán | Mô hình khớp được dữ liệu rồi, còn ngoài đời thì sao — và hỏng thì tìm ở đâu? | Mục 3, B/07 | Cốt lõi [T2] |

**Vì sao lại chia như vậy**

Cách chia này theo *độ trừu tượng của đối tượng được nói tới*, chứ không theo thứ tự lịch sử hay theo mức độ phổ biến. Mục 1 nói về một phép toán và một đồ thị; mục 2 nói về một mạng; mục 3 nói về một quá trình huấn luyện; mục 4 nói về quan hệ giữa quá trình đó và thế giới bên ngoài. Lợi ích của cách chia này là mỗi mục có một đại lượng trung tâm rõ ràng. Lần lượt: tích vector-Jacobian, hệ số nhân của tích các Jacobian, độ dài bước đi, và khoảng cách train-val. Gặp một kỹ thuật lạ, bạn biết ngay nó thuộc tầng nào.

Cách chia này cũng làm lộ ra một quan hệ mà cách chia theo chủ đề che mất: *triệu chứng luôn xuất hiện ở tầng trên, nguyên nhân thường nằm ở tầng dưới*. Một lỗi ở lượt ngược (tầng 1) biểu hiện thành một đường cong học xấu (tầng 4). Đây là lý do mục 4 dành phần lớn dung lượng cho một cây chẩn đoán đi từ trên xuống. Thấy một đường cong lạ thì phản xạ đúng là đi xuống, không đi ngang.

## Đọc theo thứ tự nào

1. Nếu bạn mới bắt đầu: đọc tuần tự  $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$ , và làm mini project của chương *ngay sau mục 4* chứ không để dành — nó là chỗ kiến thức chuyển thành phản xạ.
2. Nếu bạn đã huấn luyện mạng vài lần và muốn lấp lỗ hổng: đọc mục 3 và mục 4 trước, quay lại mục 1 khi gặp câu hỏi về bộ nhớ hoặc khi phải viết một tầng mới.
3. Nếu bạn đang gỡ một lần chạy hỏng *ngay bây giờ*: vào thẳng cây chẩn đoán và bảng triệu chứng ở mục 4, rồi đi ngược lên tầng mà cây chỉ tới.

## Cốt lõi và tham khảo

Cả bốn mục đều thuộc phần cốt lõi của cây, đây là một trong ít chương như vậy. Trong nội bộ chương, ba phần đáng quay lại nhiều lần là: ba mẫu gradient cộng quy tắc rẽ nhánh (mục 1), lập luận phương sai dẫn tới khởi tạo He (mục 2), và mạch bảy bước của optimizer (mục 3). Ba phần có thể đọc lướt lần đầu và tra lại khi cần là: chi tiết công thức lượt ngược của BatchNorm, bảng bốn biến thể normalization, và bảng bốn mức song song hoá dữ liệu.

## Nối vào phần còn lại của cây

Chương này tiêu thụ A/01-toan-toi-thieu và B/09-thiet-ke-ham-mat-mat; nó cấp đầu vào cho *toàn bộ* Phần C, vì mọi kiến trúc ở các chương sau đều được huấn luyện bằng bộ máy này. Ba nhánh nối ngang đáng nhớ: chi phí bộ nhớ kích hoạt nối sang B/11-gioi-han-tinh-toan; dải động của fp16 và bf16 nối sang D/20-toi-uu-va-nen-mo-hinh; và nhận xét “phần lớn cải tiến kiến trúc hoá ra là cải tiến công thức huấn luyện” nối sang E/24-thi-nghiem-va-ablation.

## 1.2 Cơ học của gradient (Mechanics of the Gradient)

**PHỤ THUỘC** A/01-toan-toi-thieu (quy tắc chuỗi, Jacobian, chuẩn ma trận); hữu ích nhưng không bắt buộc: B/11-gioi-han-tinh-toan nếu muốn hiểu vì sao chi phí bộ nhớ ở cuối mục này lại là ràng buộc thật chứ không phải chi tiết phụ.

### TÓM TẮT

Mục này giải thích vì sao thứ tự áp dụng quy tắc chuỗi quyết định chi phí huấn luyện, và vì sao **lan truyền ngược** (*backpropagation*) rẻ hơn vì phân thuận khoảng tám bậc độ lớn trên một mạng thật. Đọc xong bạn tự viết được lượt ngược cho một tầng bất kỳ. Bạn cũng tự bắt được lỗi của nó bằng kiểm tra gradient số, và ước lượng được bộ nhớ mà kiến trúc sẽ ngốn. Nếu chỉ nhớ một điều: lan truyền ngược không phải công thức riêng của mạng nơ-ron, nó là thuật toán vi phân cho mọi đồ thị khả vi, nên mọi kiến trúc lạ chỉ còn là một đồ thị mới.

### KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình (model)** — một hàm có tham số, nhận đầu vào và trả về dự đoán; “huấn luyện” nghĩa là đi tìm bộ tham số làm dự đoán sát nhãn nhất.
- **tham số (parameter, weight)** — các con số bên trong mô hình mà quá trình huấn luyện được phép thay đổi; một mạng thị giác cỡ vừa có hàng chục triệu con số như vậy.
- **bias** — **phân biệt hai nghĩa** — trong công thức  $Wx + b$ ,  $b$  là *tham số cộng* của một tầng. Từ “bias” trong thống kê lại nghĩa là *độ lệch hệ thống* của một bộ ước lượng. Mục này chỉ dùng nghĩa thứ nhất.

- **hàm mất mát (loss function)** — hàm biến dự đoán và nhả thành một số duy nhất đo mức sai; mọi thứ trong huấn luyện đều nhằm giảm số đó.
- **gradient** — vectơ đạo hàm riêng của hàm mất mát theo từng tham số; nó chỉ hướng làm mất mát TĂNG nhanh nhất, nên ta đi ngược lại.
- **Jacobian** — ma trận đạo hàm riêng của một hàm nhiều đầu vào, nhiều đầu ra; nó là dạng tổng quát của gradient và là đối tượng chính mà lượt ngược thao tác.
- **tensor** — mảng số nhiều chiều — một ảnh RGB là tensor ba chiều, một lô ảnh là tensor bốn chiều; đây là đơn vị dữ liệu mà mọi phép toán trong mạng làm việc với.
- **batch (lô)** — nhóm mẫu được đưa qua mạng cùng lúc để tận dụng phần cứng song song; kích thước lô ảnh hưởng tới cả tốc độ lẫn chất lượng, và ta sẽ thấy vì sao ở các mục sau.
- **kích hoạt (activation)** — giá trị đầu ra của một tầng ở giữa mạng, tức dữ liệu đang trên đường đi; đừng nhầm với “hàm kích hoạt” là phép phi tuyến sinh ra nó.

## A. Đặt vấn đề

Một mạng hiện đại là hàm hợp của vài nghìn phép toán sơ cấp. Nó phụ thuộc vào  $10^7$  đến  $10^9$  tham số vô hướng. Mỗi bước huấn luyện lại cần đạo hàm của *một* số duy nhất — giá trị **hàm mất mát (loss function)** — theo *toàn bộ* chỗ tham số đó. Câu hỏi không phải “quy tắc chuỗi có áp dụng được không”, vì nó luôn áp dụng được. Câu hỏi là: áp dụng quy tắc chuỗi *theo thứ tự nào* để chi phí là một lượt đi thay vì một trăm triệu lượt. Toàn bộ nội dung của **lan truyền ngược (backpropagation)** nằm trong câu hỏi thứ tự đó, chứ không nằm trong bản thân đạo hàm.

Bạn đã gặp mọi nguyên liệu của bài toán này ở phía tối ưu hoá phi tuyến trong SLAM: dựng Jacobian, nhân dồn các Jacobian cục bộ, giải hệ chuẩn tắc. Nhưng hình dạng bài toán ở đó ngược hẳn. Trong bundle adjustment, số dư (*residual*) rất nhiều còn tham số tương đối ít, nên việc dựng tường minh ma trận  $J$  rồi làm việc với  $J^T J$  là hợp lý. Trong huấn luyện mạng, đầu ra chỉ có *một* chiều còn đầu vào có hàng trăm triệu chiều. **Forward-mode autodiff** lan truyền một đạo hàm theo hướng: mỗi lượt cho bạn đạo hàm theo đúng một hướng đầu vào, nên cần  $10^8$  lượt. **Reverse-mode autodiff** lan truyền độ nhạy ngược từ đầu ra: mỗi lượt cho bạn đạo hàm theo *tất cả* đầu vào, nên cần đúng một lượt cho mỗi đầu ra vô hướng. Tỉ số  $10^8$  giữa hai cách là lý do tồn tại của lan truyền ngược [1]. Cùng một quy tắc chuỗi, hai thứ tự áp dụng, chi phí lệch nhau tám bậc độ lớn.

Mục này cho bạn ba việc cụ thể. Bạn tự viết được lượt ngược cho một tầng chưa có trong thư viện. Bạn tự bắt được lỗi của nó bằng kiểm tra số. Và bạn ước lượng được bộ nhớ mà một kiến trúc sẽ ngốn. Trước đó cả ba đều phải tin vào framework.

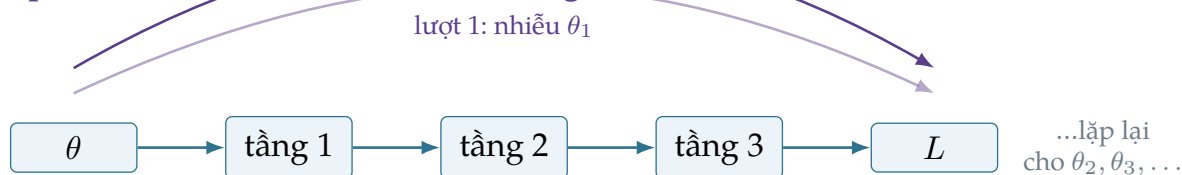
**Bảng 1.2.** Hai chế độ vi phân tự động. Chúng dùng đúng cùng một quy tắc chuỗi và chỉ khác nhau ở thứ tự nhân các Jacobian, nhưng thứ tự đó quyết định toàn bộ chi phí.

| Chế độ       | Cơ chế                                                          | Chi phí cho $n$ đầu vào, $m$ đầu ra   | Hỏng / không dùng khi                                                |
|--------------|-----------------------------------------------------------------|---------------------------------------|----------------------------------------------------------------------|
| Forward-mode | Lan truyền đạo hàm theo một hướng đầu vào, cùng chiều lượt xuôi | $n$ lượt; bộ nhớ hằng số              | $n$ lớn — huấn luyện mạng có $10^8$ tham số là bất khả thi           |
| Reverse-mode | Lan truyền độ nhạy của một đầu ra, ngược chiều lượt xuôi        | $m$ lượt; bộ nhớ tỉ lệ tổng kích hoạt | $m$ lớn (cần cả Jacobian đầy đủ), hoặc bộ nhớ không đủ giữ kích hoạt |

Bảng 1.2 nói gọn lại: huấn luyện là bài toán “một đầu ra, rất nhiều đầu vào”, và đó chính xác là hình dạng mà reverse-mode được sinh ra để phục vụ.

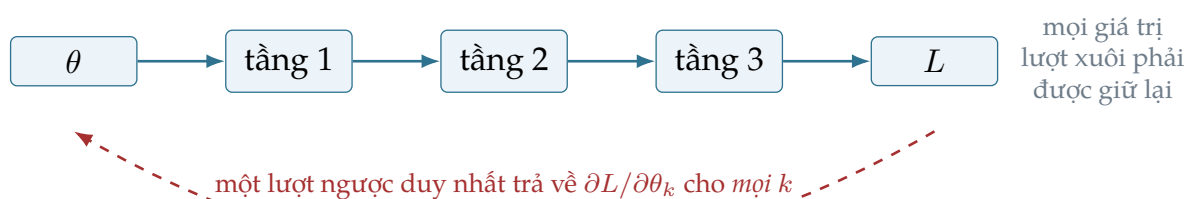


### Vi phân thuận: một lượt cho *mỗi* hướng đầu vào



**Chi phí:**  $n$  lượt cho  $n$  tham số — với  $n = 10^8$  là bất khả thi. Bộ nhớ: hằng số.

### Vi phân ngược: một lượt xuôi, rồi *một* lượt ngược cho tất cả



**Chi phí:** 1 lượt — nhưng bộ nhớ tỉ lệ với tổng kích hoạt của cả mạng.

**Hình 1.2.** Cùng một quy tắc chuỗi, hai thứ tự nhân, hai chi phí lệch nhau tám bậc độ lớn. Vi phân thuận đi cùng chiều lượt xuôi nên mỗi lượt chỉ trả lời được cho một hướng đầu vào; vi phân ngược đi ngược lại nên một lượt phủ mọi tham số. Điểm đáng nhớ nằm ở dòng cuối mỗi nửa hình: cái ta tiết kiệm được ở thời gian phải trả lại bằng bộ nhớ.

Hình 1.2 cho thấy điều mà bảng không nói được: hai chế độ dùng *cùng* một đồ thị và cùng các Jacobian cục bộ, chỉ khác chiều duyệt — và chính chiều duyệt sinh ra cả lợi thế lẫn cái giá.

Nhận thức then chốt cần mang ra khỏi mục này là: lan truyền ngược không phải công thức riêng của mạng nơ-ron. Nó là thuật toán chung cho *mọi* đồ thị có hướng không chu trình mà mỗi nút khả vi. Khi bạn đã tin điều đó, một kiến trúc lạ không còn là một thứ phải học thuộc; nó chỉ là một đồ thị mới, và bạn đã biết cách vi phân mọi đồ thị.

#### TRỰC GIÁC

Hãy nghĩ về forward pass như một dây chuyền sản xuất: nguyên liệu đi qua ba mươi trạm, mỗi trạm biến đổi một chút, cuối dây chuyền ra một sản phẩm có một con số sai lệch so với chuẩn. Forward-mode giống việc rung nhẹ *một* nút vặn rồi chạy lại cả dây chuyền để xem sai lệch đổi bao nhiêu. Cách đó đúng, nhưng phải lặp lại đúng bằng số nút vặn. Reverse-mode làm ngược: bắt đầu từ sản phẩm lỗi, hỏi trạm cuối “một đơn vị sai ở đây tương ứng với bao nhiêu sai ở đầu vào của anh”, rồi chuyển đúng câu hỏi đó ngược lên trạm trước. Một lượt đi ngược duy nhất trả lời cho mọi nút vặn trên toàn dây chuyền, vì mỗi trạm chỉ cần biết *quy tắc quy đổi cục bộ* của riêng nó.

### B. Đồ thị tính toán và tích vector-Jacobian (computational graph, VJP)

Hình thức hoá của ẩn dụ trên rất gọn. Biểu diễn phép tính thành một đồ thị: nút là giá trị trung gian, cạnh là phụ thuộc. Lượt xuôi duyệt đồ thị theo thứ tự tô-pô và tính giá trị; lượt ngược duyệt theo thứ tự tô-pô đảo và tính *adjoint*  $\bar{v} = \partial L / \partial v$  cho từng nút. Mỗi nút chỉ cần biết một thứ: cách biến adjoint ở đầu ra của nó thành adjoint ở đầu vào của nó. Nếu nút thực hiện  $y = g(x)$  thì quy



tắc đó là

$$\bar{x} = \left( \frac{\partial y}{\partial x} \right)^\top \bar{y},$$

tức là một **tích vector-Jacobian** (*vector-Jacobian product, VJP*). Điểm quan trọng về mặt kỹ thuật là ta *không bao giờ* dựng ma trận Jacobian ở giữa. Xét một tầng affine  $y = Wx + b$  với  $W \in \mathbb{R}^{4096 \times 4096}$ . Jacobian theo  $W$  là ma trận  $4096 \times 16,8$  triệu, khoảng  $6,9 \cdot 10^{10}$  phần tử. Không thể lưu nổi. Nhưng VJP của nó chỉ là ba biểu thức tầm thường:

$$\bar{x} = W^\top \bar{y}, \quad \bar{W} = \bar{y} x^\top, \quad \bar{b} = \bar{y},$$

trong đó  $\bar{W}$  là một tích ngoài với đúng 16,8 triệu phần tử — bằng kích thước của chính  $W$ . Tỉ số 4096 giữa “dựng Jacobian” và “áp dụng Jacobian” là lý do mọi framework *autodiff* đều cài đặt VJP chứ không cài đặt đạo hàm theo nghĩa ma trận. Với một batch  $X$  xếp theo hàng, quy tắc trên trở thành  $\bar{W} = \bar{Y}^\top X$ , và bạn nên coi đây là công thức phải viết lại được từ đầu bất cứ lúc nào.

↪ **trường hợp riêng của A/01** — lan truyền ngược chỉ là quy tắc chuỗi của giải tích nhiều biến, áp dụng theo thứ tự làm chi phí rẻ nhất

↪ **tiền đề C/12/02** — chính tích các Jacobian cục bộ ở đây là đại lượng mà ba hướng chống gradient tiêu biến nhắm vào

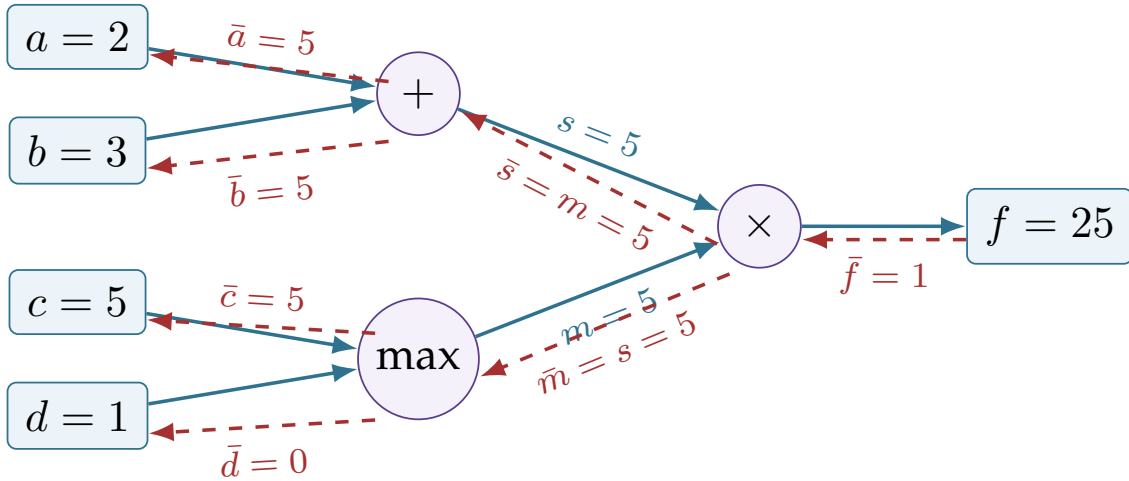
### C. Ba mẫu gradient lặp lại (the three gradient patterns)

Nghe qua thì việc phải derive backward cho từng loại tầng là công việc vô tận. Thực tế thì không, vì gần như mọi đồ thị đều được ghép từ ba mẫu, và ba mẫu đó có tên gọi trực quan.

- **Cộng là bộ phân phối** (*add = distributor*) — nếu  $y = u + v$  thì  $\bar{u} = \bar{v} = \bar{y}$ . Nói gọn: adjoint đi qua nút cộng mà không đổi giá trị, chỉ nhân bản ra mọi nhánh.
- **Nhân là bộ hoán đổi** (*multiply = swapper*) — nếu  $y = uv$  thì  $\bar{u} = \bar{v} \bar{y}$  và  $\bar{v} = \bar{u} \bar{y}$ . Mỗi nhánh nhận adjoint nhân với giá trị của *nhánh kia*; đây chính là lý do lượt ngược buộc phải nhớ các giá trị của lượt xuôi, và do đó là gốc rễ của chi phí bộ nhớ ở mục G.
- **Max là bộ định tuyến** (*max = router*) — nếu  $y = \max(u, v)$  thì toàn bộ adjoint đi về nhánh thắng, nhánh thua nhận 0. Nói gọn: max không trộn, nó chọn đường.
- **Rẽ nhánh là bộ cộng dồn** (*fan-out = accumulator*) — mẫu thứ tư, ít được nêu tên nhưng gây nhiều lỗi nhất. Nếu một giá trị được dùng ở hai chỗ trong đồ thị, adjoint của nó là *tổng* các adjoint quay về từ hai chỗ đó.

Mẫu thứ tư đáng được nói thêm một câu vì hệ quả thực hành của nó rất rộng: *lượt ngược của broadcasting là một phép cộng theo đúng trục đã được broadcast*. Cộng một vector bias shape  $(C,)$  vào một *tensor* shape  $(N, C, H, W)$  chính là rẽ nhánh giá trị bias ra  $N \cdot H \cdot W$  chỗ, nên  $\bar{b} = \sum_{n,h,w} \bar{y}_{n,c,h,w}$ . Một mình quy tắc này giải thích phần lớn lỗi shape mà người mới gặp khi tự viết lượt ngược.

Hình 1.3 chạy cả ba mẫu trên một ví dụ đủ nhỏ để tính tay. Với  $f = (a + b) \cdot \max(c, d)$  và  $a = 2, b = 3, c = 5, d = 1$ , lượt xuôi cho  $s = 5, m = 5, f = 25$ . Lượt ngược đặt  $\bar{f} = 1$ ; nút nhân hoán đổi nên  $\bar{s} = m = 5$  và  $\bar{m} = s = 5$ ; nút cộng phân phối nên  $\bar{a} = \bar{b} = 5$ ; nút max định tuyến nên  $\bar{c} = 5$  và  $\bar{d} = 0$ . Kiểm tra bằng trực giác: tăng  $d$  một chút không đổi gì cả vì  $d$  đang thua, đúng như  $\bar{d} = 0$ ; nhưng nếu  $d$  vượt  $c$  thì đạo hàm nhảy bậc, và đó chính là chỗ kiểm tra gradient bằng số sẽ báo động giả ở phần sau.



**Hình 1.3.** Ba mẫu gradient trên một đồ thị bốn nút. Mũi tên liền là lượt xuôi, mũi tên đứt là lượt ngược. Nút cộng phân phối adjoint, nút nhân hoán đổi giá trị hai nhánh, nút max định tuyến toàn bộ adjoint về nhánh thắng và cắt nhánh thua về 0.

#### D. Bốn backward đáng tự viết một lần trong đời

Việc tự derive gradient cho vài tầng cụ thể là bài tập kiểu CS231n, và nó nằm ở tầng bán rã [T2]–[T3]: không phải kiến thức vĩnh cửu, nhưng cũng không phải chi tiết framework sẽ mất giá sau một năm. Giá trị của nó là sau khi làm xong, bạn đọc bất kỳ tầng mới nào cũng tự hỏi “VJP của cái này là gì” thay vì “thư viện có hỗ trợ không”.

Tầng affine đã nêu ở trên. **ReLU** là mẫu max trá hình:  $\bar{x} = \bar{y} \cdot \mathbb{I}[x > 0]$ , và lượt ngược chỉ cần một mặt nạ bit. **Softmax ghép cross-entropy** là ví dụ đẹp nhất về việc ghép hai phép toán làm backward súp đổ: tính riêng thì Jacobian của softmax là  $\text{diag}(p) - pp^\top$ , một ma trận  $K \times K$  phải nhân với adjoint; ghép lại thì tất cả rút gọn thành  $\bar{z} = p - y$ , vừa rẻ hơn vừa ổn định hơn về số học. **Max pooling** định tuyến adjoint về vị trí argmax nên phải lưu chỉ số argmax từ lượt xuôi — một chi phí bộ nhớ thật — trong khi average pooling chỉ phân phối  $1/k^2$  và không cần nhớ gì.

**BatchNorm** là bài khó nhất và cũng bổ ích nhất, vì nó là tầng đầu tiên mà gradient của một mẫu phụ thuộc vào *mọi mẫu khác trong batch*. Chuỗi phép toán là trung bình  $\rightarrow$  phương sai  $\rightarrow$  chuẩn hoá  $\rightarrow$  scale-and-shift, và vì  $\mu$  lẫn  $\sigma^2$  đều là hàm của cả batch nên adjoint chạy ngược qua ba đường song song rồi cộng lại. Kết quả gọn có dạng

$$\bar{x}_i = \frac{\gamma}{N\sigma} \left( N \bar{\hat{x}}_i - \sum_j \bar{\hat{x}}_j - \hat{x}_i \sum_j \bar{\hat{x}}_j \hat{x}_j \right),$$

với  $\hat{x}$  là giá trị đã chuẩn hoá. Hai tổng trong ngoặc chính là hai đường phụ thuộc qua  $\mu$  và qua  $\sigma^2$ . Công thức này giải thích luôn một câu hỏi hay bị hỏi: vì sao BatchNorm làm lượt ngược tốn thêm bộ nhớ. Vì để tính được nó, lượt xuôi phải giữ lại  $\hat{x}$  và  $\sigma$  cho mỗi tầng BN, chứ không thể vứt đi sau khi dùng.

↪ **ràng buộc bởi C/12/04** — việc gradient một mẫu phụ thuộc cả batch là gốc của mọi rắc rối BatchNorm trong huấn luyện phân tán

#### E. Kiểm tra gradient bằng số (numerical gradient check)

Derive tay thì sai lúc nào không biết, và điều tệ là một gradient sai *vẫn làm loss giảm* — chỉ là giảm chậm hơn hoặc hội tụ về chỗ khác. Nên cần một phép kiểm tra độc lập. Công cụ là **sai phân hữu**

**hạn** (*finite difference*) đối xứng

$$\frac{\partial f}{\partial x_k} \approx \frac{f(x + he_k) - f(x - he_k)}{2h},$$

chứ không phải sai phân một phía. Lý do là khai triển Taylor: sai phân một phía có sai số bậc  $O(h)$ , sai phân đối xứng triệt tiêu số hạng bậc hai nên sai số bậc  $O(h^2)$ . Với  $h = 10^{-5}$ , đó là chênh lệch giữa sai số cỡ  $10^{-5}$  và cỡ  $10^{-10}$  — đủ để phân biệt “code sai” với “xấp xỉ số”.

Tiêu chí so sánh phải là sai số tương đối  $|a - n| / \max(|a|, |n|)$  chứ không phải sai số tuyệt đối, vì độ lớn gradient thay đổi hàng chục bậc giữa các tầng. Kinh nghiệm thực hành phổ biến: dưới  $10^{-7}$  là tốt, quanh  $10^{-4}$  là đáng nghi trừ khi hàm có điểm gãy, trên  $10^{-2}$  gần như chắc chắn sai. Ba điều kiện phải thoả thì con số đó mới có nghĩa. Thứ nhất, phải tính bằng *double*: với float32, epsilon máy khoảng  $1,2 \cdot 10^{-7}$ , nên phép trừ hai giá trị gần nhau ở tử số mất gần hết chữ số có nghĩa và bạn đang đo nhiễu làm tròn chứ không đo đạo hàm. Thứ hai, phải tắt mọi nguồn ngẫu nhiên — dropout, augmentation, thứ tự batch — hoặc cố định seed, vì  $f$  phải là một hàm tắt định thì sai phân mới có nghĩa. Thứ ba, chỉ kiểm vài chục toạ độ ngẫu nhiên, không kiểm hết, vì mỗi toạ độ tốn hai lượt xuôi.

Điểm gãy là cái bẫy lớn nhất. Giả sử  $x_k$  rất gần 0 ở đầu vào một ReLU. Khi đó  $x_k + h$  và  $x_k - h$  nằm hai bên điểm gãy. Sai phân số trả về một giá trị trung gian, còn gradient giải tích trả về 0 hoặc 1. Kiểm tra “thất bại” dù code hoàn toàn đúng. Cách xử lý là dùng ít điểm dữ liệu, chọn toạ độ ở xa điểm gãy, và diễn giải một vài lần thất bại cô lập một cách thận trọng thay vì lao đi viết lại backward. Kỹ năng này không hề mất giá theo thời gian: mỗi lần bạn viết một autograd.Function riêng hay một kernel CUDA cho một phép toán mới, việc đầu tiên nên làm là chạy `torch.autograd.gradcheck` với đầu vào double.

## F. Vectorization, và vì sao nó không chỉ là chuyện tốc độ

Bỏ vòng lặp Python khỏi loss và gradient là bài tập kinh điển. Lợi ích hiển nhiên là tốc độ, nhưng chi phí không nằm ở FLOPs. Nó nằm ở phần trên đầu của mỗi lời gọi: thông dịch Python cộng chi phí phóng một kernel, cỡ vài micro-giây. Một nghìn phép toán tí hon vì thế tiêu tốn vài mili-giây trước khi làm phép nhân nào. Đây đúng là hiện tượng bạn đã đo khi tối ưu SLAM: nghẽn ở số lời gọi chứ không ở số phép tính. Nhưng lợi ích thứ hai mới đáng giá lâu dài: viết dạng vector buộc bạn phát biểu rõ shape của mọi thứ, do đó phát biểu rõ *ai rẽ nhánh sang ai*, và khi đó backward gần như tự lộ ra từ ba mẫu ở trên. Đoạn dưới là toàn bộ forward và backward của softmax ghép cross-entropy cho một batch.

```
# Z: (N, K) logits, y: (N,) nhãn dung
Z = Z - Z.max(axis=1, keepdims=True) # on định so: tranh exp tràn
P = np.exp(Z); P /= P.sum(axis=1, keepdims=True)
loss = -np.log(P[np.arange(N), y]).mean()

dZ = P.copy()
dZ[np.arange(N), y] -= 1.0 # dZ = p - y
dZ /= N # vì loss lay trung bình theo batch
dW = X.T @ dZ # VJP của tang affine
db = dZ.sum(axis=0) # backward của broadcast = cong theo trục
```

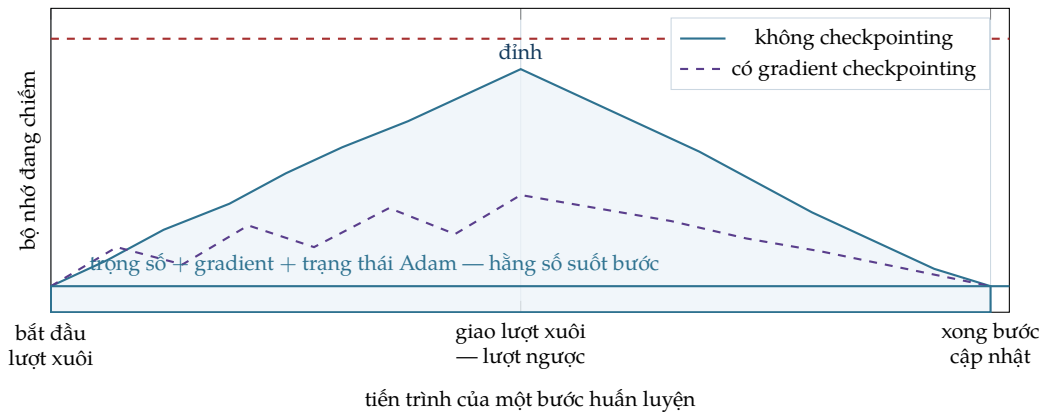
Hai dòng cuối minh hoạ chính xác hai quy tắc đã nêu: tích ngoài cho ma trận trọng số, và tổng theo trục batch cho bias.

## G. Cái giá của reverse mode: bộ nhớ kích hoạt

Reverse-mode rẻ về thời gian nhưng không miễn phí, và cái giá của nó là bộ nhớ. Mẫu nhân đòi hỏi giá trị của nhánh kia. Nên lượt ngược cần phần lớn giá trị trung gian của lượt xuôi. Bộ nhớ vì thế tăng theo tổng kích thước **kích hoạt** (*activation*) của cả mạng, không theo số tham số. Một

phép tính cụ thể cho thấy bậc độ lớn. Lấy đầu ra của một tầng conv ở batch 32, 64 kênh, độ phân giải  $112 \times 112$ , lưu ở fp32. Nó chiếm  $32 \cdot 64 \cdot 112 \cdot 112 \cdot 4 \approx 1,03 \cdot 10^8$  byte, tức khoảng 98 MiB cho một tầng. Một backbone vài chục tầng nhanh chóng đưa con số này lên vài GB. Nên trong phần lớn bài huấn luyện thị giác, thứ quyết định batch size tối đa là bộ nhớ chứ không phải tính toán.

→ **ràng buộc bởi** B/11 — bộ nhớ kích hoạt là đại lượng chặn batch size, và chương giới hạn tính toán do nó cùng với băng thông trên mô hình roofline



**Hình 1.4.** Bộ nhớ trong một bước huấn luyện. Phần không đổi là trọng số, gradient và trạng thái optimizer; phần dâng lên rồi rút xuống là kích hoạt, và nó đạt đỉnh đúng ở chỗ lượt xuôi giao lượt ngược — vì đó là lúc mọi giá trị trung gian đã được sinh ra mà chưa có giá trị nào được giải phóng. Hai hệ quả đọc thẳng từ hình: batch size tối đa bị quyết định bởi một thời điểm chứ không phải bởi mức trung bình, và checkpointing hạ đỉnh bằng cách vớt rồi tính lại, đổi lấy phần diện tích tăng thêm ở lượt ngược. Các con số là minh họa định tính, không phải số đo.

Hình 1.4 giải thích một điều mà con số 98 MiB ở trên không nói: cái chặn bạn không phải tổng bộ nhớ trung bình mà là *đỉnh tức thời* tại điểm giao giữa hai lượt. Đó cũng là lý do một mô hình chạy được ở batch 16 có thể sập ở batch 17 chứ không xuống cấp dần.

Đây là một đánh đổi thời gian-bộ nhớ quen thuộc, và nó có lời giải quen thuộc. **Gradient checkpointing** [2] chỉ lưu một tập con các nút, rồi khi lượt ngược cần một giá trị đã bị vớt thì tính lại nó từ điểm lưu gần nhất; với cách chọn điểm lưu hợp lý, bộ nhớ giảm từ  $O(L)$  xuống  $O(\sqrt{L})$  với giá khoảng một lượt xuôi phụ, tức chừng 30 phần trăm thời gian. Ở đầu kia của phổ, forward-mode có bộ nhớ hằng số nhưng cần một lượt cho mỗi hướng đầu vào, nên nó không dùng để huấn luyện mà dùng cho tích Jacobian-vector, chẳng hạn khi cần tích Hessian-vector trong các phương pháp bậc hai.

## H. Giới hạn và trường hợp hỏng

Khung đồ thị có ba chỗ gãy đáng biết trước.

Thứ nhất là *điểm không khả vi*. ReLU tại 0 và max khi hai nhánh bằng nhau không có đạo hàm; framework chọn một quy ước, thường là 0 cho ReLU và “nhánh chỉ số nhỏ hơn thắng” cho max. Quy ước đó vô hại về mặt tối ưu hoá nhưng có hệ quả về tính tái lập: khi hai giá trị bằng nhau tới từng bit, việc ai thắng phụ thuộc thứ tự duyệt, và thứ tự duyệt có thể phụ thuộc lịch trình luồng. Đây đúng là loại nguồn bất định mà bạn đã truy trong hệ SLAM đa luồng, chỉ là xuất hiện ở một tầng khác.

Thứ hai là *đồ thị bị cắt mà không ai báo*. Một phép gán tại chỗ ghi đè giá trị mà lượt ngược cần sẽ cho gradient sai; PyTorch bắt được phần lớn trường hợp này bằng bộ đếm phiên bản tensor, nhưng một kernel tự viết thì không có ai bảo vệ và bạn nhận gradient sai một cách im lặng. Cùng loại đó là việc vô tình gọi `detach()` hoặc đặt một nhánh trong `no_grad()`: loss vẫn giảm, chỉ có một mô-đun không bao giờ học, và triệu chứng duy nhất là một đường cong hơi tệ hơn kỳ vọng.

Thứ ba là *ổn định số*.  $\log(0)$ ,  $\exp$  tràn, chia cho phương sai gần 0: ba nguồn NaN kinh điển. Chúng không phải lỗi của autodiff mà là lỗi của biểu thức bạn viết ra. Cách chữa luôn là viết lại biểu thức ở dạng ổn định: trừ max trước khi lấy  $\exp$ , dùng  $\log\text{sumexp}$ , cộng  $\varepsilon$  vào mẫu số. Kẹp gradient sau khi nó đã thành NaN thì quá muộn. Cùng họ với nó là điều khiển luồng phụ thuộc dữ liệu: một câu lệnh rẽ nhánh theo giá trị tensor là hợp lệ, nhưng gradient chỉ chảy qua nhánh đã chọn, nên nếu bạn tưởng mình đang huấn luyện cả hai nhánh thì bạn nhầm.

### CẠM BẮY

Cái bẫy sinh ra nhiều gradient sai âm thầm nhất không phải công thức khó, mà là quên rằng *broadcast ở lượt xuôi là rẽ nhánh, nên lượt ngược phải cộng theo trục đã broadcast*. Nguy hiểm ở chỗ nếu bạn quên phép cộng đó, shape của kết quả nhiều khi vẫn hợp lệ nhờ chính broadcast, nên không có exception nào được ném ra; bạn chỉ nhận một gradient sai hệ số  $N$  và một quá trình huấn luyện chậm hơn đáng kể. Cách phòng duy nhất đáng tin là kiểm tra bằng số trên một ví dụ có  $N = 3$  và  $N = 1$  rồi so hai kết quả.

### CẦN NHỚ

Lan truyền ngược = duyệt đồ thị theo thứ tự tô-pô đảo, mỗi nút áp một tích vector-Jacobian cục bộ, với ba mẫu cộng-nhân-max cộng thêm quy tắc “rẽ nhánh thì cộng dồn”. Nếu bạn gọi tên được Jacobian cục bộ của một khối, bạn vi phân được khối đó — kể cả khối bạn chưa từng thấy. [T1]

## I. Kết nối

Mục này cho bạn công cụ tính gradient; mục tiếp theo, [C/12/02-lam-cho-training-chay-duoc](#), hỏi câu kè: gradient tính đúng rồi thì làm sao để nó không chết hoặc nổ khi đi qua năm mươi tầng — và câu trả lời bắt đầu chính từ tích các Jacobian cục bộ vừa nêu ở đây. Chi phí bộ nhớ kích hoạt ở cuối mục là tiền đề trực tiếp cho [B/11-gioi-han-tinh-toan](#) (roofline, chế độ nghẽn bộ nhớ) và cho phần huấn luyện phân tán ở [C/12/04-tong-quat-hoa-va-chan-doan](#). Ý “mọi kiến trúc chỉ là một đồ thị” được dùng lại nguyên vẹn ở [C/13-kien-truc-backbone](#): khi đọc ResNet hay Transformer, thứ đáng hỏi không phải sơ đồ khối mà là đường đi của adjoint qua sơ đồ đó.

### I Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao reverse-mode rẻ hơn forward-mode đúng bằng tỉ số giữa số đầu vào và số đầu ra, và trong tình huống nào tỉ số đó đảo chiều khiến forward-mode thắng?
2. Vì sao không bao giờ dựng ma trận Jacobian của một tầng affine, dù về mặt toán học nó tồn tại?
3. Kiểm tra gradient bằng số báo sai số tương đối  $3 \cdot 10^{-3}$  trên một mạng có ReLU — hai nguyên nhân khả dĩ là gì, và bạn phân biệt chúng bằng cách nào?
4. Vì sao gradient của BatchNorm với một mẫu lại phụ thuộc vào các mẫu khác trong batch, và điều đó kéo theo hệ quả gì cho huấn luyện phân tán?
5. Bạn viết một kernel CUDA cho một phép toán mới và lượt ngược cho gradient sai đúng hệ số bằng batch size. Chỗ nào trong code là nghi phạm số một?
6. Gradient checkpointing đổi cái gì lấy cái gì, và khi nào phép đổi đó *không* đáng?

- [1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [2] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [3] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *AISTATS*, 2010.
- [4] K. He, X. Zhang, S. Ren, et al., “Delving Deep into Rectifiers,” in *ICCV*, 2015.
- [5] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [6] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [7] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [8] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [9] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [10] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [11] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” in *ICLR*, 2017.
- [12] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [13] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [14] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [15] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [16] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [17] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [18] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [19] G. Huang, Y. Sun, Z. Liu, et al., “Deep Networks with Stochastic Depth,” in *ECCV*, 2016.
- [20] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [21] H. Zhang, M. Cisse, Y. N. Dauphin, et al., “mixup: Beyond Empirical Risk Minimization,” in *ICLR*, 2018.
- [22] S. Yun, D. Han, S. J. Oh, et al., “CutMix: Regularization Strategy to Train Strong Classifiers,” in *ICCV*, 2019.
- [23] E. D. Cubuk, B. Zoph, J. Shlens, et al., “RandAugment: Practical Automated Data Augmentation,” in *CVPR Workshops*, 2020.



- [24] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [25] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [26] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *NeurIPS*, 2012.
- [27] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in *ICLR*, 2015.
- [28] C. Szegedy, W. Liu, et al., “Going Deeper with Convolutions,” in *CVPR*, 2015.
- [29] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [30] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [31] G. Huang, Z. Liu, L. van der Maaten, et al., “Densely Connected Convolutional Networks,” in *CVPR*, 2017.
- [32] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [33] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [34] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [35] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [36] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [37] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.
- [38] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [39] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [40] J. Su, M. Ahmed, Y. Lu, et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, vol. 568, 2024.
- [41] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” *arXiv:2312.00752*, 2023.
- [42] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [43] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.
- [44] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [45] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [46] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [47] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [48] R. Girshick, J. Donahue, T. Darrell, et al., “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation,” in *CVPR*, 2014.



- [49] R. Girshick, "Fast R-CNN," in *ICCV*, 2015.
- [50] S. Ren, K. He, R. Girshick, et al., "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in *NeurIPS*, 2015.
- [51] T.-Y. Lin, P. Dollár, R. Girshick, et al., "Feature Pyramid Networks for Object Detection," in *CVPR*, 2017.
- [52] K. He, G. Gkioxari, P. Dollár, et al., "Mask R-CNN," in *ICCV*, 2017.
- [53] D. Bolya, S. Foley, J. Hays, et al., "TIDE: A General Toolbox for Identifying Object Detection Errors," in *ECCV*, 2020.
- [54] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [55] J. Redmon, S. Divvala, R. Girshick, et al., "You Only Look Once: Unified, Real-Time Object Detection," in *CVPR*, 2016.
- [56] W. Liu, D. Anguelov, D. Erhan, et al., "SSD: Single Shot MultiBox Detector," in *ECCV*, 2016.
- [57] T.-Y. Lin, P. Goyal, R. Girshick, et al., "Focal Loss for Dense Object Detection," in *ICCV*, 2017.
- [58] Z. Tian, C. Shen, H. Chen, et al., "FCOS: Fully Convolutional One-Stage Object Detection," in *ICCV*, 2019.
- [59] X. Zhou, D. Wang, and P. Krähenbühl, "Objects as Points," *arXiv:1904.07850*, 2019.
- [60] S. Zhang, C. Chi, Y. Yao, et al., "Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection," in *CVPR*, 2020.
- [61] Z. Ge, S. Liu, Z. Li, et al., "OTA: Optimal Transport Assignment for Object Detection," in *CVPR*, 2021.
- [62] N. Bodla, B. Singh, R. Chellappa, et al., "Soft-NMS: Improving Object Detection with One Line of Code," in *ICCV*, 2017.
- [63] X. Zhu, W. Su, L. Lu, et al., "Deformable DETR: Deformable Transformers for End-to-End Object Detection," in *ICLR*, 2021.
- [64] H. Zhang, F. Li, S. Liu, et al., "DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection," in *ICLR*, 2023.
- [65] J. Long, E. Shelhamer, and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," in *CVPR*, 2015.
- [66] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *MICCAI*, 2015.
- [67] L.-C. Chen, G. Papandreou, I. Kokkinos, et al., "DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs," *IEEE TPAMI*, vol. 40, no. 4, pp. 834–848, 2018.
- [68] E. Xie, W. Wang, Z. Yu, et al., "SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers," in *NeurIPS*, 2021.
- [69] B. Cheng, I. Misra, A. G. Schwing, et al., "Masked-Attention Mask Transformer for Universal Image Segmentation," in *CVPR*, 2022.
- [70] A. Kirillov, K. He, R. Girshick, et al., "Panoptic Segmentation," in *CVPR*, 2019.
- [71] A. Kirillov, E. Mintun, N. Ravi, et al., "Segment Anything," in *ICCV*, 2023.
- [72] K. Sun, B. Xiao, D. Liu, et al., "Deep High-Resolution Representation Learning for Human Pose Estimation," in *CVPR*, 2019.

- [73] Z. Cao, T. Simon, S.-E. Wei, et al., “Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields,” in *CVPR*, 2017.
- [74] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [75] J. Deng, J. Guo, N. Xue, et al., “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *CVPR*, 2019.
- [76] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [77] K. Xu, J. Ba, R. Kiros, et al., “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention,” in *ICML*, 2015.
- [78] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *ICLR*, 2015.
- [79] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [80] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [81] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [82] K. Simonyan and A. Zisserman, “Two-Stream Convolutional Networks for Action Recognition in Videos,” in *NeurIPS*, 2014.
- [83] J. Carreira and A. Zisserman, “Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset,” in *CVPR*, 2017.
- [84] D. Tran, H. Wang, L. Torresani, et al., “A Closer Look at Spatiotemporal Convolutions for Action Recognition,” in *CVPR*, 2018.
- [85] C. Feichtenhofer, H. Fan, J. Malik, et al., “SlowFast Networks for Video Recognition,” in *ICCV*, 2019.
- [86] A. Dosovitskiy, P. Fischer, E. Ilg, et al., “FlowNet: Learning Optical Flow with Convolutional Networks,” in *ICCV*, 2015.
- [87] D. Sun, X. Yang, M.-Y. Liu, et al., “PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume,” in *CVPR*, 2018.
- [88] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.
- [89] A. Bewley, Z. Ge, L. Ott, et al., “Simple Online and Realtime Tracking,” in *ICIP*, 2016.
- [90] Y. Zhang, P. Sun, Y. Jiang, et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *ECCV*, 2022.
- [91] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [92] K. Bernardin and R. Stiefelhagen, “Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics,” *EURASIP J. Image and Video Processing*, 2008.
- [93] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [94] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [95] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.

- [96] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *CVPR*, 2024.
- [97] D. Eigen, C. Puhrsch, and R. Fergus, “Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network,” in *NeurIPS*, 2014.
- [98] R. Ranftl, K. Lasinger, D. Hafner, et al., “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer,” *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [99] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IROS*, 2012.
- [100] C. R. Qi, H. Su, K. Mo, et al., “PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation,” in *CVPR*, 2017.
- [101] C. R. Qi, L. Yi, H. Su, et al., “PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space,” in *NeurIPS*, 2017.
- [102] A. H. Lang, S. Vora, H. Caesar, et al., “PointPillars: Fast Encoders for Object Detection from Point Clouds,” in *CVPR*, 2019.
- [103] T. Müller, A. Evans, C. Schied, et al., “Instant Neural Graphics Primitives with a Multiresolution Hash Encoding,” *ACM TOG*, vol. 41, no. 4, 2022.
- [104] B. Kerbl, G. Kopanas, T. Leimkühler, et al., “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *ACM TOG*, vol. 42, no. 4, 2023.
- [105] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.
- [106] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” in *ICLR*, 2014.
- [107] I. Goodfellow, J. Pouget-Abadie, M. Mirza, et al., “Generative Adversarial Nets,” in *NeurIPS*, 2014.
- [108] T. Karras, S. Laine, and T. Aila, “A Style-Based Generator Architecture for Generative Adversarial Networks,” in *CVPR*, 2019.
- [109] J. Ho, A. Jain, and P. Abbeel, “Denoising Diffusion Probabilistic Models,” in *NeurIPS*, 2020.
- [110] Y. Song, J. Sohl-Dickstein, D. P. Kingma, et al., “Score-Based Generative Modeling Through Stochastic Differential Equations,” in *ICLR*, 2021.
- [111] J. Johnson, A. Alahi, and L. Fei-Fei, “Perceptual Losses for Real-Time Style Transfer and Super-Resolution,” in *ECCV*, 2016.
- [112] P. Isola, J.-Y. Zhu, T. Zhou, et al., “Image-to-Image Translation with Conditional Adversarial Networks,” in *CVPR*, 2017.
- [113] L. A. Gatys, A. S. Ecker, and M. Bethge, “Image Style Transfer Using Convolutional Neural Networks,” in *CVPR*, 2016.
- [114] X. Huang and S. Belongie, “Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization,” in *ICCV*, 2017.
- [115] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.
- [116] J. Song, C. Meng, and S. Ermon, “Denoising Diffusion Implicit Models,” in *ICLR*, 2021.
- [117] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, et al., “Flow Matching for Generative Modeling,” in *ICLR*, 2023.
- [118] J. Ho and T. Salimans, “Classifier-Free Diffusion Guidance,” *arXiv:2207.12598*, 2022.
- [119] R. Rombach, A. Blattmann, D. Lorenz, et al., “High-Resolution Image Synthesis with Latent Diffusion Models,” in *CVPR*, 2022.
- [120] W. Peebles and S. Xie, “Scalable Diffusion Models with Transformers,” in *ICCV*, 2023.

- [121] L. Zhang, A. Rao, and M. Agrawala, "Adding Conditional Control to Text-to-Image Diffusion Models," in *ICCV*, 2023.
- [122] Y. Blau and T. Michaeli, "The Perception-Distortion Tradeoff," in *CVPR*, 2018.
- [123] C. Ledig, L. Theis, F. Huszár, et al., "Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network," in *CVPR*, 2017.
- [124] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., "Image Quality Assessment: From Error Visibility to Structural Similarity," *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [125] M. Heusel, H. Ramsauer, T. Unterthiner, et al., "GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium," in *NeurIPS*, 2017.
- [126] T. Brooks, B. Mildenhall, T. Xue, et al., "Unprocessing Images for Learned Raw Denoising," in *CVPR*, 2019.
- [127] A. Abdelhamed, S. Lin, and M. S. Brown, "A High-Quality Denoising Dataset for Smartphone Cameras," in *CVPR*, 2018.
- [128] J. Deng, W. Dong, R. Socher, et al., "ImageNet: A Large-Scale Hierarchical Image Database," in *CVPR*, 2009.
- [129] A. Radford, J. W. Kim, C. Hallacy, et al., "Learning Transferable Visual Models from Natural Language Supervision," in *ICML*, 2021.
- [130] J. Li, D. Li, S. Savarese, et al., "BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models," in *ICML*, 2023.
- [131] H. Liu, C. Li, Q. Wu, et al., "Visual Instruction Tuning," in *NeurIPS*, 2023.
- [132] S. Kornblith, J. Shlens, and Q. V. Le, "Do Better ImageNet Models Transfer Better?" In *CVPR*, 2019.
- [133] T. Chen, S. Kornblith, M. Norouzi, et al., "A Simple Framework for Contrastive Learning of Visual Representations," in *ICML*, 2020.
- [134] K. He, H. Fan, Y. Wu, et al., "Momentum Contrast for Unsupervised Visual Representation Learning," in *CVPR*, 2020.
- [135] J.-B. Grill, F. Strub, F. Altché, et al., "Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning," in *NeurIPS*, 2020.
- [136] M. Caron, H. Touvron, I. Misra, et al., "Emerging Properties in Self-Supervised Vision Transformers," in *ICCV*, 2021.
- [137] K. He, X. Chen, S. Xie, et al., "Masked Autoencoders Are Scalable Vision Learners," in *CVPR*, 2022.
- [138] A. van den Oord, Y. Li, and O. Vinyals, "Representation Learning with Contrastive Predictive Coding," in *arXiv:1807.03748*, 2018.
- [139] J. Kaplan, S. McCandlish, T. Henighan, et al., "Scaling Laws for Neural Language Models," *arXiv:2001.08361*, 2020.
- [140] X. Zhai, A. Kolesnikov, N. Houlsby, et al., "Scaling Vision Transformers," in *CVPR*, 2022.
- [141] J. Hoffmann, S. Borgeaud, A. Mensch, et al., "Training Compute-Optimal Large Language Models," in *NeurIPS*, 2022.
- [142] C. G. Northcutt, A. Athalye, and J. Mueller, "Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks," in *NeurIPS Datasets and Benchmarks*, 2021.
- [143] B. Recht, R. Roelofs, L. Schmidt, et al., "Do ImageNet Classifiers Generalize to ImageNet?" In *ICML*, 2019.
- [144] N. Houlsby, A. Giurgiu, S. Jastrzebski, et al., "Parameter-Efficient Transfer Learning for NLP," in *ICML*, 2019.

- [145] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [146] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *RecSys*, 2019.

### 1.3 Làm cho training chạy được (Making Training Work)

**PHỤ THUỘC** [C/12/01-co-hoc-cua-gradient](#) (tích các Jacobian cục bộ — toàn bộ mục này là hệ quả của nó); [A/01-toan-toi-thieu](#) (phương sai, giá trị kỳ dị, số điều kiện).

#### TÓM TẮT

Mục này trả lời câu hỏi trung tâm của cả chương: làm sao đưa *gradient* qua năm mươi tầng mà nó không chết cũng không nổ. Ba nhóm kỹ thuật tưởng như rời rạc — hàm kích hoạt, khởi tạo trọng số, normalization — thực ra nhằm vào đúng một đại lượng duy nhất. Đọc xong bạn nhìn một file cấu hình huấn luyện và nói được từng dòng trong đó đang giữ đại lượng nào ổn định, thay vì chép nguyên recipe của người khác. Nếu chỉ nhớ một điều: adjoint đi ngược qua  $L$  tầng là tích của  $L$  Jacobian, nên mọi thứ trong mục này chỉ là các cách khác nhau để ép hệ số nhân của tích đó về gần 1.

#### KIẾN THỨC CẦN CÓ TRƯỚC

- **tầng (layer)** — một khối phép toán có tham số riêng; “mạng sâu” nghĩa là nhiều tầng nối tiếp, và toàn bộ khó khăn của mục này sinh ra từ chữ “nối tiếp” đó.
- **kích hoạt (activation)** — giá trị đầu ra của một tầng giữa mạng. “Hàm kích hoạt” là phép phi tuyến tạo ra nó, ví dụ ReLU — hai khái niệm khác nhau nhưng tên gần giống.
- **gradient** — vectơ đạo hàm riêng của hàm mất mát theo từng tham số; đi ngược nó thì mất mát giảm. Gradient “tiêu biến” nghĩa là nó nhỏ tới mức tầng đó không học được nữa.
- **phương sai (variance)** — đại lượng đo độ tản mát của một tập số quanh trung bình của chúng; cả lập luận khởi tạo trọng số trong mục này chỉ là theo dõi phương sai đi qua từng tầng.
- **số điều kiện (condition number)** — tỉ số giữa độ cong lớn nhất và nhỏ nhất của mặt mất mát; càng lớn thì đường đồng mức càng dẹt và thuật toán tối ưu càng đi zigzag.
- **batch (lô)** — nhóm mẫu đi qua mạng cùng lúc. Với BatchNorm, kích thước lô không chỉ là chuyển tốc độ mà còn thay đổi *chính hàm số* mà mạng biểu diễn.
- **chế độ huấn luyện và suy luận (train / eval mode)** — cùng một mô hình hành xử khác nhau ở hai chế độ: BatchNorm và dropout dùng thống kê batch khi huấn luyện, dùng thống kê tích lũy khi suy luận. Quên chuyển chế độ là bug kinh điển ở cuối mục.
- **bias — phân biệt hai nghĩa** — trong  $Wx + b$ ,  $b$  là tham số cộng của một tầng, và “đẩy bias xuống rất âm” ở phần ReLU là nói về nó. Nghĩa thống kê — độ lệch hệ thống của một bộ ước lượng — không dùng trong mục này.

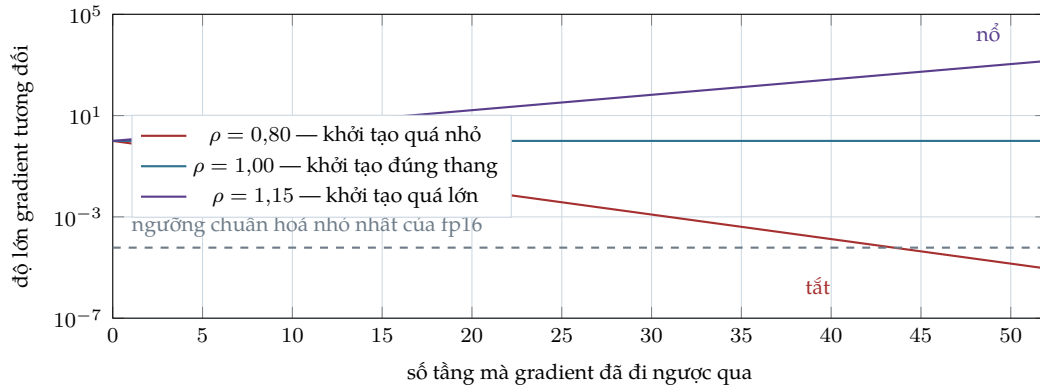
#### A. Đặt vấn đề

Gradient đã tính đúng, kiểm tra bằng số đã qua, code chạy không lỗi. Vậy mà loss vẫn nằm im ở mức phỏng đoán ngẫu nhiên, hoặc bay lên NaN ở bước thứ ba trăm. Đây là trạng thái mà mọi người từng huấn luyện mạng sâu đều đã sống qua, và nó không phải một lỗi lập trình. Nó là biểu hiện của nút thắt lịch sử lớn nhất của ngành: *vanishing and exploding gradient* — gradient tắt dần hoặc nổ tung khi đi ngược qua nhiều tầng. Giữa đầu thập niên 1990 và khoảng 2012, người ta biết cách viết mạng sâu và biết cách tính gradient của nó. Chỉ có điều không ai huấn luyện được nó. Gần như mọi “trick” trong một file cấu hình huấn luyện hiện đại tồn tại vì lý do đó.



Cơ chế thì rất gọn, và nó suy trực tiếp ra từ mục trước. Adjoint đi ngược qua  $L$  tầng là một tích của  $L$  Jacobian cục bộ. Nếu giá trị kỳ dị điển hình của mỗi Jacobian là  $\rho$ , thì độ lớn gradient ở tầng đầu tiên tỉ lệ với  $\rho^L$ . Đây là hàm mũ, nên không có vùng an toàn rộng: với  $L = 50$ ,  $\rho = 0,9$  cho  $0,9^{50} \approx 5 \cdot 10^{-3}$  và  $\rho = 1,1$  cho  $1,1^{50} \approx 117$ . Một sai lệch 10 phần trăm ở mỗi tầng biến thành hai bậc độ lớn ở toàn mạng theo hướng này và hai bậc theo hướng kia.

→ hệ quả của C/12/01 — tích các Jacobian là kết quả trực tiếp của việc adjoint đi ngược qua từng tầng một



**Hình 1.5.** Vì sao không có vùng an toàn rộng cho khởi tạo. Ba đường chỉ khác nhau ở hệ số  $\rho$  của một tầng — lệch 20 phần trăm theo hai chiều — nhưng sau 50 tầng chúng cách nhau tám bậc độ lớn, vì lũy thừa thắng mọi thứ khác. Đường  $\rho = 0,80$  chui xuống dưới ngưỡng biểu diễn của fp16 từ khoảng tầng thứ 40, tức các tầng đầu mạng không nhận được tín hiệu nào cả. Đây là đồ thị của một mô hình đơn giản hoá (mọi Jacobian có cùng phổ, độc lập nhau) nên nó cho thấy *chiều* và *tốc độ* của hiện tượng, không dự đoán giá trị cụ thể.

Hình 1.5 là toàn bộ lý do ba mục tiếp theo tồn tại: khoảng cách giữa “huấn luyện được” và “không huấn luyện được” chỉ là vài phần trăm ở hệ số của một tầng.

Do đó bài toán không phải “làm cho  $\rho$  tốt” mà là “làm cho  $\rho$  bằng 1 và giữ nó ở đó suốt quá trình huấn luyện”.

Có đúng ba hướng tấn công vào cùng con số  $\rho$  đó, và toàn bộ mục này là ba hướng ấy.

- **Sửa hệ số danh nghĩa** — chọn hàm kích hoạt có đạo hàm không nhỏ đi (mục C).
- **Đặt điểm làm việc ban đầu** — chọn khởi tạo sao cho phương sai tín hiệu giữ nguyên qua mỗi tầng (mục D).
- **Hồi tiếp trong lúc chạy** — chèn phép chuẩn hoá ở giữa mạng để ép lại phân bố mỗi khi nó trôi (mục E).

Nhưng trước cả ba, có một việc rẻ hơn nhiều phải làm trước ở đầu vào. Giá trị cụ thể của cả mục: bạn sẽ đọc được một cấu hình huấn luyện lạ và chỉ ra chính xác dòng nào đang giữ  $\rho$  gần 1, do đó biết dòng nào có thể bỏ và dòng nào không.

#### TRỰC GIÁC

Coi mạng sâu như một chuỗi năm mươi bộ khuếch đại nối tiếp nhau, mỗi bộ có hệ số khuếch đại danh nghĩa là 1 nhưng lệch chuẩn vài phần trăm. Không ai chỉnh từng bộ để tổng thể đúng bằng 1 — sai số nhân dồn sẽ thắng. Cái người ta làm trong điện tử là mắc hồi tiếp âm quanh mỗi tầng để hệ số của tầng đó bị ép về giá trị mong muốn bất kể linh kiện trôi. Ba hướng trong mục này là ba kiểu hồi tiếp khác nhau lên cùng một đại lượng: hàm kích hoạt sửa hệ số danh nghĩa, khởi tạo đặt điểm làm việc ban đầu, normalization là vòng hồi tiếp thật sự vì nó đo phân bố đang chạy rồi kéo nó về.



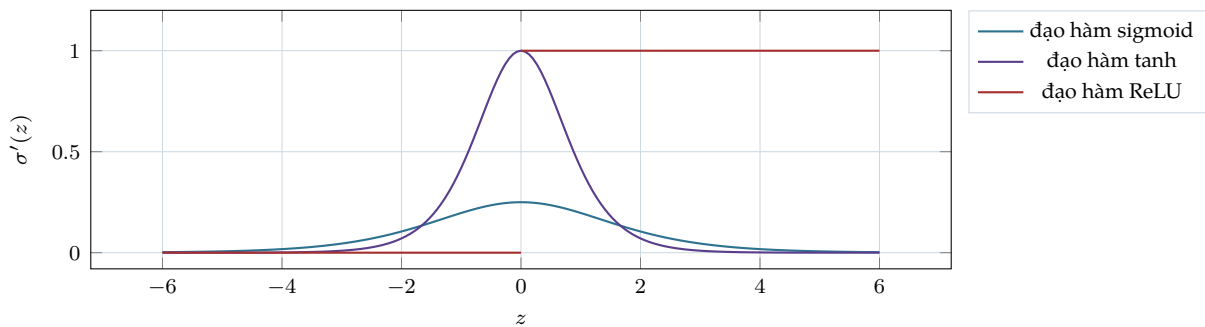
## B. Chuẩn hoá đầu vào (input normalization): việc rẻ nhất, làm trước tiên

Trước khi nói về chiều sâu, hãy nhìn một tầng tuyến tính duy nhất. Hessian của mất mát bình phương theo trọng số của tầng đó tỉ lệ với  $\mathbb{E}[xx^T]$ , tức ma trận moment bậc hai của đầu vào. Giả sử một đặc trưng nằm trong  $[0, 1]$  còn đặc trưng kia trong  $[0, 1000]$ . Hai trị riêng của Hessian khi đó lệch nhau khoảng  $10^6$ , và đường đồng mức của mất mát là những ellipse cực dẹt. Gradient descent trên một ellipse dẹt đi zigzag. Bước học phải đủ nhỏ để không phân kỳ theo trục dọc, nên nó nhỏ đến vô lý theo trục hoành. Tốc độ hội tụ khi đó do **số điều kiện** (condition number)  $\kappa$  chi phối, theo hệ số  $\left(\frac{\kappa-1}{\kappa+1}\right)^t$ . Với  $\kappa = 1000$ , hệ số đó là 0,998 mỗi bước — cần khoảng 350 bước chỉ để giảm sai số đi một nửa.

Trừ trung bình và chia độ lệch chuẩn theo từng kênh đưa  $\kappa$  từ  $10^6$  về vài đơn vị với chi phí gần bằng không. Đây là lý do vì sao nó là dòng đầu tiên của mọi pipeline, chứ không phải vì một lý do thẩm mỹ nào. Bước xa hơn về lý thuyết là *whitening* bằng PCA, đưa  $\kappa$  về đúng 1; nhưng với ảnh thì nó gần như không bao giờ được dùng, vì hai lý do có thể tính ra được. Thứ nhất là chi phí. Một ảnh  $3 \times 224 \times 224$  có 150 528 chiều, nên ma trận hiệp phương sai của nó có  $150\,528^2 \approx 2,27 \cdot 10^{10}$  phần tử. Riêng việc lưu nó đã tốn khoảng 90 GB ở fp32, chưa nói tới phân rã. Thứ hai, và quan trọng hơn, whitening trộn mọi pixel với mọi pixel nên nó *phá cấu trúc không gian*: sau khi whiten, hai pixel cạnh nhau không còn cạnh nhau theo nghĩa thống kê, và toàn bộ giả định **chia sẻ trọng số** (weight sharing) của *convolution* mất cơ sở. Đây là một ví dụ sạch về nguyên lý “một phép biến đổi tối ưu theo một tiêu chí có thể phá huỷ đúng cấu trúc mà phần còn lại của hệ thống đang dựa vào”.

## C. Hướng 1 — hàm kích hoạt (kích hoạt function), đọc như một chuỗi sửa lỗi

**Hàm kích hoạt** (activation function) đóng góp trực tiếp vào  $\rho$ : Jacobian của nó là một ma trận đường chéo chứa  $\sigma'(z)$ , nên nếu  $\sigma'$  nhỏ thì  $\rho$  nhỏ. Sigmoid có  $\sigma'$  cực đại bằng 0,25 tại  $z = 0$  và tiến nhanh về 0 ở hai đầu. Ngay cả ở điều kiện tốt nhất, mười tầng sigmoid nhân vào nhau cho hệ số  $0,25^{10} \approx 10^{-6}$ ; ở điều kiện thực tế, nơi phần lớn nơ-ron bão hoà, con số còn tệ hơn nhiều. Một phép nhân hai dòng như vậy giải thích vì sao mạng sâu không huấn luyện được trước 2010, thuyết phục hơn bất kỳ lời giải thích định tính nào.



**Hình 1.6.** Đạo hàm của ba hàm kích hoạt — đây là đại lượng thật sự đi vào tích các Jacobian, chứ không phải bản thân hàm. Đỉnh của đạo hàm sigmoid chỉ là 0,25 và tụt nhanh về 0 ở hai đầu; đạo hàm tanh đạt 1 nhưng cũng bão hoà; ReLU giữ đúng 1 trên toàn nửa dương và trả về 0 trên nửa âm, đổi tính bão hoà lấy nguy cơ nơ-ron chết.

Hình 1.6 vẽ đúng đại lượng cần nhìn: không phải hình dạng hàm kích hoạt mà là hình dạng *đạo hàm* của nó, vì chỉ đạo hàm mới đi vào tích Jacobian. Bảng 1.3 đọc tiếp bức tranh đó theo trục thời gian.

**Bảng 1.3.** Chuỗi hàm kích hoạt đọc như một chuỗi sửa lỗi. Cột cuối là cột đáng nhớ: mỗi bước sửa được một khiếm khuyết và để lại một khiếm khuyết mới.

| Hàm                    | Cơ chế                         | Sửa được gì                                             | Vấn hòng ở đâu                                                                        |
|------------------------|--------------------------------|---------------------------------------------------------|---------------------------------------------------------------------------------------|
| Sigmoid                | Ép về $(0, 1)$                 | — (khởi điểm lịch sử)                                   | Bão hoà hai đầu giết gradient; đầu ra không quanh 0 nên gradient của cả tầng cùng dấu |
| Tanh                   | Ép về $(-1, 1)$                | Đầu ra quanh 0, hết hiện tượng cùng dấu                 | Vẫn bão hoà, $\sigma' \leq 1$                                                         |
| ReLU                   | $\max(0, x)$                   | Không bão hoà phía dương ( $\sigma' = 1$ ), tính cực rẻ | Nơ-ron chết vĩnh viễn nếu rơi hẳn sang phía âm; không quanh 0                         |
| Leaky ReLU, PReLU, ELU | Cho phía âm một độ dốc nhỏ     | Nơ-ron chết                                             | Thêm siêu tham số; lợi ích không ổn định giữa các bài toán                            |
| GELU, SiLU             | Cổng mượt theo giá trị đầu vào | Đạo hàm liên tục, hợp với transformer và lịch học dài   | Đắt hơn ReLU vài chục phần trăm; khác biệt nhỏ trên CNN                               |

Bảng 1.3 đọc theo chiều dọc thì thấy rõ đây không phải một danh sách lựa chọn ngang hàng mà là một chuỗi nhân quả. Điều đáng chú ý là bước tiến lớn nhất — từ tanh sang ReLU — không đến từ một hàm “tốt hơn” theo nghĩa toán học mà từ việc *bỏ đi* tính bão hoà. Đáng chú ý không kém là ReLU tạo ra một chế độ hỏng hoàn toàn mới không tồn tại trước đó: một nơ-ron nhận bước cập nhật lớn đẩy bias xuống rất âm sẽ cho 0 trên mọi dữ liệu, do đó gradient của nó bằng 0 mãi mãi, và nó chết không hồi phục. Tỷ lệ nơ-ron chết (*dead ReLU*) vì thế là một chỉ báo sức khoẻ đáng đo, và ta sẽ gặp lại nó ở mục sau.

#### D. Hướng 2 — khởi tạo trọng số (weight initialization), hay toán đơn giản giải bài toán thật

Lập luận phương sai là ví dụ đẹp nhất trong ngành về việc vài dòng đại số giải quyết một vấn đề đã chặn cả một lĩnh vực. Xét  $y = Wx$  với  $n$  đầu vào, các  $w_{ij}$  độc lập cùng phân bố trung bình 0, và  $x$  độc lập với  $W$ . Khi đó

$$\text{Var}(y_j) = n \text{Var}(w) \text{Var}(x).$$

Muốn **phương sai** (*variance*) tín hiệu không đổi qua mỗi tầng, cần  $n \text{Var}(w) = 1$ , tức  $\text{Var}(w) = 1/n_{\text{in}}$ . Nhưng lượt ngược cũng phải giữ phương sai, và ở đó vai trò của  $n_{\text{in}}$  và  $n_{\text{out}}$  đổi chỗ; không thoả cả hai cùng lúc được nên Xavier/Glorot chọn trung hoà  $\text{Var}(w) = 2/(n_{\text{in}} + n_{\text{out}})$  [3]. Đó là lời giải cho mạng tanh.

Với ReLU, lập luận trên thiếu một hệ số. ReLU xoá một nửa số đầu ra về 0, nên phương sai sau kích hoạt chỉ còn một nửa phương sai trước nó; hệ số mỗi tầng là 1/2 chứ không phải 1. Sau 30 tầng, tín hiệu bị nhân với  $2^{-30} \approx 10^{-9}$  — tức là tắt hẳn, dù bạn đã “khởi tạo đúng” theo Xavier. Sửa lại thì hiển nhiên: nhân đôi phương sai,  $\text{Var}(w) = 2/n_{\text{in}}$ . Đó là khởi tạo He/Kaiming [4], và chính hệ số 2 đó là thứ cho phép huấn luyện mạng ReLU ba mươi tầng lần đầu tiên. Hãy để ý mạch lập luận: một giả định (“ReLU không đổi phương sai”) bị vi phạm, hậu quả được lượng hoá ra một con số, và lời sửa là một hằng số. Đây là dạng suy luận mà bạn nên tái tạo được, chứ không phải hai công thức để nhớ.

↔ **tiền đề** A/01 — toàn bộ lập luận chỉ dùng một tính chất của phương sai: phương sai của tổng các biến độc lập bằng tổng phương sai

### E. Hướng 3 — normalization, vòng hồi tiếp thật sự

Hai hướng trên chỉ đặt điều kiện *ban đầu*. Sau vài nghìn bước, trọng số đã đổi và phân bố kích hoạt trôi khỏi chỗ được thiết kế. **Chuẩn hoá** (*normalization*) đo phân bố đang chạy rồi ép nó về trung bình 0 và độ lệch chuẩn 1. Sau đó nó trả lại quyền chính cho mạng, qua hai tham số học được  $\gamma$  và  $\beta$ . Bốn biến thể khác nhau ở đúng một điểm: *lấy trung bình trên trục nào*.

**Bảng 1.4.** Bốn biến thể normalization. Chúng chỉ khác nhau ở tập chỉ số dùng để tính thống kê, và chính lựa chọn đó quyết định chúng hỏng khi nào.

| Loại          | Trung bình theo                            | Sinh ra để giải quyết                                                  | Hỏng khi nào                                                                  |
|---------------|--------------------------------------------|------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| BatchNorm [5] | Batch $\times$ không gian, riêng từng kênh | Ổn định phân bố giữa mạng, cho phép dùng learning rate lớn hơn hẳn     | Batch nhỏ (thống kê nhiễu); lệch train/eval; phải đồng bộ giữa các GPU        |
| LayerNorm [6] | Toàn bộ đặc trưng của <i>một</i> mẫu       | Batch nhỏ hoặc bằng 1; chuỗi độ dài thay đổi; không phụ thuộc mẫu khác | Trên CNN thì giác thường kém BN; trộn chung các kênh có ý nghĩa rất khác nhau |
| GroupNorm [7] | Nhóm kênh của một mẫu                      | Detection/segmentation với batch một tới hai ảnh mỗi GPU               | Thêm siêu tham số số nhóm; không có lợi thế nào khi batch đủ lớn              |
| InstanceNorm  | Mỗi kênh của mỗi mẫu                       | Style transfer: xoá thống kê style khỏi đặc trưng                      | Ở bài toán khác, thứ bị xoá lại là thông tin có ích (độ sáng, tương phản)     |

Bảng 1.4 nên đọc cùng một nhận xét về mặt cơ chế: câu chuyện gốc của BatchNorm là “giảm internal covariate shift”, nhưng lời giải thích đó đã bị chất vấn nghiêm túc. Santurkar và cộng sự bơm nhiễu để *tăng* covariate shift một cách cố ý, và BN vẫn hoạt động tốt. Họ đề xuất tác dụng thật là làm mất mát mát trơn hơn. Gradient bớt thay đổi đột ngột, nên bước đi dài hơn mà không phân kỳ [8]. Đây là một trường hợp đáng nhớ khi đọc paper. Kỹ thuật đúng, nhưng lời giải thích đi kèm nó lúc ra đời có thể sai. Học thuộc lời giải thích thay vì cơ chế thì bạn sẽ suy luận sai ở tình huống kế tiếp.

Hệ quả kỹ thuật quan trọng nhất của BatchNorm là nó *ghép các mẫu trong batch lại với nhau*. Ta đã thấy điều đó ở công thức backward trong mục trước: gradient của một mẫu phụ thuộc mọi mẫu cùng batch. Từ đó suy ra gần như mọi chế độ hỏng của nó. Batch nhỏ thì thống kê nhiễu, nên gradient nhiễu. Lúc suy luận không có batch nên phải dùng thống kê chạy tích lũy trong huấn luyện, và bất kỳ chênh lệch nào giữa hai chế độ đều thành sai lệch hệ thống. Trong huấn luyện phân tán, mỗi tiến trình tính thống kê riêng nên mô hình thực chất khác nhau ở mỗi GPU trừ khi bạn dùng SyncBN. LayerNorm và GroupNorm né toàn bộ họ vấn đề này bằng cách không dùng tới trục batch — và đó là toàn bộ lý do transformer dùng LayerNorm, chứ không phải vì LayerNorm “hiện đại hơn”.

→ **ràng buộc bởi C/14** — detection chạy một tới hai ảnh mỗi GPU, nên chính ràng buộc batch nhỏ ở đó quyết định lựa chọn GroupNorm

(1) conv → chuẩn hoá → hàm kích hoạt — thứ tự mặc định



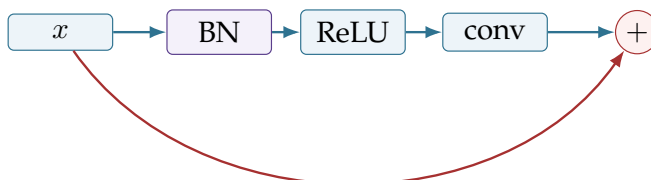
Đầu vào ReLU có trung bình 0, nên khoảng một nửa số nơ-ron hoạt động ở mọi batch. Đây là cấu hình mà lập luận phương sai của He giả định.

(2) conv → hàm kích hoạt → chuẩn hoá



BN phải chuẩn hoá một phân bố *không âm*, nên  $\beta$  phải gánh việc dời tâm và  $\gamma$  làm việc trên một phổ hẹp hơn. Chạy được, nhưng lãng phí hai tham số.

(3) tiền kích hoạt trong khối residual: BN → ReLU → conv



Gradient từ đầu ra về  $x$  đi qua một đường không bị chuẩn hoá chặn, nên thành phần bằng 1 trong Jacobian được giữ nguyên qua rất nhiều khối.

đường tắt *sạch*: không có phép toán nào chen vào

**Hình 1.7.** Ba cách đặt phép chuẩn hoá quanh hàm kích hoạt, và hệ quả lên luồng gradient. Điều hình này nói mà câu văn khó nói: vị trí của khối BN không phải chi tiết thẩm mỹ. Nó quyết định phân bố mà ReLU nhìn thấy, tức hàng 1 so với hàng 2. Nó cũng quyết định đường tắt có sạch hay không, tức hàng 3. Ở hàng 3, mọi phép toán đã bị đẩy vào *nhánh residual*, nên đường cộng từ  $x$  tới điểm  $+$  hoàn toàn trống. Đó là lý do biến thể tiền kích hoạt huấn luyện được ở độ sâu lớn hơn hẳn.

Hình 1.7 cũng là chỗ ba hướng của mục này gặp nhau trên một sơ đồ duy nhất: hàng 1 nói về quan hệ giữa chuẩn hoá và hàm kích hoạt, hàng 3 nói về quan hệ giữa chuẩn hoá và kết nối tắt.

Còn một hướng thứ tư mà mục này cố tình không kể: **kết nối tắt** (*residual connection*). Nó tấn công đúng cùng đại lượng  $\rho$ , nhưng bằng cách thêm một đường cộng để Jacobian có dạng  $I + \partial F / \partial x$  — luôn có một thành phần bằng 1 không thể triệt tiêu. Vì nó là một lựa chọn *kiến trúc* chứ không phải một thủ thuật huấn luyện, nó thuộc về [C/13/01-dong-cnn](#); nhưng khi đọc chương đó, hãy nhớ nó là thành viên thứ tư của đúng gia đình này.

↔ cùng nguyên lý với [C/13/01](#) — kết nối tắt giữ tích Jacobian gần 1 bằng một đường cộng, tức cùng mục tiêu với ba hướng ở đây nhưng thực hiện ở tầng kiến trúc

## F. Giới hạn và trường hợp hỏng

Ba hướng trên không cộng dồn vô hạn, và chúng có những chỗ giao thoa gây bất ngờ.

Khởi tạo đúng chỉ đúng ở bước 0. He/Kaiming giả định trọng số độc lập cùng phân bố và đầu vào đã chuẩn hoá; sau vài nghìn bước cả hai giả định đều sai, nên khởi tạo không thay được normalization. Ngược lại, ở các mạng rất sâu có kết nối tắt, khởi tạo quan trọng theo một kiểu khác. Nhánh residual cần được khởi tạo nhỏ hoặc bằng 0, để mạng bắt đầu gần với ánh xạ đồng nhất. Bỏ chi tiết đó đi thì mạng sâu vẫn khó huấn luyện, dù đã có đủ BN.

Normalization che giấu lỗi. Một mạng có BN ở khắp nơi vẫn huấn luyện được dù khởi tạo sai, dù đầu vào chưa chuẩn hoá, dù learning rate lệch một bậc — chỉ là kém hơn đáng kể. Đây là tin tốt cho tiến độ và tin xấu cho việc học chẩn đoán, vì nó xoá mất các triệu chứng mà lẽ ra bạn phải học đọc.

Cuối cùng, các con số như  $\rho^L$  hay  $2^{-30}$  trong mục này là ước lượng bậc độ lớn dựa trên giả định các Jacobian độc lập và có phổ đều. Giả định đó thô: trong thực tế các Jacobian tương quan với nhau, và phổ của chúng không đều. Chúng dùng để cho thấy *hàm mũ thắng mọi thứ khác*, chứ không dùng để dự đoán một giá trị cụ thể.

### CẠM BÃY

Bug kinh điển của cả nhóm này: quên gọi `model.eval()` khi đánh giá. BatchNorm khi đó dùng thống kê của chính batch kiểm thử thay vì thống kê chạy. Kết quả vì thế phụ thuộc vào việc bạn xếp mẫu nào chung batch. Tệ nhất là nó thường *tốt hơn* thực tế, vì mô hình được nhìn trộm thống kê của tập kiểm thử. Triệu chứng nhận dạng: điểm số đôi khi bạn đổi batch size lúc đánh giá, hoặc điểm số tụt hẳn khi triển khai với batch bằng 1. Cùng một dòng lệnh cũng tắt dropout, nên hai lỗi này luôn đi cùng nhau.

### CẦN NHỚ

Cả ba hướng — hàm kích hoạt, khởi tạo, normalization — chỉ nhắm vào đúng một đại lượng: hệ số nhân của tích các Jacobian qua các tầng. Mọi câu hỏi dạng “kỹ thuật X có cần thiết không” nên được dịch lại thành “trong cấu hình của tôi, cái gì đang giữ hệ số đó gần 1”. [T1]

## G. Kết nối

Mục này lấy đầu vào từ [C/12/01-co-hoc-cua-gradient](#): toàn bộ lập luận chỉ là hệ quả số học của việc adjoint là một tích Jacobian. Nó chuyển tiếp sang [C/12/03-toi-uu-hoa](#). Ở đó ta giả định gradient đã lành mạnh, rồi hỏi tiếp nên đi bước dài bao nhiêu. Warmup sẽ xuất hiện ngay tại đó, như hệ quả trực tiếp của việc thống kê BN chưa ổn định ở những bước đầu. Thành viên thứ tư của gia đình, kết nối tắt, nằm ở [C/13/01-dong-cnn](#). Lựa chọn giữa BatchNorm và GroupNorm gắn chặt với ràng buộc batch size trong detection, nên nó được nhắc lại ở [C/14-bai-toan-cv-loi](#); còn chi phí bộ nhớ của việc lưu  $\hat{x}$  cho lượt ngược thuộc về B/11-gioi-han-tinh-toan.

### Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao một sai lệch 10 phần trăm ở hệ số mỗi tầng lại là vấn đề nghiêm trọng, trong khi sai lệch 10 phần trăm ở learning rate thì không?
2. Whitening đưa số điều kiện về 1, tức tối ưu theo tiêu chí ở đầu mục. Vậy vì sao gần như không ai whiten ảnh?
3. Hệ số 2 trong khởi tạo He đến từ đâu, và nếu bạn thay ReLU bằng GELU thì lập luận đó thay đổi thế nào?
4. BatchNorm hỏng theo ba kiểu khác nhau; cả ba đều suy ra từ một tính chất duy nhất của nó. Tính chất đó là gì?
5. Vì sao lời giải thích “internal covariate shift” bị nghi ngờ, và vì sao việc học thuộc một lời giải thích sai lại nguy hiểm hơn là không có lời giải thích nào?
6. Bạn thay BatchNorm bằng LayerNorm trong một CNN thị giác và kết quả tệ đi — cách giải thích của bạn là gì, và phép đo nào kiểm chứng được nó?

- [1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [2] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [3] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *AISTATS*, 2010.
- [4] K. He, X. Zhang, S. Ren, et al., “Delving Deep into Rectifiers,” in *ICCV*, 2015.
- [5] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [6] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [7] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [8] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [9] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [10] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [11] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” in *ICLR*, 2017.
- [12] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [13] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [14] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [15] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [16] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [17] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [18] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [19] G. Huang, Y. Sun, Z. Liu, et al., “Deep Networks with Stochastic Depth,” in *ECCV*, 2016.
- [20] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [21] H. Zhang, M. Cisse, Y. N. Dauphin, et al., “mixup: Beyond Empirical Risk Minimization,” in *ICLR*, 2018.
- [22] S. Yun, D. Han, S. J. Oh, et al., “CutMix: Regularization Strategy to Train Strong Classifiers,” in *ICCV*, 2019.
- [23] E. D. Cubuk, B. Zoph, J. Shlens, et al., “RandAugment: Practical Automated Data Augmentation,” in *CVPR Workshops*, 2020.



- [24] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [25] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [26] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *NeurIPS*, 2012.
- [27] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in *ICLR*, 2015.
- [28] C. Szegedy, W. Liu, et al., “Going Deeper with Convolutions,” in *CVPR*, 2015.
- [29] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [30] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [31] G. Huang, Z. Liu, L. van der Maaten, et al., “Densely Connected Convolutional Networks,” in *CVPR*, 2017.
- [32] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [33] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [34] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [35] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [36] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [37] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.
- [38] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [39] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [40] J. Su, M. Ahmed, Y. Lu, et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, vol. 568, 2024.
- [41] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” *arXiv:2312.00752*, 2023.
- [42] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [43] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.
- [44] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [45] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [46] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [47] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [48] R. Girshick, J. Donahue, T. Darrell, et al., “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation,” in *CVPR*, 2014.

- [49] R. Girshick, "Fast R-CNN," in *ICCV*, 2015.
- [50] S. Ren, K. He, R. Girshick, et al., "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in *NeurIPS*, 2015.
- [51] T.-Y. Lin, P. Dollár, R. Girshick, et al., "Feature Pyramid Networks for Object Detection," in *CVPR*, 2017.
- [52] K. He, G. Gkioxari, P. Dollár, et al., "Mask R-CNN," in *ICCV*, 2017.
- [53] D. Bolya, S. Foley, J. Hays, et al., "TIDE: A General Toolbox for Identifying Object Detection Errors," in *ECCV*, 2020.
- [54] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [55] J. Redmon, S. Divvala, R. Girshick, et al., "You Only Look Once: Unified, Real-Time Object Detection," in *CVPR*, 2016.
- [56] W. Liu, D. Anguelov, D. Erhan, et al., "SSD: Single Shot MultiBox Detector," in *ECCV*, 2016.
- [57] T.-Y. Lin, P. Goyal, R. Girshick, et al., "Focal Loss for Dense Object Detection," in *ICCV*, 2017.
- [58] Z. Tian, C. Shen, H. Chen, et al., "FCOS: Fully Convolutional One-Stage Object Detection," in *ICCV*, 2019.
- [59] X. Zhou, D. Wang, and P. Krähenbühl, "Objects as Points," *arXiv:1904.07850*, 2019.
- [60] S. Zhang, C. Chi, Y. Yao, et al., "Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection," in *CVPR*, 2020.
- [61] Z. Ge, S. Liu, Z. Li, et al., "OTA: Optimal Transport Assignment for Object Detection," in *CVPR*, 2021.
- [62] N. Bodla, B. Singh, R. Chellappa, et al., "Soft-NMS: Improving Object Detection with One Line of Code," in *ICCV*, 2017.
- [63] X. Zhu, W. Su, L. Lu, et al., "Deformable DETR: Deformable Transformers for End-to-End Object Detection," in *ICLR*, 2021.
- [64] H. Zhang, F. Li, S. Liu, et al., "DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection," in *ICLR*, 2023.
- [65] J. Long, E. Shelhamer, and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," in *CVPR*, 2015.
- [66] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *MICCAI*, 2015.
- [67] L.-C. Chen, G. Papandreou, I. Kokkinos, et al., "DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs," *IEEE TPAMI*, vol. 40, no. 4, pp. 834–848, 2018.
- [68] E. Xie, W. Wang, Z. Yu, et al., "SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers," in *NeurIPS*, 2021.
- [69] B. Cheng, I. Misra, A. G. Schwing, et al., "Masked-Attention Mask Transformer for Universal Image Segmentation," in *CVPR*, 2022.
- [70] A. Kirillov, K. He, R. Girshick, et al., "Panoptic Segmentation," in *CVPR*, 2019.
- [71] A. Kirillov, E. Mintun, N. Ravi, et al., "Segment Anything," in *ICCV*, 2023.
- [72] K. Sun, B. Xiao, D. Liu, et al., "Deep High-Resolution Representation Learning for Human Pose Estimation," in *CVPR*, 2019.

- [73] Z. Cao, T. Simon, S.-E. Wei, et al., “Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields,” in *CVPR*, 2017.
- [74] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [75] J. Deng, J. Guo, N. Xue, et al., “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *CVPR*, 2019.
- [76] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [77] K. Xu, J. Ba, R. Kiros, et al., “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention,” in *ICML*, 2015.
- [78] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *ICLR*, 2015.
- [79] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [80] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [81] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [82] K. Simonyan and A. Zisserman, “Two-Stream Convolutional Networks for Action Recognition in Videos,” in *NeurIPS*, 2014.
- [83] J. Carreira and A. Zisserman, “Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset,” in *CVPR*, 2017.
- [84] D. Tran, H. Wang, L. Torresani, et al., “A Closer Look at Spatiotemporal Convolutions for Action Recognition,” in *CVPR*, 2018.
- [85] C. Feichtenhofer, H. Fan, J. Malik, et al., “SlowFast Networks for Video Recognition,” in *ICCV*, 2019.
- [86] A. Dosovitskiy, P. Fischer, E. Ilg, et al., “FlowNet: Learning Optical Flow with Convolutional Networks,” in *ICCV*, 2015.
- [87] D. Sun, X. Yang, M.-Y. Liu, et al., “PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume,” in *CVPR*, 2018.
- [88] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.
- [89] A. Bewley, Z. Ge, L. Ott, et al., “Simple Online and Realtime Tracking,” in *ICIP*, 2016.
- [90] Y. Zhang, P. Sun, Y. Jiang, et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *ECCV*, 2022.
- [91] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [92] K. Bernardin and R. Stiefelhagen, “Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics,” *EURASIP J. Image and Video Processing*, 2008.
- [93] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [94] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [95] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.

- [96] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *CVPR*, 2024.
- [97] D. Eigen, C. Puhrsch, and R. Fergus, “Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network,” in *NeurIPS*, 2014.
- [98] R. Ranftl, K. Lasinger, D. Hafner, et al., “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer,” *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [99] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IROS*, 2012.
- [100] C. R. Qi, H. Su, K. Mo, et al., “PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation,” in *CVPR*, 2017.
- [101] C. R. Qi, L. Yi, H. Su, et al., “PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space,” in *NeurIPS*, 2017.
- [102] A. H. Lang, S. Vora, H. Caesar, et al., “PointPillars: Fast Encoders for Object Detection from Point Clouds,” in *CVPR*, 2019.
- [103] T. Müller, A. Evans, C. Schied, et al., “Instant Neural Graphics Primitives with a Multiresolution Hash Encoding,” *ACM TOG*, vol. 41, no. 4, 2022.
- [104] B. Kerbl, G. Kopanas, T. Leimkühler, et al., “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *ACM TOG*, vol. 42, no. 4, 2023.
- [105] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.
- [106] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” in *ICLR*, 2014.
- [107] I. Goodfellow, J. Pouget-Abadie, M. Mirza, et al., “Generative Adversarial Nets,” in *NeurIPS*, 2014.
- [108] T. Karras, S. Laine, and T. Aila, “A Style-Based Generator Architecture for Generative Adversarial Networks,” in *CVPR*, 2019.
- [109] J. Ho, A. Jain, and P. Abbeel, “Denoising Diffusion Probabilistic Models,” in *NeurIPS*, 2020.
- [110] Y. Song, J. Sohl-Dickstein, D. P. Kingma, et al., “Score-Based Generative Modeling Through Stochastic Differential Equations,” in *ICLR*, 2021.
- [111] J. Johnson, A. Alahi, and L. Fei-Fei, “Perceptual Losses for Real-Time Style Transfer and Super-Resolution,” in *ECCV*, 2016.
- [112] P. Isola, J.-Y. Zhu, T. Zhou, et al., “Image-to-Image Translation with Conditional Adversarial Networks,” in *CVPR*, 2017.
- [113] L. A. Gatys, A. S. Ecker, and M. Bethge, “Image Style Transfer Using Convolutional Neural Networks,” in *CVPR*, 2016.
- [114] X. Huang and S. Belongie, “Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization,” in *ICCV*, 2017.
- [115] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.
- [116] J. Song, C. Meng, and S. Ermon, “Denoising Diffusion Implicit Models,” in *ICLR*, 2021.
- [117] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, et al., “Flow Matching for Generative Modeling,” in *ICLR*, 2023.
- [118] J. Ho and T. Salimans, “Classifier-Free Diffusion Guidance,” *arXiv:2207.12598*, 2022.
- [119] R. Rombach, A. Blattmann, D. Lorenz, et al., “High-Resolution Image Synthesis with Latent Diffusion Models,” in *CVPR*, 2022.
- [120] W. Peebles and S. Xie, “Scalable Diffusion Models with Transformers,” in *ICCV*, 2023.

- [121] L. Zhang, A. Rao, and M. Agrawala, "Adding Conditional Control to Text-to-Image Diffusion Models," in *ICCV*, 2023.
- [122] Y. Blau and T. Michaeli, "The Perception-Distortion Tradeoff," in *CVPR*, 2018.
- [123] C. Ledig, L. Theis, F. Huszár, et al., "Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network," in *CVPR*, 2017.
- [124] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., "Image Quality Assessment: From Error Visibility to Structural Similarity," *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [125] M. Heusel, H. Ramsauer, T. Unterthiner, et al., "GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium," in *NeurIPS*, 2017.
- [126] T. Brooks, B. Mildenhall, T. Xue, et al., "Unprocessing Images for Learned Raw Denoising," in *CVPR*, 2019.
- [127] A. Abdelhamed, S. Lin, and M. S. Brown, "A High-Quality Denoising Dataset for Smartphone Cameras," in *CVPR*, 2018.
- [128] J. Deng, W. Dong, R. Socher, et al., "ImageNet: A Large-Scale Hierarchical Image Database," in *CVPR*, 2009.
- [129] A. Radford, J. W. Kim, C. Hallacy, et al., "Learning Transferable Visual Models from Natural Language Supervision," in *ICML*, 2021.
- [130] J. Li, D. Li, S. Savarese, et al., "BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models," in *ICML*, 2023.
- [131] H. Liu, C. Li, Q. Wu, et al., "Visual Instruction Tuning," in *NeurIPS*, 2023.
- [132] S. Kornblith, J. Shlens, and Q. V. Le, "Do Better ImageNet Models Transfer Better?" In *CVPR*, 2019.
- [133] T. Chen, S. Kornblith, M. Norouzi, et al., "A Simple Framework for Contrastive Learning of Visual Representations," in *ICML*, 2020.
- [134] K. He, H. Fan, Y. Wu, et al., "Momentum Contrast for Unsupervised Visual Representation Learning," in *CVPR*, 2020.
- [135] J.-B. Grill, F. Strub, F. Altché, et al., "Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning," in *NeurIPS*, 2020.
- [136] M. Caron, H. Touvron, I. Misra, et al., "Emerging Properties in Self-Supervised Vision Transformers," in *ICCV*, 2021.
- [137] K. He, X. Chen, S. Xie, et al., "Masked Autoencoders Are Scalable Vision Learners," in *CVPR*, 2022.
- [138] A. van den Oord, Y. Li, and O. Vinyals, "Representation Learning with Contrastive Predictive Coding," in *arXiv:1807.03748*, 2018.
- [139] J. Kaplan, S. McCandlish, T. Henighan, et al., "Scaling Laws for Neural Language Models," *arXiv:2001.08361*, 2020.
- [140] X. Zhai, A. Kolesnikov, N. Houlsby, et al., "Scaling Vision Transformers," in *CVPR*, 2022.
- [141] J. Hoffmann, S. Borgeaud, A. Mensch, et al., "Training Compute-Optimal Large Language Models," in *NeurIPS*, 2022.
- [142] C. G. Northcutt, A. Athalye, and J. Mueller, "Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks," in *NeurIPS Datasets and Benchmarks*, 2021.
- [143] B. Recht, R. Roelofs, L. Schmidt, et al., "Do ImageNet Classifiers Generalize to ImageNet?" In *ICML*, 2019.
- [144] N. Houlsby, A. Giurgiu, S. Jastrzebski, et al., "Parameter-Efficient Transfer Learning for NLP," in *ICML*, 2019.



- [145] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [146] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *RecSys*, 2019.

## 1.4 Tối ưu hoá và siêu tham số (Optimization and Hyperparameters)

**PHỤ THUỘC** `C/12/02-lam-cho-training-chay-duoc` (gradient phải lành mạnh trước khi bàn tới bước đi); `A/01-toan-toi-thieu` (số điều kiện, hội tụ của gradient descent).

### TÓM TẮT

Mục này đọc bảy *optimizer* quen thuộc như một chuỗi vá lỗi liên tiếp chứ không như một menu lựa chọn, rồi chỉ ra siêu tham số nào thật sự quyết định kết quả. Đọc xong bạn nhìn một đường cong học và đoán được đang sai ở đâu, đặt được *learning rate* và lịch của nó có căn cứ thay vì mò, và giải thích được vì sao *mixed precision* cần *loss scaling*. Nếu chỉ nhớ một điều: optimizer chỉ định hình lại bước đi, nó không cứu được một learning rate sai — nên hãy tune learning rate trước, batch size sau, và chọn optimizer cuối cùng.

### KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình (model)** — một hàm có tham số, nhận đầu vào và trả về dự đoán; huấn luyện là đi tìm bộ tham số làm dự đoán sát nhãn nhất.
- **gradient** — vectơ đạo hàm riêng của hàm mất mát theo từng tham số; nó chỉ hướng làm mất mát TĂNG nhanh nhất, nên mọi thuật toán ở đây đi ngược lại nó.
- **learning rate** — độ dài bước mà thuật toán tối ưu đi theo hướng ngược gradient ở mỗi lần cập nhật; quá lớn thì nghiệm bật ra ngoài, quá nhỏ thì hội tụ chậm tới mức vô dụng.
- **optimizer (thuật toán tối ưu)** — quy tắc biến gradient thành bước cập nhật thực tế; toàn bộ mục này là lịch sử bảy quy tắc như vậy và lý do từng cái ra đời.
- **siêu tham số (hyperparameter)** — con số do người đặt trước khi huấn luyện chứ không được học từ dữ liệu — learning rate, batch size, cường độ weight decay.
- **bước và epoch (step / epoch)** — một “bước” là một lần cập nhật tham số dựa trên một lô dữ liệu; một “epoch” là khi mô hình đã đi hết toàn bộ tập huấn luyện một lượt.
- **weight decay** — cơ chế kéo các trọng số về gần 0 sau mỗi bước, nhằm hạn chế mô hình bám quá sát dữ liệu huấn luyện; cách nó được cài đặt là toàn bộ khác biệt giữa Adam và AdamW.
- **fp32, fp16, bf16** — ba định dạng số thực với số bit khác nhau. Điều quan trọng không phải “bao nhiêu bit” mà là bit đó chia thể nào giữa *mũ* (quyết định dải giá trị biểu diễn được) và *định trị* (quyết định độ chính xác).

### A. Đặt vấn đề

Giả sử gradient đã tính đúng và không còn chết hay nổ. Câu hỏi còn lại nghe rất đơn giản: biết hướng dốc rồi thì đi bao xa? Nhưng nó khó vì ba lý do chồng lên nhau. Thứ nhất, mất mát mát bị điều kiện xấu — có hướng dốc gấp trăm lần hướng khác, nên một bước dài theo hướng này là bước ngắn vô vọng theo hướng kia. Thứ hai, gradient bạn có không phải gradient thật mà là ước lượng từ một minibatch, nên nó nhiễu. Thứ ba, mọi thứ phi logic, nên không có lý thuyết nào nói cho bạn bước đi tối ưu.

Toàn bộ lịch sử *optimizer* trong deep learning là câu trả lời cho ba khó khăn ấy, và nó là ví dụ tốt nhất trong tài liệu này về việc *một danh sách thực ra là một mạch lập luận*. Bảy cái tên dưới đây



thường được dạy như bày lựa chọn ngang hàng trong một menu. Chúng không ngang hàng: mỗi cái ra đời để sửa một khiếm khuyết cụ thể của cái trước, và mỗi cái tạo ra một khiếm khuyết mới cho cái sau. Nếu bạn đọc được mạch đó thì bạn không cần nhớ công thức, vì bạn suy lại được chúng; nếu bạn chỉ nhớ danh sách thì bạn sẽ chọn optimizer theo thói quen và không bao giờ biết mình đang trả giá gì.

#### TRỰC GIÁC

Momentum thường được kể bằng ẩn dụ hòn bi lăn xuống dốc, nhưng ẩn dụ đó không nói cho bạn biết *vì sao* nó hiệu quả. Cách nhìn đúng hơn với người đã lọc tín hiệu IMU: chuỗi gradient theo thời gian là một tín hiệu, và momentum là một bộ lọc thông thấp một cực đặt lên tín hiệu đó. Trong một rãnh hẹp, thành phần vuông góc với rãnh đổi dấu mỗi bước — đó là thành phần tần số cao, và bộ lọc dập nó. Thành phần dọc theo rãnh giữ nguyên dấu qua nhiều bước — đó là thành phần một chiều, và bộ lọc để nó cộng dồn tới hệ số  $1/(1 - \mu)$ . Momentum không “thêm quán tính”; nó tách tín hiệu khỏi dao động, đúng như khi bạn lọc bias con quay ra khỏi rung động khung máy.

### B. Bảy bước của một mạch lập luận duy nhất (SGD → AdamW)

Trước khi vào mạch, hãy cố định một con số để có cảm giác về mức độ nghiêm trọng. Với một mặt bậc hai có số điều kiện  $\kappa$ , gradient descent hội tụ với hệ số  $\left(\frac{\kappa-1}{\kappa+1}\right)$  mỗi bước. Với  $\kappa = 1000$ , hệ số đó là 0,998, nên cần khoảng 350 bước chỉ để giảm sai số một nửa. Mọi thứ trong mạch dưới đây là những cách khác nhau để không phải trả cái giá đó.

↔ **tiền đề** A/01 — số điều kiện và hệ số hội tụ của gradient descent trên mặt bậc hai là kết quả chuẩn của giải tích số

#### I Mạch 1 — tiến hoá của optimizer

- 1 Vấn đề** gradient descent ngẫu nhiên đi zigzag trong rãnh hẹp và bò rất chậm theo hướng thoải.

**Giải pháp** **hạ gradient ngẫu nhiên** (*stochastic gradient descent, SGD*),  $\theta \leftarrow \theta - \eta g$ , là khởi điểm và cũng là chuẩn so sánh; ưu điểm thật của nó là nhiễu *minibatch* đóng vai trò **chính quy hoá ngầm** (*implicit regularization*).

**Giá phải trả** bước học  $\eta$  phải nhỏ hơn nghịch đảo độ cong lớn nhất để không phân kỳ, nên nó tự động quá nhỏ cho mọi hướng còn lại.

**Vấn đề mới** tốc độ bị khoá vào  $\kappa$ , và một giá trị  $\eta$  duy nhất phải phục vụ mọi tham số trong mạng.
- 2 Vấn đề** dao động vuông góc với rãnh làm phí gần hết công.

**Giải pháp** **momentum**:  $v \leftarrow \mu v + g$ ,  $\theta \leftarrow \theta - \eta v$ . Thành phần đổi dấu triệt tiêu trong tổng trượt, thành phần cùng dấu cộng dồn tới hệ số  $1/(1 - \mu)$ , tức gấp 10 lần khi  $\mu = 0,9$ .

**Hệ quả** tốc độ hội tụ chuyển từ phụ thuộc  $\kappa$  sang phụ thuộc  $\sqrt{\kappa}$ , một cải thiện có chứng minh cho trường hợp bậc hai.

**Vấn đề mới** momentum đã tích lũy không dừng lại được ngay: nghiệm bị vọt lố qua điểm cực tiểu, và trong một rãnh cong thì vọt lố nghĩa là văng ra thành.
- 3 Vấn đề** vọt lố vì gradient được đo ở nơi ta *đang* đứng chứ không phải nơi ta *sắp* tới.

**Giải pháp** **Nesterov**: tính gradient tại điểm nhìn trước  $\theta - \eta \mu v$ . Hiệu giữa gradient ở điểm nhìn trước và gradient tại chỗ là một xấp xỉ của số hạng bậc hai. Nó hoạt động

như lực hãm tỉ lệ với tốc độ thay đổi của gradient, tức phanh trước khi đâm vào thành chứ không sau.

**Giá phải trả** thêm một lần bookkeeping, thường được cài bằng phép đổi biến nên chi phí thực tế bằng không.

**Vấn đề mới** vẫn còn nguyên vấn đề gốc: một  $\eta$  chung cho tất cả tham số.

**4 Vấn đề** các tham số có độ lớn gradient lệch nhau hàng bậc, nên không tồn tại một  $\eta$  đúng cho tất cả.

**Giải pháp** **AdaGrad**: tích lũy  $G \leftarrow G + g^2$  theo từng tọa độ và bước theo  $\eta g / (\sqrt{G} + \varepsilon)$ . Tham số hiếm được cập nhật nhận bước lớn, tham số hay được cập nhật nhận bước nhỏ; ý tưởng sinh ra từ các bài toán lỗi thừa và rất hợp lý ở đó.

**Vấn đề mới**  $G$  là một tổng đơn điệu tăng, nên bước học hiệu dụng tắt dần về 0. Trong một bài toán lỗi thì đó là tính năng; trong một mạng sâu chạy hàng trăm nghìn bước thì đó là cái chết: mô hình ngừng học trong khi loss còn cao.

**5 Vấn đề** mẫu số của AdaGrad nhớ quá lâu.

**Giải pháp** **RMSProp**: thay tổng bằng trung bình trượt phân rã,  $G \leftarrow \beta G + (1 - \beta)g^2$ . Bộ nhớ trở thành hữu hạn với chân trời khoảng  $1/(1 - \beta)$  bước, tức chừng 1000 bước khi  $\beta = 0,999$ , nên bước học hiệu dụng không tắt.

**Vấn đề mới** hướng đi vẫn là gradient thô nên vẫn nhiễu; và trung bình trượt khởi tạo từ 0 nên trong những bước đầu  $G$  bị đánh giá thấp, mẫu số quá nhỏ, bước đi quá dài đúng vào lúc mô hình dễ tổn thương nhất.

**6 Vấn đề** cần cả hai: hướng đã lọc nhiễu và thang bước riêng cho từng tham số.

**Giải pháp** **Adam** [9] giữ hai trung bình trượt:  $m$  cho gradient và  $v$  cho bình phương gradient. Nó hiệu chỉnh độ chệch bằng  $\hat{m} = m/(1 - \beta_1^t)$  và  $\hat{v} = v/(1 - \beta_2^t)$ , rồi bước theo  $\eta \hat{m} / (\sqrt{\hat{v}} + \varepsilon)$ . Hiệu chỉnh độ chệch không phải chi tiết trang trí. Ở bước  $t = 1$  với  $\beta_1 = 0,9$  và  $\beta_2 = 0,999$ , ta có  $m = 0,1g$  và  $\sqrt{v} = 0,0316|g|$ . Bước chưa hiệu chỉnh vì thế dài gấp khoảng 3,2 lần bước mong muốn.

**Giá phải trả** hai trạng thái phụ cho mỗi tham số: với mô hình 100 triệu tham số ở fp32, tổng bộ nhớ là  $4P$  cho trọng số,  $4P$  cho gradient và  $8P$  cho trạng thái, tức 1,6 GB trước khi tính một byte kích hoạt nào.

**Vấn đề mới** hình phạt L2 cộng vào gradient bị chia cho  $\sqrt{v}$  giống hết phần còn lại, nên tham số nào có lịch sử gradient lớn thì bị suy giảm ít đi — nó không còn là weight decay theo nghĩa ban đầu nữa.

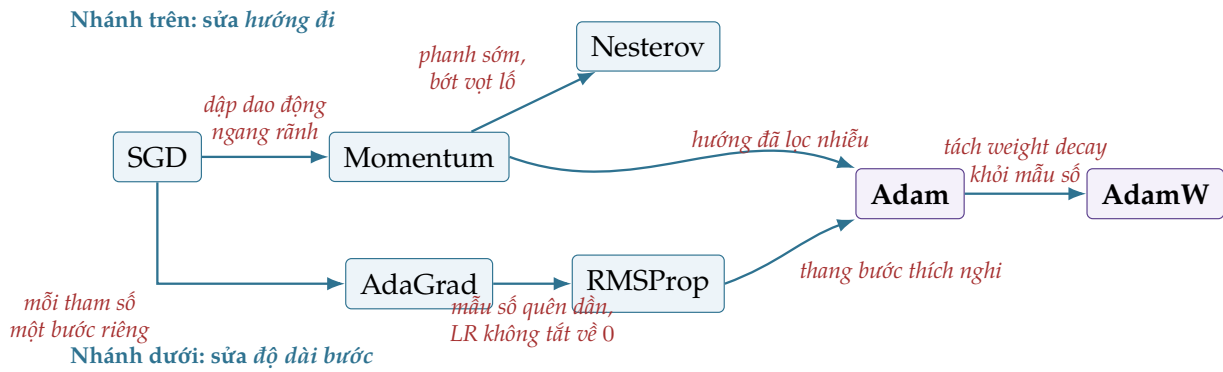
**7 Vấn đề** weight decay trong Adam không phải weight decay.

**Giải pháp** **AdamW** [10] tách hẳn hai thứ:  $\theta \leftarrow \theta - \eta(\hat{m}/(\sqrt{\hat{v}} + \varepsilon) + \lambda\theta)$ , tức phần suy giảm đi thẳng vào trọng số, không qua mẫu số thích nghi.

**Hệ quả** suy giảm trở lại thành một phép co đều, độc lập với độ lớn gradient, và đây là lý do AdamW là mặc định thực tế cho transformer lẫn cho các recipe CNN hiện đại.

**Giá phải trả**  $\lambda$  không còn so sánh được với  $\lambda$  của Adam cũ, nên mọi giá trị chép từ code cũ đều sai thang.

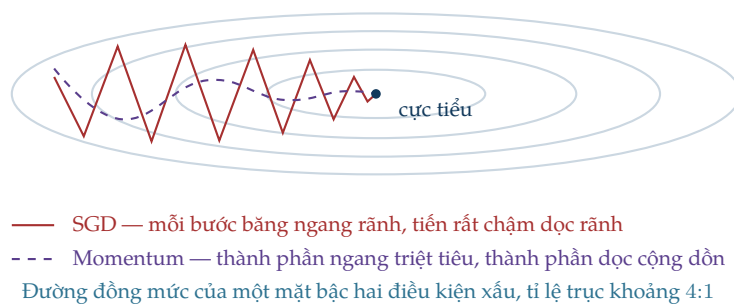
**Vấn đề mới** phần suy giảm nay tỉ lệ với  $\eta$ , nên nó tương tác với lịch học: giảm  $\eta$  theo cosine thì cũng đang giảm dần cường độ chính quy hoá, một hiệu ứng ít người nói ra.



**Hình 1.8.** Tiến hoá của optimizer không phải một chuỗi thẳng mà là hai nhánh độc lập rồi hợp lại. Nhánh trên sửa *hướng* của bước đi bằng cách lọc nhiễu theo thời gian; nhánh dưới sửa *độ dài* bước riêng cho từng tham số. Adam là chỗ hai nhánh gặp nhau, và AdamW là bản vá cho khiếm khuyết mà chính phép hợp nhất đó sinh ra. Mỗi mũi tên ghi đúng khiếm khuyết mà bước sau gỡ được.

Hình 1.8 nên được nhìn trước khi đọc mạch bảy bước, vì nó cho thấy ngay điều mà đọc tuần tự khó thấy: Adam không phải “bước tiến thứ sáu” mà là phép hợp nhất hai dòng giải quyết hai vấn đề khác nhau, và đó cũng là lý do nó thừa hưởng nhược điểm của cả hai.

Hai nhận xét khép lại mạch. Thứ nhất, hãy để ý rằng không bước nào trong bảy bước trên giải quyết vấn đề phi lồi; tất cả đều xử lý điều kiện xấu và nhiễu. Optimizer không tìm cực tiểu toàn cục và không hứa điều đó. Thứ hai, cả họ thích nghi có một cái giá chung ít được nêu: chúng *che giấu* một learning rate sai. Adam với  $\eta$  lệch một bậc vẫn cho đường cong loss trông chấp nhận được, trong khi SGD với sai lệch tương tự thì phân kỳ ngay và bạn biết mình sai. Đây là lý do đáng giữ SGD với momentum làm phép thử chẩn đoán, ngay cả khi bản chạy cuối dùng AdamW.



**Hình 1.9.** Vì sao momentum thắng trong rãnh hẹp. Thành phần vuông góc với rãnh đổi dấu mỗi bước nên triệt tiêu trong tổng trượt; thành phần dọc rãnh giữ nguyên dấu nên cộng dồn. Tỉ lệ trục trong hình chỉ khoảng 4:1 cho dễ nhìn; trong mạng thật số điều kiện lớn hơn hai tới ba bậc, nên hiệu ứng mạnh hơn nhiều.

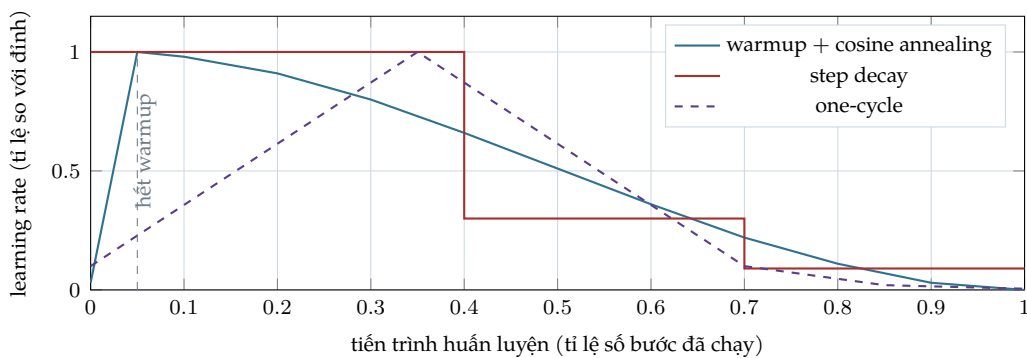
Hình 1.9 là toàn bộ trực giác hình học của bước hai trong mạch, và cũng cho thấy vì sao Nesterov cần thiết: đường đứt nét vẫn vọt qua tâm ở lần tiếp cận đầu tiên nếu  $\mu$  đủ lớn.

### C. Learning rate là siêu tham số quan trọng nhất, không có đối thủ

Nếu chỉ được tune một thứ, tune learning rate. Điều này nghe như khẩu hiệu, nhưng nó có cơ sở.  $\eta$  là đại lượng duy nhất nhân trực tiếp vào mọi bước cập nhật. Sai một bậc ở  $\eta$  đổi hẳn bản chất của quá trình; sai một bậc ở weight decay chỉ làm kết quả kém đi vài phần trăm. Kỹ năng đáng đầu tư nhất là nhận dạng *hình dạng* đường cong học, vì nó tiết kiệm hàng tuần chạy thử: loss tăng vọt hoặc thành NaN trong vài chục bước là  $\eta$  quá cao rõ ràng; loss tụt rất nhanh rồi phẳng ở mức cao và nhiễu mạnh là  $\eta$  hơi cao, hệ đang nảy quanh một lòng chảo mà không xuống được; loss giảm gần như tuyến tính và không có dấu hiệu bão hoà sau nhiều epoch là  $\eta$  quá thấp. Cách

re nhất để định vị là chạy một lượt quét: tăng  $\eta$  theo cấp số nhân trong vài trăm bước và vẽ loss theo  $\eta$ ; vùng dốc nhất trước điểm nổ là vùng đáng dùng.

*warmup* tăng  $\eta$  tuyến tính từ gần 0 lên giá trị đích, trong vài trăm tới vài nghìn bước. Thoạt nhìn nó là một thủ thuật tùy tiện. Thực ra nó có ít nhất hai lý do rõ ràng, và cả hai đều đã xuất hiện ở trên. Thứ nhất, mẫu số thích nghi của Adam ở những bước đầu được ước lượng từ rất ít mẫu, nên phương sai của  $\hat{v}$  rất lớn; một toạ độ vô tình có  $\hat{v}$  nhỏ sẽ nhận một bước khổng lồ và phá hỏng số ngay khi chưa có gì để cứu. Thứ hai, thống kê chạy của BatchNorm chưa hội tụ trong vài trăm bước đầu, nên hàm mà bạn đang tối ưu vẫn còn đang thay đổi dưới chân. Warmup mua thời gian cho cả hai bộ ước lượng đó. **Lịch learning rate** thì giải quyết một vấn đề khác: SGD với  $\eta$  cố định không hội tụ về một điểm mà tiến tới một phân bố dừng có phương sai tỉ lệ với  $\eta$ ; giảm  $\eta$  là thu nhỏ bán kính quả cầu mà nghiệm đang lang thang trong đó. Ba lịch phổ biến là step decay, cosine annealing [11] và one-cycle; khác biệt giữa chúng nhỏ hơn nhiều so với khác biệt giữa “có lịch” và “không có lịch”.



**Hình 1.10.** Ba lịch learning rate trên cùng một trục. Điều hình cho thấy mà lời văn khó nói: chúng khác nhau ở *chỗ* tiêu tiền, không ở tổng lượng. Step decay giữ nguyên bước dài rất lâu rồi cắt đột ngột. Cosine hạ liên tục nên tránh được cú sốc ở mỗi mốc cắt. One-cycle thì cố tình bắt đầu thấp và *tăng* lên đỉnh ở giữa, để đi qua vùng nhiễu nhanh hơn. Đoạn dốc đứng ở đầu đường cosine là warmup: nó tồn tại vì bộ ước lượng  $\hat{v}$  của Adam và thống kê BatchNorm đều chưa đáng tin ở vài trăm bước đầu.

Hình 1.10 cũng cho thấy vì sao so sánh hai bài báo dùng hai lịch khác nhau là việc rất dễ sai: hai mô hình cùng số epoch có thể đã trải qua hai quá trình rất khác nhau.

#### D. Đổi batch size thì đổi gì theo (linear scaling rule)

**quy tắc tỉ lệ tuyến tính** (*linear scaling rule*) nói rằng nhân batch size lên  $k$  lần thì nhân learning rate lên  $k$  lần. Quy tắc này đã được kiểm chứng trên ImageNet với warmup đủ dài, tới batch 8192 [12]. Lập luận đơn giản: một bước với batch  $kB$  nên tiến xa bằng  $k$  bước với batch  $B$ , và trung bình của  $k$  gradient liên tiếp xấp xỉ gradient của batch gộp khi trọng số chưa đổi nhiều.

Nhưng nguyên nhân sâu hơn mới đáng nhớ, vì nó nói rằng đây không chỉ là chuyện tốc độ. Phương sai của gradient minibatch tỉ lệ nghịch với batch size, nên batch lớn cho gradient sạch hơn — và nhiễu gradient chính là một dạng chính quy hoá ngầm. Batch quá lớn không chỉ “khó tune hơn”: nó *lấy đi* một nguồn chính quy hoá, và các quan sát trên ImageNet cho thấy vượt một ngưỡng nhất định thì độ chính xác kiểm thử tụt dù loss huấn luyện vẫn tốt [13]. Smith và Le đưa ra một cách nhìn thống nhất: giảm  $\eta$  và tăng batch size là hai cách làm cùng một việc, đều là giảm mức nhiễu  $\eta/B$  [14]. Cách nhìn đó giải thích cả linh cảm “tăng batch thì phải tăng LR”, lẫn lý do quy tắc vỡ. Khi  $\eta$  đã lớn tới mức không tăng được nữa vì lý do ổn định, mối liên hệ đứt.

↪ **đối lập với C/12/04** — nhiễu gradient vừa là thứ quy tắc tỉ lệ tuyến tính muốn bảo toàn, vừa là nguồn chính quy hoá ngầm mà batch lớn xoá mất

### E. Theo dõi sức khoẻ huấn luyện (training health), vì loss là chỉ báo quá chậm

Đường cong loss cho biết *đã* có vấn đề, thường sau hàng nghìn bước. Ba phép đo dưới đây rẻ như nhau nhưng phát hiện sớm hơn nhiều, vì chúng đo trạng thái bên trong chứ không đo kết quả.

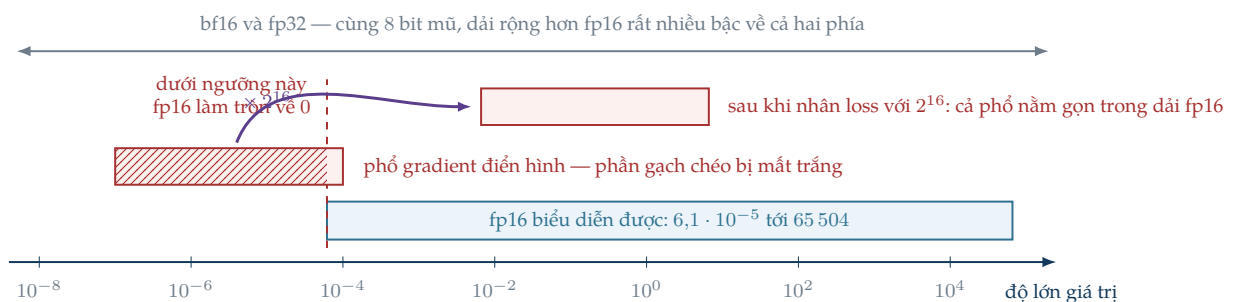
- **Tỉ lệ cập nhật trên trọng số** (*update-to-weight ratio*), theo từng tầng:  $\|\eta \Delta \theta\| / \|\theta\|$ .
  - Kinh nghiệm thực hành phổ biến: khoảng  $10^{-3}$  là lành mạnh; trên  $10^{-2}$  là mỗi bước làm đổi trọng số vài phần trăm, tức quá lớn; dưới  $10^{-4}$  là tầng đó thực chất đang đứng yên.
  - Vì sao dùng nó thay cho chuẩn gradient thô: nó không thứ nguyên, nên so sánh được giữa các tầng có thang hoàn toàn khác nhau.
- **Phân bố kích hoạt theo chiều sâu** — vẽ độ lệch chuẩn của kích hoạt từng tầng. Co dần theo chiều sâu là vanishing đang hình thành; phình lên là exploding. Đây chính là đại lượng  $\rho^L$  của mục trước, quan sát trực tiếp.
- **Tỉ lệ ReLU chết** — phần trăm nơ-ron cho 0 trên toàn bộ một batch xác thực. Vài phần trăm là bình thường; ba mươi phần trăm ở một tầng nghĩa là  $\eta$  đã có lúc quá cao, kể cả khi hiện tại nó đã ổn.

Cả ba chỉ báo chỉ tốn vài dòng hook và nên được ghi lại mặc định, vì giá trị của chúng nằm ở chỗ chúng *đi trước* loss chứ không đi sau.

### F. Mixed precision: chuyện của dải động, không phải của độ chính xác

Chuyển sang **fp16** làm training ra NaN, và nguyên nhân thường bị hiểu nhầm là “thiếu độ chính xác”. Thủ phạm thật là **dải động**. fp16 có 5 bit mũ và 10 bit định trị, nên số chuẩn hoá nhỏ nhất là khoảng  $6,1 \cdot 10^{-5}$  và số lớn nhất là 65 504. Gradient trong mạng thị giác thường nằm quanh  $10^{-7}$  tới  $10^{-4}$ ; phần lớn phổ đó rơi xuống dưới ngưỡng chuẩn hoá của fp16 và bị làm tròn về 0. Mạng không nổ, nó *tê liệt*: một phần các tầng đơn giản là không nhận được tín hiệu nào.

Lời sửa của **loss scaling** [15] tinh tế ở chỗ nó không đụng vào định dạng số. Nhân loss với một hệ số  $S$  cỡ  $2^{16}$  trước khi gọi backward; vì backward tuyến tính theo adjoint đầu vào, mọi gradient bị nhân đúng  $S$  lần và cả phổ dịch lên vùng biểu diễn được; trước khi optimizer bước, chia gradient lại cho  $S$ . Bản động của kỹ thuật này nhân  $S$  lên gấp đôi sau mỗi vài nghìn bước sạch và chia đôi ngay khi phát hiện inf hoặc NaN, bỏ luôn bước đó — tức là nó tự dò biên trên của dải động.



**Hình 1.11.** Vì sao fp16 làm huấn luyện ra NaN, và vì sao loss scaling chữa được. Trục là độ lớn giá trị theo thang log. Dải fp16 (nền xanh) không bao phủ phần dưới của phổ gradient điển hình, nên phần gạch chéo bị làm tròn về 0 — mạng không nổ mà *tê liệt*. Nhân loss với  $2^{16}$  dịch toàn bộ phổ sang phải mà không đổi hình dạng của nó, đưa cả phổ vào vùng biểu diễn được; chia lại trước bước cập nhật thì không mất gì. bf16 giải bài toán theo hướng khác: nó giữ nguyên số bit mũ của fp32 nên dải đủ rộng từ đầu, đổi lại chỉ còn 7 bit định trị.

Hình 1.11 làm rõ một điểm mà hai chữ “mixed precision” che mất: vấn đề nằm ở *vị trí* của phổ gradient so với dải biểu diễn được, không nằm ở số chữ số có nghĩa. Đó cũng là lý do lời sửa là một phép nhân vô hướng chứ không phải một thuật toán mới.

**bf16** giải bài toán theo hướng khác: giữ 8 bit mũ như fp32 nên dải động y hệt fp32 và không cần loss scaling, đổi lại chỉ còn 7 bit định trị.

Chính 7 bit định trị đó dẫn tới điểm kỹ thuật đáng nhớ nhất của cả mục. Độ chính xác tương đối của bf16 là khoảng  $2^{-8} \approx 4 \cdot 10^{-3}$ . Nhưng ta vừa nói tỉ lệ cập nhật trên trọng số lành mạnh là khoảng  $10^{-3}$ . Nghĩa là bước cập nhật *nhỏ hơn một đơn vị cuối cùng của trọng số*: nếu bạn cộng nó vào một trọng số lưu ở bf16, phép cộng làm tròn về đúng giá trị cũ và bước đi biến mất hoàn toàn. Đây là lý do mọi cài đặt mixed precision nghiêm túc giữ một bản *master weights* ở fp32 cho optimizer, dù forward và backward chạy ở nửa độ chính xác; và cũng là lý do bảng kế toán bộ nhớ không giảm một nửa như người ta hy vọng.

→ cùng nguyên lý với D/20 — lượng tử hoá lúc suy luận hỏi đúng câu này: bit chia thế nào giữa mũ và định trị.

Cuối cùng, **cắt ngưỡng gradient** (*gradient clipping*) theo chuẩn toàn cục là bắt buộc với RNN và thường cần với transformer: nó chặn bước đi khi mặt mất mát có vách dựng đứng, nơi gradient tại một điểm không còn nói gì đúng về hàm ở khoảng cách một bước.

## G. Giới hạn và trường hợp hỏng

Linear scaling rule vỡ ở batch lớn, và nó vỡ theo kiểu khó phát hiện: loss huấn luyện vẫn đẹp, chỉ có độ chính xác kiểm thử tụt. Nếu bạn chỉ theo dõi loss huấn luyện, bạn sẽ kết luận rằng batch lớn hoạt động tốt.

Loss scaling che giấu NaN thật. Bộ chia tỉ lệ động bỏ qua mọi bước có tràn số. Giả sử mô hình của bạn sinh NaN vì một lỗi thật, chẳng hạn chia cho 0 hoặc lấy log của số âm. Bạn sẽ thấy một quá trình huấn luyện tưởng như bình thường, nhưng thực chất nó bỏ một tỉ lệ lớn số bước. Đáng ghi lại số bước bị bỏ như một chỉ báo riêng.

Weight decay áp nhầm chỗ là lỗi phổ biến và tốn kém: các tham số  $\gamma, \beta$  của normalization và các bias không nên bị suy giảm, vì chúng không phải “độ phức tạp” theo nghĩa mà weight decay muốn phạt, và co chúng về 0 làm hỏng chính phép chuẩn hoá. Phần lớn recipe tốt loại các nhóm này ra khỏi decay một cách tường minh.

Cuối cùng, những con số trong mục này thuộc ba mức tin cậy khác nhau và nên được đối xử khác nhau. Hệ số  $1/(1 - \mu)$ , tỉ lệ 3,2 của bước Adam đầu tiên, và các ngưỡng của fp16 là suy dẫn hoặc tra cứu, nên chắc chắn. Kết quả linear scaling tới batch 8192 là một phép đo đã công bố trên một cấu hình cụ thể, không phải một định luật. Còn ngưỡng  $10^{-3}$  cho tỉ lệ cập nhật trên trọng số chỉ là kinh nghiệm thực hành, được truyền lại rộng rãi. Tôi chưa thấy nghiên cứu hệ thống nào xác định lại nó cho các kiến trúc hiện đại. Hãy dùng nó như một cờ báo động, đừng dùng như mục tiêu để tối ưu.

### CẠM BÃY

`torch.optim.Adam(..., weight_decay= $\lambda$ )` *không* phải AdamW: nó cộng  $\lambda\theta$  vào gradient rồi cho cả cục đi qua mẫu số thích nghi, đúng cái lỗi mà AdamW sinh ra để sửa. Muốn weight decay tách rời thì phải dùng AdamW. Đây là lỗi mà rất nhiều code cũ vẫn mắc, và nó đặc biệt khó phát hiện. Hậu quả không phải một exception mà là điểm số thấp hơn vài phần trăm. Mức đó vừa đủ nhỏ để bạn đổ cho kiến trúc, vừa đủ lớn để mất một vị trí trên bảng xếp hạng.

### CẦN NHỚ

Optimizer chỉ định hình lại bước đi; nó không cứu được một learning rate sai. Thứ tự đầu tư công sức: learning rate và lịch của nó trước, batch size cùng quy tắc tỉ lệ sau, chọn



optimizer cuối cùng. Bảy cái tên trong mạch không phải bảy lựa chọn, mà là bảy lần vá liên tiếp trên cùng một vấn đề. [T2]

## H. Kết nối

Mục này giả định gradient đã lành mạnh, tức giả định toàn bộ [C/12/02-lam-cho-training-chay-duoc](#). Nó chuyển tiếp sang [C/12/04-tong-quat-hoa-va-chan-doan](#). Ở đó, nhiều gradient của quy tắc tỉ lệ tuyến tính được nhìn lại như một công cụ chính quy hoá. Cũng ở đó, bảng kế toán bộ nhớ của trạng thái Adam trở thành lý do tồn tại của ZeRO. Phần dài dòng của fp16 và bf16 là tiền đề trực tiếp cho [D/20-toi-uu-va-nen-mo-hinh](#) (lượng tử hoá), nơi cùng một câu hỏi được đặt cho suy luận thay vì cho huấn luyện, và cho [B/11-gioi-han-tinh-toan](#) về phía băng thông. Nếu bạn muốn thấy các lựa chọn trong mục này ảnh hưởng thế nào tới kết luận của một paper, [E/24-thi-nghiem-va-ablation](#) là chương nói rằng phần lớn “cải tiến kiến trúc” hoá ra là cải tiến lịch học.

### ▮ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Momentum sửa được gì mà SGD không làm được, và nó tạo ra vấn đề mới nào buộc Nesterov phải ra đời?
2. AdaGrad và RMSProp khác nhau đúng một phép toán. Phép toán đó là gì, và vì sao khác biệt nhỏ ấy lại là ranh giới giữa “dùng được cho mạng sâu” và “không dùng được”?
3. Vì sao AdamW khác Adam cộng L2, và bạn kỳ vọng khác biệt thể hiện ở đại lượng đo được nào?
4. Vì sao warmup cần thiết với Adam hơn là với SGD, nếu trả lời bằng tính chất của bộ ước lượng  $\hat{v}$ ?
5. Bạn tăng batch size gấp 8 và nhân learning rate lên 8; loss huấn luyện tốt hơn nhưng accuracy kiểm thử tụt 1 phần trăm — hai cách giải thích là gì, và phép đo nào phân biệt chúng?
6. Vì sao huấn luyện ở bf16 vẫn cần giữ trọng số fp32, dù bf16 có dải động bằng fp32?
7. Đường cong học giảm nhanh rồi phẳng ở mức cao — ba nguyên nhân khả dĩ là gì, và ba phép đo nào phân biệt chúng?

- [1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [2] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [3] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *AISTATS*, 2010.
- [4] K. He, X. Zhang, S. Ren, et al., “Delving Deep into Rectifiers,” in *ICCV*, 2015.
- [5] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [6] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [7] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [8] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [9] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [10] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [11] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” in *ICLR*, 2017.
- [12] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [13] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [14] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [15] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [16] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [17] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [18] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [19] G. Huang, Y. Sun, Z. Liu, et al., “Deep Networks with Stochastic Depth,” in *ECCV*, 2016.
- [20] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [21] H. Zhang, M. Cisse, Y. N. Dauphin, et al., “mixup: Beyond Empirical Risk Minimization,” in *ICLR*, 2018.
- [22] S. Yun, D. Han, S. J. Oh, et al., “CutMix: Regularization Strategy to Train Strong Classifiers,” in *ICCV*, 2019.
- [23] E. D. Cubuk, B. Zoph, J. Shlens, et al., “RandAugment: Practical Automated Data Augmentation,” in *CVPR Workshops*, 2020.

- [24] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [25] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [26] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *NeurIPS*, 2012.
- [27] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in *ICLR*, 2015.
- [28] C. Szegedy, W. Liu, et al., “Going Deeper with Convolutions,” in *CVPR*, 2015.
- [29] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [30] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [31] G. Huang, Z. Liu, L. van der Maaten, et al., “Densely Connected Convolutional Networks,” in *CVPR*, 2017.
- [32] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [33] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [34] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [35] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [36] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [37] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.
- [38] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [39] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [40] J. Su, M. Ahmed, Y. Lu, et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, vol. 568, 2024.
- [41] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” *arXiv:2312.00752*, 2023.
- [42] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [43] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.
- [44] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [45] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [46] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [47] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [48] R. Girshick, J. Donahue, T. Darrell, et al., “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation,” in *CVPR*, 2014.

- [49] R. Girshick, "Fast R-CNN," in *ICCV*, 2015.
- [50] S. Ren, K. He, R. Girshick, et al., "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in *NeurIPS*, 2015.
- [51] T.-Y. Lin, P. Dollár, R. Girshick, et al., "Feature Pyramid Networks for Object Detection," in *CVPR*, 2017.
- [52] K. He, G. Gkioxari, P. Dollár, et al., "Mask R-CNN," in *ICCV*, 2017.
- [53] D. Bolya, S. Foley, J. Hays, et al., "TIDE: A General Toolbox for Identifying Object Detection Errors," in *ECCV*, 2020.
- [54] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [55] J. Redmon, S. Divvala, R. Girshick, et al., "You Only Look Once: Unified, Real-Time Object Detection," in *CVPR*, 2016.
- [56] W. Liu, D. Anguelov, D. Erhan, et al., "SSD: Single Shot MultiBox Detector," in *ECCV*, 2016.
- [57] T.-Y. Lin, P. Goyal, R. Girshick, et al., "Focal Loss for Dense Object Detection," in *ICCV*, 2017.
- [58] Z. Tian, C. Shen, H. Chen, et al., "FCOS: Fully Convolutional One-Stage Object Detection," in *ICCV*, 2019.
- [59] X. Zhou, D. Wang, and P. Krähenbühl, "Objects as Points," *arXiv:1904.07850*, 2019.
- [60] S. Zhang, C. Chi, Y. Yao, et al., "Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection," in *CVPR*, 2020.
- [61] Z. Ge, S. Liu, Z. Li, et al., "OTA: Optimal Transport Assignment for Object Detection," in *CVPR*, 2021.
- [62] N. Bodla, B. Singh, R. Chellappa, et al., "Soft-NMS: Improving Object Detection with One Line of Code," in *ICCV*, 2017.
- [63] X. Zhu, W. Su, L. Lu, et al., "Deformable DETR: Deformable Transformers for End-to-End Object Detection," in *ICLR*, 2021.
- [64] H. Zhang, F. Li, S. Liu, et al., "DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection," in *ICLR*, 2023.
- [65] J. Long, E. Shelhamer, and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," in *CVPR*, 2015.
- [66] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *MICCAI*, 2015.
- [67] L.-C. Chen, G. Papandreou, I. Kokkinos, et al., "DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs," *IEEE TPAMI*, vol. 40, no. 4, pp. 834–848, 2018.
- [68] E. Xie, W. Wang, Z. Yu, et al., "SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers," in *NeurIPS*, 2021.
- [69] B. Cheng, I. Misra, A. G. Schwing, et al., "Masked-Attention Mask Transformer for Universal Image Segmentation," in *CVPR*, 2022.
- [70] A. Kirillov, K. He, R. Girshick, et al., "Panoptic Segmentation," in *CVPR*, 2019.
- [71] A. Kirillov, E. Mintun, N. Ravi, et al., "Segment Anything," in *ICCV*, 2023.
- [72] K. Sun, B. Xiao, D. Liu, et al., "Deep High-Resolution Representation Learning for Human Pose Estimation," in *CVPR*, 2019.

- [73] Z. Cao, T. Simon, S.-E. Wei, et al., “Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields,” in *CVPR*, 2017.
- [74] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [75] J. Deng, J. Guo, N. Xue, et al., “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *CVPR*, 2019.
- [76] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [77] K. Xu, J. Ba, R. Kiros, et al., “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention,” in *ICML*, 2015.
- [78] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *ICLR*, 2015.
- [79] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [80] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [81] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [82] K. Simonyan and A. Zisserman, “Two-Stream Convolutional Networks for Action Recognition in Videos,” in *NeurIPS*, 2014.
- [83] J. Carreira and A. Zisserman, “Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset,” in *CVPR*, 2017.
- [84] D. Tran, H. Wang, L. Torresani, et al., “A Closer Look at Spatiotemporal Convolutions for Action Recognition,” in *CVPR*, 2018.
- [85] C. Feichtenhofer, H. Fan, J. Malik, et al., “SlowFast Networks for Video Recognition,” in *ICCV*, 2019.
- [86] A. Dosovitskiy, P. Fischer, E. Ilg, et al., “FlowNet: Learning Optical Flow with Convolutional Networks,” in *ICCV*, 2015.
- [87] D. Sun, X. Yang, M.-Y. Liu, et al., “PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume,” in *CVPR*, 2018.
- [88] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.
- [89] A. Bewley, Z. Ge, L. Ott, et al., “Simple Online and Realtime Tracking,” in *ICIP*, 2016.
- [90] Y. Zhang, P. Sun, Y. Jiang, et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *ECCV*, 2022.
- [91] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [92] K. Bernardin and R. Stiefelhagen, “Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics,” *EURASIP J. Image and Video Processing*, 2008.
- [93] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [94] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [95] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.

- [96] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *CVPR*, 2024.
- [97] D. Eigen, C. Puhrsch, and R. Fergus, “Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network,” in *NeurIPS*, 2014.
- [98] R. Ranftl, K. Lasinger, D. Hafner, et al., “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer,” *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [99] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IROS*, 2012.
- [100] C. R. Qi, H. Su, K. Mo, et al., “PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation,” in *CVPR*, 2017.
- [101] C. R. Qi, L. Yi, H. Su, et al., “PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space,” in *NeurIPS*, 2017.
- [102] A. H. Lang, S. Vora, H. Caesar, et al., “PointPillars: Fast Encoders for Object Detection from Point Clouds,” in *CVPR*, 2019.
- [103] T. Müller, A. Evans, C. Schied, et al., “Instant Neural Graphics Primitives with a Multiresolution Hash Encoding,” *ACM TOG*, vol. 41, no. 4, 2022.
- [104] B. Kerbl, G. Kopanas, T. Leimkühler, et al., “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *ACM TOG*, vol. 42, no. 4, 2023.
- [105] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.
- [106] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” in *ICLR*, 2014.
- [107] I. Goodfellow, J. Pouget-Abadie, M. Mirza, et al., “Generative Adversarial Nets,” in *NeurIPS*, 2014.
- [108] T. Karras, S. Laine, and T. Aila, “A Style-Based Generator Architecture for Generative Adversarial Networks,” in *CVPR*, 2019.
- [109] J. Ho, A. Jain, and P. Abbeel, “Denoising Diffusion Probabilistic Models,” in *NeurIPS*, 2020.
- [110] Y. Song, J. Sohl-Dickstein, D. P. Kingma, et al., “Score-Based Generative Modeling Through Stochastic Differential Equations,” in *ICLR*, 2021.
- [111] J. Johnson, A. Alahi, and L. Fei-Fei, “Perceptual Losses for Real-Time Style Transfer and Super-Resolution,” in *ECCV*, 2016.
- [112] P. Isola, J.-Y. Zhu, T. Zhou, et al., “Image-to-Image Translation with Conditional Adversarial Networks,” in *CVPR*, 2017.
- [113] L. A. Gatys, A. S. Ecker, and M. Bethge, “Image Style Transfer Using Convolutional Neural Networks,” in *CVPR*, 2016.
- [114] X. Huang and S. Belongie, “Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization,” in *ICCV*, 2017.
- [115] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.
- [116] J. Song, C. Meng, and S. Ermon, “Denoising Diffusion Implicit Models,” in *ICLR*, 2021.
- [117] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, et al., “Flow Matching for Generative Modeling,” in *ICLR*, 2023.
- [118] J. Ho and T. Salimans, “Classifier-Free Diffusion Guidance,” *arXiv:2207.12598*, 2022.
- [119] R. Rombach, A. Blattmann, D. Lorenz, et al., “High-Resolution Image Synthesis with Latent Diffusion Models,” in *CVPR*, 2022.
- [120] W. Peebles and S. Xie, “Scalable Diffusion Models with Transformers,” in *ICCV*, 2023.



- [121] L. Zhang, A. Rao, and M. Agrawala, “Adding Conditional Control to Text-to-Image Diffusion Models,” in *ICCV*, 2023.
- [122] Y. Blau and T. Michaeli, “The Perception-Distortion Tradeoff,” in *CVPR*, 2018.
- [123] C. Ledig, L. Theis, F. Huszár, et al., “Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network,” in *CVPR*, 2017.
- [124] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., “Image Quality Assessment: From Error Visibility to Structural Similarity,” *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [125] M. Heusel, H. Ramsauer, T. Unterthiner, et al., “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium,” in *NeurIPS*, 2017.
- [126] T. Brooks, B. Mildenhall, T. Xue, et al., “Unprocessing Images for Learned Raw Denoising,” in *CVPR*, 2019.
- [127] A. Abdelhamed, S. Lin, and M. S. Brown, “A High-Quality Denoising Dataset for Smartphone Cameras,” in *CVPR*, 2018.
- [128] J. Deng, W. Dong, R. Socher, et al., “ImageNet: A Large-Scale Hierarchical Image Database,” in *CVPR*, 2009.
- [129] A. Radford, J. W. Kim, C. Hallacy, et al., “Learning Transferable Visual Models from Natural Language Supervision,” in *ICML*, 2021.
- [130] J. Li, D. Li, S. Savarese, et al., “BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models,” in *ICML*, 2023.
- [131] H. Liu, C. Li, Q. Wu, et al., “Visual Instruction Tuning,” in *NeurIPS*, 2023.
- [132] S. Kornblith, J. Shlens, and Q. V. Le, “Do Better ImageNet Models Transfer Better?” In *CVPR*, 2019.
- [133] T. Chen, S. Kornblith, M. Norouzi, et al., “A Simple Framework for Contrastive Learning of Visual Representations,” in *ICML*, 2020.
- [134] K. He, H. Fan, Y. Wu, et al., “Momentum Contrast for Unsupervised Visual Representation Learning,” in *CVPR*, 2020.
- [135] J.-B. Grill, F. Strub, F. Altché, et al., “Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning,” in *NeurIPS*, 2020.
- [136] M. Caron, H. Touvron, I. Misra, et al., “Emerging Properties in Self-Supervised Vision Transformers,” in *ICCV*, 2021.
- [137] K. He, X. Chen, S. Xie, et al., “Masked Autoencoders Are Scalable Vision Learners,” in *CVPR*, 2022.
- [138] A. van den Oord, Y. Li, and O. Vinyals, “Representation Learning with Contrastive Predictive Coding,” in *arXiv:1807.03748*, 2018.
- [139] J. Kaplan, S. McCandlish, T. Henighan, et al., “Scaling Laws for Neural Language Models,” *arXiv:2001.08361*, 2020.
- [140] X. Zhai, A. Kolesnikov, N. Houlsby, et al., “Scaling Vision Transformers,” in *CVPR*, 2022.
- [141] J. Hoffmann, S. Borgeaud, A. Mensch, et al., “Training Compute-Optimal Large Language Models,” in *NeurIPS*, 2022.
- [142] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [143] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [144] N. Houlsby, A. Giurgiu, S. Jastrzebski, et al., “Parameter-Efficient Transfer Learning for NLP,” in *ICML*, 2019.

- [145] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [146] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *RecSys*, 2019.

## 1.5 Tổng quát hoá và chẩn đoán (Generalization and Diagnosis)

**PHỤ THUỘC** C/12/03-toi-uu-hoa (nhiều gradient, kế toán bộ nhớ của trạng thái Adam); B/04-nen-tang-hoc-tu-du-lieu và B/07-chia-du-lieu-va-danh-gia (vì sao “tổng quát hoá” chỉ có nghĩa khi tập chia đúng).

### TÓM TẮT

Mục này xử lý khoảng cách giữa “mạng khớp được dữ liệu huấn luyện” và “mạng dùng được”, rồi biến phần còn lại thành quy trình chẩn đoán. Đọc xong bạn xếp được các công cụ **chính quy hoá** (*regularization*) theo hiệu quả thực tế trong thị giác, thay vì theo độ nổi tiếng. Bạn biết đưa mô hình lên nhiều GPU theo đúng thứ tự tăng dần độ phức tạp. Và bạn có một quy trình bắt lỗi cho kết luận trong vài phút thay vì vài ngày. Nếu chỉ nhớ một điều: trước khi chạy full, hãy *overfit một batch 20–50 mẫu tới loss gần 0*. Phép thử này tách bạch “vòng lặp huấn luyện có bug” khỏi “mô hình không tổng quát hoá được”. Phần lớn thời gian bị mất là do người ta nhầm hai thứ đó với nhau.

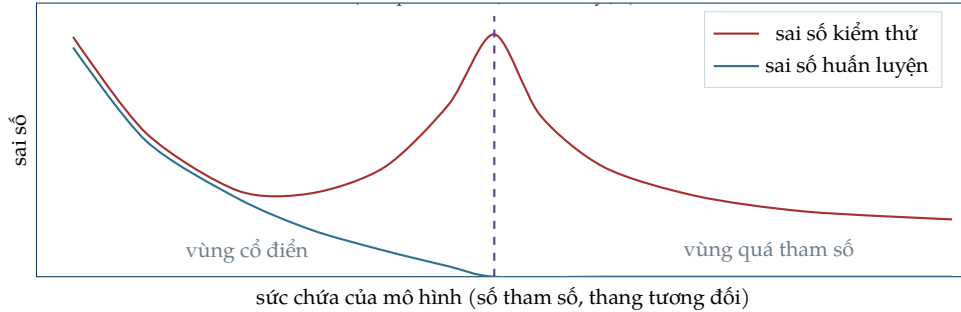
### KIẾN THỨC CẦN CÓ TRƯỚC

- **tập huấn luyện, xác thực, kiểm thử** — ba phần rời nhau của dữ liệu: phần để học, phần để chọn siêu tham số và dùng đúng lúc, phần chỉ dùng đúng một lần để báo cáo. Trộn lẫn ba phần là cách phổ biến nhất để tự lừa mình.
- **overfitting** — mô hình khớp rất tốt dữ liệu huấn luyện nhưng kém trên dữ liệu mới, vì nó đã học cả những chi tiết chỉ đúng với tập huấn luyện.
- **chính quy hoá (regularization)** — mọi kỹ thuật nhằm thu hẹp khoảng cách giữa kết quả huấn luyện và kết quả thực tế — bằng cách ràng buộc mô hình, thêm nhiễu, hoặc thêm thông tin.
- **augmentation** — sinh thêm mẫu huấn luyện bằng cách biến đổi mẫu có sẵn (lật, cắt, đổi màu); mỗi phép biến đổi là một tuyên bố “nhân không đổi dưới phép này”.
- **sức chứa (capacity)** — độ phong phú của lớp hàm mà mô hình biểu diễn được, thường xấp xỉ bằng số tham số; lý thuyết cổ điển gắn sức chứa với overfitting, còn thực nghiệm hiện đại thì không.
- **rò rỉ dữ liệu (data leakage)** — thông tin của tập đánh giá lọt vào quá trình huấn luyện, khiến điểm số đẹp giả tạo — ví dụ hai khung hình liên tiếp của cùng một video nằm ở hai bên ranh giới chia tập.
- **tiến trình và GPU** — trong huấn luyện phân tán, mỗi GPU thường chạy một tiến trình riêng giữ một bản mô hình; “đồng bộ” nghĩa là các tiến trình trao đổi gradient để giữ các bản đó giống nhau.
- **EMA (trung bình trượt lũy thừa)** — một bản trọng số phụ được cập nhật bằng trung bình có trọng số giảm dần theo thời gian; nó mượt hơn trọng số hiện thời và thường cho kết quả đánh giá tốt hơn.

### A. Đặt vấn đề

Loss huấn luyện xuống đẹp, độ chính xác trên tập kiểm thử thì không. Đây là bài toán cũ nhất của học máy, nhưng cách hiểu nó trong deep learning đã thay đổi hẳn và việc mang nguyên trực giác cũ sang là một nguồn suy luận sai lớn. Trực giác cổ điển nói: mô hình quá nhiều tham số so

với dữ liệu thì học thuộc lòng, nên hãy giảm sức chứa mô hình. Nhưng một mạng hiện đại khớp hoàn hảo được cả một tập ImageNet với nhãn đã bị xáo ngẫu nhiên [16]. Sức chứa của nó dư sức học thuộc. Vậy mà chính nó vẫn tổng quát hoá tốt khi nhãn là chân trị. Các cận sai số dựa trên sức chứa do đó gần như rỗng, và hiện tượng *double descent* còn cho thấy tăng sức chứa vượt qua điểm nội suy có thể làm sai số kiểm thử *giảm* trở lại [17], [18].



**Hình 1.12.** Hiện tượng giảm kép (*double descent*), vẽ định tính. Trực giác cổ điển chỉ mô tả đúng nửa trái: sai số kiểm thử đi xuống rồi đi lên khi mô hình lớn dần. Nhưng ngay tại ngưỡng nội suy — chỗ mô hình vừa đủ sức khớp hết dữ liệu huấn luyện — sai số kiểm thử đạt *đỉnh* rồi *giảm trở lại* khi ta tiếp tục tăng sức chứa. Đây là lý do câu “mô hình quá lớn thì sẽ overfit” không dùng được làm quy tắc, và là lý do phải chọn công cụ chính quy hoá theo đo đặc chứ không theo lý thuyết sức chứa. Hình là minh hoạ định tính; vị trí đỉnh và độ sâu của lần giảm thứ hai phụ thuộc tác vụ và lượng nhiễu nhân.

Hình 1.12 nói rõ điều mà một câu trích dẫn không nói được: hai trực giác đối nghịch nhau đều đúng, chỉ là đúng ở hai nửa khác nhau của cùng một trục.

Hệ quả thực hành rất cụ thể: đừng chọn công cụ theo lý thuyết sức chứa, hãy chọn theo hiệu quả đo được trong đúng lĩnh vực của bạn. Và vì phần lớn các lần “mô hình không tổng quát hoá” thực ra là “vòng lặp huấn luyện có lỗi”, phần thứ hai của mục này quan trọng không kém phần thứ nhất. Giá trị cụ thể: sau mục này bạn có một cây quyết định để chẩn đoán một lần chạy hỏng, và một bảng triệu chứng để tra thay vì đoán.

→ **tiền đề** B/07 — khoảng cách train-val chỉ có nghĩa nếu tập được chia đúng; chia sai thì mọi kết luận ở đây sụp.

### TRỰC GIÁC

Chính quy hoá thường được kể như “ràng buộc mô hình cho nó bớt phức tạp”. Nhưng công cụ mạnh nhất trong thị giác lại là augmentation, và nó không ràng buộc gì cả: nó *thêm thông tin*. Hãy nghĩ theo kiểu người hiệu chuẩn cảm biến: khi bạn lắc camera qua nhiều tư thế lúc hiệu chuẩn, bạn không làm mô hình camera đơn giản đi, bạn đang nói cho bộ ước lượng biết đại lượng nào *không được phép* thay đổi kết quả. Augmentation làm đúng việc đó với mạng: mỗi phép biến đổi là một câu tuyên bố “nhãn không đổi dưới phép này”. Vì thế nó thắng các công cụ ràng buộc sức chứa — nó trả lời một câu hỏi mà chúng không trả lời được.

## B. Bộ công cụ chính quy hoá, xếp theo hiệu quả thực tế trong thị giác

Thứ tự dưới đây là thứ tự *hiệu quả thực hành trong computer vision*, không phải thứ tự nổi tiếng. Nó khác đáng kể thứ tự mà sách giáo khoa trình bày.

- **Augmentation** — gần như luôn cho lợi ích lớn nhất trên mỗi giờ công bỏ ra, vì nó là công cụ duy nhất trong danh sách *thêm thông tin* thay vì lấy bớt tự do. Chi tiết ở mục C.

- **Early stopping** — dừng khi loss xác thực bắt đầu xấu. Gần như miễn phí, không có siêu tham số ngoài kiên nhẫn, và trong nhiều recipe nó là thứ đang âm thầm gánh phần lớn công việc chính quy hoá.
- **Weight decay** — vẫn hữu ích, nhưng cơ chế thật của nó trong mạng có normalization không phải cơ chế trong sách. Với một tầng conv đứng ngay trước BatchNorm, nhân  $W$  với một hằng số *không đổi* hàm số mà mạng biểu diễn, vì BN chuẩn hoá lại ngay sau đó. Vậy weight decay không thu hẹp lớp hàm; cái nó thật sự điều khiển là *độ lớn của bước học hiệu dụng*, vì bước tương đối  $\|\Delta W\|/\|W\|$  tăng khi  $\|W\|$  bị co lại. Đây là lý do weight decay và learning rate không độc lập với nhau như tên gọi gợi ý.
- **EMA của trọng số** — giữ một bản trung bình trượt của trọng số và dùng nó để đánh giá. Rẻ, gần như luôn cải thiện một chút, và cơ chế thì đã nói ở mục trước: SGD ở cuối quá trình lang thang trong một quả cầu bán kính tỉ lệ với  $\eta$ , nên lấy trung bình là đứng vào tâm quả cầu đó thay vì một điểm ngẫu nhiên trên nó.
- **Stochastic depth** — bỏ ngẫu nhiên cả một khối residual trong lúc huấn luyện [19]. Hợp với mạng rất sâu; nó là “dropout ở cấp khối” và không phá thống kê không gian như dropout cấp phần tử.
- **Dropout** — *đã mất phần lớn vai trò* trong CNN thị giác hiện đại [20]. Có hai lý do. BatchNorm đã bơm sẵn nhiễu qua thống kê batch, nên nhiễu thêm là thừa. Và hai kỹ thuật xung khắc nhau: dropout làm phương sai kích hoạt lúc huấn luyện khác lúc suy luận, đúng đại lượng mà BN đang cố cố định. Nó vẫn sống khoẻ ở tầng đầu ra của transformer và ở các mô hình ngôn ngữ.

Điểm đáng nhớ của cả danh sách không phải thứ tự mà là *vì sao* thứ tự đó lệch với sách vở: sách xếp theo mức độ ràng buộc lớp hàm, thực tế xếp theo lượng thông tin mà công cụ mang thêm vào hoặc lượng nhiễu mà nó khớp với chế độ suy luận.

### C. Augmentation là nơi bạn phát biểu invariance

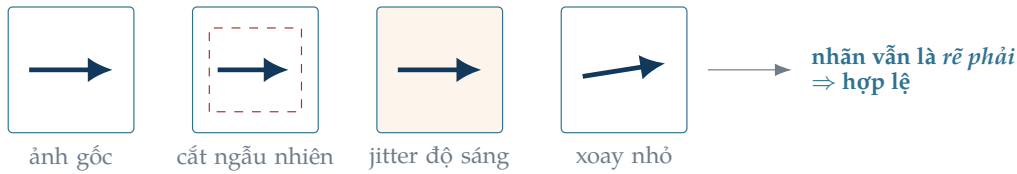
Câu hỏi “augmentation nào là đúng” không có câu trả lời chung, nhưng có một cách nhìn thống nhất làm nó trở thành câu hỏi trả lời được: *mỗi phép augmentation là một tuyên bố về tính bất biến mà bạn muốn mô hình có*. Lật ngang là tuyên bố “lớp không đổi khi lật trái phải”. Jitter màu là tuyên bố về bất biến với điều kiện chiếu sáng. Cắt ngẫu nhiên là tuyên bố “đối tượng có thể ở bất kỳ đâu và bất kỳ tỉ lệ nào”. Từ đó suy ra ngay quy tắc chọn: hãy đưa vào đúng những phép biến đổi thật sự xảy ra trong dữ liệu triển khai của bạn, và loại bỏ những phép phá huỷ nhãn.

Chính vì vậy augmentation có ý nghĩa vật lý mạnh hơn augmentation ngẫu nhiên. Nếu camera của bạn có nhiễu đọc, nhiễu bắn photon và một đường ống ISP cụ thể, thì mô phỏng đúng chuỗi đó (xem A/02-vat-ly-tao-anh) tạo ra các mẫu nằm trên đúng đa tạp mà mô hình sẽ gặp; còn cộng nhiễu Gauss vào ảnh đã qua ISP thì tạo ra các mẫu không tồn tại trong thực tế.

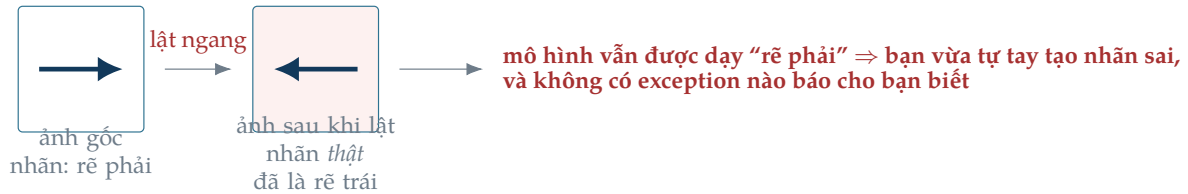
↪ **hệ quả của A/02** — chuỗi tạo ảnh vật lý quyết định phép biến đổi nào sinh ra mẫu có thật, phép nào sinh ra mẫu không bao giờ gặp.

Nhóm hiện đại thì mở rộng ý tưởng theo hai hướng khác nhau: **Mixup** và **CutMix** trộn hai mẫu và trộn cả nhãn theo cùng tỉ lệ [21], [22], tức tuyên bố một giả định về *tính tuyến tính giữa các lớp* chứ không còn về bất biến hình học; **RandAugment** thì tự động hoá việc chọn cường độ và số phép [23], giảm không gian tune xuống hai con số; còn copy-paste và mosaic ghép các đối tượng vào bối cảnh mới, đặc biệt hiệu quả cho detection vì chúng tăng số đối tượng mỗi ảnh.

**Giữ nhãn — mỗi phép này là một tuyên bố “lớp không đổi dưới phép biến đổi này”**



**Phá nhãn — cùng một phép, sai bài toán**



**Hình 1.13.** Augmentation là nơi bạn phát biểu tính bất biến, nên nó chỉ đúng khi tuyên bố đó đúng. Hàng trên: bốn phép biến đổi không đụng tới thứ quyết định nhãn, nên chúng dạy mô hình bỏ qua các chiều biến thiên vô nghĩa. Hàng dưới: cùng một phép lật ngang, nhưng ở bài toán mà *hướng* chính là lớp, nó tạo ra một cặp ảnh–nhãn mâu thuẫn. Đây cũng là lý do việc chọn augmentation phải bắt đầu từ câu hỏi “cái gì được phép đổi mà nhãn không đổi”, chứ không phải từ danh sách mặc định của thư viện.

Hình 1.13 cho thấy vì sao không tồn tại một bộ augmentation “tốt” chung: cùng một phép biến đổi vừa là công cụ mạnh nhất vừa là nguồn nhãn sai, tùy vào bài toán.

Chỗ hỏng thì rất cụ thể và đáng thuộc: augmentation phá nhãn. Lật ngang một ảnh chữ số hoặc biển báo rẽ trái là tạo ra một mẫu có nhãn sai. Jitter màu trong bài toán mà màu *chính là* lớp (đèn giao thông, phân loại quả chín) cũng vậy. Xoay lớn trong ảnh trên cao thì hợp lý, trong ảnh chụp bởi camera gắn xe thì không, vì trọng lực là một tiên nghiệm thật.

#### D. Từ một GPU tới nhiều GPU

Bốn bước dưới đây xếp theo đúng thứ tự tăng dần độ phức tạp và mức tiết kiệm bộ nhớ. Nguyên tắc là *chỉ leo lên bước sau khi bước trước đã hết dư địa*. Trước khi đọc bảng, hãy cố định bảng kế toán. Với  $P$  tham số huấn luyện ở fp32 bằng Adam, mỗi GPU phải giữ  $4P$  byte trọng số,  $4P$  byte gradient và  $8P$  byte trạng thái Adam. Tổng là  $16P$  byte, tức 4,8 GB khi  $P = 300$  triệu — trước khi tính một byte kích hoạt nào.

**Bảng 1.5.** Bốn mức song song hoá dữ liệu. Cột bộ nhớ tính cho  $N$  tiến trình và bỏ qua kích hoạt; cột cuối là cột đáng đọc kỹ nhất.

| Bước                  | Cơ chế                                                                                   | Bộ nhớ mỗi GPU                                | Hỏng khi nào                                                                                                                     |
|-----------------------|------------------------------------------------------------------------------------------|-----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| DataParallel          | Một tiến trình, phát tán mô hình sang các GPU mỗi bước                                   | $16P$                                         | Nghẽn ở GIL và ở việc sao chép mô hình; GPU 0 quá tải. Coi như đã lỗi thời                                                       |
| DDP                   | Mỗi GPU một tiến trình, all-reduce gradient sau backward                                 | $16P$                                         | Seed và sampler không đặt đúng thì các tiến trình thấy cùng dữ liệu; BatchNorm không đồng bộ nên mỗi GPU là một mô hình hơi khác |
| Gradient accumulation | Chia batch thành micro-batch, cộng dồn gradient rồi mới bước                             | $16P$ , nhưng kích hoạt giảm theo micro-batch | Không giảm bộ nhớ trạng thái; thống kê BatchNorm vẫn tính theo micro-batch nên batch hiệu dụng lớn mà thống kê thì không         |
| ZeRO / FSDP [24]      | Chia nhỏ trạng thái (giai đoạn 1), gradient (2), rồi cả trọng số (3) giữa các tiến trình | $8P + 8P/N$ tới $16P/N$                       | Lưu lượng giao tiếp tăng theo giai đoạn; nghẽn khi mạng liên kết chậm hoặc mô hình nhiều tầng nhỏ                                |

Ba cạm bẫy trong Bảng 1.5 đáng nêu tên riêng vì chúng không gây lỗi mà chỉ gây kết quả kém. Thứ nhất là *seed và sampler*: nếu mọi tiến trình dùng cùng seed cho việc xáo dữ liệu, bạn không huấn luyện trên  $N$  lần dữ liệu mà trên cùng một dòng dữ liệu  $N$  lần. Thứ hai là *BatchNorm*. Mỗi tiến trình tính thống kê trên batch cục bộ của nó. Với detection chạy một tới hai ảnh mỗi GPU, thống kê nhiều tới mức vô dụng. Đây chính là lý do GroupNorm và SyncBN tồn tại, và cũng là vấn đề đã gặp ở mục 2 của chương này. Thứ ba là *cách gộp loss*: lấy tổng thay vì trung bình làm learning rate hiệu dụng bị nhân với số tiến trình, một lỗi im lặng có hậu quả đúng bằng việc đặt sai learning rate một bậc.

↪ **hệ quả của C/12/03** — bảng kế toán  $16P$  byte cho trọng số, gradient và trạng thái Adam là lý do tồn tại của ZeRO

Song song với bốn bước trên còn một trục thứ hai đã nhắc ở mục 1: **gradient checkpointing** [2] đổi khoảng 30 phần trăm thời gian tính lấy việc giảm bộ nhớ kích hoạt từ  $O(L)$  xuống  $O(\sqrt{L})$ . Trong thị giác, kích hoạt thường lớn hơn trọng số rất nhiều, nên với một GPU đơn thì checkpointing hầu như luôn là bước nên thử trước ZeRO.

### E. Chính quy hoá ngầm: chỗ ngành chưa có câu trả lời

Cần biết mục này để *không suy luận sai*, chứ không phải để dùng. Câu hỏi mở là: vì sao một mạng quá tham số, đủ sức học thuộc nhiều, lại chọn đúng nghiệm tổng quát hoá được khi ta huấn luyện nó bằng SGD? Các mảnh câu trả lời hiện có đều là mảnh: nhiều của SGD thiên vị các nghiệm ở vùng phẳng của mặt mất mát; các thuật toán bậc nhất có xu hướng tìm nghiệm chuẩn nhỏ trong các bài toán tuyến tính quá tham số; khởi tạo và lịch học đều để lại dấu vết trên nghiệm cuối. Chưa có lý thuyết nào ghép các mảnh đó lại thành một lời giải thích đầy đủ cho mạng sâu thật.

Hệ quả cho cách bạn đọc và nói: đừng kết luận “mô hình này quá lớn nên sẽ overfit” như một mệnh đề chắc chắn, vì nó thường sai trên thực nghiệm; và cũng đừng kết luận ngược lại rằng sức chứa không quan trọng. Phát biểu trung thực là: quá trình huấn luyện, chứ không chỉ lớp hàm, quyết định nghiệm — và ta chưa mô tả được “chứ không chỉ” đó một cách định lượng.

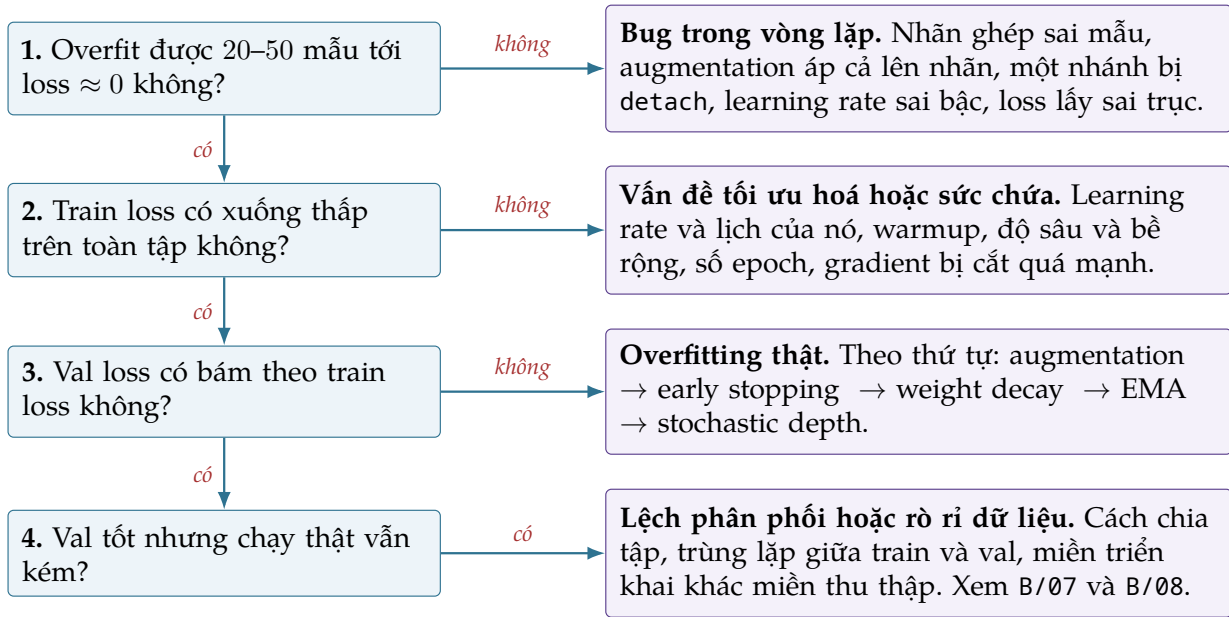
### F. Phản xạ debug số một: overfit một batch nhỏ

Trước khi chạy full, lấy 20 tới 50 mẫu, tắt mọi augmentation và mọi chính quy hoá, rồi huấn luyện tới khi loss huấn luyện gần 0. Nếu không đạt được điều đó, dừng lại: bug nằm trong vòng lặp, không nằm ở khả năng tổng quát hoá, và mọi phút bỏ ra để tune siêu tham số lúc này là phí.



Sức mạnh của phép thử nằm ở chỗ nó *loại bỏ đúng các biến gây nhiễu*. Trên 20 mẫu, một mạng có hàng triệu tham số thừa sức nhớ hết, nên nếu nó không nhớ được thì tín hiệu không đi từ dữ liệu tới loss hoặc gradient không đi ngược lại được. Nó kiểm tra toàn tuyến — đọc dữ liệu, tiền xử lý, ghép nhãn, hàm mất mát, backward, bước cập nhật — trong vài phút, thay vì để bạn suy đoán từ một đường cong sau ba giờ. Đây cũng là lý do nó phải là việc đầu tiên, không phải việc cuối cùng.

Hình 1.14 biến phép thử đó thành một cây quyết định đầy đủ. Điểm đáng chú ý là mỗi nhánh dẫn tới một *họ* nguyên nhân khác hẳn nhau, nên trả lời sai câu hỏi đầu tiên khiến bạn đi tìm ở nhầm họ suốt phần còn lại.



**Hình 1.14.** Cây chẩn đoán một lần chạy không đạt kỳ vọng. Bốn câu hỏi phải hỏi theo đúng thứ tự này, vì mỗi câu chỉ có nghĩa khi câu trước đã trả lời “có”; bỏ qua câu số một là nguyên nhân phổ biến nhất của việc tune siêu tham số cho một hệ thống đang có bug.

### G. Phá hỏng có chủ đích: xây thư viện triệu chứng

Kỹ năng cao nhất của cả chương không phải biết cách sửa mà là biết *nhìn một đường cong và đoán ra nguyên nhân*. Kỹ năng đó không đến từ đọc; nó đến từ việc cố ý làm hỏng từng thứ một và ghi lại hình dạng đường cong tương ứng. Bảng 1.6 là tám phiên bản hỏng đáng chạy, và cột cuối là cột biến bảng này từ một danh sách thành một công cụ chẩn đoán.

**Bảng 1.6.** Tám lỗi cố ý và triệu chứng của chúng. Chạy cả tám, vẽ tám đường cong vào một hình duy nhất, và giữ hình đó.

| Phá hỏng                   | Triệu chứng trên đường cong                                                                 | Phân biệt bằng phép đo nào                                                                 |
|----------------------------|---------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Learning rate $\times 100$ | Loss vọt lên rồi thành NaN trong vài chục bước, hoặc phẳng ngay ở mức phỏng đoán ngẫu nhiên | Tỉ lệ update trên trọng số ở bước đầu vượt $10^{-1}$ ; chuẩn gradient tăng theo bước       |
| Learning rate $\div 100$   | Loss giảm gần như tuyến tính, không có dấu hiệu bão hoà khi hết lịch                        | Tỉ lệ update trên trọng số dưới $10^{-5}$                                                  |
| Quên shuffle               | Loss răng cưa với chu kỳ đúng bằng một epoch                                                | Vẽ loss theo từng bước thay vì theo epoch; hình răng cưa lộ ra ngay                        |
| Quên eval()                | Val loss thấp bất thường và đôi khi đổi batch size lúc đánh giá                             | Chạy val hai lần với batch 1 và batch 64, so kết quả                                       |
| Nhân lệch một chỉ số       | Train loss giảm bình thường; val accuracy quanh mức ngẫu nhiên                              | Ma trận nhầm lẫn lệch hẳn khỏi đường chéo một cột                                          |
| Quên chuẩn hoá đầu vào     | Hội tụ chậm hơn rõ rệt; nếu có BatchNorm thì gần như không thấy gì                          | Histogram của batch đầu vào; thống kê kích hoạt tầng đầu                                   |
| Khởi tạo toàn 0            | Loss đứng đúng mức ngẫu nhiên, không nhích                                                  | Các hàng của $W$ giống hệt nhau sau vài bước: đối xứng chưa bị phá                         |
| Dropout 0,9                | Train loss cao hơn val loss; cả hai hội tụ rất chậm                                         | Khoảng cách train trừ val mang dấu âm — điều gần như không bao giờ xảy ra khi mọi thứ đúng |

## H. Giới hạn và trường hợp hỏng

Phép thử “overfit một batch nhỏ” có một chế độ hỏng đáng biết, và nó là loại chế độ hỏng khó chịu nhất: phép thử *qua* trong khi hệ thống vẫn sai. Nếu nhân của bạn bị hoán vị theo một quy tắc cố định — ví dụ chỉ số lệch một — thì trên 20 mẫu, mạng học thuộc ánh xạ sai đó dễ dàng và loss vẫn về 0. Cùng lý do, một bug trong tiền xử lý áp giống nhau cho mọi mẫu cũng không bị bắt. Nói cách khác, phép thử chứng minh “tín hiệu chảy được từ đầu vào tới loss và ngược lại”, không chứng minh “tín hiệu chảy đúng thứ”. Bổ sung rẻ nhất là nhìn trực tiếp vài chục mẫu đã qua toàn bộ pipeline, kèm nhãn, ngay trước khi vào mô hình.

↪ **tiền đề** E/24 — ablation chạy trên một hệ thống còn bug là vô nghĩa, nên phép thử này phải đứng trước mọi thí nghiệm so sánh

Bảng triệu chứng cũng có giới hạn: nó được lập trên một cấu hình cụ thể, và một số triệu chứng bị normalization che mất — “quên chuẩn hoá đầu vào” gần như vô hình khi mạng có BatchNorm ở tầng đầu. Đây là mặt trái đã nói ở mục 2: normalization làm hệ thống bền hơn với lỗi và vì thế xoá bớt các triệu chứng mà bạn muốn học đọc. Cuối cùng, thứ tự ưu tiên các công cụ chính quy hoá ở mục B chỉ là *kinh nghiệm thực hành trong thị giác*. Nó được tổng hợp từ các recipe phổ biến, không phải từ một nghiên cứu so sánh có kiểm soát. Ở lĩnh vực khác, chẳng hạn mô hình ngôn ngữ, thứ tự đó khác.

### CẠM BẮY

Rõ rĩ dữ liệu là cách phổ biến nhất để có một đường cong xác thực đẹp và một hệ thống vô dụng, và nó gần như không bao giờ trông giống một lỗi. Ba dạng hay gặp trong thị giác: chia tập theo khung hình thay vì theo chuỗi, khiến hai khung hình cách nhau 1/30 giây nằm ở hai bên ranh giới; augmentation hoặc chuẩn hoá được tính trên *toàn bộ* dữ liệu trước khi chia; và các ảnh trùng lặp hoặc gần trùng nằm ở cả hai phía. Dấu hiệu nhận biết: khoảng cách train–val nhỏ bất thường, và điểm số tụt mạnh ngay khi bạn đánh giá trên dữ liệu thu ở một buổi khác.

### CẦN NHỚ

Trước khi tune bất cứ thứ gì, overfit 20–50 mẫu tới loss gần 0. Đây là ranh giới tách “vòng lặp có bug” khỏi “mô hình không tổng quát hoá được”. Hai họ nguyên nhân đó cần hai bộ công cụ hoàn toàn khác nhau. Nhắm chúng với nhau là cách tốn thời gian nhất trong nghề. [T1]

## I. Kết nối

Mục này khép lại chương 12 và mở sang hai hướng. Về phía dữ liệu, “tổng quát hoá” chỉ có nghĩa khi tập được chia đúng, nên B/07-chia-du-lieu-va-danh-gia là tiền đề bắt buộc của cả mục, và B/08-dich-chuyen-phan-phoi là chương giải thích nhánh cuối cùng của cây chẩn đoán. Về phía kiến trúc, thư viện triệu chứng ở mục G là công cụ bạn sẽ dùng liên tục khi đọc C/13-kien-truc-backbone. Bài học lớn nhất của chương đó là ConvNeXt: phần lớn chênh lệch giữa các kiến trúc hoá ra là chênh lệch công thức huấn luyện. Bài học ấy chỉ có nghĩa nếu bạn phân biệt được hai nguồn. Cuối cùng, sự nghi ngờ có kỷ luật với một bảng so sánh là nội dung của E/24-thi-nghiem-va-ablation.

### I Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao lập luận “mô hình nhiều tham số hơn số mẫu thì sẽ học thuộc lòng” không còn dùng được, và điều đó thay đổi cách bạn chọn công cụ chính quy hoá thể nào?
2. Trong một mạng có BatchNorm, weight decay thật sự điều khiển đại lượng nào? Vì sao nó không thu hẹp lớp hàm?
3. Augmentation nào bạn phải bỏ nếu bài toán là phân loại đèn giao thông, và vì sao?
4. Gradient accumulation cho batch hiệu dụng lớn hơn nhưng không sửa được một vấn đề của BatchNorm. Vấn đề đó là gì?
5. Phép thử “overfit một batch nhỏ” qua được nhưng hệ thống vẫn sai — tình huống cụ thể nào gây ra chuyện đó, và phép kiểm tra bổ sung nào bắt được nó?
6. Bạn thấy train loss cao hơn val loss — hai nguyên nhân khả dĩ là gì?
7. ZeRO giai đoạn 3 giảm bộ nhớ xuống  $16P/N$ ; nó đổi lấy cái gì, và khi nào phép đổi đó không đáng?

- [1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [2] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [3] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *AISTATS*, 2010.
- [4] K. He, X. Zhang, S. Ren, et al., “Delving Deep into Rectifiers,” in *ICCV*, 2015.
- [5] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [6] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [7] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [8] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [9] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [10] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [11] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” in *ICLR*, 2017.
- [12] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [13] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [14] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [15] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [16] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [17] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [18] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [19] G. Huang, Y. Sun, Z. Liu, et al., “Deep Networks with Stochastic Depth,” in *ECCV*, 2016.
- [20] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [21] H. Zhang, M. Cisse, Y. N. Dauphin, et al., “mixup: Beyond Empirical Risk Minimization,” in *ICLR*, 2018.
- [22] S. Yun, D. Han, S. J. Oh, et al., “CutMix: Regularization Strategy to Train Strong Classifiers,” in *ICCV*, 2019.
- [23] E. D. Cubuk, B. Zoph, J. Shlens, et al., “RandAugment: Practical Automated Data Augmentation,” in *CVPR Workshops*, 2020.

- [24] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [25] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [26] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *NeurIPS*, 2012.
- [27] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in *ICLR*, 2015.
- [28] C. Szegedy, W. Liu, et al., “Going Deeper with Convolutions,” in *CVPR*, 2015.
- [29] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [30] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [31] G. Huang, Z. Liu, L. van der Maaten, et al., “Densely Connected Convolutional Networks,” in *CVPR*, 2017.
- [32] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [33] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [34] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [35] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [36] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [37] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.
- [38] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [39] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [40] J. Su, M. Ahmed, Y. Lu, et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, vol. 568, 2024.
- [41] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” *arXiv:2312.00752*, 2023.
- [42] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [43] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.
- [44] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [45] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [46] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [47] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [48] R. Girshick, J. Donahue, T. Darrell, et al., “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation,” in *CVPR*, 2014.

- [49] R. Girshick, "Fast R-CNN," in *ICCV*, 2015.
- [50] S. Ren, K. He, R. Girshick, et al., "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in *NeurIPS*, 2015.
- [51] T.-Y. Lin, P. Dollár, R. Girshick, et al., "Feature Pyramid Networks for Object Detection," in *CVPR*, 2017.
- [52] K. He, G. Gkioxari, P. Dollár, et al., "Mask R-CNN," in *ICCV*, 2017.
- [53] D. Bolya, S. Foley, J. Hays, et al., "TIDE: A General Toolbox for Identifying Object Detection Errors," in *ECCV*, 2020.
- [54] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [55] J. Redmon, S. Divvala, R. Girshick, et al., "You Only Look Once: Unified, Real-Time Object Detection," in *CVPR*, 2016.
- [56] W. Liu, D. Anguelov, D. Erhan, et al., "SSD: Single Shot MultiBox Detector," in *ECCV*, 2016.
- [57] T.-Y. Lin, P. Goyal, R. Girshick, et al., "Focal Loss for Dense Object Detection," in *ICCV*, 2017.
- [58] Z. Tian, C. Shen, H. Chen, et al., "FCOS: Fully Convolutional One-Stage Object Detection," in *ICCV*, 2019.
- [59] X. Zhou, D. Wang, and P. Krähenbühl, "Objects as Points," *arXiv:1904.07850*, 2019.
- [60] S. Zhang, C. Chi, Y. Yao, et al., "Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection," in *CVPR*, 2020.
- [61] Z. Ge, S. Liu, Z. Li, et al., "OTA: Optimal Transport Assignment for Object Detection," in *CVPR*, 2021.
- [62] N. Bodla, B. Singh, R. Chellappa, et al., "Soft-NMS: Improving Object Detection with One Line of Code," in *ICCV*, 2017.
- [63] X. Zhu, W. Su, L. Lu, et al., "Deformable DETR: Deformable Transformers for End-to-End Object Detection," in *ICLR*, 2021.
- [64] H. Zhang, F. Li, S. Liu, et al., "DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection," in *ICLR*, 2023.
- [65] J. Long, E. Shelhamer, and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," in *CVPR*, 2015.
- [66] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *MICCAI*, 2015.
- [67] L.-C. Chen, G. Papandreou, I. Kokkinos, et al., "DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs," *IEEE TPAMI*, vol. 40, no. 4, pp. 834–848, 2018.
- [68] E. Xie, W. Wang, Z. Yu, et al., "SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers," in *NeurIPS*, 2021.
- [69] B. Cheng, I. Misra, A. G. Schwing, et al., "Masked-Attention Mask Transformer for Universal Image Segmentation," in *CVPR*, 2022.
- [70] A. Kirillov, K. He, R. Girshick, et al., "Panoptic Segmentation," in *CVPR*, 2019.
- [71] A. Kirillov, E. Mintun, N. Ravi, et al., "Segment Anything," in *ICCV*, 2023.
- [72] K. Sun, B. Xiao, D. Liu, et al., "Deep High-Resolution Representation Learning for Human Pose Estimation," in *CVPR*, 2019.



- [73] Z. Cao, T. Simon, S.-E. Wei, et al., “Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields,” in *CVPR*, 2017.
- [74] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [75] J. Deng, J. Guo, N. Xue, et al., “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *CVPR*, 2019.
- [76] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [77] K. Xu, J. Ba, R. Kiros, et al., “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention,” in *ICML*, 2015.
- [78] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *ICLR*, 2015.
- [79] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [80] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [81] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [82] K. Simonyan and A. Zisserman, “Two-Stream Convolutional Networks for Action Recognition in Videos,” in *NeurIPS*, 2014.
- [83] J. Carreira and A. Zisserman, “Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset,” in *CVPR*, 2017.
- [84] D. Tran, H. Wang, L. Torresani, et al., “A Closer Look at Spatiotemporal Convolutions for Action Recognition,” in *CVPR*, 2018.
- [85] C. Feichtenhofer, H. Fan, J. Malik, et al., “SlowFast Networks for Video Recognition,” in *ICCV*, 2019.
- [86] A. Dosovitskiy, P. Fischer, E. Ilg, et al., “FlowNet: Learning Optical Flow with Convolutional Networks,” in *ICCV*, 2015.
- [87] D. Sun, X. Yang, M.-Y. Liu, et al., “PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume,” in *CVPR*, 2018.
- [88] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.
- [89] A. Bewley, Z. Ge, L. Ott, et al., “Simple Online and Realtime Tracking,” in *ICIP*, 2016.
- [90] Y. Zhang, P. Sun, Y. Jiang, et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *ECCV*, 2022.
- [91] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [92] K. Bernardin and R. Stiefelwagen, “Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics,” *EURASIP J. Image and Video Processing*, 2008.
- [93] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [94] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [95] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.

- [96] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *CVPR*, 2024.
- [97] D. Eigen, C. Puhrsch, and R. Fergus, “Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network,” in *NeurIPS*, 2014.
- [98] R. Ranftl, K. Lasinger, D. Hafner, et al., “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer,” *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [99] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IROS*, 2012.
- [100] C. R. Qi, H. Su, K. Mo, et al., “PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation,” in *CVPR*, 2017.
- [101] C. R. Qi, L. Yi, H. Su, et al., “PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space,” in *NeurIPS*, 2017.
- [102] A. H. Lang, S. Vora, H. Caesar, et al., “PointPillars: Fast Encoders for Object Detection from Point Clouds,” in *CVPR*, 2019.
- [103] T. Müller, A. Evans, C. Schied, et al., “Instant Neural Graphics Primitives with a Multiresolution Hash Encoding,” *ACM TOG*, vol. 41, no. 4, 2022.
- [104] B. Kerbl, G. Kopanas, T. Leimkühler, et al., “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *ACM TOG*, vol. 42, no. 4, 2023.
- [105] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.
- [106] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” in *ICLR*, 2014.
- [107] I. Goodfellow, J. Pouget-Abadie, M. Mirza, et al., “Generative Adversarial Nets,” in *NeurIPS*, 2014.
- [108] T. Karras, S. Laine, and T. Aila, “A Style-Based Generator Architecture for Generative Adversarial Networks,” in *CVPR*, 2019.
- [109] J. Ho, A. Jain, and P. Abbeel, “Denoising Diffusion Probabilistic Models,” in *NeurIPS*, 2020.
- [110] Y. Song, J. Sohl-Dickstein, D. P. Kingma, et al., “Score-Based Generative Modeling Through Stochastic Differential Equations,” in *ICLR*, 2021.
- [111] J. Johnson, A. Alahi, and L. Fei-Fei, “Perceptual Losses for Real-Time Style Transfer and Super-Resolution,” in *ECCV*, 2016.
- [112] P. Isola, J.-Y. Zhu, T. Zhou, et al., “Image-to-Image Translation with Conditional Adversarial Networks,” in *CVPR*, 2017.
- [113] L. A. Gatys, A. S. Ecker, and M. Bethge, “Image Style Transfer Using Convolutional Neural Networks,” in *CVPR*, 2016.
- [114] X. Huang and S. Belongie, “Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization,” in *ICCV*, 2017.
- [115] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.
- [116] J. Song, C. Meng, and S. Ermon, “Denoising Diffusion Implicit Models,” in *ICLR*, 2021.
- [117] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, et al., “Flow Matching for Generative Modeling,” in *ICLR*, 2023.
- [118] J. Ho and T. Salimans, “Classifier-Free Diffusion Guidance,” *arXiv:2207.12598*, 2022.
- [119] R. Rombach, A. Blattmann, D. Lorenz, et al., “High-Resolution Image Synthesis with Latent Diffusion Models,” in *CVPR*, 2022.
- [120] W. Peebles and S. Xie, “Scalable Diffusion Models with Transformers,” in *ICCV*, 2023.

- [121] L. Zhang, A. Rao, and M. Agrawala, “Adding Conditional Control to Text-to-Image Diffusion Models,” in *ICCV*, 2023.
- [122] Y. Blau and T. Michaeli, “The Perception-Distortion Tradeoff,” in *CVPR*, 2018.
- [123] C. Ledig, L. Theis, F. Huszár, et al., “Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network,” in *CVPR*, 2017.
- [124] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., “Image Quality Assessment: From Error Visibility to Structural Similarity,” *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [125] M. Heusel, H. Ramsauer, T. Unterthiner, et al., “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium,” in *NeurIPS*, 2017.
- [126] T. Brooks, B. Mildenhall, T. Xue, et al., “Unprocessing Images for Learned Raw Denoising,” in *CVPR*, 2019.
- [127] A. Abdelhamed, S. Lin, and M. S. Brown, “A High-Quality Denoising Dataset for Smartphone Cameras,” in *CVPR*, 2018.
- [128] J. Deng, W. Dong, R. Socher, et al., “ImageNet: A Large-Scale Hierarchical Image Database,” in *CVPR*, 2009.
- [129] A. Radford, J. W. Kim, C. Hallacy, et al., “Learning Transferable Visual Models from Natural Language Supervision,” in *ICML*, 2021.
- [130] J. Li, D. Li, S. Savarese, et al., “BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models,” in *ICML*, 2023.
- [131] H. Liu, C. Li, Q. Wu, et al., “Visual Instruction Tuning,” in *NeurIPS*, 2023.
- [132] S. Kornblith, J. Shlens, and Q. V. Le, “Do Better ImageNet Models Transfer Better?” In *CVPR*, 2019.
- [133] T. Chen, S. Kornblith, M. Norouzi, et al., “A Simple Framework for Contrastive Learning of Visual Representations,” in *ICML*, 2020.
- [134] K. He, H. Fan, Y. Wu, et al., “Momentum Contrast for Unsupervised Visual Representation Learning,” in *CVPR*, 2020.
- [135] J.-B. Grill, F. Strub, F. Altché, et al., “Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning,” in *NeurIPS*, 2020.
- [136] M. Caron, H. Touvron, I. Misra, et al., “Emerging Properties in Self-Supervised Vision Transformers,” in *ICCV*, 2021.
- [137] K. He, X. Chen, S. Xie, et al., “Masked Autoencoders Are Scalable Vision Learners,” in *CVPR*, 2022.
- [138] A. van den Oord, Y. Li, and O. Vinyals, “Representation Learning with Contrastive Predictive Coding,” in *arXiv:1807.03748*, 2018.
- [139] J. Kaplan, S. McCandlish, T. Henighan, et al., “Scaling Laws for Neural Language Models,” *arXiv:2001.08361*, 2020.
- [140] X. Zhai, A. Kolesnikov, N. Houlsby, et al., “Scaling Vision Transformers,” in *CVPR*, 2022.
- [141] J. Hoffmann, S. Borgeaud, A. Mensch, et al., “Training Compute-Optimal Large Language Models,” in *NeurIPS*, 2022.
- [142] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [143] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [144] N. Houlsby, A. Giurgiu, S. Jastrzebski, et al., “Parameter-Efficient Transfer Learning for NLP,” in *ICML*, 2019.

- [145] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [146] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *RecSys*, 2019.

## 1.6 Thực hành

Chương này khác các chương khác ở một điểm: đọc hiểu không tạo ra kỹ năng. Hai mục dưới đây là phần đọc và phần tự vấn; phần tay chân nằm ở khối mini project cuối chương, và đó mới là chỗ kiến thức chuyển thành phản xạ.

**Repo và tài nguyên.** Để hiểu autodiff từ gốc, [karpathy/micrograd](#) là bản cài đặt reverse-mode gọn trong khoảng một trăm dòng và đọc hết trong một buổi; [tinygrad/tinygrad](#) là bước tiếp theo, đủ nhỏ để đọc nhưng đã có đầy đủ khái niệm của một framework thật. Để tự derive gradient theo lối bài tập, bộ assignment của CS231n vẫn là tài liệu tốt nhất, và [labmlai/annotated\\_deep\\_learning\\_paper\\_implementations](#) cho bạn bản cài đặt có chú giải của phần lớn kỹ thuật nêu trong chương. Về augmentation, [alumentations-team/alumentations](#) và `transforms.v2` là hai lựa chọn mặc định. Về huấn luyện phân tán, đọc trực tiếp tài liệu DDP và FSDP của PyTorch trước, rồi mới tới [microsoft/DeepSpeed](#) nếu cần các giai đoạn ZeRO sâu hơn. Danh sách này chốt tại tháng 9/2026, và tên các mô hình dẫn đầu sẽ cũ trong sáu tới mười hai tháng. Nhưng phần lớn nội dung chương này là hạ tầng chứ không phải mô hình, nên nó bền hơn hẳn mức trung bình của cây.

**Câu hỏi kiểm tra hiểu.** Vì sao AdamW khác Adam cộng L2, và bạn kỳ vọng khác biệt đó lớn cỡ nào trong thực tế? Một đường cong học giảm rất nhanh rồi phẳng ở mức cao — nêu ba nguyên nhân khả dĩ và ba phép đo phân biệt chúng. Vì sao BatchNorm làm lượt ngược tốn thêm bộ nhớ, và điều đó gợi ý gì khi bạn phải chọn giữa BatchNorm và GroupNorm ở một bài detection chạy hai ảnh mỗi GPU?

### Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

| Thuật ngữ / câu hỏi      | Trả lời                                                                                                                                                                                                                                                               |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ••• Lan truyền ngược     | Thuật toán tính gradient bằng cách đi ngược đồ thị phép tính đúng một lượt. Nó rẻ hơn vì phân thuận khoảng tám bậc độ lớn, vì một lượt trả về đạo hàm theo <i>mọi</i> tham số. Nó không bao giờ dựng ma trận Jacobian, chỉ áp tích vector-Jacobian ở từng nút.        |
| •• Lượt xuôi, lượt ngược | Lượt xuôi tính giá trị và ghi lại kết quả trung gian; lượt ngược dùng chúng để tính gradient. Cả hai là <i>một lần chạy</i> của lan truyền ngược, đừng nhầm với chính thuật toán.                                                                                     |
| ••• Ba mẫu gradient      | Cộng phân phối adjoint đều cho mọi nhánh. Nhân hoán đổi: mỗi nhánh nhận adjoint nhân giá trị nhánh kia. Max định tuyến toàn bộ về nhánh thắng. Mẫu thứ tư hay bị quên: rẽ nhánh thì cộng dồn, nên lượt ngược của broadcast là một phép cộng theo trục.                |
| ••• Bộ nhớ kích hoạt     | Toàn bộ đầu ra trung gian của lượt xuôi, phải giữ lại vì lượt ngược cần tới. Nó đạt đỉnh ở chỗ hai lượt giao nhau, nên nó mới là thứ chặn batch chứ không phải số trọng số. Gradient checkpointing hạ đỉnh xuống $O(\sqrt{L})$ với giá khoảng 30 phần trăm thời gian. |

| Thuật ngữ / câu hỏi                                                                    | Trả lời (tiếp)                                                                                                                                                                                                                                                      |
|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ••• Vì sao gradient tắt hoặc nổ khi mạng sâu?                                          | Vì adjoint bị nhân dồn qua từng tầng. Nếu hệ số mỗi tầng là $\rho$ thì gradient ở tầng đầu tỉ lệ với $\rho^L$ ; $\rho = 0,9$ qua 50 tầng còn $5 \cdot 10^{-3}$ .                                                                                                    |
| ••• Khởi tạo He                                                                        | Đặt $\text{Var}(w) = 2/n_{\text{in}}$ để phương sai tín hiệu không đổi qua mỗi tầng. Hệ số 2 đến từ việc ReLU xoá một nửa số đầu ra, nên nó đúng cho ReLU chứ không đúng cho tanh.                                                                                  |
| ••• BatchNorm                                                                          | Chuẩn hoá kích hoạt theo thống kê của cả batch, rồi trả quyền chỉnh lại cho mạng qua $\gamma$ và $\beta$ . Vì nó ghép các mẫu trong batch lại với nhau, nó hỏng khi batch nhỏ, khi train và eval dùng hai nguồn thống kê, và khi các GPU không đồng bộ.             |
| •• Val loss thấp bất thường, và đổi khi đổi batch size lúc đánh giá — nguyên nhân?     | Quên gọi <code>model.eval()</code> , nên BatchNorm dùng thống kê của chính batch kiểm thử. Phân biệt với rò rỉ dữ liệu: rò rỉ cho điểm cao ổn định, còn lỗi này đổi điểm khi bạn đổi batch size lúc đánh giá.                                                       |
| ••• Vì sao momentum thắng trong rãnh hẹp?                                              | Thành phần vuông góc với rãnh đổi dấu mỗi bước nên triệt tiêu trong tổng trượt. Thành phần dọc rãnh giữ nguyên dấu nên cộng dồn tới hệ số $1/(1 - \mu)$ , gấp 10 lần khi $\mu = 0,9$ .                                                                              |
| ••• AdamW                                                                              | Adam với weight decay tách khỏi mẫu số thích nghi, cộng thẳng vào bước cập nhật. Nhờ đó suy giảm là một phép co đều, không phụ thuộc lịch sử gradient. <code>Adam(weight_decay=...)</code> không phải AdamW.                                                        |
| ••• Vì sao learning rate là siêu tham số quan trọng nhất?                              | Vì nó là đại lượng duy nhất nhân trực tiếp vào mọi bước cập nhật. Sai một bậc ở nó đổi hẳn bản chất quá trình; sai một bậc ở weight decay chỉ mất vài phần trăm điểm số. Warmup tồn tại cũng vì lý do này: $\hat{v}$ của Adam ở vài trăm bước đầu quá nhiều để tin. |
| •• Vì sao fp16 làm huấn luyện tê liệt?                                                 | Gradient thị giác thường nằm quanh $10^{-7}$ tới $10^{-4}$ , dưới ngưỡng chuẩn hoá $6,1 \cdot 10^{-5}$ của fp16, nên bị làm tròn về 0. Loss scaling nhân loss với $2^{16}$ để dịch cả phổ vào vùng biểu diễn được, rồi chia lại trước bước cập nhật.                |
| ••• Loss thành NaN trong vài chục bước đầu — nguyên nhân?                              | Learning rate quá lớn, hoặc tràn số ở fp16. Phân biệt: hạ learning rate mười lần mà vẫn NaN thì là tràn số, hãy thử bf16 hoặc bật loss scaling.                                                                                                                     |
| ••• Loss giảm gần tuyến tính và không bão hoà khi hết lịch — nguyên nhân?              | Learning rate quá nhỏ. Phân biệt với mô hình thiếu sức chứa: đo tỉ lệ cập nhật trên trọng số, dưới $10^{-5}$ là learning rate, còn quanh $10^{-3}$ mà loss vẫn cao thì là sức chứa hoặc dữ liệu.                                                                    |
| •• Loss răng cưa với chu kỳ đúng bằng một epoch — nguyên nhân?                         | Quên shuffle, nên mô hình gặp lại đúng thứ tự dữ liệu mỗi vòng. Phân biệt với learning rate quá cao: vẽ loss theo từng bước, răng cưa có chu kỳ đều là shuffle, còn nhiều không chu kỳ là learning rate.                                                            |
| ••• Train loss giảm bình thường nhưng val accuracy quanh mức ngẫu nhiên — nguyên nhân? | Nhân bị lệch chỉ số hoặc ghép sai mẫu: mạng học thuộc một ánh xạ sai nhưng nhất quán. Phân biệt bằng ma trận nhầm lẫn — nó lệch hẳn khỏi đường chéo đúng một cột.                                                                                                   |
| •• Augmentation                                                                        | Sinh thêm mẫu bằng cách biến đổi mẫu có sẵn. Mỗi phép là một tuyên bố “nhân không đổi dưới phép này”, nên nó thêm thông tin chứ không ràng buộc sức chứa. Nó hỏng khi phép biến đổi phá nhân, ví dụ lật ngang một biển báo rẽ.                                      |
| •• ZeRO và FSDP                                                                        | Chia trọng số, gradient và trạng thái optimizer giữa các tiến trình, theo ba giai đoạn tăng dần. Nó là hệ quả trực tiếp của bảng kế toán $16P$ byte, và trả giá bằng lưu lượng giao tiếp.                                                                           |
| ••• Overfit một batch nhỏ                                                              | Huấn luyện 20–50 mẫu tới loss gần 0 sau khi tắt mọi augmentation. Không đạt thì bug nằm trong vòng lặp chứ không ở khả năng tổng quát hoá. Nó vẫn qua nếu nhân bị hoán vị theo một quy tắc cố định.                                                                 |
| • Double descent                                                                       | Vượt qua ngưỡng nội suy, sai số kiểm thử đạt đỉnh rồi giảm trở lại khi mô hình lớn thêm. Hệ quả: trực giác chữ U của giáo trình cũ không dùng được cho mạng quá tham số.                                                                                            |



● ● ● phải thuộc ● ● nên nắm ● biết là có

**Kiến thức này đang sống ở đâu** (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

| Công trình / hệ thống                                                                                | Dùng ý tưởng nào của chương                                                                                                           | Ghi chú                                                                                |
|------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| PyTorch autograd, JAX                                                                                | Toàn bộ cơ chế tính gradient là hiện thực của vi phân ngược trên đồ thị: mỗi phép toán khai báo một tích vector-Jacobian              | Hạ tầng bền; API gần như không đổi từ 2017, nên đây là phần ít mất giá nhất của chương |
| FlashAttention                                                                                       | Tính lại theo ô thay vì lưu ma trận $T \times T$ — chính là ý tưởng gradient checkpointing áp cho attention, đổi phép tính lấy bộ nhớ | Mặc định trong phần lớn thư viện transformer hiện nay                                  |
| DeepSpeed ZeRO, PyTorch FSDP                                                                         | Hệ quả trực tiếp của bảng kế toán $16P$ byte: chia trọng số, gradient và trạng thái Adam giữa các tiến trình                          | Ba giai đoạn ZeRO tương ứng ba dòng của bảng đó                                        |
| AdamW làm mặc định trong <code>timm</code> , <code>transformers</code> , và phần lớn recipe thị giác | Tách weight decay khỏi mẫu số thích nghi — đúng bước cuối của mạch bày optimizer                                                      | Lỗi Adam cộng L2 vẫn còn trong nhiều mã nguồn cũ                                       |
| <code>torch.amp</code> , bf16 trên GPU thế hệ Ampere trở đi                                          | Loss scaling và master weights fp32 — hai hệ quả trực tiếp của phân tích dải động                                                     | Bf16 làm loss scaling gần như không cần, nhưng không bỏ được master weights            |
| RandAugment, Mixup và CutMix trong <code>timm</code> và <code>transforms.v2</code>                   | Augmentation như nơi phát biểu bất biến, và như công cụ chính quy hoá mạnh nhất trong thị giác                                        | Chính các thành phần recipe này là thứ ConvNeXt dùng để bắt kịp ViT ở chương 13        |

#### MINI PROJECT SLAM · Bảng triệu chứng cho mạng chấm điểm ghép đặc trưng · 1–2 ngày

|                 |                                                                                                                                                                                                                                                                                                                                                                                                                              |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Mục tiêu</b> | tạo một thư viện triệu chứng dùng được suốt sự nghiệp: huấn luyện một mạng nhỏ dự đoán độ tin cậy của một cặp ghép đặc trưng ORB, rồi cố ý phá nó tám cách để học nhận ra từng dạng hỏng <i>chỉ bằng hình dạng đường loss</i> , trước khi cần tới chúng trong một hệ thật.                                                                                                                                                   |
| <b>Dữ liệu</b>  | một chuỗi EuRoC bất kỳ, chẳng hạn MH_01. Trích cặp ghép ORB giữa các khung hình liên tiếp; gán nhãn dương khi sai số tái chiếu dưới ngưỡng theo chân trị pose, âm khi vượt. Vài chục nghìn cặp là quá đủ; đầu vào có thể chỉ là khoảng cách Hamming, chênh lệch góc, chênh lệch tỉ lệ và toạ độ chuẩn hoá.                                                                                                                   |
| <b>Các bước</b> | chạy một lần baseline tới khi loss xác thực phẳng và ghi lại ba chỉ báo sức khoẻ; rồi chạy tám phiên bản hỏng có chủ đích — learning rate nhân 100, learning rate chia 100, quên shuffle, quên eval(), nhãn lệch một chỉ số, chuẩn hoá đầu vào bằng thống kê của chuỗi khác, khởi tạo toàn hằng số, tắt chuẩn hoá giữa mạng; vẽ cả chín đường loss vào <i>một</i> hình; cuối cùng che nhãn và tự đoán từng đường là lỗi nào. |
| <b>Xong khi</b> | bạn có bảng ánh xạ chín đường cong sang chín nguyên nhân, nhận đúng ít nhất bảy trên chín khi che nhãn, và với mỗi cặp để nhằm nêu được một phép đo phân biệt — tỉ lệ cập nhật trên trọng số, hoặc mức đổi điểm số khi đổi batch size lúc đánh giá.                                                                                                                                                                          |
| <b>Nối với</b>  | <a href="#">C/12/04</a> cho cây chẩn đoán mà bảng này bổ sung số liệu; <a href="#">C/12/03</a> cho ba chỉ báo sức khoẻ dùng làm phép đo phân biệt; tập cặp ghép đã gán nhãn tạo ở đây được dùng lại nguyên vẹn cho mini project của chương 13.                                                                                                                                                                               |

#### TRẢ LỜI CÂU HỎI MỞ ĐẦU

**A1.** Vì vi phân ngược trả về đạo hàm theo *mọi* đầu vào chỉ trong một lượt duy nhất, còn vi phân thuận cho đạo hàm theo đúng một hướng mỗi lượt. Bài toán huấn luyện có một đầu ra và  $10^9$  đầu vào, tức đúng hình dạng mà vi phân ngược phục vụ; tỉ số chi phí giữa hai cách chính là tỉ số giữa số đầu vào và số đầu ra.



- A2. Vì quy tắc chuỗi cho ra một *tích* của  $L$  Jacobian, và tích lũy thừa thì hoặc tắt hoặc nổ — một hệ số 0,9 qua 50 tầng còn  $5 \cdot 10^{-3}$ . Cái thiếu suốt hai mươi năm không phải công thức đạo hàm mà là ba cách giữ hệ số đó gần 1: hàm kích hoạt không bão hoà, khởi tạo theo lập luận phương sai, và chuẩn hoá.
- A3. Vì chúng nằm trên hai nhánh giải quyết hai vấn đề khác nhau — momentum lọc *hướng*, Ada-Grad và RMSProp chỉnh *độ dài bước* cho từng tham số — rồi hợp lại ở Adam. AdamW là cái sinh ra chỉ để sửa lỗi của phép hợp nhất đó: khi hình phạt L2 bị chia cho mẫu số thích nghi, nó không còn là weight decay nữa.
- A4. Vì nghiệm không do lớp hàm quyết định một mình, mà do *quá trình* tìm nghiệm quyết định. Nhiều của SGD, khởi tạo và lịch học đều thiên vị những nghiệm nhất định. Ngành gọi đó là chính quy hoá ngầm và chưa mô tả được nó một cách định lượng — nên các cận sai số dựa trên sức chứa gần như rỗng.
- A5. Vì dải động và độ chính xác là hai chuyện khác nhau. bf16 chỉ có 7 bit định trị, tức độ chính xác tương đối khoảng  $4 \cdot 10^{-3}$ . Bước cập nhật lành mạnh lại chỉ bằng  $10^{-3}$  độ lớn trọng số. Cộng một số nhỏ như vậy vào trọng số bf16 thì phép cộng làm tròn về giá trị cũ, và bước đi biến mất hoàn toàn.

### NÓI LẠI THẬT ĐƠN GIẢN

Hãy tưởng tượng bạn phải chỉnh một cây đàn piano khổng lồ có mười triệu sợi dây, và mỗi lần chỉnh xong bạn chỉ được nghe đúng một hợp âm rồi nhận đúng một con số: nghe chênh bao nhiêu. Nếu thử từng dây một — vặn nhẹ, gảy lại, nghe xem con số đổi thế nào — thì bạn cần mười triệu lần gảy chỉ để biết nên vặn theo hướng nào. Không ai làm nổi.

Mẹo của cả chương là đừng đi xuôi mà đi ngược. Sau khi gảy một lần, bạn bắt đầu từ tiếng chênh ở đầu ra và lần ngược qua từng cơ cấu truyền lực, hỏi mỗi cái đúng một câu rất hẹp: kéo đầu ra của anh nhích một li thì đầu vào của anh phải nhích bao nhiêu. Mỗi cơ cấu chỉ cần biết quy đổi nội bộ của riêng nó, và một vòng đi ngược duy nhất trả lời cho cả mười triệu dây.

Cái giá có ba phần. Thứ nhất, muốn đi ngược được thì lượt đi xuôi phải ghi nhớ trạng thái của mọi cơ cấu, nên thứ chận bạn thường là chỗ để ghi chứ không phải thời gian. Thứ hai, lực truyền qua ba mươi khớp nối: mỗi khớp nuốt một chút thì tới cuối chẳng còn gì, mỗi khớp thêm một chút thì nó giật đứt dây — gần như mọi thủ thuật còn lại chỉ để giữ mỗi khớp truyền đúng bằng lực nhận vào. Thứ ba, vặn bao nhiêu mỗi lần: mạnh tay thì vọt qua, nhẹ tay thì cả đời không xong.

**Chỗ không rút gọn được:** vì sao một cách chỉnh nằm ở vùng “phẳng” — vùng mà vặn lệch đi một chút vẫn nghe được — lại chơi hay hơn trên những bản nhạc chưa từng thử, thì tới nay chưa có lời giải thích vừa gọn vừa đúng. Mọi cách nói đơn giản hiện có đều bỏ sót một phần ví dụ, nên đây là chỗ nên biết là mình chưa biết.

**Một câu:** Đi xuôi một lần để ghi nhớ, đi ngược một lần để quy trách nhiệm cho từng núm vặn, rồi vặn thật nhẹ — và mọi thứ còn lại chỉ là giữ cho lực truyền qua ba mươi khớp nối không tắt cũng không giật.

## 1.7 Kết nối

Chương này đứng ở giữa cây và có hai chiều nối rõ ràng. Ngược lên, nó tiêu thụ A/01-toan-toi-thieu cho quy tắc chuỗi và lập luận phương sai, và B/09-thiet-ke-ham-mat-mat cho thứ mà toàn bộ bộ máy này đang tối thiểu hoá. Xuôi xuống, nó là tiền đề của *mọi* chương còn lại trong Phần C: không có nó thì các kiến trúc ở chương 13 tới 18 chỉ là sơ đồ khối không chạy được. Hai nhánh nối ngang đáng đi ngay sau chương này. Thứ nhất là B/11-gioi-han-tinh-toan: ở đó bộ nhớ kích hoạt và dải động số được đặt vào mô hình roofline. Thứ hai là E/24-thi-nghiem-va-ablation: ở đó bạn học cách không để một bảng so sánh dẫn tới kết luận sai, vì bảng ấy thường chỉ phản ánh khác biệt công thức huấn luyện.

## Tổng kết chương

Ta đã đi từ một phép nhân Jacobian cục bộ tới việc chia trạng thái optimizer giữa nhiều GPU, và điều đáng giữ lại là toàn bộ hành trình đó có một sợi chỉ duy nhất. Mục 1 cho thấy vì sao đạo hàm rẽ và cái giá của nó là bộ nhớ. Mục 2 cho thấy tích các Jacobian là đại lượng phải kiểm soát, và ba nhóm kỹ thuật tưởng rời rạc đều nhắm vào nó. Mục 3 cho thấy bảy optimizer là bảy lần vá liên tiếp, và learning rate vẫn là siêu tham số không có đối thủ. Mục 4 cho thấy phần lớn các lần “mô hình không tổng quát hoá” thực ra là lỗi trong vòng lặp, và đưa ra quy trình tách hai thứ đó.

Còn bỏ ngỏ hai chỗ lớn. Thứ nhất là chính quy hoá ngầm: ta chưa biết vì sao SGD lại chọn được nghiệm tổng quát hoá tốt trong một không gian đầy nghiệm học thuộc, nên mọi kết luận về sức chứa mô hình đều nên phát biểu thận trọng. Thứ hai là chương này cố tình bỏ qua kết nối tất, thành viên thứ tư của họ “giữ tích Jacobian gần 1”, vì nó là một lựa chọn kiến trúc; nó mở đầu chương tiếp theo. Đọc tiếp [C/13-kien-truc-backbone](#) với một câu hỏi trong đầu: trong mỗi bước tiến hoá kiến trúc, bao nhiêu phần là kiến trúc thật và bao nhiêu phần là công thức huấn luyện vừa học ở đây?

### **I Câu hỏi tự kiểm** (trả lời không nhìn bài)

1. Sợi chỉ của cả chương là giữ tích các Jacobian không chết cũng không nổ. Khởi tạo, normalization và lịch learning rate tấn công tích đó ở ba chỗ khác nhau nào, và vì sao có cả ba vẫn chưa đủ để bỏ warmup?
2. BatchNorm xuất hiện ở cả bốn mục với bốn vai khác nhau: gradient của một mẫu phụ thuộc cả batch, ảnh hưởng tới số điều kiện, tương tác với weight decay, và giới hạn của gradient accumulation. Bốn điều đó suy ra từ đúng một tính chất — tính chất nào, và nó ép bạn chọn gì khi huấn luyện phân tán?
3. Bộ nhớ kích hoạt của mục 1, trạng thái optimizer của mục 3 và phân mảnh ZeRO của mục 4 là ba khoản chi bộ nhớ khác nhau. Khi một lần chạy hết bộ nhớ, bạn cắt khoản nào trước, và mỗi lựa chọn đổi lấy cái gì?
4. Loss huấn luyện phẳng ở mức cao có thể do hình dạng landscape, do công thức huấn luyện, hoặc do một lỗi trong vòng lặp dữ liệu. Ba phép thử nào tách ba nguyên nhân đó, và vì sao phép “overfit một batch” phải chạy trước hai phép còn lại?
5. Vì sao một cải tiến đo trong công thức huấn luyện cũ có thể biến mất khi đo lại trong công thức mới, và điều đó ràng buộc thế nào cách bạn thiết kế ablation cho những kỹ thuật trong chương này?
6. Mục 4 thừa nhận ta chưa biết vì sao SGD chọn được nghiệm tổng quát hoá tốt. Với khoảng trống đó, quyết định thực hành nào trong chương vẫn giữ nguyên căn cứ, và quyết định nào chỉ còn là kinh nghiệm?
7. Bạn hạ mixed precision từ fp32 xuống bf16 và loss xấu đi một chút. Ba nguyên nhân nằm ở ba mục khác nhau của chương là gì, và mỗi nguyên nhân phân biệt bằng phép đo nào?

- [1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [2] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [3] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *AISTATS*, 2010.
- [4] K. He, X. Zhang, S. Ren, et al., “Delving Deep into Rectifiers,” in *ICCV*, 2015.
- [5] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [6] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [7] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [8] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [9] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [10] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [11] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” in *ICLR*, 2017.
- [12] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [13] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [14] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [15] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [16] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [17] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [18] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [19] G. Huang, Y. Sun, Z. Liu, et al., “Deep Networks with Stochastic Depth,” in *ECCV*, 2016.
- [20] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [21] H. Zhang, M. Cisse, Y. N. Dauphin, et al., “mixup: Beyond Empirical Risk Minimization,” in *ICLR*, 2018.
- [22] S. Yun, D. Han, S. J. Oh, et al., “CutMix: Regularization Strategy to Train Strong Classifiers,” in *ICCV*, 2019.
- [23] E. D. Cubuk, B. Zoph, J. Shlens, et al., “RandAugment: Practical Automated Data Augmentation,” in *CVPR Workshops*, 2020.

- [24] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [25] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [26] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *NeurIPS*, 2012.
- [27] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in *ICLR*, 2015.
- [28] C. Szegedy, W. Liu, et al., “Going Deeper with Convolutions,” in *CVPR*, 2015.
- [29] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [30] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [31] G. Huang, Z. Liu, L. van der Maaten, et al., “Densely Connected Convolutional Networks,” in *CVPR*, 2017.
- [32] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [33] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [34] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [35] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [36] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [37] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.
- [38] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [39] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [40] J. Su, M. Ahmed, Y. Lu, et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, vol. 568, 2024.
- [41] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” *arXiv:2312.00752*, 2023.
- [42] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [43] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.
- [44] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [45] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [46] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [47] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [48] R. Girshick, J. Donahue, T. Darrell, et al., “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation,” in *CVPR*, 2014.

- [49] R. Girshick, "Fast R-CNN," in *ICCV*, 2015.
- [50] S. Ren, K. He, R. Girshick, et al., "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in *NeurIPS*, 2015.
- [51] T.-Y. Lin, P. Dollár, R. Girshick, et al., "Feature Pyramid Networks for Object Detection," in *CVPR*, 2017.
- [52] K. He, G. Gkioxari, P. Dollár, et al., "Mask R-CNN," in *ICCV*, 2017.
- [53] D. Bolya, S. Foley, J. Hays, et al., "TIDE: A General Toolbox for Identifying Object Detection Errors," in *ECCV*, 2020.
- [54] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [55] J. Redmon, S. Divvala, R. Girshick, et al., "You Only Look Once: Unified, Real-Time Object Detection," in *CVPR*, 2016.
- [56] W. Liu, D. Anguelov, D. Erhan, et al., "SSD: Single Shot MultiBox Detector," in *ECCV*, 2016.
- [57] T.-Y. Lin, P. Goyal, R. Girshick, et al., "Focal Loss for Dense Object Detection," in *ICCV*, 2017.
- [58] Z. Tian, C. Shen, H. Chen, et al., "FCOS: Fully Convolutional One-Stage Object Detection," in *ICCV*, 2019.
- [59] X. Zhou, D. Wang, and P. Krähenbühl, "Objects as Points," *arXiv:1904.07850*, 2019.
- [60] S. Zhang, C. Chi, Y. Yao, et al., "Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection," in *CVPR*, 2020.
- [61] Z. Ge, S. Liu, Z. Li, et al., "OTA: Optimal Transport Assignment for Object Detection," in *CVPR*, 2021.
- [62] N. Bodla, B. Singh, R. Chellappa, et al., "Soft-NMS: Improving Object Detection with One Line of Code," in *ICCV*, 2017.
- [63] X. Zhu, W. Su, L. Lu, et al., "Deformable DETR: Deformable Transformers for End-to-End Object Detection," in *ICLR*, 2021.
- [64] H. Zhang, F. Li, S. Liu, et al., "DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection," in *ICLR*, 2023.
- [65] J. Long, E. Shelhamer, and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," in *CVPR*, 2015.
- [66] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *MICCAI*, 2015.
- [67] L.-C. Chen, G. Papandreou, I. Kokkinos, et al., "DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs," *IEEE TPAMI*, vol. 40, no. 4, pp. 834–848, 2018.
- [68] E. Xie, W. Wang, Z. Yu, et al., "SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers," in *NeurIPS*, 2021.
- [69] B. Cheng, I. Misra, A. G. Schwing, et al., "Masked-Attention Mask Transformer for Universal Image Segmentation," in *CVPR*, 2022.
- [70] A. Kirillov, K. He, R. Girshick, et al., "Panoptic Segmentation," in *CVPR*, 2019.
- [71] A. Kirillov, E. Mintun, N. Ravi, et al., "Segment Anything," in *ICCV*, 2023.
- [72] K. Sun, B. Xiao, D. Liu, et al., "Deep High-Resolution Representation Learning for Human Pose Estimation," in *CVPR*, 2019.



- [73] Z. Cao, T. Simon, S.-E. Wei, et al., “Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields,” in *CVPR*, 2017.
- [74] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [75] J. Deng, J. Guo, N. Xue, et al., “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *CVPR*, 2019.
- [76] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [77] K. Xu, J. Ba, R. Kiros, et al., “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention,” in *ICML*, 2015.
- [78] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *ICLR*, 2015.
- [79] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [80] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [81] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [82] K. Simonyan and A. Zisserman, “Two-Stream Convolutional Networks for Action Recognition in Videos,” in *NeurIPS*, 2014.
- [83] J. Carreira and A. Zisserman, “Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset,” in *CVPR*, 2017.
- [84] D. Tran, H. Wang, L. Torresani, et al., “A Closer Look at Spatiotemporal Convolutions for Action Recognition,” in *CVPR*, 2018.
- [85] C. Feichtenhofer, H. Fan, J. Malik, et al., “SlowFast Networks for Video Recognition,” in *ICCV*, 2019.
- [86] A. Dosovitskiy, P. Fischer, E. Ilg, et al., “FlowNet: Learning Optical Flow with Convolutional Networks,” in *ICCV*, 2015.
- [87] D. Sun, X. Yang, M.-Y. Liu, et al., “PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume,” in *CVPR*, 2018.
- [88] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.
- [89] A. Bewley, Z. Ge, L. Ott, et al., “Simple Online and Realtime Tracking,” in *ICIP*, 2016.
- [90] Y. Zhang, P. Sun, Y. Jiang, et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *ECCV*, 2022.
- [91] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [92] K. Bernardin and R. Stiefelhagen, “Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics,” *EURASIP J. Image and Video Processing*, 2008.
- [93] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [94] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [95] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.



- [96] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *CVPR*, 2024.
- [97] D. Eigen, C. Puhrsch, and R. Fergus, “Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network,” in *NeurIPS*, 2014.
- [98] R. Ranftl, K. Lasinger, D. Hafner, et al., “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer,” *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [99] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IROS*, 2012.
- [100] C. R. Qi, H. Su, K. Mo, et al., “PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation,” in *CVPR*, 2017.
- [101] C. R. Qi, L. Yi, H. Su, et al., “PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space,” in *NeurIPS*, 2017.
- [102] A. H. Lang, S. Vora, H. Caesar, et al., “PointPillars: Fast Encoders for Object Detection from Point Clouds,” in *CVPR*, 2019.
- [103] T. Müller, A. Evans, C. Schied, et al., “Instant Neural Graphics Primitives with a Multiresolution Hash Encoding,” *ACM TOG*, vol. 41, no. 4, 2022.
- [104] B. Kerbl, G. Kopanas, T. Leimkühler, et al., “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *ACM TOG*, vol. 42, no. 4, 2023.
- [105] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.
- [106] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” in *ICLR*, 2014.
- [107] I. Goodfellow, J. Pouget-Abadie, M. Mirza, et al., “Generative Adversarial Nets,” in *NeurIPS*, 2014.
- [108] T. Karras, S. Laine, and T. Aila, “A Style-Based Generator Architecture for Generative Adversarial Networks,” in *CVPR*, 2019.
- [109] J. Ho, A. Jain, and P. Abbeel, “Denoising Diffusion Probabilistic Models,” in *NeurIPS*, 2020.
- [110] Y. Song, J. Sohl-Dickstein, D. P. Kingma, et al., “Score-Based Generative Modeling Through Stochastic Differential Equations,” in *ICLR*, 2021.
- [111] J. Johnson, A. Alahi, and L. Fei-Fei, “Perceptual Losses for Real-Time Style Transfer and Super-Resolution,” in *ECCV*, 2016.
- [112] P. Isola, J.-Y. Zhu, T. Zhou, et al., “Image-to-Image Translation with Conditional Adversarial Networks,” in *CVPR*, 2017.
- [113] L. A. Gatys, A. S. Ecker, and M. Bethge, “Image Style Transfer Using Convolutional Neural Networks,” in *CVPR*, 2016.
- [114] X. Huang and S. Belongie, “Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization,” in *ICCV*, 2017.
- [115] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.
- [116] J. Song, C. Meng, and S. Ermon, “Denoising Diffusion Implicit Models,” in *ICLR*, 2021.
- [117] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, et al., “Flow Matching for Generative Modeling,” in *ICLR*, 2023.
- [118] J. Ho and T. Salimans, “Classifier-Free Diffusion Guidance,” *arXiv:2207.12598*, 2022.
- [119] R. Rombach, A. Blattmann, D. Lorenz, et al., “High-Resolution Image Synthesis with Latent Diffusion Models,” in *CVPR*, 2022.
- [120] W. Peebles and S. Xie, “Scalable Diffusion Models with Transformers,” in *ICCV*, 2023.

- [121] L. Zhang, A. Rao, and M. Agrawala, “Adding Conditional Control to Text-to-Image Diffusion Models,” in *ICCV*, 2023.
- [122] Y. Blau and T. Michaeli, “The Perception-Distortion Tradeoff,” in *CVPR*, 2018.
- [123] C. Ledig, L. Theis, F. Huszár, et al., “Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network,” in *CVPR*, 2017.
- [124] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., “Image Quality Assessment: From Error Visibility to Structural Similarity,” *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [125] M. Heusel, H. Ramsauer, T. Unterthiner, et al., “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium,” in *NeurIPS*, 2017.
- [126] T. Brooks, B. Mildenhall, T. Xue, et al., “Unprocessing Images for Learned Raw Denoising,” in *CVPR*, 2019.
- [127] A. Abdelhamed, S. Lin, and M. S. Brown, “A High-Quality Denoising Dataset for Smartphone Cameras,” in *CVPR*, 2018.
- [128] J. Deng, W. Dong, R. Socher, et al., “ImageNet: A Large-Scale Hierarchical Image Database,” in *CVPR*, 2009.
- [129] A. Radford, J. W. Kim, C. Hallacy, et al., “Learning Transferable Visual Models from Natural Language Supervision,” in *ICML*, 2021.
- [130] J. Li, D. Li, S. Savarese, et al., “BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models,” in *ICML*, 2023.
- [131] H. Liu, C. Li, Q. Wu, et al., “Visual Instruction Tuning,” in *NeurIPS*, 2023.
- [132] S. Kornblith, J. Shlens, and Q. V. Le, “Do Better ImageNet Models Transfer Better?” In *CVPR*, 2019.
- [133] T. Chen, S. Kornblith, M. Norouzi, et al., “A Simple Framework for Contrastive Learning of Visual Representations,” in *ICML*, 2020.
- [134] K. He, H. Fan, Y. Wu, et al., “Momentum Contrast for Unsupervised Visual Representation Learning,” in *CVPR*, 2020.
- [135] J.-B. Grill, F. Strub, F. Altché, et al., “Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning,” in *NeurIPS*, 2020.
- [136] M. Caron, H. Touvron, I. Misra, et al., “Emerging Properties in Self-Supervised Vision Transformers,” in *ICCV*, 2021.
- [137] K. He, X. Chen, S. Xie, et al., “Masked Autoencoders Are Scalable Vision Learners,” in *CVPR*, 2022.
- [138] A. van den Oord, Y. Li, and O. Vinyals, “Representation Learning with Contrastive Predictive Coding,” in *arXiv:1807.03748*, 2018.
- [139] J. Kaplan, S. McCandlish, T. Henighan, et al., “Scaling Laws for Neural Language Models,” *arXiv:2001.08361*, 2020.
- [140] X. Zhai, A. Kolesnikov, N. Houlsby, et al., “Scaling Vision Transformers,” in *CVPR*, 2022.
- [141] J. Hoffmann, S. Borgeaud, A. Mensch, et al., “Training Compute-Optimal Large Language Models,” in *NeurIPS*, 2022.
- [142] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [143] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [144] N. Houlsby, A. Giurgiu, S. Jastrzebski, et al., “Parameter-Efficient Transfer Learning for NLP,” in *ICML*, 2019.

- [145] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [146] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *RecSys*, 2019.

## Các họ kiến trúc backbone (Backbone Architecture Families)

**PHỤ THUỘC** C/12-co-che-huan-luyen (toàn chương — không có nó thì mọi kiến trúc ở đây chỉ là sơ đồ khối); B/10-thiet-ke-kien-truc (trực đánh đổi giả định-dữ liệu); B/11-gioi-han-tinh-toan (FLOPs không phải độ trễ).

### TÓM TẮT

Chương này đọc lịch sử kiến trúc như *một cuộc tranh luận kéo dài về việc nên cài bao nhiêu giả định vào mô hình*. Mỗi thể hệ trả lời khác nhau, vì lượng dữ liệu sẵn có ở thời điểm đó khác nhau. Đọc xong bạn chọn được *backbone* theo ràng buộc thật của mình — dữ liệu, độ trễ, độ phân giải — thay vì theo bảng xếp hạng, và đọc được cấu trúc một repo thị giáic lạ trong mười phút. Nếu chỉ nhớ một điều: mọi kiến trúc trong chương là một điểm trên cùng một trục. Một đầu là “cài sẵn nhiều giả định, cần ít dữ liệu”, đầu kia là “cài ít giả định, cần nhiều dữ liệu”. Bài học đắt nhất: phần lớn chênh lệch từng được gán cho kiến trúc hoá ra thuộc về công thức huấn luyện.

### KIẾN THỨC CẦN CÓ TRƯỚC

- **backbone** — phần mạng làm nhiệm vụ trích đặc trưng từ ảnh, dùng chung được cho nhiều bài toán; phần sinh đầu ra cụ thể được lắp thêm phía sau.
- **inductive bias** — tập giả định mà mô hình mang sẵn trước khi nhìn dữ liệu; nhiều giả định đúng thì học nhanh với ít mẫu, giả định sai thì chặn luôn nghiệm đúng.
- **convolution** — phép nhân một cửa sổ trọng số nhỏ trượt khắp ảnh; vì cùng bộ trọng số dùng cho mọi vị trí, nó cài sẵn hai giả định — quan hệ gần là quan trọng, và mẫu hình có ý nghĩa như nhau ở mọi vị trí.
- **trường tiếp nhận (receptive field)** — vùng pixel đầu vào ảnh hưởng tới một giá trị đầu ra; nó lớn dần theo chiều sâu trong CNN và bằng toàn ảnh ngay từ tầng đầu trong attention toàn cục.
- **attention** — cơ chế cho mỗi vị trí tự chọn lấy thông tin từ mọi vị trí khác bằng trọng số học được, thay vì chờ thông tin lan dần qua từng tầng.
- **FLOPs** — số phép nhân-cộng mà mô hình thực hiện; nó là chỉ số đại diện phổ biến cho chi phí, nhưng *không* tỉ lệ với độ trễ thật vì còn phụ thuộc băng thông bộ nhớ.
- **pretraining** — huấn luyện trước trên một tập rất lớn để học đặc trưng chung, rồi mới huấn luyện tiếp trên tập nhỏ của bài toán đích.

### CÂU HỎI MỞ ĐẦU

- Q1. Nếu **attention** cho mọi vị trí nói chuyện trực tiếp với nhau ngay từ tầng đầu tiên, vì sao suốt mười năm người ta vẫn dùng **convolution** — thứ cổ tình chỉ cho phép nhìn một cửa sổ  $3 \times 3$ ?
- Q2. Một mạng phẳng 56 tầng có sai số *huấn luyện* cao hơn mạng 20 tầng, dù nó thừa sức chứa để bắt chước mạng nông bằng cách để các tầng thừa làm ánh xạ đồng nhất. Chuyện gì đã xảy ra?
- Q3. MobileNet giảm số phép nhân khoảng 8,7 lần. Vì sao độ trễ đo được trên GPU lại không giảm 8,7 lần?
- Q4. Nếu ConvNeXt cho thấy CNN vẫn ngang ViT khi dùng cùng công thức huấn luyện, thì mười năm “cải tiến kiến trúc” đã thực sự cải tiến cái gì?

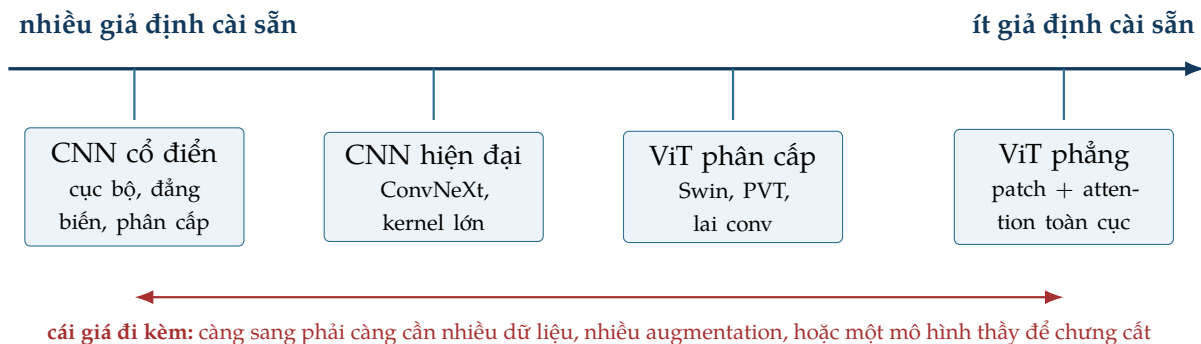
**Q5.** Vì sao cùng một kiến trúc lại thua CNN khi có một triệu ảnh và thắng khi có ba trăm triệu ảnh — điều gì trong mô hình thay đổi theo lượng dữ liệu?

## 2.1 Vì sao chương này chia làm ba mạch

Ba mục của chương không phải ba họ kiến trúc song song mà là ba lần trả lời cùng một câu hỏi, trên ba loại dữ liệu và ở ba thời điểm khác nhau. Đọc theo thứ tự này thì mục sau luôn có sẵn khái niệm mà mục trước dựng lên.

1. **Mạch 1 — dòng CNN.** Bắt đầu từ câu trả lời “cài nhiều giả định”, rồi theo dõi mười lăm năm gõ từng nút thắt. Mục này dựng lên các khái niệm nền của cả chương: trường tiếp nhận, chi phí tính toán, kết nối tắt, và bài học ConvNeXt về recipe.
2. **Mạch 2 — từ chuỗi tới attention.** Chuyển sang loại dữ liệu mà convolution không phục vụ được — dữ liệu có thứ tự và quan hệ xa — và theo dõi chuỗi RNN, LSTM, attention, transformer, rồi các cách chống chi phí bậc hai. Đây là mục cung cấp cơ chế cho mạch 3.
3. **Mạch 3 — ViT và inductive bias.** Mang attention vào ảnh, tức chọn câu trả lời “cài rất ít giả định”, rồi đo cái giá. Mục này chỉ đọc được sau hai mục trước, vì nó liên tục so sánh với cả hai.

Hình 2.1 đặt cả ba mạch lên một trục duy nhất. Điều đáng chú ý là trục này không có điểm “tốt nhất”: vị trí đúng phụ thuộc vào lượng dữ liệu bạn có, và đó chính là nội dung của cuộc tranh luận.



**Hình 2.1.** Bốn họ kiến trúc đặt trên trục “lượng giả định cài sẵn”. Trục này không có điểm tối ưu tuyệt đối: vị trí đúng phụ thuộc lượng dữ liệu có sẵn mà bạn nắm trong tay, nên cùng một bài toán có thể đổi câu trả lời khi tập dữ liệu lớn lên.

## Bản đồ chương 13 — Các họ kiến trúc backbone

### TÓM TẮT

Chương 13 gồm ba mục, và chúng là ba lần trả lời *cùng một câu hỏi*: nên cài sẵn bao nhiêu giả định vào mô hình. Ba lần đó diễn ra trên ba loại dữ liệu và ở ba thời điểm khác nhau. Thứ tự 1 → 2 → 3 là bắt buộc vì mục 3 liên tục so sánh với cả hai mục trước. Nếu chỉ nhớ một điều: mọi kiến trúc trong chương nằm trên một trục duy nhất giữa “nhiều giả định, ít dữ liệu” và “ít giả định, nhiều dữ liệu”, và không có điểm nào trên trục đó là tốt nhất một cách tuyệt đối.

**KIẾN THỨC CẦN CÓ TRƯỚC**

- **backbone** — phần mạng trích đặc trưng từ ảnh, dùng chung cho nhiều bài toán; phần sinh đầu ra cụ thể (nhân, hộp, mặt nạ) được lắp thêm phía sau.
- **inductive bias** — tập giả định mà mô hình mang sẵn trước khi nhìn dữ liệu; nó là đại lượng trung tâm mà cả chương xoay quanh.
- **convolution** — phép nhân một cửa sổ trọng số nhỏ trượt khắp ảnh, cài sẵn giả định “quan hệ gần là quan trọng” và “mẫu hình có ý nghĩa như nhau ở mọi vị trí”.
- **attention** — cơ chế cho mỗi vị trí lấy thông tin trực tiếp từ mọi vị trí khác bằng trọng số học được, không cần thông tin lan dần qua nhiều tầng.

**Có gì trong chương này****Bảng 2.1.** Ba mục của chương 13: câu hỏi trung tâm, phụ thuộc và mức độ đầu tư.

| Mục                       | Câu hỏi trung tâm                                                                | Phụ thuộc      | Mức độ       |
|---------------------------|----------------------------------------------------------------------------------|----------------|--------------|
| 1. Dòng CNN               | Mỗi thể hệ CNN gỡ nút thắt nào, và đẩy độ khó sang trục nào?                     | C/12           | Cốt lõi [T2] |
| 2. Từ chuỗi tới attention | Xử lý quan hệ xa thế nào, và cái giá của việc rút ngắn đường đi thông tin là gì? | Mục 1, C/12/02 | Cốt lõi [T2] |
| 3. ViT và inductive bias  | Vứt bỏ giả định thị giác thì mất gì, và trả lại bằng cách nào?                   | Mục 1 và 2     | Cốt lõi [T2] |

**Vì sao lại chia như vậy**

Cách chia phổ biến trong sách là chia theo *họ kiến trúc*: CNN, RNN, Transformer. Cách đó che mất điều quan trọng nhất. Ba họ này trả lời cùng một câu hỏi thiết kế, chứ không phục vụ ba thể giới tách biệt. Ở đây ta chia theo *loại dữ liệu và lượng giả định*: mục 1 là dữ liệu lưới với nhiều giả định; mục 2 là dữ liệu có thứ tự, nơi giả định lưới không dùng được; mục 3 là quay lại dữ liệu lưới nhưng với rất ít giả định.

Lợi ích của cách chia này lộ ra ở hai chỗ nổi. Thứ nhất, cổng của LSTM ở mục 2 và kết nối tắt của ResNet ở mục 1 hoá ra là cùng một mẹo — tạo một đường cộng cho gradient — và cách chia theo họ kiến trúc sẽ giấu mất điều đó. Thứ hai, mục 3 chỉ có nghĩa như một *thí nghiệm đối chứng* cho mục 1: nó đo giá trị của các giả định mà mục 1 mô tả, bằng đơn vị là lượng dữ liệu.

**Đọc theo thứ tự nào**

1. Đọc tuần tự 1 → 2 → 3. Mục 3 tham chiếu liên tục sang hai mục kia nên đọc trước sẽ hệt.
2. Nếu bạn chỉ cần chọn một backbone cho một dự án đang chạy: đọc mục 1 phần bảng dòng CNN, rồi nhảy tới mục 3 phần G (tách backbone, neck, head) và phần giới hạn. Quay lại mục 2 khi phải xử lý ảnh độ phân giải cao.
3. Nếu bạn đến từ phía SLAM hoặc hệ thống nhúng: đọc mục 1 phần C (ba phép tính tay) trước tiên, vì đó là chỗ FLOPs, cường độ số học và độ trễ thật gặp nhau.

**Cốt lõi và tham khảo**

Ba phần đáng quay lại nhiều lần: ba phép tính tay ở mục 1 (xếp chồng  $3 \times 3$ , nút thắt  $1 \times 1$ , depthwise separable); lập luận  $\sqrt{d}$  và bảng ba hướng chống chi phí bậc hai ở mục 2; và trục giả định–dữ liệu cùng sơ đồ backbone–neck–head ở mục 3. Ba phần đọc lướt lần đầu rồi tra lại khi cần: danh sách biến thể convolution, chi tiết các họ mã hoá vị trí, và phần mô hình không gian trạng thái — phần cuối này thay đổi nhanh nhất nên cũng mất giá nhanh nhất.



### Nối vào phần còn lại của cây

Chương này tiêu thụ toàn bộ [C/12-co-che-huan-luyen](#) và trực đánh đổi ở B/10-thiet-ke-kien-truc; nó cấp đầu vào cho [C/14-bai-toan-cv-loi](#) (head cụ thể lắp lên backbone), [C/16-thi-giac-3d](#) và [C/18-pretraining-va-foundation](#). Hai nhánh ngang đáng nhớ: phần FLOPs và cường độ số học nổi sang B/11-gioi-han-tinh-toan cùng D/20-toi-uu-va-nen-mo-hinh; còn bài học ConvNeXt — rằng recipe giải thích phần lớn chênh lệch được gán cho kiến trúc — là tiền đề trực tiếp của E/24-thi-nghiem-va-ablation.

## 2.2 Dòng CNN: gỡ từng nút thắt (The CNN Lineage)

**PHỤ THUỘC** [C/12-co-che-huan-luyen](#) (đặc biệt mục 2 — kết nối tất là thành viên thứ tư của họ “giữ tích Jacobian gần 1”); B/11-gioi-han-tinh-toan (FLOPs, cường độ số học, chế độ ngهن bộ nhớ).

### TÓM TẮT

Mục này đọc mười lăm năm lịch sử *CNN* như một chuỗi gỡ nút thắt: mỗi kiến trúc ra đời để phá đúng một rào cản của kiến trúc trước, và tạo ra rào cản mới cho kiến trúc sau. Đọc xong bạn nhìn một *backbone* lạ và gọi tên được nó đang tối ưu cho ràng buộc nào: tham số, độ sâu, FLOPs, hay độ trễ thật. Nhờ đó bạn chọn backbone theo ràng buộc của mình, thay vì theo bảng xếp hạng. Nếu chỉ nhớ một điều: bài học lớn nhất của cả dòng nằm ở mắt xích cuối, ConvNeXt. Rất nhiều “cải tiến kiến trúc” trong lịch sử hoá ra là cải tiến *công thức huấn luyện*. Nên mọi so sánh kiến trúc không kiểm soát recipe đều đáng ngờ.

### KIẾN THỨC CẦN CÓ TRƯỚC

- **convolution** — phép nhân một cửa sổ trọng số nhỏ, gọi là kernel hay filter, trượt khắp ảnh; vì cùng bộ trọng số dùng cho mọi vị trí nên số tham số không phụ thuộc kích thước ảnh.
- **kênh (channel)** — chiều thứ ba của một bản đồ đặc trưng — ảnh RGB có 3 kênh đầu vào, còn một tầng giữa mạng có thể có 256 kênh, mỗi kênh là một loại đặc trưng mà mạng học được.
- **stride** — bước nhảy của cửa sổ khi trượt; stride 2 làm bản đồ đặc trưng nhỏ đi một nửa mỗi chiều, nên nó là cách chính để giảm độ phân giải và tăng tầm nhìn.
- **trường tiếp nhận (receptive field)** — vùng pixel đầu vào ảnh hưởng tới một giá trị đầu ra; nó lớn dần theo chiều sâu, và cả mục này xoay quanh việc mở rộng nó cho rõ.
- **pooling** — phép gộp một vùng nhỏ thành một giá trị (lấy lớn nhất hoặc lấy trung bình), dùng để giảm độ phân giải mà không tốn tham số.
- **FLOPs** — số phép nhân-cộng mà mô hình thực hiện; đây là chỉ số đại diện phổ biến cho chi phí, nhưng *không* tỉ lệ với độ trễ thật.
- **cường độ số học (arithmetic intensity)** — số phép tính thực hiện trên mỗi byte dữ liệu đọc từ bộ nhớ; thấp thì kernel bị nghẽn ở băng thông chứ không ở khả năng tính toán, và giảm FLOPs lúc đó gần như vô ích.
- **recipe (công thức huấn luyện)** — toàn bộ các lựa chọn không thuộc kiến trúc — optimizer, lịch học, số epoch, augmentation — và bài học cuối mục là chúng giải thích phần lớn chênh lệch giữa các kiến trúc.

### A. Đặt vấn đề

Toàn bộ chương 13 là một cuộc tranh luận kéo dài về một câu hỏi: *nên cài sẵn bao nhiêu giả định vào mô hình?* **Convolution** là một câu trả lời rất mạnh cho câu hỏi đó, và nó cài sẵn đúng ba giả định. Thứ nhất là **tính cục bộ (locality)** — một pixel chỉ liên quan trực tiếp tới các pixel lân cận, nên trọng số bằng 0 ở ngoài cửa sổ  $k \times k$ . Thứ hai là **chia sẻ trọng số (weight sharing)**, tương đương

với **đẳng biến tịnh tiến** (*translation equivariance*) — một mẫu hình có ý nghĩa như nhau dù nó nằm ở đâu trong ảnh. Thứ ba là **phân cấp** (*hierarchy*) — đặc trưng phức tạp được ghép từ đặc trưng đơn giản, nên tăng dần độ trừu tượng và giảm dần độ phân giải.

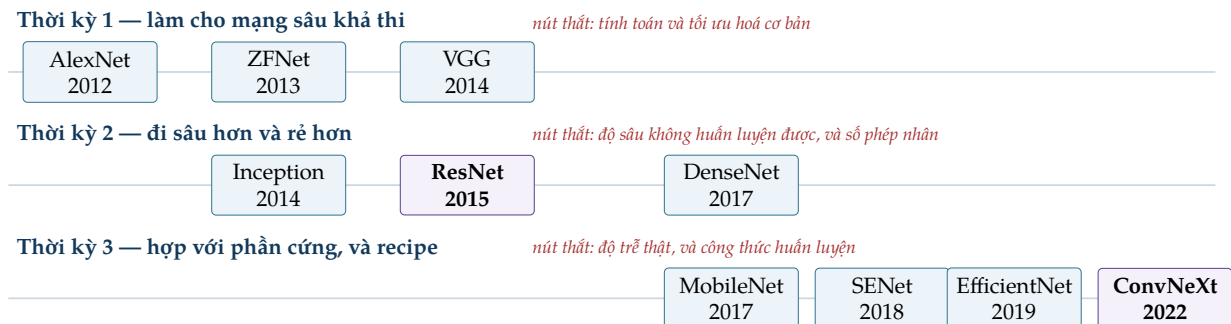
Ba giả định này đến từ vật lý của việc tạo ảnh chứ không từ toán học, và chúng đúng đủ tốt để một mạng conv cần ít dữ liệu hơn hẳn một mạng dày đặc cùng sức chứa. Nhưng chúng cũng khoá lại một không gian thiết kế, và lịch sử CNN chính là lịch sử của việc lần lượt phát hiện ra rào cản nào trong không gian đó đang chặn, rồi gỡ nó. Giá trị cụ thể của mục này: cầm một backbone lạ lên, bạn trả lời được ba câu mà không cần chạy thử. Nó đang tiết kiệm cái gì? Nó trả giá ở đâu? Và ràng buộc nào khiến người ta chọn nó?

#### TRỰC GIÁC

Đừng đọc dòng CNN như một cuộc thi xem ai chính xác hơn. Hãy đọc nó như lịch sử tối ưu một đường ống sản xuất, nơi mỗi thế hệ tìm ra *nút cổ chai hiện thời* rồi mở rộng đúng chỗ đó. Mở đúng nút cổ chai thì thông lượng tăng vọt, và nút cổ chai lập tức chuyển sang chỗ khác. Đó là lý do mỗi bước tiến trong bảng dưới đây lại sinh ra một vấn đề mới ở trực khác. Người đọc lịch sử theo cách này đoán được bước tiếp theo; người đọc theo cách “ai chính xác hơn” chỉ nhớ được tên.

### B. Dòng thời gian: ba thời kỳ, ba nút thắt khác nhau

Nhìn từ xa, mười lăm năm này chia làm ba thời kỳ rõ rệt, và *nút thắt của mỗi thời kỳ nằm ở một trực khác nhau*. Thời kỳ đầu vướng ở tính khả thi. Thời kỳ giữa vướng ở khả năng tối ưu hoá. Thời kỳ sau vướng ở phần cứng và ở công thức huấn luyện.



**Hình 2.2.** Dòng CNN đặt trên trục thời gian và tách theo nút thắt được gỡ. Hai mắt xích được tô đậm vì chúng thay đổi cách ngành suy nghĩ chứ không chỉ thay đổi con số: ResNet cho thấy rào cản của mạng sâu là *tối ưu hoá* chứ không phải sức chứa, còn ConvNeXt cho thấy phần lớn khoảng cách giữa các kiến trúc là khoảng cách công thức huấn luyện.

Bảng 2.2 đọc cùng Hình 2.2 theo chiều dọc; cột cuối là cột đáng nhớ, vì nó cho thấy không bước nào miễn phí.

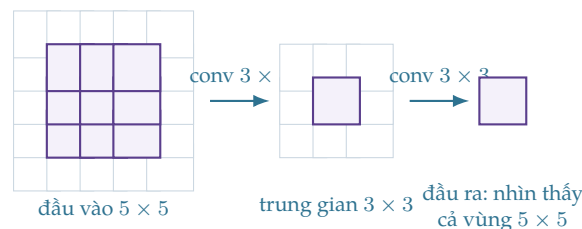
**Bảng 2.2.** Dòng CNN dọc theo mạch gỡ nút thắt. Cột cuối cho thấy mỗi bước tiến đẩy độ khó sang một trục khác chứ không xoá nó.

| Bước         | Gỡ nút thắt gì                                                                              | Cái giá / vấn đề mới                                                                                            |
|--------------|---------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| LeNet        | Chứng minh conv + pooling + huấn luyện gradient học được đặc trưng thị giác [25]            | Chỉ chạy ở quy mô chữ số; chưa có dữ liệu và phần cứng để mở rộng                                               |
| AlexNet      | Chứng minh mạng sâu + GPU + ReLU + dropout là khả thi ở quy mô ImageNet [26]                | Kiến trúc làm bằng tay, filter lớn $11 \times 11$ , rất nhiều tham số ở tầng FC                                 |
| ZFNet        | Trực quan hoá để thấy mạng học gì, từ đó chỉnh filter và stride                             | Đóng góp là phương pháp quan sát, không phải kiến trúc mới                                                      |
| VGG          | Xếp chồng $3 \times 3$ : cùng trường tiếp nhận, ít tham số hơn, nhiều phi tuyến hơn [27]    | Tầng FC chiếm phần lớn tham số; tổng FLOPs rất cao                                                              |
| Inception    | Nhiều tỉ lệ song song; $1 \times 1$ làm nút thắt giảm chiều trước phép tính nặng [28], [29] | Kiến trúc phức tạp, nhiều siêu tham số nhánh, khó chỉnh và khó mở rộng                                          |
| ResNet       | Vấn đề <i>tối ưu hoá</i> của mạng sâu, bằng đường cộng đồng nhất [30]                       | Vẫn nhiều FLOPs; nhánh tắt buộc giữ thêm kích hoạt nên tốn bộ nhớ hơn                                           |
| DenseNet     | Tái dùng đặc trưng tối đa, ít tham số hơn ở cùng độ chính xác [31]                          | Phép nối làm bộ nhớ và băng thông phình ra nếu cài đặt ngây thơ                                                 |
| MobileNet    | Chi phí tính toán trên thiết bị yếu, bằng depthwise separable [32]                          | Giảm FLOPs nhiều hơn giảm độ trễ thật, vì cường độ số học tụt                                                   |
| SENet        | Mạng không tự điều chỉnh được tầm quan trọng giữa các kênh [33]                             | Thêm một điểm đồng bộ toàn cục; lợi ích nhỏ dần khi backbone đã mạnh                                            |
| EfficientNet | Việc mở rộng mô hình bị làm tùy tiện theo một chiều [34]                                    | Tối ưu cho FLOPs, không tối ưu cho một phần cứng cụ thể                                                         |
| ConvNeXt     | Nghi vấn “ViT thắng nhờ kiến trúc”, bằng cách hiện đại hoá recipe của CNN [35]              | Cho thấy phần lớn chênh lệch trước đó là do công thức huấn luyện, làm mọi so sánh kiến trúc cũ trở nên đáng ngờ |

### C. Ba phép tính tay giải thích ba bước quan trọng nhất

Ba bước có ảnh hưởng lớn nhất trong bảng đều rút gọn được thành một phép tính ngắn. Tự làm ba phép tính này một lần thì bạn nhớ chúng mãi.

**VGG — vì sao xếp chồng  $3 \times 3$ .** Hai tầng  $3 \times 3$  nối tiếp có cùng trường tiếp nhận với một tầng  $5 \times 5$ ; ba tầng thì bằng  $7 \times 7$ . Nhưng chi phí tham số khác hẳn. Với  $C$  kênh vào và ra, một tầng  $5 \times 5$  tốn  $25C^2$  tham số, còn hai tầng  $3 \times 3$  chỉ tốn  $2 \cdot 9C^2 = 18C^2$  — ít hơn 28 phần trăm. So  $7 \times 7$  với ba tầng  $3 \times 3$  thì là  $49C^2$  đối với  $27C^2$ , ít hơn 45 phần trăm. Và ta còn được thêm hai lần phi tuyến. Nói gọn: cùng tầm nhìn, ít tham số hơn, nhiều khả năng biểu diễn hơn — một trong những phép đổi hiểm hơi tốt ở cả ba mặt.

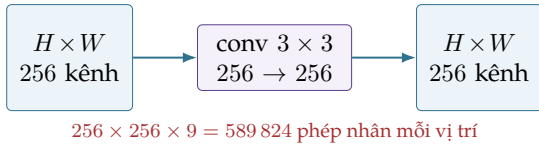


**Hình 2.3.** Hai tầng  $3 \times 3$  nối tiếp cho một ô đầu ra nhìn thấy đúng vùng  $5 \times 5$  của đầu vào. Cùng trường tiếp nhận với một tầng  $5 \times 5$ , nhưng chỉ tốn  $18C^2$  tham số thay vì  $25C^2$ . Và có thêm một lần phi tuyến ở giữa.

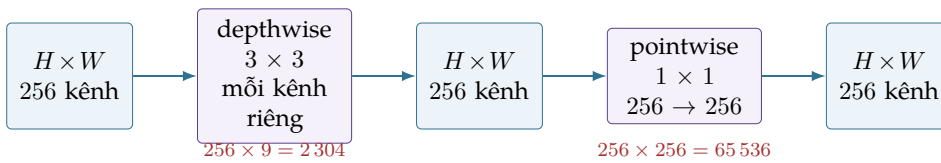
Hình 2.3 là toàn bộ lập luận của VGG dưới dạng hình. Điều nó *không* nói ra là nút thắt kế tiếp: tầng kết nối đầy đủ đầu tiên của VGG-16 có  $7 \cdot 7 \cdot 512 \cdot 4096 \approx 103$  triệu tham số, tức khoảng ba phần tư toàn bộ mô hình nằm trong một tầng duy nhất không phải conv. Đó là lý do các thế hệ sau thay tầng FC bằng *global average pooling*.

**Inception — vì sao  $1 \times 1$  là nút thắt hiệu quả.** Giả sử cần một conv  $3 \times 3$  từ 256 kênh sang 256 kênh. Chi phí trực tiếp là  $256 \cdot 256 \cdot 9 = 589\,824$  phép nhân-cộng cho mỗi vị trí không gian. Bây giờ chèn hai tầng  $1 \times 1$ : giảm  $256 \rightarrow 64$ , làm  $3 \times 3$  trong không gian 64 kênh, rồi mở lại  $64 \rightarrow 256$ . Tổng chi phí là  $256 \cdot 64 + 64 \cdot 64 \cdot 9 + 64 \cdot 256 = 16\,384 + 36\,864 + 16\,384 = 69\,632$ , tức rẻ hơn 8,5 lần. Ý nghĩa của phép tính: phần đắt nhất của conv là tương tác giữa *mọi* kênh vào với *mọi* kênh ra, nhân với  $k^2$ ; nếu ta làm phép tính không gian ở một không gian kênh hẹp thì cái  $k^2$  chỉ nhân với một số nhỏ. Đây cũng chính là khối *bottleneck* của ResNet, và là ý tưởng nền của phần lớn kiến trúc hiệu quả về sau.

**conv  $3 \times 3$  thường: mọi kênh vào nói chuyện với mọi kênh ra, ở mọi vị trí**



**depthwise separable: tách phần không gian khỏi phần kênh**



Tổng = 67 840, tức  $\frac{1}{C_{\text{out}}} + \frac{1}{k^2} = \frac{1}{256} + \frac{1}{9} \approx 0,115$  lần chi phí gốc — rẻ hơn khoảng 8,7 lần.

Nhưng bước depthwise đọc mỗi phần tử để làm vòn vẹn 9 phép nhân, nên cường độ số học của nó tụt xuống mức nghen bằng thông.

**Hình 2.4.** Vì sao depthwise separable rẻ, và vì sao “rẻ” theo FLOPs không có nghĩa là “nhanh”. Conv thường trả tiền cho tích của ba thứ cùng lúc: kênh vào, kênh ra, và diện tích của số. Phép tách chia nó thành hai bước, mỗi bước chỉ trả cho *hai* trong ba thứ đó — đây là toàn bộ nguồn gốc của tỉ số  $1/C_{\text{out}} + 1/k^2$ . Cái giá nằm ở dòng cuối: bước depthwise có rất ít phép tính trên mỗi byte đọc vào, nên nó rơi vào phía nghen bộ nhớ và thời gian thực tế không giảm theo tỉ lệ FLOPs.

Hình 2.4 bày ra cả hai vế của phép đổi trên cùng một sơ đồ, và đó là lý do nó đáng vẽ: con số 8,7 và lời cảnh báo về độ trễ luôn phải đọc cùng nhau.

**MobileNet — và vì sao FLOPs không phải độ trễ.** Depthwise separable tách một conv thành hai: một conv  $k \times k$  chạy độc lập trên từng kênh, rồi một conv  $1 \times 1$  trộn kênh. Tỉ số chi phí so với conv thường là  $\frac{1}{C_{\text{out}}} + \frac{1}{k^2}$ ; với  $k = 3$  và  $C_{\text{out}} = 256$  thì bằng  $0,0039 + 0,111 \approx 0,115$ , tức rẻ hơn khoảng 8,7 lần. Nhưng độ trễ thực đo được hầu như không bao giờ giảm 8,7 lần, và lý do là thứ bạn đã gặp khi tối ưu CUDA: phần depthwise đọc mỗi phần tử đầu vào để làm vòn vẹn  $k^2 = 9$  phép nhân, nên **cường độ số học** (*arithmetic intensity*) của nó cực thấp và nó rơi vào chế độ nghen bằng thông bộ nhớ. Giảm FLOPs của một kernel vốn đã nghen ở bộ nhớ thì gần như không giảm thời gian. Đây là ví dụ rõ nhất trong cả cây về việc một chỉ số đại diện — FLOPs — đo sai thứ mà ta quan tâm.

↪ **ràng buộc bởi B/11** — mô hình roofline nói chính xác khi nào giảm FLOPs không giảm thời gian: khi kernel đã nằm ở phía nghen bằng thông

#### D. ResNet: nút thắt là tối ưu hoá, không phải sức chứa

Đóng góp của ResNet hay bị kể sai thành “mạng sâu hơn thì mạnh hơn”. Bằng chứng gốc nói điều khác và mạnh hơn nhiều: một mạng phẳng 56 tầng có *sai số huấn luyện* cao hơn một mạng phẳng 20 tầng [30]. Sai số huấn luyện, không phải sai số kiểm thử. Nên đây không phải overfitting, và cũng không phải thiếu sức chứa. Mạng sâu hơn về nguyên tắc biểu diễn được mọi thứ mạng

nông biểu diễn được: chỉ cần các tầng thừa làm ánh xạ đồng nhất. Vấn đề là gradient descent không tìm ra nghiệm đó.

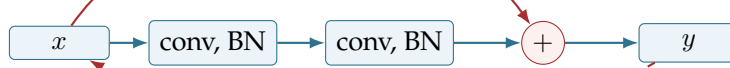
Kết nối tắt  $y = x + F(x)$  làm hai việc cùng lúc, và cả hai đều đã xuất hiện ở chương 12. Về mặt biểu diễn, ánh xạ đồng nhất trở thành phương án *mặc định* — chỉ cần  $F \approx 0$ , điều mà khởi tạo nhỏ cho sẵn. Về mặt gradient, Jacobian của khối là  $I + \partial F / \partial x$ , nên tích các Jacobian qua nhiều khối luôn còn một thành phần bằng 1 không thể triệt tiêu; đây chính xác là thành viên thứ tư của họ kỹ thuật “giữ  $\rho$  gần 1” mà mục C/12/02 đã mở ngoặc để dành.

### Khối phẳng



$\frac{\partial y}{\partial x} = J_2 J_1$  — mọi thừa số đều có thể nhỏ hơn 1  
đường tắt:  $x$  đi thẳng, không qua phép toán nào

### Khối residual



$\frac{\partial y}{\partial x} = I + \frac{\partial F}{\partial x}$  — số hạng  $I$  không bao giờ triệt tiêu

Hệ quả kép: ánh xạ đồng nhất trở thành phương án *mặc định* (chỉ cần  $F \approx 0$ ), và gradient luôn có một đường về không bị nhân dồn.

**Hình 2.5.** Kết nối tắt sửa vấn đề gì, nhìn từ phía lượt ngược. Ở khối phẳng, đạo hàm của đầu ra theo đầu vào là một tích thuần tuý, nên nếu mỗi thừa số hơi nhỏ hơn 1 thì sau vài chục khối nó tắt — đúng cơ chế ở C/12/02. Ở khối residual, đường cộng thêm một số hạng  $I$  vào Jacobian, và số hạng đó không phụ thuộc trọng số nên không thể bị nhân cho tiêu biến. Đây là lý do bằng chứng gốc của ResNet là sai số *huấn luyện* chứ không phải sai số kiểm thử: thứ được sửa là khả năng tối ưu hoá, không phải sức chứa.

Hình 2.5 cũng cho thấy cái giá đã nhắc ở bảng: mũi tên đường tắt buộc tensor  $x$  phải sống tới tận điểm cộng, nên bộ nhớ kích hoạt của một mạng residual cao hơn một mạng phẳng cùng độ sâu.

Cái giá thì cụ thể: nhánh tắt buộc giữ tensor  $x$  sống tới tận chỗ cộng, nên bộ nhớ kích hoạt tăng so với một mạng phẳng cùng độ sâu.

↔ cùng nguyên lý với C/12/02 — kết nối tắt, hàm kích hoạt, khởi tạo và chuẩn hoá đều nhắm vào một đại lượng: hệ số nhân của tích các Jacobian

## E. Các biến thể convolution và mục đích riêng

Đây là một bộ sưu tập chứ không phải một chuỗi, nên đọc theo kiểu tra cứu. Điểm chung: mỗi biến thể *nói lỏng hoặc siết chặt* đúng một trong ba giả định gốc ở mục A.

- **Depthwise separable** — tách tương tác không gian khỏi tương tác kênh.
  - **Được:** chi phí giảm khoảng  $k^2$  lần, như phép tính ở mục C.
  - **Mất:** cường độ số học tụt nên độ trễ thật giảm ít hơn nhiều; và giả định “không gian và kênh tách rời được” không phải lúc nào cũng đúng.
- **Group convolution** — chia kênh thành  $g$  nhóm, mỗi nhóm chỉ nói chuyện trong nội bộ. Ra đời từ ràng buộc bộ nhớ của AlexNet trên hai GPU, sau đó thành công cụ giảm tham số. **Mất:** thông tin không đi qua ranh giới nhóm, nên cần một phép trộn kênh (ví dụ shuffle hoặc  $1 \times 1$ ) để bù.

- **Dilated convolution** [36] — giãn khoảng cách giữa các điểm lấy mẫu trong cửa sổ, nên trường tiếp nhận tăng theo cấp số nhân mà không cần giảm độ phân giải. Sinh ra cho segmentation, nơi giảm độ phân giải là mất mát trực tiếp. *Mất*: hiện tượng lưới — các ô lấy mẫu không phủ kín nên xuất hiện hoa văn răng lược nếu xếp chồng nhiều tầng cùng hệ số giãn.
- **Deformable convolution** [37] — học luôn cả *vị trí* lấy mẫu thay vì cố định trên lưới, tức nói lỏng giả định cục bộ theo một cách có điều khiển. *Mất*: truy cập bộ nhớ không đều nên rất khó tối ưu trên GPU, và là một trong những lý do phổ biến khiến một mô hình không xuất được sang runtime suy luận.
- **Transposed convolution** — tăng độ phân giải ở nhánh giải mã. *Mất*: hiện tượng bàn cờ khi kích thước kernel không chia hết cho stride; nhiều cài đặt hiện đại thay nó bằng *upsample rồi conv thường* để tránh hẳn.

## F. ConvNeXt và bài học đắt nhất của cả mục

Khi ViT vượt CNN trên ImageNet, cách đọc tự nhiên là “*attention* tốt hơn convolution”. ConvNeXt kiểm chứng cách đọc đó bằng một thí nghiệm có kỷ luật. Nhóm tác giả giữ nguyên convolution, rồi lần lượt áp từng thành phần của recipe hiện đại và đo sau mỗi bước [35]. Danh sách gồm lịch học dài hơn, AdamW, augmentation mạnh, stochastic depth, LayerNorm thay Batch-Norm, kernel lớn hơn, và tỉ lệ tăng theo kiểu transformer. Kết quả là một CNN thuần bắt kịp ViT cùng hạng.

Kết luận đúng không phải “CNN thắng” mà là: *phần lớn khoảng cách được gán cho kiến trúc thực ra thuộc về công thức huấn luyện*. Điều này thay đổi cách bạn đọc mọi paper kiến trúc về sau. Câu hỏi đầu tiên khi thấy một bảng so sánh không còn là “kiến trúc nào cao hơn”. Nó thành “hai dòng này có cùng recipe không, và nếu không thì ai được tune kỹ hơn”. Đây cũng là lý do chương E/24-thi-nghiem-va-ablation tồn tại.

↪ **hệ quả của E/24** — một bảng so sánh chỉ có nghĩa khi hai dòng được huấn luyện bằng cùng công thức và được tune ngang nhau

## G. Giới hạn và trường hợp hồng

Ba giới hạn đáng mang theo. Thứ nhất, *FLOPs không phải độ trễ, và số tham số cũng không*. Ba đại lượng này tương quan yếu với nhau và với thời gian chạy thật; MobileNet và DenseNet là hai ví dụ ngược chiều nhau về đúng điểm này. Bất kỳ lựa chọn backbone nào cũng phải đo trên đúng phần cứng và đúng chế độ batch mà bạn sẽ triển khai.

Thứ hai, *độ chính xác ImageNet là một chỉ số đại diện, không phải mục tiêu*. Tương quan giữa nó và chất lượng ở tác vụ khác là có thật, nhưng không chặt. Tương quan đó yếu đi rõ rệt với các tác vụ dự đoán dày như segmentation. Ở đó, độ phân giải và đặc trưng đa tỉ lệ quan trọng hơn khả năng phân loại toàn ảnh.

Thứ ba, các con số trong mục này thuộc hai loại khác nhau. Các phép tính tham số và FLOPs ở mục C là số học thuần túy nên chính xác. Ngược lại, các phát biểu kiểu “rẻ hơn 8,7 lần nhưng độ trễ giảm ít hơn nhiều” là mô tả định tính của một hiện tượng đã được quan sát rộng rãi; mức giảm cụ thể phụ thuộc GPU, thư viện và kích thước tensor tới mức tôi không nêu một con số nào ở đây.

### CẠM BẤY

Cạm bẫy phổ biến nhất khi chọn backbone là so sánh hai mô hình được huấn luyện bằng hai recipe khác nhau rồi kết luận về kiến trúc. Một ResNet-50 theo recipe 2015 và một ResNet-50 theo recipe 2022 chênh nhau vài điểm phần trăm trên ImageNet. Khoảng cách



đó lớn hơn khoảng cách giữa nhiều cặp kiến trúc mà người ta viết hẳn một paper để so. Trước khi tin bất kỳ bảng nào, hãy kiểm tra ba thứ: số epoch, bộ augmentation, và optimizer cùng lịch học.

#### CẦN NHỚ

Mỗi bước trong dòng CNN gỡ đúng một nút thắt và đẩy độ khó sang một trục khác: tham số  $\rightarrow$  khả năng tối ưu hoá  $\rightarrow$  chi phí tính toán  $\rightarrow$  độ trễ trên phần cứng thật  $\rightarrow$  công thức huấn luyện. Khi cầm một backbone mới, hỏi nó đang đứng ở nút nào trong chuỗi đó — câu trả lời cho bạn biết nó sẽ hỏng ở đâu. [T2]

## H. Kết nối

Mục này giả định toàn bộ [C/12-co-che-huan-luyen](#), và trả lại cho chương đó thành viên thứ tư của họ “giữ tích Jacobian gần 1”. Nó đi thẳng vào [C/13/02-tu-chuoi-toi-attention](#), nơi câu hỏi “bao nhiêu giả định là đủ” được đặt cho dữ liệu có thứ tự. Rồi vào [C/13/03-vit-va-inductive-bias](#), nơi ba giả định ở mục A bị gỡ bỏ có chủ đích để xem chuyện gì xảy ra. Phần FLOPs và cường độ số học nối trực tiếp sang [B/11-gioi-han-tinh-toan](#) và [D/20-toi-uu-va-nen-mo-hinh](#); còn bài học ConvNeXt là tiền đề của [E/24-thi-nghiem-va-ablation](#). Nếu bạn quan tâm việc lắp backbone vào một hệ thống thật, [C/14-bai-toan-cv-loi](#) là chương nói về phần neck và head gắn lên nó.

### Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Ba giả định mà convolution cài sẵn là gì, và mỗi biến thể ở mục E nới lỏng giả định nào?
2. Vì sao xếp chồng hai tầng  $3 \times 3$  tốt hơn một tầng  $5 \times 5$  ở cả ba mặt, và trong tình huống nào nó không tốt hơn?
3. Bằng chứng gốc của ResNet là sai số *huấn luyện* chứ không phải kiểm thử. Vì sao chi tiết đó thay đổi hoàn toàn kết luận rút ra?
4. MobileNet giảm FLOPs khoảng 8,7 lần nhưng độ trễ giảm ít hơn nhiều — giải thích bằng cường độ số học thì thế nào, và phép đo nào xác nhận chẩn đoán đó?
5. Nếu ConvNeXt đúng khi nói recipe quan trọng hơn kiến trúc, điều đó thay đổi thứ tự bạn đọc một paper kiến trúc thế nào?
6. Deformable convolution mạnh hơn conv thường ở đâu, và vì sao nó thường bị loại khi triển khai lên thiết bị nhúng?

- [1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [2] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [3] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *AISTATS*, 2010.
- [4] K. He, X. Zhang, S. Ren, et al., “Delving Deep into Rectifiers,” in *ICCV*, 2015.
- [5] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [6] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [7] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [8] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [9] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [10] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [11] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” in *ICLR*, 2017.
- [12] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [13] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [14] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [15] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [16] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [17] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [18] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [19] G. Huang, Y. Sun, Z. Liu, et al., “Deep Networks with Stochastic Depth,” in *ECCV*, 2016.
- [20] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [21] H. Zhang, M. Cisse, Y. N. Dauphin, et al., “mixup: Beyond Empirical Risk Minimization,” in *ICLR*, 2018.
- [22] S. Yun, D. Han, S. J. Oh, et al., “CutMix: Regularization Strategy to Train Strong Classifiers,” in *ICCV*, 2019.
- [23] E. D. Cubuk, B. Zoph, J. Shlens, et al., “RandAugment: Practical Automated Data Augmentation,” in *CVPR Workshops*, 2020.

- [24] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [25] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [26] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *NeurIPS*, 2012.
- [27] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in *ICLR*, 2015.
- [28] C. Szegedy, W. Liu, et al., “Going Deeper with Convolutions,” in *CVPR*, 2015.
- [29] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [30] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [31] G. Huang, Z. Liu, L. van der Maaten, et al., “Densely Connected Convolutional Networks,” in *CVPR*, 2017.
- [32] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [33] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [34] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [35] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [36] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [37] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.
- [38] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [39] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [40] J. Su, M. Ahmed, Y. Lu, et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, vol. 568, 2024.
- [41] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” *arXiv:2312.00752*, 2023.
- [42] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [43] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.
- [44] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [45] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [46] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [47] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [48] R. Girshick, J. Donahue, T. Darrell, et al., “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation,” in *CVPR*, 2014.

- [49] R. Girshick, "Fast R-CNN," in *ICCV*, 2015.
- [50] S. Ren, K. He, R. Girshick, et al., "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in *NeurIPS*, 2015.
- [51] T.-Y. Lin, P. Dollár, R. Girshick, et al., "Feature Pyramid Networks for Object Detection," in *CVPR*, 2017.
- [52] K. He, G. Gkioxari, P. Dollár, et al., "Mask R-CNN," in *ICCV*, 2017.
- [53] D. Bolya, S. Foley, J. Hays, et al., "TIDE: A General Toolbox for Identifying Object Detection Errors," in *ECCV*, 2020.
- [54] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [55] J. Redmon, S. Divvala, R. Girshick, et al., "You Only Look Once: Unified, Real-Time Object Detection," in *CVPR*, 2016.
- [56] W. Liu, D. Anguelov, D. Erhan, et al., "SSD: Single Shot MultiBox Detector," in *ECCV*, 2016.
- [57] T.-Y. Lin, P. Goyal, R. Girshick, et al., "Focal Loss for Dense Object Detection," in *ICCV*, 2017.
- [58] Z. Tian, C. Shen, H. Chen, et al., "FCOS: Fully Convolutional One-Stage Object Detection," in *ICCV*, 2019.
- [59] X. Zhou, D. Wang, and P. Krähenbühl, "Objects as Points," *arXiv:1904.07850*, 2019.
- [60] S. Zhang, C. Chi, Y. Yao, et al., "Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection," in *CVPR*, 2020.
- [61] Z. Ge, S. Liu, Z. Li, et al., "OTA: Optimal Transport Assignment for Object Detection," in *CVPR*, 2021.
- [62] N. Bodla, B. Singh, R. Chellappa, et al., "Soft-NMS: Improving Object Detection with One Line of Code," in *ICCV*, 2017.
- [63] X. Zhu, W. Su, L. Lu, et al., "Deformable DETR: Deformable Transformers for End-to-End Object Detection," in *ICLR*, 2021.
- [64] H. Zhang, F. Li, S. Liu, et al., "DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection," in *ICLR*, 2023.
- [65] J. Long, E. Shelhamer, and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," in *CVPR*, 2015.
- [66] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *MICCAI*, 2015.
- [67] L.-C. Chen, G. Papandreou, I. Kokkinos, et al., "DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs," *IEEE TPAMI*, vol. 40, no. 4, pp. 834–848, 2018.
- [68] E. Xie, W. Wang, Z. Yu, et al., "SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers," in *NeurIPS*, 2021.
- [69] B. Cheng, I. Misra, A. G. Schwing, et al., "Masked-Attention Mask Transformer for Universal Image Segmentation," in *CVPR*, 2022.
- [70] A. Kirillov, K. He, R. Girshick, et al., "Panoptic Segmentation," in *CVPR*, 2019.
- [71] A. Kirillov, E. Mintun, N. Ravi, et al., "Segment Anything," in *ICCV*, 2023.
- [72] K. Sun, B. Xiao, D. Liu, et al., "Deep High-Resolution Representation Learning for Human Pose Estimation," in *CVPR*, 2019.

- [73] Z. Cao, T. Simon, S.-E. Wei, et al., “Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields,” in *CVPR*, 2017.
- [74] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [75] J. Deng, J. Guo, N. Xue, et al., “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *CVPR*, 2019.
- [76] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [77] K. Xu, J. Ba, R. Kiros, et al., “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention,” in *ICML*, 2015.
- [78] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *ICLR*, 2015.
- [79] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [80] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [81] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [82] K. Simonyan and A. Zisserman, “Two-Stream Convolutional Networks for Action Recognition in Videos,” in *NeurIPS*, 2014.
- [83] J. Carreira and A. Zisserman, “Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset,” in *CVPR*, 2017.
- [84] D. Tran, H. Wang, L. Torresani, et al., “A Closer Look at Spatiotemporal Convolutions for Action Recognition,” in *CVPR*, 2018.
- [85] C. Feichtenhofer, H. Fan, J. Malik, et al., “SlowFast Networks for Video Recognition,” in *ICCV*, 2019.
- [86] A. Dosovitskiy, P. Fischer, E. Ilg, et al., “FlowNet: Learning Optical Flow with Convolutional Networks,” in *ICCV*, 2015.
- [87] D. Sun, X. Yang, M.-Y. Liu, et al., “PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume,” in *CVPR*, 2018.
- [88] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.
- [89] A. Bewley, Z. Ge, L. Ott, et al., “Simple Online and Realtime Tracking,” in *ICIP*, 2016.
- [90] Y. Zhang, P. Sun, Y. Jiang, et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *ECCV*, 2022.
- [91] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [92] K. Bernardin and R. Stiefelwagen, “Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics,” *EURASIP J. Image and Video Processing*, 2008.
- [93] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [94] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [95] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.

- [96] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *CVPR*, 2024.
- [97] D. Eigen, C. Puhrsch, and R. Fergus, “Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network,” in *NeurIPS*, 2014.
- [98] R. Ranftl, K. Lasinger, D. Hafner, et al., “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer,” *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [99] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IROS*, 2012.
- [100] C. R. Qi, H. Su, K. Mo, et al., “PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation,” in *CVPR*, 2017.
- [101] C. R. Qi, L. Yi, H. Su, et al., “PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space,” in *NeurIPS*, 2017.
- [102] A. H. Lang, S. Vora, H. Caesar, et al., “PointPillars: Fast Encoders for Object Detection from Point Clouds,” in *CVPR*, 2019.
- [103] T. Müller, A. Evans, C. Schied, et al., “Instant Neural Graphics Primitives with a Multiresolution Hash Encoding,” *ACM TOG*, vol. 41, no. 4, 2022.
- [104] B. Kerbl, G. Kopanas, T. Leimkühler, et al., “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *ACM TOG*, vol. 42, no. 4, 2023.
- [105] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.
- [106] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” in *ICLR*, 2014.
- [107] I. Goodfellow, J. Pouget-Abadie, M. Mirza, et al., “Generative Adversarial Nets,” in *NeurIPS*, 2014.
- [108] T. Karras, S. Laine, and T. Aila, “A Style-Based Generator Architecture for Generative Adversarial Networks,” in *CVPR*, 2019.
- [109] J. Ho, A. Jain, and P. Abbeel, “Denoising Diffusion Probabilistic Models,” in *NeurIPS*, 2020.
- [110] Y. Song, J. Sohl-Dickstein, D. P. Kingma, et al., “Score-Based Generative Modeling Through Stochastic Differential Equations,” in *ICLR*, 2021.
- [111] J. Johnson, A. Alahi, and L. Fei-Fei, “Perceptual Losses for Real-Time Style Transfer and Super-Resolution,” in *ECCV*, 2016.
- [112] P. Isola, J.-Y. Zhu, T. Zhou, et al., “Image-to-Image Translation with Conditional Adversarial Networks,” in *CVPR*, 2017.
- [113] L. A. Gatys, A. S. Ecker, and M. Bethge, “Image Style Transfer Using Convolutional Neural Networks,” in *CVPR*, 2016.
- [114] X. Huang and S. Belongie, “Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization,” in *ICCV*, 2017.
- [115] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.
- [116] J. Song, C. Meng, and S. Ermon, “Denoising Diffusion Implicit Models,” in *ICLR*, 2021.
- [117] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, et al., “Flow Matching for Generative Modeling,” in *ICLR*, 2023.
- [118] J. Ho and T. Salimans, “Classifier-Free Diffusion Guidance,” *arXiv:2207.12598*, 2022.
- [119] R. Rombach, A. Blattmann, D. Lorenz, et al., “High-Resolution Image Synthesis with Latent Diffusion Models,” in *CVPR*, 2022.
- [120] W. Peebles and S. Xie, “Scalable Diffusion Models with Transformers,” in *ICCV*, 2023.



- [121] L. Zhang, A. Rao, and M. Agrawala, “Adding Conditional Control to Text-to-Image Diffusion Models,” in *ICCV*, 2023.
- [122] Y. Blau and T. Michaeli, “The Perception-Distortion Tradeoff,” in *CVPR*, 2018.
- [123] C. Ledig, L. Theis, F. Huszár, et al., “Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network,” in *CVPR*, 2017.
- [124] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., “Image Quality Assessment: From Error Visibility to Structural Similarity,” *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [125] M. Heusel, H. Ramsauer, T. Unterthiner, et al., “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium,” in *NeurIPS*, 2017.
- [126] T. Brooks, B. Mildenhall, T. Xue, et al., “Unprocessing Images for Learned Raw Denoising,” in *CVPR*, 2019.
- [127] A. Abdelhamed, S. Lin, and M. S. Brown, “A High-Quality Denoising Dataset for Smartphone Cameras,” in *CVPR*, 2018.
- [128] J. Deng, W. Dong, R. Socher, et al., “ImageNet: A Large-Scale Hierarchical Image Database,” in *CVPR*, 2009.
- [129] A. Radford, J. W. Kim, C. Hallacy, et al., “Learning Transferable Visual Models from Natural Language Supervision,” in *ICML*, 2021.
- [130] J. Li, D. Li, S. Savarese, et al., “BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models,” in *ICML*, 2023.
- [131] H. Liu, C. Li, Q. Wu, et al., “Visual Instruction Tuning,” in *NeurIPS*, 2023.
- [132] S. Kornblith, J. Shlens, and Q. V. Le, “Do Better ImageNet Models Transfer Better?” In *CVPR*, 2019.
- [133] T. Chen, S. Kornblith, M. Norouzi, et al., “A Simple Framework for Contrastive Learning of Visual Representations,” in *ICML*, 2020.
- [134] K. He, H. Fan, Y. Wu, et al., “Momentum Contrast for Unsupervised Visual Representation Learning,” in *CVPR*, 2020.
- [135] J.-B. Grill, F. Strub, F. Altché, et al., “Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning,” in *NeurIPS*, 2020.
- [136] M. Caron, H. Touvron, I. Misra, et al., “Emerging Properties in Self-Supervised Vision Transformers,” in *ICCV*, 2021.
- [137] K. He, X. Chen, S. Xie, et al., “Masked Autoencoders Are Scalable Vision Learners,” in *CVPR*, 2022.
- [138] A. van den Oord, Y. Li, and O. Vinyals, “Representation Learning with Contrastive Predictive Coding,” in *arXiv:1807.03748*, 2018.
- [139] J. Kaplan, S. McCandlish, T. Henighan, et al., “Scaling Laws for Neural Language Models,” *arXiv:2001.08361*, 2020.
- [140] X. Zhai, A. Kolesnikov, N. Houlsby, et al., “Scaling Vision Transformers,” in *CVPR*, 2022.
- [141] J. Hoffmann, S. Borgeaud, A. Mensch, et al., “Training Compute-Optimal Large Language Models,” in *NeurIPS*, 2022.
- [142] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [143] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [144] N. Houlsby, A. Giurgiu, S. Jastrzebski, et al., “Parameter-Efficient Transfer Learning for NLP,” in *ICML*, 2019.

- [145] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [146] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *RecSys*, 2019.

## 2.3 Từ chuỗi tới attention (From Sequences to Attention)

**PHỤ THUỘC** C/12/02-lam-cho-training-chay-duoc (tích các Jacobian — vanishing gradient ở đây là đúng hiện tượng đó, chỉ trải theo thời gian); C/13/01-dong-cnn (khái niệm trường tiếp nhận và inductive bias).

### TÓM TẮT

Mục này đi từ mạng hồi quy tới **attention** và tới các mô hình không gian trạng thái, đọc như một chuỗi gỡ nút thắt chứ không như một danh sách kiến trúc. Đọc xong bạn giải thích được vì sao **transformer** bỏ được tính tuần tự, và vì sao nó phải trả bằng chi phí bậc hai. Bạn cũng nói được ba hướng tấn công chi phí đó khác nhau ở tầng nào: toán, kiến trúc, hay hệ thống. Nếu chỉ nhớ một điều: LSTM và ResNet giải cùng một bài toán bằng cùng một mẹo. Cả hai tạo một *đường cộng* để gradient đi qua mà không bị nhân dồn. Nhận ra điều đó thì hai kiến trúc trông rất khác nhau trở thành một ý tưởng duy nhất.

### KIẾN THỨC CẦN CÓ TRƯỚC

- **chuỗi (sequence)** — dữ liệu có thứ tự, ví dụ một câu gồm nhiều từ hoặc một video gồm nhiều khung hình; “thứ tự là thông tin” nghĩa là hoán vị các phần tử sẽ đổi ý nghĩa.
- **token** — một phần tử của chuỗi sau khi đã được cắt và mã hoá thành vectơ số — một từ, một mảnh từ, hoặc với ảnh là một ô vuông  $16 \times 16$  pixel.
- **embedding** — vectơ số biểu diễn một token; các token có nghĩa gần nhau được đặt gần nhau trong không gian này, và toàn bộ mô hình chỉ làm việc với vectơ chứ không với ký tự.
- **trạng thái ẩn (hidden state)** — vectơ mà mạng hồi quy mang theo từ bước thời gian này sang bước sau; nó là toàn bộ “trí nhớ” của mạng, nên mọi thứ không nhét được vào nó là mất.
- **BPTT (backpropagation through time)** — cách áp lan truyền ngược cho mạng hồi quy: trải mạng ra thành  $T$  bản sao theo thời gian rồi lan truyền ngược qua cả  $T$  bước, nên một chuỗi dài 100 hành xử như một mạng sâu 100 tầng.
- **attention** — cơ chế cho mỗi vị trí tự chọn lấy thông tin từ mọi vị trí khác bằng một trọng số học được, thay vì phải chờ thông tin truyền dần qua từng bước.
- **độ phức tạp bậc hai** — chi phí tỉ lệ với  $T^2$  khi độ dài chuỗi là  $T$ ; gấp đôi độ dài thì chi phí gấp bốn, nên đây là rào cản chính khi mang attention sang ảnh độ phân giải cao.
- **HBM** — bộ nhớ băng thông cao gắn ngoài chip GPU — rất lớn nhưng chậm hơn nhiều so với bộ nhớ trên chip; số lần đọc ghi HBM thường quyết định tốc độ hơn cả số phép nhân.

### A. Đặt vấn đề

Convolution giả định dữ liệu nằm trên một lưới đều và quan hệ quan trọng là quan hệ gần. Nhưng nhiều dữ liệu có *thứ tự* chứ không có lưới, và quan hệ quan trọng có thể ở rất xa: từ đầu câu quyết định nghĩa từ cuối câu, khung hình đầu video quyết định cách hiểu khung hình cuối. Câu hỏi của mục này là: cấu trúc nào xử lý được loại quan hệ đó, và mỗi câu trả lời phải trả giá bằng gì?

Giá trị cụ thể của mục: nhìn một kiến trúc xử lý chuỗi, bạn trả lời được ba câu. Thông tin đi từ vị trí  $i$  tới vị trí  $j$  qua bao nhiêu bước? Chi phí tăng theo độ dài như thế nào? Và cái gì trong kiến trúc đang giữ vai trò “biết thứ tự”? Ba câu đó đủ để phân loại mọi biến thể mà bạn sẽ gặp.

### TRỰC GIÁC

Hãy đo mọi kiến trúc trong mục này bằng một đại lượng duy nhất: *độ dài đường đi của thông tin* giữa hai vị trí xa nhau. Trong mạng hồi quy, thông tin phải đi qua  $T$  trạm nối tiếp, và mỗi trạm nhân nó với một ma trận. Giống như truyền một tin nhắn qua dây chuyền thì thăm dài 100 người. Trong attention, mọi vị trí nói chuyện trực tiếp với mọi vị trí, đường đi bằng 1 — giống như thay dây chuyền thì thăm bằng một phòng họp mà ai cũng nghe thấy ai. Toàn bộ ưu điểm lẫn toàn bộ chi phí của transformer nằm gọn trong phép đổi đó: phòng họp không tam sao thất bản, nhưng số cặp người phải nghe nhau tăng theo bình phương số.

## B. Mạch từ RNN tới state space model

Sáu bước dưới đây là một chuỗi liên tục, và mỗi bước chỉ hiểu được khi đã thấy vấn đề mà bước trước để lại.

### I Mạch 2 — từ chuỗi tới attention

- 1 Vấn đề** dữ liệu có thứ tự, nhưng mạng dày đặc và convolution đều không có khái niệm “bước trước”.

**Giải pháp** **mạng hồi quy** (*recurrent neural network, RNN*): giữ một trạng thái ẩn  $h_t = f(Wh_{t-1} + Ux_t)$  và cập nhật nó theo từng bước, nên cùng một bộ trọng số dùng lại cho mọi độ dài chuỗi.

**Vấn đề mới** huấn luyện bằng BPTT nghĩa là lan truyền ngược qua  $T$  tầng dùng chung trọng số. Gradient vì thế tỉ lệ với lũy thừa  $T$  của cùng một ma trận. Đó đúng là hiện tượng vanishing và exploding của mục C/12/02, nhưng tệ hơn: thừa số lặp lại y hệt nhau nên không có triệt tiêu ngẫu nhiên nào cứu.
- 2 Vấn đề** trí nhớ dài không tồn tại: sau vài chục bước tín hiệu đã tắt.

**Giải pháp** **LSTM** [38] và GRU dùng **cổng** (*gate*) để tạo một trạng thái ô  $c_t = f_t \odot c_{t-1} + i_t \odot g_t$ .

**Hệ quả** đạo hàm  $\partial c_t / \partial c_{t-1} = f_t$ , nên khi cổng quên mở gần 1 thì gradient chảy qua mà không bị nhân với ma trận trọng số. Đây chính xác là mẹo của kết nối tắt trong ResNet, chỉ khác là hệ số của đường cộng ở đây học được thay vì cố định bằng 1.

**Vấn đề mới** tính tuần tự vẫn còn nguyên:  $h_t$  cần  $h_{t-1}$ , nên không song song hoá được theo thời gian, và độ dài dùng được vẫn giới hạn.
- 3 Vấn đề** tuần tự nghĩa là không tận dụng được GPU, và thông tin xa vẫn phải đi qua mọi trạm trung gian.

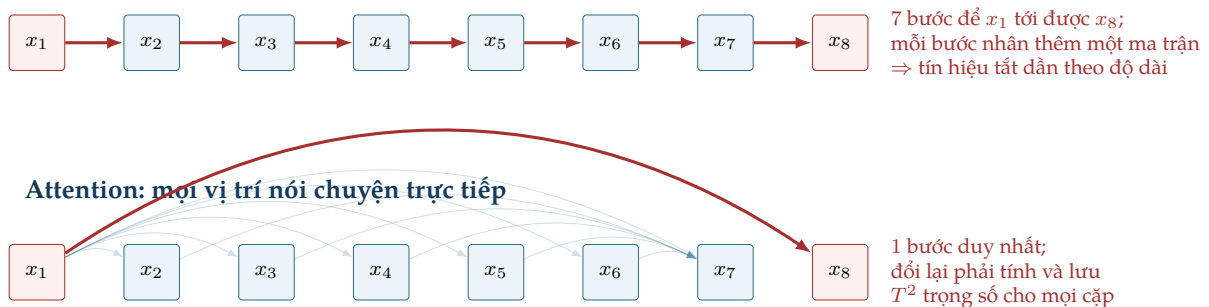
**Giải pháp** **attention**. Mỗi vị trí sinh một truy vấn  $q$ ; mọi vị trí sinh khoá  $k$  và giá trị  $v$ . Đầu ra là tổ hợp  $\text{softmax}(qK^\top / \sqrt{d})V$ , tức truy cập trực tiếp mọi vị trí trong một bước. **Transformer** [39] bỏ hẳn phần hồi quy và chỉ giữ attention cộng mạng truyền thẳng theo vị trí, nên toàn bộ chuỗi được xử lý song song.

**Giá phải trả** chi phí và bộ nhớ bậc hai theo độ dài, và mất hoàn toàn khái niệm thứ tự vì phép tính bất biến với hoán vị.
- 4 Vấn đề** mô hình không còn biết token nào đứng trước token nào.

- Giải pháp** **mã hoá vị trí** (*positional encoding*): cộng hoặc trộn thêm thông tin vị trí vào embedding.
- Giả định** vị trí có thể mã hoá thành một vectơ mà mô hình học cách dùng.
- Hệ quả** bốn họ với đánh đổi khác nhau — tuyệt đối dạng sin, tuyệt đối học được, tương đối, và RoPE quay embedding theo góc tỉ lệ vị trí [40]. Tuyệt đối đơn giản nhưng ngoại suy kém ra ngoài độ dài đã thấy khi huấn luyện; tương đối và RoPE ngoại suy tốt hơn vì chúng mã hoá *khoảng cách* chứ không mã hoá chỉ số. Với ảnh còn cần biến thể hai chiều, vì vị trí là một cặp  $(x, y)$  chứ không phải một số.
- 5 Vấn đề** chi phí bậc hai chặn đường tới ảnh độ phân giải cao.
- Giải pháp** ba hướng tấn công ở ba tầng khác nhau — xấp xỉ toán học, giới hạn phạm vi kiến trúc, và tối ưu truy cập bộ nhớ — xem Bảng 2.3.
- Hệ quả** đây là một bài tập rất tốt về việc *cùng một vấn đề có thể tấn công ở nhiều tầng trừu tượng*, và ba lời giải không loại trừ nhau: người ta dùng FlashAttention *bên trong* Swin.
- 6 Vấn đề** cả ba hướng trên đều vá chứ không xoá được bậc hai.
- Giải pháp** **mô hình không gian trạng thái** (*state space model, SSM*) như Mamba [41] quay lại dạng hồi quy tuyến tính, với tham số phụ thuộc đầu vào. Nó tính được bằng phép quét song song. Nhờ vậy chi phí tuyến tính theo độ dài mà vẫn huấn luyện song song được.
- Giá phải trả** đây là hướng đang lên, chưa phải kết luận: tính tới thời điểm viết, SSM chưa thay thế attention trong thị giác, và phần lớn kết quả mạnh nhất vẫn thuộc về các kiến trúc dựa trên attention.

↔ cùng nguyên lý với C/13/01 — cổng của LSTM và kết nối tắt của ResNet là cùng một mẹo — tạo một đường cộng để gradient không bị nhân dồn

### Mạng hồi quy: thông tin bò từng bước



**Hình 2.6.** Đại lượng đáng dùng để so mọi kiến trúc chuỗi: *độ dài đường đi của thông tin* giữa hai vị trí xa nhau. Ở mạng hồi quy nó bằng khoảng cách giữa hai vị trí, nên tín hiệu phải sống sót qua từng ấy phép nhân ma trận — đây chính là vanishing gradient của mục C/12/02, trải theo trục thời gian. Ở attention nó luôn bằng 1, và cái giá là các cạnh mờ trong hình: mọi cặp đều phải có một trọng số, nên số cạnh tăng theo  $T^2$ . LSTM nằm giữa hai thái cực: đường đi vẫn dài, nhưng cổng quên tạo một đường cộng nên tín hiệu không bị nhân dồn.

Hình 2.6 là dạng hình học của ẩn dụ ở đầu mục, và nó cũng cho thấy vì sao ba bước đầu của mạch phải đi theo đúng thứ tự đó.

### C. Vì sao chia cho $\sqrt{d}$ — một phép tính hai dòng

Chi tiết  $\sqrt{d}$  trong công thức attention trông như một hệ số làm đẹp. Nhưng bỏ nó đi thì mô hình không huấn luyện được, và lý do tính ra được. Giả sử các thành phần của  $q$  và  $k$  độc lập, trung bình 0, phương sai 1. Tích vô hướng  $q \cdot k = \sum_{i=1}^d q_i k_i$  là tổng của  $d$  số hạng độc lập, mỗi số hạng trung bình 0 và phương sai 1. Nên nó có phương sai  $d$ , tức độ lệch chuẩn  $\sqrt{d}$ . Với  $d = 64$ , độ lệch chuẩn của các logit là 8.

Ý nghĩa của con số đó: softmax trên một tập logit có độ lệch chuẩn 8 gần như luôn dồn toàn bộ khối lượng vào một phần tử, tức nó bão hoà thành một vectơ gần one-hot. Mà đạo hàm của softmax ở vùng bão hoà xấp xỉ 0 — đúng hiện tượng đã gặp với sigmoid ở C/12/02. Chia cho  $\sqrt{d}$  đưa độ lệch chuẩn về 1, giữ softmax ở vùng có gradient. Nói gọn:  $\sqrt{d}$  không phải chuẩn hoá cho đẹp, nó là điều kiện để attention có gradient khi  $d$  lớn.

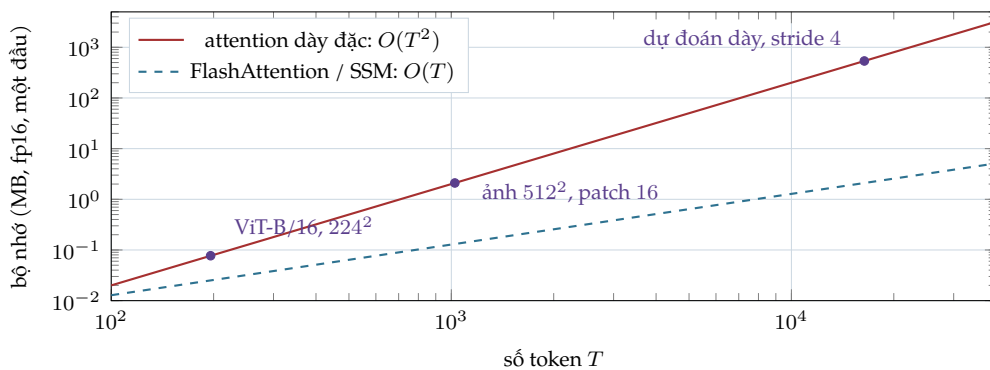
↪ trường hợp riêng của C/12/02 — đây đúng là hiện tượng bão hoà của sigmoid, chỉ khác là xảy ra ở softmax và nguyên nhân là thang của tích vô hướng

### D. Chi phí bậc hai: con số cụ thể trước, giải pháp sau

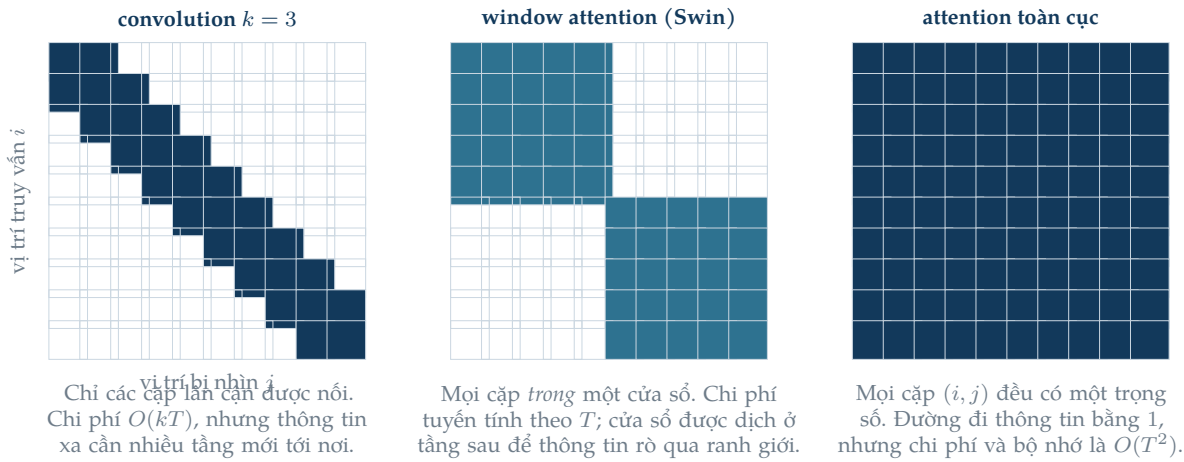
Trước khi bàn giải pháp, hãy làm phép tính để biết vấn đề nghiêm trọng tới mức nào. Ma trận attention có kích thước  $T \times T$  cho mỗi đầu và mỗi tầng.

- **Phân loại ảnh, ViT-B/16 ở  $224 \times 224$ :**  $T = (224/16)^2 = 196$  token, nên ma trận có  $196^2 = 38\,416$  phần tử — không đáng kể.
- **Ảnh  $512 \times 512$ , patch 16:**  $T = 1024$ , ma trận có hơn một triệu phần tử. Vẫn chạy được.
- **Dự đoán dài ở stride 4 trên ảnh  $512 \times 512$ :**  $T = 128^2 = 16\,384$ , ma trận có  $2,68 \cdot 10^8$  phần tử, tức khoảng 537 MB ở fp16 — cho một đầu, một tầng. Nhân với 12 đầu là 6,4 GB chỉ để chứa các trọng số attention của một tầng.

Con số cuối là toàn bộ lý do các biến thể ở Bảng 2.3 tồn tại, và cũng là lý do ViT phải không dùng trực tiếp được cho segmentation. Hình 2.7 vẽ cùng ba mốc đó trên trục log để thấy chỗ đường cong bẻ gãy ngân sách.



**Hình 2.7.** Bộ nhớ cần để chứa ma trận attention theo số token, thang log hai chiều. Ba chấm là ba chế độ dùng thật. Khoảng cách giữa hai đường là lý do tồn tại của cả ba hướng tấn công ở Bảng 2.3; lưu ý FlashAttention nằm trên đường tuyến tính về bộ nhớ nhưng vẫn làm đủ  $O(T^2)$  phép tính.



**Hình 2.8.** Ba kiến trúc, cùng một cách nhìn: ô  $(i, j)$  được tô nghĩa là vị trí  $i$  được phép lấy thông tin trực tiếp từ vị trí  $j$ . Convolution là một dải hẹp quanh đường chéo — đó chính là “tính cục bộ” phát biểu dưới dạng ma trận. Window attention là các khối trên đường chéo, tức một thỏa hiệp: dày đặc trong cửa sổ, rỗng ngoài cửa sổ. Attention toàn cục tô kín, và diện tích phần tô chính là chi phí phải trả. Nhìn ba lưới cạnh nhau thì “inductive bias” thôi không còn là một khái niệm trừu tượng: nó chính là các ô bị bỏ trắng vì kiến trúc quyết định trước rằng chúng không đáng quan tâm.

Hình 2.8 nên được xem trước khi đọc Bảng 2.3: hai trong ba hướng tấn công ở đó chính là hai cách khác nhau để *bớt tô* lưới bên phải, còn hướng thứ ba giữ nguyên lưới nhưng đổi cách nó được tính.

**Bảng 2.3.** Ba hướng tấn công chi phí bậc hai, ở ba tầng trừu tượng khác nhau. Chúng không loại trừ nhau và trên thực tế hay được dùng chồng lên nhau.

| Tầng tấn công | Đại diện                    | Cơ chế                                                                                    | Mất gì / hỏng khi nào                                                                                     |
|---------------|-----------------------------|-------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Toán học      | Linear attention            | Đổi thứ tự nhân ma trận nhờ một xấp xỉ của softmax, biến $O(T^2d)$ thành $O(Td^2)$        | Chất lượng giảm ở tác vụ cần attention “sắc”, tức cần chọn đúng một vị trí trong nhiều vị trí giống nhau  |
| Kiến trúc     | Swin, window attention [42] | Chỉ cho attention trong cửa sổ cố định, rồi dịch cửa sổ giữa các tầng để thông tin rò qua | Quan hệ xa cần nhiều tầng mới nối được; xuất hiện lại một dạng “đường đi dài” mà attention sinh ra để xóa |
| Hệ thống      | FlashAttention [43]         | Chia ô và tính softmax trực tuyến để không bao giờ dựng ma trận $T \times T$ trong HBM    | Không giảm FLOPs và không giảm độ phức tạp tiệm cận; lợi ích phụ thuộc kích thước SRAM của GPU cụ thể     |

Bảng 2.3 đáng đọc kỹ vì nó dạy một thói quen chung hơn cả nội dung: khi gặp một rào cản, hãy hỏi nó có thể bị tấn công ở tầng nào. FlashAttention là ví dụ sạch nhất — nó không sửa thuật toán, nó sửa *cách thuật toán chạm vào bộ nhớ*, và với người đã tối ưu kernel CUDA thì đây là câu chuyện quen thuộc: tăng cường độ số học bằng cách giữ dữ liệu trên chip và giảm số vòng đọc ghi ra bộ nhớ ngoài.

↔ **ràng buộc bởi** B/11 — FlashAttention chỉ có ý nghĩa trong mô hình roofline: nó dời kernel từ phía ngiên bằng thông sang phía ngiên tính toán

## E. Giới hạn và trường hợp hỏng

Attention không phải luôn thắng, và ba chỗ nó thua đáng nhớ.



Thứ nhất, với chuỗi ngắn và dữ liệu ít, mạng hồi quy vẫn cạnh tranh. Chi phí bậc hai chỉ là vấn đề khi  $T$  lớn; khi  $T$  nhỏ, phần chi phí thật nằm ở mạng truyền thẳng chứ không ở attention, còn transformer thì lại cần nhiều dữ liệu hơn vì nó cài ít giả định hơn.

Thứ hai, mã hoá vị trí là chỗ rò rỉ giả định. Mọi phép ngoại suy ra độ dài chưa từng thấy khi huấn luyện đều là ngoại suy, và mô hình có thể xuống cấp lạng lẽ. Với ảnh, đổi độ phân giải đầu vào mà không nội suy lại positional embedding là một lỗi phổ biến, cho ra kết quả tệ mà không báo lỗi.

Thứ ba, linear attention và window attention đánh đổi ở hai chỗ khác nhau, nên đừng coi chúng thay thế được nhau. Linear attention mất khả năng chọn sắc; window attention giữ được độ sắc trong cửa sổ nhưng mất quan hệ xa. Nếu tác vụ của bạn cần cả hai — ví dụ theo vết một đối tượng nhỏ qua toàn ảnh — thì cả hai đều không đủ và bạn phải quay lại kiến trúc phân cấp.

Về mức độ tin cậy: các con số bộ nhớ ở mục D là số học thuần túy nên chắc chắn. Còn phát biểu “SSM chưa thay thế attention trong thị giác” chỉ là đánh giá của tôi tính tới tháng 9/2026. Nó dựa trên việc các kết quả dần dần vẫn dùng attention, và có thể sai trong vòng một năm.

#### CẠM BẤY

Cạm bẫy thực hành hay gặp nhất khi mang một ViT sang bài toán của mình: đổi kích thước ảnh đầu vào mà quên nội suy positional embedding. Số token thay đổi, nhưng bảng vị trí đã học vẫn có kích thước cũ; tùy cài đặt mà bạn nhận một exception — trường hợp may mắn — hoặc một phép cắt/dệm âm thầm cho ra một mô hình hoạt động nhưng kém hẳn. Dấu hiệu nhận biết: điểm số tụt đúng một bậc ngay khi đổi độ phân giải, trong khi mọi thứ khác giữ nguyên.

#### CẦN NHỚ

Đo mọi kiến trúc chuỗi bằng ba câu: thông tin đi từ  $i$  tới  $j$  qua bao nhiêu bước, chi phí tăng theo  $T$  như thế nào, và cái gì đang giữ vai trò “biết thứ tự”. RNN cho đường đi  $O(T)$  và chi phí  $O(T)$ ; attention cho đường đi  $O(1)$  và chi phí  $O(T^2)$ ; mọi biến thể là một điểm nằm giữa hai thái cực đó. [T2]

## F. Kết nối

Bước hai của mạch — cổng tạo đường cộng — là cùng một ý tưởng với kết nối tắt ở [C/13/01-dong-cnn](#), và cả hai đều là hệ quả của lập luận tích Jacobian ở [C/12/02-lam-cho-training-chay-duoc](#); nếu chỉ nhớ một liên kết trong cả chương 13 thì nên nhớ liên kết này. Mục tiếp theo, [C/13/03-vit-va-inductive-bias](#), mang attention vào ảnh và hỏi cái giá của việc vứt bỏ tính cục bộ. Phần FlashAttention nối trực tiếp sang [B/11-gioi-han-tinh-toan](#) (cường độ số học, roofline) và sang [D/20-toi-uu-va-nen-mo-hinh](#). Còn attention với vai trò cơ chế lựa chọn trong bài toán thị giác cụ thể xuất hiện lại ở [C/14-bai-toan-cv-loi](#) qua DETR [44].

### ❏ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao vanishing gradient trong RNN tệ hơn trong một mạng truyền thẳng cùng độ sâu, dù cơ chế là như nhau?
2. Cổng của LSTM và kết nối tắt của ResNet giống nhau ở điểm cốt lõi nào, và khác nhau ở điểm nào?
3. Nếu bỏ hệ số  $1/\sqrt{d}$  trong attention thì hỏng ở đâu, và vì sao hậu quả nặng hơn khi  $d$  lớn?

4. FlashAttention không giảm FLOPs — vậy nó nhanh hơn nhờ đâu, và trên phần cứng nào lợi ích của nó nhỏ đi?
5. Ba hướng tấn công chi phí bậc hai nằm ở ba tầng khác nhau; với bài toán segmentation ảnh  $1024 \times 1024$  thì bạn chọn hướng nào trước, và vì sao?
6. Bạn fine-tune một ViT ở độ phân giải gấp đôi lúc pretrain và kết quả tệ đi. Nghi phạm số một là gì?

- [1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [2] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [3] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *AISTATS*, 2010.
- [4] K. He, X. Zhang, S. Ren, et al., “Delving Deep into Rectifiers,” in *ICCV*, 2015.
- [5] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [6] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [7] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [8] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [9] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [10] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [11] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” in *ICLR*, 2017.
- [12] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [13] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [14] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [15] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [16] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [17] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [18] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [19] G. Huang, Y. Sun, Z. Liu, et al., “Deep Networks with Stochastic Depth,” in *ECCV*, 2016.
- [20] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [21] H. Zhang, M. Cisse, Y. N. Dauphin, et al., “mixup: Beyond Empirical Risk Minimization,” in *ICLR*, 2018.
- [22] S. Yun, D. Han, S. J. Oh, et al., “CutMix: Regularization Strategy to Train Strong Classifiers,” in *ICCV*, 2019.
- [23] E. D. Cubuk, B. Zoph, J. Shlens, et al., “RandAugment: Practical Automated Data Augmentation,” in *CVPR Workshops*, 2020.

- [24] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [25] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [26] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *NeurIPS*, 2012.
- [27] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in *ICLR*, 2015.
- [28] C. Szegedy, W. Liu, et al., “Going Deeper with Convolutions,” in *CVPR*, 2015.
- [29] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [30] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [31] G. Huang, Z. Liu, L. van der Maaten, et al., “Densely Connected Convolutional Networks,” in *CVPR*, 2017.
- [32] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [33] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [34] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [35] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [36] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [37] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.
- [38] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [39] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [40] J. Su, M. Ahmed, Y. Lu, et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, vol. 568, 2024.
- [41] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” *arXiv:2312.00752*, 2023.
- [42] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [43] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.
- [44] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [45] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [46] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [47] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [48] R. Girshick, J. Donahue, T. Darrell, et al., “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation,” in *CVPR*, 2014.

- [49] R. Girshick, "Fast R-CNN," in *ICCV*, 2015.
- [50] S. Ren, K. He, R. Girshick, et al., "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in *NeurIPS*, 2015.
- [51] T.-Y. Lin, P. Dollár, R. Girshick, et al., "Feature Pyramid Networks for Object Detection," in *CVPR*, 2017.
- [52] K. He, G. Gkioxari, P. Dollár, et al., "Mask R-CNN," in *ICCV*, 2017.
- [53] D. Bolya, S. Foley, J. Hays, et al., "TIDE: A General Toolbox for Identifying Object Detection Errors," in *ECCV*, 2020.
- [54] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [55] J. Redmon, S. Divvala, R. Girshick, et al., "You Only Look Once: Unified, Real-Time Object Detection," in *CVPR*, 2016.
- [56] W. Liu, D. Anguelov, D. Erhan, et al., "SSD: Single Shot MultiBox Detector," in *ECCV*, 2016.
- [57] T.-Y. Lin, P. Goyal, R. Girshick, et al., "Focal Loss for Dense Object Detection," in *ICCV*, 2017.
- [58] Z. Tian, C. Shen, H. Chen, et al., "FCOS: Fully Convolutional One-Stage Object Detection," in *ICCV*, 2019.
- [59] X. Zhou, D. Wang, and P. Krähenbühl, "Objects as Points," *arXiv:1904.07850*, 2019.
- [60] S. Zhang, C. Chi, Y. Yao, et al., "Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection," in *CVPR*, 2020.
- [61] Z. Ge, S. Liu, Z. Li, et al., "OTA: Optimal Transport Assignment for Object Detection," in *CVPR*, 2021.
- [62] N. Bodla, B. Singh, R. Chellappa, et al., "Soft-NMS: Improving Object Detection with One Line of Code," in *ICCV*, 2017.
- [63] X. Zhu, W. Su, L. Lu, et al., "Deformable DETR: Deformable Transformers for End-to-End Object Detection," in *ICLR*, 2021.
- [64] H. Zhang, F. Li, S. Liu, et al., "DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection," in *ICLR*, 2023.
- [65] J. Long, E. Shelhamer, and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," in *CVPR*, 2015.
- [66] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *MICCAI*, 2015.
- [67] L.-C. Chen, G. Papandreou, I. Kokkinos, et al., "DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs," *IEEE TPAMI*, vol. 40, no. 4, pp. 834–848, 2018.
- [68] E. Xie, W. Wang, Z. Yu, et al., "SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers," in *NeurIPS*, 2021.
- [69] B. Cheng, I. Misra, A. G. Schwing, et al., "Masked-Attention Mask Transformer for Universal Image Segmentation," in *CVPR*, 2022.
- [70] A. Kirillov, K. He, R. Girshick, et al., "Panoptic Segmentation," in *CVPR*, 2019.
- [71] A. Kirillov, E. Mintun, N. Ravi, et al., "Segment Anything," in *ICCV*, 2023.
- [72] K. Sun, B. Xiao, D. Liu, et al., "Deep High-Resolution Representation Learning for Human Pose Estimation," in *CVPR*, 2019.

- [73] Z. Cao, T. Simon, S.-E. Wei, et al., “Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields,” in *CVPR*, 2017.
- [74] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [75] J. Deng, J. Guo, N. Xue, et al., “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *CVPR*, 2019.
- [76] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [77] K. Xu, J. Ba, R. Kiros, et al., “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention,” in *ICML*, 2015.
- [78] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *ICLR*, 2015.
- [79] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [80] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [81] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [82] K. Simonyan and A. Zisserman, “Two-Stream Convolutional Networks for Action Recognition in Videos,” in *NeurIPS*, 2014.
- [83] J. Carreira and A. Zisserman, “Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset,” in *CVPR*, 2017.
- [84] D. Tran, H. Wang, L. Torresani, et al., “A Closer Look at Spatiotemporal Convolutions for Action Recognition,” in *CVPR*, 2018.
- [85] C. Feichtenhofer, H. Fan, J. Malik, et al., “SlowFast Networks for Video Recognition,” in *ICCV*, 2019.
- [86] A. Dosovitskiy, P. Fischer, E. Ilg, et al., “FlowNet: Learning Optical Flow with Convolutional Networks,” in *ICCV*, 2015.
- [87] D. Sun, X. Yang, M.-Y. Liu, et al., “PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume,” in *CVPR*, 2018.
- [88] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.
- [89] A. Bewley, Z. Ge, L. Ott, et al., “Simple Online and Realtime Tracking,” in *ICIP*, 2016.
- [90] Y. Zhang, P. Sun, Y. Jiang, et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *ECCV*, 2022.
- [91] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [92] K. Bernardin and R. Stiefelhagen, “Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics,” *EURASIP J. Image and Video Processing*, 2008.
- [93] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [94] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [95] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.



- [96] S. Wang, V. Leroy, Y. Cabon, et al., "DUST3R: Geometric 3D Vision Made Easy," in *CVPR*, 2024.
- [97] D. Eigen, C. Puhrsch, and R. Fergus, "Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network," in *NeurIPS*, 2014.
- [98] R. Ranftl, K. Lasinger, D. Hafner, et al., "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer," *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [99] J. Sturm, N. Engelhard, F. Endres, et al., "A Benchmark for the Evaluation of RGB-D SLAM Systems," in *IROS*, 2012.
- [100] C. R. Qi, H. Su, K. Mo, et al., "PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation," in *CVPR*, 2017.
- [101] C. R. Qi, L. Yi, H. Su, et al., "PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space," in *NeurIPS*, 2017.
- [102] A. H. Lang, S. Vora, H. Caesar, et al., "PointPillars: Fast Encoders for Object Detection from Point Clouds," in *CVPR*, 2019.
- [103] T. Müller, A. Evans, C. Schied, et al., "Instant Neural Graphics Primitives with a Multiresolution Hash Encoding," *ACM TOG*, vol. 41, no. 4, 2022.
- [104] B. Kerbl, G. Kopanas, T. Leimkühler, et al., "3D Gaussian Splatting for Real-Time Radiance Field Rendering," *ACM TOG*, vol. 42, no. 4, 2023.
- [105] R. Zhang, P. Isola, A. A. Efros, et al., "The Unreasonable Effectiveness of Deep Features as a Perceptual Metric," in *CVPR*, 2018.
- [106] D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," in *ICLR*, 2014.
- [107] I. Goodfellow, J. Pouget-Abadie, M. Mirza, et al., "Generative Adversarial Nets," in *NeurIPS*, 2014.
- [108] T. Karras, S. Laine, and T. Aila, "A Style-Based Generator Architecture for Generative Adversarial Networks," in *CVPR*, 2019.
- [109] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," in *NeurIPS*, 2020.
- [110] Y. Song, J. Sohl-Dickstein, D. P. Kingma, et al., "Score-Based Generative Modeling Through Stochastic Differential Equations," in *ICLR*, 2021.
- [111] J. Johnson, A. Alahi, and L. Fei-Fei, "Perceptual Losses for Real-Time Style Transfer and Super-Resolution," in *ECCV*, 2016.
- [112] P. Isola, J.-Y. Zhu, T. Zhou, et al., "Image-to-Image Translation with Conditional Adversarial Networks," in *CVPR*, 2017.
- [113] L. A. Gatys, A. S. Ecker, and M. Bethge, "Image Style Transfer Using Convolutional Neural Networks," in *CVPR*, 2016.
- [114] X. Huang and S. Belongie, "Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization," in *ICCV*, 2017.
- [115] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," in *NeurIPS Deep Learning Workshop*, 2015.
- [116] J. Song, C. Meng, and S. Ermon, "Denoising Diffusion Implicit Models," in *ICLR*, 2021.
- [117] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, et al., "Flow Matching for Generative Modeling," in *ICLR*, 2023.
- [118] J. Ho and T. Salimans, "Classifier-Free Diffusion Guidance," *arXiv:2207.12598*, 2022.
- [119] R. Rombach, A. Blattmann, D. Lorenz, et al., "High-Resolution Image Synthesis with Latent Diffusion Models," in *CVPR*, 2022.
- [120] W. Peebles and S. Xie, "Scalable Diffusion Models with Transformers," in *ICCV*, 2023.

- [121] L. Zhang, A. Rao, and M. Agrawala, “Adding Conditional Control to Text-to-Image Diffusion Models,” in *ICCV*, 2023.
- [122] Y. Blau and T. Michaeli, “The Perception-Distortion Tradeoff,” in *CVPR*, 2018.
- [123] C. Ledig, L. Theis, F. Huszár, et al., “Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network,” in *CVPR*, 2017.
- [124] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., “Image Quality Assessment: From Error Visibility to Structural Similarity,” *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [125] M. Heusel, H. Ramsauer, T. Unterthiner, et al., “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium,” in *NeurIPS*, 2017.
- [126] T. Brooks, B. Mildenhall, T. Xue, et al., “Unprocessing Images for Learned Raw Denoising,” in *CVPR*, 2019.
- [127] A. Abdelhamed, S. Lin, and M. S. Brown, “A High-Quality Denoising Dataset for Smartphone Cameras,” in *CVPR*, 2018.
- [128] J. Deng, W. Dong, R. Socher, et al., “ImageNet: A Large-Scale Hierarchical Image Database,” in *CVPR*, 2009.
- [129] A. Radford, J. W. Kim, C. Hallacy, et al., “Learning Transferable Visual Models from Natural Language Supervision,” in *ICML*, 2021.
- [130] J. Li, D. Li, S. Savarese, et al., “BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models,” in *ICML*, 2023.
- [131] H. Liu, C. Li, Q. Wu, et al., “Visual Instruction Tuning,” in *NeurIPS*, 2023.
- [132] S. Kornblith, J. Shlens, and Q. V. Le, “Do Better ImageNet Models Transfer Better?” In *CVPR*, 2019.
- [133] T. Chen, S. Kornblith, M. Norouzi, et al., “A Simple Framework for Contrastive Learning of Visual Representations,” in *ICML*, 2020.
- [134] K. He, H. Fan, Y. Wu, et al., “Momentum Contrast for Unsupervised Visual Representation Learning,” in *CVPR*, 2020.
- [135] J.-B. Grill, F. Strub, F. Altché, et al., “Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning,” in *NeurIPS*, 2020.
- [136] M. Caron, H. Touvron, I. Misra, et al., “Emerging Properties in Self-Supervised Vision Transformers,” in *ICCV*, 2021.
- [137] K. He, X. Chen, S. Xie, et al., “Masked Autoencoders Are Scalable Vision Learners,” in *CVPR*, 2022.
- [138] A. van den Oord, Y. Li, and O. Vinyals, “Representation Learning with Contrastive Predictive Coding,” in *arXiv:1807.03748*, 2018.
- [139] J. Kaplan, S. McCandlish, T. Henighan, et al., “Scaling Laws for Neural Language Models,” *arXiv:2001.08361*, 2020.
- [140] X. Zhai, A. Kolesnikov, N. Houlsby, et al., “Scaling Vision Transformers,” in *CVPR*, 2022.
- [141] J. Hoffmann, S. Borgeaud, A. Mensch, et al., “Training Compute-Optimal Large Language Models,” in *NeurIPS*, 2022.
- [142] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [143] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [144] N. Houlsby, A. Giurgiu, S. Jastrzebski, et al., “Parameter-Efficient Transfer Learning for NLP,” in *ICML*, 2019.

- [145] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [146] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *RecSys*, 2019.

## 2.4 ViT và bài học về inductive bias (Vision Transformers and Inductive Bias)

**PHỤ THUỘC** C/13/01-dong-cnn (ba giả định mà convolution cài sẵn); C/13/02-tu-chuoi-toi-attention (attention, mã hoá vị trí, chỉ phí bậc hai).

### TÓM TẮT

Mục này dùng *Vision Transformer* làm thí nghiệm sạch nhất cho một câu hỏi trung tâm của cả cây: cài sẵn giả định vào mô hình thì được gì và mất gì. ViT vứt bỏ hai giả định mà convolution cài sẵn, và cái giá lộ ra rất rõ ràng dưới dạng *lượng dữ liệu cần có*. Đọc xong bạn quyết định được khi nào nên dùng ViT, khi nào nên dùng CNN, khi nào nên lai. Bạn cũng đọc được mã nguồn của một hệ detection trong mười phút, nhờ cách tách *backbone*, *neck* và *head*. Nếu chỉ nhớ một điều: *inductive bias là một khoản vay trả trước cho dữ liệu* — bạn hoặc trả bằng giả định, hoặc trả bằng mẫu, và ViT chỉ đơn giản chọn cách trả thứ hai.

### KIẾN THỨC CẦN CÓ TRƯỚC

- **inductive bias** — tập giả định mà mô hình mang sẵn trước khi nhìn dữ liệu, ví dụ “pixel gần nhau thì liên quan”; nó thu hẹp không gian giả thuyết nên học nhanh hơn với ít dữ liệu, nhưng nếu giả định sai thì nó chặn luôn nghiệm đúng.
- **patch** — ô vuông nhỏ được cắt ra từ ảnh, thường  $16 \times 16$  pixel, rồi làm phẳng thành một vector để coi như một token; đây là cách ViT biến ảnh thành chuỗi.
- **đẳng biến và bất biến (equivariance vs invariance)** — đẳng biến tịnh tiến nghĩa là dịch ảnh thì bản đồ đặc trưng dịch theo đúng như vậy; bất biến nghĩa là dịch ảnh thì đầu ra *không đổi*. Convolution cho đẳng biến, còn phép gộp toàn cục ở cuối mới tạo ra bất biến.
- **pretraining và fine-tuning** — huấn luyện trước trên một tập rất lớn để học đặc trưng chung, rồi huấn luyện tiếp trên tập nhỏ của bài toán đích; đây là cách phổ biến nhất để bù lượng dữ liệu mà một mô hình ít giả định đòi hỏi.
- **chưng cất (distillation)** — huấn luyện mô hình học sinh bắt chước đầu ra của một mô hình thầy đã tốt, thay vì chỉ học từ nhãn cứng; nó chuyển được cả các giả định ngầm của thầy sang trò.
- **backbone, neck, head** — ba tầng chức năng của một hệ thị giác: backbone trích đặc trưng từ ảnh, neck hợp nhất đặc trưng ở nhiều tỉ lệ, head sinh đầu ra theo đúng bài toán (nhân lớp, hộp, mặt nạ).
- **FPN (feature pyramid network)** — một kiểu neck ghép đặc trưng độ phân giải cao nhưng ngữ nghĩa yếu ở tầng nông với đặc trưng ngữ nghĩa mạnh nhưng thô ở tầng sâu, để phát hiện được cả vật nhỏ lẫn vật lớn.

### A. Đặt vấn đề

Suốt chương 13 ta lặp lại một câu hỏi: nên cài sẵn bao nhiêu giả định vào mô hình? Câu hỏi đó rất khó trả lời bằng lý luận, vì hai vế của nó nằm ở hai đơn vị khác nhau. Một bên là “mô hình biết trước điều gì”; bên kia là “có bao nhiêu dữ liệu”. *Vision Transformer* (ViT) [45] biến câu hỏi đó thành một thí nghiệm đo được: lấy một kiến trúc gần như không có giả định thị giác nào, huấn luyện nó ở nhiều quy mô dữ liệu khác nhau, và xem điểm giao cắt nằm ở đâu.

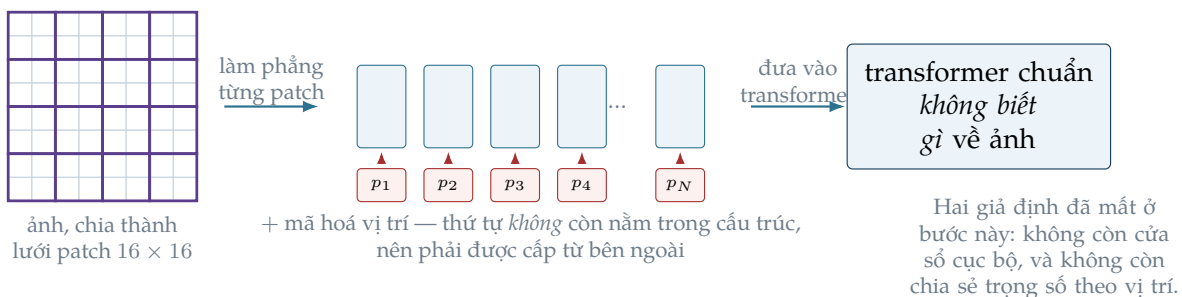
Kết quả là câu trả lời sạch nhất mà ngành có: với dữ liệu ít, ViT thua CNN rõ rệt; với dữ liệu rất nhiều, ViT vượt. Giá trị cụ thể của mục này là biến quan sát đó thành một quy tắc quyết định dùng được. Quy tắc ấy chạy trên ba dữ kiện: bạn có bao nhiêu ảnh có nhãn, bạn có mô hình pretrain nào, và ràng buộc độ phân giải của bạn là gì. Mục này cũng cho bạn một cách đọc mã nguồn nhanh cho bất kỳ hệ thị giác nào.

→ **tiền đề** B/10 — trực đánh đổi giả định—dữ liệu được phát biểu ở chương thiết kế kiến trúc; mục này là phép đo thực nghiệm cho chính trực đó

## B. ViT bỏ đi cái gì

ViT cắt ảnh thành các patch  $16 \times 16$ , làm phẳng mỗi patch thành một vectơ, cộng mã hoá vị trí, rồi đưa cả dãy vào một transformer chuẩn. Phép biến đổi đó vứt bỏ hai trong ba giả định của convolution nêu ở C/13/01.

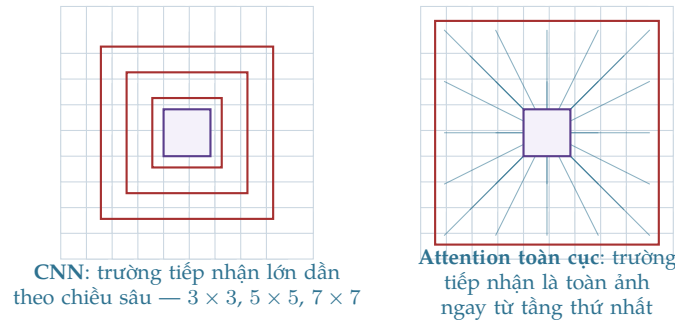
- **Tính cục bộ biến mất.** Ngay từ tầng đầu tiên, mọi patch nói chuyện với mọi patch. Không có gì trong kiến trúc nói rằng hai patch cạnh nhau liên quan nhiều hơn hai patch ở hai góc ảnh — mô hình phải học điều đó từ dữ liệu.
- **Đăng biến tịnh tiến biến mất.** Convolution dùng chung một bộ trọng số cho mọi vị trí nên dịch ảnh thì đặc trưng dịch theo. ViT gán một mã vị trí riêng cho từng ô, nên cùng một vật ở hai vị trí khác nhau đi qua hai đường tính toán hơi khác nhau.
- **Phân cấp cũng gần như biến mất** ở bản ViT gốc: mọi tầng chạy ở cùng một độ phân giải token, không có chuỗi giảm dần độ phân giải như CNN. Đây là giả định thứ ba, và chính nó sẽ được các biến thể phân cấp lấy lại đầu tiên.



**Hình 2.9.** Ba bước biến một ảnh thành thứ mà transformer xử lý được, và chỗ inductive bias rơi ra. Việc cắt patch vẫn giữ lại một chút tính cục bộ: 256 điểm ảnh trong một ô bị gộp cứng. Nhưng *quan hệ giữa* các patch thì không còn được mã hoá ở đâu trong kiến trúc. Nó phải được cấp bằng một bảng vị trí học được. Đó là lý do đối độ phân giải đầu vào mà quên nội suy bảng này làm mô hình xuống cấp lạng lẹ: bạn vừa cấp sai đúng cái thông tin mà kiến trúc không tự có.

Hình 2.9 cho thấy điều mà câu “ViT coi ảnh như một chuỗi” còn giấu: chuỗi đó không có thứ tự tự nhiên, và toàn bộ hiểu biết về không gian hai chiều bị dồn vào một bảng tham số phụ.

Cần nói ngay một điều để tránh hiểu quá đà: ViT *không phải không có giả định*. Bản thân việc cắt thành patch là một giả định cục bộ khá mạnh — 256 pixel trong một ô bị gộp thành một token và không bao giờ được xét riêng nữa. Ngoài ra, các phân tích attention cho thấy ở những tầng đầu, nhiều đầu tự học ra hành vi gần như cục bộ, tức mô hình khám phá lại một phần giả định đã bị vứt. Phát biểu đúng là: ViT giảm mạnh lượng giả định cài sẵn, chứ không xoá sạch.

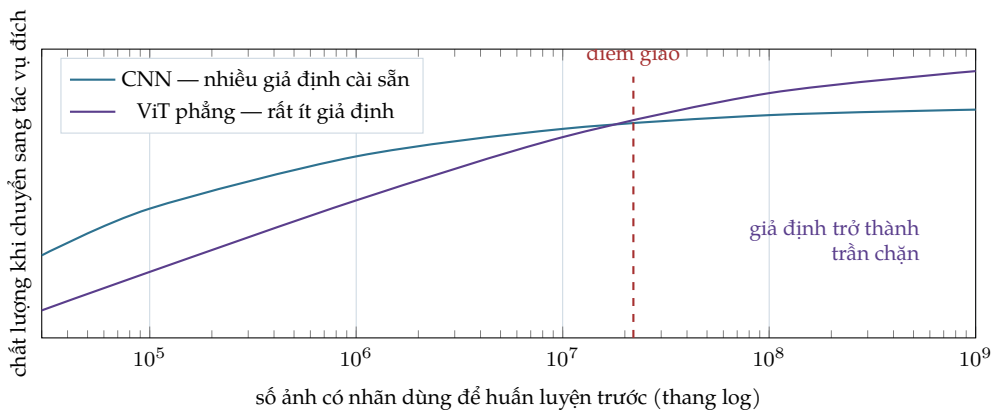


**Hình 2.10.** Hai cách phủ sóng thông tin. CNN mở rộng trường tiếp nhận dần dần và do đó *cài sẵn* rằng quan hệ gần quan trọng hơn quan hệ xa; attention toàn cục nối mọi vị trí ngay lập tức và không cài giả định nào về khoảng cách, đổi lại phải học quan hệ đó từ dữ liệu và trả chi phí bậc hai. Swin và các biến thể cửa sổ nằm giữa hai thái cực này.

Hình 2.10 là cách gọn nhất để thấy cả ưu điểm lẫn cái giá: bên trái, thông tin xa cần nhiều tầng nhưng mỗi tầng rẻ và có sẵn tiên nghiệm đúng cho ảnh tự nhiên; bên phải, thông tin xa có ngay nhưng phải trả bằng  $T^2$  cặp quan hệ và bằng việc mô hình không được cho biết cặp nào đáng quan tâm.

### C. Cái giá hiện ra dưới dạng lượng dữ liệu

Kết quả gốc của ViT rất dễ tóm tắt: huấn luyện từ đầu trên ImageNet-1k thì nó thua các ResNet cùng hạng; pretrain trên một tập lớn hơn nhiều bậc rồi fine-tune thì nó vượt [45]. Đây là minh chứng trực tiếp cho trục đánh đổi giả định–dữ liệu: giả định mà bạn không cài vào kiến trúc thì phải mua bằng mẫu.



**Hình 2.11.** Trục đánh đổi giả định–dữ liệu, vẽ định tính. Ở phía trái, các giả định mà convolution cài sẵn là món quà miễn phí: chúng thu hẹp không gian giả thuyết đúng chỗ, nên CNN học được từ ít mẫu. Ở phía phải, chính các giả định ấy thành trần — chúng cấm mô hình biểu diễn những quan hệ mà dữ liệu đã đủ để dạy nó. Điều quan trọng nhất trong hình là *có* một điểm giao chứ không phải vị trí của nó: vị trí phụ thuộc tác vụ, recipe và quy mô mô hình, và ba cách trả nợ giả định ở mục C đều là cách kéo đường tím sang trái.

Hình 2.11 biến một câu định tính thành một hình có thể tranh luận được: câu hỏi “CNN hay ViT” thực chất là câu hỏi “tôi đang đứng ở đâu trên trục hoành”.

Điều làm câu chuyện thú vị là có ít nhất ba cách trả khoản nợ đó, và chúng trả bằng ba loại tiền khác nhau.

- **Trả bằng dữ liệu** — pretrain trên tập cực lớn, rồi fine-tune. Hiệu quả nhất về chất lượng, nhưng chỉ khả thi nếu bạn có tập đó hoặc có checkpoint của người khác; và nó đẩy toàn bộ chi phí sang một pha huấn luyện mà bạn không kiểm soát.
- **Trả bằng chính quy hoá và chứng cất** — DeiT cho thấy với augmentation mạnh, lịch học dài và một thầy CNN để chứng cất, ViT huấn luyện được trên riêng ImageNet-1k [46]. Đáng chú ý là thầy CNN chuyển sang trò không chỉ nhận mà cả *giả định* của mình — đây là một cách cài inductive bias qua cửa sau.
- **Trả bằng kiến trúc** — lắp lại một phần giả định đã bỏ, tức các biến thể phân cấp ở mục D. Rẻ nhất về dữ liệu, nhưng đánh mất sự thuần khiết và một phần khả năng mở rộng theo quy mô.

↪ **hệ quả của C/18** — pretraining tự giám sát là cách rẻ nhất để trả khoản nợ giả định; các backbone mạnh nhất hiện nay đều đến từ hướng này.

↪ **cùng nguyên lý với C/12/04** — augmentation mạnh và chứng cất đều bơm thêm giả định vào mô hình, chỉ khác đường đi.

#### D. Phản ứng thứ nhất: ViT phân cấp

ViT phẳng có hai vấn đề thực dụng ngoài chuyện dữ liệu: nó chỉ sinh ra đặc trưng ở một tỉ lệ duy nhất, trong khi detection và segmentation cần nhiều tỉ lệ; và chi phí bậc hai chặn đường tới độ phân giải cao, như đã tính ở C/13/02. Swin giải cả hai bằng một thay đổi: giới hạn attention trong các cửa sổ cố định rồi *dịch* cửa sổ ở tầng kế tiếp để thông tin rò qua ranh giới, đồng thời gộp patch dần để tạo kim tự tháp đặc trưng [42]. Chi phí trở thành tuyến tính theo số token, và đầu ra có đúng dạng mà FPN cần. PVT đi theo hướng gần giống nhưng giảm số khoá và giá trị thay vì giới hạn cửa sổ. Còn lại ghép đơn giản nhất là thay phép cắt patch bằng vài tầng convolution làm “stem”. Cách này được biết là làm giai đoạn huấn luyện đầu ổn định hơn hẳn. Đây là kinh nghiệm thực hành rất phổ biến, không phải một kết quả tôi kiểm chứng lại.

Cái giá thì cụ thể: cửa sổ tạo lại một dạng “đường đi dài” cho quan hệ xa, đúng thứ mà attention sinh ra để xoá; và kiến trúc nhiều giai đoạn khó mở rộng theo quy mô một cách trơn tru hơn so với một chồng khối đồng nhất.

#### E. Phản ứng thứ hai: ConvNeXt, và vì sao nó là phản chứng quan trọng

Hướng ngược lại không sửa transformer mà sửa cách so sánh. ConvNeXt giữ nguyên convolution rồi lần lượt áp từng thành phần recipe của transformer, đo sau mỗi bước, và kết quả là CNN bất kịp [35]. Kết luận đã nêu ở C/13/01 và nhắc lại ở đây vì nó thay đổi cách đọc chính mục này: phần lớn khoảng cách được gán cho “attention thay convolution” thực ra thuộc về công thức huấn luyện. Nói cách khác, thí nghiệm ViT vẫn đúng về trục giả định–dữ liệu, nhưng *độ lớn* của hiệu ứng kiến trúc đã bị phóng đại trong cách trình bày đương thời.

#### F. Ba dạng tổ chức và ba lớp bài toán

Chọn sai dạng tổ chức là lỗi thiết kế cơ bản, không phải lỗi tinh chỉnh. Không có siêu tham số nào cứu được một encoder-only đang bị bắt sinh chuỗi tự hồi quy.

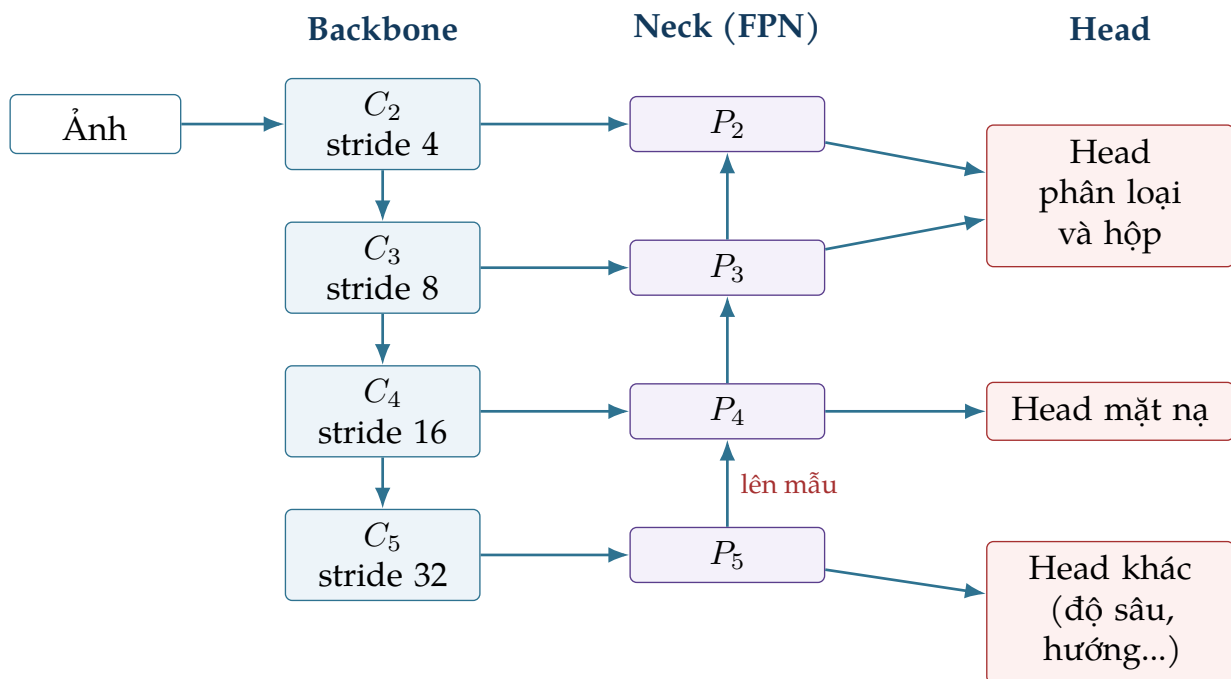


**Bảng 2.4.** Ba dạng tổ chức của một mô hình dựa trên transformer, và lớp bài toán tương ứng.

| Dạng            | Cơ chế                                                                           | Hợp với bài toán nào                                                                    | Hỏng khi nào                                                         |
|-----------------|----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| Encoder-only    | Mọi token thấy mọi token, một lượt                                               | Hiểu và dự đoán từ toàn bộ đầu vào: phân loại, dự đoán đầy, trích đặc trưng cho hệ khác | Cần sinh chuỗi có độ dài chưa biết trước, hoặc cần sinh tự hồi quy   |
| Decoder-only    | Attention có mặt nạ nhân quả, sinh từng token dựa trên các token đã sinh         | Sinh tự hồi quy: ngôn ngữ, chuỗi hành động, mã hoá ảnh thành token rồi sinh             | Suy luận chậm vì tuần tự; lãng phí khi đầu ra là một nhãn duy nhất   |
| Encoder-decoder | Encoder đọc đầu vào, decoder sinh đầu ra và tra cứu encoder bằng cross-attention | Biến đổi chuỗi sang chuỗi; trong thị giác là DETR với tập truy vấn học được [44]        | Nhiều tham số và nhiều đường nối hơn; hội tụ chậm hơn ở bài toán nhỏ |

### G. Tách backbone, neck, head — cách đọc code nhanh nhất

Ba khái niệm này không phải lý thuyết mà là quy ước kỹ thuật, và chúng có giá trị thực dụng rất cao: gần như mọi repo detection và segmentation hiện đại đều tổ chức theo đúng ba khối này, nên nắm được chúng là đọc được cấu trúc một repo lạ trong mười phút thay vì một ngày.



**Hình 2.12.** Ba khối chức năng của một hệ thị giác đầy. Backbone sinh đặc trưng ở nhiều stride; neck hợp nhất chúng theo chiều từ sâu về nông để mọi tỉ lệ đều có ngữ nghĩa mạnh; head chuyển đặc trưng thành đầu ra của bài toán. Thay backbone là thao tác rẻ nhất và hay làm nhất; thay neck ảnh hưởng chủ yếu tới vật nhỏ; thay head là đổi bài toán.

Hình 3.1 cũng giải thích vì sao ViT phẳng khó lắp vào hệ detection: nó chỉ sinh ra một cột duy nhất chứ không sinh ra bốn mức  $C_2$  tới  $C_5$ , nên hoặc phải chế thêm một neck đặc biệt, hoặc phải chuyển sang biến thể phân cấp. Đây là một ràng buộc kỹ thuật rất thực tế và thường quyết định lựa chọn backbone hơn cả điểm số ImageNet.

→ **tiền đề C/14** — các head của detection và segmentation giả định có sẵn bốn mức đặc trưng, nên dạng đầu ra của backbone quyết định head nào lắp được

## H. Giới hạn và trường hợp hỏng

Bốn chỗ hỏng đáng nhớ, xếp theo tần suất gặp trong thực hành.

Thứ nhất, *ViT huấn luyện từ đầu trên tập nhỏ*. Đây là cách phổ biến nhất để kết luận sai rằng “transformer không hợp bài toán của tôi”. Nếu bạn có vài nghìn ảnh, kết luận đúng là bạn không đủ dữ liệu để trả khoản nợ giả định, chứ không phải kiến trúc kém.

Thứ hai, *kích thước patch so với kích thước đối tượng*. Một patch  $16 \times 16$  nghiền mọi đối tượng nhỏ hơn 16 pixel thành một phần của một token duy nhất. Với ảnh camera gắn xe, người đi bộ ở xa chỉ chiếm mười pixel. Đây không phải chi tiết nhỏ mà là giới hạn cứng. Nó giải thích vì sao các hệ dự đoán dày dúng patch nhỏ hoặc dùng conv stem.

Thứ ba, *mã hoá vị trí khi đổi độ phân giải*, đã nói ở mục trước và nhắc lại vì nó thực sự hay xảy ra.

Thứ tư, *mất đẳng biến tịnh tiến là có thật và đo được*. Cùng một vật ở hai vị trí có thể cho đặc trưng hơi khác nhau. Trong phần lớn trường hợp, augmentation ngẫu nhiên đã bù đủ. Nhưng ở các bài toán đo lường chính xác như ước lượng tư thế hay đo khoảng cách, đây là một nguồn sai số hệ thống đáng kiểm tra chứ không nên giả định là không có.

Về độ tin cậy: quan hệ “ít dữ liệu thì CNN thắng, nhiều dữ liệu thì ViT thắng” là kết quả đã công bố và được lặp lại rộng rãi; còn mức dữ liệu cụ thể ở điểm giao cắt phụ thuộc tác vụ, recipe và quy mô mô hình tới mức tôi không nêu một con số nào ở đây.

### CẠM BÃY

Cạm bẫy tư duy lớn nhất của mục này là đọc “ViT không có inductive bias” theo nghĩa đen rồi suy ra “ViT là mô hình trung lập, không giả định gì”. Không đúng. Phép cắt patch là một giả định cục bộ rất mạnh. Mã hoá vị trí là một giả định về cách biểu diễn không gian. Và cách chọn dữ liệu pretrain là một giả định khổng lồ về thế giới mà mô hình sẽ gặp. Phát biểu chính xác là ViT *dịch chuyển* giả định từ kiến trúc sang dữ liệu và recipe — và giả định nằm trong dữ liệu thì khó nhìn thấy hơn nhiều so với giả định nằm trong kiến trúc.

### CẦN NHỚ

Inductive bias là khoản vay trả trước cho dữ liệu: giả định bạn không cài vào kiến trúc thì phải mua bằng mẫu, bằng chưng cất, hoặc bằng augmentation. Khi chọn giữa CNN, ViT và các biến thể lai, câu hỏi đầu tiên không phải “cái nào mạnh hơn” mà là “tôi có bao nhiêu dữ liệu và bao nhiêu tiền nghiệm đáng tin về bài toán này”. [T2]

## I. Kết nối

Mục này khép lại chương 13 bằng cách trả lời câu hỏi mở ở [C/13/01-dong-cnn](#): ba giả định của convolution đáng giá bao nhiêu, đo bằng lượng dữ liệu cần để thay thế chúng. Nó dựa hoàn toàn vào cơ chế attention ở [C/13/02-tu-chuoi-toi-attention](#), và cách tách backbone-neck-head ở mục G là tiền đề trực tiếp cho [C/14-bai-toan-cv-loi](#), nơi các head cụ thể được thiết kế. Về phía dữ liệu, việc trả nợ giả định bằng pretraining là nội dung của [C/18-pretraining-va-foundation](#), còn việc trả bằng augmentation đã nằm ở [C/12/04-tong-quat-hoa-va-chan-doan](#). Cuối cùng, bài học ConvNeXt nối sang [E/24-thi-nghiem-va-ablation](#), chương duy nhất trong cây dạy cách không bị một bảng so sánh đánh lừa.

### Câu hỏi tự kiểm (trả lời không nhìn bài)

- ViT bỏ đi hai giả định nào của convolution, và mỗi giả định bị bỏ gây ra hậu quả đo được nào?

2. Vì sao nói ViT vẫn có inductive bias, dù nó bỏ tính cục bộ và đẳng biến tịnh tiến?
3. Ba cách bù lại lượng dữ liệu mà ViT đòi hỏi là gì, và mỗi cách trả giá bằng loại tài nguyên nào?
4. Vì sao Swin lắp vào một hệ detection dễ hơn ViT phẳng — câu trả lời nằm ở khối nào trong sơ đồ backbone–neck–head?
5. Nếu ConvNeXt cho thấy recipe quan trọng hơn kiến trúc, điều đó thay đổi cách bạn đọc một bảng so sánh backbone thế nào?
6. Bạn cần phát hiện người đi bộ cách xa 80 mét, chỉ chiếm khoảng 12 pixel chiều cao. Vì sao ViT patch 16 là lựa chọn tệ, và hai cách sửa là gì?
7. Trong hệ của bạn, thay backbone, thay neck và thay head có tác động khác nhau thế nào — và vì sao thứ tự thứ nên là backbone trước?

- [1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [2] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [3] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *AISTATS*, 2010.
- [4] K. He, X. Zhang, S. Ren, et al., “Delving Deep into Rectifiers,” in *ICCV*, 2015.
- [5] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [6] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [7] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [8] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [9] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [10] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [11] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” in *ICLR*, 2017.
- [12] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [13] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [14] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [15] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [16] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [17] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [18] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [19] G. Huang, Y. Sun, Z. Liu, et al., “Deep Networks with Stochastic Depth,” in *ECCV*, 2016.
- [20] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [21] H. Zhang, M. Cisse, Y. N. Dauphin, et al., “mixup: Beyond Empirical Risk Minimization,” in *ICLR*, 2018.
- [22] S. Yun, D. Han, S. J. Oh, et al., “CutMix: Regularization Strategy to Train Strong Classifiers,” in *ICCV*, 2019.
- [23] E. D. Cubuk, B. Zoph, J. Shlens, et al., “RandAugment: Practical Automated Data Augmentation,” in *CVPR Workshops*, 2020.

- [24] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [25] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [26] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *NeurIPS*, 2012.
- [27] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in *ICLR*, 2015.
- [28] C. Szegedy, W. Liu, et al., “Going Deeper with Convolutions,” in *CVPR*, 2015.
- [29] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [30] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [31] G. Huang, Z. Liu, L. van der Maaten, et al., “Densely Connected Convolutional Networks,” in *CVPR*, 2017.
- [32] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [33] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [34] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [35] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [36] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [37] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.
- [38] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [39] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [40] J. Su, M. Ahmed, Y. Lu, et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, vol. 568, 2024.
- [41] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” *arXiv:2312.00752*, 2023.
- [42] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [43] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.
- [44] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [45] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [46] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [47] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [48] R. Girshick, J. Donahue, T. Darrell, et al., “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation,” in *CVPR*, 2014.

- [49] R. Girshick, "Fast R-CNN," in *ICCV*, 2015.
- [50] S. Ren, K. He, R. Girshick, et al., "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in *NeurIPS*, 2015.
- [51] T.-Y. Lin, P. Dollár, R. Girshick, et al., "Feature Pyramid Networks for Object Detection," in *CVPR*, 2017.
- [52] K. He, G. Gkioxari, P. Dollár, et al., "Mask R-CNN," in *ICCV*, 2017.
- [53] D. Bolya, S. Foley, J. Hays, et al., "TIDE: A General Toolbox for Identifying Object Detection Errors," in *ECCV*, 2020.
- [54] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [55] J. Redmon, S. Divvala, R. Girshick, et al., "You Only Look Once: Unified, Real-Time Object Detection," in *CVPR*, 2016.
- [56] W. Liu, D. Anguelov, D. Erhan, et al., "SSD: Single Shot MultiBox Detector," in *ECCV*, 2016.
- [57] T.-Y. Lin, P. Goyal, R. Girshick, et al., "Focal Loss for Dense Object Detection," in *ICCV*, 2017.
- [58] Z. Tian, C. Shen, H. Chen, et al., "FCOS: Fully Convolutional One-Stage Object Detection," in *ICCV*, 2019.
- [59] X. Zhou, D. Wang, and P. Krähenbühl, "Objects as Points," *arXiv:1904.07850*, 2019.
- [60] S. Zhang, C. Chi, Y. Yao, et al., "Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection," in *CVPR*, 2020.
- [61] Z. Ge, S. Liu, Z. Li, et al., "OTA: Optimal Transport Assignment for Object Detection," in *CVPR*, 2021.
- [62] N. Bodla, B. Singh, R. Chellappa, et al., "Soft-NMS: Improving Object Detection with One Line of Code," in *ICCV*, 2017.
- [63] X. Zhu, W. Su, L. Lu, et al., "Deformable DETR: Deformable Transformers for End-to-End Object Detection," in *ICLR*, 2021.
- [64] H. Zhang, F. Li, S. Liu, et al., "DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection," in *ICLR*, 2023.
- [65] J. Long, E. Shelhamer, and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," in *CVPR*, 2015.
- [66] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *MICCAI*, 2015.
- [67] L.-C. Chen, G. Papandreou, I. Kokkinos, et al., "DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs," *IEEE TPAMI*, vol. 40, no. 4, pp. 834–848, 2018.
- [68] E. Xie, W. Wang, Z. Yu, et al., "SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers," in *NeurIPS*, 2021.
- [69] B. Cheng, I. Misra, A. G. Schwing, et al., "Masked-Attention Mask Transformer for Universal Image Segmentation," in *CVPR*, 2022.
- [70] A. Kirillov, K. He, R. Girshick, et al., "Panoptic Segmentation," in *CVPR*, 2019.
- [71] A. Kirillov, E. Mintun, N. Ravi, et al., "Segment Anything," in *ICCV*, 2023.
- [72] K. Sun, B. Xiao, D. Liu, et al., "Deep High-Resolution Representation Learning for Human Pose Estimation," in *CVPR*, 2019.



- [73] Z. Cao, T. Simon, S.-E. Wei, et al., “Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields,” in *CVPR*, 2017.
- [74] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [75] J. Deng, J. Guo, N. Xue, et al., “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *CVPR*, 2019.
- [76] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [77] K. Xu, J. Ba, R. Kiros, et al., “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention,” in *ICML*, 2015.
- [78] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *ICLR*, 2015.
- [79] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [80] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [81] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [82] K. Simonyan and A. Zisserman, “Two-Stream Convolutional Networks for Action Recognition in Videos,” in *NeurIPS*, 2014.
- [83] J. Carreira and A. Zisserman, “Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset,” in *CVPR*, 2017.
- [84] D. Tran, H. Wang, L. Torresani, et al., “A Closer Look at Spatiotemporal Convolutions for Action Recognition,” in *CVPR*, 2018.
- [85] C. Feichtenhofer, H. Fan, J. Malik, et al., “SlowFast Networks for Video Recognition,” in *ICCV*, 2019.
- [86] A. Dosovitskiy, P. Fischer, E. Ilg, et al., “FlowNet: Learning Optical Flow with Convolutional Networks,” in *ICCV*, 2015.
- [87] D. Sun, X. Yang, M.-Y. Liu, et al., “PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume,” in *CVPR*, 2018.
- [88] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.
- [89] A. Bewley, Z. Ge, L. Ott, et al., “Simple Online and Realtime Tracking,” in *ICIP*, 2016.
- [90] Y. Zhang, P. Sun, Y. Jiang, et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *ECCV*, 2022.
- [91] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [92] K. Bernardin and R. Stiefelhagen, “Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics,” *EURASIP J. Image and Video Processing*, 2008.
- [93] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [94] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [95] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.

- [96] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *CVPR*, 2024.
- [97] D. Eigen, C. Puhrsch, and R. Fergus, “Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network,” in *NeurIPS*, 2014.
- [98] R. Ranftl, K. Lasinger, D. Hafner, et al., “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer,” *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [99] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IROS*, 2012.
- [100] C. R. Qi, H. Su, K. Mo, et al., “PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation,” in *CVPR*, 2017.
- [101] C. R. Qi, L. Yi, H. Su, et al., “PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space,” in *NeurIPS*, 2017.
- [102] A. H. Lang, S. Vora, H. Caesar, et al., “PointPillars: Fast Encoders for Object Detection from Point Clouds,” in *CVPR*, 2019.
- [103] T. Müller, A. Evans, C. Schied, et al., “Instant Neural Graphics Primitives with a Multiresolution Hash Encoding,” *ACM TOG*, vol. 41, no. 4, 2022.
- [104] B. Kerbl, G. Kopanas, T. Leimkühler, et al., “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *ACM TOG*, vol. 42, no. 4, 2023.
- [105] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.
- [106] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” in *ICLR*, 2014.
- [107] I. Goodfellow, J. Pouget-Abadie, M. Mirza, et al., “Generative Adversarial Nets,” in *NeurIPS*, 2014.
- [108] T. Karras, S. Laine, and T. Aila, “A Style-Based Generator Architecture for Generative Adversarial Networks,” in *CVPR*, 2019.
- [109] J. Ho, A. Jain, and P. Abbeel, “Denoising Diffusion Probabilistic Models,” in *NeurIPS*, 2020.
- [110] Y. Song, J. Sohl-Dickstein, D. P. Kingma, et al., “Score-Based Generative Modeling Through Stochastic Differential Equations,” in *ICLR*, 2021.
- [111] J. Johnson, A. Alahi, and L. Fei-Fei, “Perceptual Losses for Real-Time Style Transfer and Super-Resolution,” in *ECCV*, 2016.
- [112] P. Isola, J.-Y. Zhu, T. Zhou, et al., “Image-to-Image Translation with Conditional Adversarial Networks,” in *CVPR*, 2017.
- [113] L. A. Gatys, A. S. Ecker, and M. Bethge, “Image Style Transfer Using Convolutional Neural Networks,” in *CVPR*, 2016.
- [114] X. Huang and S. Belongie, “Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization,” in *ICCV*, 2017.
- [115] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.
- [116] J. Song, C. Meng, and S. Ermon, “Denoising Diffusion Implicit Models,” in *ICLR*, 2021.
- [117] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, et al., “Flow Matching for Generative Modeling,” in *ICLR*, 2023.
- [118] J. Ho and T. Salimans, “Classifier-Free Diffusion Guidance,” *arXiv:2207.12598*, 2022.
- [119] R. Rombach, A. Blattmann, D. Lorenz, et al., “High-Resolution Image Synthesis with Latent Diffusion Models,” in *CVPR*, 2022.
- [120] W. Peebles and S. Xie, “Scalable Diffusion Models with Transformers,” in *ICCV*, 2023.

- [121] L. Zhang, A. Rao, and M. Agrawala, "Adding Conditional Control to Text-to-Image Diffusion Models," in *ICCV*, 2023.
- [122] Y. Blau and T. Michaeli, "The Perception-Distortion Tradeoff," in *CVPR*, 2018.
- [123] C. Ledig, L. Theis, F. Huszár, et al., "Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network," in *CVPR*, 2017.
- [124] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., "Image Quality Assessment: From Error Visibility to Structural Similarity," *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [125] M. Heusel, H. Ramsauer, T. Unterthiner, et al., "GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium," in *NeurIPS*, 2017.
- [126] T. Brooks, B. Mildenhall, T. Xue, et al., "Unprocessing Images for Learned Raw Denoising," in *CVPR*, 2019.
- [127] A. Abdelhamed, S. Lin, and M. S. Brown, "A High-Quality Denoising Dataset for Smartphone Cameras," in *CVPR*, 2018.
- [128] J. Deng, W. Dong, R. Socher, et al., "ImageNet: A Large-Scale Hierarchical Image Database," in *CVPR*, 2009.
- [129] A. Radford, J. W. Kim, C. Hallacy, et al., "Learning Transferable Visual Models from Natural Language Supervision," in *ICML*, 2021.
- [130] J. Li, D. Li, S. Savarese, et al., "BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models," in *ICML*, 2023.
- [131] H. Liu, C. Li, Q. Wu, et al., "Visual Instruction Tuning," in *NeurIPS*, 2023.
- [132] S. Kornblith, J. Shlens, and Q. V. Le, "Do Better ImageNet Models Transfer Better?" In *CVPR*, 2019.
- [133] T. Chen, S. Kornblith, M. Norouzi, et al., "A Simple Framework for Contrastive Learning of Visual Representations," in *ICML*, 2020.
- [134] K. He, H. Fan, Y. Wu, et al., "Momentum Contrast for Unsupervised Visual Representation Learning," in *CVPR*, 2020.
- [135] J.-B. Grill, F. Strub, F. Altché, et al., "Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning," in *NeurIPS*, 2020.
- [136] M. Caron, H. Touvron, I. Misra, et al., "Emerging Properties in Self-Supervised Vision Transformers," in *ICCV*, 2021.
- [137] K. He, X. Chen, S. Xie, et al., "Masked Autoencoders Are Scalable Vision Learners," in *CVPR*, 2022.
- [138] A. van den Oord, Y. Li, and O. Vinyals, "Representation Learning with Contrastive Predictive Coding," in *arXiv:1807.03748*, 2018.
- [139] J. Kaplan, S. McCandlish, T. Henighan, et al., "Scaling Laws for Neural Language Models," *arXiv:2001.08361*, 2020.
- [140] X. Zhai, A. Kolesnikov, N. Houlsby, et al., "Scaling Vision Transformers," in *CVPR*, 2022.
- [141] J. Hoffmann, S. Borgeaud, A. Mensch, et al., "Training Compute-Optimal Large Language Models," in *NeurIPS*, 2022.
- [142] C. G. Northcutt, A. Athalye, and J. Mueller, "Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks," in *NeurIPS Datasets and Benchmarks*, 2021.
- [143] B. Recht, R. Roelofs, L. Schmidt, et al., "Do ImageNet Classifiers Generalize to ImageNet?" In *ICML*, 2019.
- [144] N. Houlsby, A. Giurgiu, S. Jastrzebski, et al., "Parameter-Efficient Transfer Learning for NLP," in *ICML*, 2019.

- [145] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [146] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *RecSys*, 2019.

## 2.5 Thực hành

Chương này dễ đọc thụ động và khó nhớ, vì phần lớn nội dung là tên riêng. Hai mục dưới đây là phần đọc và phần tự vấn; phần chuyển nó thành thứ dùng được nằm ở khối mini project cuối chương, nơi bạn tự tạo ra bảng đo của chính mình thay vì tin bảng của người khác.

**Repo và SOTA (chốt tháng 9/2026).** Backbone dùng thật: [facebookresearch/dinov3](#) phủ dải từ khoảng 21 triệu tới vài tỉ tham số nhưng có giấy phép riêng, nên nếu cần điều khoản thoáng thì DINOv2 (Apache 2.0) là lựa chọn thay thế [47]; ConvNeXt-V2 và Swin lấy qua [timm](#); ngoài ra EVA-02 và SigLIP. Cài đặt tối giản để đọc chứ không để chạy production: [lucidrains/vit-pytorch](#) và [karpathy/nanoGPT](#). Attention nhanh: [Dao-AILab/flash-attention](#). Mô hình không gian trạng thái: [state-spaces/mamba](#). Danh sách tên mô hình này chốt tại tháng 9/2026 và sẽ cũ trong sáu tới mười hai tháng; ngược lại, các repo hạ tầng như [timm](#) hay COLMAP bền hơn nhiều, nên hãy phân bổ thời gian học theo đúng tỉ lệ đó.

**Câu hỏi kiểm tra hiểu.** Vì sao ViT cần nhiều dữ liệu hơn CNN, và ba cách bù lại — pretraining, augmentation mạnh, lai ghép — trả giá bằng loại tài nguyên nào? FlashAttention không giảm FLOPs, vậy nó nhanh hơn nhờ đâu, và trên phần cứng nào lợi ích của nó thu hẹp lại? Nếu ConvNeXt cho thấy recipe quan trọng hơn kiến trúc, điều đó thay đổi thứ tự bạn đọc một paper kiến trúc như thế nào?

### Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

| Thuật ngữ / câu hỏi                                                    | Trả lời                                                                                                                                                                                                                                             |
|------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ●●● Ba giả định của convolution                                        | Tính cục bộ: chỉ pixel lân cận liên quan trực tiếp. Chia sẻ trọng số, tức đẳng biến tịnh tiến. Và phân cấp: đặc trưng phức tạp ghép từ đặc trưng đơn giản. Cả chương là chuyện thêm hoặc bớt ba giả định này.                                       |
| ●● Trường tiếp nhận                                                    | Vùng pixel đầu vào ảnh hưởng tới một giá trị đầu ra. Trong CNN nó lớn dần theo chiều sâu; trong attention toàn cục nó bằng cả ảnh ngay từ tầng đầu.                                                                                                 |
| ●● Vì sao xếp chồng hai tầng $3 \times 3$ thay một tầng $5 \times 5$ ? | Cùng trường tiếp nhận, nhưng tốn $18C^2$ tham số thay vì $25C^2$ , và có thêm một lần phi tuyến ở giữa. Một trong ít phép đổi tốt ở cả ba mặt.                                                                                                      |
| ●● Vì sao nút thắt $1 \times 1$ rẻ hơn gần chín lần?                   | Vì phần đắt nhất của conv là tích của kênh vào, kênh ra và diện tích cửa sổ. Giảm kênh xuống 64 rồi mới làm $3 \times 3$ thì $k^2$ chỉ nhân với một số nhỏ: 589 824 phép nhân còn 69 632.                                                           |
| ●●● Kết nối tắt                                                        | Đường cộng $y = x + F(x)$ đưa ánh xạ đồng nhất thành phương án mặc định. Jacobian thành $I + \partial F / \partial x$ , nên tích qua nhiều khối luôn còn một thành phần bằng 1. Giá phải trả: phải giữ $x$ sống tới điểm cộng, nên tốn thêm bộ nhớ. |
| ●● Depthwise separable                                                 | Tách conv thành một bước không gian chạy riêng từng kênh, rồi một bước $1 \times 1$ trộn kênh. Chi phí còn $1/C_{\text{out}} + 1/k^2$ lần, khoảng 0,115 với $k = 3$ và $C_{\text{out}} = 256$ .                                                     |

| Thuật ngữ / câu hỏi                                               | Trả lời (tiếp)                                                                                                                                                                                                                      |
|-------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ●●● Giảm 8,7 lần FLOPs mà độ trễ gần như không đổi — nguyên nhân? | Bước depthwise đọc mỗi phần tử để làm vòn vẹn chín phép nhân, nên nó nghe ở bằng thông chứ không ở khả năng tính toán. Phân biệt với ngheh tính toán: đo thông lượng bộ nhớ, nếu đã sát trần thì giảm FLOPs vô ích.                 |
| ● Dilated convolution                                             | Giãn khoảng cách giữa các điểm lấy mẫu, nên trường tiếp nhận tăng mà không phải giảm độ phân giải. Sinh ra cho segmentation; hồng thành hoa văn rằng lược nếu xếp chồng nhiều tầng cùng hệ số giãn.                                 |
| ●● Cổng của LSTM                                                  | Cơ chế cho trạng thái ô đi qua theo dạng cộng:<br>$c_t = f_t \odot c_{t-1} + \dots$ Khi cổng quên mở gần 1, gradient chảy qua mà không bị nhân với ma trận trọng số — đúng mẹo của kết nối tắt, chỉ khác là hệ số học được.         |
| ●●● Attention                                                     | Mỗi vị trí sinh truy vấn, mọi vị trí sinh khoá và giá trị; đầu ra là tổ hợp có trọng số của mọi vị trí. Đường đi thông tin bằng 1 thay vì $T$ , đổi lại chi phí và bộ nhớ $O(T^2)$ .                                                |
| ●●● Vì sao attention chia cho $\sqrt{d}$ ?                        | Tích vô hướng $d$ chiều có độ lệch chuẩn $\sqrt{d}$ , tức 8 khi $d = 64$ . Softmax trên logit lệch chuẩn 8 bão hoà thành gần one-hot, và ở vùng đó đạo hàm bằng 0.                                                                  |
| ●● Mã hoá vị trí                                                  | Thông tin vị trí được cấp từ bên ngoài, vì phép attention bất biến với hoán vị. Bốn họ: sin tuyệt đối, học được, tương đối, và RoPE. Loại tuyệt đối ngoại suy kém ra ngoài độ dài đã thấy khi huấn luyện.                           |
| ●● Window attention                                               | Chỉ cho attention trong một cửa sổ cố định, rồi dịch cửa sổ ở tầng sau để thông tin rò qua ranh giới. Chi phí thành tuyến tính theo $T$ ; giá phải trả là quan hệ xa cần nhiều tầng mới nối được.                                   |
| ● FlashAttention                                                  | Chia ô và tính softmax trực tuyến để không bao giờ dựng ma trận $T \times T$ trong HBM. Số FLOPs không đổi và độ phức tạp tiệm cận cũng không đổi; cái giảm là số vòng đọc ghi bộ nhớ ngoài.                                        |
| ●●● ViT                                                           | Cắt ảnh thành patch, coi mỗi patch là một token, rồi đưa vào transformer chuẩn. Nó vứt bỏ tính cục bộ và đẳng biến tịnh tiến, nên thua CNN khi dữ liệu ít và vượt khi dữ liệu rất nhiều.                                            |
| ●●● Vì sao ViT cần nhiều dữ liệu hơn CNN?                         | Vì giả định không cài vào kiến trúc thì phải mua bằng mẫu. Ba cách trả: pretrain trên tập rất lớn, chưng cất từ thầy CNN cộng augmentation mạnh, hoặc lắp lại giả định bằng kiến trúc phân cấp.                                     |
| ●● Đổi độ phân giải đầu vào rồi ViT tẻ hẳn — nguyên nhân?         | Quên nội suy bằng mã hoá vị trí: số token đổi nhưng bảng vẫn kích thước cũ. Phân biệt với thiếu fine-tune: lỗi này làm điểm tụt ngay cả khi bạn fine-tune lại đủ lâu.                                                               |
| ●● ViT phẳng hết bộ nhớ ở dự đoán dày stride 4 — nguyên nhân?     | Số token là $128^2 = 16\,384$ , nên ma trận attention chiếm khoảng 537 MB cho một đầu, một tầng. Lời sửa là chuyển sang biến thể cửa sổ hoặc phân cấp.                                                                              |
| ●●● ConvNeXt                                                      | Một CNN được hiện đại hoá bằng chính công thức huấn luyện của ViT, và nó bắt kịp ViT cùng hạng. Bài học: phần lớn chênh lệch từng được gán cho kiến trúc thực ra thuộc về recipe.                                                   |
| ●●● Backbone, neck, head                                          | Backbone trích đặc trưng ở nhiều stride, neck hợp nhất chúng theo chiều từ sâu về nông, head sinh đầu ra của bài toán. Đây là cách đọc một repo detection nhanh nhất, và là lý do ViT phẳng khó lắp: nó chỉ sinh một mức đặc trưng. |

●●● phải thuộc    ●● nên nắm    ● biết là có

**I Kiến thức này đang sống ở đâu** (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)



| Công trình / hệ thống           | Dùng ý tưởng nào của chương                                                                                                                                    | Ghi chú                                                                                       |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <a href="#">timm</a>            | Kho backbone chuẩn của ngành: ConvNeXt, Swin, EVA, ResNet hiện đại hoá — và là nơi thấy rõ nhất rằng cùng kiến trúc với recipe khác nhau cho điểm số khác nhau | Hạ tầng bền; đáng đầu tư hơn bất kỳ tên mô hình nào                                           |
| DINOv2 và DINOv3                | Backbone ViT trả khoản nợ giả định bằng pretraining tự giám sát ở quy mô rất lớn — đúng cách thứ nhất trong ba cách ở mạch 3                                   | DINOv2 giấy phép Apache 2.0; DINOv3 giấy phép riêng, chốt 9/2026                              |
| Swin và các biến thể phân cấp   | Lắp lại tính cục bộ và đa tỉ lệ mà ViT phẳng đánh mất, để sinh ra bốn mức đặc trưng mà FPN cần                                                                 | Vẫn là backbone mặc định cho nhiều hệ dự đoán dày                                             |
| <a href="#">flash-attention</a> | Tấn công chi phí bậc hai ở tầng hệ thống: không dựng ma trận $T \times T$ trong HBM, giữ nguyên FLOPs                                                          | Đã thành mặc định trong phần lớn thư viện transformer                                         |
| ConvNeXt-V2                     | Bằng chứng đang sống rằng convolution vẫn cạnh tranh khi được huấn luyện bằng recipe hiện đại                                                                  | Đọc kèm bảng ablation của nó là cách nhanh nhất để thấy từng thành phần recipe đáng bao nhiêu |

MINI PROJECT SLAM · Chọn backbone cho bộ mô tả đặc trưng dưới ngân sách độ trễ của Orin · 1–2 ngày

|                 |                                                                                                                                                                                                                                                                                                                                                                                                |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Mục tiêu</b> | biến câu hỏi “backbone nào tốt hơn” thành một quyết định có bằng chứng, dưới đúng ràng buộc của bạn. Và thấy tận mắt rằng thứ hạng theo FLOPs có thể ngược thứ hạng theo độ trễ thật.                                                                                                                                                                                                          |
| <b>Dữ liệu</b>  | dùng lại tập cặp ghép đã gán nhãn từ mini project chương 12, nhưng thay đầu vào thủ công bằng miếng ảnh $32 \times 32$ cắt quanh mỗi góc ORB trên EuRoC. Ba backbone cùng hạng lấy từ <a href="#">timm</a> : một CNN cổ điển (ResNet-18), một CNN hiện đại hoặc hướng thiết bị nhúng (ConvNeXt-T hoặc MobileNetV3), và một ViT phân cấp (Swin-T).                                              |
| <b>Các bước</b> | fine-tune cả ba ở cùng ngân sách epoch và cùng công thức huấn luyện — đây là điều kiện bắt buộc, nếu không thì bạn đang đo recipe chứ không đo kiến trúc; đo trên Jetson Orin ở batch bằng 1, tức đúng chế độ mà SLAM chạy, sau khi khoá xung nhịp, lấy cả trung vị lẫn phân vị 95; lập bảng sáu cột gồm độ chính xác ghép, số tham số, FLOPs, độ trễ trung vị, độ trễ $p_{95}$ , bộ nhớ đỉnh. |
| <b>Xong khi</b> | bạn chỉ ra được backbone nào thắng ở một ngân sách cụ thể, chẳng hạn 8 ms mỗi khung hình. Bạn giải thích thứ hạng bằng hai đại lượng: trường tiếp nhận so với kích thước miếng ảnh, và cường độ số học so với ngưỡng roofline của Orin. Và bạn chỉ ra được ít nhất một cặp trong bảng có thứ hạng FLOPs ngược thứ hạng độ trễ.                                                                 |
| <b>Nối với</b>  | B/11 cho mô hình roofline dùng để giải thích cột độ trễ; C/12/03 cho việc giữ công thức huấn luyện ngang nhau giữa ba lần chạy; C/14 cho bước tiếp theo là lắp backbone đã chọn vào một head thật.                                                                                                                                                                                             |

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- Vì cửa sổ  $3 \times 3$  không phải một giới hạn mà là một *giả định*: quan hệ gần quan trọng hơn quan hệ xa, và mẫu hình có ý nghĩa như nhau ở mọi vị trí. Giả định đó đúng với ảnh tự nhiên, nên nó thay thế cho dữ liệu. Attention toàn cục không giả định gì về khoảng cách, nên phải học quan hệ đó từ mẫu và trả thêm chi phí  $O(T^2)$ .
- Gradient descent không tìm ra nghiệm “các tầng thừa làm ánh xạ đồng nhất”, dù nghiệm đó tồn tại. Đây là vấn đề tối ưu hoá, không phải sức chứa — và kết nối tắt sửa nó bằng cách biến ánh xạ đồng nhất thành phương án mặc định, đồng thời để lại một thành phần bằng 1 trong tích các Jacobian.
- Vì phần depthwise đọc mỗi phần tử đầu vào để làm vòn vện  $k^2 = 9$  phép nhân, nên cường độ số học của nó rất thấp và kernel rơi vào phía nghèo băng thông bộ nhớ. Giảm FLOPs của một kernel vốn đã nghèo ở bộ nhớ thì gần như không giảm thời gian.



- A4. Cải tiến công thức huấn luyện — optimizer, lịch học, số epoch, augmentation — nhiều hơn là cải tiến kiến trúc. Điều đó không xóa bỏ tiến bộ về kiến trúc, nhưng nó nói rằng mọi bảng so sánh không kiểm soát recipe đều không tách được hai nguồn ấy, nên câu hỏi đầu tiên khi đọc bảng phải là câu hỏi về recipe.
- A5. Không có gì trong mô hình thay đổi — thứ thay đổi là *ai trả tiền cho các giả định*. Với ít dữ liệu, giả định cài sẵn của convolution là món quà miễn phí; với rất nhiều dữ liệu, chính giả định đó thành trần, còn mô hình ít giả định thì học ra được các quan hệ mà convolution cấm nó biểu diễn.

### NÓI LẠI THẬT ĐƠN GIẢN

Giả sử bạn thuê một người thợ mới và giao việc phân loại đồ vật qua ảnh. Bạn có hai cách. Cách thứ nhất là dặn trước thật nhiều quy tắc: hai chỗ cạnh nhau trong ảnh thường liên quan tới nhau; một cái cốc vẫn là cái cốc dù nó nằm ở góc nào của bàn; nhìn nét nhỏ trước rồi mới ghép thành hình lớn. Cách thứ hai là không dặn gì cả, chỉ đưa cho họ thật nhiều ví dụ và để họ tự rút ra. Cả chương là câu chuyện về việc chọn giữa hai cách đó, và phát hiện quan trọng nhất là chúng thay thế được cho nhau. Ai chỉ có vài nghìn tấm ảnh thì dặn trước nhiều quy tắc là món hời, vì quy tắc làm thay phần việc mà ví dụ đáng ra phải làm. Ai có hàng trăm triệu tấm thì chính những quy tắc ấy lại thành xiềng: chúng cấm người thợ nhận ra các mối liên hệ mà bạn không nghĩ tới khi dặn. Có hai cái giá phải nhớ. Một là bỏ bớt quy tắc thì phải trả bằng ví dụ, hoặc bằng việc mượn một người thợ đã được luyện sẵn ở nơi khác. Hai là “rẻ hơn trên giấy” không có nghĩa là “nhanh hơn trong xương”: một cách làm cắt được tám phần chín số phép tính vẫn có thể chạy chậm y như cũ, vì thời gian thật đi vào việc lấy vật liệu ra khỏi kho chứ không vào việc gia công. Và bất ngờ cuối cùng: phần lớn khác biệt giữa hai người thợ hoá ra không nằm ở quy tắc họ được dặn, mà ở cách họ được luyện tập.

*Một câu: Quy tắc dặn trước là khoản vay trả trước cho ví dụ — ít ví dụ thì nên dặn nhiều, nhiều ví dụ thì chính lời dặn thành xiềng, và phần lớn chênh lệch bạn thấy trên bảng xếp hạng lại đến từ cách luyện tập chứ không từ lời dặn.*

## 2.6 Kết nối

Chương này nhận đầu vào từ [C/12-co-che-huan-luyen](#): không có bộ máy huấn luyện thì các kiến trúc ở đây chỉ là sơ đồ khối. Nó cũng nhận từ [B/10-thiet-ke-kien-truc](#), nơi trực đánh đổi giả định–dữ liệu được phát biểu lần đầu. Xuôi xuống, nó cấp backbone cho [C/14-bai-toan-cv-loi](#), [C/16-thi-giac-3d](#) và [C/17-mo-hinh-sinh](#), và đặt sẵn câu hỏi mà [C/18-pretraining-va-foundation](#) trả lời: nếu giả định phải mua bằng dữ liệu, thì mua ở đâu cho rẻ nhất. Hai nhánh ngang nên đọc ngay sau chương này. Thứ nhất là [B/11-gioi-han-tinh-toan](#), để biến các phép tính FLOPs ở đây thành dự đoán độ trễ có căn cứ. Thứ hai là [E/24-thi-nghiem-va-ablation](#), để không bị chính các bảng so sánh trong chương này dẫn tới kết luận quá mạnh.

### Tổng kết chương

Ta đã đi qua ba mạch và về cùng một chỗ. Mạch 1 cho thấy dòng CNN là một chuỗi gỡ nút thắt, mỗi bước đẩy độ khó sang một trục khác, và mất xích cuối — ConvNeXt — buộc ta xét lại toàn bộ các so sánh trước đó. Mạch 2 cho thấy attention mua được đường đi thông tin ngắn nhất có thể bằng chi phí bậc hai, và ba hướng chống chi phí đó nằm ở ba tầng trừu tượng khác nhau. Mạch 3 cho thấy khi vứt bỏ giả định thị giác, cái giá hiện ra rất sạch dưới dạng lượng dữ liệu cần có — và có ít nhất ba loại tiền để trả khoản đó.

Còn bỏ ngỏ ba chỗ. Thứ nhất, chương này chỉ nói về *trích đặc trưng*; việc biến đặc trưng thành đầu ra của một bài toán cụ thể — hộp, mặt nạ, khoảng cách — là nội dung của [C/14-bai-toan-cv-loi](#). Thứ hai, cách rẻ nhất để trả khoản nợ giả định là dùng một mô hình đã pretrain, và toàn bộ chủ đề đó nằm ở [C/18-pretraining-va-foundation](#). Thứ ba, mô hình không gian trạng thái là hướng đang chuyển động nhanh nhất trong ba mạch. Chương này đánh giá rằng nó chưa thay thế attention trong thị giác. Đó là đánh giá tính tới tháng 9/2026, không phải một kết luận bền. Đọc tiếp [C/14-bai-toan-cv-loi](#) với câu hỏi: khi lắp một head cụ thể lên các backbone này, ràng buộc nào của backbone sẽ trở thành ràng buộc của cả hệ thống?

### ■ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Trục duy nhất của chương là “cài sẵn nhiều giả định, cần ít dữ liệu” đổi lại “cài ít giả định, cần nhiều dữ liệu”. Hãy đặt ResNet, Swin và ViT phẳng lên trục đó, rồi nói cái gì trong mô hình thực sự thay đổi khi dữ liệu tăng, khiến thứ hạng đảo chiều?
2. Kết nối tắt của ResNet, cổng của LSTM và chuẩn hoá theo tầng cùng phục vụ một mục tiêu đã đặt ra ở [C/12-co-che-huan-luyen](#). Mục tiêu đó là gì, và vì sao attention giải quyết nó theo một cách khác hẳn ba thứ kia?
3. Nếu ConvNeXt đúng khi nói recipe quan trọng hơn kiến trúc, thì các bảng so sánh trong chính chương này còn đọc được ở mức nào, và bạn đòi thêm cột nào trước khi tin một dòng?
4. FLOPs xuất hiện ở cả ba mục nhưng không mục nào coi nó là độ trễ. MobileNet, FlashAttention và attention cửa sổ mỗi cái phá vỡ quan hệ FLOPs–độ trễ theo kiểu riêng nào, và đại lượng chung nào giải thích được cả ba?
5. Bạn cần một backbone cho ảnh rất lớn, vật thể chỉ vài chục pixel, và ngân sách bộ nhớ chặt. Ràng buộc nào loại bớt phương án trước, và vì sao câu trả lời đôi khi bạn có hay không có một mô hình pretrain?
6. Cả ba mục đều kể lịch sử như một chuỗi gỡ nút thắt. Nếu bạn tin nút thắt hiện tại nằm ở dữ liệu chứ không ở kiến trúc, thì quan sát nào trong chính chương này ủng hộ niềm tin đó, và quan sát nào phản bác?

- [1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [2] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [3] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *AISTATS*, 2010.
- [4] K. He, X. Zhang, S. Ren, et al., “Delving Deep into Rectifiers,” in *ICCV*, 2015.
- [5] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [6] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [7] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [8] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [9] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [10] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [11] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” in *ICLR*, 2017.
- [12] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [13] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [14] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [15] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [16] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [17] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [18] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [19] G. Huang, Y. Sun, Z. Liu, et al., “Deep Networks with Stochastic Depth,” in *ECCV*, 2016.
- [20] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [21] H. Zhang, M. Cisse, Y. N. Dauphin, et al., “mixup: Beyond Empirical Risk Minimization,” in *ICLR*, 2018.
- [22] S. Yun, D. Han, S. J. Oh, et al., “CutMix: Regularization Strategy to Train Strong Classifiers,” in *ICCV*, 2019.
- [23] E. D. Cubuk, B. Zoph, J. Shlens, et al., “RandAugment: Practical Automated Data Augmentation,” in *CVPR Workshops*, 2020.

- [24] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [25] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [26] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *NeurIPS*, 2012.
- [27] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in *ICLR*, 2015.
- [28] C. Szegedy, W. Liu, et al., “Going Deeper with Convolutions,” in *CVPR*, 2015.
- [29] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [30] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [31] G. Huang, Z. Liu, L. van der Maaten, et al., “Densely Connected Convolutional Networks,” in *CVPR*, 2017.
- [32] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [33] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [34] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [35] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [36] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [37] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.
- [38] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [39] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [40] J. Su, M. Ahmed, Y. Lu, et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, vol. 568, 2024.
- [41] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” *arXiv:2312.00752*, 2023.
- [42] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [43] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.
- [44] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [45] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [46] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [47] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [48] R. Girshick, J. Donahue, T. Darrell, et al., “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation,” in *CVPR*, 2014.

- [49] R. Girshick, "Fast R-CNN," in *ICCV*, 2015.
- [50] S. Ren, K. He, R. Girshick, et al., "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in *NeurIPS*, 2015.
- [51] T.-Y. Lin, P. Dollár, R. Girshick, et al., "Feature Pyramid Networks for Object Detection," in *CVPR*, 2017.
- [52] K. He, G. Gkioxari, P. Dollár, et al., "Mask R-CNN," in *ICCV*, 2017.
- [53] D. Bolya, S. Foley, J. Hays, et al., "TIDE: A General Toolbox for Identifying Object Detection Errors," in *ECCV*, 2020.
- [54] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [55] J. Redmon, S. Divvala, R. Girshick, et al., "You Only Look Once: Unified, Real-Time Object Detection," in *CVPR*, 2016.
- [56] W. Liu, D. Anguelov, D. Erhan, et al., "SSD: Single Shot MultiBox Detector," in *ECCV*, 2016.
- [57] T.-Y. Lin, P. Goyal, R. Girshick, et al., "Focal Loss for Dense Object Detection," in *ICCV*, 2017.
- [58] Z. Tian, C. Shen, H. Chen, et al., "FCOS: Fully Convolutional One-Stage Object Detection," in *ICCV*, 2019.
- [59] X. Zhou, D. Wang, and P. Krähenbühl, "Objects as Points," *arXiv:1904.07850*, 2019.
- [60] S. Zhang, C. Chi, Y. Yao, et al., "Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection," in *CVPR*, 2020.
- [61] Z. Ge, S. Liu, Z. Li, et al., "OTA: Optimal Transport Assignment for Object Detection," in *CVPR*, 2021.
- [62] N. Bodla, B. Singh, R. Chellappa, et al., "Soft-NMS: Improving Object Detection with One Line of Code," in *ICCV*, 2017.
- [63] X. Zhu, W. Su, L. Lu, et al., "Deformable DETR: Deformable Transformers for End-to-End Object Detection," in *ICLR*, 2021.
- [64] H. Zhang, F. Li, S. Liu, et al., "DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection," in *ICLR*, 2023.
- [65] J. Long, E. Shelhamer, and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," in *CVPR*, 2015.
- [66] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *MICCAI*, 2015.
- [67] L.-C. Chen, G. Papandreou, I. Kokkinos, et al., "DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs," *IEEE TPAMI*, vol. 40, no. 4, pp. 834–848, 2018.
- [68] E. Xie, W. Wang, Z. Yu, et al., "SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers," in *NeurIPS*, 2021.
- [69] B. Cheng, I. Misra, A. G. Schwing, et al., "Masked-Attention Mask Transformer for Universal Image Segmentation," in *CVPR*, 2022.
- [70] A. Kirillov, K. He, R. Girshick, et al., "Panoptic Segmentation," in *CVPR*, 2019.
- [71] A. Kirillov, E. Mintun, N. Ravi, et al., "Segment Anything," in *ICCV*, 2023.
- [72] K. Sun, B. Xiao, D. Liu, et al., "Deep High-Resolution Representation Learning for Human Pose Estimation," in *CVPR*, 2019.

- [73] Z. Cao, T. Simon, S.-E. Wei, et al., “Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields,” in *CVPR*, 2017.
- [74] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [75] J. Deng, J. Guo, N. Xue, et al., “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *CVPR*, 2019.
- [76] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [77] K. Xu, J. Ba, R. Kiros, et al., “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention,” in *ICML*, 2015.
- [78] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *ICLR*, 2015.
- [79] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [80] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [81] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [82] K. Simonyan and A. Zisserman, “Two-Stream Convolutional Networks for Action Recognition in Videos,” in *NeurIPS*, 2014.
- [83] J. Carreira and A. Zisserman, “Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset,” in *CVPR*, 2017.
- [84] D. Tran, H. Wang, L. Torresani, et al., “A Closer Look at Spatiotemporal Convolutions for Action Recognition,” in *CVPR*, 2018.
- [85] C. Feichtenhofer, H. Fan, J. Malik, et al., “SlowFast Networks for Video Recognition,” in *ICCV*, 2019.
- [86] A. Dosovitskiy, P. Fischer, E. Ilg, et al., “FlowNet: Learning Optical Flow with Convolutional Networks,” in *ICCV*, 2015.
- [87] D. Sun, X. Yang, M.-Y. Liu, et al., “PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume,” in *CVPR*, 2018.
- [88] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.
- [89] A. Bewley, Z. Ge, L. Ott, et al., “Simple Online and Realtime Tracking,” in *ICIP*, 2016.
- [90] Y. Zhang, P. Sun, Y. Jiang, et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *ECCV*, 2022.
- [91] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [92] K. Bernardin and R. Stiefelhagen, “Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics,” *EURASIP J. Image and Video Processing*, 2008.
- [93] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [94] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [95] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.



- [96] S. Wang, V. Leroy, Y. Cabon, et al., "DUST3R: Geometric 3D Vision Made Easy," in *CVPR*, 2024.
- [97] D. Eigen, C. Puhrsch, and R. Fergus, "Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network," in *NeurIPS*, 2014.
- [98] R. Ranftl, K. Lasinger, D. Hafner, et al., "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer," *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [99] J. Sturm, N. Engelhard, F. Endres, et al., "A Benchmark for the Evaluation of RGB-D SLAM Systems," in *IROS*, 2012.
- [100] C. R. Qi, H. Su, K. Mo, et al., "PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation," in *CVPR*, 2017.
- [101] C. R. Qi, L. Yi, H. Su, et al., "PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space," in *NeurIPS*, 2017.
- [102] A. H. Lang, S. Vora, H. Caesar, et al., "PointPillars: Fast Encoders for Object Detection from Point Clouds," in *CVPR*, 2019.
- [103] T. Müller, A. Evans, C. Schied, et al., "Instant Neural Graphics Primitives with a Multiresolution Hash Encoding," *ACM TOG*, vol. 41, no. 4, 2022.
- [104] B. Kerbl, G. Kopanas, T. Leimkühler, et al., "3D Gaussian Splatting for Real-Time Radiance Field Rendering," *ACM TOG*, vol. 42, no. 4, 2023.
- [105] R. Zhang, P. Isola, A. A. Efros, et al., "The Unreasonable Effectiveness of Deep Features as a Perceptual Metric," in *CVPR*, 2018.
- [106] D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," in *ICLR*, 2014.
- [107] I. Goodfellow, J. Pouget-Abadie, M. Mirza, et al., "Generative Adversarial Nets," in *NeurIPS*, 2014.
- [108] T. Karras, S. Laine, and T. Aila, "A Style-Based Generator Architecture for Generative Adversarial Networks," in *CVPR*, 2019.
- [109] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," in *NeurIPS*, 2020.
- [110] Y. Song, J. Sohl-Dickstein, D. P. Kingma, et al., "Score-Based Generative Modeling Through Stochastic Differential Equations," in *ICLR*, 2021.
- [111] J. Johnson, A. Alahi, and L. Fei-Fei, "Perceptual Losses for Real-Time Style Transfer and Super-Resolution," in *ECCV*, 2016.
- [112] P. Isola, J.-Y. Zhu, T. Zhou, et al., "Image-to-Image Translation with Conditional Adversarial Networks," in *CVPR*, 2017.
- [113] L. A. Gatys, A. S. Ecker, and M. Bethge, "Image Style Transfer Using Convolutional Neural Networks," in *CVPR*, 2016.
- [114] X. Huang and S. Belongie, "Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization," in *ICCV*, 2017.
- [115] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," in *NeurIPS Deep Learning Workshop*, 2015.
- [116] J. Song, C. Meng, and S. Ermon, "Denoising Diffusion Implicit Models," in *ICLR*, 2021.
- [117] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, et al., "Flow Matching for Generative Modeling," in *ICLR*, 2023.
- [118] J. Ho and T. Salimans, "Classifier-Free Diffusion Guidance," *arXiv:2207.12598*, 2022.
- [119] R. Rombach, A. Blattmann, D. Lorenz, et al., "High-Resolution Image Synthesis with Latent Diffusion Models," in *CVPR*, 2022.
- [120] W. Peebles and S. Xie, "Scalable Diffusion Models with Transformers," in *ICCV*, 2023.

- [121] L. Zhang, A. Rao, and M. Agrawala, "Adding Conditional Control to Text-to-Image Diffusion Models," in *ICCV*, 2023.
- [122] Y. Blau and T. Michaeli, "The Perception-Distortion Tradeoff," in *CVPR*, 2018.
- [123] C. Ledig, L. Theis, F. Huszár, et al., "Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network," in *CVPR*, 2017.
- [124] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., "Image Quality Assessment: From Error Visibility to Structural Similarity," *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [125] M. Heusel, H. Ramsauer, T. Unterthiner, et al., "GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium," in *NeurIPS*, 2017.
- [126] T. Brooks, B. Mildenhall, T. Xue, et al., "Unprocessing Images for Learned Raw Denoising," in *CVPR*, 2019.
- [127] A. Abdelhamed, S. Lin, and M. S. Brown, "A High-Quality Denoising Dataset for Smartphone Cameras," in *CVPR*, 2018.
- [128] J. Deng, W. Dong, R. Socher, et al., "ImageNet: A Large-Scale Hierarchical Image Database," in *CVPR*, 2009.
- [129] A. Radford, J. W. Kim, C. Hallacy, et al., "Learning Transferable Visual Models from Natural Language Supervision," in *ICML*, 2021.
- [130] J. Li, D. Li, S. Savarese, et al., "BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models," in *ICML*, 2023.
- [131] H. Liu, C. Li, Q. Wu, et al., "Visual Instruction Tuning," in *NeurIPS*, 2023.
- [132] S. Kornblith, J. Shlens, and Q. V. Le, "Do Better ImageNet Models Transfer Better?" In *CVPR*, 2019.
- [133] T. Chen, S. Kornblith, M. Norouzi, et al., "A Simple Framework for Contrastive Learning of Visual Representations," in *ICML*, 2020.
- [134] K. He, H. Fan, Y. Wu, et al., "Momentum Contrast for Unsupervised Visual Representation Learning," in *CVPR*, 2020.
- [135] J.-B. Grill, F. Strub, F. Altché, et al., "Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning," in *NeurIPS*, 2020.
- [136] M. Caron, H. Touvron, I. Misra, et al., "Emerging Properties in Self-Supervised Vision Transformers," in *ICCV*, 2021.
- [137] K. He, X. Chen, S. Xie, et al., "Masked Autoencoders Are Scalable Vision Learners," in *CVPR*, 2022.
- [138] A. van den Oord, Y. Li, and O. Vinyals, "Representation Learning with Contrastive Predictive Coding," in *arXiv:1807.03748*, 2018.
- [139] J. Kaplan, S. McCandlish, T. Henighan, et al., "Scaling Laws for Neural Language Models," *arXiv:2001.08361*, 2020.
- [140] X. Zhai, A. Kolesnikov, N. Houlsby, et al., "Scaling Vision Transformers," in *CVPR*, 2022.
- [141] J. Hoffmann, S. Borgeaud, A. Mensch, et al., "Training Compute-Optimal Large Language Models," in *NeurIPS*, 2022.
- [142] C. G. Northcutt, A. Athalye, and J. Mueller, "Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks," in *NeurIPS Datasets and Benchmarks*, 2021.
- [143] B. Recht, R. Roelofs, L. Schmidt, et al., "Do ImageNet Classifiers Generalize to ImageNet?" In *ICML*, 2019.
- [144] N. Houlsby, A. Giurgiu, S. Jastrzebski, et al., "Parameter-Efficient Transfer Learning for NLP," in *ICML*, 2019.

- [145] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [146] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *RecSys*, 2019.

## Bài toán CV lõi: nhận dạng, phát hiện, phân vùng (Core Vision Tasks: Recognition, Detection, Segmentation)

**PHỤ THUỘC** C/13-kien-truc-backbone (backbone, stride, attention); B/09-thiet-ke-ham-mat-mat (loss đa nhiệm, mất cân bằng); B/07-chia-du-lieu-va-danh-gia (mAP, mIoU, đánh giá theo lát). [T3]

### TÓM TẮT

Chương này lắp các “đầu” bài toán lên trên backbone của chương trước, và đọc toàn bộ lịch sử detection lẫn segmentation theo một trục duy nhất: từng bước nuốt các heuristic bên ngoài vào bên trong mạng, tức biến một khối không nhận gradient thành một khối được tối ưu cho đúng mục tiêu cuối. Đọc xong bạn phát biểu được một bài toán thị giác mới dưới dạng đúng, chọn được họ mô hình từ ràng buộc chứ không từ bảng xếp hạng, và đọc được một bảng so sánh detector với đúng mức hoài nghi. Nếu chỉ nhớ một điều: những chi tiết không được viết trong abstract — cách gán mẫu, tỉ lệ mẫu dương/âm, lịch huấn luyện — thường giải thích chênh lệch kết quả nhiều hơn hình vẽ kiến trúc ở trang đầu.

### KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình và huấn luyện** — mô hình là một hàm có hàng triệu tham số biến ảnh thành dự đoán; huấn luyện là chỉnh dần các tham số đó để dự đoán khớp nhãn, bằng cách đi ngược hướng *gradient* của hàm mất mát.
- **hàm mất mát (loss function)** — một số duy nhất đo mức sai của dự đoán. Đây là thứ định nghĩa “đúng” nghĩa là gì, nên đối hàm mất mát là đối bài toán chứ không phải đối mô hình.
- **backbone, neck, head** — backbone biến ảnh thành lưới đặc trưng; neck trộn các lưới ở nhiều độ phân giải; head là phần nhỏ phía trên sinh ra đầu ra của một bài toán cụ thể. Ba bài toán khác nhau thường dùng chung backbone và chỉ khác head.
- **chân trị (ground truth)** — đáp án do người gán cho mỗi ảnh — khung, mask, hay nhãn lớp. Mô hình chỉ học được đúng những gì nhãn nói, kể cả khi nhãn sai.
- **IoU** — diện tích giao chia diện tích hợp của hai vùng; bằng 1 khi trùng khít, 0 khi rời nhau. Nó là đơn vị đo cơ bản của cả detection lẫn segmentation.
- **mAP và mIoU** — hai thước đo tổng: mAP cho detection (diện tích dưới đường precision-recall, trung bình theo lớp và theo ngưỡng IoU), mIoU cho segmentation (IoU trung bình theo lớp).
- **heuristic** — một quy tắc viết tay, không có tham số học được, do đó không nhận gradient và không tự chỉnh theo lỗi cuối. Ví dụ: NMS, ngưỡng IoU khi gán mẫu, công thức chọn tầng của FPN.

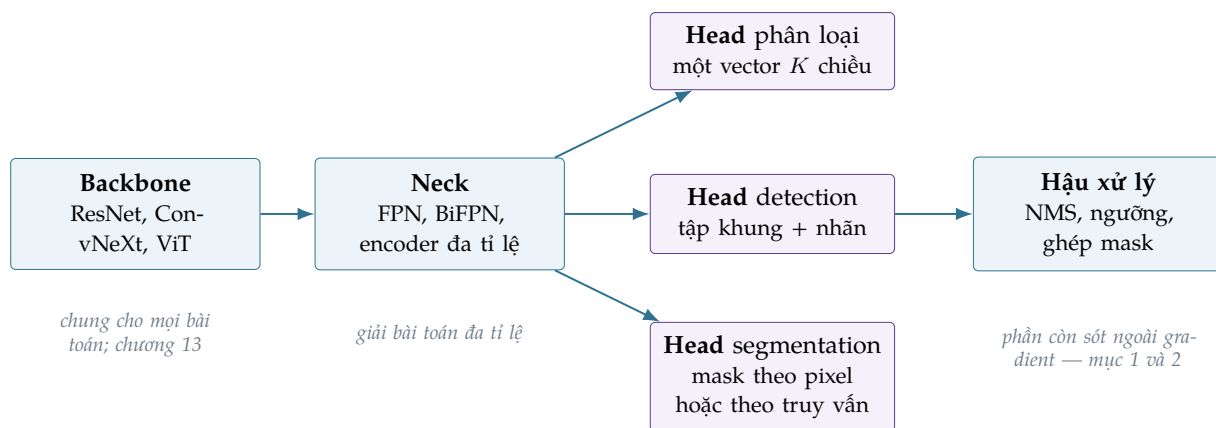
### CÂU HỎI MỞ ĐẦU

- Q1. Vì sao thay đúng một quy tắc gán mẫu — một hàm chục dòng nằm trong code, không có trong abstract — lại đổi kết quả nhiều hơn thay cả backbone?
- Q2. R-CNN chạy backbone hai nghìn lần mỗi ảnh, Fast R-CNN chạy một lần. Vì sao bản một lần lại *chính xác hơn* chứ không chỉ nhanh hơn?
- Q3. NMS chỉ là một vòng lặp xoá khung trùng. Vì sao suốt mười năm không ai “nuốt” được nó vào mạng, trong khi selective search thì nuốt được ngay?

- Q4.** Nếu bỏ đúng một phép làm tròn toạ độ mà chất lượng mask nhảy vọt, điều đó nói gì về việc bạn nên tin phần nào của một paper?
- Q5.** Semantic, instance và panoptic segmentation từng là ba code base riêng. Việc chúng gộp được vào một mô hình là dấu hiệu của cái gì?

Chương trước cho bạn cái máy trích đặc trưng; chương này hỏi phải gắn cái gì lên trên nó để trả lời những câu hỏi mà thế giới thật đặt ra. Ba câu hỏi kinh điển — *ảnh này là gì, có những vật gì ở đâu, mỗi pixel thuộc về cái gì* — trông như ba mức độ chi tiết tăng dần của cùng một bài toán. Nhưng chỉ câu thứ nhất có đầu ra kích thước cố định, và chính điều đó chia đôi chương này: mọi khó khăn kỹ thuật của detection và segmentation đều bắt nguồn từ việc đầu ra là một *tập hợp* có lực lượng thay đổi, thứ mà bộ máy gradient không biết chấm điểm trực tiếp.

Lịch sử của lĩnh vực này, đọc theo đúng trục, là một câu chuyện duy nhất: **từng bước nuốt các heuristic bên ngoài vào bên trong mạng**. Selective search bị RPN nuốt, thiết kế đa tỉ lệ thủ công bị FPN nuốt, lấy mẫu cứng bị focal loss nuốt, NMS bị Hungarian matching nuốt. Mỗi lần nuốt, một khối vốn không nhận được gradient trở thành một khối được tối ưu cho đúng mục tiêu cuối, và gần như lần nào hệ thống cũng vừa nhanh hơn vừa chính xác hơn. Đọc theo mạch đó, bạn không chỉ nhớ được lịch sử mà còn đoán được bước tiếp theo của bất kỳ dòng mô hình nào: hãy liệt kê những gì vẫn còn nằm ngoài.



**Hình 3.1.** Giải phẫu chung của một hệ thống thị giác hiện đại. Ba đầu bài toán khác nhau dùng chung backbone và neck; cái phân biệt chúng là *dạng đầu ra* và do đó là hàm mất mát. Khối hậu xử lý bên phải là phần duy nhất không nhận gradient, và toàn bộ chương này là câu chuyện thu nhỏ khối đó lại.

Hình 3.1 là bản đồ nên giữ trong đầu suốt chương: mỗi khi gặp một mô hình mới, hãy hỏi nó thay đổi khối nào trong bốn khối này.

Chương chia làm bốn mục theo đúng thứ tự phụ thuộc đó. Mục 1 đi hết dòng hai giai đoạn, nơi mẫu hình “nuốt heuristic” hiện ra rõ nhất và nơi các khái niệm nền — anchor, RoI, NMS — được định nghĩa cho cả chương. Mục 2 bỏ giai đoạn đề xuất để lấy tốc độ, và đúng lúc đó một vấn đề mới lộ ra mà mục 1 chưa hề gặp: mất cân bằng foreground–background, thứ được giải quyết ở tầng hàm mất mát chứ không ở tầng kiến trúc. Mục 2 cũng là nơi đặt bài học trung tâm của cả chương: cách gán mẫu cho các vị trí dự đoán — thành phần hiểm khi nằm trong abstract và thường nằm trong code — giải thích phần lớn chênh lệch giữa hai detector, nhiều hơn cả kiến trúc. Mục 3 chuyển sang segmentation. Ở đó cùng một mâu thuẫn “ngữ nghĩa sâu chống lại định vị chính xác” được tấn công từ bốn phía. Mục kết thúc bằng một lần đổi phát biểu, hợp nhất semantic, instance và panoptic. Lần đổi phát biểu đó mượn nguyên ý tưởng dự đoán tập hợp của mục 2, nên hai mục phải đọc theo thứ tự. Mục 4 gom các bài toán lân cận mà kỹ sư gặp hằng ngày: pose, OCR, retrieval, cùng hai chế độ phân bố dữ liệu mà mọi con số tổng đều che giấu.

Nếu bạn chỉ có thời gian cho một phần, hãy đọc mục 2. Nó chứa cái mà roadmap gọi là bài học lặp lại của cả lĩnh vực, và nó thay đổi cách bạn đọc mọi bảng kết quả detection về sau.

## Bản đồ chương 14 — Bài toán CV lõi (Core Vision Tasks)

### TÓM TẮT

Chương 14 gồm bốn mục, chia theo *dạng đầu ra* chứ không theo thứ tự lịch sử: hai mục đầu về đầu ra dạng tập khung (detection), mục ba về đầu ra dạng mask, mục bốn về những dạng đầu ra còn lại mà kỹ sư gặp hằng ngày. Đọc bản đồ này để biết mục nào là cốt lõi phải nắm chắc, mục nào tra khi cần, và thứ tự phụ thuộc giữa chúng. Nếu chỉ nhớ một điều: mục 2 là mục cốt lõi của cả chương, vì nó chứa bài học “cách gán mẫu quyết định nhiều hơn kiến trúc”.

### KIẾN THỨC CẦN CÓ TRƯỚC

- **mục (section) và chương** — chương là một chủ đề lớn; mỗi mục là một file riêng đọc độc lập được. Số thứ tự trong tên thư mục phản ánh *thứ tự nên đọc*, không phải thứ tự lịch sử.
- **cốt lõi và tham khảo** — “cốt lõi” là phần phải nắm chắc và quay lại nhiều lần; “tham khảo” là phần biết-là-có, đọc kỹ khi thật sự gặp bài toán đó.
- **detection và segmentation** — detection trả về danh sách khung chữ nhật kèm nhãn; segmentation trả về nhãn cho từng pixel. Khác biệt cốt lõi là *dạng đầu ra*, và nó kéo theo hàm mất mát lẫn thước đo.
- **gán mẫu (label assignment)** — quy tắc quyết định vị trí dự đoán nào phải học vật nào. Nó nằm trong định nghĩa của hàm mất mát chứ không phải trong kiến trúc — lý do nó là bài học trung tâm của chương.

### Có gì trong chương này

- **Mục 1 — Detection hai giai đoạn (cốt lõi).** R-CNN → Fast R-CNN → Faster R-CNN với RPN → FPN → Mask R-CNN với RoIAlign. Đây là nơi định nghĩa các khái niệm nền dùng lại suốt chương: anchor, RoI, NMS, IoU, loss đa nhiệm.
- **Mục 2 — Một giai đoạn và cuộc đánh đổi tốc độ (cốt lõi, đọc kỹ nhất).** YOLO/SSD, mất cân bằng nền-vật thể, focal loss, anchor-free, và thực sự quan trọng: **gán mẫu (label assignment)** từ IoU cố định qua ATSS tới SimOTA, rồi DETR bỏ NMS bằng Hungarian matching.
- **Mục 3 — Phân vùng ảnh và sự hợp nhất (cốt lõi).** FCN, U-Net, DeepLab với atrous/ASPP, SegFormer, Mask2Former, rồi bước dịch sang mô hình nền tảng nhận prompt.
- **Mục 4 — Các bài toán lân cận (tham khảo có trọng điểm).** Pose, OCR, metric learning và truy hồi, fine-grained và long-tail. Đọc lướt để biết là có, quay lại khi thật sự gặp.

### Vì sao chia như vậy

Ba câu hỏi kinh điển của thị giác — *ảnh này là gì, có gì ở đâu, mỗi pixel thuộc về cái gì* — trông như ba mức chi tiết của cùng một bài toán, nhưng chỉ câu đầu có đầu ra kích thước cố định. Chương chia theo đúng ranh giới đó: mục 1 và 2 xử lý đầu ra dạng tập hợp khung, mục 3 xử lý đầu ra dạng mask, mục 4 gom các dạng còn lại. Cách chia này quan trọng vì *dạng đầu ra* kéo theo hàm mất mát, metric và chế độ hồng — ba thứ khó đổi hơn kiến trúc rất nhiều.



### Thứ tự đọc và phụ thuộc

1. Đọc [C/13-kien-truc-backbone](#) trước; toàn chương giả định bạn hiểu stride, trường tiếp nhận và feature map đa tỉ lệ.
2. Mục 1 trước mục 2: mất cân bằng nền–vật thể chỉ lộ ra *sau khi* bỏ giai đoạn đề xuất, nên đọc ngược thứ tự thì mất luôn nguyên nhân.
3. Mục 2 trước mục 3: khung truy vấn của Mask2Former mượn nguyên ý tưởng dự đoán tập hợp của DETR.
4. Mục 4 độc lập, đọc lúc nào cũng được.
5. Đọc [B/09-thiet-ke-ham-mat-mat](#) song song với mục 2 nếu phần focal loss chưa rõ, và [B/07-chia-du-lieu-va-danh-gia](#) song song với mục 4.

### Nếu chỉ có một buổi

Đọc phần đặt vấn đề của mục 1, rồi đọc trọn mục 2. Mục 2 chứa bài học lặp lại của cả lĩnh vực — rằng những chi tiết không được viết trong abstract thường quyết định kết quả nhiều hơn kiến trúc — và bài học đó thay đổi cách bạn đọc mọi bảng kết quả về sau.

## 3.1 Detection hai giai đoạn nuốt dần heuristic (Two-Stage Detection)

**PHỤ THUỘC** [C/13-kien-truc-backbone/01-dong-cnn](#) (feature map, stride, trường tiếp nhận (*receptive field*)); [B/09-thiet-ke-ham-mat-mat](#) (loss đa nhiệm); [B/05-thiet-ke-bai-toan](#) (bộ ba đầu vào – đầu ra – tín hiệu giám sát).

### TÓM TẮT

Mục này theo dấu bốn thể hệ detection hai giai đoạn: R-CNN, Fast R-CNN, Faster R-CNN với FPN, và Mask R-CNN. Chúng không phải bốn ý tưởng rời rạc, mà là bốn lần thay một bước thủ công bằng một khối học được. Đọc xong, cầm bất kỳ paper detection nào bạn cũng chỉ ra được hai thứ: khối nào nằm trong tầm với của gradient, và khối nào là heuristic cố định. Từ đó bạn đoán được bước cải tiến hợp lý tiếp theo. Đó là kỹ năng dự đoán, không phải ghi nhớ. Nếu chỉ nhớ một điều: ranh giới “cái gì nhận được gradient” dịch xuống qua từng thể hệ. Hai thứ sống sót được là triết phi cực đại và cách gán mẫu. Chúng sống sót vì lý do cấu trúc, không phải ngẫu nhiên, nên gỡ chúng đòi hỏi đổi phát biểu bài toán chứ không đổi kiến trúc.

### KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình (model)** — một hàm có tham số, ánh xạ ảnh đầu vào thành dự đoán; “huấn luyện” là tìm bộ tham số làm dự đoán khớp nhãn nhất có thể.
- **gradient** — vector đạo hàm của hàm mất mát theo từng tham số; nó chỉ hướng phải chỉnh tham số để lỗi giảm. Một khối “không nhận được gradient” là khối không tự chỉnh được, dù nó gây ra lỗi.
- **backbone** — phần mạng dùng chung để biến ảnh thành *feature map* — một lưới vector đặc trưng, mỗi ô mô tả nội dung của một vùng ảnh. Phần gần phía trên để trả lời một bài toán cụ thể gọi là *head*.
- **stride** — tỉ lệ thu nhỏ giữa ảnh và feature map. Stride 16 nghĩa là một ô đặc trưng ứng với một vùng  $16 \times 16$  pixel ảnh, nên mọi toạ độ tính trên feature map đều thô tới 16 pixel.
- **IoU** — diện tích giao chia diện tích hợp của hai khung chữ nhật; bằng 1 khi trùng khít, 0 khi rời nhau. Đây là thước đo “khung dự đoán có đúng chỗ không”.

- **anchor** — một khung chữ nhật mẫu, kích thước và tỉ số cạnh định sẵn, rải đều trên mọi vị trí của feature map. Mô hình không đoán khung từ số không mà đoán *độ lệch* so với anchor gần nhất — một bài toán dễ hơn nhiều.
- **proposal (đề xuất vùng)** — một vùng ảnh ứng viên “có thể chứa vật gì đó”, sinh ra ở giai đoạn một để giai đoạn hai chỉ phải phân loại vài trăm vùng thay vì hàng chục nghìn.
- **NMS** — bước hậu xử lý xoá các khung trùng lặp: giữ khung điểm cao nhất, xoá mọi khung có IoU với nó vượt ngưỡng, lặp lại. Nó là thuật toán viết tay, không học được.
- **mAP** — thước đo chuẩn của detection: với mỗi lớp, vẽ đường precision-recall rồi lấy diện tích dưới đường đó, sau đó trung bình trên mọi lớp và trên nhiều ngưỡng IoU.

### Đặt vấn đề

Phân loại ảnh có một đặc tính dễ chịu mà ta ít khi để ý: đầu ra của nó có kích thước cố định. Một ảnh vào, một vector  $K$  chiều ra, và toàn bộ bộ máy huấn luyện bằng *gradient* chạy trơn tru vì **hàm mất mát** (*loss function*) được định nghĩa trên một đối tượng có hình dạng biết trước. **Phát hiện vật thể** (*object detection*) phá vỡ đúng đặc tính đó. Số vật thể trong ảnh thay đổi từ không tới hàng trăm, mỗi vật cần một nhãn và một vị trí, nên đầu ra là một **tập hợp** (*set*) có lực lượng thay đổi. Không có cách hiển nhiên nào viết một hàm mất mát khả vi trên tập hợp, và đây chính là nút thắt mà toàn bộ lịch sử detection xoay quanh.

Cách né đầu tiên, và trong nhiều năm là cách duy nhất, là biến bài toán tập hợp thành nhiều bài toán phân loại: đề xuất trước một tập ứng viên vùng ảnh, rồi phân loại từng ứng viên độc lập. Mẹo này giải quyết được vấn đề hình dạng đầu ra, nhưng nó đẩy độ khó sang chỗ khác — sang câu hỏi *ai sinh ra các ứng viên đó*. Trong R-CNN, câu trả lời là một thuật toán phân đoạn thủ công tên là *selective search*, thứ không có tham số nào học được và không bao giờ nhận được một chút *gradient* nào từ lỗi cuối cùng.

**Học xong mục này bạn làm được gì mà trước đó không làm được?** Bạn đọc một paper detection bất kỳ và chỉ ra ngay được ba thứ: khối nào nằm trong tầm với của gradient, khối nào là heuristic cố định, và vì thế bước cải tiến hợp lý tiếp theo là gì. Đó là một kỹ năng dự đoán, không phải kỹ năng ghi nhớ.

### TRỰC GIÁC

Hãy hình dung hệ thống như một xưởng máy mà một số máy được nối vào bảng điều khiển trung tâm, còn một số thì quay tay. Máy nối vào bảng điều khiển tự chỉnh lại khi sản phẩm cuối bị lỗi; máy quay tay thì không, dù nó có gây ra lỗi hay không. “Nuốt một heuristic vào mạng” theo nghĩa cơ học chính xác là *nối thêm một máy vào bảng điều khiển*: biến một bước không khả vi, không tham số, thành một khối khả vi nhận được gradient từ hàm mất mát detection. Vì thế mỗi lần nuốt, hệ thống không chỉ nhanh hơn (bớt một công đoạn rồi) mà còn chính xác hơn (thêm một khối được tối ưu cho đúng mục tiêu cuối).

## A. Bốn bước nuốt

### I MẠCH 1 — detection hai giai đoạn

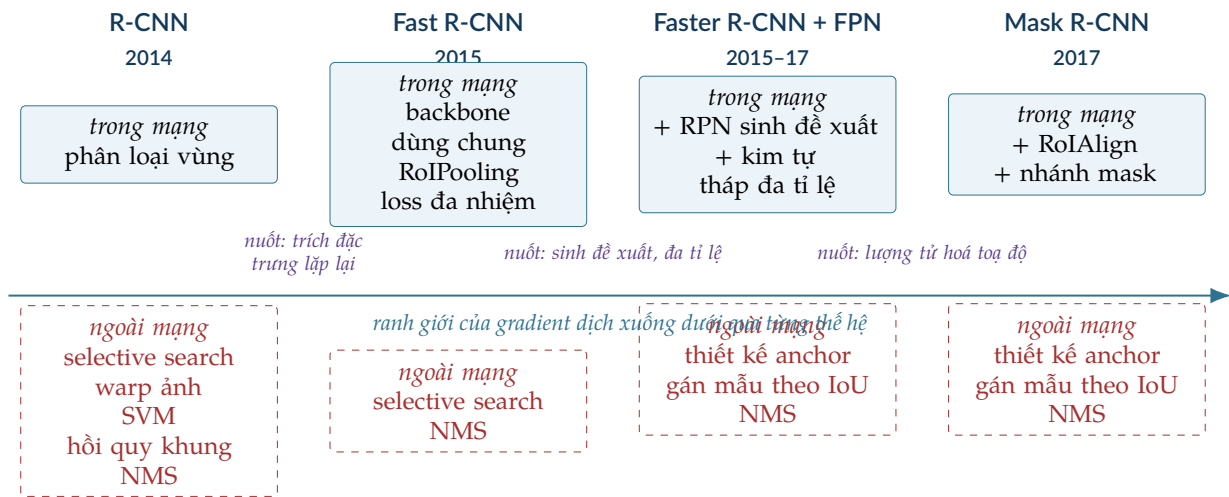
#### 1 Vấn đề

không thể chấm điểm một tập hợp có lực lượng thay đổi.

#### Giải pháp

R-CNN cắt khoảng hai nghìn vùng ứng viên do *selective search* sinh ra. Mỗi vùng được warp về kích thước cố định rồi cho qua CNN để lấy đặc trưng. Một SVM phân loại, và một bộ hồi quy riêng tinh chỉnh khung [48].

- Giả định** các vùng đáng quan tâm đều nằm trong tập ứng viên; nếu selective search bỏ sót thì không gì cứu được.
- Giá phải trả** mỗi ảnh phải chạy CNN hai nghìn lần, huấn luyện chia làm ba pha rời rạc, và không pha nào tối ưu cho mục tiêu của pha kia.
- 2 Vấn đề** hai nghìn lượt chạy CNN cho cùng một ảnh là lãng phí gần như toàn bộ.
- Giải pháp** Fast R-CNN đảo thứ tự: chạy *backbone* **một lần** trên cả ảnh, rồi với mỗi vùng ứng viên chỉ cắt phần tương ứng trên feature map bằng *RoIPooling* và cho qua vài tầng **kết nối đầy đủ** (*fully connected*) [49]. Phân loại và hồi quy khung được gộp thành một **hàm mất mát đa nhiệm** (*multi-task loss*) duy nhất, huấn luyện một pha.
- ↔ **ràng buộc bởi** B/09 — trọng số giữa hạng thức phân loại và hạng thức hồi quy là một siêu tham số ẩn, và chương thiết kế hàm mất mát giải thích vì sao nó không thể chọn tùy tiện.
- Giả định** đặc trưng dùng chung cho mọi vùng vẫn đủ tốt cho từng vùng — đúng khi các vùng chồng lấn nhiều, và chúng chồng lấn thật.
- Vấn đề mới** khi phần tính toán nặng đã bị loại bỏ, selective search trở thành nút cổ chai.
- 3 Vấn đề** bộ sinh đề xuất là công đoạn quay tay cuối cùng còn nặng.
- Giải pháp** Faster R-CNN thay nó bằng **mạng đề xuất vùng** (*region proposal network, RPN*), một mạng con trượt trên chính feature map đã có, tại mỗi vị trí dự đoán độ khách quan và độ lệch khung so với một tập *anchor* định sẵn [50]. Vì RPN dùng chung backbone, chi phí sinh đề xuất gần như bằng không.
- Giả định** tập anchor phủ đủ dày dãi tỉ lệ và tỉ số cạnh của vật thể trong dữ liệu.
- Giá phải trả** anchor trở thành một **siêu tham số** (*hyperparameter*) nhạy và *mang theo thống kê của tập dữ liệu gốc*; đổi miền ảnh thì phải thiết kế lại.
- 4 Vấn đề** một feature map duy nhất không phục vụ nổi mọi tỉ lệ. Feature ở tầng sâu có ngữ nghĩa mạnh nhưng độ phân giải thô, feature nông thì ngược lại; vật nhỏ vì thế bị thiệt kép.
- Giải pháp** **kim tự tháp đặc trưng** (*feature pyramid network, FPN*) dựng một kim tự tháp bằng nhánh *top-down* cộng với kết nối ngang, để mọi tầng đều vừa sâu về ngữ nghĩa vừa đúng độ phân giải [51].
- Hệ quả** đây là lần đầu “đa tỉ lệ” được đưa vào kiến trúc thay vì đưa vào pipeline dưới dạng chạy ảnh ở nhiều kích thước.
- ↔ **tiền đề** C/13/01 — stride và trường tiếp nhận của backbone chính là thứ tạo ra mâu thuẫn ngữ nghĩa–độ phân giải mà FPN sinh ra để gỡ.
- 5 Vấn đề** RoIPooling làm tròn toạ độ, và việc làm tròn đó phá hỏng mọi bài toán cần độ chính xác dưới pixel.
- Giải pháp** Mask R-CNN thay bằng *RoIAlign*: không làm tròn, lấy mẫu song tuyến tại các điểm có toạ độ thực, rồi thêm một nhánh dự đoán **mặt nạ** (*mask*) nhị phân song song với nhánh phân loại [52].
- Hệ quả** nhánh mask gần như miễn phí về ý tưởng nhưng chỉ hoạt động tốt *sau khi* lỗi lượng tử hoá được gỡ bỏ — một minh hoạ sắc nét rằng chi tiết cài đặt có thể chặn đứng cả một bài toán mới.

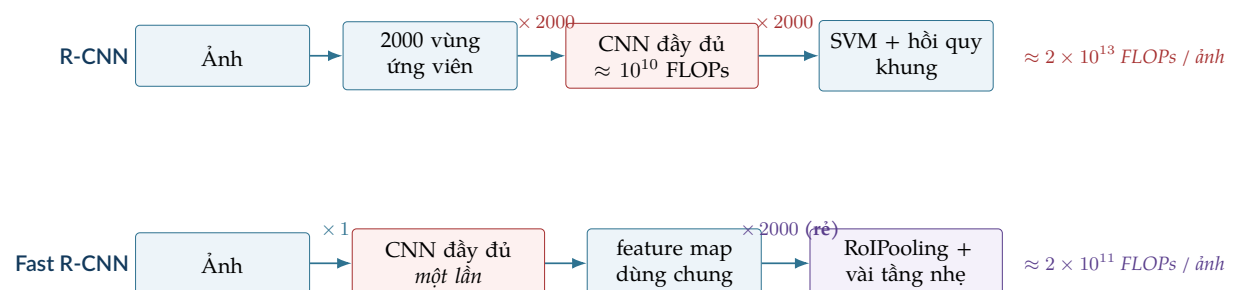


**Hình 3.2.** Dòng thời gian của họ hai giai đoạn, vẽ theo trục “cái gì nhận được gradient”. Khối trên đường kẻ nằm trong tầm với của hàm mất mát detection và tự chỉnh; khối dưới đường kẻ là heuristic cố định. Nhấn tím giữa hai cột ghi rõ mỗi bước nuốt được heuristic nào. Qua bốn thể hệ, phần dưới teo dần nhưng NMS và cách gán mẫu thì ở lại — đó là dư địa mà Mục 2 khai thác.

Hình 3.2 cho thấy điều mà một bảng thời gian không cho thấy: cái còn sót lại sau bốn bước không phải là ngẫu nhiên. **Triệt phi cực đại** (*non-maximum suppression, NMS*) sống sót vì nó thao tác trên *quan hệ giữa các dự đoán* chứ không trên một dự đoán đơn lẻ, mà loss của kiến trúc này lại chấm điểm từng dự đoán độc lập. Cách **gán mẫu** (*label assignment*) sống sót vì nó là một phần của *định nghĩa* loss chứ không phải một khối tính toán — không có gì để “nuốt” theo nghĩa thông thường. Hai chỗ đó đòi hỏi đối phát biểu bài toán, và đó đúng là việc DETR làm ở mục sau.

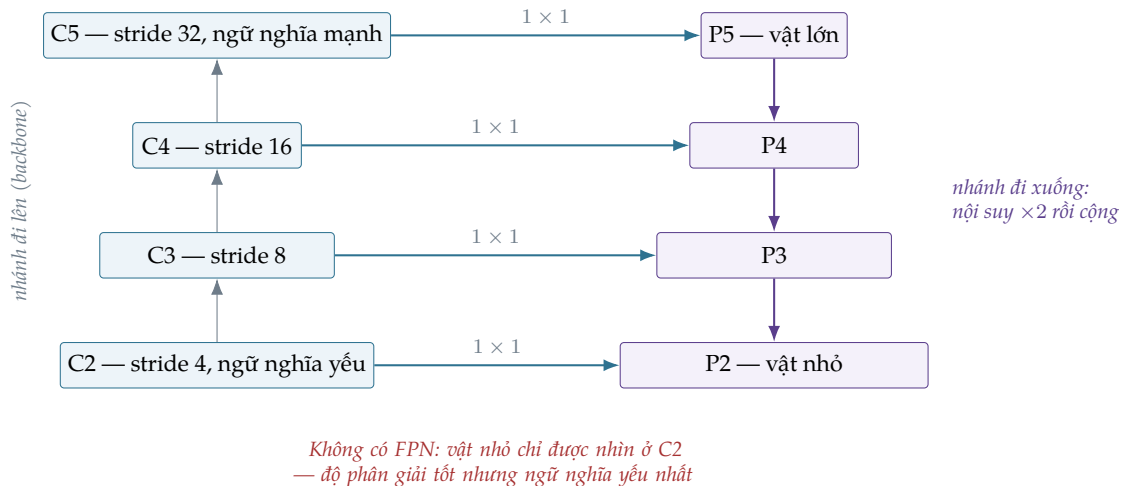
### B. Ba phép tính nhỏ giải thích ba bước nhảy

- **Vì sao chia sẻ feature map lại nhanh hơn hai bậc.** Giả sử backbone tốn khoảng  $10^{10}$  FLOPs cho một ảnh  $224 \times 224$ .
  - **R-CNN** chạy nó cho từng vùng trong  $N = 2000$  vùng:  $2 \times 10^{13}$  FLOPs mỗi ảnh.
  - **Fast R-CNN** chạy backbone một lần trên ảnh lớn hơn (giả sử đất gấp mười,  $10^{11}$ ), rồi mỗi vùng chỉ qua vài tầng nhẹ cỡ  $10^8$ , tổng  $2 \times 10^{11}$ .
  - **Tỉ số** khoảng  $2 \times 10^{13} / 3 \times 10^{11} \approx 70$  lần, và thực tế còn lớn hơn vì R-CNN phải ghi/đọc đặc trưng ra đĩa. *Ý nghĩa:* chi phí chuyển từ  $O(N)$  lượt backbone sang  $O(1)$  lượt backbone cộng  $O(N)$  lượt đầu nhẹ — đó mới là điều cần nhớ, không phải con số.



**Hình 3.3.** Cùng số vùng ứng viên, cùng backbone, khác nhau ở *thứ tự* phép cắt và phép trích đặc trưng. R-CNN đặt phép cắt trước nên khối đắt nhất bị nhân với  $N$ ; Fast R-CNN đặt phép cắt sau nên khối đắt chỉ chạy một lần và chỉ phần nhẹ mới bị nhân. Toàn bộ khoảng cách hai bậc độ lớn nằm ở chỗ đổi chỗ hai mũi tên, không ở chỗ nào khác.

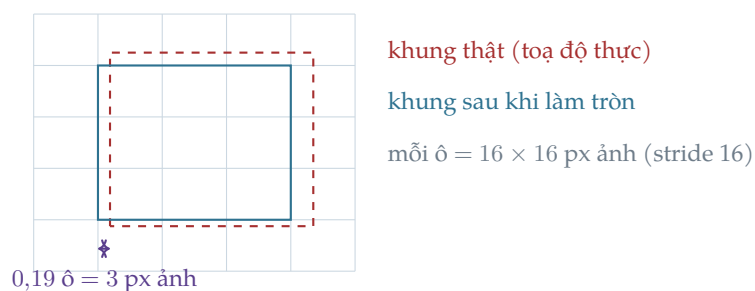
Hình 3.3 là dạng nhìn thấy được của phép tính vừa rồi, và nó cũng là mẫu hình đáng nhớ: khi một pipeline lập một khối đất, hãy hỏi liệu có đẩy được khối đó ra ngoài vòng lặp hay không. **Vì sao làm tròn lại đầu đến thế.** Backbone thông dụng có *stride* 16, nên một pixel feature ứng với ô  $16 \times 16$  pixel ảnh. Khung có cạnh trái tại  $x = 19$  rơi vào toạ độ feature  $19/16 = 1,1875$ ; RoIPooling làm tròn xuống 1, tức dịch biên  $0,1875 \times 16 = 3$  pixel ảnh gốc, rồi chia ô pooling làm tròn lần thứ hai. Ba pixel là nhiều với người cao 200 pixel, nhưng là 10% kích thước với một vật 30 pixel. Đó là lý do RoIAlign gần như không đổi gì ở ngưỡng **giao trên hợp** (*intersection over union, IoU*) lỏng nhưng cải thiện mạnh ở ngưỡng chặt và với nhánh mask [52].



**Hình 3.4.** FPN gộp hai thứ vốn nằm ở hai đầu đối lập của backbone. Đi lên, độ phân giải giảm còn ngữ nghĩa mạnh dần; đi xuống, ngữ nghĩa của tầng sâu được nội suy trả về từng mức và cộng vào đặc trưng nông qua kết nối ngang  $1 \times 1$ . Kết quả là mọi mức  $P_i$  đều vừa đủ sâu vừa đủ mịn, nên vật nhỏ không còn bị thiệt kép.

- Hình 3.4 làm rõ vì sao đây là một bước “nuốt heuristic”: trước FPN, cách xử lý đa tỉ lệ là chạy *cả ảnh* ở nhiều kích thước rồi gộp kết quả — một vòng lặp bên ngoài mạng, tốn gấp nhiều lần. FPN đưa đúng chức năng đó vào trong một nhánh có tham số học được.

**FPN gán vùng vào tầng nào.** Công thức trong bài gốc là  $k = \lfloor k_0 + \log_2(\sqrt{wh}/224) \rfloor$  với  $k_0 = 4$ . Nó nói gì: vật càng nhỏ thì càng được đẩy về tầng có độ phân giải cao. Khung  $112 \times 112$  cho  $\log_2(112/224) = -1$  nên  $k = 3$ ; khung  $448 \times 448$  cho  $k = 5$ . Đáng chú ý là công thức này vẫn là heuristic thủ công nằm ngoài gradient — lại một vòng nữa của cùng mẫu hình.



**Hình 3.5.** Vì sao RoIPooling làm hỏng bài toán mask. Trên lưới đặc trưng stride 16, việc làm tròn toạ độ khung về ô nguyên dịch biên đi tới vài pixel trên ảnh gốc; sai số này cố định, không giảm khi huấn luyện lâu hơn, và tỉ lệ nghịch với kích thước vật. RoIAlign lấy mẫu song tuyến tại toạ độ thực nên không sinh ra khoảng lệch màu tím.

Hình 3.5 là một lời nhắc quen thuộc với người làm hình học. Đây đúng là lỗi lượng tử hoá mà mọi hệ bám đặc trưng dưới pixel đều phải xử lý. Cách chữa cũng giống hệt: nội suy thay vì làm tròn, đúng như một bộ *tracker* KLT tinh chỉnh vị trí đặc trưng.

### C. Giới hạn và trường hợp hỏng

Họ hai giai đoạn hỏng theo ba kiểu rất đặc trưng, và mỗi kiểu có một triệu chứng quan sát được:

- **Cảnh đông đúc.** Khi hai vật cùng lớp chồng lấn nhiều hơn ngưỡng NMS, một trong hai chắc chắn bị xoá. *Triệu chứng:* hạ ngưỡng thì mất vật, nâng ngưỡng thì sinh trùng lặp — không có giá trị nào cứu được cả hai, vì vấn đề nằm ở phát biểu chứ không ở tham số.
- **Miền anchor lệch.** Tập anchor mã hoá phân bố tỉ lệ và tỉ số cạnh của dữ liệu huấn luyện. Đưa sang ống kính góc rộng, ảnh *fisheye*, hay lớp vật dài mảnh thì recall của RPN sụt trước khi bộ phân loại kịp nói gì. *Triệu chứng:* recall giai đoạn đề xuất thấp trong khi độ chính xác trên các đề xuất còn lại vẫn cao.
- **Độ trễ không ổn định.** Số đề xuất qua ngưỡng thay đổi theo ảnh, nên thời gian giai đoạn hai dao động theo nội dung cảnh. *Triệu chứng:* phân vị 99 của độ trễ cao hơn nhiều so với trung vị — một tính chất rất khó chịu khi bạn phải cam kết ngân sách mili-giây cho mỗi khung hình, đúng kiểu ràng buộc cứng đã bàn ở B/03-giai-phau-he-thong.

#### CẠM BẤY

Rất nhiều so sánh “kiến trúc A tốt hơn B 2 mAP” thực chất là so sánh hai công thức huấn luyện: số epoch, **tăng cường dữ liệu** (*data augmentation*), tỉ lệ mẫu dương/âm trong mỗi *batch*, ngưỡng IoU khi gán mẫu. Trước khi tin một cải tiến kiến trúc, hãy tìm xem phần gán mẫu và recipe có được giữ nguyên không — và trong đa số paper, thông tin đó nằm trong code chứ không nằm trong bài.

↔ **hệ quả của E/24** — muốn tách phần đóng góp của một khối khỏi phần đóng góp của recipe thì phải ablate đúng một thứ — đó là phương pháp, không phải trực giác.

Công cụ như [dbolya/tide](#) tách mAP thành các loại lỗi thành phần [53], [54], và thường phát hiện rằng hai mô hình “chênh 2 mAP” hỏng theo hai kiểu hoàn toàn khác nhau.

#### CẦN NHỚ

Mỗi bước tiến của detection hai giai đoạn là một heuristic bị kéo vào trong tầm với của gradient. Muốn đoán bước tiếp theo của một dòng mô hình bất kỳ, kể cả *SLAM*, hãy liệt kê những gì còn nằm *ngoài* vòng tối ưu. Rồi hỏi cái nào trong số đó viết lại được thành một khối khả vi. [T3]

### D. Kết nối

Mục tiếp theo ([C/14-bai-toan-cv-loi/02-detection-mot-giai-doan](#)) là mặt đối lập: bỏ hẳn giai đoạn đề xuất để đổi lấy tốc độ, và trả giá bằng một vấn đề mất cân bằng mà mục này chưa gặp. [C/13-kien-truc-backbone/01-dong-cnn](#) là tiền đề: RoIPooling, stride và trường tiếp nhận chỉ hiểu được sau khi nắm cơ học của *convolution*. B/09-thiet-ke-ham-mat-mat giải thích vì sao gộp phân loại và hồi quy vào một loss đa nhiệm không hiển nhiên đúng. E/24-thi-nghiem-va-ablation là nơi kiểm chứng cạm bẫy vừa nêu.

#### Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao Fast R-CNN nhanh hơn R-CNN nhiều bậc trong khi cả hai dùng cùng backbone và cùng số vùng ứng viên?
2. RoIAlign khác RoIPooling ở đúng một chỗ. Vì sao chỗ đó ảnh hưởng tới nhánh mask mạnh hơn nhiều so với nhánh phân loại?
3. Miền ảnh cụ thể nào làm tập anchor mặc định của COCO sụt recall, và triệu chứng quan sát được ở đó là gì?



4. Vì sao NMS và cách gán mẫu là hai thứ duy nhất sống sót qua cả bốn thế hệ? Hai lý do đó khác nhau ở điểm nào?
5. Nếu bạn phải cam kết một ngân sách mili-giây cố định cho mỗi khung hình, kiến trúc hai giai đoạn gây khó khăn gì mà một giai đoạn không gây?

- [1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [2] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [3] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *AISTATS*, 2010.
- [4] K. He, X. Zhang, S. Ren, et al., “Delving Deep into Rectifiers,” in *ICCV*, 2015.
- [5] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [6] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [7] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [8] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [9] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [10] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [11] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” in *ICLR*, 2017.
- [12] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [13] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [14] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [15] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [16] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [17] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [18] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [19] G. Huang, Y. Sun, Z. Liu, et al., “Deep Networks with Stochastic Depth,” in *ECCV*, 2016.
- [20] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [21] H. Zhang, M. Cisse, Y. N. Dauphin, et al., “mixup: Beyond Empirical Risk Minimization,” in *ICLR*, 2018.
- [22] S. Yun, D. Han, S. J. Oh, et al., “CutMix: Regularization Strategy to Train Strong Classifiers,” in *ICCV*, 2019.
- [23] E. D. Cubuk, B. Zoph, J. Shlens, et al., “RandAugment: Practical Automated Data Augmentation,” in *CVPR Workshops*, 2020.

- [24] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [25] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [26] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *NeurIPS*, 2012.
- [27] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in *ICLR*, 2015.
- [28] C. Szegedy, W. Liu, et al., “Going Deeper with Convolutions,” in *CVPR*, 2015.
- [29] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [30] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [31] G. Huang, Z. Liu, L. van der Maaten, et al., “Densely Connected Convolutional Networks,” in *CVPR*, 2017.
- [32] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [33] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [34] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [35] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [36] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [37] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.
- [38] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [39] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [40] J. Su, M. Ahmed, Y. Lu, et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, vol. 568, 2024.
- [41] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” *arXiv:2312.00752*, 2023.
- [42] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [43] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.
- [44] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [45] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [46] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [47] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [48] R. Girshick, J. Donahue, T. Darrell, et al., “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation,” in *CVPR*, 2014.

- [49] R. Girshick, "Fast R-CNN," in *ICCV*, 2015.
- [50] S. Ren, K. He, R. Girshick, et al., "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in *NeurIPS*, 2015.
- [51] T.-Y. Lin, P. Dollár, R. Girshick, et al., "Feature Pyramid Networks for Object Detection," in *CVPR*, 2017.
- [52] K. He, G. Gkioxari, P. Dollár, et al., "Mask R-CNN," in *ICCV*, 2017.
- [53] D. Bolya, S. Foley, J. Hays, et al., "TIDE: A General Toolbox for Identifying Object Detection Errors," in *ECCV*, 2020.
- [54] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [55] J. Redmon, S. Divvala, R. Girshick, et al., "You Only Look Once: Unified, Real-Time Object Detection," in *CVPR*, 2016.
- [56] W. Liu, D. Anguelov, D. Erhan, et al., "SSD: Single Shot MultiBox Detector," in *ECCV*, 2016.
- [57] T.-Y. Lin, P. Goyal, R. Girshick, et al., "Focal Loss for Dense Object Detection," in *ICCV*, 2017.
- [58] Z. Tian, C. Shen, H. Chen, et al., "FCOS: Fully Convolutional One-Stage Object Detection," in *ICCV*, 2019.
- [59] X. Zhou, D. Wang, and P. Krähenbühl, "Objects as Points," *arXiv:1904.07850*, 2019.
- [60] S. Zhang, C. Chi, Y. Yao, et al., "Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection," in *CVPR*, 2020.
- [61] Z. Ge, S. Liu, Z. Li, et al., "OTA: Optimal Transport Assignment for Object Detection," in *CVPR*, 2021.
- [62] N. Bodla, B. Singh, R. Chellappa, et al., "Soft-NMS: Improving Object Detection with One Line of Code," in *ICCV*, 2017.
- [63] X. Zhu, W. Su, L. Lu, et al., "Deformable DETR: Deformable Transformers for End-to-End Object Detection," in *ICLR*, 2021.
- [64] H. Zhang, F. Li, S. Liu, et al., "DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection," in *ICLR*, 2023.
- [65] J. Long, E. Shelhamer, and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," in *CVPR*, 2015.
- [66] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *MICCAI*, 2015.
- [67] L.-C. Chen, G. Papandreou, I. Kokkinos, et al., "DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs," *IEEE TPAMI*, vol. 40, no. 4, pp. 834–848, 2018.
- [68] E. Xie, W. Wang, Z. Yu, et al., "SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers," in *NeurIPS*, 2021.
- [69] B. Cheng, I. Misra, A. G. Schwing, et al., "Masked-Attention Mask Transformer for Universal Image Segmentation," in *CVPR*, 2022.
- [70] A. Kirillov, K. He, R. Girshick, et al., "Panoptic Segmentation," in *CVPR*, 2019.
- [71] A. Kirillov, E. Mintun, N. Ravi, et al., "Segment Anything," in *ICCV*, 2023.
- [72] K. Sun, B. Xiao, D. Liu, et al., "Deep High-Resolution Representation Learning for Human Pose Estimation," in *CVPR*, 2019.

- [73] Z. Cao, T. Simon, S.-E. Wei, et al., “Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields,” in *CVPR*, 2017.
- [74] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [75] J. Deng, J. Guo, N. Xue, et al., “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *CVPR*, 2019.
- [76] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [77] K. Xu, J. Ba, R. Kiros, et al., “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention,” in *ICML*, 2015.
- [78] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *ICLR*, 2015.
- [79] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [80] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [81] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [82] K. Simonyan and A. Zisserman, “Two-Stream Convolutional Networks for Action Recognition in Videos,” in *NeurIPS*, 2014.
- [83] J. Carreira and A. Zisserman, “Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset,” in *CVPR*, 2017.
- [84] D. Tran, H. Wang, L. Torresani, et al., “A Closer Look at Spatiotemporal Convolutions for Action Recognition,” in *CVPR*, 2018.
- [85] C. Feichtenhofer, H. Fan, J. Malik, et al., “SlowFast Networks for Video Recognition,” in *ICCV*, 2019.
- [86] A. Dosovitskiy, P. Fischer, E. Ilg, et al., “FlowNet: Learning Optical Flow with Convolutional Networks,” in *ICCV*, 2015.
- [87] D. Sun, X. Yang, M.-Y. Liu, et al., “PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume,” in *CVPR*, 2018.
- [88] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.
- [89] A. Bewley, Z. Ge, L. Ott, et al., “Simple Online and Realtime Tracking,” in *ICIP*, 2016.
- [90] Y. Zhang, P. Sun, Y. Jiang, et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *ECCV*, 2022.
- [91] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [92] K. Bernardin and R. Stiefelhagen, “Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics,” *EURASIP J. Image and Video Processing*, 2008.
- [93] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [94] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [95] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.

- [96] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *CVPR*, 2024.
- [97] D. Eigen, C. Puhrsch, and R. Fergus, “Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network,” in *NeurIPS*, 2014.
- [98] R. Ranftl, K. Lasinger, D. Hafner, et al., “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer,” *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [99] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IROS*, 2012.
- [100] C. R. Qi, H. Su, K. Mo, et al., “PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation,” in *CVPR*, 2017.
- [101] C. R. Qi, L. Yi, H. Su, et al., “PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space,” in *NeurIPS*, 2017.
- [102] A. H. Lang, S. Vora, H. Caesar, et al., “PointPillars: Fast Encoders for Object Detection from Point Clouds,” in *CVPR*, 2019.
- [103] T. Müller, A. Evans, C. Schied, et al., “Instant Neural Graphics Primitives with a Multiresolution Hash Encoding,” *ACM TOG*, vol. 41, no. 4, 2022.
- [104] B. Kerbl, G. Kopanas, T. Leimkühler, et al., “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *ACM TOG*, vol. 42, no. 4, 2023.
- [105] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.
- [106] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” in *ICLR*, 2014.
- [107] I. Goodfellow, J. Pouget-Abadie, M. Mirza, et al., “Generative Adversarial Nets,” in *NeurIPS*, 2014.
- [108] T. Karras, S. Laine, and T. Aila, “A Style-Based Generator Architecture for Generative Adversarial Networks,” in *CVPR*, 2019.
- [109] J. Ho, A. Jain, and P. Abbeel, “Denoising Diffusion Probabilistic Models,” in *NeurIPS*, 2020.
- [110] Y. Song, J. Sohl-Dickstein, D. P. Kingma, et al., “Score-Based Generative Modeling Through Stochastic Differential Equations,” in *ICLR*, 2021.
- [111] J. Johnson, A. Alahi, and L. Fei-Fei, “Perceptual Losses for Real-Time Style Transfer and Super-Resolution,” in *ECCV*, 2016.
- [112] P. Isola, J.-Y. Zhu, T. Zhou, et al., “Image-to-Image Translation with Conditional Adversarial Networks,” in *CVPR*, 2017.
- [113] L. A. Gatys, A. S. Ecker, and M. Bethge, “Image Style Transfer Using Convolutional Neural Networks,” in *CVPR*, 2016.
- [114] X. Huang and S. Belongie, “Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization,” in *ICCV*, 2017.
- [115] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.
- [116] J. Song, C. Meng, and S. Ermon, “Denoising Diffusion Implicit Models,” in *ICLR*, 2021.
- [117] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, et al., “Flow Matching for Generative Modeling,” in *ICLR*, 2023.
- [118] J. Ho and T. Salimans, “Classifier-Free Diffusion Guidance,” *arXiv:2207.12598*, 2022.
- [119] R. Rombach, A. Blattmann, D. Lorenz, et al., “High-Resolution Image Synthesis with Latent Diffusion Models,” in *CVPR*, 2022.
- [120] W. Peebles and S. Xie, “Scalable Diffusion Models with Transformers,” in *ICCV*, 2023.



- [121] L. Zhang, A. Rao, and M. Agrawala, "Adding Conditional Control to Text-to-Image Diffusion Models," in *ICCV*, 2023.
- [122] Y. Blau and T. Michaeli, "The Perception-Distortion Tradeoff," in *CVPR*, 2018.
- [123] C. Ledig, L. Theis, F. Huszár, et al., "Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network," in *CVPR*, 2017.
- [124] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., "Image Quality Assessment: From Error Visibility to Structural Similarity," *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [125] M. Heusel, H. Ramsauer, T. Unterthiner, et al., "GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium," in *NeurIPS*, 2017.
- [126] T. Brooks, B. Mildenhall, T. Xue, et al., "Unprocessing Images for Learned Raw Denoising," in *CVPR*, 2019.
- [127] A. Abdelhamed, S. Lin, and M. S. Brown, "A High-Quality Denoising Dataset for Smartphone Cameras," in *CVPR*, 2018.
- [128] J. Deng, W. Dong, R. Socher, et al., "ImageNet: A Large-Scale Hierarchical Image Database," in *CVPR*, 2009.
- [129] A. Radford, J. W. Kim, C. Hallacy, et al., "Learning Transferable Visual Models from Natural Language Supervision," in *ICML*, 2021.
- [130] J. Li, D. Li, S. Savarese, et al., "BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models," in *ICML*, 2023.
- [131] H. Liu, C. Li, Q. Wu, et al., "Visual Instruction Tuning," in *NeurIPS*, 2023.
- [132] S. Kornblith, J. Shlens, and Q. V. Le, "Do Better ImageNet Models Transfer Better?" In *CVPR*, 2019.
- [133] T. Chen, S. Kornblith, M. Norouzi, et al., "A Simple Framework for Contrastive Learning of Visual Representations," in *ICML*, 2020.
- [134] K. He, H. Fan, Y. Wu, et al., "Momentum Contrast for Unsupervised Visual Representation Learning," in *CVPR*, 2020.
- [135] J.-B. Grill, F. Strub, F. Altché, et al., "Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning," in *NeurIPS*, 2020.
- [136] M. Caron, H. Touvron, I. Misra, et al., "Emerging Properties in Self-Supervised Vision Transformers," in *ICCV*, 2021.
- [137] K. He, X. Chen, S. Xie, et al., "Masked Autoencoders Are Scalable Vision Learners," in *CVPR*, 2022.
- [138] A. van den Oord, Y. Li, and O. Vinyals, "Representation Learning with Contrastive Predictive Coding," in *arXiv:1807.03748*, 2018.
- [139] J. Kaplan, S. McCandlish, T. Henighan, et al., "Scaling Laws for Neural Language Models," *arXiv:2001.08361*, 2020.
- [140] X. Zhai, A. Kolesnikov, N. Houlsby, et al., "Scaling Vision Transformers," in *CVPR*, 2022.
- [141] J. Hoffmann, S. Borgeaud, A. Mensch, et al., "Training Compute-Optimal Large Language Models," in *NeurIPS*, 2022.
- [142] C. G. Northcutt, A. Athalye, and J. Mueller, "Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks," in *NeurIPS Datasets and Benchmarks*, 2021.
- [143] B. Recht, R. Roelofs, L. Schmidt, et al., "Do ImageNet Classifiers Generalize to ImageNet?" In *ICML*, 2019.
- [144] N. Houlsby, A. Giurgiu, S. Jastrzebski, et al., "Parameter-Efficient Transfer Learning for NLP," in *ICML*, 2019.

- [145] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [146] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *RecSys*, 2019.

### 3.2 Một giai đoạn và cuộc đánh đổi tốc độ (One-Stage Detection and the Speed Trade-off)

**PHỤ THUỘC** C/14-bai-toan-cv-loi/01-detection-hai-giai-doan (anchor, NMS, Rol); B/09-thiet-ke-ham-mat-mat (focal loss, mất cân bằng lớp); C/13/02-tu-chuoi-toi-attention (attention, cần cho phần DETR).

#### TÓM TẮT

Mục này trả lời ba câu hỏi. Vì sao bỏ giai đoạn đề xuất vùng lại làm lộ ra mất cân bằng nền–vật thể? Vì sao vấn đề đó được chữa ở tầng hàm mất mát chứ không ở tầng kiến trúc? Và vì sao chuỗi ATSS → SimOTA → Hungarian matching mới là trực tiến hoá thật của detection hiện đại? Đọc xong, trong bất kỳ bảng so sánh detector nào bạn cũng tách được phần chênh lệch *do kiến trúc* khỏi phần chênh lệch *do gán mẫu và công thức huấn luyện*. Bạn cũng biết tìm thông tin đó ở đâu khi paper không nói. Nếu chỉ nhớ một điều: cách gán mẫu cho  $10^5$  vị trí dự đoán là một phần của *định nghĩa bài toán tối ưu*. Hai detector gán mẫu khác nhau vì thế là hai bài toán khác nhau. So sánh chúng rồi quy kết cho kiến trúc là sai ngay từ tiền đề.

#### KIẾN THỨC CẦN CÓ TRƯỚC

- **hàm mất mát (loss function)** — một số duy nhất đo mức sai của dự đoán so với nhãn; toàn bộ việc huấn luyện là chỉnh tham số để số đó nhỏ đi. Đổi hàm mất mát là đổi *mục tiêu*, không phải đổi mô hình.
- **cross-entropy** — hàm mất mát chuẩn cho phân loại:  $-\log p_t$  với  $p_t$  là xác suất mô hình gán cho lớp đúng. Đoán đúng chắc chắn thì mất mát gần 0; đoán sai chắc chắn thì mất mát rất lớn.
- **mẫu dương / mẫu âm** — với mỗi vị trí dự đoán, “dương” nghĩa là vị trí đó được coi là chứa vật thể và phải học nhãn của vật đó; “âm” nghĩa là nền. Ai được coi là dương là do bước gán mẫu quyết định.
- **gán mẫu (label assignment)** — quy tắc quyết định mỗi trong hàng trăm nghìn vị trí dự đoán phải học điều gì. Nó không phải một khối tính toán mà là một phần của *định nghĩa* hàm mất mát — đây là ý trung tâm của cả mục.
- **anchor và anchor-free** — có anchor là dự đoán độ lệch so với các khung mẫu định sẵn; anchor-free là dự đoán trực tiếp từ mỗi ô lưới, chẳng hạn bốn khoảng cách tới bốn cạnh của khung.
- **NMS** — hậu xử lý xoá khung trùng: giữ khung điểm cao nhất rồi xoá mọi khung chồng lấn với nó quá ngưỡng IoU. Vì nó xét quan hệ *giữa* các dự đoán nên không thể huấn luyện cùng phần còn lại.
- **truy vấn (query) trong DETR** — một vector học được đóng vai “một chỗ trống để điền một vật thể”. Mô hình có  $N$  truy vấn cố định, mỗi truy vấn tự đi hỏi feature map rồi trả về một khung cộng một nhãn.
- **epoch** — một lượt đi hết toàn bộ tập huấn luyện. “Hội tụ sau 500 epoch” nghĩa là phải quét dữ liệu 500 lần mới đạt kết quả tốt — rất đắt.

#### Đặt vấn đề

Giai đoạn đề xuất vùng trong Faster R-CNN giải quyết được một vấn đề rất cụ thể: nó *lọc* hàng chục nghìn vị trí ứng viên xuống còn vài trăm, và mọi thứ phía sau chỉ phải làm việc trên một

tập nhỏ, đã tương đối cân bằng giữa mẫu dương và mẫu âm. Cái giá là một vòng lặp nữa qua bộ nhớ và một lượng tính toán tỉ lệ với số đề xuất. Với robot phải chạy 30 khung hình mỗi giây trên một máy nhúng, cái giá đó thường không trả nổi. Câu hỏi tự nhiên là: bỏ hẳn giai đoạn đề xuất, cho mạng dự đoán thẳng trên lưới đặc trưng, thì mất gì?

**Học xong mục này bạn làm được gì mà trước đó không làm được?** Đọc phần thực nghiệm của một paper detection, bạn biết ngay cần kiểm tra ba dòng nào trong code trước khi tin bằng kết quả.

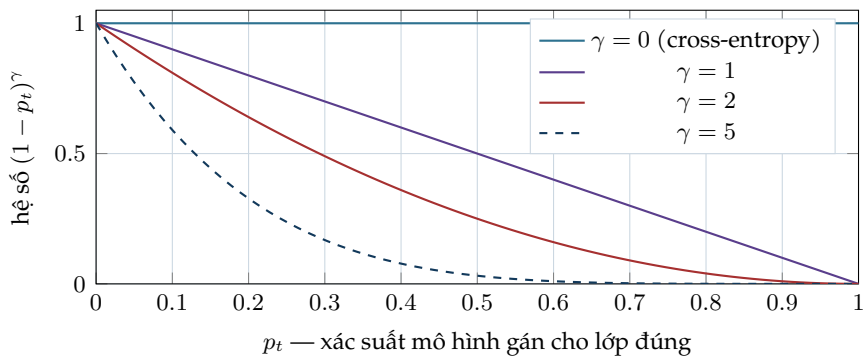
YOLO và SSD trả lời bằng cách làm đúng thể [55], [56]. Kết quả là tốc độ tăng vọt, nhưng một vấn đề mới xuất hiện mà kiến trúc hai giai đoạn chưa từng phải đối mặt, bởi vì bộ lọc đề xuất đã âm thầm che nó đi: **mất cân bằng nền–vật thể** (*foreground–background imbalance*). Bài học lớn nhất của mục này nằm ở chỗ vấn đề đó được giải quyết ở tầng **hàm mất mát** (*loss function*), không phải ở tầng kiến trúc. Câu chuyện sau đó còn cho thấy một điều nữa: phần lớn khoảng cách giữa các detector hiện đại nằm ở những chi tiết *không được viết trong abstract*.

#### A. Mất cân bằng, và vì sao nó là vấn đề của loss (Class Imbalance)

Hãy đếm. Một mạng một giai đoạn với **kim tự tháp đặc trưng** (*feature pyramid network, FPN*) năm tầng, ảnh  $800 \times 1333$ , chín **anchor** mỗi vị trí, sinh ra cỡ  $10^5$  dự đoán mỗi ảnh. Một ảnh COCO trung bình có bảy vật thể; kể cả khi mỗi vật khớp vài anchor thì số mẫu dương vẫn chỉ vài chục. Tỉ lệ âm trên dương do đó vào khoảng  $10^3$  tới  $10^4$ .

Con số đó chưa đủ để thấy vấn đề. Cái đáng sợ nằm ở tổng gradient. Giả sử sau vài epoch, mạng đã học được rằng gần như mọi thứ là nền. Mỗi anchor nền khi đó có xác suất đúng  $p_t = 0,99$ , đóng góp **entropy chéo** (*cross-entropy*)  $-\log 0,99 \approx 0,01$ . Nghe như không đáng kể — nhưng  $10^5$  anchor nền cho tổng  $10^5 \times 0,01 = 1000$ . Trong khi đó ba mươi anchor dương khó, mỗi cái  $p_t = 0,5$  và loss 0,69, chỉ cho tổng 21. Tín hiệu học thật bị nhấn chìm dưới tỉ lệ 50:1 bởi những mẫu mà mạng đã làm đúng rồi. Đây là ví dụ nhỏ nhất giải thích vì sao “hạ learning rate” hay “train lâu hơn” không cứu được: bản thân hàm mất mát đang chỉ sai hướng.

Cách chữa cổ điển là **hard negative mining** — SSD giữ tỉ lệ âm/dương 3:1 bằng cách chỉ lấy các mẫu âm có loss cao nhất. Cách này hoạt động nhưng đưa thêm một heuristic rời rạc, không khả vi vào giữa vòng huấn luyện. **Focal loss** thay nó bằng một hệ số điều biến trơn:  $FL(p_t) = -\alpha_t(1-p_t)^\gamma \log p_t$  [57]. Với  $\gamma = 2$ , mẫu dễ  $p_t = 0,99$  bị nhân với  $(0,01)^2 = 10^{-4}$ . Mẫu khó  $p_t = 0,5$  chỉ bị nhân 0,25. Lặp lại phép đếm ở trên: tổng đóng góp của nền tụt từ 1000 xuống 0,1, trong khi mẫu dương khó chỉ giảm bốn lần. Tỉ lệ đảo ngược hoàn toàn.



**Hình 3.6.** Hệ số điều biến của focal loss. Điều quan trọng không phải mẫu khó được đề cao, mà là mẫu *đã đúng* bị hạ xuống gần như bằng không — với  $\gamma = 2$ , một anchor nền chắc chắn đóng góp ít hơn bốn bậc so với cross-entropy. Vì  $\gamma$  chỉ nén chứ không đảo thứ tự, nó không thể sửa được một tập nhãn sai.

Hình 3.6 cũng cho thấy giới hạn. Hệ số luôn dương và đơn điệu. Nên khi một mẫu bị gán nhãn sai, mô hình dự đoán *đúng theo thực tế nhưng sai theo nhãn*, focal loss coi nó là mẫu khó và *tăng* trọng số cho nó. Trên dữ liệu nhiều nhãn, đây là chế độ hồng thật, không phải lý thuyết.

↪ **trường hợp riêng của** B/09 — focal loss là một ca cụ thể của nguyên tắc tổng quát “sửa mất cân bằng ở tầng hàm mất mát thay vì ở tầng lấy mẫu dữ liệu”.

Kết luận rút ra từ RetinaNet vượt xa bản thân hàm loss. Suốt vài năm, khoảng cách giữa một giai đoạn và hai giai đoạn được giải thích bằng “kiến trúc hai giai đoạn tinh vi hơn”. Hoá ra phần lớn khoảng cách đó là hệ quả của việc giai đoạn đề xuất đã ngầm cân bằng lại tập mẫu. Sửa ở tầng loss là đủ.

## B. Bỏ anchor, và câu hỏi thật sự là gán mẫu (Label Assignment)

### TRỰC GIÁC

Kiến trúc là thí sinh; cách gán mẫu là *đáp án*. Một detector một giai đoạn có  $10^5$  ô trên lưới, và gán mẫu chính là việc điền vào đáp án cho từng ô: ô này phải nói “có xe ở đây”, ô kia phải nói “không”. Đối cách gán mẫu không phải là đối thí sinh, mà là đối đáp án — tức đối hần bài toán tối ưu. Vì thế so hai detector dùng hai cách gán mẫu khác nhau rồi kết luận về kiến trúc cũng vô nghĩa như so điểm hai thí sinh làm hai đề khác nhau.

## I MẠCH 2 — một giai đoạn, từ anchor tới tập hợp

### 1 Vấn đề

#### Giải pháp

#### Giá phải trả

anchor là một siêu tham số thủ công mang theo thông kê của tập dữ liệu.

họ *anchor-free* dự đoán trực tiếp từ mỗi vị trí trên lưới: FCOS dự đoán bốn khoảng cách từ điểm đó tới bốn cạnh của khung [58], CenterNet dự đoán một heatmap tâm vật cộng kích thước [59].

không còn tập anchor, nhưng bây giờ phải trả lời một câu hỏi mới và khó hơn: *vị trí nào trên lưới chịu trách nhiệm cho vật nào*. Bài toán không biến mất, nó đổi tên thành gán mẫu.

### 2 Vấn đề

#### Giải pháp

#### Hệ quả

gán mẫu theo ngưỡng IoU cố định là một lựa chọn tùy tiện. Ngưỡng 0,5 khiến vật nhỏ chỉ nhận được một hai mẫu dương trong khi vật lớn nhận hàng chục, tức mô hình được huấn luyện với mức giám sát rất khác nhau tùy kích thước.

*ATSS* — viết tắt của *adaptive training sample selection* — chọn ngưỡng thích nghi cho từng vật: lấy  $k$  ứng viên gần tâm nhất ở mỗi tầng, tính trung bình và độ lệch chuẩn IoU của chúng, dùng  $\mu + \sigma$  làm ngưỡng [60].

bài báo cho thấy sau khi thống nhất cách gán mẫu, khác biệt giữa detector có anchor và không anchor gần như biến mất — một trong những kết quả ablation quan trọng nhất của cả lĩnh vực.

### 3 Vấn đề

#### Giải pháp

#### Giá phải trả

gán mẫu từng vật độc lập là tối ưu cục bộ. Khi hai vật chồng lấn, một vị trí lưới có thể tốt cho cả hai, và quyết định tham lam cho từng vật riêng lẻ dẫn tới xung đột.

*optimal transport assignment* (OTA) và bản rút gọn SimOTA phát biểu việc gán mẫu thành một bài toán vận tải tối ưu: mỗi ground truth là một nguồn cung  $k$  nhãn dương, mỗi vị trí lưới là một điểm cầu, chi phí là tổ hợp loss phân loại và loss hồi quy [61].

thêm chi phí tính toán trong lúc huấn luyện, không thêm gì lúc suy luận.

4 Vấn đề

NMS là heuristic cuối cùng, và nó là heuristic duy nhất thao tác trên quan hệ giữa các dự đoán. Soft-NMS làm mềm bằng cách giảm điểm thay vì xóa hẳn [62], nhưng vẫn là hậu xử lý.

Giải pháp

DETR đổi phát biểu: dự đoán một tập hợp cố định  $N$  truy vấn, ghép chúng với **chân trị** (*ground truth*) bằng **ghép cặp Hungary** (*Hungarian matching*) một-một, và loss chỉ được định nghĩa sau khi ghép [44]. Đây là ý tưởng **dự đoán tập hợp** (*set prediction*) mà Mục 3 sẽ mượn nguyên vẹn sang segmentation. Vì mỗi vật chỉ có đúng một dự đoán được thưởng, mô hình học cách tự triệt tiêu trùng lặp; NMS biến mất khỏi pipeline.

Giá phải trả

hội tụ cực chậm — bản gốc cần khoảng 500 epoch trong khi Faster R-CNN quen dùng lịch 12–36 epoch — và kém trên vật nhỏ.

5 Vấn đề

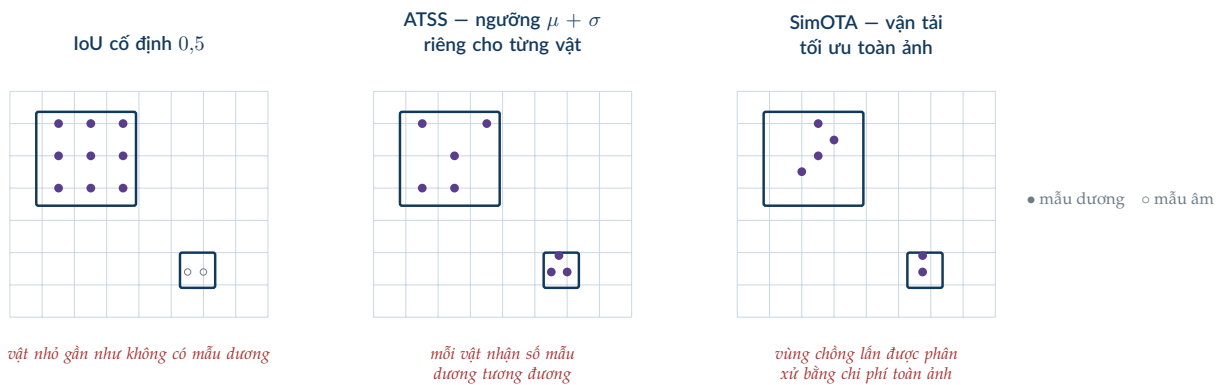
vì sao DETR chậm hội tụ? Hai nguyên nhân được các bản sửa chỉ ra: **attention** dày đặc trên toàn feature map khởi đầu gần như đồng đều nên mất rất lâu để tập trung, và phép ghép một-một dao động giữa các epoch nên tín hiệu giám sát cho mỗi truy vấn không ổn định.

Giải pháp

Deformable DETR cho mỗi truy vấn chỉ lấy mẫu một số ít điểm quanh một điểm tham chiếu học được, giảm cả chi phí lẫn thời gian hội tụ xuống cỡ 50 epoch [37], [63]; DINO thêm truy vấn khởi nhiễu (cho mạng các khung ground truth đã bị nhiễu và bắt khôi phục) để ổn định hoá tín hiệu [64].

Hệ quả

tới 2026, dòng thực dụng như RT-DETR và RF-DETR đã đạt được cả tốc độ lẫn độ chính xác mà không cần NMS.



**Hình 3.7.** Cùng một ảnh, cùng một mạng, cùng một tập vị trí dự đoán — ba quy tắc gán mẫu cho ba tập mẫu dương khác nhau, và do đó ba hàm mất mát khác nhau. Khung đậm là hai vật thật, một lớn một nhỏ. Với ngưỡng IoU cố định, vật lớn nuốt gần hết ngân sách giám sát còn vật nhỏ hầu như không được dạy; ATSS cân lại bằng ngưỡng tính riêng cho từng vật; SimOTA thêm một bước phân xử toàn ảnh nên vị trí tranh chấp không bị gán cho cả hai. Vị trí các chấm ở đây là minh hoạ nguyên lý, không phải kết quả chạy thật.

Hình 3.7 cho thấy thứ mà một bảng số không cho thấy: cái thay đổi giữa ba cột không phải mô hình mà là *đề bài* đưa cho mô hình.

**Bảng 3.1.** Bốn cách trả lời câu hỏi “vị trí nào chịu trách nhiệm cho vật nào”. Đây là thành phần hiểm khi nằm trong abstract nhưng thường giải thích phần lớn chênh lệch giữa hai detector.

| Cách gán          | Cơ chế                                                                     | Giả định                                      | Hỏng khi nào                                                                 |
|-------------------|----------------------------------------------------------------------------|-----------------------------------------------|------------------------------------------------------------------------------|
| IoU cố định       | Anchor nào có IoU vượt ngưỡng thì là dương                                 | Ngưỡng tốt cho mọi kích thước và tỉ lệ        | Vật nhỏ nhận rất ít mẫu dương; đổi phân bố kích thước là phải chỉnh lại tay  |
| Centerness / FCOS | Điểm trong vùng trung tâm của khung là dương, có trọng số theo độ lệch tâm | Tâm hình học là điểm giàu thông tin nhất      | Vật hình khuyên, hình chữ L, hoặc bị che ở giữa — tâm rơi vào nền            |
| ATSS              | Ngưỡng $\mu + \sigma$ tính riêng cho từng vật từ $k$ ứng viên gần nhất     | Phân bố IoU của ứng viên là đơn đỉnh          | Cảnh rất đông; các ứng viên “gần nhất” thuộc về vật khác                     |
| OTA / SimOTA      | Vận tải tối ưu toàn ảnh trên chi phí phân loại + hồi quy                   | Chi phí ghép phản ánh đúng chất lượng dự đoán | Nhạy với chi phí khởi đầu khi mô hình còn kém; tốn thêm thời gian huấn luyện |

Bảng 3.1 nên đọc cùng với một quan sát khó chịu: bốn dòng này thay đổi *hàm mất mát*, không thay đổi mô hình. Hai detector khác nhau ở dòng gán mẫu là hai bài toán tối ưu khác nhau, nên so sánh chúng rồi quy kết chênh lệch cho “kiến trúc” là một suy luận sai ngay từ tiền đề.

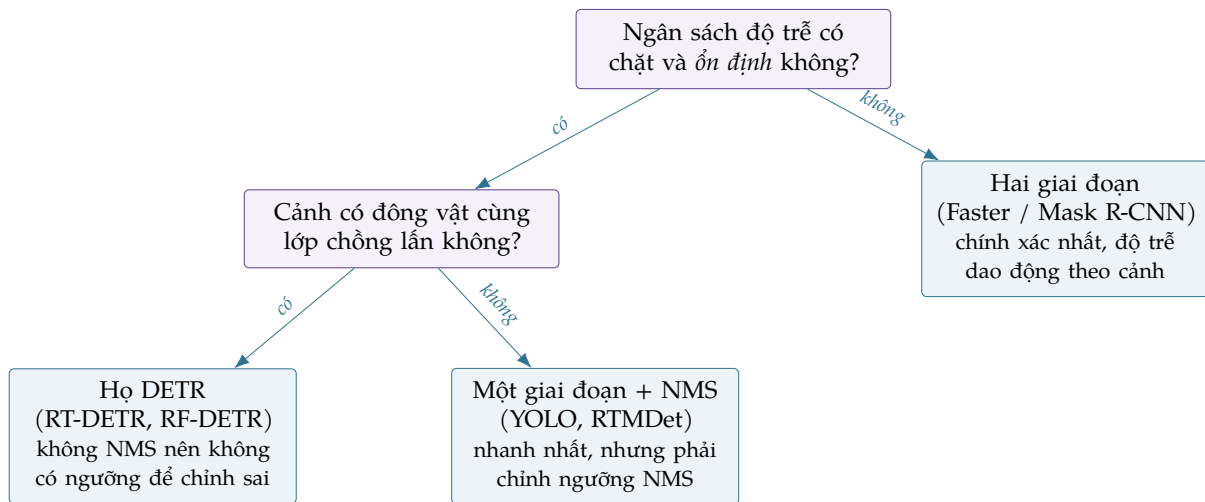
↔ cùng nguyên lý với C/14/03 — Mask2Former mượn nguyên phép ghép cặp một-một này sang miền mask, và nhờ đó hợp nhất được semantic, instance và panoptic.

Hungarian matching của DETR, nhìn qua lăng kính này, chỉ là dòng thứ năm — một cách gán mẫu một-một thay vì một-nhiều — và chính điều đó, chứ không phải Transformer, mới là thứ khiến NMS trở nên thừa.

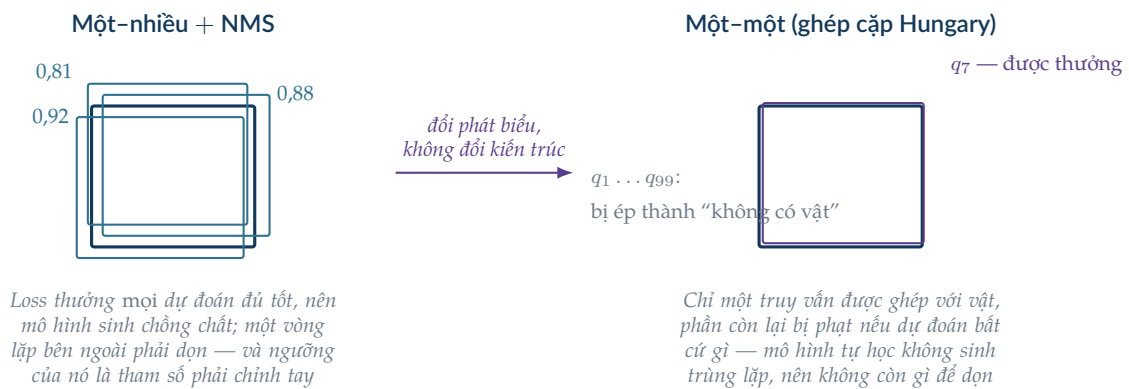
#### CẠM BÃY

Điểm số mà NMS dùng để xếp hạng là điểm *phân loại*, nhưng thứ ta cần giữ lại là khung *định vị* tốt nhất. Hai đại lượng này tương quan yếu hơn ta tưởng, nên một khung tự tin về nhãn nhưng lệch vị trí vẫn có thể xóa mất một khung đúng hơn. Các bản vá là dự đoán thêm một nhánh ước lượng IoU hoặc centerness rồi nhân vào điểm; nhưng nếu bạn thấy mAP ở ngưỡng IoU chặt ( $AP_{75}$ ) tụt nhiều hơn ở ngưỡng lỏng ( $AP_{50}$ ), hãy nghi ngờ đúng chỗ này trước khi đổi backbone.





**Hình 3.8.** Cây quyết định thực dụng: ràng buộc loại bỏ phương án trước khi độ chính xác kịp lên tiếng. Hai nhánh trái đều là một giai đoạn, nhưng khác nhau ở chỗ NMS còn hay không — và trong cảnh đông đúc, đó là khác biệt quyết định chứ không phải vài điểm mAP.



**Hình 3.9.** Vì sao NMS biến mất khi đổi cách ghép. Bên trái, hàm mất mát chấm điểm từng dự đoán độc lập nên nhiều dự đoán cùng đúng và cùng được thưởng; việc chọn một trong số đó phải làm ở hậu xử lý, bằng một heuristic có ngưỡng. Bên phải, phép ghép một-một biến “sinh trùng lặp” thành hành vi bị phạt ngay trong hàm mất mát, nên bước dọn không còn lý do tồn tại. Các số tin cậy chỉ để minh họa.

Hình 3.9 cũng giải thích cái giá: vì mỗi vật chỉ có đúng một truy vấn được thưởng, tín hiệu giám sát cho mỗi truy vấn thưa hơn hẳn, và phép ghép còn dao động giữa các epoch — hai lý do khiến DETR bản gốc hội tụ chậm.

↪ **ràng buộc bởi B/03** — cây quyết định này là hiện thân của nguyên tắc đọc hệ thống: ràng buộc cứng loại phương án trước, độ chính xác chỉ lên tiếng sau.

Hình 3.8 tóm lại mục này dưới dạng dùng được ngay: câu hỏi đầu tiên không phải “mô hình nào mAP cao nhất” mà “ràng buộc nào của tôi là cứng” — đúng cách đọc hệ thống ở B/03-giai-phau-he-thong.

### C. Giới hạn và trường hợp hỏng

Detector một giai đoạn hỏng ở ba chỗ đáng nhớ, mỗi chỗ có một triệu chứng quan sát được:

- **Nhãn nhiễu gặp focal loss.** Focal loss giả định nhãn sạch; trên dữ liệu gán bằng đám đông hoặc bán tự động, nó khuếch đại đúng những mẫu bị gán sai và có thể tệ hơn cross-entropy

thường. *Triệu chứng*: loss huấn luyện đứng yên trong khi một nhóm rất nhỏ mẫu chiếm gần hết *gradient*.

- **Gán nhãn tinh vi ở giai đoạn đầu.** SimOTA và họ hàng dựa vào chi phí ghép, mà chi phí đó vô nghĩa khi mô hình còn kém, nên chúng phải kèm một giai đoạn khởi động. *Triệu chứng*: phân kỳ hoặc hội tụ về nghiệm suy biến trong vài epoch đầu — và chi tiết khắc phục nằm trong code chứ không trong bài.
- **Ngân sách truy vấn cố định của DETR.** Ảnh chứa nhiều vật hơn  $N$  thì phần dư bị mất im lặng, không có cảnh báo nào. *Triệu chứng*: recall giảm đều theo mật độ vật thể; không lộ ra trên COCO nhưng lộ ra ngay khi bạn đếm người trong đám đông.

Với người làm hệ thống thời gian thực, còn một giới hạn nữa ít được nói: bỏ NMS không có nghĩa là bỏ hậu xử lý không xác định. Một giai đoạn có thời gian chạy gần như cố định theo nội dung cảnh, đó là ưu điểm lớn nhất của nó so với hai giai đoạn; nhưng nếu bạn thêm lại NMS trên đầu ra (như hầu hết bản triển khai YOLO làm), thời gian NMS lại tỉ lệ với số khung sống sót, và phương sai độ trễ quay về.

#### CẦN NHỚ

Trước khi tin bất kỳ so sánh detector nào, hãy hỏi hai câu: hai bên dùng cách gán mẫu nào, và hai bên chạy bao nhiêu epoch với augmentation gì. Rất nhiều “cải tiến kiến trúc” công bố thực chất là cải tiến gán mẫu, và nó bị chôn trong code. [T3]

#### D. Kết nối

[C/14-bai-toan-cv-loi/01-detection-hai-giai-doan](#) là tiền đề trực tiếp: mất cân bằng chỉ lộ ra khi bỏ giai đoạn đề xuất, nên mục này chỉ có nghĩa sau mục kia. B/09-thiet-ke-ham-mat-mat là nơi focal loss được đặt cạnh các cách xử lý mất cân bằng khác, và là nơi trả lời câu hỏi tổng quát “khi nào nên sửa ở loss thay vì sửa ở dữ liệu”. Ý tưởng dự đoán tập hợp bằng Hungarian matching quay lại nguyên vẹn ở [C/14-bai-toan-cv-loi/03-segmentation](#) dưới dạng khung truy vấn của Mask2Former — đó là trường hợp hiếm hoi một ý tưởng chuyển miền mà không mất gì. E/24-thi-nghiem-va-ablation cung cấp phương pháp để tự kiểm chứng khẳng định trung tâm của mục này.

#### Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao mất cân bằng foreground-background hầu như không xuất hiện trong Faster R-CNN nhưng lại nghiêm trọng trong SSD, dù cả hai dùng cùng anchor?
2. Hệ số điều biến với  $\gamma = 2$  bằng bao nhiêu tại  $p_t = 0,9$  và tại  $p_t = 0,3$ ? Hai con số đó giải thích thế nào việc focal loss nguy hiểm trên dữ liệu nhiều nhãn?
3. ATSS cho thấy điều gì về câu hỏi “anchor hay anchor-free”? Kết quả đó thay đổi cách bạn đọc một paper detection như thế nào?
4. DETR bỏ được NMS nhờ Hungarian matching chứ không nhờ Transformer. Hãy giải thích tại sao, và điều gì sẽ xảy ra nếu ta ghép một-nhiều thay vì một-một?
5. Bạn đo được  $AP_{50}$  cao nhưng  $AP_{75}$  thấp — hai giả thuyết nào giải thích được, và thí nghiệm nào phân biệt chúng?
6. Trên một robot có ngân sách 20 ms mỗi khung hình, vì sao phương sai độ trễ có thể quan trọng hơn độ trễ trung bình?

- [1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [2] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [3] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *AISTATS*, 2010.
- [4] K. He, X. Zhang, S. Ren, et al., “Delving Deep into Rectifiers,” in *ICCV*, 2015.
- [5] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [6] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [7] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [8] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [9] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [10] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [11] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” in *ICLR*, 2017.
- [12] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [13] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [14] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [15] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [16] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [17] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [18] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [19] G. Huang, Y. Sun, Z. Liu, et al., “Deep Networks with Stochastic Depth,” in *ECCV*, 2016.
- [20] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [21] H. Zhang, M. Cisse, Y. N. Dauphin, et al., “mixup: Beyond Empirical Risk Minimization,” in *ICLR*, 2018.
- [22] S. Yun, D. Han, S. J. Oh, et al., “CutMix: Regularization Strategy to Train Strong Classifiers,” in *ICCV*, 2019.
- [23] E. D. Cubuk, B. Zoph, J. Shlens, et al., “RandAugment: Practical Automated Data Augmentation,” in *CVPR Workshops*, 2020.

- [24] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [25] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [26] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *NeurIPS*, 2012.
- [27] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in *ICLR*, 2015.
- [28] C. Szegedy, W. Liu, et al., “Going Deeper with Convolutions,” in *CVPR*, 2015.
- [29] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [30] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [31] G. Huang, Z. Liu, L. van der Maaten, et al., “Densely Connected Convolutional Networks,” in *CVPR*, 2017.
- [32] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [33] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [34] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [35] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [36] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [37] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.
- [38] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [39] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [40] J. Su, M. Ahmed, Y. Lu, et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, vol. 568, 2024.
- [41] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” *arXiv:2312.00752*, 2023.
- [42] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [43] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.
- [44] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [45] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [46] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [47] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [48] R. Girshick, J. Donahue, T. Darrell, et al., “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation,” in *CVPR*, 2014.

- [49] R. Girshick, "Fast R-CNN," in *ICCV*, 2015.
- [50] S. Ren, K. He, R. Girshick, et al., "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in *NeurIPS*, 2015.
- [51] T.-Y. Lin, P. Dollár, R. Girshick, et al., "Feature Pyramid Networks for Object Detection," in *CVPR*, 2017.
- [52] K. He, G. Gkioxari, P. Dollár, et al., "Mask R-CNN," in *ICCV*, 2017.
- [53] D. Bolya, S. Foley, J. Hays, et al., "TIDE: A General Toolbox for Identifying Object Detection Errors," in *ECCV*, 2020.
- [54] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [55] J. Redmon, S. Divvala, R. Girshick, et al., "You Only Look Once: Unified, Real-Time Object Detection," in *CVPR*, 2016.
- [56] W. Liu, D. Anguelov, D. Erhan, et al., "SSD: Single Shot MultiBox Detector," in *ECCV*, 2016.
- [57] T.-Y. Lin, P. Goyal, R. Girshick, et al., "Focal Loss for Dense Object Detection," in *ICCV*, 2017.
- [58] Z. Tian, C. Shen, H. Chen, et al., "FCOS: Fully Convolutional One-Stage Object Detection," in *ICCV*, 2019.
- [59] X. Zhou, D. Wang, and P. Krähenbühl, "Objects as Points," *arXiv:1904.07850*, 2019.
- [60] S. Zhang, C. Chi, Y. Yao, et al., "Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection," in *CVPR*, 2020.
- [61] Z. Ge, S. Liu, Z. Li, et al., "OTA: Optimal Transport Assignment for Object Detection," in *CVPR*, 2021.
- [62] N. Bodla, B. Singh, R. Chellappa, et al., "Soft-NMS: Improving Object Detection with One Line of Code," in *ICCV*, 2017.
- [63] X. Zhu, W. Su, L. Lu, et al., "Deformable DETR: Deformable Transformers for End-to-End Object Detection," in *ICLR*, 2021.
- [64] H. Zhang, F. Li, S. Liu, et al., "DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection," in *ICLR*, 2023.
- [65] J. Long, E. Shelhamer, and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," in *CVPR*, 2015.
- [66] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *MICCAI*, 2015.
- [67] L.-C. Chen, G. Papandreou, I. Kokkinos, et al., "DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs," *IEEE TPAMI*, vol. 40, no. 4, pp. 834–848, 2018.
- [68] E. Xie, W. Wang, Z. Yu, et al., "SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers," in *NeurIPS*, 2021.
- [69] B. Cheng, I. Misra, A. G. Schwing, et al., "Masked-Attention Mask Transformer for Universal Image Segmentation," in *CVPR*, 2022.
- [70] A. Kirillov, K. He, R. Girshick, et al., "Panoptic Segmentation," in *CVPR*, 2019.
- [71] A. Kirillov, E. Mintun, N. Ravi, et al., "Segment Anything," in *ICCV*, 2023.
- [72] K. Sun, B. Xiao, D. Liu, et al., "Deep High-Resolution Representation Learning for Human Pose Estimation," in *CVPR*, 2019.

- [73] Z. Cao, T. Simon, S.-E. Wei, et al., “Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields,” in *CVPR*, 2017.
- [74] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [75] J. Deng, J. Guo, N. Xue, et al., “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *CVPR*, 2019.
- [76] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [77] K. Xu, J. Ba, R. Kiros, et al., “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention,” in *ICML*, 2015.
- [78] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *ICLR*, 2015.
- [79] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [80] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [81] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [82] K. Simonyan and A. Zisserman, “Two-Stream Convolutional Networks for Action Recognition in Videos,” in *NeurIPS*, 2014.
- [83] J. Carreira and A. Zisserman, “Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset,” in *CVPR*, 2017.
- [84] D. Tran, H. Wang, L. Torresani, et al., “A Closer Look at Spatiotemporal Convolutions for Action Recognition,” in *CVPR*, 2018.
- [85] C. Feichtenhofer, H. Fan, J. Malik, et al., “SlowFast Networks for Video Recognition,” in *ICCV*, 2019.
- [86] A. Dosovitskiy, P. Fischer, E. Ilg, et al., “FlowNet: Learning Optical Flow with Convolutional Networks,” in *ICCV*, 2015.
- [87] D. Sun, X. Yang, M.-Y. Liu, et al., “PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume,” in *CVPR*, 2018.
- [88] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.
- [89] A. Bewley, Z. Ge, L. Ott, et al., “Simple Online and Realtime Tracking,” in *ICIP*, 2016.
- [90] Y. Zhang, P. Sun, Y. Jiang, et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *ECCV*, 2022.
- [91] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [92] K. Bernardin and R. Stiefelhagen, “Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics,” *EURASIP J. Image and Video Processing*, 2008.
- [93] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [94] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [95] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.



- [96] S. Wang, V. Leroy, Y. Cabon, et al., "DUST3R: Geometric 3D Vision Made Easy," in *CVPR*, 2024.
- [97] D. Eigen, C. Puhrsch, and R. Fergus, "Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network," in *NeurIPS*, 2014.
- [98] R. Ranftl, K. Lasinger, D. Hafner, et al., "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer," *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [99] J. Sturm, N. Engelhard, F. Endres, et al., "A Benchmark for the Evaluation of RGB-D SLAM Systems," in *IROS*, 2012.
- [100] C. R. Qi, H. Su, K. Mo, et al., "PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation," in *CVPR*, 2017.
- [101] C. R. Qi, L. Yi, H. Su, et al., "PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space," in *NeurIPS*, 2017.
- [102] A. H. Lang, S. Vora, H. Caesar, et al., "PointPillars: Fast Encoders for Object Detection from Point Clouds," in *CVPR*, 2019.
- [103] T. Müller, A. Evans, C. Schied, et al., "Instant Neural Graphics Primitives with a Multiresolution Hash Encoding," *ACM TOG*, vol. 41, no. 4, 2022.
- [104] B. Kerbl, G. Kopanas, T. Leimkühler, et al., "3D Gaussian Splatting for Real-Time Radiance Field Rendering," *ACM TOG*, vol. 42, no. 4, 2023.
- [105] R. Zhang, P. Isola, A. A. Efros, et al., "The Unreasonable Effectiveness of Deep Features as a Perceptual Metric," in *CVPR*, 2018.
- [106] D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," in *ICLR*, 2014.
- [107] I. Goodfellow, J. Pouget-Abadie, M. Mirza, et al., "Generative Adversarial Nets," in *NeurIPS*, 2014.
- [108] T. Karras, S. Laine, and T. Aila, "A Style-Based Generator Architecture for Generative Adversarial Networks," in *CVPR*, 2019.
- [109] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," in *NeurIPS*, 2020.
- [110] Y. Song, J. Sohl-Dickstein, D. P. Kingma, et al., "Score-Based Generative Modeling Through Stochastic Differential Equations," in *ICLR*, 2021.
- [111] J. Johnson, A. Alahi, and L. Fei-Fei, "Perceptual Losses for Real-Time Style Transfer and Super-Resolution," in *ECCV*, 2016.
- [112] P. Isola, J.-Y. Zhu, T. Zhou, et al., "Image-to-Image Translation with Conditional Adversarial Networks," in *CVPR*, 2017.
- [113] L. A. Gatys, A. S. Ecker, and M. Bethge, "Image Style Transfer Using Convolutional Neural Networks," in *CVPR*, 2016.
- [114] X. Huang and S. Belongie, "Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization," in *ICCV*, 2017.
- [115] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," in *NeurIPS Deep Learning Workshop*, 2015.
- [116] J. Song, C. Meng, and S. Ermon, "Denoising Diffusion Implicit Models," in *ICLR*, 2021.
- [117] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, et al., "Flow Matching for Generative Modeling," in *ICLR*, 2023.
- [118] J. Ho and T. Salimans, "Classifier-Free Diffusion Guidance," *arXiv:2207.12598*, 2022.
- [119] R. Rombach, A. Blattmann, D. Lorenz, et al., "High-Resolution Image Synthesis with Latent Diffusion Models," in *CVPR*, 2022.
- [120] W. Peebles and S. Xie, "Scalable Diffusion Models with Transformers," in *ICCV*, 2023.

- [121] L. Zhang, A. Rao, and M. Agrawala, "Adding Conditional Control to Text-to-Image Diffusion Models," in *ICCV*, 2023.
- [122] Y. Blau and T. Michaeli, "The Perception-Distortion Tradeoff," in *CVPR*, 2018.
- [123] C. Ledig, L. Theis, F. Huszár, et al., "Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network," in *CVPR*, 2017.
- [124] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., "Image Quality Assessment: From Error Visibility to Structural Similarity," *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [125] M. Heusel, H. Ramsauer, T. Unterthiner, et al., "GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium," in *NeurIPS*, 2017.
- [126] T. Brooks, B. Mildenhall, T. Xue, et al., "Unprocessing Images for Learned Raw Denoising," in *CVPR*, 2019.
- [127] A. Abdelhamed, S. Lin, and M. S. Brown, "A High-Quality Denoising Dataset for Smartphone Cameras," in *CVPR*, 2018.
- [128] J. Deng, W. Dong, R. Socher, et al., "ImageNet: A Large-Scale Hierarchical Image Database," in *CVPR*, 2009.
- [129] A. Radford, J. W. Kim, C. Hallacy, et al., "Learning Transferable Visual Models from Natural Language Supervision," in *ICML*, 2021.
- [130] J. Li, D. Li, S. Savarese, et al., "BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models," in *ICML*, 2023.
- [131] H. Liu, C. Li, Q. Wu, et al., "Visual Instruction Tuning," in *NeurIPS*, 2023.
- [132] S. Kornblith, J. Shlens, and Q. V. Le, "Do Better ImageNet Models Transfer Better?" In *CVPR*, 2019.
- [133] T. Chen, S. Kornblith, M. Norouzi, et al., "A Simple Framework for Contrastive Learning of Visual Representations," in *ICML*, 2020.
- [134] K. He, H. Fan, Y. Wu, et al., "Momentum Contrast for Unsupervised Visual Representation Learning," in *CVPR*, 2020.
- [135] J.-B. Grill, F. Strub, F. Altché, et al., "Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning," in *NeurIPS*, 2020.
- [136] M. Caron, H. Touvron, I. Misra, et al., "Emerging Properties in Self-Supervised Vision Transformers," in *ICCV*, 2021.
- [137] K. He, X. Chen, S. Xie, et al., "Masked Autoencoders Are Scalable Vision Learners," in *CVPR*, 2022.
- [138] A. van den Oord, Y. Li, and O. Vinyals, "Representation Learning with Contrastive Predictive Coding," in *arXiv:1807.03748*, 2018.
- [139] J. Kaplan, S. McCandlish, T. Henighan, et al., "Scaling Laws for Neural Language Models," *arXiv:2001.08361*, 2020.
- [140] X. Zhai, A. Kolesnikov, N. Houlsby, et al., "Scaling Vision Transformers," in *CVPR*, 2022.
- [141] J. Hoffmann, S. Borgeaud, A. Mensch, et al., "Training Compute-Optimal Large Language Models," in *NeurIPS*, 2022.
- [142] C. G. Northcutt, A. Athalye, and J. Mueller, "Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks," in *NeurIPS Datasets and Benchmarks*, 2021.
- [143] B. Recht, R. Roelofs, L. Schmidt, et al., "Do ImageNet Classifiers Generalize to ImageNet?" In *ICML*, 2019.
- [144] N. Houlsby, A. Giurgiu, S. Jastrzebski, et al., "Parameter-Efficient Transfer Learning for NLP," in *ICML*, 2019.

- [145] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [146] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *RecSys*, 2019.

### 3.3 Phân vùng ảnh và sự hợp nhất (Segmentation and Unification)

**PHỤ THUỘC** C/13-kien-truc-backbone/01-dong-cnn (stride, trường tiếp nhận (*receptive field*), upsampling); C/14-bai-toan-cv-loi/02-detection-mot-giai-doan (dự đoán tập hợp bằng Hungarian matching — cần cho phần Mask2Former).

#### TÓM TẮT

Mục này theo một mâu thuẫn duy nhất: mạng phân loại giỏi nhờ vứt bỏ thông tin không gian, còn phân vùng lại cần đúng thông tin vừa bị vứt. Bốn kiến trúc lớn — FCN, U-Net, DeepLab, SegFormer — là bốn cách khác nhau để lấy lại thông tin đó, mỗi cách trả một loại giá khác nhau; Mask2Former thì không lấy lại mà đổi hẳn phát biểu, và nhờ vậy hợp nhất được semantic, instance và panoptic. Đọc xong bạn chọn được kiến trúc phân vùng từ ràng buộc (bộ nhớ, lượng nhân, yêu cầu về biên) thay vì từ bảng xếp hạng, và đọc được mIoU với đúng mức hoài nghi. Nếu chỉ nhớ một điều: mọi kiến trúc segmentation đều là một câu trả lời cho “chi tiết không gian đã mất ở đâu, lấy lại bằng cách nào” — và mIoU cao không hề bảo đảm biên đúng hay vật nhỏ không bị bỏ sót.

#### KIẾN THỨC CẦN CÓ TRƯỚC

- **semantic segmentation** — gán cho mỗi *pixel* một nhãn lớp. Hai chiếc xe cạnh nhau đều là “xe” và không được tách ra.
- **instance segmentation** — ngoài nhãn lớp còn tách từng cá thể: xe thứ nhất và xe thứ hai là hai mask khác nhau.
- **panoptic segmentation** — gộp cả hai: các lớp đếm được thì tách cá thể, các lớp không đếm được (bầu trời, mặt đường) thì chỉ cần nhãn lớp.
- **mask** — một lưới nhị phân cùng kích thước ảnh, đánh dấu pixel nào thuộc về vùng đang nói tới. Đây là “dạng đầu ra” của mọi bài toán trong mục này.
- **output stride** — tỉ lệ thu nhỏ giữa ảnh và feature map. Output stride 32 nghĩa là ảnh  $1024 \times 1024$  còn lại lưới  $32 \times 32$ , nên mỗi ô đại diện cho  $32 \times 32$  pixel và biên chỉ định vị được tới độ thô đó.
- **encoder-decoder** — encoder là phần đi xuống, thu nhỏ dần độ phân giải để lấy ngữ nghĩa; decoder là phần đi lên, khôi phục độ phân giải. *Skip connection* là đường nối tắt từ một tầng encoder sang tầng decoder cùng cỡ.
- **trường tiếp nhận (receptive field)** — vùng ảnh gốc mà một ô đặc trưng “nhìn thấy”. Muốn hiểu ngữ cảnh rộng thì cần trường tiếp nhận lớn, mà cách rẻ nhất để mở rộng nó lại là thu nhỏ độ phân giải — đó chính là mâu thuẫn của mục này.
- **mIoU** — thước đo chuẩn của segmentation: với mỗi lớp lấy diện tích giao chia diện tích hợp giữa vùng dự đoán và vùng thật, rồi trung bình trên các lớp.

#### Đặt vấn đề

Detection trả lời “có cái gì, ở khoảng nào”. Nhiều ứng dụng cần chặt hơn: một khung chữ nhật quanh một con đường không nói được đâu là mặt đường có thể đi. Segmentation đẩy độ phân giải của câu trả lời xuống tận pixel, và ngay lập tức va vào một mâu thuẫn mà cả một thập kỷ nghiên cứu xoay quanh: mạng phân loại tốt nhờ vứt bỏ thông tin không gian — pooling và stride là cơ chế tạo ra bất biến tịnh tiến — trong khi segmentation lại cần đúng thông tin không gian

vừa bị vớt bỏ. Ta muốn ngữ nghĩa sâu và định vị chính xác cùng lúc, còn kiến trúc phân loại được thiết kế để đánh đổi cái sau lấy cái trước.

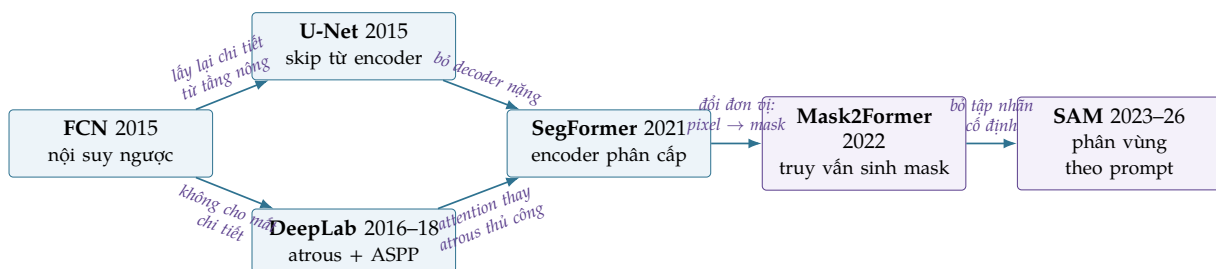
Một phép tính nhỏ cho thấy mâu thuẫn này cụ thể tới mức nào. Backbone phân loại điển hình có *output stride* 32: ảnh  $1024 \times 1024$  ra feature map  $32 \times 32$ . Mỗi ô đặc trưng đại diện cho một mảng  $32 \times 32$  pixel. Dù bạn nội suy lên lại bằng phép gì, thông tin để phân biệt hai pixel kề nhau bên trong cùng một ô đã không còn tồn tại; sai số định vị biên có sẵn cỡ chục pixel. Toàn bộ mạch dưới đây là các cách khác nhau để phá cái sào đó, và mỗi cách trả một loại giá khác nhau.

**Học xong mục này bạn làm được gì mà trước đó không làm được?** Cho một yêu cầu cụ thể — “bám mép đường tới 2 pixel”, “chạy trên Jetson với 4 GB”, “chỉ có 200 ảnh gán nhãn” — bạn suy ra được kiến trúc nào khả thi và vì sao, trước khi chạy thí nghiệm nào.

#### TRỰC GIÁC

Semantic segmentation cổ điển hỏi mỗi pixel: “bạn thuộc lớp nào?” — một trăm nghìn câu hỏi độc lập, và mô hình không có chỗ nào để nói “ba nghìn pixel này cùng thuộc *một* vật”. Mask2Former đảo chiều điểm danh: thay vì hỏi từng pixel, nó đưa ra một trăm cái tên (truy vấn) và bắt mỗi cái tên tự nhận “tôi là vùng nào, và tôi là cái gì”. Đảo chiều câu hỏi là lý do duy nhất khiến semantic, instance và panoptic gộp được vào một mô hình: một vật thể trở thành một đơn vị được nói tới, thay vì một tính chất phát sinh từ hậu xử lý.

#### A. Bốn cách phá sào độ phân giải, rồi một lần đổi phát biểu (From FCN to Mask2Former)



**Hình 3.10.** Dòng thời gian phân vùng ảnh, đọc theo nút thắt được gỡ ở mỗi bước. Hai nhánh giữa (U-Net và DeepLab) tấn công *cùng một* vấn đề mất chi tiết từ hai phía đối lập nên có thể kết hợp; hai bước cuối không gỡ nút thắt kỹ thuật mà đổi phát biểu bài toán — đơn vị được nói tới chuyển từ pixel sang mask, rồi tập nhãn cố định bị bỏ hẳn.

Hình 3.10 đáng nhìn trước khi đọc mạch bên dưới, vì nó cho thấy ngay một điều mà đọc tuần tự khó thấy: bốn bước đầu là kỹ thuật, hai bước cuối là đổi phát biểu.

#### Mạch 3 — segmentation và sự hợp nhất

##### 1 Vấn đề

mạng phân loại không sinh được đầu ra theo pixel.

##### Giải pháp

FCN vớt các tầng fully connected, coi chúng như conv  $1 \times 1$ , rồi dùng transposed conv để nâng độ phân giải trở lại kích thước ảnh [65].

##### Hệ quả

lần đầu tiên toàn bộ pipeline là một mạng duy nhất huấn luyện đầu-cuối.

##### Vấn đề mới

biên rất thô, vì chi tiết đã mất ở đường xuống chứ không phải ở đường lên — nội suy không thể tạo lại thứ đã bị bỏ.

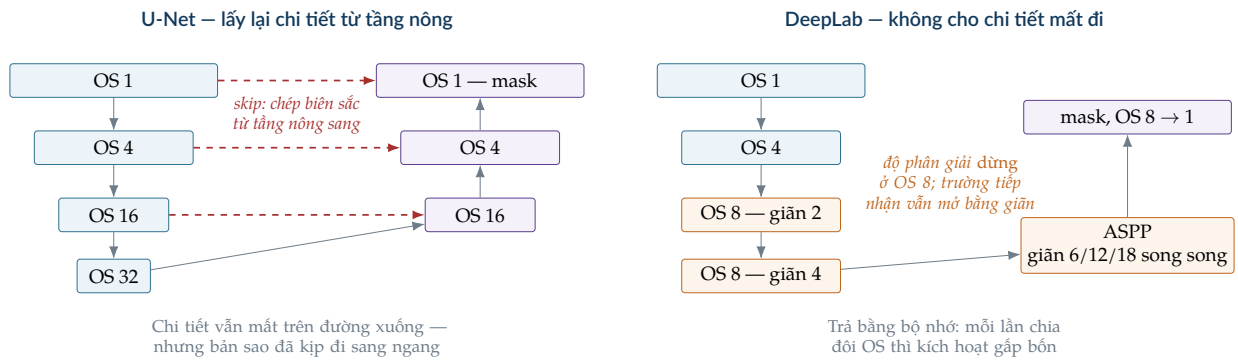
##### 2 Vấn đề

chi tiết mất ở đường xuống thì phải lấy lại từ đường xuống.

##### Giải pháp

U-Net nối thẳng feature ở mỗi mức của encoder sang mức tương ứng của decoder [66]. Feature nông giữ biên sắc, feature sâu giữ ngữ nghĩa, decoder hợp nhất hai loại.

- Giả định biên trong ảnh nông và biên ngữ nghĩa trùng nhau — đúng với ảnh y tế và ảnh vệ tinh, kém đúng khi vật có hoa văn mạnh bên trong.
- Hệ quả đây vẫn là kiến trúc mặc định của ảnh y tế sau mười năm, và [MIC-DKFZ/nnUNet](#) cho thấy một U-Net cấu hình tự động vẫn rất khó bị đánh bại.
- 3 Vấn đề** một hướng tấn công khác: dừng để mất chi tiết ngay từ đầu.
- Giải pháp** **tích chập giãn** (*atrous / dilated convolution*) chen khoảng trống giữa các trọng số, mở rộng trường tiếp nhận mà không giảm độ phân giải [36]; DeepLab dùng nó để giữ output stride ở mức 8 hoặc 16, và ghép nhiều tỉ lệ giãn song song thành *ASPP* [67].
- Giá phải trả** bộ nhớ và tính toán, vì kích hoạt trung gian to gấp bốn lần mỗi khi giảm output stride một nửa ở cả hai chiều.
- Vấn đề mới** hiệu ứng lưới (gridding) khi các tỉ lệ giãn có ước chung, làm mất mẫu ở một số vị trí.
- 4 Vấn đề** decoder nặng và thiết kế đa tỉ lệ thủ công.
- Giải pháp** SegFormer dùng encoder Transformer phân cấp sinh sẵn đặc trưng ở bốn tỉ lệ, rồi ghép chúng bằng một decoder chỉ gồm vài tầng MLP [68].
- Hệ quả attention lo phần ngữ cảnh xa mà trước đây phải mua bằng atrous, nên decoder được phép rẻ.
- Giả định đủ dữ liệu để Transformer bù lại phần inductive bias đã bỏ.
- **ràng buộc bởi** [C/13/03](#) — bỏ bớt thiên lệch quy nạp thì phải mua lại bằng dữ liệu — đó là lý do SegFormer đói dữ liệu hơn U-Net chứ không phải vì nó “phức tạp hơn”.
- 5 Vấn đề** semantic, instance và panoptic là ba bài toán với ba code base, ba loss, ba metric.
- Giải pháp** Mask2Former phát biểu cả ba như **phân loại mặt nạ** (*mask classification*): một tập cố định **truy vấn** (*query*), mỗi truy vấn sinh ra một mask nhị phân cộng một nhãn lớp, ghép với ground truth bằng Hungarian matching — đúng ý tưởng dự đoán tập hợp của DETR, chuyển sang miền mask [69], [70].
- **tiền đề** [C/14/02](#) — khung truy vấn ở đây chỉ là dự đoán tập hợp của DETR chuyển sang miền mask; chưa hiểu vì sao ghép cặp một-một bỏ được NMS thì không hiểu được vì sao ba bài toán hợp nhất được.
- Đóng góp kỹ thuật then chốt là **attention có mặt nạ** (*masked attention*): mỗi truy vấn chỉ chú ý vào vùng mà nó đang dự đoán, khiến hội tụ nhanh hơn nhiều so với attention toàn cục.
- Hệ quả ba bài toán chỉ khác nhau ở cách hậu xử lý tập mask, không khác nhau ở mô hình. Khi ba bài toán hợp nhất được, thường là dấu hiệu ta đã tìm ra phát biểu đúng.
- 6 Vấn đề** tập nhãn cố định là ràng buộc cuối cùng. Mọi mô hình ở trên chỉ biết những lớp có trong tập huấn luyện.
- Giải pháp** mô hình nền tảng nhận prompt — SAM phân vùng theo điểm, khung hay vùng do người chỉ, không cần biết vật đó tên gì [71]; các hệ open-vocabulary như Grounding DINO nhận mô tả văn bản; tới 2026, SAM 3 nhận prompt ở mức khái niệm và chạy được cả trên video.
- Giá phải trả** mô hình lớn, khó kiểm soát hành vi ở biên, và điều quan trọng với kỹ sư: đầu ra không kèm một khái niệm “lớp” ổn định để hệ thống phía sau dựa vào.



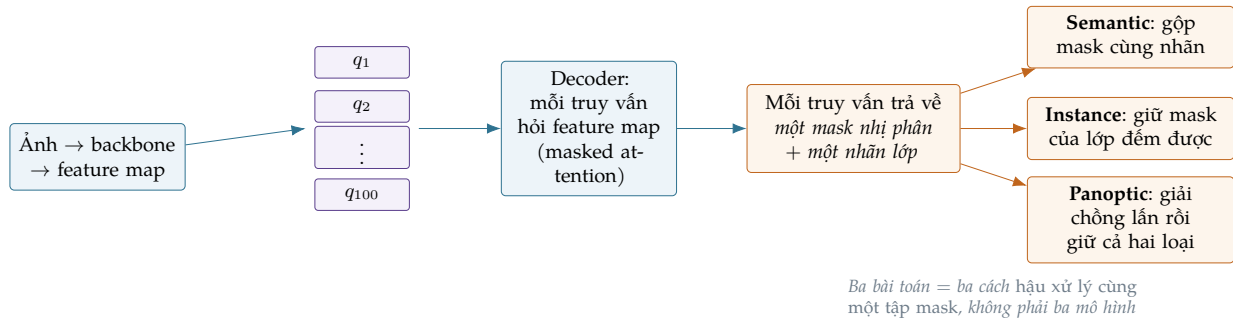
**Hình 3.11.** Hai lời giải đối lập cho cùng một vấn đề. U-Net chấp nhận để chi tiết mất trên đường xuống rồi chép nó sang ngang bằng skip connection; DeepLab từ chối để mất, giữ output stride (OS) ở mức 8 và mua trường tiếp nhận bằng tích chập giãn thay vì bằng thu nhỏ. Vì hai cơ chế độc lập nhau nên chúng cộng được: nhiều kiến trúc thực dụng dùng cả hai.

Hình 3.11 là lý do nên đọc mạch trên theo trục “chi tiết mất ở đâu” chứ không theo trục thời gian: hai bước liền nhau trong lịch sử lại là hai hướng tấn công khác nhau, không phải cái sau thay thế cái trước.

**Bảng 3.2.** Năm cách lấy lại độ phân giải, cùng cái giá và chế độ hồng riêng. Ba dòng đầu tấn công cùng một vấn đề từ ba phía khác nhau, nên chúng có thể kết hợp; dòng cuối đối phát biểu chứ không đối cơ chế.

| Cách          | Cơ chế                                               | Mạnh khi nào                            | Hồng khi nào                                                            |
|---------------|------------------------------------------------------|-----------------------------------------|-------------------------------------------------------------------------|
| FCN           | Nội suy ngược từ feature sâu                         | Vùng lớn, biên không quan trọng         | Biên thô; vật mảnh (dây, cột) biến mất hẳn                              |
| U-Net         | Skip connection encoder → decoder                    | Ít dữ liệu, biên rõ, ảnh y tế           | Vật có kết cấu nội bộ mạnh: skip mang cả nhiễu lên                      |
| Atrous / ASPP | Giữ output stride nhỏ, mở trường tiếp nhận bằng giãn | Cần ngữ cảnh xa mà vẫn cần biên         | Bộ nhớ tăng mạnh; gridding nếu tỉ lệ giãn không nguyên tố cùng nhau     |
| SegFormer     | Encoder phân cấp + decoder MLP                       | Dữ liệu nhiều, cần ngữ cảnh toàn ảnh    | Dữ liệu ít; chi phí attention ở độ phân giải cao                        |
| Mask2Former   | Truy vấn sinh mask + Hungarian matching              | Cần instance/panoptic trong một mô hình | Số truy vấn cố định; ảnh nhiều instance hơn số truy vấn thì mất im lặng |





**Hình 3.12.** Khung truy vấn của Mask2Former. Mô hình không hỏi từng pixel “bạn thuộc lớp nào” mà đưa ra một tập cố định truy vấn, mỗi truy vấn tự nhận một vùng và một nhãn; huấn luyện ghép các truy vấn với chân trị bằng ghép cặp Hungary như DETR. Vì đơn vị được nói tới là *một mask kèm một nhãn*, cả ba bài toán phân vùng chỉ còn khác nhau ở bước đọc tập mask đó, nên chúng dùng chung một mô hình và một hàm mất mát.

Hình 3.12 làm rõ điều mà hộp trực giác ở đầu mục chỉ nói bằng lời: cái được hợp nhất không phải ba kiến trúc mà là ba *cách phát biểu đầu ra*.

### B. Một phép tính về cái giá của độ phân giải (The Resolution Budget)

Bảng 3.2 nói “bộ nhớ tăng mạnh” — con số cụ thể đáng ghi nhớ. Với ảnh  $1024 \times 1024$  và một tầng đặc trưng 256 kênh: ở output stride 16, feature map là  $64 \times 64 \times 256 \approx 1,05$  triệu phần tử, tức 4,2 MB ở fp32. Hạ xuống output stride 8, kích thước không gian nhân đôi ở cả hai chiều nên số phần tử nhân bốn: 16,8 MB cho *một* tensor. **Lượt ngược** (*backward pass*) phải giữ kích hoạt của mọi tầng, nên hệ số bốn này nhân vào toàn bộ phần sâu của mạng. Đây là lý do rất thực tế khiến hệ thống triển khai hiếm khi chạy ở output stride 4. Nó cũng giải thích vì sao U-Net rẻ hơn nhiều so với cảm giác ban đầu: U-Net chỉ giữ đặc trưng phân giải cao ở các tầng *ngang*, nơi số kênh còn ít.

Ba tầng cùng giãn 2



một nửa số pixel không bao giờ được nhìn — hiệu ứng lưới

Ba tầng giãn 1, 2, 3



mọi pixel đều đóng góp, cùng trường tiếp nhận

(một chiều, kernel  $3 \times 1$ , tâm ở giữa; ô tô đậm là pixel thực sự ảnh hưởng tới đầu ra ở tâm)

**Hình 3.13.** Hiệu ứng lưới của tích chập giãn, vẽ trên một hàng pixel. Khi mọi tầng dùng cùng tỉ lệ giãn chẵn, các vị trí lấy mẫu luôn rơi vào cùng một lớp chẵn lẻ, nên trường tiếp nhận trông thì rộng mà thực chất thừa và bỏ sót một nửa tín hiệu. Chọn dãy tỉ lệ nguyên tố cùng nhau làm các lớp dư khác nhau phủ lẫn nhau, và trường tiếp nhận trở lại đặc.

Hình 3.13 là ví dụ nhỏ nhất cho thấy vì sao “mở trường tiếp nhận” và “nhìn thấy mọi pixel” là hai chuyện khác nhau.

Về atrous: một kernel  $3 \times 3$  với tỉ lệ giãn  $r$  có trường tiếp nhận hiệu dụng  $3 + 2(r - 1)$ . Xếp chồng ba tầng giãn 1, 2, 4 cho trường tiếp nhận  $3 \rightarrow 7 \rightarrow 15$  mà không hề giảm độ phân giải. Nhưng nếu ba tầng cùng giãn 2, các vị trí lấy mẫu luôn rơi vào cùng một lớp chẵn lẻ. Một nửa số pixel khi đó không bao giờ được nhìn tới. Đó là hiệu ứng gridding, và cách chữa là chọn dãy tỉ lệ giãn nguyên tố cùng nhau.

### C. Giới hạn và trường hợp hỏng

Chế độ hỏng nguy hiểm nhất của segmentation không nằm ở mô hình mà ở thước đo. **giao trên hợp trung bình** (*mean intersection over union, mIoU*) trung bình theo lớp, nên một lớp hiếm nhưng quan trọng (người đi bộ, vật cản nhỏ) đóng góp bằng một lớp phủ kín ảnh (bầu trời), điều này

tốt; nhưng bên trong mỗi lớp, IoU trung bình theo *diện tích*, nên sai vài pixel ở biên của một vùng lớn gần như không bị phạt. Hệ quả là một mô hình có mIoU 0,85 vẫn có thể cho biên răng cưa tới mức không bám nổi mép đường. Nó cũng có thể bỏ sót hoàn toàn một vật cản  $20 \times 20$  pixel mà chỉ mất một phần nghìn điểm. Nếu ứng dụng của bạn quan tâm tới biên hay tới vật nhỏ, mIoU là thước đo sai và phải bổ sung metric theo biên hoặc theo instance.

→ **hệ quả của B/07** — một con số tổng luôn che giấu chế độ khó; cách chữa duy nhất là báo cáo theo lát, ở đây là lát theo kích thước vật và lát theo độ dài biên.

Ba chế độ hỏng còn lại đáng nêu tên riêng:

- **Vật mảnh.** Dây điện, chân bàn, cột — biến mất ở output stride lớn, và không có hậu xử lý nào lấy lại được thứ đã không được lấy mẫu. *Dấu hiệu:* IoU của lớp đó gần 0 trong khi mIoU tổng vẫn cao.
- **Nhiều instance cùng lớp chồng lấn.** Các mô hình theo truy vấn chia sẻ một ngân sách cố định, nên phần vượt ngân sách mất im lặng — đúng chế độ hỏng của DETR ở Mục 2, xuất hiện lại ở miền mask.
- **Khái niệm không ổn định ở mô hình nhận prompt.** Cùng một ảnh, hai prompt gần như đồng nghĩa cho ra hai phân vùng ở hai mức chi tiết khác nhau (cái cửa, hay tay nắm cửa?). Hệ thống hạ nguồn giả định “một vật, một mask” sẽ hỏng theo cách rất khó tái hiện, vì lỗi phụ thuộc câu chữ chứ không phụ thuộc ảnh.

#### CẦN NHỚ

Đừng nhớ danh sách kiến trúc; hãy nhớ rằng mọi kiến trúc segmentation đều là một câu trả lời cho cùng một câu hỏi “chi tiết không gian đã mất ở đâu, và lấy lại bằng cách nào”. Skip lấy lại từ tầng nông, atrous không cho mất, attention học lại quan hệ xa, query đổi hần đơn vị được nói tới. [T3]

#### D. Kết nối

[C/14-bai-toan-cv-loi/02-detection-mot-giai-doan](#) là tiền đề của phần Mask2Former: nếu chưa hiểu vì sao Hungarian matching bỏ được NMS thì khung truy vấn ở đây chỉ là một kiến trúc lạ. [C/13-kien-truc-backbone/03-vit-va-inductive-bias](#) giải thích cái giá dữ liệu mà SegFormer phải trả. [C/18-pretraining-va-foundation](#) là nơi bàn kỹ về bước dịch chuyển sang mô hình nền tảng và kinh tế học của nó. [C/15-thi-giac-khong-thoi-gian/03-flow-tracking-va-rang-buoc](#) nối tiếp trực tiếp: một mô hình segmentation per-frame có mIoU cao vẫn cho video nhấp nháy, và lý do là mô hình này chưa hề có khái niệm thời gian.

#### ❏ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao nội suy ngược từ feature ở output stride 32 không thể tạo lại biên sắc, dù ta dùng phép nội suy tốt đến đâu?
2. U-Net và DeepLab tấn công cùng một vấn đề từ hai phía nào, và trong tình huống nào thì cách của U-Net tốt hơn hẳn?
3. Ba tầng atrous cùng tỉ lệ giãn 2 gây ra chuyện gì, và vì sao chọn tỉ lệ nguyên tố cùng nhau lại chữa được?
4. Vì sao việc semantic, instance và panoptic hợp nhất được vào một mô hình lại là bằng chứng cho thấy phát biểu bài toán đúng, chứ không chỉ là tiện lợi kỹ thuật?
5. Mô hình của bạn đạt mIoU 0,85 nhưng bỏ sót vật cản nhỏ trên đường. Vì sao metric không phát hiện được, và bạn thay bằng metric nào?

- [1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [2] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [3] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *AISTATS*, 2010.
- [4] K. He, X. Zhang, S. Ren, et al., “Delving Deep into Rectifiers,” in *ICCV*, 2015.
- [5] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [6] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [7] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [8] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [9] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [10] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [11] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” in *ICLR*, 2017.
- [12] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [13] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [14] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [15] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [16] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [17] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [18] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [19] G. Huang, Y. Sun, Z. Liu, et al., “Deep Networks with Stochastic Depth,” in *ECCV*, 2016.
- [20] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [21] H. Zhang, M. Cisse, Y. N. Dauphin, et al., “mixup: Beyond Empirical Risk Minimization,” in *ICLR*, 2018.
- [22] S. Yun, D. Han, S. J. Oh, et al., “CutMix: Regularization Strategy to Train Strong Classifiers,” in *ICCV*, 2019.
- [23] E. D. Cubuk, B. Zoph, J. Shlens, et al., “RandAugment: Practical Automated Data Augmentation,” in *CVPR Workshops*, 2020.

- [24] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [25] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [26] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *NeurIPS*, 2012.
- [27] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in *ICLR*, 2015.
- [28] C. Szegedy, W. Liu, et al., “Going Deeper with Convolutions,” in *CVPR*, 2015.
- [29] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [30] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [31] G. Huang, Z. Liu, L. van der Maaten, et al., “Densely Connected Convolutional Networks,” in *CVPR*, 2017.
- [32] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [33] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [34] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [35] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [36] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [37] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.
- [38] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [39] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [40] J. Su, M. Ahmed, Y. Lu, et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, vol. 568, 2024.
- [41] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” *arXiv:2312.00752*, 2023.
- [42] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [43] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.
- [44] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [45] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [46] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [47] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [48] R. Girshick, J. Donahue, T. Darrell, et al., “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation,” in *CVPR*, 2014.

- [49] R. Girshick, "Fast R-CNN," in *ICCV*, 2015.
- [50] S. Ren, K. He, R. Girshick, et al., "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in *NeurIPS*, 2015.
- [51] T.-Y. Lin, P. Dollár, R. Girshick, et al., "Feature Pyramid Networks for Object Detection," in *CVPR*, 2017.
- [52] K. He, G. Gkioxari, P. Dollár, et al., "Mask R-CNN," in *ICCV*, 2017.
- [53] D. Bolya, S. Foley, J. Hays, et al., "TIDE: A General Toolbox for Identifying Object Detection Errors," in *ECCV*, 2020.
- [54] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [55] J. Redmon, S. Divvala, R. Girshick, et al., "You Only Look Once: Unified, Real-Time Object Detection," in *CVPR*, 2016.
- [56] W. Liu, D. Anguelov, D. Erhan, et al., "SSD: Single Shot MultiBox Detector," in *ECCV*, 2016.
- [57] T.-Y. Lin, P. Goyal, R. Girshick, et al., "Focal Loss for Dense Object Detection," in *ICCV*, 2017.
- [58] Z. Tian, C. Shen, H. Chen, et al., "FCOS: Fully Convolutional One-Stage Object Detection," in *ICCV*, 2019.
- [59] X. Zhou, D. Wang, and P. Krähenbühl, "Objects as Points," *arXiv:1904.07850*, 2019.
- [60] S. Zhang, C. Chi, Y. Yao, et al., "Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection," in *CVPR*, 2020.
- [61] Z. Ge, S. Liu, Z. Li, et al., "OTA: Optimal Transport Assignment for Object Detection," in *CVPR*, 2021.
- [62] N. Bodla, B. Singh, R. Chellappa, et al., "Soft-NMS: Improving Object Detection with One Line of Code," in *ICCV*, 2017.
- [63] X. Zhu, W. Su, L. Lu, et al., "Deformable DETR: Deformable Transformers for End-to-End Object Detection," in *ICLR*, 2021.
- [64] H. Zhang, F. Li, S. Liu, et al., "DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection," in *ICLR*, 2023.
- [65] J. Long, E. Shelhamer, and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," in *CVPR*, 2015.
- [66] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *MICCAI*, 2015.
- [67] L.-C. Chen, G. Papandreou, I. Kokkinos, et al., "DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs," *IEEE TPAMI*, vol. 40, no. 4, pp. 834–848, 2018.
- [68] E. Xie, W. Wang, Z. Yu, et al., "SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers," in *NeurIPS*, 2021.
- [69] B. Cheng, I. Misra, A. G. Schwing, et al., "Masked-Attention Mask Transformer for Universal Image Segmentation," in *CVPR*, 2022.
- [70] A. Kirillov, K. He, R. Girshick, et al., "Panoptic Segmentation," in *CVPR*, 2019.
- [71] A. Kirillov, E. Mintun, N. Ravi, et al., "Segment Anything," in *ICCV*, 2023.
- [72] K. Sun, B. Xiao, D. Liu, et al., "Deep High-Resolution Representation Learning for Human Pose Estimation," in *CVPR*, 2019.



- [73] Z. Cao, T. Simon, S.-E. Wei, et al., “Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields,” in *CVPR*, 2017.
- [74] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [75] J. Deng, J. Guo, N. Xue, et al., “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *CVPR*, 2019.
- [76] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [77] K. Xu, J. Ba, R. Kiros, et al., “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention,” in *ICML*, 2015.
- [78] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *ICLR*, 2015.
- [79] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [80] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [81] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [82] K. Simonyan and A. Zisserman, “Two-Stream Convolutional Networks for Action Recognition in Videos,” in *NeurIPS*, 2014.
- [83] J. Carreira and A. Zisserman, “Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset,” in *CVPR*, 2017.
- [84] D. Tran, H. Wang, L. Torresani, et al., “A Closer Look at Spatiotemporal Convolutions for Action Recognition,” in *CVPR*, 2018.
- [85] C. Feichtenhofer, H. Fan, J. Malik, et al., “SlowFast Networks for Video Recognition,” in *ICCV*, 2019.
- [86] A. Dosovitskiy, P. Fischer, E. Ilg, et al., “FlowNet: Learning Optical Flow with Convolutional Networks,” in *ICCV*, 2015.
- [87] D. Sun, X. Yang, M.-Y. Liu, et al., “PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume,” in *CVPR*, 2018.
- [88] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.
- [89] A. Bewley, Z. Ge, L. Ott, et al., “Simple Online and Realtime Tracking,” in *ICIP*, 2016.
- [90] Y. Zhang, P. Sun, Y. Jiang, et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *ECCV*, 2022.
- [91] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [92] K. Bernardin and R. Stiefelwagen, “Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics,” *EURASIP J. Image and Video Processing*, 2008.
- [93] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [94] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [95] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.



- [96] S. Wang, V. Leroy, Y. Cabon, et al., "DUST3R: Geometric 3D Vision Made Easy," in *CVPR*, 2024.
- [97] D. Eigen, C. Puhrsch, and R. Fergus, "Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network," in *NeurIPS*, 2014.
- [98] R. Ranftl, K. Lasinger, D. Hafner, et al., "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer," *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [99] J. Sturm, N. Engelhard, F. Endres, et al., "A Benchmark for the Evaluation of RGB-D SLAM Systems," in *IROS*, 2012.
- [100] C. R. Qi, H. Su, K. Mo, et al., "PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation," in *CVPR*, 2017.
- [101] C. R. Qi, L. Yi, H. Su, et al., "PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space," in *NeurIPS*, 2017.
- [102] A. H. Lang, S. Vora, H. Caesar, et al., "PointPillars: Fast Encoders for Object Detection from Point Clouds," in *CVPR*, 2019.
- [103] T. Müller, A. Evans, C. Schied, et al., "Instant Neural Graphics Primitives with a Multiresolution Hash Encoding," *ACM TOG*, vol. 41, no. 4, 2022.
- [104] B. Kerbl, G. Kopanas, T. Leimkühler, et al., "3D Gaussian Splatting for Real-Time Radiance Field Rendering," *ACM TOG*, vol. 42, no. 4, 2023.
- [105] R. Zhang, P. Isola, A. A. Efros, et al., "The Unreasonable Effectiveness of Deep Features as a Perceptual Metric," in *CVPR*, 2018.
- [106] D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," in *ICLR*, 2014.
- [107] I. Goodfellow, J. Pouget-Abadie, M. Mirza, et al., "Generative Adversarial Nets," in *NeurIPS*, 2014.
- [108] T. Karras, S. Laine, and T. Aila, "A Style-Based Generator Architecture for Generative Adversarial Networks," in *CVPR*, 2019.
- [109] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," in *NeurIPS*, 2020.
- [110] Y. Song, J. Sohl-Dickstein, D. P. Kingma, et al., "Score-Based Generative Modeling Through Stochastic Differential Equations," in *ICLR*, 2021.
- [111] J. Johnson, A. Alahi, and L. Fei-Fei, "Perceptual Losses for Real-Time Style Transfer and Super-Resolution," in *ECCV*, 2016.
- [112] P. Isola, J.-Y. Zhu, T. Zhou, et al., "Image-to-Image Translation with Conditional Adversarial Networks," in *CVPR*, 2017.
- [113] L. A. Gatys, A. S. Ecker, and M. Bethge, "Image Style Transfer Using Convolutional Neural Networks," in *CVPR*, 2016.
- [114] X. Huang and S. Belongie, "Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization," in *ICCV*, 2017.
- [115] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," in *NeurIPS Deep Learning Workshop*, 2015.
- [116] J. Song, C. Meng, and S. Ermon, "Denoising Diffusion Implicit Models," in *ICLR*, 2021.
- [117] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, et al., "Flow Matching for Generative Modeling," in *ICLR*, 2023.
- [118] J. Ho and T. Salimans, "Classifier-Free Diffusion Guidance," *arXiv:2207.12598*, 2022.
- [119] R. Rombach, A. Blattmann, D. Lorenz, et al., "High-Resolution Image Synthesis with Latent Diffusion Models," in *CVPR*, 2022.
- [120] W. Peebles and S. Xie, "Scalable Diffusion Models with Transformers," in *ICCV*, 2023.

- [121] L. Zhang, A. Rao, and M. Agrawala, "Adding Conditional Control to Text-to-Image Diffusion Models," in *ICCV*, 2023.
- [122] Y. Blau and T. Michaeli, "The Perception-Distortion Tradeoff," in *CVPR*, 2018.
- [123] C. Ledig, L. Theis, F. Huszár, et al., "Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network," in *CVPR*, 2017.
- [124] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., "Image Quality Assessment: From Error Visibility to Structural Similarity," *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [125] M. Heusel, H. Ramsauer, T. Unterthiner, et al., "GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium," in *NeurIPS*, 2017.
- [126] T. Brooks, B. Mildenhall, T. Xue, et al., "Unprocessing Images for Learned Raw Denoising," in *CVPR*, 2019.
- [127] A. Abdelhamed, S. Lin, and M. S. Brown, "A High-Quality Denoising Dataset for Smartphone Cameras," in *CVPR*, 2018.
- [128] J. Deng, W. Dong, R. Socher, et al., "ImageNet: A Large-Scale Hierarchical Image Database," in *CVPR*, 2009.
- [129] A. Radford, J. W. Kim, C. Hallacy, et al., "Learning Transferable Visual Models from Natural Language Supervision," in *ICML*, 2021.
- [130] J. Li, D. Li, S. Savarese, et al., "BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models," in *ICML*, 2023.
- [131] H. Liu, C. Li, Q. Wu, et al., "Visual Instruction Tuning," in *NeurIPS*, 2023.
- [132] S. Kornblith, J. Shlens, and Q. V. Le, "Do Better ImageNet Models Transfer Better?" In *CVPR*, 2019.
- [133] T. Chen, S. Kornblith, M. Norouzi, et al., "A Simple Framework for Contrastive Learning of Visual Representations," in *ICML*, 2020.
- [134] K. He, H. Fan, Y. Wu, et al., "Momentum Contrast for Unsupervised Visual Representation Learning," in *CVPR*, 2020.
- [135] J.-B. Grill, F. Strub, F. Altché, et al., "Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning," in *NeurIPS*, 2020.
- [136] M. Caron, H. Touvron, I. Misra, et al., "Emerging Properties in Self-Supervised Vision Transformers," in *ICCV*, 2021.
- [137] K. He, X. Chen, S. Xie, et al., "Masked Autoencoders Are Scalable Vision Learners," in *CVPR*, 2022.
- [138] A. van den Oord, Y. Li, and O. Vinyals, "Representation Learning with Contrastive Predictive Coding," in *arXiv:1807.03748*, 2018.
- [139] J. Kaplan, S. McCandlish, T. Henighan, et al., "Scaling Laws for Neural Language Models," *arXiv:2001.08361*, 2020.
- [140] X. Zhai, A. Kolesnikov, N. Houlsby, et al., "Scaling Vision Transformers," in *CVPR*, 2022.
- [141] J. Hoffmann, S. Borgeaud, A. Mensch, et al., "Training Compute-Optimal Large Language Models," in *NeurIPS*, 2022.
- [142] C. G. Northcutt, A. Athalye, and J. Mueller, "Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks," in *NeurIPS Datasets and Benchmarks*, 2021.
- [143] B. Recht, R. Roelofs, L. Schmidt, et al., "Do ImageNet Classifiers Generalize to ImageNet?" In *ICML*, 2019.
- [144] N. Houlsby, A. Giurgiu, S. Jastrzebski, et al., "Parameter-Efficient Transfer Learning for NLP," in *ICML*, 2019.

- [145] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [146] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *RecSys*, 2019.

### 3.4 Các bài toán lân cận, mỗi cái một đánh đổi riêng (Neighbouring Tasks)

**PHỤ THUỘC** C/14-bai-toan-cv-loi/01-detection-hai-giai-doan và 03-segmentation (RoIAlign, heatmap, độ phân giải); B/07-chia-du-lieu-va-danh-gia (slice, metric theo nhóm); B/08-dich-chuyen-phan-phoi (long-tail).

#### TÓM TẮT

Mục này gom bốn họ bài toán mà kỹ sư gặp hằng ngày nhưng không nằm gọn trong trục khung/pixel: ước lượng tư thế, OCR, học biểu diễn cho truy hồi, và hai chế độ phân bố dữ liệu (fine-grained, long-tail). Điểm chung của cả bốn là quyết định quan trọng nhất không phải chọn kiến trúc mà là *chọn dạng đầu ra*: điểm, heatmap, chuỗi, hay embedding. Dạng đầu ra kéo theo luôn cả loss, metric và chế độ hỏng. Đọc xong bạn phát biểu được một bài toán thì giác lạ dưới dạng đúng ngay từ đầu, thay vì phát hiện mình chọn sai sau khi đã huấn luyện xong. Nếu chỉ nhớ một điều: chọn sai dạng đầu ra thì không kiến trúc nào cứu được, và triệu chứng của nó luôn xuất hiện ở chỗ mô hình phải biểu diễn sự *mơ hồ* — khớp bị che, ký tự nhoè, hai người rất giống nhau.

#### KIẾN THỨC CẦN CÓ TRƯỚC

- **dạng đầu ra** — kiểu dữ liệu mà mô hình phải sinh ra — một nhãn, một khung, một mask, một điểm, một chuỗi ký tự, hay một vector. Nó quyết định luôn hàm mất mát và thước đo, nên đây là quyết định thiết kế đắt giá nhất.
- **hồi quy (regression) và phân loại (classification)** — hồi quy là dự đoán một số thực (tọa độ  $x$ ); phân loại là chọn một trong các lựa chọn rời rạc (ô lưới nào chứa khớp). Hai cách này hỏng theo kiểu rất khác nhau khi câu trả lời mơ hồ.
- **keypoint và pose** — keypoint là một điểm có ý nghĩa trên vật (khủy tay, góc biến số); pose là tập hợp các keypoint của một đối tượng cùng quan hệ giữa chúng.
- **heatmap** — một bản đồ xác suất cùng cỡ với feature map, giá trị cao ở nơi mô hình tin rằng có keypoint. Khác với một cặp tọa độ, heatmap biểu diễn được “có hai khả năng”.
- **embedding** — một vector số chiều cố định biểu diễn một ảnh, học sao cho hai ảnh cùng danh tính thì gần nhau. Nhận dạng trở thành đo khoảng cách, nên thêm một người mới không cần huấn luyện lại.
- **tập lớp mở (open-set)** — tình huống mà các lớp lúc chạy thật không có trong tập huấn luyện — lý do phân loại softmax không dùng được và metric learning ra đời.
- **long-tail** — phân bố dữ liệu mà vài lớp chiếm gần hết số mẫu còn hàng trăm lớp chỉ có vài chục mẫu; accuracy tổng bị các lớp đông thống trị nên che kín phần đuôi.
- **lát (slice)** — một tập con dữ liệu chia theo một thuộc tính có ý nghĩa — theo lớp, theo kích thước vật, theo điều kiện chụp. Báo cáo metric theo lát là cách duy nhất thấy được các chế độ khó.

#### Đặt vấn đề

Ba mục trước đi theo hai trục quen thuộc: nhãn ở mức khung và nhãn ở mức pixel. Nhưng phần lớn hệ thống thị giác thật lại đứng ở chỗ khác. Cánh tay robot cần *tọa độ khớp* chứ không cần khung; hệ đọc biển số cần một *chuỗi ký tự* đi qua ba khối nối tiếp; hệ nhận dạng khuôn mặt cần trả lời “có phải cùng một người” cho những người chưa từng có trong tập huấn luyện, tức đầu ra không thể là một nhãn trong tập cố định. Mục này gom bốn họ bài toán như vậy.

↪ **tiền đề** B/05 — chọn dạng đầu ra chính là bước thứ hai trong bộ ba đầu vào – đầu ra – tín hiệu giám sát; sai ở bước này thì mọi lựa chọn sau đó đều sai theo.

Chúng không nổi thành một mạch lịch sử duy nhất, nhưng chúng chia sẻ một điểm chung đáng học: ở mỗi họ, quyết định quan trọng nhất là *chọn dạng đầu ra*, và chọn sai dạng đầu ra thì không kiến trúc nào cứu được.

**Học xong mục này bạn làm được gì mà trước đó không làm được?** Nhận một yêu cầu mơ hồ kiểu “đọc số công-tơ từ camera” hay “nhận ra người quen ở cửa”, bạn viết ngay được bộ ba đầu vào – đầu ra – tín hiệu giám sát cùng metric đúng. Kèm theo là danh sách chỗ hệ thống sẽ hỏng.

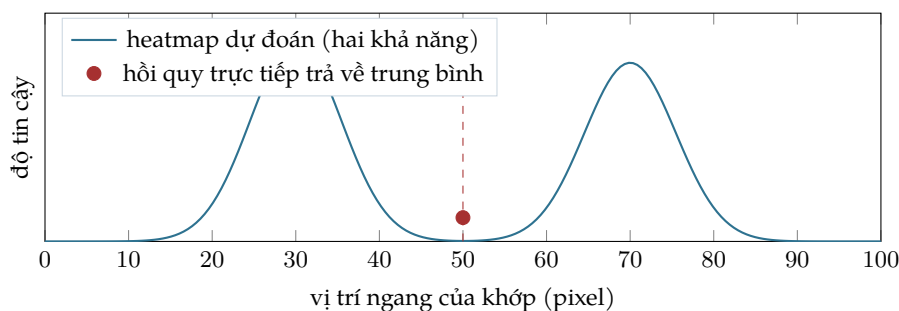
#### TRỰC GIÁC

Phân loại xây một cánh cửa cho mỗi lớp: muốn thêm lớp thì phải xây thêm cửa, tức huấn luyện lại. Metric learning không xây cửa mà vẽ *bản đồ*: mọi ảnh được đặt vào một không gian sao cho khoảng cách phản ánh “cùng danh tính hay không”. Thêm một người mới chỉ là ghim thêm một điểm móc lên bản đồ đã có. Sức mạnh và điểm yếu đều nằm ở đó: bạn không phải xây lại gì cả, nhưng nếu tấm bản đồ vẽ sai thì mọi điểm móc ghim lên nó đều sai theo, và không có cách nào sửa cục bộ.

#### A. Keypoint và pose: hai lần chọn dạng (Keypoint and Pose Estimation)

Ước lượng tư thế phải chọn hai lần. Lần thứ nhất là *dạng đầu ra của một khớp*, và chỉ có hai phương án.

- **Hồi quy trực tiếp hai số**  $(x, y)$  — hiển nhiên và rẻ nhất. Nhưng mạng phải nén một bản đồ tin cậy hai chiều thành một điểm, qua các tầng fully connected. Cách này không biểu diễn được sự mơ hồ: khi một khớp bị che, câu trả lời đúng là “có hai khả năng”, còn hồi quy trả về trung bình của hai khả năng — một điểm mà cả hai giả thuyết đều bác bỏ.
- **Dự đoán một bản đồ nhiệt (heatmap)** — một bản đồ xác suất cho mỗi khớp, lấy argmax làm vị trí [72]. Heatmap giữ được sự mơ hồ. Huấn luyện cũng ổn định hơn nhiều vì bài toán trở thành phân loại theo pixel. Cái giá là bộ nhớ, và một lỗi lượng tử hoá đi kèm.



**Hình 3.14.** Vì sao dạng đầu ra quyết định chế độ hỏng. Khi một khớp bị che, câu trả lời đúng là “một trong hai chỗ này”. Heatmap biểu diễn được điều đó bằng hai đỉnh, và tầng sau có thể chọn hoặc hoãn quyết định. Hồi quy trực tiếp bị hàm mất mát bình phương kéo về trung bình của hai khả năng — một điểm mà cả hai giả thuyết đều bác bỏ. Đây đúng là lý do một bộ ước lượng trạng thái không bao giờ thay một phân bố đa đỉnh bằng kỳ vọng của nó.

Hình 3.14 là ví dụ nhỏ nhất cho luận điểm trung tâm của mục này.

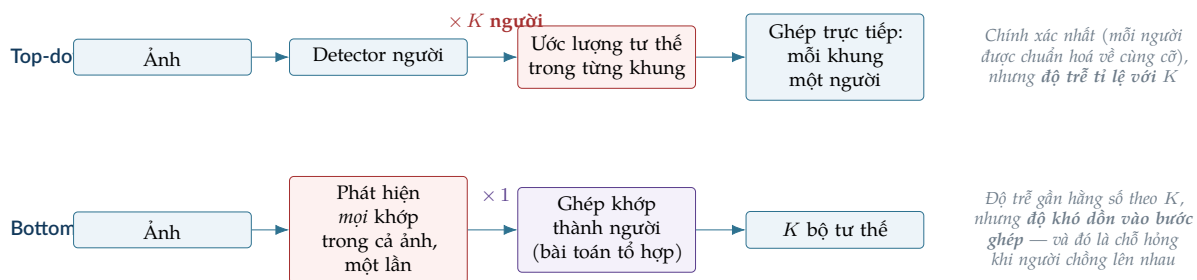
Lỗi lượng tử hoá đó tính được bằng tay. Với heatmap ở stride 4 trên ảnh  $256 \times 192$ , lưới đầu ra là  $64 \times 48$ ; argmax chỉ định vị được tới bội số của 4 pixel, nên sai số hệ thống là  $\pm 2$  pixel ngay cả khi mô hình hoàn hảo. Với người cao 200 pixel trong ảnh, 2 pixel là 1% — chấp nhận được; nhưng

với bàn tay 40 pixel thì đó là 5%, và mọi bản triển khai nghiêm túc đều phải thêm một nhánh hồi quy độ lệch dưới pixel. Về bộ nhớ:  $17 \text{ khớp} \times 64 \times 48 \times 4 \text{ byte} \approx 209 \text{ KB}$  cho một người — không đáng kể; nhưng ở độ phân giải  $384 \times 288$  và 133 điểm (cả bàn tay và mặt) thì con số nhân lên hơn hai chục lần, và đó là lúc lựa chọn bắt đầu đau.

Lần chọn thứ hai là *cách xử lý nhiều người*. **top-down** chạy detector trước rồi ước lượng tư thế trong từng khung. Cách này chính xác nhất, vì mỗi người được chuẩn hoá về cùng kích thước. Nhưng chi phí tỉ lệ tuyến tính với số người, nên độ trễ phụ thuộc nội dung cảnh — một tính chất tệ hại cho hệ thời gian thực. **bottom-up** phát hiện mọi khớp trong cả ảnh một lần rồi ghép chúng thành người, chẳng hạn bằng trường liên kết từng phần trong OpenPose [73]: thời gian gần như hằng số theo số người, nhưng bước ghép là một bài toán tổ hợp và nó chính là chỗ hỏng khi người chồng lên nhau.

↔ cùng nguyên lý với C/15/03 — top-down và bottom-up lặp lại đúng dạng của tracking-by-detection và joint tracking: tách rời thì dễ gỡ lỗi nhưng chi phí tỉ lệ số đối tượng, gộp chung thì chi phí ổn định nhưng độ khó dồn vào một bước liên kết mờ ám.

Đây là cùng một cấu trúc đánh đổi mà bạn sẽ gặp lại ở tracking: tách rời thì dễ gỡ lỗi nhưng chi phí tỉ lệ với số đối tượng; gộp chung thì ổn định về chi phí nhưng đẩy độ khó vào một bước liên kết mờ ám.



**Hình 3.15.** Cùng một bài toán, hai chỗ đặt độ khó. Top-down mua độ chính xác bằng cách chạy lại khối nặng cho từng người, nên chi phí — và quan trọng hơn, *phương sai* của chi phí — phụ thuộc nội dung cảnh. Bottom-up cố định chi phí phát hiện nhưng phải giải một bài toán ghép tổ hợp mà không có lời giải chắc chắn. Đây đúng là cấu trúc đánh đổi sẽ trở lại ở tracking dưới tên tracking-by-detection và joint tracking.

Hình 3.15 cho thấy vì sao lựa chọn này là lựa chọn hệ thống chứ không phải lựa chọn độ chính xác: với ngân sách mili-giây cố định, cột phải mới là cột quyết định.

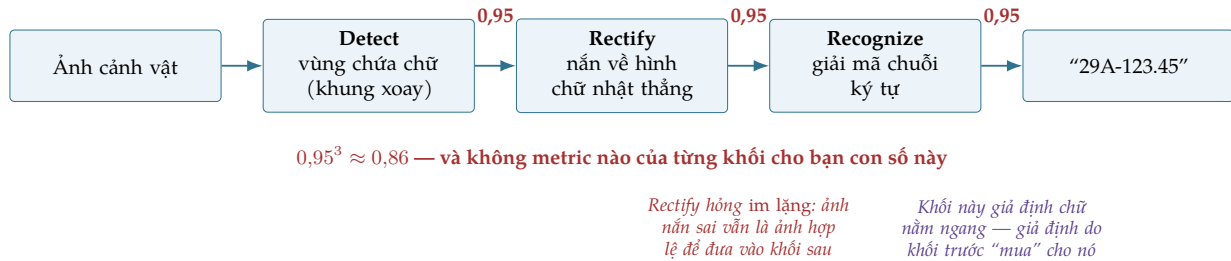
## B. OCR: một pipeline hoàn chỉnh trong một hệ thống nhỏ (Optical Character Recognition)

OCR đáng được mổ xẻ không phải vì nó khó nhất mà vì nó là hệ thống nhỏ nhất chứa đủ mọi thứ: phát hiện, biến đổi hình học, mô hình chuỗi, và một thước đo cấp hệ thống. Pipeline điển hình gồm ba khối nối tiếp:

1. **Detect** vùng chứa chữ. Thường là segmentation hoặc detection với khung xoay, vì chữ hiếm khi nằm ngang.
2. **Rectify** nắn vùng đó về một hình chữ nhật thẳng, bằng phép biến đổi phối cảnh hoặc một lưới biến dạng học được.
3. **Recognize** giải mã ảnh đã nắn thành chuỗi ký tự.

Điều đáng học nằm ở chỗ nối. Thứ nhất, khối rectify tồn tại vì khối recognize giả định chữ nằm ngang và đọc từ trái sang phải; đó là một giả định được đưa vào để làm khối cuối rẻ đi, đúng theo nguyên lý “độ khó không biến mất, chỉ bị đẩy đi chỗ khác”. Thứ hai, lỗi của ba khối nhân lên chứ không cộng lại: nếu mỗi khối đúng 95% thì hệ thống đúng  $0,95^3 \approx 86\%$ , và bạn không bao giờ đọc được con số đó từ metric của từng khối. Thứ ba, thước đo đúng của OCR là tỉ lệ khớp

*cả chuỗi* hoặc khoảng cách chỉnh sửa, chứ không phải độ chính xác từng ký tự — một biến số sai một ký tự là một biến số sai. Bất kỳ ai từng dựng một pipeline thì giác nhiều tầng đều nhận ra hai bài học này, và OCR là nơi rẻ nhất để học chúng.

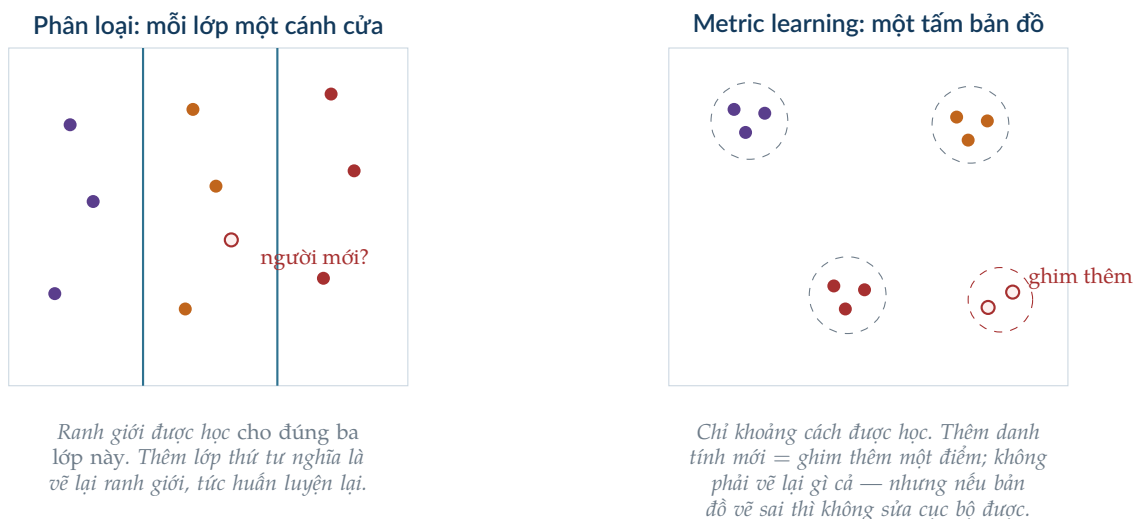


**Hình 3.16.** OCR là hệ thống nhỏ nhất chứa đủ mọi bệnh của một pipeline nhiều tầng. Độ chính xác của các khối *nhân* với nhau chứ không cộng, nên ba khối rất tốt vẫn cho một hệ thống tầm thường; khối giữa tồn tại chỉ để mua một giả định cho khối cuối; và thước đo đúng là khớp cả chuỗi chứ không phải độ chính xác từng ký tự — một biến số sai một ký tự là một biến số sai.

Hình 3.16 đáng nhớ vì nó áp dụng cho mọi pipeline nhiều tầng, kể cả pipeline SLAM: nếu bạn chỉ theo dõi metric của từng khối, bạn không bao giờ nhìn thấy con số mà người dùng thật sự trải nghiệm.

### C. Học biểu diễn cho truy hồi (Metric Learning and Retrieval)

Nhận dạng khuôn mặt, re-identification và tìm kiếm ảnh có chung một ràng buộc mà phân loại không đáp ứng được: tập lớp *mở*. Không thể huấn luyện một softmax trên tập nhân viên của công ty rồi mỗi lần có người mới lại huấn luyện lại. Giải pháp là học một hàm *embedding*  $f$  sao cho khoảng cách trong không gian đích phản ánh quan hệ đồng nhất, rồi việc nhận dạng trở thành tìm láng giềng gần nhất. FaceNet huấn luyện bằng *triplet loss* — kéo cặp cùng danh tính lại gần hơn cặp khác danh tính ít nhất một biên  $\alpha$  [74]; ArcFace thay bằng một biên góc cộng vào logit của softmax, vừa dễ huấn luyện hơn vừa cho ranh giới hình học rõ ràng trên mặt cầu đơn vị [75].



**Hình 3.17.** Vì sao tập lớp mở buộc phải đổi dạng đầu ra. Bên trái, mô hình học *ranh giới*, nên số lớp là một phần của kiến trúc và mọi lớp mới đòi một lần huấn luyện lại. Bên phải, mô hình chỉ học một phép đo khoảng cách, nên danh tính trở thành dữ liệu chứ không còn là tham số. Cái giá là chất lượng phụ thuộc hoàn toàn vào hình dạng của không gian nhúng — thứ không sửa được cho riêng một truy vấn.



Hình 3.17 giải thích luôn vì sao tầng tìm láng giềng xấp xỉ lại quan trọng đến thế: khi danh tính là dữ liệu, việc tra cứu trở thành một phần của hệ thống chứ không còn là một phép nhân ma trận.

Cái giá của cách tiếp cận này gồm ba phần. Thứ nhất, chất lượng phụ thuộc *hoàn toàn* vào không gian nhúng; không có cơ chế nào sửa được một truy vấn cụ thể mà không động vào toàn cục. Thứ hai, việc lấy mẫu quan trọng ngang hàm loss: phần lớn cặp ngẫu nhiên đã thoả biên nên không sinh gradient, và mọi phương pháp thực dụng đều phải khai thác mẫu khó — lại là một chi tiết nằm trong code. Thứ ba, ở quy mô lớn phải thêm một tầng *approximate nearest neighbour* (ANN), và tầng đó có sai số riêng: recall của ANN đặt trần lên recall của cả hệ thống, độc lập với chất lượng embedding.

#### CẠM BẦY

Một khảo sát có kiểm soát đã chỉ ra rằng phần lớn “tiền bộ” công bố trong metric learning biến mất khi so sánh công bằng [76]. Công bằng ở đây nghĩa là cùng backbone, cùng cách lấy mẫu, cùng quy trình chọn siêu tham số. Bài học không chỉ dành cho metric learning: khi một lĩnh vực có nhiều bậc tự do nằm ngoài phần được viết trong bài (cách lấy mẫu, augmentation, lịch huấn luyện), bảng kết quả đo lường công sức tinh chỉnh nhiều hơn đo lường ý tưởng.

#### D. Fine-grained và long-tail: hai chế độ mà accuracy tổng che giấu

Hai chế độ khó cuối cùng không phải là bài toán mới mà là hai kiểu phân bố dữ liệu khiến mọi con số tổng trở nên vô nghĩa. **phân loại chi tiết** (*fine-grained classification*) là khi khác biệt giữa các lớp nhỏ hơn khác biệt bên trong một lớp: hai loài chim khác nhau ở màu một mảng lông, trong khi cùng một loài trông rất khác giữa tư thế bay và tư thế đậu. Hệ quả kỹ thuật là mô hình cần độ phân giải cao ở đúng vùng phân biệt và cần bất biến mạnh với tư thế — hai yêu cầu kéo ngược nhau. **phân bố đuôi dài** (*long-tail distribution*) là khi vài lớp chiếm phần lớn dữ liệu còn hàng trăm lớp chỉ có vài chục mẫu; accuracy tổng bị các lớp đầu thống trị nên có thể rất cao trong khi mô hình gần như không bao giờ dự đoán các lớp đuôi.

→ cùng nguyên lý với B/08 — long-tail ở đây và dịch chuyển phân phối ở chương 8 có chung một cơ chế: phân bố lúc đánh giá không giống phân bố mà hệ thống thật sự gặp.

Điểm chung của hai chế độ là chúng *vô hình* với một con số tổng. Cách phát hiện duy nhất là chia dữ liệu theo lát và báo cáo metric trên từng lát — theo lớp, theo kích thước vật, theo điều kiện chụp. Đây chính là kỹ thuật ở B/07-chia-du-lieu-va-danh-gia, và lý do nó được nhắc lại ở đây là vì detection và segmentation có thêm một chiều lát riêng: kích thước vật. Bộ công cụ như [dbolya/tide](#) tách mAP thành lỗi phân loại, lỗi định vị, lỗi trùng lặp và lỗi nền [53], [54]; đọc bảng đó thường có ích hơn đọc thêm mười paper kiến trúc.

**Bảng 3.3.** Bốn họ bài toán lân cận, đọc theo trục “dạng đầu ra quyết định mọi thứ còn lại”.

| Bài toán                 | Dạng đầu ra          | Metric đúng                          | Hỏng khi nào                                                            |
|--------------------------|----------------------|--------------------------------------|-------------------------------------------------------------------------|
| Pose                     | Heatmap hoặc toạ độ  | PCK, OKS                             | Khớp bị che: hồi quy trả về trung bình của hai khả năng                 |
| OCR                      | Chuỗi ký tự          | Khớp cả chuỗi, khoảng cách chỉnh sửa | Lỗi nhân qua ba khối; rectify hỏng im lặng                              |
| Retrieval                | Embedding            | Recall@ $k$ , mAP truy hồi           | Không gian nhúng sai thì không sửa cục bộ được; recall của ANN đặt trần |
| Fine-grained / long-tail | Nhân (như phân loại) | Accuracy theo lớp và theo lát        | Con số tổng bị các lớp đầu thống trị, che kín các lớp đuôi              |

Bảng 3.3 nên đọc từ cột thứ hai sang: chọn cột hai thì cột ba và cột bốn gần như bị quyết định theo, và đó chính là lý do bước chọn dạng đầu ra đáng dành thời gian hơn bước chọn backbone.

### E. Giới hạn và trường hợp hỏng

Bốn họ trên chia nhau bốn chế độ hỏng, và điểm chung là cả bốn đều *im lặng*:

- **Pose — khớp bị che.** Heatmap trả một đỉnh yếu ở vị trí sai, và hệ thống hạ nguồn không phân biệt được “dự đoán yếu” với “không có”. *Cách chữa duy nhất*: truyền độ tin cậy đi cùng toạ độ, để tăng sau quyết định. Đây đúng là nguyên tắc mà bộ lọc trạng thái trong SLAM đã dùng từ lâu — không ai truyền một phép đo mà không truyền hiệp phương sai của nó.
- **OCR — nhân lỗi và thất bại im lặng.** Ba khối nhân xác suất đúng với nhau, và khối rectify hỏng vẫn sinh ra một ảnh hợp lệ để đưa vào khối nhận dạng, nên không có ngoại lệ nào được ném ra.
- **Retrieval — gallery trôi theo thời gian.** Chất lượng giảm dần mà không có tín hiệu nào lúc chạy, vì hệ thống không bao giờ biết câu trả lời đúng. *Cách phát hiện*: giữ một tập truy vấn cố định và đo lại định kỳ.
- **Fine-grained và long-tail — chính bạn.** Đo bằng một con số tổng rồi tin rằng hệ thống hoạt động. *Cách chữa*: báo cáo theo lát, không bao giờ chỉ báo cáo một số.

#### CẦN NHỚ

Ở mỗi bài toán lân cận, câu hỏi đầu tiên không phải “dùng kiến trúc nào” mà là “đầu ra nên có dạng gì” — điểm, heatmap, chuỗi, hay embedding. Dạng đầu ra quyết định loss, metric, chế độ hỏng và cả cách hệ thống hạ nguồn phải đối xử với sự mơ hồ. [T3]

### F. Kết nối

B/05-thiet-ke-bai-toan là tiền đề khái niệm: chọn dạng đầu ra chính là bước hai trong bộ ba đầu vào – đầu ra – tín hiệu giám sát. B/07-chia-du-lieu-va-danh-gia và B/08-dich-chuyen-phan-phoi là chỗ xử lý nghiêm túc fine-grained và long-tail. C/15-thi-giac-khong-thoi-gian/03-flow-tracking-va-rang-buoc nối trực tiếp với phần top-down/bottom-up: tracking lặp lại đúng cấu trúc đánh đổi đó ở quy mô thời gian. C/18-pretraining-va-foundation giải thích vì sao không gian nhúng học từ pretraining quy mô lớn thường tốt hơn không gian học riêng cho một bài toán retrieval nhỏ.

**Câu hỏi tự kiểm** (trả lời không nhìn bài)

1. Vì sao hồi quy trực tiếp toạ độ khớp lại xử lý sai trường hợp khớp bị che, trong khi heatmap thì không?
2. Với heatmap stride 4, sai số lượng tử hoá là bao nhiêu pixel, và khi nào con số đó bắt đầu quan trọng?
3. Top-down chính xác hơn nhưng vì sao lại là lựa chọn tệ cho một hệ thống có ngân sách mili-giây cố định?
4. Ba khối OCR mỗi khối đúng 95%. Hệ thống đúng bao nhiêu, và vì sao metric từng khối không cho bạn biết điều đó?
5. Metric learning cho phép thêm lớp mới không cần huấn luyện lại. Cái giá của khả năng đó là gì, và nó lộ ra ở đâu khi hệ thống chạy thật?
6. Vì sao accuracy tổng che giấu cả fine-grained lẫn long-tail, và bạn thay nó bằng cách báo cáo nào?

- [1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [2] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [3] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *AISTATS*, 2010.
- [4] K. He, X. Zhang, S. Ren, et al., “Delving Deep into Rectifiers,” in *ICCV*, 2015.
- [5] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [6] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [7] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [8] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [9] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [10] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [11] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” in *ICLR*, 2017.
- [12] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [13] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [14] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [15] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [16] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [17] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [18] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [19] G. Huang, Y. Sun, Z. Liu, et al., “Deep Networks with Stochastic Depth,” in *ECCV*, 2016.
- [20] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [21] H. Zhang, M. Cisse, Y. N. Dauphin, et al., “mixup: Beyond Empirical Risk Minimization,” in *ICLR*, 2018.
- [22] S. Yun, D. Han, S. J. Oh, et al., “CutMix: Regularization Strategy to Train Strong Classifiers,” in *ICCV*, 2019.
- [23] E. D. Cubuk, B. Zoph, J. Shlens, et al., “RandAugment: Practical Automated Data Augmentation,” in *CVPR Workshops*, 2020.

- [24] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [25] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [26] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *NeurIPS*, 2012.
- [27] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in *ICLR*, 2015.
- [28] C. Szegedy, W. Liu, et al., “Going Deeper with Convolutions,” in *CVPR*, 2015.
- [29] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [30] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [31] G. Huang, Z. Liu, L. van der Maaten, et al., “Densely Connected Convolutional Networks,” in *CVPR*, 2017.
- [32] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [33] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [34] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [35] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [36] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [37] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.
- [38] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [39] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [40] J. Su, M. Ahmed, Y. Lu, et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, vol. 568, 2024.
- [41] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” *arXiv:2312.00752*, 2023.
- [42] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [43] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.
- [44] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [45] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [46] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [47] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [48] R. Girshick, J. Donahue, T. Darrell, et al., “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation,” in *CVPR*, 2014.

- [49] R. Girshick, "Fast R-CNN," in *ICCV*, 2015.
- [50] S. Ren, K. He, R. Girshick, et al., "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in *NeurIPS*, 2015.
- [51] T.-Y. Lin, P. Dollár, R. Girshick, et al., "Feature Pyramid Networks for Object Detection," in *CVPR*, 2017.
- [52] K. He, G. Gkioxari, P. Dollár, et al., "Mask R-CNN," in *ICCV*, 2017.
- [53] D. Bolya, S. Foley, J. Hays, et al., "TIDE: A General Toolbox for Identifying Object Detection Errors," in *ECCV*, 2020.
- [54] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [55] J. Redmon, S. Divvala, R. Girshick, et al., "You Only Look Once: Unified, Real-Time Object Detection," in *CVPR*, 2016.
- [56] W. Liu, D. Anguelov, D. Erhan, et al., "SSD: Single Shot MultiBox Detector," in *ECCV*, 2016.
- [57] T.-Y. Lin, P. Goyal, R. Girshick, et al., "Focal Loss for Dense Object Detection," in *ICCV*, 2017.
- [58] Z. Tian, C. Shen, H. Chen, et al., "FCOS: Fully Convolutional One-Stage Object Detection," in *ICCV*, 2019.
- [59] X. Zhou, D. Wang, and P. Krähenbühl, "Objects as Points," *arXiv:1904.07850*, 2019.
- [60] S. Zhang, C. Chi, Y. Yao, et al., "Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection," in *CVPR*, 2020.
- [61] Z. Ge, S. Liu, Z. Li, et al., "OTA: Optimal Transport Assignment for Object Detection," in *CVPR*, 2021.
- [62] N. Bodla, B. Singh, R. Chellappa, et al., "Soft-NMS: Improving Object Detection with One Line of Code," in *ICCV*, 2017.
- [63] X. Zhu, W. Su, L. Lu, et al., "Deformable DETR: Deformable Transformers for End-to-End Object Detection," in *ICLR*, 2021.
- [64] H. Zhang, F. Li, S. Liu, et al., "DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection," in *ICLR*, 2023.
- [65] J. Long, E. Shelhamer, and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," in *CVPR*, 2015.
- [66] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *MICCAI*, 2015.
- [67] L.-C. Chen, G. Papandreou, I. Kokkinos, et al., "DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs," *IEEE TPAMI*, vol. 40, no. 4, pp. 834–848, 2018.
- [68] E. Xie, W. Wang, Z. Yu, et al., "SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers," in *NeurIPS*, 2021.
- [69] B. Cheng, I. Misra, A. G. Schwing, et al., "Masked-Attention Mask Transformer for Universal Image Segmentation," in *CVPR*, 2022.
- [70] A. Kirillov, K. He, R. Girshick, et al., "Panoptic Segmentation," in *CVPR*, 2019.
- [71] A. Kirillov, E. Mintun, N. Ravi, et al., "Segment Anything," in *ICCV*, 2023.
- [72] K. Sun, B. Xiao, D. Liu, et al., "Deep High-Resolution Representation Learning for Human Pose Estimation," in *CVPR*, 2019.



- [73] Z. Cao, T. Simon, S.-E. Wei, et al., “Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields,” in *CVPR*, 2017.
- [74] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [75] J. Deng, J. Guo, N. Xue, et al., “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *CVPR*, 2019.
- [76] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [77] K. Xu, J. Ba, R. Kiros, et al., “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention,” in *ICML*, 2015.
- [78] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *ICLR*, 2015.
- [79] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [80] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [81] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [82] K. Simonyan and A. Zisserman, “Two-Stream Convolutional Networks for Action Recognition in Videos,” in *NeurIPS*, 2014.
- [83] J. Carreira and A. Zisserman, “Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset,” in *CVPR*, 2017.
- [84] D. Tran, H. Wang, L. Torresani, et al., “A Closer Look at Spatiotemporal Convolutions for Action Recognition,” in *CVPR*, 2018.
- [85] C. Feichtenhofer, H. Fan, J. Malik, et al., “SlowFast Networks for Video Recognition,” in *ICCV*, 2019.
- [86] A. Dosovitskiy, P. Fischer, E. Ilg, et al., “FlowNet: Learning Optical Flow with Convolutional Networks,” in *ICCV*, 2015.
- [87] D. Sun, X. Yang, M.-Y. Liu, et al., “PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume,” in *CVPR*, 2018.
- [88] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.
- [89] A. Bewley, Z. Ge, L. Ott, et al., “Simple Online and Realtime Tracking,” in *ICIP*, 2016.
- [90] Y. Zhang, P. Sun, Y. Jiang, et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *ECCV*, 2022.
- [91] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [92] K. Bernardin and R. Stiefelhagen, “Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics,” *EURASIP J. Image and Video Processing*, 2008.
- [93] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [94] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [95] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.

- [96] S. Wang, V. Leroy, Y. Cabon, et al., "DUST3R: Geometric 3D Vision Made Easy," in *CVPR*, 2024.
- [97] D. Eigen, C. Puhrsch, and R. Fergus, "Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network," in *NeurIPS*, 2014.
- [98] R. Ranftl, K. Lasinger, D. Hafner, et al., "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer," *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [99] J. Sturm, N. Engelhard, F. Endres, et al., "A Benchmark for the Evaluation of RGB-D SLAM Systems," in *IROS*, 2012.
- [100] C. R. Qi, H. Su, K. Mo, et al., "PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation," in *CVPR*, 2017.
- [101] C. R. Qi, L. Yi, H. Su, et al., "PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space," in *NeurIPS*, 2017.
- [102] A. H. Lang, S. Vora, H. Caesar, et al., "PointPillars: Fast Encoders for Object Detection from Point Clouds," in *CVPR*, 2019.
- [103] T. Müller, A. Evans, C. Schied, et al., "Instant Neural Graphics Primitives with a Multiresolution Hash Encoding," *ACM TOG*, vol. 41, no. 4, 2022.
- [104] B. Kerbl, G. Kopanas, T. Leimkühler, et al., "3D Gaussian Splatting for Real-Time Radiance Field Rendering," *ACM TOG*, vol. 42, no. 4, 2023.
- [105] R. Zhang, P. Isola, A. A. Efros, et al., "The Unreasonable Effectiveness of Deep Features as a Perceptual Metric," in *CVPR*, 2018.
- [106] D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," in *ICLR*, 2014.
- [107] I. Goodfellow, J. Pouget-Abadie, M. Mirza, et al., "Generative Adversarial Nets," in *NeurIPS*, 2014.
- [108] T. Karras, S. Laine, and T. Aila, "A Style-Based Generator Architecture for Generative Adversarial Networks," in *CVPR*, 2019.
- [109] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," in *NeurIPS*, 2020.
- [110] Y. Song, J. Sohl-Dickstein, D. P. Kingma, et al., "Score-Based Generative Modeling Through Stochastic Differential Equations," in *ICLR*, 2021.
- [111] J. Johnson, A. Alahi, and L. Fei-Fei, "Perceptual Losses for Real-Time Style Transfer and Super-Resolution," in *ECCV*, 2016.
- [112] P. Isola, J.-Y. Zhu, T. Zhou, et al., "Image-to-Image Translation with Conditional Adversarial Networks," in *CVPR*, 2017.
- [113] L. A. Gatys, A. S. Ecker, and M. Bethge, "Image Style Transfer Using Convolutional Neural Networks," in *CVPR*, 2016.
- [114] X. Huang and S. Belongie, "Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization," in *ICCV*, 2017.
- [115] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," in *NeurIPS Deep Learning Workshop*, 2015.
- [116] J. Song, C. Meng, and S. Ermon, "Denoising Diffusion Implicit Models," in *ICLR*, 2021.
- [117] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, et al., "Flow Matching for Generative Modeling," in *ICLR*, 2023.
- [118] J. Ho and T. Salimans, "Classifier-Free Diffusion Guidance," *arXiv:2207.12598*, 2022.
- [119] R. Rombach, A. Blattmann, D. Lorenz, et al., "High-Resolution Image Synthesis with Latent Diffusion Models," in *CVPR*, 2022.
- [120] W. Peebles and S. Xie, "Scalable Diffusion Models with Transformers," in *ICCV*, 2023.

- [121] L. Zhang, A. Rao, and M. Agrawala, "Adding Conditional Control to Text-to-Image Diffusion Models," in *ICCV*, 2023.
- [122] Y. Blau and T. Michaeli, "The Perception-Distortion Tradeoff," in *CVPR*, 2018.
- [123] C. Ledig, L. Theis, F. Huszár, et al., "Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network," in *CVPR*, 2017.
- [124] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., "Image Quality Assessment: From Error Visibility to Structural Similarity," *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [125] M. Heusel, H. Ramsauer, T. Unterthiner, et al., "GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium," in *NeurIPS*, 2017.
- [126] T. Brooks, B. Mildenhall, T. Xue, et al., "Unprocessing Images for Learned Raw Denoising," in *CVPR*, 2019.
- [127] A. Abdelhamed, S. Lin, and M. S. Brown, "A High-Quality Denoising Dataset for Smartphone Cameras," in *CVPR*, 2018.
- [128] J. Deng, W. Dong, R. Socher, et al., "ImageNet: A Large-Scale Hierarchical Image Database," in *CVPR*, 2009.
- [129] A. Radford, J. W. Kim, C. Hallacy, et al., "Learning Transferable Visual Models from Natural Language Supervision," in *ICML*, 2021.
- [130] J. Li, D. Li, S. Savarese, et al., "BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models," in *ICML*, 2023.
- [131] H. Liu, C. Li, Q. Wu, et al., "Visual Instruction Tuning," in *NeurIPS*, 2023.
- [132] S. Kornblith, J. Shlens, and Q. V. Le, "Do Better ImageNet Models Transfer Better?" In *CVPR*, 2019.
- [133] T. Chen, S. Kornblith, M. Norouzi, et al., "A Simple Framework for Contrastive Learning of Visual Representations," in *ICML*, 2020.
- [134] K. He, H. Fan, Y. Wu, et al., "Momentum Contrast for Unsupervised Visual Representation Learning," in *CVPR*, 2020.
- [135] J.-B. Grill, F. Strub, F. Altché, et al., "Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning," in *NeurIPS*, 2020.
- [136] M. Caron, H. Touvron, I. Misra, et al., "Emerging Properties in Self-Supervised Vision Transformers," in *ICCV*, 2021.
- [137] K. He, X. Chen, S. Xie, et al., "Masked Autoencoders Are Scalable Vision Learners," in *CVPR*, 2022.
- [138] A. van den Oord, Y. Li, and O. Vinyals, "Representation Learning with Contrastive Predictive Coding," in *arXiv:1807.03748*, 2018.
- [139] J. Kaplan, S. McCandlish, T. Henighan, et al., "Scaling Laws for Neural Language Models," *arXiv:2001.08361*, 2020.
- [140] X. Zhai, A. Kolesnikov, N. Houlsby, et al., "Scaling Vision Transformers," in *CVPR*, 2022.
- [141] J. Hoffmann, S. Borgeaud, A. Mensch, et al., "Training Compute-Optimal Large Language Models," in *NeurIPS*, 2022.
- [142] C. G. Northcutt, A. Athalye, and J. Mueller, "Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks," in *NeurIPS Datasets and Benchmarks*, 2021.
- [143] B. Recht, R. Roelofs, L. Schmidt, et al., "Do ImageNet Classifiers Generalize to ImageNet?" In *ICML*, 2019.
- [144] N. Houlsby, A. Giurgiu, S. Jastrzebski, et al., "Parameter-Efficient Transfer Learning for NLP," in *ICML*, 2019.

- [145] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [146] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *RecSys*, 2019.

### 3.5 Thực hành

Chương này là chương dễ thực hành nhất của cả cây, vì mọi khẳng định trong đó đều kiểm chứng được trong vài giờ trên một GPU tầm trung. Hãy tận dụng: đọc mười paper detection không dạy bạn được bằng một lần tự tay đổi cách gán mẫu và nhìn con số nhúc nhích.

**Repo và tài nguyên.** Về detection, [roboflow/rf-detr](#) là dòng DETR thực dụng nhất tại thời điểm viết (giấy phép Apache 2.0, backbone DINOv2 nên khá bền với vật bị che); [ultralytics](#) YOLO11/YOLO26 nhanh và hợp với thiết bị biên nhưng dùng giấy phép **AGPL-3.0**, hãy kiểm tra kỹ trước khi đưa vào sản phẩm đóng; [lyuwenyu/RT-DETR](#) và RTMDet trong [open-mmlab/mmdetection](#) (MIT) là hai lựa chọn trung dung. Về segmentation, SAM 3 cho phân vùng theo prompt trên cả ảnh lẫn video, Mask2Former và SegFormer cho tập nhãn đóng, còn [MIC-DKFZ/nnUNet](#) là baseline y tế rất khó đánh bại. Open-vocabulary: Grounding DINO, Grounded-SAM. Khung làm việc: [facebookresearch/detectron2](#), [open-mmlab/mmdetection](#), [qubvel-org/segmentation\\_models.pytorch](#). Soi lỗi: [voxel51/fiftyone](#) để nhìn tận mắt, [dbolya/tide](#) để tách mAP thành các loại lỗi. Danh sách này chốt tại tháng 9/2026 và tên mô hình dẫn đầu sẽ cũ trong 6–12 tháng, trong khi các repo hạ tầng (detectron2, mmdetection, fiftyone) bền hơn nhiều — hãy đầu tư thời gian theo tỉ lệ đó.

**Câu hỏi kiểm tra hiểu.** Vì sao DETR bản gốc hội tụ chậm hơn Faster R-CNN nhiều lần, và hai bản sửa lớn tấn công vào nguyên nhân nào? Nếu hai detector chênh nhau 2 mAP, bạn làm thế nào để biết chênh lệch đó đến từ kiến trúc hay đến từ cách gán mẫu và công thức huấn luyện? Khi nào một mô hình nền tảng zero-shot vẫn thua một mô hình nhỏ fine-tune, và dấu hiệu nào cho bạn biết điều đó trước khi đo?

**Mini project (vài giờ đến hai ngày).** Huấn luyện RF-DETR và một biến thể YOLO trên cùng một tập dữ liệu tự thu, với cùng ngân sách thời gian chứ không phải cùng số epoch. So sánh mAP, độ trễ trung bình và phương sai độ trễ, dung lượng mô hình, và giấy phép. Kết thúc bằng một khuyến nghị viết cho một tình huống triển khai cụ thể, trong đó nêu rõ ràng buộc nào loại bỏ phương án nào.

#### Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

| Thuật ngữ / câu hỏi | Trả lời                                                                                                                                                                                                                                                                       |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ••• Anchor          | Khung chữ nhật mẫu, kích thước và tỉ số cạnh định sẵn, rải đều trên mọi vị trí của feature map. Mô hình đoán <i>độ lệch</i> so với anchor gần nhất chứ không đoán khung từ số không. Nó mã hoá thống kê hình dạng của tập dữ liệu gốc, nên đối miền ảnh là phải thiết kế lại. |
| ••• RPN             | Mạng con trượt trên chính feature map của backbone, mỗi vị trí trả về độ khách quan và độ lệch khung. Nó thay selective search, và vì dùng chung backbone nên chi phí sinh đề xuất gần như bằng không.                                                                        |

| Thuật ngữ / câu hỏi                                                        | Trả lời (tiếp)                                                                                                                                                                                                                                    |
|----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ●● Vì sao Fast R-CNN vừa nhanh hơn vừa chính xác hơn R-CNN?                | Nhanh hơn vì backbone chạy một lần thay vì hai nghìn lần, đưa chi phí từ $O(N)$ lượt backbone về $O(1)$ . Chính xác hơn vì phân loại và hồi quy khung gộp vào một hàm mất mát huấn luyện một pha, nên mọi khối cùng tối ưu cho mục tiêu cuối.     |
| ●● RoIAlign                                                                | Phép cắt đặc trưng theo vùng, lấy mẫu song tuyến tại tọa độ thực thay vì làm tròn về ô nguyên. Ở stride 16, làm tròn dịch biên tới 3 pixel ảnh, đủ để phá nhánh mask; bỏ nó đi thì nhánh mask mới chạy được.                                      |
| ●● Vì sao vật nhỏ bị thiệt kém trước khi có FPN?                           | Tầng sâu có ngữ nghĩa mạnh nhưng độ phân giải thô, tầng nông thì ngược lại. Vật nhỏ cần cả hai mà không tầng nào cho đủ. FPN gộp chúng bằng nhánh đi xuống cộng kết nối ngang $1 \times 1$ .                                                      |
| ●● Recall đề xuất thấp mà độ chính xác trên đề xuất vẫn cao — nguyên nhân? | Tập anchor lệch miền: nó không phủ dải tỉ lệ hay tỉ số cạnh của dữ liệu mới. Phân biệt với detector kém bằng cách đo riêng recall của giai đoạn một; nếu recall đó thấp thì lỗi nằm ở anchor, không ở bộ phân loại.                               |
| ●●● Triệt phi cực đại (NMS)                                                | Bước hậu xử lý giữ khung điểm cao nhất rồi xoá mọi khung chồng lấn quá ngưỡng IoU. Nó không học được vì nó xét quan hệ giữa các dự đoán. Hồng trong cảnh đông: hai vật cùng lớp chồng nhau thì chắc chắn mất một.                                 |
| ●●● Gán mẫu                                                                | Quy tắc quyết định vị trí dự đoán nào phải học vật nào. Nó nằm trong định nghĩa hàm mất mát chứ không phải trong kiến trúc, nên đổi nó là đổi bài toán tối ưu. Đây là thành phần giải thích phần lớn chênh lệch giữa các detector.                |
| ●●● Focal loss                                                             | Cross-entropy nhân thêm hệ số $(1 - p_t)^\gamma$ để nén đóng góp của mẫu đã đúng. Với $\gamma = 2$ , một anchor nền chắc chắn bị hạ hơn bốn bậc, nên $10^5$ mẫu nền hết chi phối gradient. Hồng trên nhãn nhiễu: mẫu gán sai trông y hệt mẫu khó. |
| ●● Vì sao chỉ detector một giai đoạn cần focal loss?                       | Vì giai đoạn đề xuất đã ngậm lọc hàng chục nghìn vị trí xuống còn vài trăm, tức đã cân bằng lại tập mẫu. Bỏ nó đi thì tỉ lệ nền trên vật quay về cỡ $10^3$ tới $10^4$ , và mẫu dễ nhấn chìm tín hiệu học.                                         |

●●● phải thuộc ●● nên nắm ● biết là có

## Active recall — phần 2 (che cột phải, tự trả lời, rồi mở ra đối chiếu)

| Thuật ngữ / câu hỏi                                               | Trả lời                                                                                                                                                                                                                                                          |
|-------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ●● ATSS                                                           | Cách gán mẫu chọn ngưỡng IoU riêng cho từng vật: lấy $k$ ứng viên gần tâm nhất, tính trung bình và độ lệch chuẩn IoU của chúng, dùng $\mu + \sigma$ làm ngưỡng. Nó cho thấy sau khi thống nhất gán mẫu, khác biệt anchor với anchor-free gần như biến mất.       |
| ●● SimOTA                                                         | Cách gán mẫu phát biểu việc chọn mẫu dương thành bài toán vận tải tối ưu trên toàn ảnh, chi phí là tổ hợp loss phân loại và loss hồi quy. Ưu điểm là phân xử được vùng chồng lấn; nhược điểm là vô nghĩa ở giai đoạn đầu huấn luyện nên phải kèm bước khởi động. |
| ●●● Ghép cặp Hungary                                              | Phép gán một-một giữa $N$ truy vấn và các vật thật, tìm phép ghép có tổng chi phí nhỏ nhất. Vì chỉ một truy vấn được thưởng cho mỗi vật, mô hình tự học không sinh trùng lặp, và NMS hết lý do tồn tại. Giá là hội tụ chậm.                                      |
| ● AP <sub>50</sub> cao nhưng AP <sub>75</sub> thấp — nguyên nhân? | Định vị kém, không phải phân loại kém. Thường vì NMS xếp hạng bằng điểm phân loại, nên khung tự tin về nhãn nhưng lệch vị trí xoá mất khung đúng hơn. Phân biệt bằng TIDE: nếu lỗi định vị chiếm phần lớn thì đúng là nguyên nhân này.                           |



| Thuật ngữ / câu hỏi                                                    | Trả lời (tiếp)                                                                                                                                                                                                                                                     |
|------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| •• Output stride                                                       | Tỉ lệ thu nhỏ giữa ảnh và feature map. Output stride 32 nghĩa là mỗi ô đặc trưng đại diện cho một mảng $32 \times 32$ pixel, nên sai số định vị biên có sà cỡ chục pixel. Hạ nó một nửa thì lượng kích hoạt gấp bốn.                                               |
| •• Vì sao tích chập giãn không làm mất độ phân giải?                   | Vì nó chèn khoảng trống giữa các trọng số thay vì thu nhỏ ảnh. Kernel $3 \times 3$ giãn $r$ có trường tiếp nhận hiệu dụng $3 + 2(r - 1)$ , trong khi số ô đầu ra giữ nguyên. Cái giá là bộ nhớ, vì độ phân giải không giảm.                                        |
| •• Vì sao ba tầng cùng giãn 2 bỏ sót nửa số pixel?                     | Vì mọi vị trí lấy mẫu rơi vào cùng một lớp chẵn lẻ, nên trường tiếp nhận trống rỗng mà thực chất thừa. Đó là hiệu ứng lưới, và cách chữa là chọn dãy tỉ lệ giãn nguyên tố cùng nhau.                                                                               |
| ••• Mask classification                                                | Phát biểu coi đầu ra là một tập truy vấn, mỗi truy vấn trả một mask nhị phân cộng một nhãn lớp, ghép với chân trị bằng ghép cặp Hungary. Nhờ nó semantic, instance và panoptic chỉ còn khác nhau ở cách đọc tập mask, dùng chung một mô hình.                      |
| •• Heatmap                                                             | Bản đồ xác suất cho mỗi keypoint, lấy argmax làm vị trí. Nó biểu diễn được sự mơ hồ khi khớp bị che, điều mà hồi quy tọa độ không làm được. Giá là bộ nhớ, cộng sai số lượng tử hoá $\pm 2$ pixel ở stride 4.                                                      |
| • Số tổng đẹp — mAP cao, mIoU 0,85 — nhưng lớp vật nhỏ gần như bằng 0? | Vì cả hai thước đo đều trung bình theo diện tích bên trong mỗi lớp, nên vật tí hon đóng góp gần như bằng không, và một lớp hiếm bị các lớp lớn che lấp. Phân biệt bằng cách báo cáo theo lát kích thước vật; nếu chỉ lát nhỏ nhất sập thì đúng là nguyên nhân này. |

••• phải thuộc •• nên nắm • biết là có

**I Kiến thức này đang sống ở đâu** (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

| Công trình / hệ thống                              | Dùng ý tưởng nào của chương                                                                                    | Ghi chú                                                                         |
|----------------------------------------------------|----------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| <code>detectron2</code> , <code>mmdetection</code> | Hiện thực trực tiếp mạch hai giai đoạn: RPN, RoIAlign, FPN, và các bộ gán mẫu thay thế được                    | Hạ tầng, bền hơn tên mô hình dẫn đầu rất nhiều                                  |
| Họ RT-DETR / RF-DETR (2024–2026)                   | Dùng dự đoán tập hợp và ghép cặp Hungary để bỏ hẳn NMS, cộng attention biến dạng để chữa hội tụ chậm           | Dòng detector không NMS thực dụng tính tới 9/2026                               |
| Ultralytics YOLO                                   | Bộ gán mẫu động kiểu SimOTA/TAL là một trong những thứ đóng góp nhiều nhất vào chênh lệch giữa các phiên bản   | Được cho là vậy dựa trên code; ghi chú phát hành không phải lúc nào cũng nêu rõ |
| Segment Anything (SAM, và SAM 3 tính tới 9/2026)   | Segmentation theo prompt — bỏ ràng buộc tập nhãn cố định, giữ nguyên đầu ra dạng mask                          | Đang được dùng làm bộ gán nhãn bán tự động trong nhiều pipeline dữ liệu         |
| <code>nnUNet</code>                                | U-Net với skip connection, cấu hình tự động theo tập dữ liệu; vẫn là baseline rất khó đánh bại trong ảnh y tế  | Bằng chứng rằng kiến trúc 2015 chưa hề lỗi thời khi ràng buộc là ít dữ liệu     |
| <code>tide</code> , <code>fiftyone</code>          | Tách mAP thành lỗi phân loại, định vị, trùng lặp, nền — hiện thực của ý “một con số tổng che giấu chế độ hỏng” | Công cụ chẩn đoán, không phải mô hình                                           |

MINI PROJECT SLAM • Lọc vật thể động bằng segmentation trước khi vào frontend SLAM • 1–2 ngày

**Mục tiêu** trả lời một câu hỏi kỹ thuật cụ thể thay vì một câu hỏi học thuật: đưa một mô hình phân vùng có sẵn vào trước frontend để vớt bỏ đặc trưng nằm trên người đi lại, thì quỹ đạo có tốt lên *vượt sà nhiều* của chính harness của bạn không?



|          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Dữ liệu  | một chuỗi EuRoC có người di chuyển trong khung hình, hoặc một chuỗi tự quay trong hành lang có người qua lại; chân trị lấy từ hệ bắt chuyển động hoặc từ một lần chạy tham chiếu. Mô hình phân vùng dùng sẵn, không huấn luyện lại — đây là bài toán tích hợp, không phải bài toán huấn luyện.                                                                                                                                                                            |
| Các bước | (1) chạy mô hình phân vùng ngoại tuyến trên cả chuỗi, lưu mask nhị phân của các lớp động (người, xe) ra đĩa để không làm nhiều phép đo thời gian; (2) chèn một bước lọc vào ngay sau khối trích đặc trưng: bỏ mọi đặc trưng có tâm rơi vào mask; (3) chạy cấu hình gốc và cấu hình có lọc, <i>mỗi cấu hình ít nhất ba lần</i> , ghi ATE bằng <b>evo</b> ; (4) ghi thêm hai số phụ mỗi lần chạy: số đặc trưng còn lại trung bình mỗi khung hình, và số khung hình mất bám. |
| Xong khi | bạn có một bảng ATE trung bình và độ lệch chuẩn cho cả hai cấu hình trên ít nhất ba lần chạy mỗi bên, và phát biểu được một trong hai kết luận có bằng chứng: “cải thiện lớn hơn hai lần độ lệch chuẩn nên không phải nhiễu”, hoặc “cải thiện nằm trong sản nhiễu nên chưa kết luận được”. Kèm một câu giải thích dựa trên hai số phụ: nếu ATE tệ đi, có phải vì mask ăn mất quá nhiều đặc trưng ở vùng giàu kết cấu không?                                               |
| Nối với  | E/24 cho cách đọc chèn lệch trong nhiễu; B/07 cho giao thức đánh giá và việc không được chọn lần chạy đẹp nhất; kết quả của bài này là baseline cho mini project của chương 15.                                                                                                                                                                                                                                                                                           |

### TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì quy tắc gán mẫu nằm trong *định nghĩa* hàm mất mát chứ không trong mô hình: đổi nó là đổi bài toán tối ưu, trong khi đổi backbone chỉ đổi cách giải cùng một bài toán. ATSS cho thấy sau khi thống nhất gán mẫu, khác biệt anchor / anchor-free gần như biến mất.
- A2. Vì chạy backbone một lần cho phép gộp phân loại và hồi quy khung vào một hàm mất mát duy nhất huấn luyện một pha; ở R-CNN ba pha rời rạc, mỗi pha tối ưu cho mục tiêu của riêng nó nên không pha nào tối ưu cho kết quả cuối.
- A3. Vì NMS thao tác trên *quan hệ giữa các dự đoán*, trong khi hàm mất mát của các kiến trúc đó chấm điểm từng dự đoán độc lập — không có chỗ để gán gradient. Gỡ được nó đòi hỏi đổi phát biểu sang dự đoán tập hợp với ghép cặp một-một, chứ không phải thêm một khối mới.
- A4. Rằng chi tiết cài đặt có thể là biến quyết định chứ không phải chi tiết phụ, nên phần đáng soi kỹ nhất của một paper thường là code và bảng ablation, không phải sơ đồ kiến trúc ở trang đầu.
- A5. Rằng ta đã tìm được phát biểu đúng. Khi ba bài toán trở thành ba cách hậu xử lý cùng một tập mask, đơn vị mà mô hình nói tới là một mask kèm một nhãn. Đơn vị đó khớp với cấu trúc thật của bài toán, không còn là quy ước của từng code base.

### NÓI LẠI THẬT ĐƠN GIẢN

Bạn làm rơi chùm chìa khoá trong một căn phòng và nhờ một người máy tìm hộ. Cách hiển nhiên là bảo nó “nhìn cả phòng rồi nói chìa nằm ở đâu”, nhưng người máy không làm được: có hôm một chùm chìa, có hôm ba chùm, có hôm không có chùm nào, mà nó thì chỉ biết trả lời những câu hỏi có hình dạng cố định.

Mẹo của cả chương này là chia phòng thành hàng trăm nghìn ô vuông nhỏ rồi hỏi mỗi ô đúng hai câu giống hệt nhau: “trong ô này có thứ gì không” và “nếu có thì mép của nó cách đây bao xa”. Bây giờ mọi câu hỏi đều cùng một dạng, nên người máy học được. Toàn bộ lịch sử của lĩnh vực là chuyện đưa dần những việc con người vẫn phải làm tay vào bên trong phần mà máy tự chỉnh được. Những việc đó là: khoanh trước vùng đáng nhìn, chọn cỡ ô, và dọn các câu trả lời trùng nhau.

Cái giá có hai khoản. Thứ nhất, hàng chục ô cạnh nhau cùng nói “có”, nên phải có một bước dọn trùng, và bước dọn ấy chính là chỗ hai chùm chìa nằm sát nhau bị nhập làm một. Thứ hai, cái luật quyết định ô nào *được phép* nói “có” cho chùm chìa nào là do người đặt ra, nằm ngoài phần máy học — và đổi luật đó thường làm kết quả thay đổi nhiều hơn đổi cả cái máy.

**Chỗ không rút gọn được:** vì sao một luật chọn ô lại tốt hơn luật khác thì tới nay chủ yếu vẫn là kết quả đo được chứ chưa có lời giải thích gọn; ai nói chắc chắn về điều này là đang nói quá.

**Một câu:** Chia bài toán thành hàng trăm nghìn câu hỏi giống hệt nhau thì máy học được — nhưng luật chấm điểm cho những câu hỏi ấy mới là thứ quyết định kết quả.

## Tổng kết chương

Ta đã đi từ R-CNN, nơi mạng chỉ làm mỗi việc phân loại một mẫu ảnh, tới các hệ thống mà gần như mọi quyết định đều nằm trong tầm với của gradient. Trục xuyên suốt là ranh giới giữa phần học được và phần cố định, và mỗi thế hệ đẩy ranh giới đó xuống thấp hơn. Hai thứ còn sót lại là NMS và cách gán mẫu. Chúng sót lại không ngẫu nhiên: một cái thao tác trên quan hệ *giữa* các dự đoán, cái kia nằm trong chính định nghĩa của loss. Cả hai chỉ gỡ được bằng cách đổi phát biểu bài toán. DETR và Mask2Former là hai lần đổi phát biểu như vậy, và điều đáng chú ý là chúng dùng chung một ý tưởng: dự đoán một tập hợp và ghép một-một.

Bài học thực dụng thì nằm ở chỗ khác và quan trọng hơn: phần quyết định kết quả thường không phải phần được viết trong bài. Cách gán mẫu, tỉ lệ mẫu dương/âm, lịch huấn luyện, ngưỡng NMS — những thứ này giải thích chênh lệch mAP nhiều hơn hình vẽ kiến trúc trên trang đầu. Mang thói quen đó sang mọi chương sau: trước khi tin một cái tiến, hỏi xem có gì khác đã thay đổi cùng lúc.

Còn bỏ ngo hai hướng. Thứ nhất, mọi thứ trong chương này chạy trên một ảnh tĩnh; thêm chiều thời gian sinh ra ba vấn đề hoàn toàn mới và đó là [C/15-thi-giac-khong-thoi-gian](#). Thứ hai, khi tập nhãn không còn cố định, câu hỏi chuyển từ “kiến trúc nào” sang “pretraining trên nguồn giám sát nào”, và đó là [C/18-pretraining-va-foundation](#).

### **Câu hỏi tự kiểm** (trả lời không nhìn bài)

1. Trục của chương là nuốt dần heuristic vào trong mạng. Selective search, RoIPooling và NMS bị nuốt ở ba thời điểm rất khác nhau — điều gì ở mỗi heuristic quyết định nó dễ hay khó nuốt?
2. Quy tắc gán mẫu xuất hiện ở cả detection hai giai đoạn, một giai đoạn và Hungarian matching của DETR. Ba cách gán đó khác nhau ở giả định nào về quan hệ giữa dự đoán và chân trị, và giả định nào hỏng khi vật thể chen chúc?
3. Mất cân bằng lớp trong detection và mất cân bằng lớp trong segmentation đều được chữa bằng cách đổi hàm mất mát, nhưng vì hai lý do khác nhau. Hai lý do đó là gì, và vì sao cùng một liều focal loss lại nguy hiểm ở một bên hơn bên kia?
4. Hệ của bạn chạy detector rồi mới cắt vùng cho head tư thế hoặc OCR. Kiểu hỏng nào của giai đoạn đầu bị thước đo của từng khối che giấu, và bạn đo cả chuỗi bằng cách nào?
5. Hộp, mặt nạ, heatmap khớp và nhúng metric learning là bốn cách phát biểu đầu ra. Với mỗi cách, chỗ nào trong đó là một phép lượng tử hoá hay làm tròn, và cái giá của nó lộ ra ở loại vật thể nào?
6. Vì sao việc ba bài segmentation gộp được vào một mô hình lại là bằng chứng về *phát biểu bài toán*, trong khi việc detection vẫn tách hai nhánh anchor và anchor-free thì không nói lên điều tương tự?
7. Bạn nhận một bảng so sánh detector với mọi dòng cùng backbone. Ba chi tiết nào không nằm trong bảng vẫn đủ sức đảo thứ tự, và bạn hỏi tác giả câu nào trước?

- [1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [2] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [3] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *AISTATS*, 2010.
- [4] K. He, X. Zhang, S. Ren, et al., “Delving Deep into Rectifiers,” in *ICCV*, 2015.
- [5] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [6] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [7] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [8] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [9] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [10] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [11] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” in *ICLR*, 2017.
- [12] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [13] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [14] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [15] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [16] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [17] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [18] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [19] G. Huang, Y. Sun, Z. Liu, et al., “Deep Networks with Stochastic Depth,” in *ECCV*, 2016.
- [20] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [21] H. Zhang, M. Cisse, Y. N. Dauphin, et al., “mixup: Beyond Empirical Risk Minimization,” in *ICLR*, 2018.
- [22] S. Yun, D. Han, S. J. Oh, et al., “CutMix: Regularization Strategy to Train Strong Classifiers,” in *ICCV*, 2019.
- [23] E. D. Cubuk, B. Zoph, J. Shlens, et al., “RandAugment: Practical Automated Data Augmentation,” in *CVPR Workshops*, 2020.

- [24] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [25] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [26] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *NeurIPS*, 2012.
- [27] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in *ICLR*, 2015.
- [28] C. Szegedy, W. Liu, et al., “Going Deeper with Convolutions,” in *CVPR*, 2015.
- [29] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [30] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [31] G. Huang, Z. Liu, L. van der Maaten, et al., “Densely Connected Convolutional Networks,” in *CVPR*, 2017.
- [32] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [33] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [34] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [35] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [36] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [37] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.
- [38] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [39] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [40] J. Su, M. Ahmed, Y. Lu, et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, vol. 568, 2024.
- [41] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” *arXiv:2312.00752*, 2023.
- [42] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [43] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.
- [44] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [45] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [46] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [47] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [48] R. Girshick, J. Donahue, T. Darrell, et al., “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation,” in *CVPR*, 2014.

- [49] R. Girshick, "Fast R-CNN," in *ICCV*, 2015.
- [50] S. Ren, K. He, R. Girshick, et al., "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in *NeurIPS*, 2015.
- [51] T.-Y. Lin, P. Dollár, R. Girshick, et al., "Feature Pyramid Networks for Object Detection," in *CVPR*, 2017.
- [52] K. He, G. Gkioxari, P. Dollár, et al., "Mask R-CNN," in *ICCV*, 2017.
- [53] D. Bolya, S. Foley, J. Hays, et al., "TIDE: A General Toolbox for Identifying Object Detection Errors," in *ECCV*, 2020.
- [54] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [55] J. Redmon, S. Divvala, R. Girshick, et al., "You Only Look Once: Unified, Real-Time Object Detection," in *CVPR*, 2016.
- [56] W. Liu, D. Anguelov, D. Erhan, et al., "SSD: Single Shot MultiBox Detector," in *ECCV*, 2016.
- [57] T.-Y. Lin, P. Goyal, R. Girshick, et al., "Focal Loss for Dense Object Detection," in *ICCV*, 2017.
- [58] Z. Tian, C. Shen, H. Chen, et al., "FCOS: Fully Convolutional One-Stage Object Detection," in *ICCV*, 2019.
- [59] X. Zhou, D. Wang, and P. Krähenbühl, "Objects as Points," *arXiv:1904.07850*, 2019.
- [60] S. Zhang, C. Chi, Y. Yao, et al., "Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection," in *CVPR*, 2020.
- [61] Z. Ge, S. Liu, Z. Li, et al., "OTA: Optimal Transport Assignment for Object Detection," in *CVPR*, 2021.
- [62] N. Bodla, B. Singh, R. Chellappa, et al., "Soft-NMS: Improving Object Detection with One Line of Code," in *ICCV*, 2017.
- [63] X. Zhu, W. Su, L. Lu, et al., "Deformable DETR: Deformable Transformers for End-to-End Object Detection," in *ICLR*, 2021.
- [64] H. Zhang, F. Li, S. Liu, et al., "DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection," in *ICLR*, 2023.
- [65] J. Long, E. Shelhamer, and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," in *CVPR*, 2015.
- [66] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *MICCAI*, 2015.
- [67] L.-C. Chen, G. Papandreou, I. Kokkinos, et al., "DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs," *IEEE TPAMI*, vol. 40, no. 4, pp. 834–848, 2018.
- [68] E. Xie, W. Wang, Z. Yu, et al., "SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers," in *NeurIPS*, 2021.
- [69] B. Cheng, I. Misra, A. G. Schwing, et al., "Masked-Attention Mask Transformer for Universal Image Segmentation," in *CVPR*, 2022.
- [70] A. Kirillov, K. He, R. Girshick, et al., "Panoptic Segmentation," in *CVPR*, 2019.
- [71] A. Kirillov, E. Mintun, N. Ravi, et al., "Segment Anything," in *ICCV*, 2023.
- [72] K. Sun, B. Xiao, D. Liu, et al., "Deep High-Resolution Representation Learning for Human Pose Estimation," in *CVPR*, 2019.



- [73] Z. Cao, T. Simon, S.-E. Wei, et al., “Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields,” in *CVPR*, 2017.
- [74] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [75] J. Deng, J. Guo, N. Xue, et al., “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *CVPR*, 2019.
- [76] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [77] K. Xu, J. Ba, R. Kiros, et al., “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention,” in *ICML*, 2015.
- [78] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *ICLR*, 2015.
- [79] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [80] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [81] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [82] K. Simonyan and A. Zisserman, “Two-Stream Convolutional Networks for Action Recognition in Videos,” in *NeurIPS*, 2014.
- [83] J. Carreira and A. Zisserman, “Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset,” in *CVPR*, 2017.
- [84] D. Tran, H. Wang, L. Torresani, et al., “A Closer Look at Spatiotemporal Convolutions for Action Recognition,” in *CVPR*, 2018.
- [85] C. Feichtenhofer, H. Fan, J. Malik, et al., “SlowFast Networks for Video Recognition,” in *ICCV*, 2019.
- [86] A. Dosovitskiy, P. Fischer, E. Ilg, et al., “FlowNet: Learning Optical Flow with Convolutional Networks,” in *ICCV*, 2015.
- [87] D. Sun, X. Yang, M.-Y. Liu, et al., “PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume,” in *CVPR*, 2018.
- [88] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.
- [89] A. Bewley, Z. Ge, L. Ott, et al., “Simple Online and Realtime Tracking,” in *ICIP*, 2016.
- [90] Y. Zhang, P. Sun, Y. Jiang, et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *ECCV*, 2022.
- [91] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [92] K. Bernardin and R. Stiefelhagen, “Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics,” *EURASIP J. Image and Video Processing*, 2008.
- [93] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [94] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [95] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.



- [96] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *CVPR*, 2024.
- [97] D. Eigen, C. Puhrsch, and R. Fergus, “Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network,” in *NeurIPS*, 2014.
- [98] R. Ranftl, K. Lasinger, D. Hafner, et al., “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer,” *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [99] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IROS*, 2012.
- [100] C. R. Qi, H. Su, K. Mo, et al., “PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation,” in *CVPR*, 2017.
- [101] C. R. Qi, L. Yi, H. Su, et al., “PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space,” in *NeurIPS*, 2017.
- [102] A. H. Lang, S. Vora, H. Caesar, et al., “PointPillars: Fast Encoders for Object Detection from Point Clouds,” in *CVPR*, 2019.
- [103] T. Müller, A. Evans, C. Schied, et al., “Instant Neural Graphics Primitives with a Multiresolution Hash Encoding,” *ACM TOG*, vol. 41, no. 4, 2022.
- [104] B. Kerbl, G. Kopanas, T. Leimkühler, et al., “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *ACM TOG*, vol. 42, no. 4, 2023.
- [105] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.
- [106] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” in *ICLR*, 2014.
- [107] I. Goodfellow, J. Pouget-Abadie, M. Mirza, et al., “Generative Adversarial Nets,” in *NeurIPS*, 2014.
- [108] T. Karras, S. Laine, and T. Aila, “A Style-Based Generator Architecture for Generative Adversarial Networks,” in *CVPR*, 2019.
- [109] J. Ho, A. Jain, and P. Abbeel, “Denoising Diffusion Probabilistic Models,” in *NeurIPS*, 2020.
- [110] Y. Song, J. Sohl-Dickstein, D. P. Kingma, et al., “Score-Based Generative Modeling Through Stochastic Differential Equations,” in *ICLR*, 2021.
- [111] J. Johnson, A. Alahi, and L. Fei-Fei, “Perceptual Losses for Real-Time Style Transfer and Super-Resolution,” in *ECCV*, 2016.
- [112] P. Isola, J.-Y. Zhu, T. Zhou, et al., “Image-to-Image Translation with Conditional Adversarial Networks,” in *CVPR*, 2017.
- [113] L. A. Gatys, A. S. Ecker, and M. Bethge, “Image Style Transfer Using Convolutional Neural Networks,” in *CVPR*, 2016.
- [114] X. Huang and S. Belongie, “Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization,” in *ICCV*, 2017.
- [115] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.
- [116] J. Song, C. Meng, and S. Ermon, “Denoising Diffusion Implicit Models,” in *ICLR*, 2021.
- [117] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, et al., “Flow Matching for Generative Modeling,” in *ICLR*, 2023.
- [118] J. Ho and T. Salimans, “Classifier-Free Diffusion Guidance,” *arXiv:2207.12598*, 2022.
- [119] R. Rombach, A. Blattmann, D. Lorenz, et al., “High-Resolution Image Synthesis with Latent Diffusion Models,” in *CVPR*, 2022.
- [120] W. Peebles and S. Xie, “Scalable Diffusion Models with Transformers,” in *ICCV*, 2023.

- [121] L. Zhang, A. Rao, and M. Agrawala, "Adding Conditional Control to Text-to-Image Diffusion Models," in *ICCV*, 2023.
- [122] Y. Blau and T. Michaeli, "The Perception-Distortion Tradeoff," in *CVPR*, 2018.
- [123] C. Ledig, L. Theis, F. Huszár, et al., "Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network," in *CVPR*, 2017.
- [124] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., "Image Quality Assessment: From Error Visibility to Structural Similarity," *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [125] M. Heusel, H. Ramsauer, T. Unterthiner, et al., "GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium," in *NeurIPS*, 2017.
- [126] T. Brooks, B. Mildenhall, T. Xue, et al., "Unprocessing Images for Learned Raw Denoising," in *CVPR*, 2019.
- [127] A. Abdelhamed, S. Lin, and M. S. Brown, "A High-Quality Denoising Dataset for Smartphone Cameras," in *CVPR*, 2018.
- [128] J. Deng, W. Dong, R. Socher, et al., "ImageNet: A Large-Scale Hierarchical Image Database," in *CVPR*, 2009.
- [129] A. Radford, J. W. Kim, C. Hallacy, et al., "Learning Transferable Visual Models from Natural Language Supervision," in *ICML*, 2021.
- [130] J. Li, D. Li, S. Savarese, et al., "BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models," in *ICML*, 2023.
- [131] H. Liu, C. Li, Q. Wu, et al., "Visual Instruction Tuning," in *NeurIPS*, 2023.
- [132] S. Kornblith, J. Shlens, and Q. V. Le, "Do Better ImageNet Models Transfer Better?" In *CVPR*, 2019.
- [133] T. Chen, S. Kornblith, M. Norouzi, et al., "A Simple Framework for Contrastive Learning of Visual Representations," in *ICML*, 2020.
- [134] K. He, H. Fan, Y. Wu, et al., "Momentum Contrast for Unsupervised Visual Representation Learning," in *CVPR*, 2020.
- [135] J.-B. Grill, F. Strub, F. Altché, et al., "Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning," in *NeurIPS*, 2020.
- [136] M. Caron, H. Touvron, I. Misra, et al., "Emerging Properties in Self-Supervised Vision Transformers," in *ICCV*, 2021.
- [137] K. He, X. Chen, S. Xie, et al., "Masked Autoencoders Are Scalable Vision Learners," in *CVPR*, 2022.
- [138] A. van den Oord, Y. Li, and O. Vinyals, "Representation Learning with Contrastive Predictive Coding," in *arXiv:1807.03748*, 2018.
- [139] J. Kaplan, S. McCandlish, T. Henighan, et al., "Scaling Laws for Neural Language Models," *arXiv:2001.08361*, 2020.
- [140] X. Zhai, A. Kolesnikov, N. Houlsby, et al., "Scaling Vision Transformers," in *CVPR*, 2022.
- [141] J. Hoffmann, S. Borgeaud, A. Mensch, et al., "Training Compute-Optimal Large Language Models," in *NeurIPS*, 2022.
- [142] C. G. Northcutt, A. Athalye, and J. Mueller, "Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks," in *NeurIPS Datasets and Benchmarks*, 2021.
- [143] B. Recht, R. Roelofs, L. Schmidt, et al., "Do ImageNet Classifiers Generalize to ImageNet?" In *ICML*, 2019.
- [144] N. Houlsby, A. Giurgiu, S. Jastrzebski, et al., "Parameter-Efficient Transfer Learning for NLP," in *ICML*, 2019.

- [145] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [146] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *RecSys*, 2019.

## Thị giác chuỗi và không-thời gian (Sequential and Spatio-Temporal Vision)

**PHỤ THUỘC** C/13-kien-truc-backbone (CNN, attention); C/14-bai-toan-cv-loi (detection, segmentation — tracking dựng thẳng trên detection); B/11-gioi-han-tinh-toan (ngân sách bộ nhớ và tính toán); B/07-chia-du-lieu-va-danh-gia (chọn metric, rò rỉ dữ liệu). [T3]

### TÓM TẮT

Chương này hỏi: thêm chiều thời gian vào ảnh thì phát sinh những vấn đề nào mà ảnh tĩnh không có? Câu trả lời là đúng ba vấn đề, và ba mục của chương tương ứng với ba vấn đề đó. Thứ nhất, đầu ra có độ dài thay đổi — đây là nơi *attention* ra đời. Thứ hai, chi phí bộ nhớ nhân theo số khung hình. Thứ ba, mô hình chạy trực tuyến không được nhìn khung hình tương lai. Đọc xong bạn thiết kế được một hệ thị giác thời gian thực bắt đầu từ ràng buộc chứ không từ bảng xếp hạng: biết kiến trúc nào bị loại ngay, ngân sách bộ nhớ phải chia thế nào, và metric nào không đánh lừa. Nếu chỉ nhớ một điều: hai bài toán khó nhất của thị giác thời gian là tương ứng điểm và liên kết dữ liệu. Đó đúng là hai bài toán mà một hệ SLAM giải ở mỗi khung hình. Phần lớn chương này vì thế là dịch từ vựng, không phải học lại từ đầu.

### KIẾN THỨC CẦN CÓ TRƯỚC

- **khung hình (frame) và clip** — khung hình là một ảnh tại một thời điểm; clip là một đoạn gồm  $T$  khung liên tiếp, và là đơn vị đầu vào của mô hình video — tương đương “một ảnh” của mô hình ảnh.
- **trực tuyến (online / streaming)** — chế độ chạy mà dữ liệu đến dần theo thời gian và phải trả kết quả ngay, khác với chế độ ngoại tuyến được xem toàn bộ video rồi mới quyết định.
- **nhân quả (causal)** — tính chất của một mô hình chỉ dùng thông tin tới thời điểm hiện tại. Mô hình hai chiều — nhìn cả trước lẫn sau — không nhân quả, nên không dùng được cho hệ trực tuyến.
- **độ trễ (latency) và thông lượng (throughput)** — độ trễ là thời gian từ lúc khung hình vào tới lúc có kết quả; thông lượng là số khung xử lý được mỗi giây. Hai đại lượng này độc lập: gộp batch làm thông lượng tăng nhưng độ trễ tăng theo.
- **optical flow** — trường vector cho biết mỗi pixel dịch chuyển đi đâu giữa hai khung hình — biểu diễn tường minh của chuyển động, và là dạng dày đặc của bài toán tương ứng điểm.
- **tracking và track ID** — tracking là giữ danh tính của từng đối tượng qua thời gian; track ID là mã số phải giữ nguyên suốt thời gian đối tượng xuất hiện. Đánh tráo mã số giữa hai đối tượng gọi là *ID switch*.
- **attention** — cơ chế cho bên cần thông tin tính một bộ trọng số trên các phần của bên có thông tin rồi lấy tổ hợp có trọng số — tức hoãn việc nén dữ liệu lại cho tới khi biết đang cần gì.

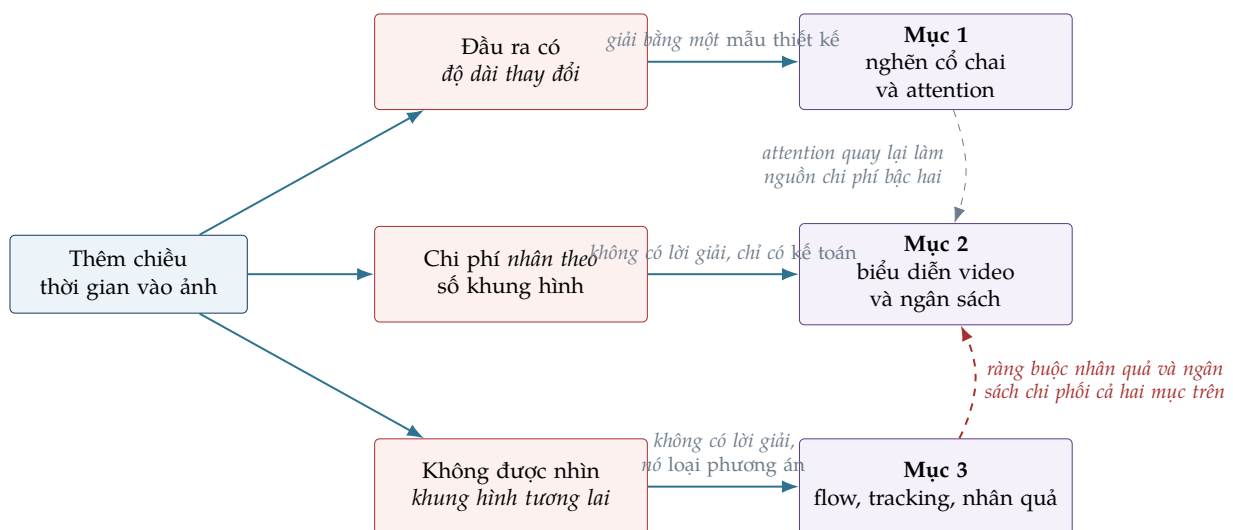
### CÂU HỎI MỞ ĐẦU

- Q1. Vì sao một mô hình đạt mIoU cao trên từng khung hình vẫn cho ra video nhấp nháy, và vì sao không có thước đo phổ biến nào bắt được lỗi đó?

- Q2.** *attention* thường được kể như một phát minh của Transformer. Vậy nó thực sự ra đời để giải quyết nút thắt nào, và vì sao nút thắt đó xuất hiện lần đầu ở bài toán mô tả ảnh chứ không ở phân loại?
- Q3.** Một hệ tracking có thể đánh tráo danh tính đối tượng liên tục mà chỉ số MOTA gần như không suy chuyển. Công thức nào cho phép chuyển đó xảy ra?
- Q4.** Vì sao “thời gian là trục thứ ba của tensor” là một câu nói sai, dù về mặt lập trình nó hoàn toàn đúng?
- Q5.** Nếu bạn tăng gấp bốn độ dài clip mà giữ nguyên bộ nhớ, bạn phải trả bằng gì — và vì sao ba cách trả giá đó không hề tương đương?

Chương 14 xử lý từng ảnh như một ốc đảo. Đó là một đơn giản hoá rất tiện nhưng rất xa thực tế. Camera không sinh ra ảnh rời rạc mà sinh ra một dòng liên tục. Phần lớn thông tin hữu ích chỉ tồn tại *giữa* các khung hình, không nằm trong khung nào cả. Một vật đang tiến lại gần, một người vừa ngã, cùng một chiếc xe hay hai chiếc khác nhau — không câu nào trong số đó trả lời được từ một ảnh tĩnh.

Thêm chiều thời gian không phải là thêm một chiều nữa cho tensor. Nó sinh ra đúng ba vấn đề mới, và ba vấn đề đó độc lập với nhau tới mức mỗi cái đáng một mục riêng. Thứ nhất, **đầu ra có thể có độ dài thay đổi**: mô tả một cảnh cần một câu, không phải một nhãn, và việc phải nén cả ảnh vào một vector trước khi sinh câu là nút thắt đã khai sinh ra *attention* trong thị giác. Thứ hai, **chi phí nhân theo số khung hình**. Ngân sách bộ nhớ của một mô hình ảnh vốn đã chật, giờ bị chia tiếp cho ba chiều: độ dài clip, độ phân giải, batch size. Mọi lựa chọn kiến trúc vì thế là lựa chọn hy sinh chiều nào. Thứ ba, **ràng buộc nhân quả**: hệ chạy trực tuyến không được nhìn khung hình tương lai, và ràng buộc đó loại bỏ nguyên một lớp kiến trúc trước khi độ chính xác kịp lên tiếng.



**Hình 4.1.** Bản đồ chương: ba mục không phải ba chủ đề rời mà là ba *loại vấn đề* khác nhau mà chiều thời gian sinh ra, và ba loại câu trả lời khác nhau về bản chất. Mũi tên đứt cho thấy ba nhánh không độc lập: ràng buộc nhân quả và ngân sách bộ nhớ quyết định cách chữa nào ở mục 3 là khả thi, còn *attention* của mục 1 quay lại chính là nguồn chi phí bậc hai của mục 2.

Hình 4.1 nên xem trước khi đọc, vì nó cho biết mục nào trả lời loại câu hỏi nào — và vì thế mục nào đáng đọc kỹ với bài toán cụ thể của bạn.

Ba mục của chương lần lượt xử lý ba vấn đề đó, theo thứ tự phụ thuộc chứ không theo thứ tự lịch sử. Mục 1 đi từ sinh mô tả ảnh kiểu CNN–RNN tới “Show, Attend and Tell”. Nên đọc nó *trước* phần Transformer của chương 13. Lý do là nó cho thấy *attention* ra đời để gỡ một nút thắt cụ thể, không phải để làm kiến trúc đẹp hơn. Mục 2 chuyển sang biểu diễn video và biến ngân

sách bộ nhớ thành những con số tính được bằng tay, vì đó là ràng buộc mà mọi quyết định phía sau phải sống chung. Mục 3 gom ba bài toán mà một kỹ sư robot gặp hằng ngày — optical flow, tracking, và tính nhất quán thời gian — rồi kết thúc bằng ràng buộc nhân quả, thứ chi phối tất cả.

Với người đọc đến từ hình học và SLAM, chương này gần chuyên môn hơn bất kỳ chương nào khác trong Phần C, và cách đọc hiệu quả nhất là đọc như một cuốn từ điển đối chiếu. Optical flow là bài toán tương ứng điểm ở dạng dày đặc, với đúng bài toán khẩu độ và đúng ma trận cấu trúc mà bạn dùng để chọn đặc trưng góc. Tracking là bài toán liên kết dữ liệu, với đúng bộ đôi dự đoán bằng bộ lọc trạng thái cộng gán tối ưu. Còn ràng buộc nhân quả và ngân sách mili-giây thì bạn đã sống cùng nó hằng ngày. Cái mới ở đây không phải cấu trúc bài toán mà là các thành phần học được đặt vào từng vị trí, cùng với câu hỏi quen thuộc: mỗi lần thay một khối hình học bằng một khối học được thì ta mua được độ bền gì và mất đi tính chất gì.

**Bảng 4.1.** Từ điển đối chiếu cho người đến từ SLAM. Cột giữa và cột phải là cùng một bài toán với hai truyền thống thuật ngữ khác nhau; chỉ cột cuối là thứ thực sự mới.

| Bài toán                 | Tên trong SLAM / hình học                                    | Tên trong thị giác học sâu, và cái mới                                                               |
|--------------------------|--------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| Tương ứng điểm           | Ghép đặc trưng, KLT, tìm kiếm dọc đường epipolar             | Optical flow dày đặc; cái mới là học được prior về chuyển động nên vùng không kết cấu cũng có đáp án |
| Liên kết dữ liệu         | Ghép đặc trưng với điểm bản đồ, công lọc Mahalanobis, RANSAC | Tracking-by-detection; cái mới là dùng thêm đặc trưng ngoại hình học được làm chi phí ghép           |
| Dự đoán trạng thái       | Bộ lọc Kalman, mô hình chuyển động hằng vận tốc              | Vẫn là Kalman trong đa số bộ tracker; cái mới nằm ở phần đo, không ở phần dự đoán                    |
| Chọn đặc trưng tốt       | Giá trị riêng nhỏ nhất của ma trận cấu trúc                  | Cùng ma trận đó xuất hiện trong Lucas–Kanade; mô hình học sâu thay tiêu chí này bằng khối tương quan |
| Nhận diện nơi đã qua     | Túi từ trực quan, kiểm chứng hình học                        | Nhúng toàn cục cộng bước quay lại đặc trưng — đúng cấu trúc của attention                            |
| Ngân sách thời gian thực | Giới hạn số vòng lặp tối ưu, kích thước cửa sổ trượt         | Số bước tinh chỉnh của RAFT, độ dài clip, cửa sổ nhân quả                                            |

Bảng 4.1 nên đọc trước cả ba mục. Nó cho biết chỗ nào bạn được phép đọc lướt vì cấu trúc bài toán đã quen. Nó cũng chỉ ra chỗ phải đọc kỹ: nơi một thành phần học được thay vào, mang theo giả định mới.

## Bản đồ chương 15 – Thị giác chuỗi và không-thời gian

### TÓM TẮT

Chương 15 chia làm ba mục theo đúng ba vấn đề mà chiều thời gian sinh ra, chứ không theo thứ tự lịch sử của các kiến trúc. Ba vấn đề đó là: đầu ra có độ dài thay đổi, chi phí bộ nhớ nhân theo số khung hình, và ràng buộc nhân quả. Bản đồ này nói mục nào cốt lõi, mục nào tra khi cần, và mục nào nên đọc chen vào giữa chương 13. Nếu chỉ nhớ một điều: mục 3 là mục gần chuyên môn của một kỹ sư SLAM nhất và cũng là mục dùng được ngay nhất, còn mục 1 nên đọc *trước* phần Transformer của chương 13.



**KIẾN THỨC CẦN CÓ TRƯỚC**

- **chương và mục** — chương là một chủ đề lớn, mỗi mục là một file đọc độc lập được. Số trong tên thư mục là thứ tự nên đọc, không phải thứ tự lịch sử.
- **trực tuyến (streaming) và ngoại tuyến** — trực tuyến là xử lý dữ liệu đến dần và trả kết quả ngay; ngoại tuyến là được xem toàn bộ video rồi mới quyết định. Phần lớn số liệu công bố là ngoại tuyến.
- **nhân quả (causal)** — chỉ dùng thông tin tới thời điểm hiện tại, không nhìn khung hình tương lai — ràng buộc cứng của mọi hệ chạy trên robot.
- **ngân sách bộ nhớ ba chiều** — lượng bộ nhớ khi huấn luyện video tỉ lệ với độ dài clip  $\times$  độ phân giải<sup>2</sup>  $\times$  batch size, nên tăng một chiều buộc phải giảm chiều khác.

**Có gì trong chương này**

- **Mục 1 — Đầu ra độ dài thay đổi và sự ra đời của attention (cốt lõi, đọc sớm)**. Sinh mô tả ảnh kiểu CNN-RNN, nghẽn cổ chai của một vector duy nhất, và “Show, Attend and Tell”. Giá trị lớn nhất của mục không phải bài toán captioning mà là việc nó cho thấy *động cơ* của attention.
- **Mục 2 — Biểu diễn video và cái giá của thời gian (cốt lõi)**. Gộp theo khung, hai luồng, tích chập 3D, bản tách (2+1)D, SlowFast, video transformer — và ba phép tính bộ nhớ quyết định tất cả.
- **Mục 3 — Flow, tracking và các ràng buộc riêng (cốt lõi, gần chuyên môn nhất)**. Lucas-Kanade tới RAFT, định vị hành động theo khoảng thời gian, tracking như bài toán liên kết dữ liệu, bầy của MOTA và cách HOTA sửa nó, tính nhất quán thời gian, và ràng buộc nhân quả.

**Vì sao chia như vậy**

Cách chia hiển nhiên sẽ là chia theo bài toán — captioning, phân loại hành động, tracking — nhưng cách đó che mất điều đáng học. Ba mục ở đây chia theo *ba vấn đề cấu trúc* mà chiều thời gian sinh ra, và mỗi vấn đề có một dạng câu trả lời khác hẳn. Vấn đề thứ nhất được giải bằng một mẫu thiết kế, tức attention. Vấn đề thứ hai không có lời giải, chỉ có kế toán. Vấn đề thứ ba cũng không có lời giải; nó chỉ loại bỏ phương án. Nhìn theo trục này thì các kiến trúc cụ thể trở thành hệ quả, và bạn không phải nhớ chúng như một danh sách.

**Thứ tự đọc và phụ thuộc**

1. Đọc mục 1 *trước* [C/13-kien-truc-backbone/02-tu-chuoi-toi-attention](#): hiểu nút thắt trước thì self-attention thành hệ quả hiển nhiên thay vì một phát minh khó hiểu.
2. Mục 2 trước mục 3: ngân sách bộ nhớ ở mục 2 là thứ quyết định cách chữa nào cho hiện tượng nhấp nháy là khả thi ở mục 3.
3. Mục 3 giả định bạn đã đọc [C/14-bai-toan-cv-loi](#), vì tracking-by-detection dựng thẳng trên một detector.
4. Đọc [B/07-chia-du-lieu-va-danh-gia](#) song song với mục 3 nếu phần metric và rò rỉ dữ liệu theo clip còn mờ.
5. [C/16-thi-giac-3d](#) là chương tiếp nối tự nhiên: nó nâng bài toán tương ứng điểm từ mặt phẳng ảnh lên không gian ba chiều.

**Nếu chỉ có một buổi**

Đọc mục 3, và trong mục 3 đọc kỹ hai chỗ. Thứ nhất là bài toán khâu độ với ma trận cấu trúc: nó nổi thẳng vào việc chọn đặc trưng mà bạn đã làm. Thứ hai là hộp cạm bẫy về MOTA: nó thay đổi ngay cách bạn báo cáo kết quả tracking. Phần còn lại có thể đọc lướt và quay lại khi cần.

## 4.1 Đầu ra độ dài thay đổi và sự ra đời của attention (Variable-Length Outputs and the Birth of Attention)

**PHỤ THUỘC** C/13-kien-truc-backbone/02-tu-chuoi-toi-attention (RNN, LSTM, attention — mục này là *động cơ* của mục đó, nên đọc mục này trước sẽ dễ hơn); B/05-thiet-ke-bai-toan (dạng đầu ra).

### TÓM TẮT

Mục này kể lại đúng một sự kiện: **attention** không sinh ra từ mong muốn làm kiến trúc đẹp hơn, mà sinh ra để gỡ một nút thắt rất cụ thể — việc nén cả một ảnh vào một vector duy nhất trước khi sinh câu mô tả. Đọc xong bạn hiểu *vì sao* self-attention của Transformer là hiển nhiên chứ không bí ẩn, và bạn nhận ra được cùng một nút thắt ở những nơi khác, kể cả trong pipeline SLAM của chính mình. Nếu chỉ nhớ một điều: ở đâu một hệ thống nén thông tin có cấu trúc thành một vector cố định *trước khi* biết sẽ cần phần nào, ở đó có một nút thắt. Attention là cách chuẩn để gỡ nó, bằng cách hoãn phép nén tới lúc biết đang cần gì.

### KIẾN THỨC CẦN CÓ TRƯỚC

- **token** — một đơn vị rời rạc của chuỗi đầu ra — thường là một từ hoặc một mảnh từ. Mô hình sinh câu bằng cách dự đoán token tiếp theo, lặp lại tới khi gặp token kết thúc.
- **RNN và LSTM** — mạng xử lý chuỗi bằng cách mang theo một *trạng thái ẩn* (một vector) từ bước này sang bước sau. LSTM là biến thể có cổng, giúp trạng thái giữ được thông tin qua nhiều bước hơn.
- **encoder-decoder** — encoder biến đầu vào thành biểu diễn số; decoder sinh đầu ra từ biểu diễn đó. Ở đây encoder là CNN nhìn ảnh, decoder là mạng sinh câu.
- **feature map (lưới đặc trưng)** — đầu ra không gian của CNN: một lưới, ví dụ  $14 \times 14$ , mỗi ô là một vector mô tả một vùng ảnh. Giữ nguyên lưới nghĩa là giữ nguyên thông tin “ở đâu”.
- **global average pooling** — phép lấy trung bình mọi ô của lưới đặc trưng để ra một vector duy nhất. Rất tiện cho phân loại, nhưng nó xoá sạch thông tin vị trí.
- **attention** — cơ chế cho một bên (bên cần thông tin) tính một bộ trọng số trên các phần của bên kia (bên có thông tin), rồi lấy tổ hợp có trọng số. Trọng số phụ thuộc vào việc đang cần gì, nên phép nén được “hoãn” tới lúc có truy vấn.
- **self-attention** — trường hợp hai bên là cùng một tập: mỗi phần tử tự tính trọng số trên mọi phần tử khác của chính chuỗi đó. Đây là khối cơ bản của Transformer.

### Đặt vấn đề

Toàn bộ chương 14 giả định một điều mà ta không để ý: đầu ra tuy có thể là tập hợp nhưng *mỗi phần tử* của nó vẫn có hình dạng cố định — một khung là bốn số, một mask là một lưới nhị phân. **Sinh mô tả ảnh** (*image captioning*) phá vỡ cả điều đó: đầu ra là một câu, dài ngắn tùy ảnh, và mỗi từ phụ thuộc vào các từ trước. Đây là lần đầu thị giác máy tính phải sinh ra một chuỗi, và nó buộc phải mượn bộ máy của xử lý ngôn ngữ.

Giải pháp đầu tiên nghe rất hợp lý. Cho ảnh qua CNN, lấy vector đặc trưng ở tầng áp chót, dùng nó làm trạng thái khởi tạo cho một **mạng hồi quy** (*recurrent neural network, RNN*) hoặc **LSTM** [38], rồi để mạng đó sinh từng **token** một. Kiến trúc này chạy được và cho ra những câu nghe khá trôi chảy. Nhưng nó chứa một giả định chết người: rằng *một* vector cố định — thường 512 hay 2048 số — đủ để trả lời mọi câu hỏi mà mọi từ trong câu sẽ đặt ra. Khi mạng đang sinh từ “con chó”, nó cần biết chi tiết ở góc dưới bên trái; khi sinh từ “bãi cỏ”, nó cần chi tiết ở nền. Vector kia phải chứa sẵn cả hai, và cả những chi tiết cho mọi câu khác mà nó không biết trước.

**Học xong mục này bạn làm được gì mà trước đó không làm được?** Bạn phát hiện được nút thắt kiểu “nén trước, dùng sau” trong một kiến trúc bất kỳ, kể cả kiến trúc không dính gì tới ngôn ngữ. Bạn cũng biết attention là công cụ chuẩn để gỡ nó, và biết cái giá phải trả khi gỡ.

### TRỰC GIÁC

Hãy tưởng tượng bạn phải kể lại một bộ phim cho người khác, nhưng luật chơi là: xem xong, bạn chỉ được ghi *một* câu tóm tắt vào giấy, rồi trả lại bằng phim, và từ đó chỉ được nhìn tờ giấy để kể suốt hai mươi phút. Bạn sẽ kể trôi chảy, nhưng mọi chi tiết đều là bịa hợp lý. Attention là việc bỏ luật đó đi: mỗi khi kể đến một cảnh, bạn được phép tua lại băng và nhìn đúng cảnh ấy. Không có gì huyền bí — toàn bộ ý tưởng chỉ là *hoãn việc nén lại cho tới khi biết mình đang cần gì*.

## A. Từ một vector tới một cái nhìn có chọn lọc

### I Mạch 1 — đầu ra độ dài thay đổi

- 1 Vấn đề** đầu ra là một câu, không phải một nhãn.

**Giải pháp** gán một bộ giải mã chuỗi lên trên bộ trích đặc trưng: CNN cho vector, RNN/LSTM sinh từng token, huấn luyện bằng **entropy chéo** (*cross-entropy*) trên từng vị trí token.

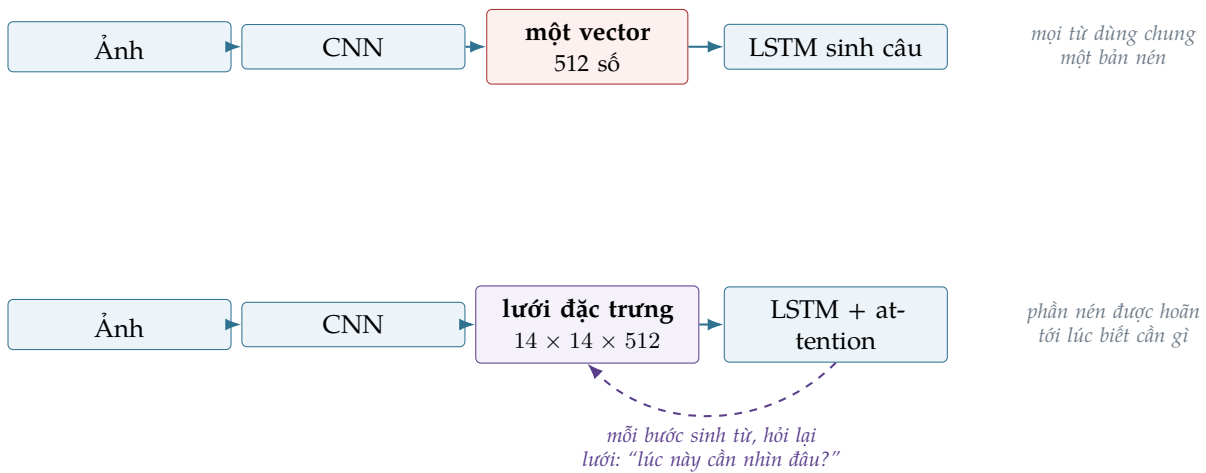
**Giả định** đặc trưng toàn cục của ảnh chứa đủ thông tin cho toàn bộ câu.

**Hệ quả** lần đầu tiên một hệ thị giác sinh được đầu ra có độ dài thay đổi.
- 2 Vấn đề** một vector duy nhất là nghẽn cổ chai thông tin. Mọi chi tiết không gian bị nén trước khi ta biết sẽ cần chi tiết nào; và nén *có mất mát* thì không đảo ngược được.

**Giải pháp** “Show, Attend and Tell” giữ nguyên lưới đặc trưng không gian [77]. Ở mỗi bước sinh từ, bộ giải mã tính một phân bố trọng số trên các ô lưới. Tổ hợp có trọng số của các ô đó là đầu vào cho bước ấy.

**Hệ quả** đây là nơi attention xuất hiện lần đầu trong thị giác, và nó ra đời để gỡ một nút thắt cụ thể chứ không phải vì đẹp. Ý tưởng gốc đến từ dịch máy, nơi cùng một nghẽn cổ chai xuất hiện ở dạng vector câu nguồn [78].
- 3 Vấn đề** nếu attention hữu ích cho quan hệ giữa *câu và ảnh*, tại sao không dùng cho quan hệ giữa *các phần của cùng một ảnh*?

**Giải pháp** đó chính là bước đi tới self-attention và **transformer** [39], và đó là lý do nên đọc mục này *trước* [C/13-kien-truc-backbone/02-tu-chuoi-toi-attention](#): khi đã thấy nút thắt, self-attention trở thành hệ quả hiển nhiên thay vì một phát minh từ trên trời rơi xuống.



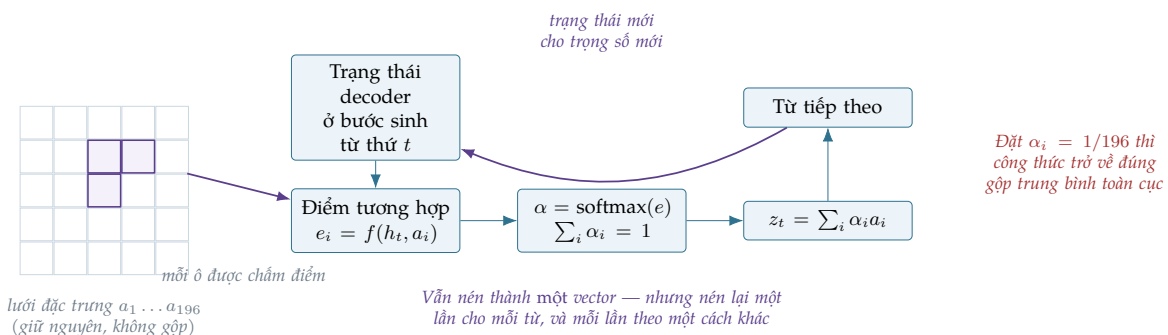
**Hình 4.2.** Cùng một pipeline, khác nhau đúng một chỗ. Nhánh trên nén lưới đặc trưng thành một vector trước khi biết câu sẽ nói gì; nhánh dưới giữ nguyên lưới và để bộ giải mã truy vấn lại nó ở mỗi bước. Mỗi tên dứt là toàn bộ nội dung của attention — một vòng phản hồi từ nơi cần thông tin về nơi có thông tin.

### B. Nút thắt lớn tới mức nào — một phép tính

Hình 4.2 sẽ thuyết phục hơn nhiều khi kèm con số. Lưới đặc trưng của một CNN trước tầng gộp toàn cục có kích thước điển hình  $14 \times 14 \times 512$ , tức  $196 \times 512 = 100\,352$  số. Sau **gộp trung bình toàn cục** (*global average pooling*) chỉ còn 512 số. Tỉ số nén là  $196:1$ , và điều quan trọng không phải tỉ số mà là *cái gì bị mất*: phép gộp lấy trung bình theo không gian, nên nó xoá sạch thông tin “đặc trưng này nằm ở đâu”. Một câu mô tả cần đúng thông tin đó để nói “con chó bên trái người đàn ông”.

↔ **hệ quả của C/13/02** — self-attention là bước tiếp theo hiển nhiên khi đem chính cơ chế này áp cho quan hệ giữa các phần bên trong một đầu vào, thay vì giữa hai đầu vào khác nhau.

Nói gọn thì attention thay phép trung bình đều bằng phép trung bình có trọng số, với trọng số phụ thuộc vào việc bộ giải mã đang cần gì: gọi  $a_i$  là vector đặc trưng của ô thứ  $i$ , và  $\alpha_i$  là trọng số attention tại bước sinh từ hiện tại. Đầu vào của bước đó là  $z = \sum_i \alpha_i a_i$ , với  $\sum_i \alpha_i = 1$ . Công thức này nói một điều rất đơn giản: *vẫn nén thành một vector, nhưng nén lại một lần cho mỗi từ, và mỗi lần nén theo một cách khác nhau*. Trường hợp  $\alpha_i = 1/196$  với mọi  $i$  chính là gộp trung bình toàn cục — mô hình cũ là một trường hợp riêng của mô hình mới.



**Hình 4.3.** Cơ chế attention viết ra thành luồng tính. Điều đáng chú ý không phải công thức mà là vòng phản hồi: trạng thái của bộ giải mã quyết định trọng số, trọng số quyết định vector đưa vào bước sau, và bước sau lại đổi trạng thái. Mô hình cũ là trường hợp riêng khi trọng số bị ép bằng nhau, nên đây là một phép tổng quát hoá chứ không phải một kiến trúc cạnh tranh.

Hình 4.3 cũng cho thấy chi phí: phần chấm điểm phải chạy lại cho *mỗi* từ trên *mọi* ô lưới, nên tổng chi phí tỉ lệ với tích số ô nhân số bước sinh — mằm mông của vấn đề bậc hai ở mục sau.

**Bảng 4.2.** Ba cách đưa thông tin ảnh vào bộ giải mã chuỗi.

| Cách                | Cơ chế                                 | Giá phải trả                      | Hỏng khi nào                                                                |
|---------------------|----------------------------------------|-----------------------------------|-----------------------------------------------------------------------------|
| Một vector toàn cục | Gộp trung bình rồi khởi tạo trạng thái | Rẻ nhất, một lần tính             | Câu cần chi tiết cục bộ hoặc quan hệ không gian; mô hình bị chi tiết hợp lý |
| Soft attention      | Trung bình có trọng số, khả vi         | Chi phí tỉ lệ số ô $\times$ số từ | Ảnh nhiều vật giống nhau: trọng số dàn đều, không chọn được                 |
| Hard attention      | Lấy mẫu rồi rạc một ô                  | Rẻ khi chạy, nhưng không khả vi   | Phải huấn luyện bằng ước lượng gradient có phương sai cao; kém ổn định      |

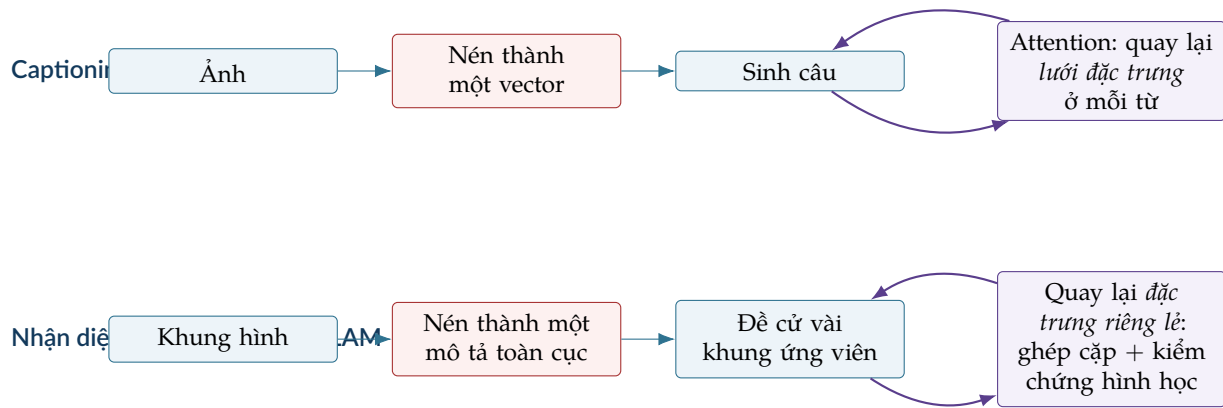
↔ cùng nguyên lý với C/14/02 — mỗi truy vấn của DETR “hỏi” feature map xem vật của nó ở đâu — đúng vòng phản hồi từ nơi cần thông tin về nơi có thông tin.

### C. Cùng một nút thắt trong pipeline SLAM

Chỗ nổi này đáng dừng lại, vì nó cho thấy nút thắt trên không phải chuyện riêng của ngôn ngữ. Một hệ *SLAM* nhận diện nơi đã đi qua bằng cách nén cả khung hình thành một mô tả toàn cục — túi từ trực quan hoặc một vector nhúng — rồi so mô tả đó với cơ sở dữ liệu [79], [80]. Bước nén ấy có *đúng* tính chất của nghẽn cổ chai: nó xảy ra trước khi biết sẽ cần kiểm chứng điều gì, nên nó vứt bỏ hình học.

Và giải pháp mà cộng đồng SLAM dùng cũng có đúng cấu trúc của attention: sau khi mô tả toàn cục đề cử vài ứng viên, hệ thống *quay lại* các đặc trưng riêng lẻ để ghép cặp và kiểm chứng bằng ràng buộc hình học. Nói cách khác, mô tả toàn cục chỉ dùng để *chọn nơi cần nhìn kỹ*, chứ không dùng để kết luận — y hệt vai trò của phân bố  $\alpha$  trong công thức trên. Nếu bạn thấy self-attention khó hình dung, hãy nghĩ nó như bước kiểm chứng đặc trưng đó, chỉ khác là trọng số được học thay vì được tính bằng RANSAC.

↔ cùng nguyên lý với C/16 — nhận diện nơi đã đi qua nén khung hình thành một mô tả toàn cục rồi phải quay lại đặc trưng để kiểm chứng hình học — cùng cấu trúc “đề cử rồi nhìn kỹ” của attention.



Cùng một cấu trúc: bản nén toàn cục chỉ dùng để chọn nơi cần nhìn kỹ, không dùng để kết luận. Khác biệt duy nhất là trọng số quay lại được học (attention) hay được tính bằng ràng buộc hình học (RANSAC).

**Hình 4.4.** Nghẽn cổ chai “nén trước, dùng sau” không phải chuyện riêng của ngôn ngữ. Hệ SLAM gặp đúng nó khi nén một khung hình thành một mô tả toàn cục để tra cứu, và giải đúng theo cách của attention: dùng bản nén để đề cử, rồi quay về nguồn thông tin đầy đủ để kiểm chứng. Nhận ra được cấu trúc này thì self-attention thôi trông giống một phát minh xa lạ.

Hình 4.4 là lý do mục này nên đọc trước phần Transformer: khi mẫu thiết kế đã quen, phần công thức chỉ còn là chi tiết.

#### D. Giới hạn và trường hợp hỏng

- **Thước đo của captioning rất yếu.** Các độ đo dựa trên trùng khớp  $n$ -gram thường cho câu “an toàn”: đúng ngữ pháp và giống câu tham chiếu. Vì thế một mô hình sinh mô tả chung chung có thể ghi điểm cao hơn một mô hình mô tả đúng nhưng diễn đạt khác. *Hệ quả thực tế:* cải tiến trên bảng xếp hạng không bảo đảm cải tiến khi người dùng thật đọc.
- **Mô hình bịa chi tiết.** Vì bộ giải mã được huấn luyện để sinh câu *hợp lý*, nó lấp chỗ trống bằng thứ thường đúng — “người đàn ông cầm ván lướt sóng” cho một ảnh bãi biển không có ván. Nút thắt càng chặt thì hiện tượng này càng nặng, nên đây là triệu chứng chẩn đoán được.
- **Bản đồ attention không phải lời giải thích.** Rất dễ nhìn  $\alpha$  rồi tin rằng mô hình “đang nhìn vào đó vì lý do đó”. Các kiểm tra tính đúng đắn cho thấy nhiều loại bản đồ nổi bật vẫn trông hợp lý ngay cả khi trọng số mô hình bị làm hỏng [81], nên trọng số attention là *manh mối chẩn đoán*, không phải bằng chứng.
- **Chi phí tăng theo tích số ô  $\times$  số bước.** Với ảnh phân giải cao hoặc câu dài, phần attention có thể vượt chi phí của chính CNN. Đây là mầm mống của vấn đề bậc hai ở mục sau, khi số ô còn nhân thêm với số khung hình.

#### CẦN NHỚ

Nghẽn cổ chai thông tin xuất hiện ở *mọi* chỗ mà hệ thống nén dữ liệu có cấu trúc thành một vector cố định trước khi biết sẽ cần phần nào. Attention không phải một loại tầng mạng, nó là một mẫu thiết kế: *giữ nguyên nguồn, và hoãn việc nén tới lúc có truy vấn*. Nhận ra mẫu này rồi thì Transformer, cross-attention trong DETR, và cả bước kiểm chứng hình học trong SLAM đều là cùng một thứ. [T2]

#### E. Kết nối

C/13-kien-truc-backbone/02-tu-chuoi-toi-attention là nơi hình thức hoá đầy đủ; mục này là động cơ của nó, nên thứ tự đọc tự nhiên là mục này trước.



[C/14-bai-toan-cv-loi/02-detection-mot-giai-doan](#) dùng lại chính cơ chế này dưới dạng truy vấn của DETR: mỗi truy vấn “hỏi” feature map xem vật của nó ở đâu. [C/15-thi-giac-khong-thoi-gian/02-bieu-dien-video](#) là bước kế tiếp trực tiếp: khi nguồn không chỉ là một lưới không gian mà là một khối không-thời gian, chi phí attention nhân lên và trở thành ràng buộc thiết kế chính.

#### ■ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao nén ảnh thành một vector *trước* khi sinh câu lại là một quyết định tồi, trong khi nén ảnh thành một vector trước khi phân loại thì hoàn toàn ổn?
2. Chứng minh bằng một câu rằng gộp trung bình toàn cục là trường hợp riêng của soft attention. Điều đó nói gì về quan hệ giữa hai kiến trúc?
3. Hard attention rẻ hơn khi chạy nhưng ít được dùng. Vì sao?
4. Vì sao không nên đọc bản đồ attention như một lời giải thích về nguyên nhân?
5. Trong một hệ thống bạn đã làm, nghẽn cổ chai kiểu “nén trước, dùng sau” nằm ở đâu, và hệ thống đó bù lại bằng bước “quay lại nguồn” nào?

- [1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [2] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [3] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *AISTATS*, 2010.
- [4] K. He, X. Zhang, S. Ren, et al., “Delving Deep into Rectifiers,” in *ICCV*, 2015.
- [5] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [6] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [7] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [8] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [9] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [10] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [11] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” in *ICLR*, 2017.
- [12] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [13] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [14] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [15] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [16] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [17] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [18] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [19] G. Huang, Y. Sun, Z. Liu, et al., “Deep Networks with Stochastic Depth,” in *ECCV*, 2016.
- [20] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [21] H. Zhang, M. Cisse, Y. N. Dauphin, et al., “mixup: Beyond Empirical Risk Minimization,” in *ICLR*, 2018.
- [22] S. Yun, D. Han, S. J. Oh, et al., “CutMix: Regularization Strategy to Train Strong Classifiers,” in *ICCV*, 2019.
- [23] E. D. Cubuk, B. Zoph, J. Shlens, et al., “RandAugment: Practical Automated Data Augmentation,” in *CVPR Workshops*, 2020.

- [24] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [25] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [26] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *NeurIPS*, 2012.
- [27] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in *ICLR*, 2015.
- [28] C. Szegedy, W. Liu, et al., “Going Deeper with Convolutions,” in *CVPR*, 2015.
- [29] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [30] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [31] G. Huang, Z. Liu, L. van der Maaten, et al., “Densely Connected Convolutional Networks,” in *CVPR*, 2017.
- [32] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [33] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [34] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [35] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [36] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [37] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.
- [38] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [39] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [40] J. Su, M. Ahmed, Y. Lu, et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, vol. 568, 2024.
- [41] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” *arXiv:2312.00752*, 2023.
- [42] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [43] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.
- [44] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [45] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [46] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [47] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [48] R. Girshick, J. Donahue, T. Darrell, et al., “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation,” in *CVPR*, 2014.

- [49] R. Girshick, "Fast R-CNN," in *ICCV*, 2015.
- [50] S. Ren, K. He, R. Girshick, et al., "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in *NeurIPS*, 2015.
- [51] T.-Y. Lin, P. Dollár, R. Girshick, et al., "Feature Pyramid Networks for Object Detection," in *CVPR*, 2017.
- [52] K. He, G. Gkioxari, P. Dollár, et al., "Mask R-CNN," in *ICCV*, 2017.
- [53] D. Bolya, S. Foley, J. Hays, et al., "TIDE: A General Toolbox for Identifying Object Detection Errors," in *ECCV*, 2020.
- [54] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [55] J. Redmon, S. Divvala, R. Girshick, et al., "You Only Look Once: Unified, Real-Time Object Detection," in *CVPR*, 2016.
- [56] W. Liu, D. Anguelov, D. Erhan, et al., "SSD: Single Shot MultiBox Detector," in *ECCV*, 2016.
- [57] T.-Y. Lin, P. Goyal, R. Girshick, et al., "Focal Loss for Dense Object Detection," in *ICCV*, 2017.
- [58] Z. Tian, C. Shen, H. Chen, et al., "FCOS: Fully Convolutional One-Stage Object Detection," in *ICCV*, 2019.
- [59] X. Zhou, D. Wang, and P. Krähenbühl, "Objects as Points," *arXiv:1904.07850*, 2019.
- [60] S. Zhang, C. Chi, Y. Yao, et al., "Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection," in *CVPR*, 2020.
- [61] Z. Ge, S. Liu, Z. Li, et al., "OTA: Optimal Transport Assignment for Object Detection," in *CVPR*, 2021.
- [62] N. Bodla, B. Singh, R. Chellappa, et al., "Soft-NMS: Improving Object Detection with One Line of Code," in *ICCV*, 2017.
- [63] X. Zhu, W. Su, L. Lu, et al., "Deformable DETR: Deformable Transformers for End-to-End Object Detection," in *ICLR*, 2021.
- [64] H. Zhang, F. Li, S. Liu, et al., "DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection," in *ICLR*, 2023.
- [65] J. Long, E. Shelhamer, and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," in *CVPR*, 2015.
- [66] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *MICCAI*, 2015.
- [67] L.-C. Chen, G. Papandreou, I. Kokkinos, et al., "DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs," *IEEE TPAMI*, vol. 40, no. 4, pp. 834–848, 2018.
- [68] E. Xie, W. Wang, Z. Yu, et al., "SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers," in *NeurIPS*, 2021.
- [69] B. Cheng, I. Misra, A. G. Schwing, et al., "Masked-Attention Mask Transformer for Universal Image Segmentation," in *CVPR*, 2022.
- [70] A. Kirillov, K. He, R. Girshick, et al., "Panoptic Segmentation," in *CVPR*, 2019.
- [71] A. Kirillov, E. Mintun, N. Ravi, et al., "Segment Anything," in *ICCV*, 2023.
- [72] K. Sun, B. Xiao, D. Liu, et al., "Deep High-Resolution Representation Learning for Human Pose Estimation," in *CVPR*, 2019.

- [73] Z. Cao, T. Simon, S.-E. Wei, et al., “Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields,” in *CVPR*, 2017.
- [74] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [75] J. Deng, J. Guo, N. Xue, et al., “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *CVPR*, 2019.
- [76] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [77] K. Xu, J. Ba, R. Kiros, et al., “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention,” in *ICML*, 2015.
- [78] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *ICLR*, 2015.
- [79] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [80] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [81] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [82] K. Simonyan and A. Zisserman, “Two-Stream Convolutional Networks for Action Recognition in Videos,” in *NeurIPS*, 2014.
- [83] J. Carreira and A. Zisserman, “Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset,” in *CVPR*, 2017.
- [84] D. Tran, H. Wang, L. Torresani, et al., “A Closer Look at Spatiotemporal Convolutions for Action Recognition,” in *CVPR*, 2018.
- [85] C. Feichtenhofer, H. Fan, J. Malik, et al., “SlowFast Networks for Video Recognition,” in *ICCV*, 2019.
- [86] A. Dosovitskiy, P. Fischer, E. Ilg, et al., “FlowNet: Learning Optical Flow with Convolutional Networks,” in *ICCV*, 2015.
- [87] D. Sun, X. Yang, M.-Y. Liu, et al., “PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume,” in *CVPR*, 2018.
- [88] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.
- [89] A. Bewley, Z. Ge, L. Ott, et al., “Simple Online and Realtime Tracking,” in *ICIP*, 2016.
- [90] Y. Zhang, P. Sun, Y. Jiang, et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *ECCV*, 2022.
- [91] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [92] K. Bernardin and R. Stiefelhagen, “Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics,” *EURASIP J. Image and Video Processing*, 2008.
- [93] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [94] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [95] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.

- [96] S. Wang, V. Leroy, Y. Cabon, et al., "DUST3R: Geometric 3D Vision Made Easy," in *CVPR*, 2024.
- [97] D. Eigen, C. Puhrsch, and R. Fergus, "Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network," in *NeurIPS*, 2014.
- [98] R. Ranftl, K. Lasinger, D. Hafner, et al., "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer," *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [99] J. Sturm, N. Engelhard, F. Endres, et al., "A Benchmark for the Evaluation of RGB-D SLAM Systems," in *IROS*, 2012.
- [100] C. R. Qi, H. Su, K. Mo, et al., "PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation," in *CVPR*, 2017.
- [101] C. R. Qi, L. Yi, H. Su, et al., "PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space," in *NeurIPS*, 2017.
- [102] A. H. Lang, S. Vora, H. Caesar, et al., "PointPillars: Fast Encoders for Object Detection from Point Clouds," in *CVPR*, 2019.
- [103] T. Müller, A. Evans, C. Schied, et al., "Instant Neural Graphics Primitives with a Multiresolution Hash Encoding," *ACM TOG*, vol. 41, no. 4, 2022.
- [104] B. Kerbl, G. Kopanas, T. Leimkühler, et al., "3D Gaussian Splatting for Real-Time Radiance Field Rendering," *ACM TOG*, vol. 42, no. 4, 2023.
- [105] R. Zhang, P. Isola, A. A. Efros, et al., "The Unreasonable Effectiveness of Deep Features as a Perceptual Metric," in *CVPR*, 2018.
- [106] D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," in *ICLR*, 2014.
- [107] I. Goodfellow, J. Pouget-Abadie, M. Mirza, et al., "Generative Adversarial Nets," in *NeurIPS*, 2014.
- [108] T. Karras, S. Laine, and T. Aila, "A Style-Based Generator Architecture for Generative Adversarial Networks," in *CVPR*, 2019.
- [109] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," in *NeurIPS*, 2020.
- [110] Y. Song, J. Sohl-Dickstein, D. P. Kingma, et al., "Score-Based Generative Modeling Through Stochastic Differential Equations," in *ICLR*, 2021.
- [111] J. Johnson, A. Alahi, and L. Fei-Fei, "Perceptual Losses for Real-Time Style Transfer and Super-Resolution," in *ECCV*, 2016.
- [112] P. Isola, J.-Y. Zhu, T. Zhou, et al., "Image-to-Image Translation with Conditional Adversarial Networks," in *CVPR*, 2017.
- [113] L. A. Gatys, A. S. Ecker, and M. Bethge, "Image Style Transfer Using Convolutional Neural Networks," in *CVPR*, 2016.
- [114] X. Huang and S. Belongie, "Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization," in *ICCV*, 2017.
- [115] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," in *NeurIPS Deep Learning Workshop*, 2015.
- [116] J. Song, C. Meng, and S. Ermon, "Denoising Diffusion Implicit Models," in *ICLR*, 2021.
- [117] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, et al., "Flow Matching for Generative Modeling," in *ICLR*, 2023.
- [118] J. Ho and T. Salimans, "Classifier-Free Diffusion Guidance," *arXiv:2207.12598*, 2022.
- [119] R. Rombach, A. Blattmann, D. Lorenz, et al., "High-Resolution Image Synthesis with Latent Diffusion Models," in *CVPR*, 2022.
- [120] W. Peebles and S. Xie, "Scalable Diffusion Models with Transformers," in *ICCV*, 2023.



- [121] L. Zhang, A. Rao, and M. Agrawala, “Adding Conditional Control to Text-to-Image Diffusion Models,” in *ICCV*, 2023.
- [122] Y. Blau and T. Michaeli, “The Perception-Distortion Tradeoff,” in *CVPR*, 2018.
- [123] C. Ledig, L. Theis, F. Huszár, et al., “Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network,” in *CVPR*, 2017.
- [124] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., “Image Quality Assessment: From Error Visibility to Structural Similarity,” *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [125] M. Heusel, H. Ramsauer, T. Unterthiner, et al., “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium,” in *NeurIPS*, 2017.
- [126] T. Brooks, B. Mildenhall, T. Xue, et al., “Unprocessing Images for Learned Raw Denoising,” in *CVPR*, 2019.
- [127] A. Abdelhamed, S. Lin, and M. S. Brown, “A High-Quality Denoising Dataset for Smartphone Cameras,” in *CVPR*, 2018.
- [128] J. Deng, W. Dong, R. Socher, et al., “ImageNet: A Large-Scale Hierarchical Image Database,” in *CVPR*, 2009.
- [129] A. Radford, J. W. Kim, C. Hallacy, et al., “Learning Transferable Visual Models from Natural Language Supervision,” in *ICML*, 2021.
- [130] J. Li, D. Li, S. Savarese, et al., “BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models,” in *ICML*, 2023.
- [131] H. Liu, C. Li, Q. Wu, et al., “Visual Instruction Tuning,” in *NeurIPS*, 2023.
- [132] S. Kornblith, J. Shlens, and Q. V. Le, “Do Better ImageNet Models Transfer Better?” In *CVPR*, 2019.
- [133] T. Chen, S. Kornblith, M. Norouzi, et al., “A Simple Framework for Contrastive Learning of Visual Representations,” in *ICML*, 2020.
- [134] K. He, H. Fan, Y. Wu, et al., “Momentum Contrast for Unsupervised Visual Representation Learning,” in *CVPR*, 2020.
- [135] J.-B. Grill, F. Strub, F. Altché, et al., “Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning,” in *NeurIPS*, 2020.
- [136] M. Caron, H. Touvron, I. Misra, et al., “Emerging Properties in Self-Supervised Vision Transformers,” in *ICCV*, 2021.
- [137] K. He, X. Chen, S. Xie, et al., “Masked Autoencoders Are Scalable Vision Learners,” in *CVPR*, 2022.
- [138] A. van den Oord, Y. Li, and O. Vinyals, “Representation Learning with Contrastive Predictive Coding,” in *arXiv:1807.03748*, 2018.
- [139] J. Kaplan, S. McCandlish, T. Henighan, et al., “Scaling Laws for Neural Language Models,” *arXiv:2001.08361*, 2020.
- [140] X. Zhai, A. Kolesnikov, N. Houlsby, et al., “Scaling Vision Transformers,” in *CVPR*, 2022.
- [141] J. Hoffmann, S. Borgeaud, A. Mensch, et al., “Training Compute-Optimal Large Language Models,” in *NeurIPS*, 2022.
- [142] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [143] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [144] N. Houlsby, A. Giurgiu, S. Jastrzebski, et al., “Parameter-Efficient Transfer Learning for NLP,” in *ICML*, 2019.

- [145] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [146] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *RecSys*, 2019.

## 4.2 Biểu diễn video và cái giá của thời gian (Video Representation and the Cost of Time)

**PHỤ THUỘC** C/13/01-dong-cnn (conv, số tham số, FLOPs); C/13/02-tu-chuoi-toi-attention (chi phí bậc hai của attention); B/11-gioi-han-tinh-toan (bảng thông bộ nhớ, ngân sách tính toán).

### TÓM TẮT

Mục này trả lời câu hỏi “thêm chiều thời gian vào ảnh thì phải trả giá gì”. Câu trả lời ngắn: bộ nhớ, và nó bị chia cho ba chiều cùng lúc — độ dài clip, độ phân giải, và *batch* — nên tăng một chiều buộc phải giảm chiều khác. Bốn họ kiến trúc (gộp theo khung, hai luồng, tích chập 3D và bản tách (2+1)D, video transformer) là bốn cách khác nhau để mua thông tin thời gian với ít bộ nhớ nhất có thể. Đọc xong bạn tính được ngân sách bộ nhớ của một cấu hình video *trước khi* chạy, và biết chiều nào nên hy sinh cho bài toán của mình. Nếu chỉ nhớ một điều: trước khi tin bất kỳ mô hình video nào, hãy chạy baseline “mỗi khung một lần rồi gộp”. Trên rất nhiều tập dữ liệu, baseline đó gần bằng mô hình chuyển động. Chênh lệch còn lại mới là thứ đáng trả tiền.

### KIẾN THỨC CẦN CÓ TRƯỚC

- **clip** — một đoạn video ngắn gồm  $T$  khung hình liên tiếp (hoặc lấy mẫu thưa), là đơn vị mà mô hình video nhận vào — tương đương “một ảnh” của mô hình ảnh.
- **kích hoạt (activation)** — tensor giá trị trung gian mà mỗi tầng mạng sinh ra — khác với *hàm kích hoạt*, là phép phi tuyến áp lên nó. Khi huấn luyện phải giữ các tensor này trong bộ nhớ để tính gradient ở **lượt ngược** (*backward pass*), nên chúng — chứ không phải tham số — thường là thứ làm tràn bộ nhớ.
- **batch size** — số mẫu xử lý cùng lúc trong một bước cập nhật. Batch lớn cho ước lượng gradient ổn định hơn nhưng tốn bộ nhớ tỉ lệ thuận.
- **batch normalization** — phép chuẩn hoá kích hoạt bằng trung bình và phương sai *tính trong batch*. Batch quá nhỏ thì hai thống kê này rất nhiễu, nên mô hình video hay phải đổi sang chuẩn hoá theo nhóm hoặc theo tầng.
- **FLOPs** — số phép tính dấu phẩy động cần cho một lần chạy; dùng để so chi phí tính toán độc lập với phần cứng. Nó không tỉ lệ chặt với thời gian chạy thật, vì bộ nhớ mới hay là nút thắt.
- **optical flow** — trường vector cho biết mỗi pixel dịch đi đâu giữa hai khung hình liên tiếp — biểu diễn tường minh của chuyển động, dùng làm đầu vào cho nhánh chuyển động của kiến trúc hai luồng.
- **token và chi phí bậc hai** — Transformer chia đầu vào thành  $n$  token rồi tính quan hệ giữa mọi cặp, nên chi phí tỉ lệ  $n^2$ . Với video,  $n$  nhân thêm số khung hình, và  $n^2$  nhân thêm bình phương số đó.
- **inductive bias** — giả định cài sẵn trong kiến trúc, ví dụ tích chập giả định tính cục bộ và **đẳng biến tịnh tiến** (*translation equivariance*). Bỏ bớt giả định thì mô hình linh hoạt hơn nhưng cần nhiều dữ liệu hơn để tự học lại điều đã bị bỏ.

### Đặt vấn đề

Cách hiển nhiên nhất để xử lý video là coi nó như một khối ba chiều và dùng **tích chập ba chiều** (*3D convolution*) thay cho tích chập hai chiều. Cách này đúng về mặt hình thức và sai về mặt kinh

tế: số phần tử phải giữ trong bộ nhớ nhân với số khung hình, số tham số của mỗi kernel nhân với chiều dài thời gian, và ngân sách vốn đã chặt của một mô hình ảnh lập tức vỡ. Vì thế toàn bộ lĩnh vực này không phải là câu chuyện “kiến trúc nào tốt hơn” mà là câu chuyện “mua thông tin thời gian bằng cách nào cho rẻ”.

Nhưng còn một lý do sâu hơn khiến ta không nên coi thời gian là trục không gian thứ ba. Trục thời gian có *hướng*: đảo ngược nó thì “mở cửa” thành “đóng cửa”, trong khi lật ngược trục  $x$  của một ảnh vẫn cho một ảnh hợp lệ của cùng một cảnh. Trục thời gian cũng có *đơn vị khác*: 16 khung hình ở 30 Hz là nửa giây, còn 16 pixel là một chi tiết nhỏ — không có lý do gì để cùng một kernel  $3 \times 3 \times 3$  là kích thước hợp lý ở cả ba trục. Hai bất đối xứng này là nguyên nhân sâu xa của mọi thiết kế tách trục mà ta sẽ gặp.

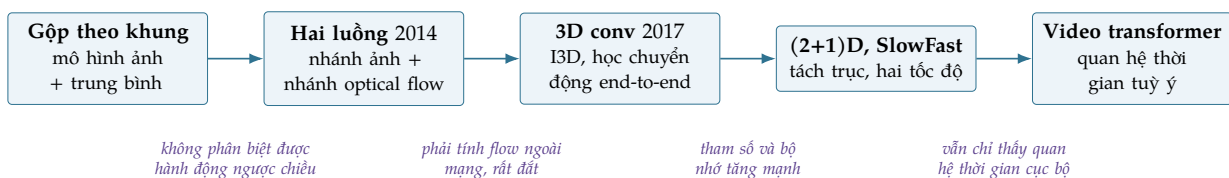
**Học xong mục này bạn làm được gì mà trước đó không làm được?** Cầm một yêu cầu như “phân loại hành động trên clip 4 giây, chạy được trên GPU 8 GB”, bạn suy ra ngay cấu hình khả thi và thứ phải hy sinh. Không phải thử ngẫu nhiên tới khi hết bộ nhớ.

#### TRỰC GIÁC

Ngân sách bộ nhớ của video giống một tấm chăn ngắn kéo theo ba hướng: độ dài clip, độ phân giải, và batch size. Tích của ba đại lượng đó gần như là một hằng số do phần cứng ấn định, nên mọi quyết định kiến trúc thực chất là quyết định kéo chăn về phía nào. Điều đáng nói là ba hướng ấy không tương đương về mặt khoa học: giảm batch làm hỏng thống kê của **chuẩn hoá theo batch** (*batch normalization*), giảm độ phân giải làm mất vật nhỏ, còn giảm độ dài clip làm mất chính thứ ta đang muốn học. Không có hướng nào miễn phí, và biết trước điều đó tiết kiệm hàng tuần thí nghiệm.

↔ **ràng buộc bởi**  $B/11 - \text{tích độ dài clip} \times \text{độ phân giải}^2 \times \text{batch}$  bị chặn bởi bộ nhớ thiết bị — cùng một trần đã chi phối mọi lựa chọn ở chương giới hạn tính toán.

#### A. Bốn họ biểu diễn, và nút thắt mà mỗi họ gỡ



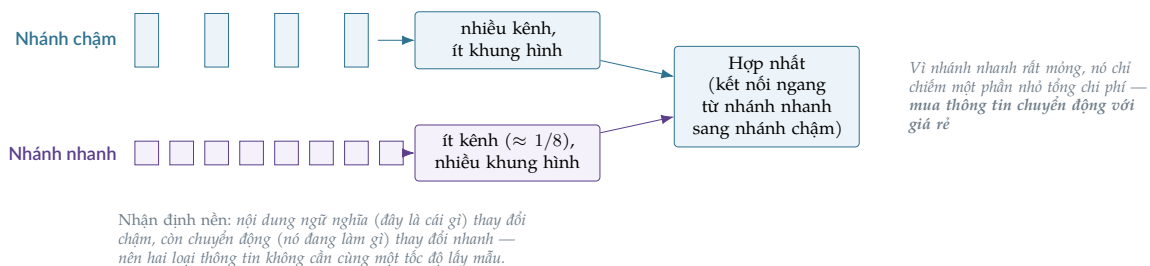
**Hình 4.5.** Dòng thời gian biểu diễn video, đọc theo nút thắt được gỡ. Nhân tím dưới mỗi mũi tên là *vấn đề của bước trước* — đó là thứ khiến bước sau ra đời. Chú ý rằng đây không phải một thang chất lượng: bước đầu tiên vẫn là baseline mạnh và bắt buộc phải chạy.

Hình 4.5 cho thấy mạch phát triển, còn Bảng 4.3 cho thấy cái mà mạch đó che đi: mỗi họ giả định một điều khác nhau về bản chất của chuyển động, và chính giả định — chứ không phải số tầng — quyết định nó hỏng ở đâu.

**Bảng 4.3.** Năm cách biểu diễn video. Cột chi phí ghi theo bậc độ lớn tương đối so với một mô hình ảnh chạy trên một khung hình, không phải số đo thực nghiệm.

| Cách              | Giả định                                           | Chi phí                        | Được                                                        | Hỏng khi nào                                                         |
|-------------------|----------------------------------------------------|--------------------------------|-------------------------------------------------------------|----------------------------------------------------------------------|
| Mỗi khung + gộp   | Thời gian không quan trọng lắm                     | $\sim T \times$                | Rẻ, tái dùng nguyên mô hình ảnh                             | Không phân biệt được hành động ngược chiều (mở/đóng, nhặt/đặt)       |
| Hai luồng         | Chuyển động tách được thành một nhánh riêng [82]   | $\sim 2T \times + \text{flow}$ | Rõ ràng, hiệu quả với ít dữ liệu                            | Phải tính optical flow ngoài mạng; flow sai thì nhánh đó thành nhiễu |
| 3D conv           | Không-thời gian đối xứng [83]                      | $\sim 3T \times$               | Học chuyển động đầu-cuối                                    | Tham số và bộ nhớ tăng mạnh; giả định đối xứng vốn sai               |
| (2+1)D            | Trục thời gian tách được khỏi trục không gian [84] | $\sim 1,5T \times$             | Cùng số tham số nhưng thêm một hàm kích hoạt, dễ tối ưu hơn | Chuyển động thực sự xoắn không-thời gian (xoáy, biến dạng nhanh)     |
| Video transformer | Quan hệ thời gian tùy ý, không cần cục bộ          | bậc hai theo số token          | Mạnh nhất khi đủ dữ liệu                                    | Chi phí bậc hai theo $T$ ; rất đói dữ liệu                           |

Một họ nữa nằm ngoài bảng vì nó là một ý tưởng chéo chứ không phải một loại tích chập: **Slow-Fast** chạy *hai* nhánh trên cùng video — một nhánh tốc độ khung thấp nhưng nhiều kênh để nắm ngữ nghĩa, một nhánh tốc độ khung cao nhưng ít kênh để nắm chuyển động [85]. Ý nghĩa của thiết kế này: *nội dung ngữ nghĩa thay đổi chậm, còn chuyển động thay đổi nhanh, nên hai loại thông tin không cần cùng một tốc độ lấy mẫu*. Nhánh nhanh rất mỏng: bài báo dùng khoảng một phần tám số kênh. Nó vì thế chỉ chiếm một phần nhỏ tổng chi phí. Đây là ví dụ đẹp của việc dùng cấu trúc bài toán để mua thông tin rẻ hơn.



**Hình 4.6.** SlowFast dùng cấu trúc của tín hiệu để mua thông tin rẻ hơn. Thay vì tăng đồng loạt tần số lấy mẫu cho toàn mạng — cách tốn kém nhất — nó tách thành hai nhánh có tần số và bề dày kênh ngược nhau, rồi hợp nhất. Nếu nhận định nền sai với dữ liệu của bạn (ngữ nghĩa thay đổi nhanh, chẳng hạn cảnh cắt dựng liên tục) thì lợi thế chi phí biến mất.

Hình 4.6 đáng đặt cạnh Hình 4.7: cả hai đều là ví dụ của cùng một chiến lược — đọc ra một bất đối xứng thật của bài toán rồi đưa nó vào kiến trúc, thay vì trả tiền đồng đều cho mọi chiều.

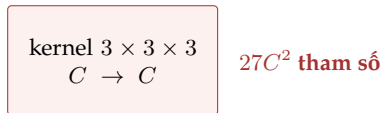
## B. Ba phép tính quyết định mọi thứ

- **Bộ nhớ của một tensor kích hoạt.** Một clip 16 khung,  $224 \times 224$ , 64 kênh, fp32:  $16 \times 224 \times 224 \times 64 \times 4 \text{ byte} \approx 205 \text{ MB}$  — cho *một* tensor, ở *batch* bằng 1. Lướt ngược phải giữ kích hoạt

của mọi tầng, nên con số này nhân với số tầng nông. Đây là lý do thực tế vì sao mô hình video hầu như luôn chạy ở batch nhỏ, và vì sao chuẩn hoá theo batch trở thành vấn đề (xem [C/12-co-che-huan-luyen/02-lam-cho-training-chay-duoc](#)).

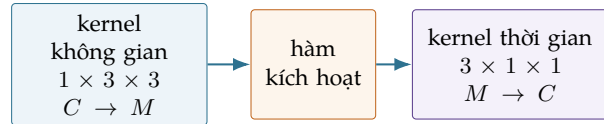
- **Vì sao (2+1)D không tốn thêm tham số.** Một kernel 3D  $3 \times 3 \times 3$  từ  $C$  kênh sang  $C$  kênh có  $27C^2$  tham số. Bản tách gồm hai kernel. Kernel không gian  $1 \times 3 \times 3$  đi từ  $C$  sang  $M$  kênh, tốn  $9CM$ ; kernel thời gian  $3 \times 1 \times 1$  đi từ  $M$  về  $C$ , tốn  $3MC$ . Tổng là  $12CM$ . Cho hai vế bằng nhau:  $M = 2,25C$ . *Ý nghĩa:* với cùng ngân sách tham số, ta mua được một hàm kích hoạt bổ sung ở giữa và một bài toán tối ưu dễ hơn — lợi ích đến từ dạng của hàm, không từ sức chứa (capacity) [84].

Tích chập 3D



Một khối duy nhất trộn cả ba trục, ngâm giữ định thời gian đối xứng với không gian

Bản tách (2+1)D



tổng  $12CM$ ; cho bằng  $27C^2$  thì  $M = 2,25C$

Cùng ngân sách tham số, nhưng mua thêm được một hàm kích hoạt ở giữa — lợi ích đến từ dạng của hàm, không từ sức chứa

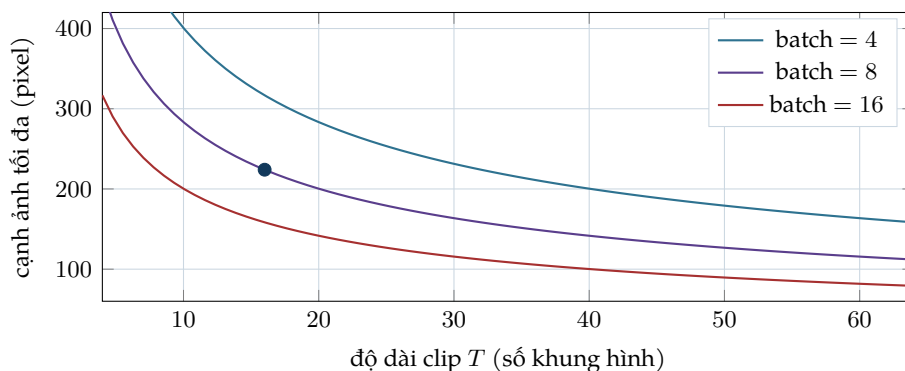
**Hình 4.7.** Phân rã (2+1)D vẽ theo khối. Kernel 3D duy nhất được tách thành một kernel chỉ tác động theo không gian rồi một kernel chỉ tác động theo thời gian, với số kênh trung gian  $M$  chọn sao cho tổng tham số bằng bản gốc. Phép tách phản ánh đúng bất đối xứng của bài toán: trục thời gian có hướng và có đơn vị khác, nên không có lý do để cùng một kernel đối xứng là hợp lý ở cả ba trục.

Hình 4.7 làm rõ vì sao đây không phải mẹo tiết kiệm: nếu chỉ để tiết kiệm thì người ta đã giảm số kênh, chứ không giữ nguyên tham số rồi chèn thêm một phi tuyến.

- **Vì sao attention trên video đắt tới mức đó.** Một ViT chia ảnh  $224 \times 224$  thành  $14 \times 14 = 196$  token, nên ma trận attention có  $196^2 \approx 3,8 \times 10^4$  phần tử. Với clip 16 khung, số token thành 3136 và ma trận thành  $9,8 \times 10^6$  — gấp 256 lần, đúng bằng  $16^2$ .

↪ **ràng buộc bởi C/13/03** — video transformer bỏ thêm một tầng thiên lệch quy nạp nữa — lần này theo trục thời gian — nên nó đòi dữ liệu hơn cả ViT trên ảnh.

*Ý nghĩa:* chi phí bậc hai theo số token nghĩa là bậc hai theo độ dài clip, nên mọi video transformer thực dụng đều phải phân tách attention (theo thời gian rồi theo không gian) hoặc giới hạn cửa sổ.



**Hình 4.8.** Đường đẳng ngân sách bộ nhớ. Vì lượng kích hoạt tỉ lệ với  $T \cdot R^2 \cdot B$ , một ngân sách cố định cho  $R \propto 1/\sqrt{TB}$ . Điểm xanh đậm là cấu hình tham chiếu (16 khung, 224 px, batch 8). Hình cho thấy điều mà một bảng số không cho thấy: muốn gấp đôi độ dài clip, bạn chỉ phải giảm cạnh ảnh khoảng 30%, nhưng muốn gấp bốn thì phải giảm một nửa — và đường cong dốc rất nhanh ở phía trái.

Hình 4.8 nên dùng như một công cụ chứ không phải một minh họa: khi ai đó đề nghị “tăng độ dài clip lên 64 khung”, hãy chỉ ngay vào đường cong và hỏi họ định trả bằng độ phân giải hay bằng batch.

### C. Giới hạn và trường hợp hỏng

- **Rất nhiều tập dữ liệu không thực sự cần thời gian.** Nhiều tập dữ liệu hành động phổ biến có thể đoán đúng phần lớn chỉ từ *bối cảnh* của một khung hình — thấy bể bơi thì đoán “bơi”. Đây là quan sát được lặp lại rộng rãi trong lĩnh vực, và hệ quả là một mô hình thời gian tinh vi có thể trông tốt vì lý do hoàn toàn khác với lý do bạn nghĩ.
- **Optical flow là một phụ thuộc ngoài mạng.** Nhánh chuyển động của kiến trúc hai luồng đòi hỏi flow được tính trước, thường tốn hơn cả mô hình chính. Chất lượng flow khi đó thành trần cho toàn hệ thống — đúng mẫu “đẩy độ khó sang chỗ khác”.
- **Batch nhỏ phá thống kê chuẩn hoá.**  
 ⇔ hệ quả của C/12/02 — chuẩn hoá theo batch ước lượng trung bình và phương sai từ chính batch, nên batch nhỏ làm ước lượng nhiễu — đúng cơ chế đã bàn ở phần làm cho huấn luyện chạy được.  
 Mô hình video thường chạy batch 2–8 trên mỗi GPU, nên batch norm cho ước lượng phương sai rất nhiễu; đây là lý do các kiến trúc video hay dùng chuẩn hoá theo nhóm hoặc theo tầng [7].
- **Lấy mẫu khung là một siêu tham số ẩn.** Lấy 16 khung liên tiếp, hay 16 khung rải đều trên 4 giây, là hai bài toán khác nhau; và cách lấy mẫu lúc đánh giá thường khác lúc huấn luyện, nên số công bố phụ thuộc vào một chi tiết ít khi được nêu rõ.

#### CẠM BẮY

Trước khi so sánh bất kỳ hai kiến trúc video nào, hãy chạy baseline rẻ nhất: lấy một khung hình ngẫu nhiên, chạy mô hình ảnh, rồi trung bình dự đoán trên vài khung. Nếu baseline này đạt gần bằng mô hình thời gian, thì tập dữ liệu của bạn không đo cái bạn tưởng nó đo, và mọi so sánh giữa các mô hình thời gian phía trên đều là nhiễu. Đây là bài kiểm tra rẻ nhất và bị bỏ qua nhiều nhất trong cả lĩnh vực.

#### CẦN NHỚ

Ngân sách bộ nhớ của video chia cho ba chiều nhân với nhau: độ dài clip  $\times$  độ phân giải<sup>2</sup>  $\times$  batch. Mọi lựa chọn kiến trúc vì thế là lựa chọn hy sinh chiều nào. Và trước khi trả tiền cho chiều thời gian, hãy chứng minh rằng bài toán của bạn thật sự cần nó, bằng baseline một khung hình. [T2]

### D. Kết nối

B/11-gioi-han-tinh-toan là nơi ngân sách bộ nhớ và băng thông được xử lý tổng quát; mục này chỉ là trường hợp riêng nhưng khác nghiệt nhất. C/13-kien-truc-backbone/03-vit-va-inductive-bias giải thích vì sao video transformer đòi dữ liệu: bỏ inductive bias theo trục thời gian còn tốn dữ liệu hơn bỏ theo trục không gian. C/15-thi-giac-khong-thoi-gian/03-flow-tracking-va-rang-buoc tiếp nối trực tiếp bằng nhánh optical flow — thứ mà kiến trúc hai luồng dựa vào — và bằng ràng buộc nhân quả, thứ loại bỏ phần lớn các kiến trúc trong bảng trên khi bạn phải chạy streaming.

❏ **Câu hỏi tự kiểm** (trả lời không nhìn bài)



1. Vì sao coi thời gian như trục không gian thứ ba là một giả định sai, và sai đó biểu hiện thành lỗi gì trong thực tế?
2. Tính lượng bộ nhớ cho một tensor kích hoạt clip 32 khung,  $112 \times 112$ , 128 kênh, fp16. Nếu vượt ngân sách, bạn cắt chiều nào trước và vì sao?
3. (2+1)D có cùng số tham số với 3D conv nhưng thường tốt hơn. Lợi ích đó đến từ đâu, nếu không phải từ sức chứa của mô hình?
4. Vì sao chi phí attention trên clip 16 khung gấp 256 lần trên một ảnh, chứ không phải 16 lần?
5. SlowFast dựa trên nhận định nào về bản chất của tín hiệu? Nếu nhận định đó sai với dữ liệu của bạn thì kiến trúc hồng thể nào?
6. Baseline “một khung hình rồi gộp” đạt 92% so với mô hình 3D đạt 94%. Bạn kết luận gì, và bước tiếp theo là gì?

- [1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [2] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [3] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *AISTATS*, 2010.
- [4] K. He, X. Zhang, S. Ren, et al., “Delving Deep into Rectifiers,” in *ICCV*, 2015.
- [5] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [6] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [7] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [8] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [9] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [10] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [11] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” in *ICLR*, 2017.
- [12] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [13] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [14] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [15] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [16] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [17] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [18] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [19] G. Huang, Y. Sun, Z. Liu, et al., “Deep Networks with Stochastic Depth,” in *ECCV*, 2016.
- [20] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [21] H. Zhang, M. Cisse, Y. N. Dauphin, et al., “mixup: Beyond Empirical Risk Minimization,” in *ICLR*, 2018.
- [22] S. Yun, D. Han, S. J. Oh, et al., “CutMix: Regularization Strategy to Train Strong Classifiers,” in *ICCV*, 2019.
- [23] E. D. Cubuk, B. Zoph, J. Shlens, et al., “RandAugment: Practical Automated Data Augmentation,” in *CVPR Workshops*, 2020.

- [24] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [25] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [26] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *NeurIPS*, 2012.
- [27] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in *ICLR*, 2015.
- [28] C. Szegedy, W. Liu, et al., “Going Deeper with Convolutions,” in *CVPR*, 2015.
- [29] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [30] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [31] G. Huang, Z. Liu, L. van der Maaten, et al., “Densely Connected Convolutional Networks,” in *CVPR*, 2017.
- [32] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [33] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [34] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [35] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [36] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [37] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.
- [38] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [39] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [40] J. Su, M. Ahmed, Y. Lu, et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, vol. 568, 2024.
- [41] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” *arXiv:2312.00752*, 2023.
- [42] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [43] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.
- [44] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [45] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [46] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [47] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [48] R. Girshick, J. Donahue, T. Darrell, et al., “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation,” in *CVPR*, 2014.

- [49] R. Girshick, "Fast R-CNN," in *ICCV*, 2015.
- [50] S. Ren, K. He, R. Girshick, et al., "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in *NeurIPS*, 2015.
- [51] T.-Y. Lin, P. Dollár, R. Girshick, et al., "Feature Pyramid Networks for Object Detection," in *CVPR*, 2017.
- [52] K. He, G. Gkioxari, P. Dollár, et al., "Mask R-CNN," in *ICCV*, 2017.
- [53] D. Bolya, S. Foley, J. Hays, et al., "TIDE: A General Toolbox for Identifying Object Detection Errors," in *ECCV*, 2020.
- [54] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [55] J. Redmon, S. Divvala, R. Girshick, et al., "You Only Look Once: Unified, Real-Time Object Detection," in *CVPR*, 2016.
- [56] W. Liu, D. Anguelov, D. Erhan, et al., "SSD: Single Shot MultiBox Detector," in *ECCV*, 2016.
- [57] T.-Y. Lin, P. Goyal, R. Girshick, et al., "Focal Loss for Dense Object Detection," in *ICCV*, 2017.
- [58] Z. Tian, C. Shen, H. Chen, et al., "FCOS: Fully Convolutional One-Stage Object Detection," in *ICCV*, 2019.
- [59] X. Zhou, D. Wang, and P. Krähenbühl, "Objects as Points," *arXiv:1904.07850*, 2019.
- [60] S. Zhang, C. Chi, Y. Yao, et al., "Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection," in *CVPR*, 2020.
- [61] Z. Ge, S. Liu, Z. Li, et al., "OTA: Optimal Transport Assignment for Object Detection," in *CVPR*, 2021.
- [62] N. Bodla, B. Singh, R. Chellappa, et al., "Soft-NMS: Improving Object Detection with One Line of Code," in *ICCV*, 2017.
- [63] X. Zhu, W. Su, L. Lu, et al., "Deformable DETR: Deformable Transformers for End-to-End Object Detection," in *ICLR*, 2021.
- [64] H. Zhang, F. Li, S. Liu, et al., "DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection," in *ICLR*, 2023.
- [65] J. Long, E. Shelhamer, and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," in *CVPR*, 2015.
- [66] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *MICCAI*, 2015.
- [67] L.-C. Chen, G. Papandreou, I. Kokkinos, et al., "DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs," *IEEE TPAMI*, vol. 40, no. 4, pp. 834–848, 2018.
- [68] E. Xie, W. Wang, Z. Yu, et al., "SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers," in *NeurIPS*, 2021.
- [69] B. Cheng, I. Misra, A. G. Schwing, et al., "Masked-Attention Mask Transformer for Universal Image Segmentation," in *CVPR*, 2022.
- [70] A. Kirillov, K. He, R. Girshick, et al., "Panoptic Segmentation," in *CVPR*, 2019.
- [71] A. Kirillov, E. Mintun, N. Ravi, et al., "Segment Anything," in *ICCV*, 2023.
- [72] K. Sun, B. Xiao, D. Liu, et al., "Deep High-Resolution Representation Learning for Human Pose Estimation," in *CVPR*, 2019.

- [73] Z. Cao, T. Simon, S.-E. Wei, et al., “Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields,” in *CVPR*, 2017.
- [74] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [75] J. Deng, J. Guo, N. Xue, et al., “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *CVPR*, 2019.
- [76] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [77] K. Xu, J. Ba, R. Kiros, et al., “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention,” in *ICML*, 2015.
- [78] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *ICLR*, 2015.
- [79] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [80] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [81] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [82] K. Simonyan and A. Zisserman, “Two-Stream Convolutional Networks for Action Recognition in Videos,” in *NeurIPS*, 2014.
- [83] J. Carreira and A. Zisserman, “Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset,” in *CVPR*, 2017.
- [84] D. Tran, H. Wang, L. Torresani, et al., “A Closer Look at Spatiotemporal Convolutions for Action Recognition,” in *CVPR*, 2018.
- [85] C. Feichtenhofer, H. Fan, J. Malik, et al., “SlowFast Networks for Video Recognition,” in *ICCV*, 2019.
- [86] A. Dosovitskiy, P. Fischer, E. Ilg, et al., “FlowNet: Learning Optical Flow with Convolutional Networks,” in *ICCV*, 2015.
- [87] D. Sun, X. Yang, M.-Y. Liu, et al., “PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume,” in *CVPR*, 2018.
- [88] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.
- [89] A. Bewley, Z. Ge, L. Ott, et al., “Simple Online and Realtime Tracking,” in *ICIP*, 2016.
- [90] Y. Zhang, P. Sun, Y. Jiang, et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *ECCV*, 2022.
- [91] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [92] K. Bernardin and R. Stiefelhagen, “Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics,” *EURASIP J. Image and Video Processing*, 2008.
- [93] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [94] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [95] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.

- [96] S. Wang, V. Leroy, Y. Cabon, et al., "DUST3R: Geometric 3D Vision Made Easy," in *CVPR*, 2024.
- [97] D. Eigen, C. Puhrsch, and R. Fergus, "Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network," in *NeurIPS*, 2014.
- [98] R. Ranftl, K. Lasinger, D. Hafner, et al., "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer," *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [99] J. Sturm, N. Engelhard, F. Endres, et al., "A Benchmark for the Evaluation of RGB-D SLAM Systems," in *IROS*, 2012.
- [100] C. R. Qi, H. Su, K. Mo, et al., "PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation," in *CVPR*, 2017.
- [101] C. R. Qi, L. Yi, H. Su, et al., "PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space," in *NeurIPS*, 2017.
- [102] A. H. Lang, S. Vora, H. Caesar, et al., "PointPillars: Fast Encoders for Object Detection from Point Clouds," in *CVPR*, 2019.
- [103] T. Müller, A. Evans, C. Schied, et al., "Instant Neural Graphics Primitives with a Multiresolution Hash Encoding," *ACM TOG*, vol. 41, no. 4, 2022.
- [104] B. Kerbl, G. Kopanas, T. Leimkühler, et al., "3D Gaussian Splatting for Real-Time Radiance Field Rendering," *ACM TOG*, vol. 42, no. 4, 2023.
- [105] R. Zhang, P. Isola, A. A. Efros, et al., "The Unreasonable Effectiveness of Deep Features as a Perceptual Metric," in *CVPR*, 2018.
- [106] D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," in *ICLR*, 2014.
- [107] I. Goodfellow, J. Pouget-Abadie, M. Mirza, et al., "Generative Adversarial Nets," in *NeurIPS*, 2014.
- [108] T. Karras, S. Laine, and T. Aila, "A Style-Based Generator Architecture for Generative Adversarial Networks," in *CVPR*, 2019.
- [109] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," in *NeurIPS*, 2020.
- [110] Y. Song, J. Sohl-Dickstein, D. P. Kingma, et al., "Score-Based Generative Modeling Through Stochastic Differential Equations," in *ICLR*, 2021.
- [111] J. Johnson, A. Alahi, and L. Fei-Fei, "Perceptual Losses for Real-Time Style Transfer and Super-Resolution," in *ECCV*, 2016.
- [112] P. Isola, J.-Y. Zhu, T. Zhou, et al., "Image-to-Image Translation with Conditional Adversarial Networks," in *CVPR*, 2017.
- [113] L. A. Gatys, A. S. Ecker, and M. Bethge, "Image Style Transfer Using Convolutional Neural Networks," in *CVPR*, 2016.
- [114] X. Huang and S. Belongie, "Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization," in *ICCV*, 2017.
- [115] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," in *NeurIPS Deep Learning Workshop*, 2015.
- [116] J. Song, C. Meng, and S. Ermon, "Denoising Diffusion Implicit Models," in *ICLR*, 2021.
- [117] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, et al., "Flow Matching for Generative Modeling," in *ICLR*, 2023.
- [118] J. Ho and T. Salimans, "Classifier-Free Diffusion Guidance," *arXiv:2207.12598*, 2022.
- [119] R. Rombach, A. Blattmann, D. Lorenz, et al., "High-Resolution Image Synthesis with Latent Diffusion Models," in *CVPR*, 2022.
- [120] W. Peebles and S. Xie, "Scalable Diffusion Models with Transformers," in *ICCV*, 2023.



- [121] L. Zhang, A. Rao, and M. Agrawala, "Adding Conditional Control to Text-to-Image Diffusion Models," in *ICCV*, 2023.
- [122] Y. Blau and T. Michaeli, "The Perception-Distortion Tradeoff," in *CVPR*, 2018.
- [123] C. Ledig, L. Theis, F. Huszár, et al., "Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network," in *CVPR*, 2017.
- [124] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., "Image Quality Assessment: From Error Visibility to Structural Similarity," *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [125] M. Heusel, H. Ramsauer, T. Unterthiner, et al., "GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium," in *NeurIPS*, 2017.
- [126] T. Brooks, B. Mildenhall, T. Xue, et al., "Unprocessing Images for Learned Raw Denoising," in *CVPR*, 2019.
- [127] A. Abdelhamed, S. Lin, and M. S. Brown, "A High-Quality Denoising Dataset for Smartphone Cameras," in *CVPR*, 2018.
- [128] J. Deng, W. Dong, R. Socher, et al., "ImageNet: A Large-Scale Hierarchical Image Database," in *CVPR*, 2009.
- [129] A. Radford, J. W. Kim, C. Hallacy, et al., "Learning Transferable Visual Models from Natural Language Supervision," in *ICML*, 2021.
- [130] J. Li, D. Li, S. Savarese, et al., "BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models," in *ICML*, 2023.
- [131] H. Liu, C. Li, Q. Wu, et al., "Visual Instruction Tuning," in *NeurIPS*, 2023.
- [132] S. Kornblith, J. Shlens, and Q. V. Le, "Do Better ImageNet Models Transfer Better?" In *CVPR*, 2019.
- [133] T. Chen, S. Kornblith, M. Norouzi, et al., "A Simple Framework for Contrastive Learning of Visual Representations," in *ICML*, 2020.
- [134] K. He, H. Fan, Y. Wu, et al., "Momentum Contrast for Unsupervised Visual Representation Learning," in *CVPR*, 2020.
- [135] J.-B. Grill, F. Strub, F. Altché, et al., "Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning," in *NeurIPS*, 2020.
- [136] M. Caron, H. Touvron, I. Misra, et al., "Emerging Properties in Self-Supervised Vision Transformers," in *ICCV*, 2021.
- [137] K. He, X. Chen, S. Xie, et al., "Masked Autoencoders Are Scalable Vision Learners," in *CVPR*, 2022.
- [138] A. van den Oord, Y. Li, and O. Vinyals, "Representation Learning with Contrastive Predictive Coding," in *arXiv:1807.03748*, 2018.
- [139] J. Kaplan, S. McCandlish, T. Henighan, et al., "Scaling Laws for Neural Language Models," *arXiv:2001.08361*, 2020.
- [140] X. Zhai, A. Kolesnikov, N. Houlsby, et al., "Scaling Vision Transformers," in *CVPR*, 2022.
- [141] J. Hoffmann, S. Borgeaud, A. Mensch, et al., "Training Compute-Optimal Large Language Models," in *NeurIPS*, 2022.
- [142] C. G. Northcutt, A. Athalye, and J. Mueller, "Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks," in *NeurIPS Datasets and Benchmarks*, 2021.
- [143] B. Recht, R. Roelofs, L. Schmidt, et al., "Do ImageNet Classifiers Generalize to ImageNet?" In *ICML*, 2019.
- [144] N. Houlsby, A. Giurgiu, S. Jastrzebski, et al., "Parameter-Efficient Transfer Learning for NLP," in *ICML*, 2019.

- [145] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [146] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *RecSys*, 2019.

### 4.3 Flow, tracking và các ràng buộc riêng (Optical Flow, Tracking and Their Constraints)

**PHỤ THUỘC** C/15-thi-giac-khong-thoi-gian/02-bieu-dien-video (ngân sách bộ nhớ); B/07-chia-du-lieu-va-danh-gia (rò rỉ dữ liệu, chọn metric); A/01-toan-toi-thieu (bình phương tối thiểu, điều kiện của ma trận).

#### TÓM TẮT

Mục này bàn ba thứ mà một kỹ sư SLAM đã sống cùng hằng ngày, chỉ dưới tên khác: **dòng quang** (*optical flow*) là bài toán tương ứng điểm, **bám nhiều mục tiêu** (*multi-object tracking*) là bài toán liên kết dữ liệu, và ràng buộc nhân quả là ràng buộc cứng của mọi hệ chạy thời gian thực. Đọc xong bạn ánh xạ được các kỹ thuật học sâu ở đây về đúng chỗ tương ứng trong pipeline hình học của mình. Bạn chọn được metric tracking không đánh lừa. Và bạn biết kiến trúc nào bị loại ngay từ đầu khi không được nhìn khung hình tương lai. Nếu chỉ nhớ một điều: trong cả flow lẫn tracking, chỗ hệ thống sống hay chết là bước *liên kết* chứ không phải bước phát hiện — và cả hai lĩnh vực đều có một metric phổ biến che giấu đúng chỗ đó.

#### KIẾN THỨC CẦN CÓ TRƯỚC

- **optical flow** — trường vector  $(u, v)$  cho biết mỗi pixel của khung này nằm ở đâu trong khung kia. “Dày đặc” nghĩa là tính cho mọi pixel, khác với việc chỉ bám vài trăm đặc trưng.
- **gradient ảnh**  $I_x, I_y, I_t$  — tốc độ thay đổi độ sáng theo trục ngang, trục dọc và theo thời gian tại một pixel. Ba đại lượng này là toàn tập dữ liệu mà phương trình flow cổ điển có.
- **bài toán khẩu độ (aperture problem)** — khi chỉ nhìn qua một cửa sổ nhỏ, chỉ thành phần chuyển động vuông góc với cạnh là quan sát được; thành phần dọc theo cạnh hoàn toàn vô hình. Đây là thiếu thông tin thật, không phải lỗi thuật toán.
- **liên kết dữ liệu (data association)** — bước quyết định phép đo nào ở khung này ứng với đối tượng nào đã biết. Trong tracking đó là ghép hộp với track; trong SLAM đó là ghép đặc trưng với điểm bản đồ. Gán sai gây hậu quả nặng và khó rút lại.
- **track ID** — mã số mà hệ thống gán cho một đối tượng và phải giữ nguyên suốt thời gian nó xuất hiện. *ID switch* là khi mã số bị đánh tráo giữa hai đối tượng — lỗi liên kết điển hình.
- **bộ lọc Kalman** — bộ ước lượng đệ quy giữ vị trí và vận tốc cùng độ bất định của chúng; nó dự đoán vị trí khung tới rồi cập nhật khi có phép đo mới. Trong tracking, nó cung cấp dự đoán để ghép cặp.
- **thuật toán Hungary** — thuật toán giải bài toán gán tối ưu: cho ma trận chi phí giữa  $m$  dự đoán và  $n$  phát hiện, tìm phép ghép một-một có tổng chi phí nhỏ nhất.
- **MOTA và HOTA** — hai thước đo tracking. MOTA cộng số phát hiện sót, phát hiện thừa và số lần đổi mã rồi chia cho tổng số vật thật. HOTA tách riêng chất lượng phát hiện (DetA) và chất lượng liên kết (AssA) rồi lấy trung bình nhân của chúng.
- **nhân quả / streaming** — ràng buộc rằng quyết định tại thời điểm  $t$  chỉ được dùng dữ liệu tới  $t$ , không được dùng khung hình tương lai. Nó loại bỏ mọi kiến trúc hai chiều và mọi phép làm mượt nhìn về phía trước.

## Đặt vấn đề

Hai mục trước xử lý video như một khối cần phân loại. Nhưng phần lớn ứng dụng thật lại cần thứ tinh hơn nhiều: *cùng một điểm vật lý ở khung này nằm ở đâu trong khung kia*. Trả lời câu đó cho mọi pixel là optical flow; trả lời cho từng vật thể qua thời gian là tracking; và trong cả hai trường hợp, khó khăn thật không nằm ở việc nhìn thấy mà ở việc *ghép cặp*. Đây chính xác là cấu trúc của bài toán mà một hệ SLAM giải mỗi khung hình, nên mục này ít giới thiệu khái niệm mới hơn là dịch từ vựng.

Thêm vào đó là một ràng buộc mà ảnh tĩnh không có: mô hình chạy trực tuyến không được nhìn khung hình tương lai, và cũng không được nhìn quá lâu vào quá khứ nếu ngân sách mili-giây có hạn. Ràng buộc này loại bỏ nguyên một lớp kiến trúc trước khi độ chính xác kịp lên tiếng — đúng nghĩa **ràng buộc cứng** ở B/03-giai-phau-he-thong.

**Học xong mục này bạn làm được gì mà trước đó không làm được?** Bạn đọc một paper flow hay tracking và biết ngay nó thay khối nào trong pipeline hình học quen thuộc, nó mua độ bền bằng giả định gì, và nó có chạy được dưới ràng buộc nhân quả của bạn hay không.

### TRỰC GIÁC

Nhìn một cạnh thẳng đang chuyển động qua một cái ống hút: bạn chỉ thấy được thành phần chuyển động *vuông góc* với cạnh, còn thành phần dọc theo cạnh thì hoàn toàn vô hình. Đó không phải hạn chế của mắt hay của thuật toán, mà là thiếu thông tin thật sự: một phương trình, hai ẩn. Mọi thứ trong optical flow — lấy cửa sổ, làm trơn, kim tự tháp, khối tương quan — đều là các cách khác nhau để mượn thêm ràng buộc từ nơi khác. Và ma trận đo mức độ “mượn được hay không” chính là ma trận mà bạn đã dùng để chọn đặc trưng góc trong SLAM.

## A. Optical flow: từ Lucas–Kanade tới RAFT

Giả định gốc là **độ sáng không đổi** (*brightness constancy*): cùng một điểm vật lý giữ nguyên giá trị sáng giữa hai khung. Viết  $I(x, y, t) = I(x + u, y + v, t + 1)$  rồi khai triển Taylor bậc nhất cho phương trình ràng buộc

$$I_x u + I_y v + I_t = 0.$$

Câu này nói gì bằng lời: *tốc độ thay đổi độ sáng theo thời gian tại một pixel phải bằng đúng phần độ sáng bị “trôi vào” do chuyển động*. Một phương trình, hai ẩn — đó là **bài toán khẩu độ** (*aperture problem*) ở dạng đại số.

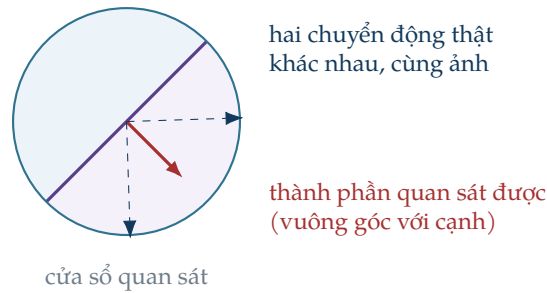
Lucas–Kanade mượn ràng buộc bằng cách giả định flow không đổi trong một cửa sổ nhỏ, cho  $N$  phương trình và hai ẩn, rồi giải bình phương tối thiểu. Phương trình chuẩn có ma trận hệ số

$$A^T A = \begin{bmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{bmatrix},$$

và đây đúng là ma trận cấu trúc trong tiêu chuẩn chọn đặc trưng góc. Ý nghĩa: bài toán giải được khi và chỉ khi cửa sổ có gradient theo *hai* hướng độc lập. Ví dụ nhỏ nhất: một cửa sổ nằm trên một cạnh dọc lý tưởng có  $I_x = g$  và  $I_y = 0$  ở mọi pixel, nên  $A^T A = \begin{bmatrix} Ng^2 & 0 \\ 0 & 0 \end{bmatrix}$  — suy biến, giá trị riêng nhỏ nhất bằng 0, và thành phần flow dọc theo cạnh không xác định.

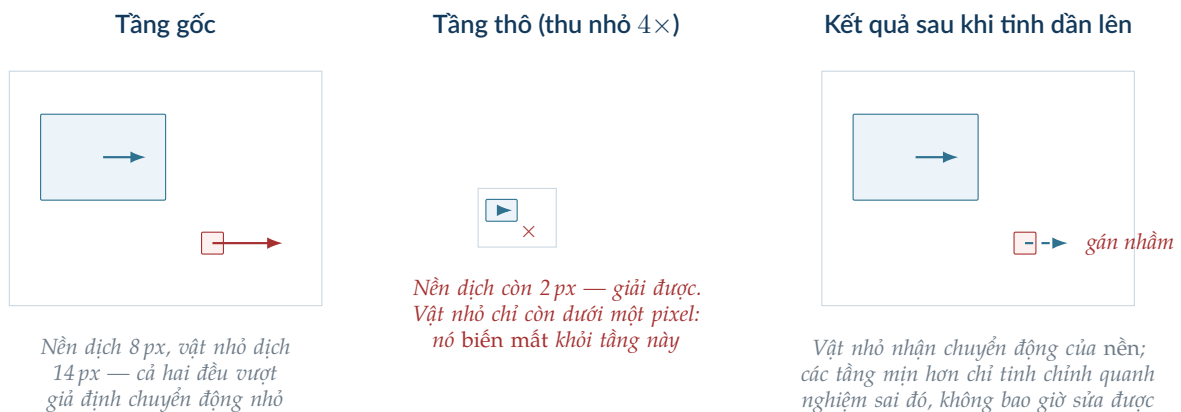
↪ **tiền đề** A/01 — đây đúng là câu hỏi điều kiện của một hệ bình phương tối thiểu: nghiệm ổn định khi giá trị riêng nhỏ nhất đủ xa 0, và suy biến khi nó bằng 0.

Chọn “good features to track” chính là chọn các cửa sổ mà giá trị riêng nhỏ nhất của ma trận này đủ lớn; nói cách khác, việc chọn đặc trưng trong SLAM và tính giải được của flow là *cùng một điều kiện*.



**Hình 4.9.** Bài toán khẩu độ. Hai chuyển động thật rất khác nhau (mũi tên đứt) tạo ra cùng một hình ảnh bên trong cửa sổ, vì chỉ thành phần vuông góc với cạnh là quan sát được. Về đại số, đó là việc ma trận  $A^T A$  suy biến; về thực hành, đó là lý do mọi bộ bám đặc trưng đều phải chọn góc chứ không chọn cạnh.

Hình 4.9 cũng giải thích luôn giả định thứ ba của Lucas–Kanade — *chuyển động nhỏ* — vì khai triển Taylor bậc nhất chỉ đúng khi dịch chuyển nhỏ hơn cỡ một pixel gradient. Cách chữa cổ điển là kim tự tháp thô–tối–mịn: giảm ảnh xuống bốn lần thì chuyển động 8 pixel thành 2 pixel. Cái giá là một chế độ hỏng đặc trưng: vật *nhỏ* và *nhạy* biến mất ở tầng thô, nên hệ thống ước lượng chuyển động của nền thay vì của vật, và sai lầm đó không bao giờ được sửa ở các tầng mịn hơn.



**Hình 4.10.** Chế độ hỏng đặc trưng của chiến lược thô–tối–mịn. Thu nhỏ ảnh làm chuyển động lớn trở nên nhỏ — đó là mục đích — nhưng nó cũng làm vật nhỏ biến mất trước khi kịp được ước lượng. Vì mỗi tầng chỉ *tinh chỉnh* nghiệm của tầng thô hơn, một sai lầm ở tầng thô là sai lầm vĩnh viễn. Đây chính là lý do RAFT bỏ kim tự tháp và tính tương quan toàn cặp một lần.

Hình 4.10 nên đọc như một cảnh báo quen thuộc với người làm bám đặc trưng: mọi sơ đồ nhiều tầng chỉ tinh chỉnh chứ không sửa lại điều thừa hưởng đúng chế độ hỏng này, kể cả bộ tối ưu bạn tự viết.

**Bảng 4.4.** Bốn thể hệ optical flow. Cột cuối là cột đáng giá nhất: mỗi phương pháp mượn ràng buộc từ một nguồn khác nhau, nên nó hỏng khi đúng nguồn đó cạn.

| Phương pháp      | Mượn ràng buộc từ đâu                                                                     | Mạnh khi nào                             | Hỏng khi nào                                                            |
|------------------|-------------------------------------------------------------------------------------------|------------------------------------------|-------------------------------------------------------------------------|
| Lucas–Kanade     | Giả định flow hằng trong cửa sổ nhỏ                                                       | Chuyển động nhỏ, kết cấu giàu, rất rẻ    | Vùng trơn, cạnh dài, chuyển động lớn; thay đổi độ sáng phá giả định gốc |
| LK + kim tự tháp | Thêm giả định trơn qua các tỉ lệ                                                          | Chuyển động lớn của vùng lớn             | Vật nhỏ chuyển động nhanh biến mất ở tầng thô                           |
| FlowNet, PWC-Net | Học từ dữ liệu; PWC-Net đưa kim tự tháp, warping, khối chi phí thành kiến trúc [86], [87] | Cân bằng tốt giữa chính xác và tốc độ    | Lệch miễn so với dữ liệu tổng hợp dùng để huấn luyện                    |
| RAFT             | Khối tương quan <i>toàn cặp</i> + tinh chỉnh lặp bằng GRU [88]                            | Chuyển động lớn, biên sắc, khái quát tốt | Bộ nhớ của khối tương quan; số vòng lặp là ngân sách phải tự chọn       |

RAFT đáng chú ý riêng vì hai lý do rất gần với kinh nghiệm tối ưu phi tuyến. Thứ nhất, nó bỏ chiến lược thô–tối–mịn bằng cách tính tương quan giữa *mọi* cặp pixel một lần, nhờ đó chuyển động lớn không còn cần kim tự tháp. Cái giá đo được: ở độ phân giải  $1/8$  của ảnh  $1024 \times 436$ , lưới là  $128 \times 54 \approx 6900$  điểm, nên khối toàn cặp có  $6900^2 \approx 4,8 \times 10^7$  phần tử — khoảng 190 MB ở fp32, trước khi tính bất cứ thứ gì. Thứ hai, nó tinh chỉnh nghiệm bằng một số vòng lặp cố định thay vì giải một lần; điều này khiến *số vòng lặp trở thành nút vận độ trễ*, đúng như việc bạn giới hạn số bước Gauss–Newton trong một bộ tối ưu thời gian thực. Đây là chỗ nổi đáng nhớ: mô hình học sâu ở đây không thay thế tư duy tối ưu lặp, nó mượn lại tư duy đó.

## B. Nhãn nằm trên khoảng thời gian, không nằm trên khung hình

Nhận dạng hành động trên clip đã cắt sẵn là bài toán dễ; *temporal action localization* mới là bài toán thật: cho một video dài, tìm *khoảng* nào chứa hành động gì. Việc nhãn nằm trên khoảng kéo theo hai hệ quả mà người quen bài toán ảnh dễ vấp.

- **Metric phải đổi.** Ta không đánh giá theo khung hình mà theo khoảng, với ngưỡng *temporal IoU*. Ranh giới hành động vốn mơ hồ ngay cả với người gán nhãn — “bắt đầu ngồi xuống” là khung nào? — nên độ chính xác ở ngưỡng tIoU chặt phản ánh sự bất đồng của người gán nhãn nhiều hơn phản ánh chất lượng mô hình.
- **Chia dữ liệu phải theo video, không theo clip.** Hai clip cắt từ cùng một video có nền, ánh sáng và diễn viên giống nhau; để chúng nằm hai bên tập train/test là rò rỉ dữ liệu kinh điển, và nó thổi phồng kết quả một cách rất khó phát hiện.

↔ **trường hợp riêng của B/07** — hai clip cắt từ cùng một video là một nhóm phụ thuộc; chia ngẫu nhiên theo clip là rò rỉ dữ liệu kinh điển, chỉ đổi trục từ không gian sang thời gian.

## C. Tracking chính là liên kết dữ liệu

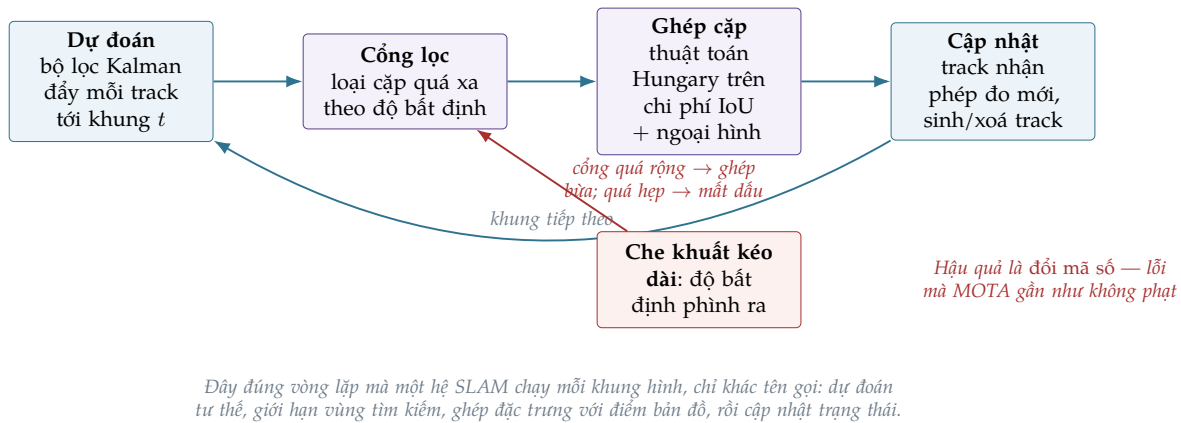
Có hai triết lý. *Tracking-by-detection* tách hẳn hai việc: một detector chạy độc lập trên mỗi khung, rồi một khối liên kết nối các phát hiện qua thời gian bằng một mô hình chuyển động cộng một độ đo tương đồng. *Joint detection-tracking* huấn luyện cả hai cùng lúc, mạnh hơn trên giấy nhưng khó gỡ lỗi vì không còn ranh giới rõ để đặt điểm dừng. Với hệ thống sản xuất, cách thứ nhất

thường thắng vì lý do kỹ thuật hệ thống chứ không phải vì độ chính xác: bạn thay được detector mà không phải huấn luyện lại phần liên kết.

Khối liên kết là chỗ mà người làm SLAM sẽ thấy quen tới mức bất ngờ. SORT dùng đúng bộ đôi kinh điển — bộ lọc Kalman dự đoán vị trí khung tới, rồi thuật toán Hungary ghép cặp dự đoán với phát hiện theo chi phí IoU [89]. ByteTrack thêm một quan sát đơn giản mà hiệu quả: đừng vứt các phát hiện điểm thấp, hãy dùng chúng ở vòng ghép thứ hai, vì một vật bị che một phần thường cho điểm thấp chứ không phải không có [90]. Về mặt cấu trúc, đây là cùng bài toán **liên kết dữ liệu** (*data association*) mà một hệ SLAM giải khi ghép đặc trưng với điểm bản đồ: có dự đoán từ mô hình chuyển động, có cổng lọc theo khoảng cách thống kê, có một bài toán gán tối ưu, và có hậu quả nghiêm trọng khi gán sai.

→ cùng nguyên lý với C/16 — ghép hộp với track và ghép đặc trưng với điểm bản đồ là cùng một bài toán gán tối ưu có cổng lọc theo độ bất định; chỉ khác hậu quả khi gán sai.

Khác biệt lớn nhất là ở SLAM, một liên kết sai làm hỏng bản đồ và rất khó rút lại, trong khi ở tracking nó chỉ đổi một mã số — nhưng chính vì hậu quả nhẹ hơn nên metric mới dễ che giấu nó.



**Hình 4.11.** Tracking là một vòng lặp liên kết dữ liệu, không phải một mạng nơ-ron. Ba trong bốn khối không chứa tham số học được nào; phần học được nằm ở detector cấp phép đo và ở đặc trưng ngoại hình dùng làm chi phí ghép. Vì thế cải thiện detector và cải thiện liên kết là hai việc khác nhau, và một thước đo gộp chúng vào một số sẽ che mất việc nào đang hỏng.

Hình 4.11 là lý do hộp cạm bẫy ngay dưới đây quan trọng đến vậy: nếu vòng lặp này hỏng mà thước đo không phản ánh, bạn sẽ dành công sức tối ưu nhầm khối.

#### CẠM BÃY

MOTA cộng ba loại lỗi rồi chia cho tổng số vật thể thật:  $MOTA = 1 - (FN + FP + IDSW) / GT$ . Vấn đề nằm ở việc FN và FP được đếm *trên từng khung hình cho từng vật*, còn mỗi lần đổi mã số chỉ được đếm *một lần*. Ví dụ: 1000 vật–khung, 50 FN, 30 FP, 40 lần đổi mã cho  $MOTA = 0,88$ ; cải thiện khối liên kết để chỉ còn 20 lần đổi mã — tức tốt lên gấp đôi — chỉ nâng MOTA lên 0,90. Nói cách khác, một hệ thống liên tục đánh tráo danh tính vẫn có thể khoe MOTA cao. HOTA sửa điều này bằng cách tách riêng chất lượng phát hiện và chất lượng liên kết, rồi lấy trung bình nhân  $\sqrt{DetA \cdot AssA}$ . Hai thành phần khi đó có trọng số ngang nhau, và không thành phần nào bù đắp được cho thành phần kia [91], [92].



#### D. Nhất quán thời gian và ràng buộc nhân quả

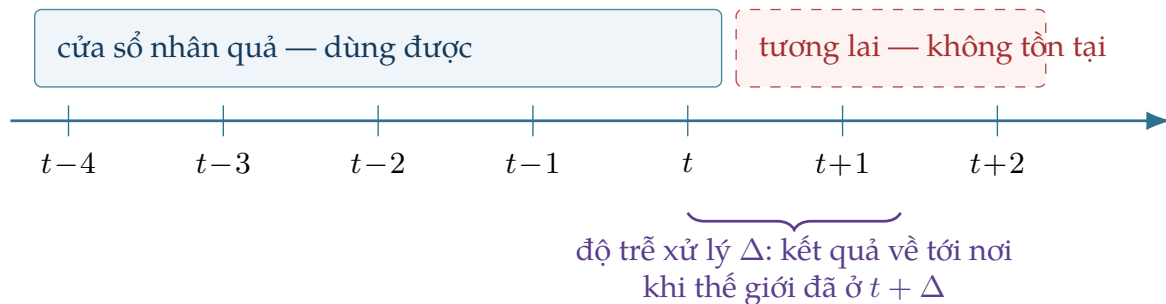
Một mô hình phân vùng chạy độc lập trên từng khung có thể đạt mIoU cao mà vẫn cho video nhấp nháy. Lý do là hàm mất mát không có số hạng nào phạt việc dự đoán đối qua đối lại giữa hai khung gần như giống nhau. Ba cách xử lý, với ba mức chi phí khác nhau:

**Bảng 4.5.** Ba cách chữa hiện tượng nhấp nháy, xếp theo mức xâm lấn vào hệ thống.

| Cách                      | Cơ chế                                                          | Chi phí                              | Hỏng khi nào                                              |
|---------------------------|-----------------------------------------------------------------|--------------------------------------|-----------------------------------------------------------|
| Làm mượt hậu xử lý        | Lọc dự đoán qua vài khung, có thể warp bằng flow                | Rẻ, nhưng thêm độ trễ                | Chuyển cảnh và chuyển động nhanh: làm mượt thành làm nhoè |
| Đưa thời gian vào mô hình | Mô hình nhận nhiều khung, hoặc mang trạng thái ẩn               | Đắt nhất, chạm vào ngân sách ở Mục 2 | Trạng thái ẩn có thể “dính” vào một sai lầm và tự củng cố |
| Loss nhất quán thời gian  | Phạt sai khác giữa dự đoán của hai khung liên tiếp sau khi warp | Trung dung, chỉ tốn lúc huấn luyện   | Warp dựa vào flow, nên flow sai thì loss dạy điều sai     |

↔ **ràng buộc bởi C/15/02** — cách chữa “đưa thời gian vào mô hình” nghe hấp dẫn nhất nhưng chính là cách đắt nhất, vì nó tiêu vào đúng ngân sách ba chiều đã cạn ở mục trước.

Cả ba đều bị một ràng buộc lớn hơn chi phối: hệ thống chạy trực tuyến *không được nhìn khung hình tương lai*. Ràng buộc này loại thẳng mọi kiến trúc hai chiều, mọi phép làm mượt trung tâm, và mọi lịch trình đánh giá “xem toàn bộ video rồi mới quyết định” — và nó loại chúng trước khi độ chính xác được đem ra bàn.



**Hình 4.12.** Ràng buộc nhân quả, vẽ trên trục thời gian. Vùng xanh là toàn bộ thông tin mà một hệ streaming được phép dùng để quyết định tại thời điểm  $t$ ; vùng đỏ là thứ mà mọi mô hình hai chiều lạng lẽ giả định là có. Ngoặc tím nhắc thêm một điều thường bị quên: ngay cả quyết định đúng cũng đến muộn mất  $\Delta$ , nên hệ thống hạ nguồn phải hoặc dự đoán bù, hoặc chấp nhận hành động theo một trạng thái đã cũ.

Hình 4.12 nói thêm một điều mà người làm robot cảm nhận rõ hơn ai hết: ràng buộc nhân quả không chỉ là “không được nhìn tương lai” mà còn là “kết quả luôn nói về quá khứ”. Vì thế, khi so sánh hai mô hình cho hệ thời gian thực, đại lượng đáng so không phải độ chính xác trên khung hình  $t$ . Đáng so là độ chính xác *tại thời điểm quyết định được dùng*. Cách đánh giá này vẫn còn ít được chuẩn hoá, và nhiều bảng kết quả công bố không nói gì về nó.

#### E. Giới hạn và trường hợp hỏng

- **Giả định độ sáng không đổi bị vi phạm thường xuyên hơn ta nghĩ.** Tự động phơi sáng, đèn nhấp nháy, phản xạ gương và bề mặt bóng đều làm sai giả định gốc của flow — và đây đúng là những cảnh làm hỏng cả các hệ thống hình học dựa trên độ sáng. Mô hình học sâu chịu

đựng tốt hơn nhưng không miễn nhiễm, vì dữ liệu huấn luyện của chúng phần lớn là ảnh tổng hợp phối sáng cố định.

- **Che khuất kéo dài là kẻ thù của mọi bộ bám.** Mô hình chuyển động dự đoán được vài khung, sau đó độ bất định lớn tới mức cổng lọc hoặc quá rộng (ghép bù) hoặc quá hẹp (mất dấu). Không có cách chỉnh tham số nào thoát khỏi đánh đổi này; chỉ có thêm thông tin — ngoại hình, hoặc một cảm biến khác.
- **Metric đánh lừa theo hai kiểu.** MOTA che giấu lỗi liên kết như đã nói; còn mIoU theo khung che giấu hiện tượng nhấp nháy. Cả hai đều là ví dụ của cùng một sai lầm: chọn một con số tổng không đo được đại lượng mà hệ thống hạ nguồn thực sự cần.
- **Lệch miền của dữ liệu flow.** Vì nhãn flow dày đặc gần như không thể gán bằng tay, các mô hình được huấn luyện chủ yếu trên dữ liệu tổng hợp; chất lượng trên cảnh thật vì thế phụ thuộc vào mức độ giống nhau giữa hai miền, và điều đó thay đổi theo camera của bạn.

#### CẦN NHỚ

Trong flow cũng như trong tracking, phần khó không phải phát hiện mà là *liên kết* — và cả hai lĩnh vực đều có một metric phổ biến che giấu đúng phần đó. Hãy chọn metric tách riêng chất lượng phát hiện khỏi chất lượng liên kết (HOTA cho tracking, sai số theo từng chế độ chuyển động cho flow), rồi mới chọn mô hình. Ràng buộc nhân quả thì đứng trên tất cả: nó loại phương án trước khi độ chính xác kịp lên tiếng. [T2]

## F. Kết nối

[C/15-thi-giac-khong-thoi-gian/02-bieu-dien-video](#) là tiền đề về chi phí: mọi cách chữa nhấp nháy bằng “đưa thời gian vào mô hình” đều phải trả bằng ngân sách ba chiều ở đó. [B/07-chia-du-lieu-va-danh-gia](#) là nơi bàn tổng quát về việc chọn metric và về rõ ràng do nhóm phụ thuộc, mà chia dữ liệu theo clip là một ca điển hình. [C/16-thi-giac-3d](#) nối tiếp trực tiếp: flow là bài toán tương ứng hai chiều, còn chương 16 nâng nó lên tương ứng ba chiều và ràng buộc epipolar. [C/14-bai-toan-cv-loi/04-bai-toan-lan-can](#) chứa cấu trúc đánh đổi song song: top-down và bottom-up ở pose lặp lại đúng dạng của tracking-by-detection và joint tracking.

### Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao một phương trình độ sáng không đủ để giải ra flow, và ba giả định của Lucas–Kanade mượn ràng buộc còn thiếu từ đâu?
2. Viết ma trận  $A^T A$  cho một cửa sổ nằm trên cạnh dọc lý tưởng và giải thích vì sao nó suy biến. Điều đó liên hệ thế nào với việc chọn đặc trưng góc?
3. Kim tự tháp thô–tới–mịn chứa được chuyển động lớn nhưng sinh ra chế độ hỏng nào, và nó biểu hiện ra sao trong một cảnh giao thông cụ thể?
4. RAFT bỏ kim tự tháp bằng khối tương quan toàn cặp. Cái giá tính được bằng con số là gì, và số vòng lặp đóng vai trò gì với một hệ thời gian thực?
5. Một hệ thống có MOTA 0,90 nhưng đổi mã số liên tục. Giải thích bằng công thức vì sao điều đó có thể xảy ra, và HOTA sửa nó thế nào?
6. Ràng buộc nhân quả loại bỏ những loại kiến trúc và loại đánh giá nào? Vì sao độ trễ  $\Delta$  khiến “độ chính xác trên khung  $t$ ” là một đại lượng không đủ?
7. Bạn có mô hình phân vùng mIoU cao nhưng video nhấp nháy, và ngân sách tính toán đã cạn. Bạn chọn cách chữa nào trong ba cách, và chấp nhận hỏng ở đâu?

- [1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [2] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [3] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *AISTATS*, 2010.
- [4] K. He, X. Zhang, S. Ren, et al., “Delving Deep into Rectifiers,” in *ICCV*, 2015.
- [5] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [6] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [7] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [8] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [9] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [10] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [11] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” in *ICLR*, 2017.
- [12] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [13] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [14] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [15] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [16] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [17] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [18] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [19] G. Huang, Y. Sun, Z. Liu, et al., “Deep Networks with Stochastic Depth,” in *ECCV*, 2016.
- [20] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [21] H. Zhang, M. Cisse, Y. N. Dauphin, et al., “mixup: Beyond Empirical Risk Minimization,” in *ICLR*, 2018.
- [22] S. Yun, D. Han, S. J. Oh, et al., “CutMix: Regularization Strategy to Train Strong Classifiers,” in *ICCV*, 2019.
- [23] E. D. Cubuk, B. Zoph, J. Shlens, et al., “RandAugment: Practical Automated Data Augmentation,” in *CVPR Workshops*, 2020.

- [24] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [25] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [26] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *NeurIPS*, 2012.
- [27] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in *ICLR*, 2015.
- [28] C. Szegedy, W. Liu, et al., “Going Deeper with Convolutions,” in *CVPR*, 2015.
- [29] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [30] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [31] G. Huang, Z. Liu, L. van der Maaten, et al., “Densely Connected Convolutional Networks,” in *CVPR*, 2017.
- [32] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [33] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [34] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [35] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [36] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [37] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.
- [38] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [39] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [40] J. Su, M. Ahmed, Y. Lu, et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, vol. 568, 2024.
- [41] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” *arXiv:2312.00752*, 2023.
- [42] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [43] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.
- [44] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [45] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [46] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [47] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [48] R. Girshick, J. Donahue, T. Darrell, et al., “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation,” in *CVPR*, 2014.

- [49] R. Girshick, "Fast R-CNN," in *ICCV*, 2015.
- [50] S. Ren, K. He, R. Girshick, et al., "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in *NeurIPS*, 2015.
- [51] T.-Y. Lin, P. Dollár, R. Girshick, et al., "Feature Pyramid Networks for Object Detection," in *CVPR*, 2017.
- [52] K. He, G. Gkioxari, P. Dollár, et al., "Mask R-CNN," in *ICCV*, 2017.
- [53] D. Bolya, S. Foley, J. Hays, et al., "TIDE: A General Toolbox for Identifying Object Detection Errors," in *ECCV*, 2020.
- [54] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [55] J. Redmon, S. Divvala, R. Girshick, et al., "You Only Look Once: Unified, Real-Time Object Detection," in *CVPR*, 2016.
- [56] W. Liu, D. Anguelov, D. Erhan, et al., "SSD: Single Shot MultiBox Detector," in *ECCV*, 2016.
- [57] T.-Y. Lin, P. Goyal, R. Girshick, et al., "Focal Loss for Dense Object Detection," in *ICCV*, 2017.
- [58] Z. Tian, C. Shen, H. Chen, et al., "FCOS: Fully Convolutional One-Stage Object Detection," in *ICCV*, 2019.
- [59] X. Zhou, D. Wang, and P. Krähenbühl, "Objects as Points," *arXiv:1904.07850*, 2019.
- [60] S. Zhang, C. Chi, Y. Yao, et al., "Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection," in *CVPR*, 2020.
- [61] Z. Ge, S. Liu, Z. Li, et al., "OTA: Optimal Transport Assignment for Object Detection," in *CVPR*, 2021.
- [62] N. Bodla, B. Singh, R. Chellappa, et al., "Soft-NMS: Improving Object Detection with One Line of Code," in *ICCV*, 2017.
- [63] X. Zhu, W. Su, L. Lu, et al., "Deformable DETR: Deformable Transformers for End-to-End Object Detection," in *ICLR*, 2021.
- [64] H. Zhang, F. Li, S. Liu, et al., "DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection," in *ICLR*, 2023.
- [65] J. Long, E. Shelhamer, and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," in *CVPR*, 2015.
- [66] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *MICCAI*, 2015.
- [67] L.-C. Chen, G. Papandreou, I. Kokkinos, et al., "DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs," *IEEE TPAMI*, vol. 40, no. 4, pp. 834–848, 2018.
- [68] E. Xie, W. Wang, Z. Yu, et al., "SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers," in *NeurIPS*, 2021.
- [69] B. Cheng, I. Misra, A. G. Schwing, et al., "Masked-Attention Mask Transformer for Universal Image Segmentation," in *CVPR*, 2022.
- [70] A. Kirillov, K. He, R. Girshick, et al., "Panoptic Segmentation," in *CVPR*, 2019.
- [71] A. Kirillov, E. Mintun, N. Ravi, et al., "Segment Anything," in *ICCV*, 2023.
- [72] K. Sun, B. Xiao, D. Liu, et al., "Deep High-Resolution Representation Learning for Human Pose Estimation," in *CVPR*, 2019.

- [73] Z. Cao, T. Simon, S.-E. Wei, et al., “Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields,” in *CVPR*, 2017.
- [74] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [75] J. Deng, J. Guo, N. Xue, et al., “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *CVPR*, 2019.
- [76] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [77] K. Xu, J. Ba, R. Kiros, et al., “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention,” in *ICML*, 2015.
- [78] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *ICLR*, 2015.
- [79] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [80] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [81] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [82] K. Simonyan and A. Zisserman, “Two-Stream Convolutional Networks for Action Recognition in Videos,” in *NeurIPS*, 2014.
- [83] J. Carreira and A. Zisserman, “Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset,” in *CVPR*, 2017.
- [84] D. Tran, H. Wang, L. Torresani, et al., “A Closer Look at Spatiotemporal Convolutions for Action Recognition,” in *CVPR*, 2018.
- [85] C. Feichtenhofer, H. Fan, J. Malik, et al., “SlowFast Networks for Video Recognition,” in *ICCV*, 2019.
- [86] A. Dosovitskiy, P. Fischer, E. Ilg, et al., “FlowNet: Learning Optical Flow with Convolutional Networks,” in *ICCV*, 2015.
- [87] D. Sun, X. Yang, M.-Y. Liu, et al., “PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume,” in *CVPR*, 2018.
- [88] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.
- [89] A. Bewley, Z. Ge, L. Ott, et al., “Simple Online and Realtime Tracking,” in *ICIP*, 2016.
- [90] Y. Zhang, P. Sun, Y. Jiang, et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *ECCV*, 2022.
- [91] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [92] K. Bernardin and R. Stiefelhagen, “Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics,” *EURASIP J. Image and Video Processing*, 2008.
- [93] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [94] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [95] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.



- [96] S. Wang, V. Leroy, Y. Cabon, et al., "DUST3R: Geometric 3D Vision Made Easy," in *CVPR*, 2024.
- [97] D. Eigen, C. Puhrsch, and R. Fergus, "Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network," in *NeurIPS*, 2014.
- [98] R. Ranftl, K. Lasinger, D. Hafner, et al., "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer," *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [99] J. Sturm, N. Engelhard, F. Endres, et al., "A Benchmark for the Evaluation of RGB-D SLAM Systems," in *IROS*, 2012.
- [100] C. R. Qi, H. Su, K. Mo, et al., "PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation," in *CVPR*, 2017.
- [101] C. R. Qi, L. Yi, H. Su, et al., "PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space," in *NeurIPS*, 2017.
- [102] A. H. Lang, S. Vora, H. Caesar, et al., "PointPillars: Fast Encoders for Object Detection from Point Clouds," in *CVPR*, 2019.
- [103] T. Müller, A. Evans, C. Schied, et al., "Instant Neural Graphics Primitives with a Multiresolution Hash Encoding," *ACM TOG*, vol. 41, no. 4, 2022.
- [104] B. Kerbl, G. Kopanas, T. Leimkühler, et al., "3D Gaussian Splatting for Real-Time Radiance Field Rendering," *ACM TOG*, vol. 42, no. 4, 2023.
- [105] R. Zhang, P. Isola, A. A. Efros, et al., "The Unreasonable Effectiveness of Deep Features as a Perceptual Metric," in *CVPR*, 2018.
- [106] D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," in *ICLR*, 2014.
- [107] I. Goodfellow, J. Pouget-Abadie, M. Mirza, et al., "Generative Adversarial Nets," in *NeurIPS*, 2014.
- [108] T. Karras, S. Laine, and T. Aila, "A Style-Based Generator Architecture for Generative Adversarial Networks," in *CVPR*, 2019.
- [109] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," in *NeurIPS*, 2020.
- [110] Y. Song, J. Sohl-Dickstein, D. P. Kingma, et al., "Score-Based Generative Modeling Through Stochastic Differential Equations," in *ICLR*, 2021.
- [111] J. Johnson, A. Alahi, and L. Fei-Fei, "Perceptual Losses for Real-Time Style Transfer and Super-Resolution," in *ECCV*, 2016.
- [112] P. Isola, J.-Y. Zhu, T. Zhou, et al., "Image-to-Image Translation with Conditional Adversarial Networks," in *CVPR*, 2017.
- [113] L. A. Gatys, A. S. Ecker, and M. Bethge, "Image Style Transfer Using Convolutional Neural Networks," in *CVPR*, 2016.
- [114] X. Huang and S. Belongie, "Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization," in *ICCV*, 2017.
- [115] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," in *NeurIPS Deep Learning Workshop*, 2015.
- [116] J. Song, C. Meng, and S. Ermon, "Denoising Diffusion Implicit Models," in *ICLR*, 2021.
- [117] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, et al., "Flow Matching for Generative Modeling," in *ICLR*, 2023.
- [118] J. Ho and T. Salimans, "Classifier-Free Diffusion Guidance," *arXiv:2207.12598*, 2022.
- [119] R. Rombach, A. Blattmann, D. Lorenz, et al., "High-Resolution Image Synthesis with Latent Diffusion Models," in *CVPR*, 2022.
- [120] W. Peebles and S. Xie, "Scalable Diffusion Models with Transformers," in *ICCV*, 2023.

- [121] L. Zhang, A. Rao, and M. Agrawala, "Adding Conditional Control to Text-to-Image Diffusion Models," in *ICCV*, 2023.
- [122] Y. Blau and T. Michaeli, "The Perception-Distortion Tradeoff," in *CVPR*, 2018.
- [123] C. Ledig, L. Theis, F. Huszár, et al., "Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network," in *CVPR*, 2017.
- [124] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., "Image Quality Assessment: From Error Visibility to Structural Similarity," *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [125] M. Heusel, H. Ramsauer, T. Unterthiner, et al., "GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium," in *NeurIPS*, 2017.
- [126] T. Brooks, B. Mildenhall, T. Xue, et al., "Unprocessing Images for Learned Raw Denoising," in *CVPR*, 2019.
- [127] A. Abdelhamed, S. Lin, and M. S. Brown, "A High-Quality Denoising Dataset for Smartphone Cameras," in *CVPR*, 2018.
- [128] J. Deng, W. Dong, R. Socher, et al., "ImageNet: A Large-Scale Hierarchical Image Database," in *CVPR*, 2009.
- [129] A. Radford, J. W. Kim, C. Hallacy, et al., "Learning Transferable Visual Models from Natural Language Supervision," in *ICML*, 2021.
- [130] J. Li, D. Li, S. Savarese, et al., "BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models," in *ICML*, 2023.
- [131] H. Liu, C. Li, Q. Wu, et al., "Visual Instruction Tuning," in *NeurIPS*, 2023.
- [132] S. Kornblith, J. Shlens, and Q. V. Le, "Do Better ImageNet Models Transfer Better?" In *CVPR*, 2019.
- [133] T. Chen, S. Kornblith, M. Norouzi, et al., "A Simple Framework for Contrastive Learning of Visual Representations," in *ICML*, 2020.
- [134] K. He, H. Fan, Y. Wu, et al., "Momentum Contrast for Unsupervised Visual Representation Learning," in *CVPR*, 2020.
- [135] J.-B. Grill, F. Strub, F. Altché, et al., "Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning," in *NeurIPS*, 2020.
- [136] M. Caron, H. Touvron, I. Misra, et al., "Emerging Properties in Self-Supervised Vision Transformers," in *ICCV*, 2021.
- [137] K. He, X. Chen, S. Xie, et al., "Masked Autoencoders Are Scalable Vision Learners," in *CVPR*, 2022.
- [138] A. van den Oord, Y. Li, and O. Vinyals, "Representation Learning with Contrastive Predictive Coding," in *arXiv:1807.03748*, 2018.
- [139] J. Kaplan, S. McCandlish, T. Henighan, et al., "Scaling Laws for Neural Language Models," *arXiv:2001.08361*, 2020.
- [140] X. Zhai, A. Kolesnikov, N. Houlsby, et al., "Scaling Vision Transformers," in *CVPR*, 2022.
- [141] J. Hoffmann, S. Borgeaud, A. Mensch, et al., "Training Compute-Optimal Large Language Models," in *NeurIPS*, 2022.
- [142] C. G. Northcutt, A. Athalye, and J. Mueller, "Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks," in *NeurIPS Datasets and Benchmarks*, 2021.
- [143] B. Recht, R. Roelofs, L. Schmidt, et al., "Do ImageNet Classifiers Generalize to ImageNet?" In *ICML*, 2019.
- [144] N. Houlsby, A. Giurgiu, S. Jastrzebski, et al., "Parameter-Efficient Transfer Learning for NLP," in *ICML*, 2019.

- [145] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [146] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *RecSys*, 2019.

## 4.4 Thực hành

Chương này có một đặc điểm may mắn: mọi khẳng định trong đó đều kiểm chứng được trên một video 60 giây và một GPU tầm trung. Hãy tận dụng điều đó. Và hãy tự tay làm hỏng hệ thống: các bài dưới đây được thiết kế để bạn quan sát *metric nào tụt trước*. Đó mới là thứ dạy bạn về metric.

**Repo và tài nguyên.** Tracking: [ifzhang/ByteTrack](#) (đơn giản, mạnh, đọc được trong một buổi), [NirAharon/BoT-SORT](#), và [mikel-brostrom/boxmot](#) nếu muốn thử nhiều bộ liên kết trên cùng một detector. Phân vùng và bám trong video: SAM 3. Bám điểm dày: [facebookresearch/co-tracker](#). Optical flow: [princeton-vl/RAFT](#) và [autonomousvision/unimatch](#). Nhận dạng hành động: [open-mmlab/mmdetection](#), VideoMAE-V2, InternVideo. Video qua mô hình ngôn ngữ-thị giác: Qwen3-VL (định vị trong video dài), Molmo2 (mở cả dữ liệu, theo dõi đối tượng qua khung hình). Đánh giá: [JonathonLuiten/TrackEval](#) — đây là thứ đáng đầu tư thời gian nhất trong danh sách, vì nó tính HOTA đúng cách và tách được lỗi phát hiện khỏi lỗi liên kết. Danh sách này chốt tại tháng 9/2026 và các tên mô hình dẫn đầu sẽ cũ trong 6–12 tháng, trong khi hạ tầng đánh giá như TrackEval thì bền hơn nhiều.

**Câu hỏi kiểm tra hiểu.** Vì sao MOTA có thể cao trong khi hệ thống liên tục đổi mã số đối tượng, và HOTA sửa điều đó bằng cơ chế nào? Một mô hình phân vùng đạt mIoU 0,85 nhưng video nhấp nháy — ba cách xử lý là gì, và mỗi cách trả giá bằng gì? Nếu bạn phải chạy trực tuyến với ngân sách 30 ms mỗi khung hình, những họ kiến trúc nào trong chương này bị loại ngay từ đầu, và vì lý do gì?

**Mini project (vài giờ đến hai ngày).** Ghép một detector với ByteTrack trên một video 60 giây, đánh giá bằng TrackEval để có cả MOTA lẫn HOTA. Sau đó *cố tình làm hỏng* detector bằng cách hạ dần ngưỡng tin cậy, và ghi lại metric nào tụt trước, tụt bao nhiêu. Kết thúc bằng một đoạn giải thích vì sao thứ tụt lại như vậy — đó là bài học thật của bài tập này.

### Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

| Thuật ngữ / câu hỏi                                               | Trả lời                                                                                                                                                                                                                                              |
|-------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ●●● <b>Nghẽn cổ chai thông tin</b>                                | Chỗ mà hệ thống nén dữ liệu có cấu trúc thành một vector cố định <i>trước khi</i> biết sẽ cần phần nào. Trong sinh mô tả ảnh, đó là việc nén lưới $14 \times 14 \times 512$ thành 512 số, tỉ lệ 196:1, xoá sạch thông tin “đặc trưng này nằm ở đâu”. |
| ●●● <b>Attention</b>                                              | Phép trung bình có trọng số trên các phần của nguồn, với trọng số phụ thuộc vào thứ đang cần. Nó vẫn nén thành một vector, nhưng nén lại một lần cho mỗi bước đầu ra. Gộp trung bình toàn cục là trường hợp riêng khi mọi trọng số bằng nhau.        |
| ●● <b>Vì sao gộp trung bình toàn cục làm hỏng sinh mô tả ảnh?</b> | Vì nó lấy trung bình theo không gian, nên xoá thông tin vị trí. Câu “con chó bên trái người đàn ông” cần đúng thông tin đó. Triệu chứng là mô hình bịa chi tiết hợp lý thay vì mô tả cái có thật.                                                    |

| Thuật ngữ / câu hỏi                                                        | Trả lời (tiếp)                                                                                                                                                                                                                                                     |
|----------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ●●● <b>Kích hoạt</b>                                                       | Tensor giá trị trung gian mà mỗi tầng sinh ra, phải giữ trong bộ nhớ vì lượt ngược cần tới. Một clip 16 khung $224 \times 224 \times 64$ ở fp32 chiếm khoảng 205 MB cho <i>một</i> tensor ở batch 1. Đây mới là thứ làm tràn bộ nhớ, không phải số trọng số.       |
| ●●● <b>Ngân sách bộ nhớ</b>                                                | Lượng kích hoạt tỉ lệ với độ dài clip $\times$ độ phân giải <sup>2</sup> $\times$ batch. Vì tích của ba đại lượng bị phần cứng chặn, mọi lựa chọn kiến trúc video thực chất là chọn hy sinh chiều nào.                                                             |
| ●● <b>(2+1)D</b>                                                           | Tách kernel 3D thành một kernel chỉ theo không gian rồi một kernel chỉ theo thời gian. Chọn số kênh trung gian $M = 2,25 C$ thì tổng tham số bằng bản 3D. Lợi ích đến từ một hàm kích hoạt thêm vào giữa, không từ sức chứa.                                       |
| ●● <b>SlowFast</b>                                                         | Hai nhánh chạy trên cùng video: nhánh chậm ít khung nhưng nhiều kênh để nắm ngữ nghĩa, nhánh nhanh nhiều khung nhưng ít kênh để nắm chuyển động. Nhận định nền là ngữ nghĩa đổi chậm còn chuyển động đổi nhanh; nếu nhận định đó sai thì lợi thế chi phí biến mất. |
| ●● <b>Vì sao chi phí video transformer là bậc hai theo độ dài clip?</b>    | Vì attention tính quan hệ giữa mọi cặp token, và số token tỉ lệ với số khung. Một ảnh có 196 token cho ma trận $3,8 \times 10^4$ ô; clip 16 khung có 3136 token cho $9,8 \times 10^6$ ô, gấp 256 lần.                                                              |
| ●● <b>Mô hình 3D chỉ hơn baseline một khung hình 2 điểm — nghĩa là gì?</b> | Nhiều khả năng tập dữ liệu đoán đúng được từ bối cảnh một khung, tức nó không đo thông tin thời gian. Phân biệt bằng cách chạy thêm trên dữ liệu cần thứ tự, ví dụ phân biệt mở với đóng; nếu baseline sập ở đó thì phép đo mới có nghĩa.                          |
| ●●● <b>Dòng quang</b>                                                      | Trường vector cho biết mỗi pixel của khung này nằm ở đâu trong khung kia. Giả định gốc là độ sáng không đổi, nên nó hỏng khi tự động phơi sáng, đèn nhấp nháy hay bề mặt bóng làm giá trị sáng đổi.                                                                |

●●● phải thuộc    ●● nên nắm    ● biết là có

## I Active recall – phần 2 (che cột phải, tự trả lời, rồi mở ra đối chiếu)

| Thuật ngữ / câu hỏi                                                         | Trả lời                                                                                                                                                                                                                                             |
|-----------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ●●● <b>Bài toán khẩu độ</b>                                                 | Một phương trình độ sáng nhưng hai ẩn, nên qua một cửa sổ nhỏ chỉ đo được thành phần chuyển động vuông góc với cạnh. Đây là thiếu thông tin thật, không phải lỗi thuật toán, nên mọi cách chữa đều là mượn thêm ràng buộc từ nơi khác.              |
| ●● <b>Vì sao phải chọn góc chứ không chọn cạnh để bấm?</b>                  | Vì nghiệm bình phương tối thiểu của Lucas–Kanade cần ma trận $A^T A$ không suy biến, tức cửa sổ phải có gradient theo hai hướng độc lập. Trên một cạnh lý tưởng, giá trị riêng nhỏ nhất bằng 0 và flow dọc cạnh không xác định.                     |
| ●● <b>RAFT</b>                                                              | Bộ ước lượng dòng quang dùng khối tương quan toàn cặp cộng một số vòng tinh chỉnh cố định bằng GRU. Nó bỏ được kim tự tháp nên xử lý tốt chuyển động lớn; giá là khối tương quan cỡ 190 MB ở 1/8 độ phân giải, và số vòng lặp thành nùm vận độ trễ. |
| ●● <b>Vì sao kim tự tháp thô–tối–mịn làm mất vật nhỏ chuyển động nhanh?</b> | Vì thu nhỏ ảnh làm vật nhỏ tụt xuống dưới một pixel ở tầng thô. Nó nhận chuyển động của nền, và các tầng mịn hơn chỉ tinh chỉnh quanh nghiệm sai đó nên không bao giờ sửa lại được.                                                                 |
| ●●● <b>Liên kết dữ liệu</b>                                                 | Bước quyết định phép đo nào ở khung này ứng với đối tượng nào đã biết. Trong tracking là ghép hộp với track, trong SLAM là ghép đặc trưng với điểm bản đồ. Đây là chỗ hệ thống hỏng, chứ không phải bước phát hiện.                                 |

| Thuật ngữ / câu hỏi                                         | Trả lời (tiếp)                                                                                                                                                                                                                       |
|-------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ●● HOTA                                                     | Thước đo tracking tách riêng chất lượng phát hiện (DetA) và chất lượng liên kết (AssA), rồi lấy trung bình nhân $\sqrt{\text{DetA} \cdot \text{AssA}}$ . Vì là trung bình nhân, không thành phần nào bù đắp được cho thành phần kia. |
| ●● Vì sao MOTA che được lỗi đổi mã số?                      | Vì phát hiện sót và phát hiện thừa đếm trên từng khung cho từng vật, còn mỗi lần đổi mã chỉ đếm một lần. Với 1000 vật–khung, giảm số lần đổi mã từ 40 xuống 20 chỉ nâng MOTA từ 0,88 lên 0,90.                                       |
| ●●● Cửa sổ nhân quả                                         | Toàn bộ dữ liệu tới thời điểm hiện tại mà một hệ trực tuyến được phép dùng. Nó loại thẳng mọi kiến trúc hai chiều và mọi phép làm mượt nhìn về phía trước. Thêm nữa, độ trễ xử lý khiến câu trả lời luôn nói về một quá khứ gần.     |
| ●● mIoU từng khung cao nhưng video nhấp nháy — nghĩa là gì? | Mô hình chạy độc lập trên từng khung và hàm mất mát không phạt việc đổi dự đoán giữa hai khung gần giống nhau. Phân biệt với lỗi phát hiện bằng cách đo độ ổn định giữa các khung liên tiếp, không đo mIoU.                          |
| ● Bám mắt dấu sau khi vật bị che vài giây — nguyên nhân?    | Độ bất định của mô hình chuyển động phình ra, nên cổng lọc hoặc quá rộng và ghép bù, hoặc quá hẹp và mất dấu. Không có giá trị ngưỡng nào thoát khỏi đánh đổi này; chỉ thêm thông tin ngoại hình mới cứu được.                       |

●●● phải thuộc ●● nên nắm ● biết là có

## I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

| Công trình / hệ thống                 | Dùng ý tưởng nào của chương                                                                                                        | Ghi chú                                                                                 |
|---------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| RAFT và các bản kế thừa               | Khối tương quan toàn cặp cộng tính chính lập; số vòng lặp làm núng vận độ trễ                                                      | Vẫn là kiến trúc tham chiếu của dòng quang học sâu tính tới 9/2026                      |
| ByteTrack, BoT-SORT                   | Tracking-by-detection: bộ lọc Kalman dự đoán, thuật toán Hungary ghép cặp, và ý “dùng vết phát hiện điểm thấp”                     | Được dùng rộng rãi trong hệ đếm và giám sát thực tế                                     |
| TrackEval                             | Hiện thực HOTA — tách chất lượng phát hiện khỏi chất lượng liên kết thay vì gộp vào một số                                         | Hạ tầng đánh giá; bền hơn mọi tên mô hình trong danh sách này                           |
| SAM 3 (2026) cho video                | Phân vùng theo prompt cộng bám danh tính qua khung hình — đúng bài toán liên kết dữ liệu, chỉ khác là đơn vị được liên kết là mask | Ranh giới giữa segmentation và tracking đang mờ dần                                     |
| co-tracker                            | Bám điểm dày qua thời gian; nối trực tiếp bài toán tương ứng điểm với bài toán giữ danh tính                                       | Hữu ích khi bạn cần tương ứng dài hạn chứ không chỉ giữa hai khung liên tiếp            |
| Kiến trúc SlowFast và các bản kế thừa | Ý “ngữ nghĩa thay đổi chậm, chuyển động thay đổi nhanh nên không cần cùng tốc độ lấy mẫu”                                          | Vẫn xuất hiện trong nhiều mô hình video hai nhánh; tên mô hình dẫn đầu thì đổi liên tục |

MINI PROJECT SLAM · Dòng quang hay mô tả đặc trưng — ngưỡng chuyển động nào làm mỗi bên gãy? · 1–2 ngày

|          |                                                                                                                                                                                                                                                                                                      |
|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Mục tiêu | đo trực tiếp thứ mà cả chương chỉ nói bằng lời: hai cách giải bài toán tương ứng điểm — bám theo dòng quang và ghép theo mô tả đặc trưng — gãy ở đâu, và ngưỡng gãy đó nằm ở độ lớn chuyển động nào. Đây chính là quyết định liên kết dữ liệu mà frontend SLAM của bạn phải đưa ra ở mỗi khung hình. |
| Dữ liệu  | một chuỗi EuRoC có dải chuyển động rộng (chuỗi “khó” như V2_03 là lựa chọn tốt vì nó có cả đoạn chậm lẫn đoạn lắc mạnh và mờ). Chân trị tương ứng điểm không có sẵn, nên dùng <i>kiểm tra vòng</i> làm thước đo thay thế: ghép từ khung $t$ sang $t + 1$ rồi ghép                                    |



|          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|          | ngược lại, cặp nào quay về đúng vị trí ban đầu trong ngưỡng một pixel thì tính là ghép đúng.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| Các bước | (1) trích một tập đặc trưng góc chung cho cả hai phương pháp trên mỗi khung hình, để hai bên xuất phát từ cùng một tập điểm; (2) nhánh A: bám các điểm đó sang khung sau bằng dòng quang (KLT có kim tự tháp, hoặc <b>RAFT</b> rồi lấy mẫu tại các điểm); nhánh B: tính mô tả đặc trưng và ghép theo khoảng cách gần nhất có kiểm tra tỉ số; (3) với mỗi cặp khung hình, tính độ lớn chuyển động trung vị (từ chân trị tư thế, hoặc từ chính trung vị dòng quang) và tỉ lệ ghép đúng theo kiểm tra vòng; (4) chia các cặp khung hình thành các nhóm theo độ lớn chuyển động và vẽ tỉ lệ ghép đúng theo nhóm cho cả hai nhánh. |
| Xong khi | bạn có một đồ thị tỉ lệ ghép đúng theo độ lớn chuyển động, hai đường cho hai phương pháp. Bạn chỉ ra được bằng số ngưỡng mà mỗi đường bắt đầu sụt, tính bằng pixel dịch chuyển trung vị. Và bạn giải thích được cơ chế: dòng quang gây vì khai triển bậc nhất hết hiệu lực và kim tự tháp làm mất vật nhỏ, còn mô tả đặc trưng gây vì mờ chuyển động phá kết cấu cục bộ.                                                                                                                                                                                                                                                      |
| Nối với  | <b>C/15/03</b> cho cơ chế của cả hai bên; <b>A/01</b> cho lý do ma trận cấu trúc quyết định điểm nào bám được; <b>E/24</b> cho cách trình bày kết quả; dùng lại đúng chuỗi và đúng harness của mini project chương 14.                                                                                                                                                                                                                                                                                                                                                                                                        |

### TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì hàm mất mát của mô hình per-frame không có số hạng nào phạt việc dự đoán đổi qua đổi lại giữa hai khung gần như giống nhau; và mIoU tính độc lập trên từng khung nên trung bình của nhiều khung nhấp nháy vẫn cao. Muốn thấy lỗi đó phải đo sự ổn định *giữa* các khung, thứ chưa được chuẩn hoá thành một thước đo phổ biến.
- A2. Nó ra đời để gỡ nghẽn cổ chai thông tin: cả ảnh bị nén vào một vector trước khi biết câu sẽ nói gì. Nút thắt này không xuất hiện ở phân loại vì ở đó đầu ra chỉ có một câu hỏi duy nhất và một vector là đủ; nó chỉ lộ ra khi đầu ra gồm nhiều bước, mỗi bước cần một phần khác nhau của ảnh.
- A3. Vì  $MOTA = 1 - (FN + FP + IDSW)/GT$  đếm phát hiện sót và phát hiện thừa *trên từng khung cho từng vật*, nhưng mỗi lần đổi mã số chỉ được đếm một lần. Với nghìn vật–khung, giảm một nửa số lần đổi mã chỉ nâng MOTA vài phần trăm điểm. HOTA sửa bằng trung bình nhân  $\sqrt{DetA \cdot AssA}$ , nên không thành phần nào bù cho thành phần kia.
- A4. Vì trục thời gian có hướng: đảo ngược nó thì “mở” thành “đóng”, trong khi lật trục  $x$  vẫn cho một ảnh hợp lệ. Nó lại còn có đơn vị khác hẳn hai trục kia. Vậy không có lý do gì để cùng một kernel  $3 \times 3 \times 3$  là kích thước hợp lý ở cả ba trục. Hai bất đối xứng đó là nguyên nhân sâu xa của mọi thiết kế tách trục.
- A5. Bằng độ phân giải hoặc bằng batch, theo quan hệ  $R \propto 1/\sqrt{TB}$ . Ba cách không tương đương: giảm batch làm hỏng thống kê của chuẩn hoá theo batch, giảm độ phân giải làm mất vật nhỏ vĩnh viễn, còn giảm độ dài clip thì làm mất chính thông tin thời gian mà bạn đang trả tiền để có.

### NÓI LẠI THẬT ĐƠN GIẢN

Hãy tưởng tượng bạn ngồi trước một cái bàn nhỏ, và cứ mỗi một phần ba mươi giây lại có người đặt thêm lên bàn một tấm ảnh chụp cùng một hành lang. Nhìn một tấm ảnh, bạn nói được “có một người” — nhưng để nói “người đó đang bước tới” hoặc “vẫn là người lúc này chứ không phải người khác”, bạn buộc phải đặt vài tấm cạnh nhau và so.

Mẹo thì đơn giản: so từng chỗ trên tấm này với chỗ tương ứng trên tấm kia. Hoặc so từng điểm nhỏ, để biết mọi thứ dịch đi đâu; hoặc so cả một vùng, để nói được “cái hộp này” ở tấm trước với “cái hộp kia” ở tấm sau. Việc nói đó, chứ không phải việc nhìn thấy, mới là phần khó.

Cái giá gồm ba khoản, và cả ba đều đắt. Thứ nhất, mặt bàn có hạn: muốn để bốn mươi tấm thay vì mười tấm thì phải in ảnh nhỏ lại, mà ảnh nhỏ thì mất những vật bé. Thứ hai, các tấm ảnh của tương lai chưa được chụp, nên bạn chỉ được dùng những gì đã có tới lúc này — mọi cách làm cần



“xem hết rồi mới nói” đều bị loại ngay. Thứ ba, đến khi bạn nói xong thì người trong ảnh đã bước thêm mấy bước, nên câu trả lời của bạn luôn nói về một quá khứ gần.

**Chỗ không rút gọn được:** ngưỡng chuyển động mà một cách nối bắt đầu sai phụ thuộc vào kết cấu bề mặt, độ mờ và cách phơi sáng của đúng cái camera bạn dùng, nên không có con số chung để nhớ — chỗ này bắt buộc phải tự đo.

**Một câu:** Thêm thời gian vào ảnh không tặng bạn một chiều dữ liệu, nó gửi cho bạn ba hoá đơn: bộ nhớ nhân lên, câu trả lời luôn muộn, và phần khó nhất là nối đúng vật ở tấm này với vật ở tấm kia.

## Tổng kết chương

Ba vấn đề mà chiều thời gian sinh ra hoá ra có ba câu trả lời rất khác nhau về bản chất. Đầu ra độ dài thay đổi được giải bằng một *mẫu thiết kế*: giữ nguyên nguồn thông tin và hoãn việc nén lại cho tới khi có truy vấn — và mẫu thiết kế đó, dưới cái tên attention, sau này chiếm lĩnh cả lĩnh vực. Chi phí bộ nhớ không có lời giải, chỉ có kế toán: tích của độ dài clip, độ phân giải bình phương và batch size gần như là hằng số, nên mọi kiến trúc là một cách chọn hy sinh. Ràng buộc nhân quả cũng không có lời giải, nó chỉ đơn giản loại bỏ phương án — và loại bỏ trước khi ta kịp bàn tới độ chính xác.

Bài học chung nhất thì nằm ở phần liên kết. Trong optical flow cũng như trong tracking, phần khó không phải nhìn thấy mà là ghép cặp, và trong cả hai lĩnh vực đều tồn tại một thước đo phổ biến che giấu đúng phần khó đó. Đây là biến thể của bài học ở chương 14: chi tiết không được viết trong abstract thường quyết định kết quả. Ở đây nó xuất hiện dưới dạng thước đo, không phải dưới dạng cách gán mẫu. Thói quen cần mang theo là như nhau: trước khi tin một con số, hãy hỏi nó cộng những loại lỗi lại với nhau và với trọng số bao nhiêu.

Còn bỏ ngỏ hai hướng. Thứ nhất, toàn chương này làm việc trên mặt phẳng ảnh; khi bài toán tương ứng điểm được nâng lên không gian ba chiều với ràng buộc epipolar và độ sâu, ta bước sang [C/16-thi-giac-3d](#) — và đó là chỗ mà nền hình học của bạn phát huy trực tiếp nhất. Thứ hai, mọi mô hình ở đây đều cần nhãn thời gian đắt đỏ; [C/18-pretraining-va-foundation](#) bàn về việc lấy giám sát từ chính cấu trúc thời gian của video, thứ có sẵn miễn phí.

## ❏ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Ba vấn đề riêng của chiều thời gian — đầu ra độ dài thay đổi, chi phí bộ nhớ, ràng buộc nhân quả — có xung đột với nhau không? Chỉ ra một thiết kế mà chữa vấn đề này thì làm nặng thêm vấn đề kia.
2. Mục 1 sinh ra attention để tránh nghẽn “nén trước, dùng sau”, còn mục 2 cho thấy chính attention nổ chi phí trên clip dài. Hai mục vì thế đẩy về hai hướng ngược nhau; bạn hoà giải thế nào cho một mô hình chạy trực tuyến?
3. Tương ứng điểm và liên kết dữ liệu là hai bài toán mà một hệ SLAM đã giải ở mỗi khung hình. Với mỗi bài, phần nào của từ vựng SLAM dịch thẳng sang được và phần nào không, và chỗ không dịch được bắt nguồn từ giả định nào?
4. Nhấp nháy không xuất hiện trong mIoU, đổi danh tính gần như không xuất hiện trong MOTA, và độ trễ không xuất hiện trong “độ chính xác trên khung  $t$ ”. Ba lỗ hổng đó có chung một nguyên nhân về cách xây thước đo không, và nó là gì?
5. Cửa sổ trượt, trạng thái hồi quy và lấy mẫu thưa theo thời gian là ba cách trả giá cho việc kéo dài clip. Chúng hỏng theo ba kiểu khác nhau nào, và loại chuyển động nào phơi bày từng kiểu?
6. Baseline chỉ nhìn một khung hình rồi gộp lại gần bằng mô hình thời gian đắt hơn nhiều. Trước khi kết luận rằng thời gian không quan trọng, bạn kiểm ba khả năng nào ở dữ liệu và ở thước đo?

- [1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [2] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [3] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *AISTATS*, 2010.
- [4] K. He, X. Zhang, S. Ren, et al., “Delving Deep into Rectifiers,” in *ICCV*, 2015.
- [5] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [6] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [7] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [8] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [9] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [10] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [11] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” in *ICLR*, 2017.
- [12] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [13] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [14] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [15] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [16] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [17] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [18] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [19] G. Huang, Y. Sun, Z. Liu, et al., “Deep Networks with Stochastic Depth,” in *ECCV*, 2016.
- [20] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [21] H. Zhang, M. Cisse, Y. N. Dauphin, et al., “mixup: Beyond Empirical Risk Minimization,” in *ICLR*, 2018.
- [22] S. Yun, D. Han, S. J. Oh, et al., “CutMix: Regularization Strategy to Train Strong Classifiers,” in *ICCV*, 2019.
- [23] E. D. Cubuk, B. Zoph, J. Shlens, et al., “RandAugment: Practical Automated Data Augmentation,” in *CVPR Workshops*, 2020.

- [24] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [25] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [26] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *NeurIPS*, 2012.
- [27] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in *ICLR*, 2015.
- [28] C. Szegedy, W. Liu, et al., “Going Deeper with Convolutions,” in *CVPR*, 2015.
- [29] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [30] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [31] G. Huang, Z. Liu, L. van der Maaten, et al., “Densely Connected Convolutional Networks,” in *CVPR*, 2017.
- [32] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [33] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [34] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [35] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [36] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [37] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.
- [38] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [39] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [40] J. Su, M. Ahmed, Y. Lu, et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, vol. 568, 2024.
- [41] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” *arXiv:2312.00752*, 2023.
- [42] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [43] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.
- [44] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [45] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [46] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [47] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [48] R. Girshick, J. Donahue, T. Darrell, et al., “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation,” in *CVPR*, 2014.

- [49] R. Girshick, "Fast R-CNN," in *ICCV*, 2015.
- [50] S. Ren, K. He, R. Girshick, et al., "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in *NeurIPS*, 2015.
- [51] T.-Y. Lin, P. Dollár, R. Girshick, et al., "Feature Pyramid Networks for Object Detection," in *CVPR*, 2017.
- [52] K. He, G. Gkioxari, P. Dollár, et al., "Mask R-CNN," in *ICCV*, 2017.
- [53] D. Bolya, S. Foley, J. Hays, et al., "TIDE: A General Toolbox for Identifying Object Detection Errors," in *ECCV*, 2020.
- [54] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [55] J. Redmon, S. Divvala, R. Girshick, et al., "You Only Look Once: Unified, Real-Time Object Detection," in *CVPR*, 2016.
- [56] W. Liu, D. Anguelov, D. Erhan, et al., "SSD: Single Shot MultiBox Detector," in *ECCV*, 2016.
- [57] T.-Y. Lin, P. Goyal, R. Girshick, et al., "Focal Loss for Dense Object Detection," in *ICCV*, 2017.
- [58] Z. Tian, C. Shen, H. Chen, et al., "FCOS: Fully Convolutional One-Stage Object Detection," in *ICCV*, 2019.
- [59] X. Zhou, D. Wang, and P. Krähenbühl, "Objects as Points," *arXiv:1904.07850*, 2019.
- [60] S. Zhang, C. Chi, Y. Yao, et al., "Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection," in *CVPR*, 2020.
- [61] Z. Ge, S. Liu, Z. Li, et al., "OTA: Optimal Transport Assignment for Object Detection," in *CVPR*, 2021.
- [62] N. Bodla, B. Singh, R. Chellappa, et al., "Soft-NMS: Improving Object Detection with One Line of Code," in *ICCV*, 2017.
- [63] X. Zhu, W. Su, L. Lu, et al., "Deformable DETR: Deformable Transformers for End-to-End Object Detection," in *ICLR*, 2021.
- [64] H. Zhang, F. Li, S. Liu, et al., "DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection," in *ICLR*, 2023.
- [65] J. Long, E. Shelhamer, and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," in *CVPR*, 2015.
- [66] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *MICCAI*, 2015.
- [67] L.-C. Chen, G. Papandreou, I. Kokkinos, et al., "DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs," *IEEE TPAMI*, vol. 40, no. 4, pp. 834–848, 2018.
- [68] E. Xie, W. Wang, Z. Yu, et al., "SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers," in *NeurIPS*, 2021.
- [69] B. Cheng, I. Misra, A. G. Schwing, et al., "Masked-Attention Mask Transformer for Universal Image Segmentation," in *CVPR*, 2022.
- [70] A. Kirillov, K. He, R. Girshick, et al., "Panoptic Segmentation," in *CVPR*, 2019.
- [71] A. Kirillov, E. Mintun, N. Ravi, et al., "Segment Anything," in *ICCV*, 2023.
- [72] K. Sun, B. Xiao, D. Liu, et al., "Deep High-Resolution Representation Learning for Human Pose Estimation," in *CVPR*, 2019.

- [73] Z. Cao, T. Simon, S.-E. Wei, et al., “Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields,” in *CVPR*, 2017.
- [74] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [75] J. Deng, J. Guo, N. Xue, et al., “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *CVPR*, 2019.
- [76] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [77] K. Xu, J. Ba, R. Kiros, et al., “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention,” in *ICML*, 2015.
- [78] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *ICLR*, 2015.
- [79] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [80] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [81] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [82] K. Simonyan and A. Zisserman, “Two-Stream Convolutional Networks for Action Recognition in Videos,” in *NeurIPS*, 2014.
- [83] J. Carreira and A. Zisserman, “Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset,” in *CVPR*, 2017.
- [84] D. Tran, H. Wang, L. Torresani, et al., “A Closer Look at Spatiotemporal Convolutions for Action Recognition,” in *CVPR*, 2018.
- [85] C. Feichtenhofer, H. Fan, J. Malik, et al., “SlowFast Networks for Video Recognition,” in *ICCV*, 2019.
- [86] A. Dosovitskiy, P. Fischer, E. Ilg, et al., “FlowNet: Learning Optical Flow with Convolutional Networks,” in *ICCV*, 2015.
- [87] D. Sun, X. Yang, M.-Y. Liu, et al., “PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume,” in *CVPR*, 2018.
- [88] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.
- [89] A. Bewley, Z. Ge, L. Ott, et al., “Simple Online and Realtime Tracking,” in *ICIP*, 2016.
- [90] Y. Zhang, P. Sun, Y. Jiang, et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *ECCV*, 2022.
- [91] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [92] K. Bernardin and R. Stiefelhagen, “Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics,” *EURASIP J. Image and Video Processing*, 2008.
- [93] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [94] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [95] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.



- [96] S. Wang, V. Leroy, Y. Cabon, et al., "DUST3R: Geometric 3D Vision Made Easy," in *CVPR*, 2024.
- [97] D. Eigen, C. Puhrsch, and R. Fergus, "Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network," in *NeurIPS*, 2014.
- [98] R. Ranftl, K. Lasinger, D. Hafner, et al., "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer," *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [99] J. Sturm, N. Engelhard, F. Endres, et al., "A Benchmark for the Evaluation of RGB-D SLAM Systems," in *IROS*, 2012.
- [100] C. R. Qi, H. Su, K. Mo, et al., "PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation," in *CVPR*, 2017.
- [101] C. R. Qi, L. Yi, H. Su, et al., "PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space," in *NeurIPS*, 2017.
- [102] A. H. Lang, S. Vora, H. Caesar, et al., "PointPillars: Fast Encoders for Object Detection from Point Clouds," in *CVPR*, 2019.
- [103] T. Müller, A. Evans, C. Schied, et al., "Instant Neural Graphics Primitives with a Multiresolution Hash Encoding," *ACM TOG*, vol. 41, no. 4, 2022.
- [104] B. Kerbl, G. Kopanas, T. Leimkühler, et al., "3D Gaussian Splatting for Real-Time Radiance Field Rendering," *ACM TOG*, vol. 42, no. 4, 2023.
- [105] R. Zhang, P. Isola, A. A. Efros, et al., "The Unreasonable Effectiveness of Deep Features as a Perceptual Metric," in *CVPR*, 2018.
- [106] D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," in *ICLR*, 2014.
- [107] I. Goodfellow, J. Pouget-Abadie, M. Mirza, et al., "Generative Adversarial Nets," in *NeurIPS*, 2014.
- [108] T. Karras, S. Laine, and T. Aila, "A Style-Based Generator Architecture for Generative Adversarial Networks," in *CVPR*, 2019.
- [109] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," in *NeurIPS*, 2020.
- [110] Y. Song, J. Sohl-Dickstein, D. P. Kingma, et al., "Score-Based Generative Modeling Through Stochastic Differential Equations," in *ICLR*, 2021.
- [111] J. Johnson, A. Alahi, and L. Fei-Fei, "Perceptual Losses for Real-Time Style Transfer and Super-Resolution," in *ECCV*, 2016.
- [112] P. Isola, J.-Y. Zhu, T. Zhou, et al., "Image-to-Image Translation with Conditional Adversarial Networks," in *CVPR*, 2017.
- [113] L. A. Gatys, A. S. Ecker, and M. Bethge, "Image Style Transfer Using Convolutional Neural Networks," in *CVPR*, 2016.
- [114] X. Huang and S. Belongie, "Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization," in *ICCV*, 2017.
- [115] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," in *NeurIPS Deep Learning Workshop*, 2015.
- [116] J. Song, C. Meng, and S. Ermon, "Denoising Diffusion Implicit Models," in *ICLR*, 2021.
- [117] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, et al., "Flow Matching for Generative Modeling," in *ICLR*, 2023.
- [118] J. Ho and T. Salimans, "Classifier-Free Diffusion Guidance," *arXiv:2207.12598*, 2022.
- [119] R. Rombach, A. Blattmann, D. Lorenz, et al., "High-Resolution Image Synthesis with Latent Diffusion Models," in *CVPR*, 2022.
- [120] W. Peebles and S. Xie, "Scalable Diffusion Models with Transformers," in *ICCV*, 2023.



- [121] L. Zhang, A. Rao, and M. Agrawala, “Adding Conditional Control to Text-to-Image Diffusion Models,” in *ICCV*, 2023.
- [122] Y. Blau and T. Michaeli, “The Perception-Distortion Tradeoff,” in *CVPR*, 2018.
- [123] C. Ledig, L. Theis, F. Huszár, et al., “Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network,” in *CVPR*, 2017.
- [124] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., “Image Quality Assessment: From Error Visibility to Structural Similarity,” *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [125] M. Heusel, H. Ramsauer, T. Unterthiner, et al., “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium,” in *NeurIPS*, 2017.
- [126] T. Brooks, B. Mildenhall, T. Xue, et al., “Unprocessing Images for Learned Raw Denoising,” in *CVPR*, 2019.
- [127] A. Abdelhamed, S. Lin, and M. S. Brown, “A High-Quality Denoising Dataset for Smartphone Cameras,” in *CVPR*, 2018.
- [128] J. Deng, W. Dong, R. Socher, et al., “ImageNet: A Large-Scale Hierarchical Image Database,” in *CVPR*, 2009.
- [129] A. Radford, J. W. Kim, C. Hallacy, et al., “Learning Transferable Visual Models from Natural Language Supervision,” in *ICML*, 2021.
- [130] J. Li, D. Li, S. Savarese, et al., “BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models,” in *ICML*, 2023.
- [131] H. Liu, C. Li, Q. Wu, et al., “Visual Instruction Tuning,” in *NeurIPS*, 2023.
- [132] S. Kornblith, J. Shlens, and Q. V. Le, “Do Better ImageNet Models Transfer Better?” In *CVPR*, 2019.
- [133] T. Chen, S. Kornblith, M. Norouzi, et al., “A Simple Framework for Contrastive Learning of Visual Representations,” in *ICML*, 2020.
- [134] K. He, H. Fan, Y. Wu, et al., “Momentum Contrast for Unsupervised Visual Representation Learning,” in *CVPR*, 2020.
- [135] J.-B. Grill, F. Strub, F. Altché, et al., “Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning,” in *NeurIPS*, 2020.
- [136] M. Caron, H. Touvron, I. Misra, et al., “Emerging Properties in Self-Supervised Vision Transformers,” in *ICCV*, 2021.
- [137] K. He, X. Chen, S. Xie, et al., “Masked Autoencoders Are Scalable Vision Learners,” in *CVPR*, 2022.
- [138] A. van den Oord, Y. Li, and O. Vinyals, “Representation Learning with Contrastive Predictive Coding,” in *arXiv:1807.03748*, 2018.
- [139] J. Kaplan, S. McCandlish, T. Henighan, et al., “Scaling Laws for Neural Language Models,” *arXiv:2001.08361*, 2020.
- [140] X. Zhai, A. Kolesnikov, N. Houlsby, et al., “Scaling Vision Transformers,” in *CVPR*, 2022.
- [141] J. Hoffmann, S. Borgeaud, A. Mensch, et al., “Training Compute-Optimal Large Language Models,” in *NeurIPS*, 2022.
- [142] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [143] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [144] N. Houlsby, A. Giurgiu, S. Jastrzebski, et al., “Parameter-Efficient Transfer Learning for NLP,” in *ICML*, 2019.

- [145] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [146] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *RecSys*, 2019.

## Thị giác 3D và hình học (3D Vision and Geometry)

**PHỤ THUỘC** A/02-vat-ly-cao-anh/03-camera-va-hinh-hoc-nhieu-view (mô hình chiếu, hiệu chỉnh); B/04-nen-tang-hoc-tu-du-lieu (đánh đổi giả định  $\leftrightarrow$  dữ liệu); B/07-chia-du-lieu-va-danh-gia (giao thức đánh giá, ATE/RPE); C/13-kien-truc-backbone (transformer, thiên lệch quy nạp).

### TÓM TẮT

Đây là lĩnh vực có mạch lịch sử rõ nhất trong computer vision: cùng một bài toán — suy hình dạng 3D và pose camera từ ảnh 2D — được giải lại từ đầu bốn lần, mỗi lần đổi một loại giả định lấy một loại độ bền. Chương này dựng khung bốn thể hệ đó, rồi đi vào các quyết định con (biểu diễn, độ sâu, SLAM, học trên point cloud) và hai thứ mới nhất là *neural rendering* và hợp nhất cảm biến. Đọc xong bạn chọn được thể hệ đúng cho dữ liệu đang có và biết cách phát hiện lúc một mô hình học đang nói dối về hình học. Nếu chỉ nhớ một điều: mỗi lần bỏ bớt một giả định về thế giới, bạn phải bù bằng dữ liệu, và bạn mất một phần khả năng tự chẩn đoán. Đó là cái giá ít được nói tới nhất của thị giác 3D hiện đại.

### KIẾN THỨC CẦN CÓ TRƯỚC

- **thị giác 3D (3D vision)** — nhánh của computer vision lo việc suy hình dạng ba chiều và chuyển động camera từ ảnh hai chiều — ngược lại với nhánh nhận dạng, vốn chỉ hỏi “trong ảnh có gì”.
- **phép chiếu phối cảnh (perspective projection)** — cách camera biến một điểm 3D thành một pixel: chia toạ độ ngang và dọc cho độ sâu. Chính phép chia này làm mất chiều thứ ba.
- **prior (prior)** — những gì mô hình “tin sẵn” về thế giới trước khi nhìn dữ liệu hiện tại — có thể do người viết cài cứng (mô hình vật lý) hoặc do mạng học từ dữ liệu lớn.
- **metric (đơn vị mét)** — một kết quả gọi là metric khi nó cho ra số đo tuyệt đối tính bằng mét, chứ không chỉ đúng sai khác một hệ số tỉ lệ chưa biết.
- **SLAM và VIO** — hai hệ thống ước lượng vị trí robot theo thời gian thực trong lúc dựng bản đồ, dùng camera (SLAM) hoặc camera cộng cảm biến quán tính (VIO). Chương này giả sử bạn đã quen chúng và chỉ đặt chúng vào bức tranh chung.
- **neural rendering** — họ phương pháp học một biểu diễn cảnh bằng cách bắt nó render ra ảnh giống các ảnh đã chụp, thay vì bằng cách đo hình học trực tiếp.

### CÂU HỎI MỞ ĐẦU

- Q1. Vì sao thêm bao nhiêu dữ liệu cũng không cho một camera đơn biết vật thể trước mặt lớn thật sự bao nhiêu mét?
- Q2. Nếu một *transformer* suy được pose và hình học chỉ trong một lượt xuôi, vì sao COLMAP vẫn chưa bị thay thế?
- Q3. Vì sao NeRF phải mô tả cảnh bằng khói mờ thay vì bằng bề mặt cứng, trong khi thứ tồn tại thật là bề mặt?
- Q4. Vì sao một hệ hợp nhất camera–LiDAR có thể chạy hoàn hảo lúc robot đứng yên rồi hỏng đúng lúc nó xoay nhanh, dù không một tham số nào thay đổi?

Vì sao chương này chia làm ba mục

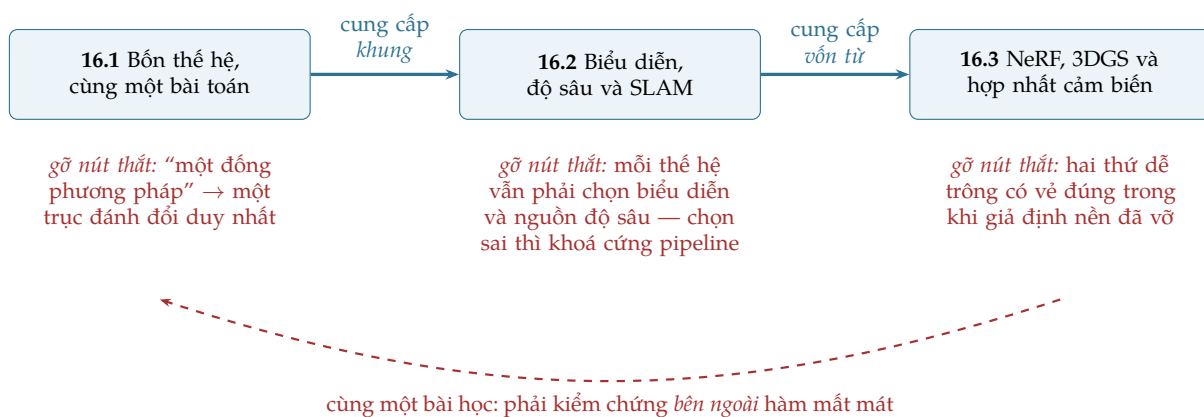
Chương được chia theo ba tầng câu hỏi, và thứ tự là thứ tự phụ thuộc chứ không phải thứ tự lịch sử.

Mục thứ nhất dựng *khung*: bốn thể hệ giải cùng một bài toán, mỗi thể hệ thay một đoạn khác nhau của cùng một pipeline. Không có khung này thì mọi thứ còn lại rơi vào tình trạng “một đồng phương pháp”, và ta không trả lời được câu hỏi thực dụng nhất — với dữ liệu đang có thì nên dùng cái gì. Khung này cũng là chỗ trực đánh đổi “giả định mạnh + ít dữ liệu  $\leftrightarrow$  giả định yếu + nhiều dữ liệu” của chương 4 hiện ra rõ nhất trong toàn bộ CV.

Mục thứ hai đi vào các *quyết định con* mà mọi thể hệ đều phải đưa ra: lưu cảnh bằng biểu diễn nào, lấy độ sâu từ đâu và cái gì cố định tỉ lệ, SLAM đứng ở đâu trong bức tranh, và làm sao đưa mạng nơ-ron lên một tập điểm không thứ tự. Đây là tầng “vốn từ”: không nắm nó thì không đọc được mục ba.

Mục thứ ba đào sâu đúng hai thứ xa nền tảng hình học cổ điển nhất — neural rendering (NeRF, 3DGS) và hợp nhất camera–LiDAR–IMU. Hai chủ đề này được gộp vào một mục vì chúng chia chung một bài học: cả hai đều dễ trông có vẻ hoạt động trong khi giả định nền đã bị vi phạm, và cả hai đều đòi một cơ chế kiểm chứng *bên ngoài* hàm mất mát của chính chúng.

Hình 5.1 vẽ ba mục đó cùng nút thắt mà mỗi mục gỡ.



**Hình 5.1.** Ba mục của chương và quan hệ phụ thuộc giữa chúng. Thứ tự là thứ tự *phụ thuộc*, không phải thứ tự lịch sử: mục 16.1 dựng khung để hai mục sau không biến thành danh sách phương pháp rời rạc, mục 16.2 cung cấp vốn từ cho mục 16.3. Mũi tên đứt phía dưới là điều mà cả ba mục cùng dẫn tới, và cũng là điều dễ bị bỏ sót nhất khi đọc riêng từng mục.

Với người đọc đang làm visual-inertial SLAM thật, ba mục này có mật độ thông tin mới rất khác nhau: mục một chủ yếu sắp xếp lại thứ đã biết, mục hai bổ sung phần học sâu trên dữ liệu 3D, còn mục ba mới là phần thực sự mới. Nếu thời gian eo hẹp, đọc mục một để lấy khung rồi nhảy thẳng sang mục ba.

## Bản đồ chương 16 — Thị giác 3D và hình học

**TÓM TẮT**

Chương 16 gồm ba mục, chia theo ba tầng câu hỏi chứ không theo thứ tự lịch sử: khung bốn thể hệ, các quyết định con, rồi hai chủ đề mới nhất. Bản đồ này nói mục nào là cốt lõi, mục nào có thể tra khi cần, và đọc theo trình tự nào tùy theo nền sẵn có của bạn. Nếu chỉ nhớ một điều: mục 16.1 là khung để đọc hai mục còn lại — bỏ qua nó thì phần sau biến thành một danh sách phương pháp rời rạc.

**KIẾN THỨC CẦN CÓ TRƯỚC**

- **SfM / MVS** — hai bước của pipeline hình học cổ điển: suy pose camera và cấu trúc thưa từ ảnh (SfM), rồi dựng bề mặt dày đặc khi đã biết pose (MVS).
- **neural rendering** — học một biểu diễn cảnh bằng cách bắt nó dựng ra ảnh giống các ảnh đã chụp, thay vì đo hình học trực tiếp.
- **feed-forward geometry** — mô hình suy pose và hình học trong *một* lần chạy mạng, không cần vòng tối ưu và không cần pose cho trước.
- **point cloud / voxel / mesh** — ba cách lưu hình dạng 3D: tập điểm rời rạc, lưới ô đều, và bề mặt tam giác.

**Ba mục và vai trò của từng mục**

- **16.1 — Bốn thể hệ, cùng một bài toán (cốt lõi).** Khung của cả chương: hình học cổ điển, lai học-hình học, neural rendering, feed-forward geometry. Trục đánh đổi “giả định mạnh + ít dữ liệu ↔ giả định yếu + nhiều dữ liệu”. Đọc mục này trước mọi thứ khác.
- **16.2 — Chọn biểu diễn 3D, độ sâu và SLAM (cốt lõi phần biểu diễn và độ sâu; phần SLAM là tham khảo với người đã làm SLAM).** Vốn từ: voxel, point cloud, mesh, trường ẩn, Gaussian; bốn nguồn độ sâu và cái gì cố định tỉ lệ; PointNet và bất biến hoán vị.
- **16.3 — NeRF, 3DGS và hợp nhất cảm biến (cốt lõi).** Cơ chế dựng ảnh thể tích, vì sao phải dùng mật độ liên tục, 3DGS đổi gì lấy gì, ba giả định nền dễ vỡ; early đối lại late fusion và ngân sách sai số hiệu chỉnh.

**Trình tự đọc**

1. Nếu bạn **chưa có nền hình học nhiều view**: đọc mục *camera và hình học nhiều view* ở A/02-vat-ly-tao-anh trước, rồi 16.1 → 16.2 → 16.3 theo đúng thứ tự.
2. Nếu bạn **đã làm SLAM hoặc SfM**: đọc 16.1 để lấy khung (nhánh, phần lớn là sắp xếp lại thứ đã biết), lướt mục 16.2 phần D về SLAM, đọc kỹ 16.2 phần C và E (độ sâu học được, point cloud) và toàn bộ 16.3 — đó là phần có mật độ thông tin mới cao nhất với bạn.
3. Khối *Thực hành* nằm ở file chương, sau ba mục.

**Chương này phụ thuộc gì và được dùng lại ở đâu**

- **Phụ thuộc lên trên:** A/02-vat-ly-tao-anh (mô hình chiếu, hiệu chỉnh, phơi sáng); B/04-nen-tang-hoc-tu-du-lieu (đánh đổi giả định ↔ dữ liệu, tính xác định được); B/07-chia-du-lieu-va-danh-gia (ATE/RPE, giao thức); C/13-kien-truc-backbone (transformer).
- **Được dùng lại ở dưới:** C/17-mo-hinh-sinh/02-toi-uu-tren-pixel dùng lại nguyên ý tưởng tối ưu trên chính đối tượng cần tạo ra; C/17-mo-hinh-sinh/04-low-level-vision dùng lại

khái niệm bài toán ill-posed và vai trò của prior; E/25-do-tin-cay-va-van-hanh dùng lại yêu cầu “cơ chế phát hiện sai từ bên ngoài”.

## I 5.1 Bốn thế hệ, cùng một bài toán (Four Generations of 3D Reconstruction)

**PHỤ THUỘC** A/02-vat-ly-tao-anh/03-camera-va-hinh-hoc-nhieu-view (mô hình chiếu, ràng buộc epipolar); B/04-nen-tang-hoc-tu-du-lieu (trục “giả định mạnh + ít dữ liệu”  $\leftrightarrow$  “giả định yếu + nhiều dữ liệu”); B/03-giai-phau-he-thong (tách bài toán  $\rightarrow$  nguyên lý  $\rightarrow$  cài đặt).

### TÓM TẮT

Bài toán khôi phục hình học 3D từ ảnh đã được giải lại từ đầu bốn lần. Bốn thế hệ đó không thay thế nhau: mỗi thế hệ thay một *đoạn khác nhau* của cùng một pipeline. Vì vậy chúng chồng lớp được, và câu hỏi “VGGT có thay được COLMAP không” là câu hỏi sai. Đọc xong bạn làm được ba việc. Chọn thế hệ đúng cho dữ liệu đang có: bao nhiêu ảnh, có pose hay không, cảnh loại gì. Ghép hai thế hệ thành một pipeline, một bên khởi tạo một bên tinh chỉnh. Và kiểm tra được lúc một mô hình tiến thẳng đang nói dối. Nếu chỉ nhớ một điều: đi từ thế hệ một tới thế hệ bốn, thứ mất đi không chỉ là độ chính xác metric mà là *khả năng tự chẩn đoán*. Hiệu chỉnh chùm tia trả về phần dư và hiệp phương sai; mạng tiến thẳng không trả về gì cả. Vì vậy đưa nó vào hệ thống thật là phải tự dựng lại cơ chế phát hiện sai từ bên ngoài.

### KIẾN THỨC CẦN CÓ TRƯỚC

- **SfM (structure from motion)** — quy trình cổ điển suy đồng thời hình dạng 3D của cảnh và vị trí camera từ một tập ảnh chụp cùng cảnh ở nhiều góc.
- **MVS (multi-view stereo)** — bước sau SfM: khi đã biết pose camera, dựng bề mặt dày đặc bằng cách khớp từng vùng ảnh giữa nhiều view.
- **pose camera (camera pose)** — vị trí và hướng của camera trong không gian, sáu bậc tự do; “biết pose” là biết ảnh được chụp từ đâu, nhìn về đâu.
- **tam giác hoá (triangulation)** — tìm điểm 3D bằng giao của hai hay nhiều tia nhìn xuất phát từ các camera đã biết pose; đây là chỗ chiều bị mất khi chiếu được lấy lại.
- **hiệu chỉnh chùm tia (bundle adjustment)** — bài toán bình phương tối thiểu phi tuyến. Nó tinh chỉnh đồng thời mọi pose và mọi điểm 3D sao cho sai số chiếu lại nhỏ nhất. Bạn đã quen nó; điều cần nhớ ở đây là nó trả về *phần dư* và *hiệp phương sai*.
- **RANSAC** — thủ tục lọc điểm sai bằng cách thử ngẫu nhiên nhiều giả thuyết nhỏ rồi giữ giả thuyết được nhiều dữ liệu ủng hộ nhất. Nó giả định điểm sai là *ngẫu nhiên*, không phải sai đồng loạt theo một kiểu.
- **point map** — ảnh mà mỗi pixel chứa một toạ độ 3D thay vì một màu; đây là dạng đầu ra của các mô hình tiến thẳng hiện đại, thay cho “point cloud rời rạc”.
- **disparity** — độ lệch ngang của cùng một điểm giữa ảnh trái và ảnh phải trong hệ stereo; nó tỉ lệ nghịch với độ sâu.

### A. Đặt vấn đề

Bài toán gốc của thị giác 3D không đổi trong bốn mươi năm: cho một tập ảnh 2D của cùng một cảnh, hãy khôi phục hình dạng ba chiều của cảnh và vị trí camera lúc chụp. Nó khó vì phép chiếu phối cảnh làm mất một chiều — mỗi pixel chỉ nói “có gì đó nằm đâu đó trên tia này”, chứ không nói nó xa bao nhiêu. Chỉ khi nhiều tia từ nhiều vị trí camera khác nhau cắt nhau thì độ sâu mới được xác định, và cả ngành 3D là các cách khác nhau để lấy lại thông tin đã mất đó.



Điều đáng học ở lĩnh vực này không phải thuật toán cụ thể mà là mạch lịch sử của nó: đây là chỗ duy nhất trong computer vision mà ta thấy một bài toán được giải lại từ đầu bốn lần, mỗi lần đổi một loại giả định lấy một loại độ bền. Thế hệ đầu viết thẳng mô hình vật lý vào bài toán tối ưu, nên chỉ cần vài chục ảnh nhưng đòi cảnh phải cư xử đúng như mô hình. Thế hệ cuối vứt gần hết mô hình đó đi. Nó thay bằng một *prior* học từ hàng triệu cảnh, nên chạy được ở chỗ mô hình vật lý bó tay. Cái giá là không còn cách nào kiểm chứng kết quả từ bên trong. Đây chính xác là trục “giả định mạnh + ít dữ liệu  $\leftrightarrow$  giả định yếu + nhiều dữ liệu” của chương 4, và không chỗ nào trong CV nó hiện ra sạch sẽ như ở đây.

$\leftrightarrow$  **trường hợp riêng của B/04** — bốn thế hệ 3D là hiện thân cụ thể nhất của trục “giả định mạnh + ít dữ liệu  $\leftrightarrow$  giả định yếu + nhiều dữ liệu”; ở đây ta đo được cái giá bằng centimét.

**Học mục này thì làm được gì mà trước đó không làm được?** Trả lời được ba câu hỏi thực dụng: (i) với dữ liệu bạn đang có — bao nhiêu ảnh, có pose hay không, cảnh loại gì — thế hệ nào là lựa chọn đúng; (ii) khi ghép hai thế hệ vào một pipeline thì ai làm khởi tạo, ai làm tinh chỉnh; (iii) làm thế nào phát hiện lúc một mô hình **tiền thẳng** (*feed-forward*) đang nói dối, việc mà **hiệu chỉnh chùm tia** (*bundle adjustment*) làm hộ bạn miễn phí còn mạng học thì không.

Với người đọc đang làm *visual-inertial SLAM*, ba thế hệ đầu phần lớn là sân nhà. Phần lạ nằm ở hai thế hệ sau — *neural rendering* và *feed-forward geometry* — nên chương này viết kỹ đúng hai chỗ đó.

#### TRỰC GIÁC

Bốn thế hệ là bốn cách trả lời câu hỏi “căn phòng này rộng bao nhiêu”. Thế hệ hình học mang thước ra đo từng cạnh rồi cộng sai số theo quy tắc rõ ràng: đúng tới milimét, và nếu các số đo mâu thuẫn nhau thì phần dư của phép bình phương tối thiểu tự nó tố cáo. Thế hệ lai vẫn dùng thước, nhưng nhờ mắt chỉ chỗ đặt thước cho nhanh. *Neural rendering* không đo phòng: nó dựng một mô hình phòng bằng sương mù có màu rồi chỉnh đám sương cho tới khi ảnh chụp mô hình trùng khít mọi ảnh thật — đúng về mặt *nhìn*, chưa chắc đúng về mặt *chạm*. Thế hệ tiền thẳng liếc một cái rồi nói “khoảng bốn nhân năm mét”, vì đã ở trong hàng vạn căn phòng: nhanh, thường đúng, và khi sai thì sai rất tự tin vì không có phép đo nào để mâu thuẫn với nó.

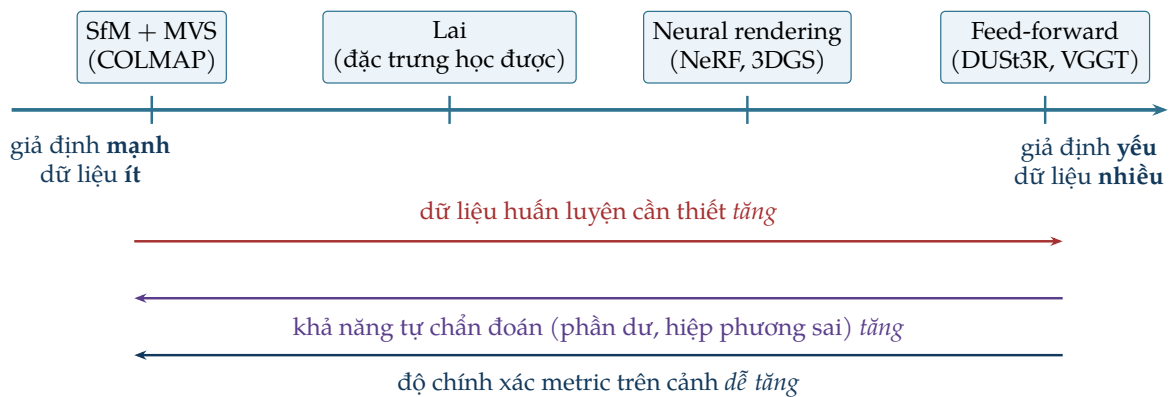
#### B. Bốn thế hệ đặt cạnh nhau

Bảng 5.1 đặt bốn thế hệ trên cùng bốn tiêu chí. Cột đáng đọc kỹ nhất là cột cuối: mỗi thế hệ ra đời vì thế hệ trước hỏng ở một chỗ cụ thể, rồi lại hỏng ở một chỗ khác.

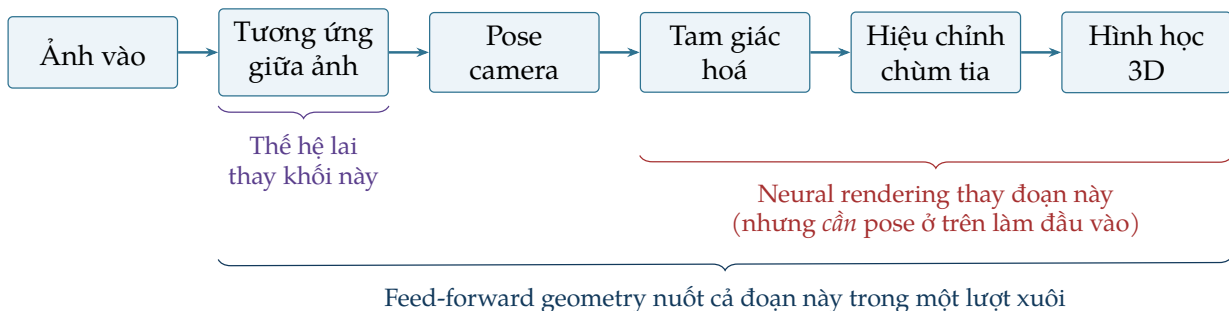
**Bảng 5.1.** Bốn thể hệ giải bài toán khôi phục hình học 3D từ ảnh. Không thể hệ nào thay thế hoàn toàn thể hệ trước; trên thực tế chúng chồng lên nhau.

| Thể hệ                               | Cơ chế                                                                      | Giả định                                              | Mạnh khi                                       | Hồng khi                                                            |
|--------------------------------------|-----------------------------------------------------------------------------|-------------------------------------------------------|------------------------------------------------|---------------------------------------------------------------------|
| Hình học cổ điển (SfM/MVS, COLMAP)   | Tìm tương ứng, ước lượng pose, tam giác hoá, hiệu chỉnh chùm tia            | Cảnh tĩnh, đủ texture, đủ chồng lấn giữa các view     | Cần độ chính xác metric và cần kiểm chứng được | Ít texture, ít ảnh, bề mặt phản chiếu hoặc trong suốt               |
| Lai học + hình học                   | Học phần tương ứng hoặc phần độ sâu, giữ nguyên phần tối ưu hình học        | Hình học vẫn đúng; chỉ khâu nhận dạng cần học         | Có prior mạnh nhưng vẫn cần ràng buộc cứng     | Khi khâu học sai một cách <i>có hệ thống</i> : RANSAC không lọc nổi |
| Neural rendering (NeRF, 3DGS)        | Tối ưu một biểu diễn riêng cho từng cảnh bằng loss render khả vi            | Đã biết pose; cảnh tĩnh; phơi sáng nhất quán          | Cần chất lượng ảnh ở góc nhìn mới              | Không có pose sẵn; phải tối ưu lại từ đầu cho mỗi cảnh              |
| Feed-forward geometry (DUST3R, VGGT) | Một <i>transformer</i> suy ra pose + độ sâu + point map trong một lượt xuôi | Prior học từ dữ liệu lớn khái quát được sang cảnh mới | Ít ảnh, không có pose, cần kết quả nhanh       | Cảnh dễ mà cần độ chính xác metric: thua hiệu chỉnh chùm tia        |

Bảng cho biết *cái gì*, hai hình sau cho biết *tại sao*. Hình 5.2 đặt bốn thể hệ lên một trục duy nhất và cho thấy ba đại lượng cùng biến thiên theo trục đó; Hình 5.3 cho thấy mỗi thể hệ thay một *đoạn* khác nhau của cùng một pipeline. Hình thứ hai giải thích vì sao câu hỏi “VGGT có thay được COLMAP không” là câu hỏi sai — câu hỏi đúng là “thay được đoạn nào”.



**Hình 5.2.** Một trục duy nhất giải thích cả bốn thể hệ. Đi sang phải, ta bỏ bớt giả định về thể giới và bù bằng dữ liệu; đổi lại, hai thứ cùng giảm — độ chính xác metric ở cảnh dễ, và quan trọng hơn, khả năng hệ thống tự biết mình đang sai.



**Hình 5.3.** Cùng một pipeline, ba cách thay thế khác nhau. Thể hệ lai thay đúng khối dễ hỏng nhất mà giữ nguyên phần tối ưu; neural rendering nằm *sau* khâu pose nên nó là khách hàng của SfM chứ không phải đối thủ; chỉ thể hệ tiến thẳng mới thực sự cạnh tranh trực tiếp với COLMAP.

## C. Vì sao thế hệ sau ra đời

### I Mạch 1 — bốn thế hệ, cùng một bài toán

- 1 Vấn đề** ảnh mất một chiều, và không có cách nào lấy lại nó từ một ảnh đơn lẻ.

**Giải pháp** viết thẳng mô hình chiếu vào một bài toán bình phương tối thiểu: tìm tương ứng giữa các ảnh, ước lượng pose tương đối, tam giác hoá điểm, rồi tinh chỉnh toàn bộ bằng hiệu chỉnh chùm tia [93], [94].

**Giả định** cảnh tĩnh, bề mặt *Lambertian* đủ texture, nhiễu đo là Gauss độc lập.

**Hệ quả** mô hình sinh ảnh đã được cài cứng, nên phương pháp cần rất ít dữ liệu: vài chục ảnh là ra kết quả metric. Quan trọng hơn, nó trả về *phần dư* cùng *ma trận hiệp phương sai*, tức là tự nói cho ta biết nó tin kết quả tới đâu.
- 2 Vấn đề** khâu yếu nhất của thế hệ một là tìm tương ứng: nó dựa vào *descriptor* thủ công, và descriptor thủ công chết ở thay đổi ánh sáng lớn, góc nhìn lệch nhiều, vùng hoa văn lặp lại.

**Giải pháp** thay đúng khối đó bằng một mạng học — detector/descriptor học được, matcher học được, hoặc **khớp dày đặc** (*dense matching*) — và giữ nguyên phần tối ưu hình học phía sau.

**Giả định** hình học vẫn đúng; chỉ phần “nhận dạng” mới cần học.

**Giá phải trả** mất tính tái lập bit-for-bit và mất khả năng suy luận giải tích về sai số của khâu đầu.

**Vấn đề mới** mạng học sai theo kiểu *tương quan*. Ở một mặt tường lặp hoa văn, nó không cho ra *outlier* ngẫu nhiên mà cho ra một tập tương ứng sai *nhất quán*. RANSAC không lọc được loại lỗi đó, vì nó giả định outlier là ngẫu nhiên.

↔ **hệ quả của** B/08 — sai đồng loạt của khâu học là lệch phân phối biểu hiện ở tầng hình học; đó là lý do một bộ lọc thống kê không cứu được nó.
- 3 Vấn đề** ngay cả khi hình học đúng, đầu ra của SfM/MVS là point cloud hoặc mesh — render lại thì thùng lỗ, không có hiệu ứng phụ thuộc góc nhìn.

**Giải pháp** bỏ luôn ý định khôi phục bề mặt; tham số hoá cảnh bằng một trường liên tục và tối ưu trường đó sao cho ảnh render khớp mọi ảnh quan sát [95].

**Giả định** pose đã biết trước, cảnh tĩnh, phơi sáng và cân bằng trắng nhất quán.

**Hệ quả** chất lượng ảnh ở góc nhìn mới nhảy vọt.

**Giá phải trả** hàm mất mát chỉ ràng buộc *ảnh render*, không ràng buộc *hình học*.

**Vấn đề mới** mỗi cảnh phải tối ưu lại từ đầu, và pose vẫn phải xin từ thế hệ một.
- 4 Vấn đề** cả ba thế hệ trên đều gậy ở cùng một điều kiện: quá ít ảnh, hoặc chồng lấn không đủ để tam giác hoá. Ba tấm ảnh chụp ba góc một căn phòng, mỗi cặp chồng lấn 20 %, thì COLMAP thường không khởi tạo nổi.

**Giải pháp** huấn luyện một transformer trên rất nhiều cảnh có chân trị 3D, cho nó nhận thẳng một tập ảnh không pose và xuất ra *point map* trong một hệ toạ độ chung [96]; dòng kế tiếp (DUST3R → MAST3R → VGGT) mở rộng từ hai ảnh lên hàng chục ảnh trong một lượt xuôi.

**Giả định** cảnh mới nằm trong phân phối mà prior đã học.

**Hệ quả** bài toán đổi bản chất: từ ước lượng có ràng buộc cứng thành hồi quy.

**Giá phải trả** độ chính xác metric ở cảnh dễ thua hiệu chỉnh chùm tia, và — điểm quan trọng nhất — không còn phần dư hình học để phát hiện lúc nó sai.

Đọc theo mạch này thì thấy cả bốn bước là cùng một phép đánh đổi lặp lại: mỗi lần bỏ bớt một giả định về thế giới, ta phải bù bằng dữ liệu, và mất đi một phần khả năng tự kiểm tra — đúng ba mũi tên của Hình 5.2.

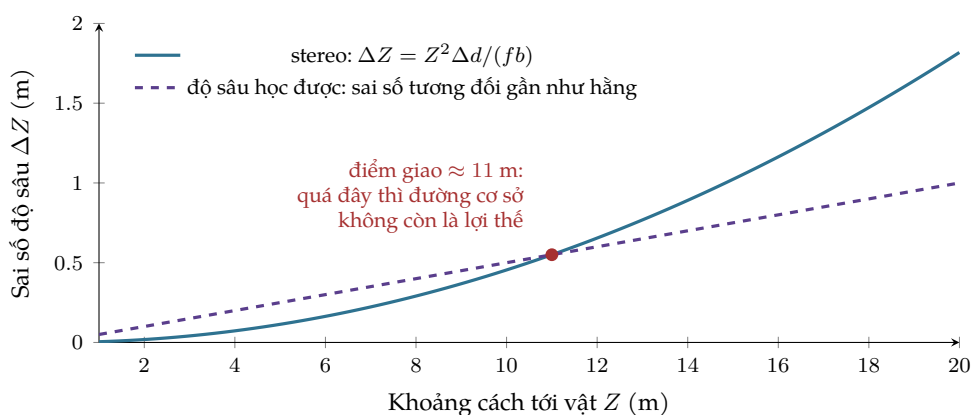
#### D. Một phép tính nhỏ: vì sao “chính xác metric” là đặc sản của thế hệ một

Nói gọn thì: sai số độ sâu của phương pháp hình học có một *công thức*, còn sai số của mạng học thì không. Với một cặp *stereo* đã hiệu chỉnh, độ sâu suy ra từ **độ lệch ngang** (*disparity*) theo  $Z = fb/d$ , nên

$$\Delta Z = \frac{Z^2}{fb} \Delta d.$$

Câu này nói: sai khớp điểm *một chút* ở gần thì hầu như không sao, nhưng cùng sai số đó ở xa thì bị khuếch đại theo bình phương khoảng cách. Lấy con số của một bộ EuRoC quen thuộc,  $f \approx 400$  pixel và  $b \approx 0,11$  m: ở  $Z = 10$  m thì  $d = 4,4$  pixel, và  $\Delta d = 0,2$  pixel cho  $\Delta Z \approx 0,45$  m (4,5%); ở  $Z = 2$  m thì cùng  $\Delta d$  đó chỉ cho  $\Delta Z \approx 1,8$  cm.

Hình 5.4 vẽ hai đường sai số đó cạnh nhau và cho thấy chúng cắt nhau ở đâu.



**Hình 5.4.** Hai loại sai số độ sâu có *hình dạng* khác nhau, và đó mới là điều đáng nhớ. Sai số stereo tăng theo  $Z^2$  nên rất tốt ở gần và vô dụng ở xa; sai số của một mạng học được tăng gần như tuyến tính vì nó là sai số *tương đối* do prior quyết định, không do hình học. Vì vậy tồn tại một khoảng cách mà quá nó, phép đo hình học thua phép đoán có học. *Con số minh họa* ( $f = 400$  px,  $b = 0,11$  m,  $\Delta d = 0,2$  px, sai số tương đối 5%), không phải số đo trên một hệ cụ thể.

Ba hệ quả rút ra:

- **Có một “chân trời” độ sâu.** Vượt qua nó, độ sâu stereo vô dụng — không phải vì thuật toán kém mà vì hình học.
- **Cách duy nhất đẩy chân trời ra xa là tăng  $fb$ .** Bundle adjustment trên một chuỗi ảnh chính là cách tăng **đường cơ sở** (*baseline*) hiệu dụng bằng thời gian; đây là lý do SLAM đơn mắt vẫn có độ sâu tốt khi robot đi đủ xa.
- **Mạng suy độ sâu từ một ảnh không có phương trình này.** Sai số của nó do prior quyết định, nên không tuân theo  $Z^2$  và cũng không hưởng lợi khi ta đi thêm một quãng để mở rộng đường cơ sở. Trộn hai loại sai số này vào cùng một bộ lọc mà không mô hình hoá đúng thì hiệp phương sai sẽ sai.

#### E. Giới hạn và trường hợp hồng

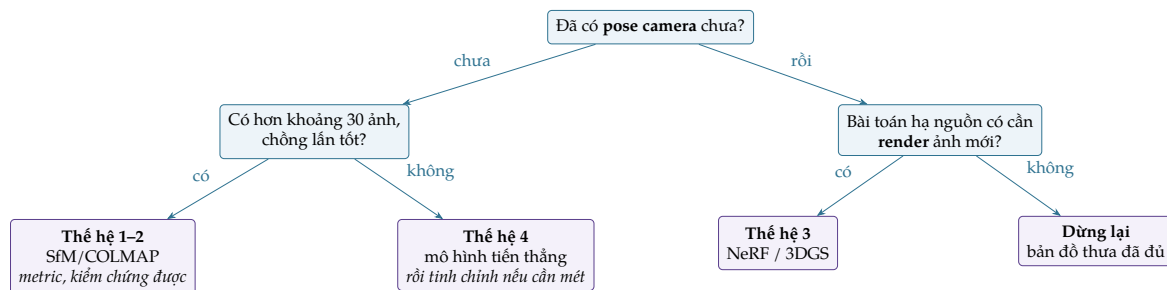
Giới hạn lớn nhất của cách kể “bốn thế hệ” là nó gợi ý một sự thay thế tuần tự, trong khi thực tế là chồng lớp. Pipeline mạnh nhất hiện nay thường lai, gồm ba chặng. Thứ nhất, dùng feed-forward geometry để có khởi tạo pose và point map thô, không cần khởi tạo thủ công. Thứ hai, ném khởi

tạo đó vào hiệu chỉnh chùm tia cổ điển để lấy lại độ chính xác metric. Thứ ba, dựng biểu diễn render lên trên. Thế hệ bốn giải bài toán *khởi tạo* (vốn là chỗ SfM hay chết), thế hệ một giải bài toán *tinh chỉnh*.

Các chế độ hỏng riêng:

- **Thế hệ một** hỏng khi giả định vật lý bị vi phạm — mặt kính, mặt nước, tường trắng, cảnh động — và nó hỏng *ôn ào*: số *inlier* tụt, phần dư tăng, hệ thống biết mình đang hỏng.
- **Thế hệ hai** kế thừa toàn bộ chỗ hỏng đó và thêm **lệch phân phối** (*domain shift*) của khâu học.
- **Thế hệ ba** hỏng ở ba chỗ: pose đầu vào sai, phơi sáng tự động thay đổi giữa các ảnh, hoặc cảnh có vật chuyển động. Riêng pose thì chỉ cần sai vài phần mười độ: ảnh render mờ đi, và tối ưu bù lại bằng vật chất giả lơ lửng.
- **Thế hệ bốn** hỏng khi cảnh ra khỏi phân phối huấn luyện — và đây là chế độ nguy hiểm nhất vì nó *im lặng*.

Gộp lại, bốn chế độ hỏng đó cho một quy tắc chọn dùng được ngay: Hình 5.5 hỏi theo đúng thứ tự các ràng buộc, từ ràng buộc cứng nhất trở đi.



**Hình 5.5.** Chọn thế hệ nào, hỏi theo đúng thứ tự. Câu hỏi đầu tiên không phải “phương pháp nào mới nhất” mà “tôi đã có pose chưa” — vì neural rendering nằm sau khâu pose nên nó không bao giờ là câu trả lời khi chưa có pose. Nhánh phải dưới cùng là nhánh hay bị bỏ quên nhất: rất nhiều bài toán robot chỉ cần định vị, và ở đó bản đồ thừa cổ điển vẫn là lựa chọn đúng.

### CẠM BẮY

Cạm bẫy phổ biến nhất khi so sánh thế hệ bốn với COLMAP là so trên chính loại cảnh mà thế hệ bốn được huấn luyện. Nhiều bộ *benchmark* 3D lớn đã được dùng làm dữ liệu huấn luyện cho các mô hình tiền thẳng gần đây, nên con số “tốt hơn COLMAP” trên các bộ đó không nói được gì về cảnh của bạn. Phép thử trung thực duy nhất là chạy trên dữ liệu bạn tự quay. Và cách phát hiện lỗi duy nhất là kiểm tra chéo bằng một ràng buộc bên ngoài. Ba ràng buộc rẻ nhất: chiều dài một vật đã biết, một đoạn quỹ đạo có chân trị, hoặc phần dư sau khi ép kết quả tiền thẳng qua một vòng hiệu chỉnh chùm tia.

↪ **cùng nguyên lý với C/17/04** — khôi phục ảnh cũng đối ràng buộc cứng lấy prior học được, và cũng mất khả năng tự phát hiện sai theo đúng cách này.

↪ **tiền đề A/02** — mọi phép tam giác hoá đứng trên mô hình chiếu và giả định hiệu chỉnh; sai ở tầng đó thì không thể hệ nào cứu được.

### CẦN NHỚ

Đi từ thế hệ một tới thế hệ bốn, thứ mất đi không chỉ là độ chính xác metric mà là *khả năng tự chẩn đoán*. Hiệu chỉnh chùm tia trả về phần dư và hiệp phương sai, nên hệ thống biết lúc nào nó không tin được; một mạng tiền thẳng trả về point map với cùng vẽ tự tin dù cảnh

nằm trong hay ngoài phân phối. Khi đưa thể hệ bốn vào hệ thống thật, việc đầu tiên phải làm là dựng lại một cơ chế phát hiện sai từ bên ngoài. [T3]

## F. Kết nối

Mục này là khung để đọc hai mục còn lại của chương. Mục kế tiếp, [C/16-thi-giac-3d/02-bieu-dien-3d-va-depth](#), đi vào các quyết định con bên trong mỗi thể hệ; mục cuối, [C/16-thi-giac-3d/03-nerf-3dgs-va-hop-nhat](#), đào sâu đúng hai thứ lạ nhất với người làm SLAM.

Trục đánh đổi ở đây là trường hợp riêng, cụ thể nhất, của B/04-nen-tang-hoc-tu-du-lieu. Cơ chế “prior thay ràng buộc cứng” sẽ trở lại nguyên vẹn dưới tên bài toán *ill-posed* ở [C/17-mo-hinh-sinh/04-low-level-vision](#). Mô hình chiếu và các giả định hiệu chỉnh nằm ở A/02-vat-ly-tao-anh/03-camera-va-hinh-hoc-nhieu-view; còn nếu khâu tương ứng hỏng vì domain shift thì B/08-dich-chuyen-phan-phoi là chỗ đọc tiếp.

### ■ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao thể hệ lai “chỉ học khâu tương ứng” lại nguy hiểm hơn thể hệ cổ điển ở đúng một loại cảnh, dù RANSAC vẫn được giữ nguyên?
2. Neural rendering đúng *sau* khâu pose trong pipeline. Điều đó nói gì về câu “NeRF thay thế SfM”?
3. Một mạng tiến thẳng cho ra point map đẹp trên video bạn quay. Bạn có ba mươi phút để kiểm tra xem nó đúng hay chỉ trông đúng — bạn làm gì?
4. Sai số độ sâu stereo tăng theo  $Z^2$ . Vì sao sai số của độ sâu đơn ảnh học được lại *không* tuân theo quy luật đó, và điều này ảnh hưởng thế nào tới việc gán trọng số cho nó trong một bộ lọc?
5. Nếu phải chọn đúng một thể hệ để đưa vào robot chạy 24/7 ngoài trời, bạn chọn thể hệ nào và dựng thêm cơ chế an toàn nào?



- [1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [2] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [3] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *AISTATS*, 2010.
- [4] K. He, X. Zhang, S. Ren, et al., “Delving Deep into Rectifiers,” in *ICCV*, 2015.
- [5] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [6] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [7] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [8] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [9] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [10] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [11] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” in *ICLR*, 2017.
- [12] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [13] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [14] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [15] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [16] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [17] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [18] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [19] G. Huang, Y. Sun, Z. Liu, et al., “Deep Networks with Stochastic Depth,” in *ECCV*, 2016.
- [20] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [21] H. Zhang, M. Cisse, Y. N. Dauphin, et al., “mixup: Beyond Empirical Risk Minimization,” in *ICLR*, 2018.
- [22] S. Yun, D. Han, S. J. Oh, et al., “CutMix: Regularization Strategy to Train Strong Classifiers,” in *ICCV*, 2019.
- [23] E. D. Cubuk, B. Zoph, J. Shlens, et al., “RandAugment: Practical Automated Data Augmentation,” in *CVPR Workshops*, 2020.

- [24] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [25] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [26] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *NeurIPS*, 2012.
- [27] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in *ICLR*, 2015.
- [28] C. Szegedy, W. Liu, et al., “Going Deeper with Convolutions,” in *CVPR*, 2015.
- [29] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [30] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [31] G. Huang, Z. Liu, L. van der Maaten, et al., “Densely Connected Convolutional Networks,” in *CVPR*, 2017.
- [32] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [33] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [34] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [35] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [36] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [37] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.
- [38] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [39] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [40] J. Su, M. Ahmed, Y. Lu, et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, vol. 568, 2024.
- [41] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” *arXiv:2312.00752*, 2023.
- [42] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [43] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.
- [44] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [45] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [46] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [47] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [48] R. Girshick, J. Donahue, T. Darrell, et al., “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation,” in *CVPR*, 2014.

- [49] R. Girshick, "Fast R-CNN," in *ICCV*, 2015.
- [50] S. Ren, K. He, R. Girshick, et al., "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in *NeurIPS*, 2015.
- [51] T.-Y. Lin, P. Dollár, R. Girshick, et al., "Feature Pyramid Networks for Object Detection," in *CVPR*, 2017.
- [52] K. He, G. Gkioxari, P. Dollár, et al., "Mask R-CNN," in *ICCV*, 2017.
- [53] D. Bolya, S. Foley, J. Hays, et al., "TIDE: A General Toolbox for Identifying Object Detection Errors," in *ECCV*, 2020.
- [54] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [55] J. Redmon, S. Divvala, R. Girshick, et al., "You Only Look Once: Unified, Real-Time Object Detection," in *CVPR*, 2016.
- [56] W. Liu, D. Anguelov, D. Erhan, et al., "SSD: Single Shot MultiBox Detector," in *ECCV*, 2016.
- [57] T.-Y. Lin, P. Goyal, R. Girshick, et al., "Focal Loss for Dense Object Detection," in *ICCV*, 2017.
- [58] Z. Tian, C. Shen, H. Chen, et al., "FCOS: Fully Convolutional One-Stage Object Detection," in *ICCV*, 2019.
- [59] X. Zhou, D. Wang, and P. Krähenbühl, "Objects as Points," *arXiv:1904.07850*, 2019.
- [60] S. Zhang, C. Chi, Y. Yao, et al., "Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection," in *CVPR*, 2020.
- [61] Z. Ge, S. Liu, Z. Li, et al., "OTA: Optimal Transport Assignment for Object Detection," in *CVPR*, 2021.
- [62] N. Bodla, B. Singh, R. Chellappa, et al., "Soft-NMS: Improving Object Detection with One Line of Code," in *ICCV*, 2017.
- [63] X. Zhu, W. Su, L. Lu, et al., "Deformable DETR: Deformable Transformers for End-to-End Object Detection," in *ICLR*, 2021.
- [64] H. Zhang, F. Li, S. Liu, et al., "DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection," in *ICLR*, 2023.
- [65] J. Long, E. Shelhamer, and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," in *CVPR*, 2015.
- [66] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *MICCAI*, 2015.
- [67] L.-C. Chen, G. Papandreou, I. Kokkinos, et al., "DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs," *IEEE TPAMI*, vol. 40, no. 4, pp. 834–848, 2018.
- [68] E. Xie, W. Wang, Z. Yu, et al., "SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers," in *NeurIPS*, 2021.
- [69] B. Cheng, I. Misra, A. G. Schwing, et al., "Masked-Attention Mask Transformer for Universal Image Segmentation," in *CVPR*, 2022.
- [70] A. Kirillov, K. He, R. Girshick, et al., "Panoptic Segmentation," in *CVPR*, 2019.
- [71] A. Kirillov, E. Mintun, N. Ravi, et al., "Segment Anything," in *ICCV*, 2023.
- [72] K. Sun, B. Xiao, D. Liu, et al., "Deep High-Resolution Representation Learning for Human Pose Estimation," in *CVPR*, 2019.

- [73] Z. Cao, T. Simon, S.-E. Wei, et al., “Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields,” in *CVPR*, 2017.
- [74] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [75] J. Deng, J. Guo, N. Xue, et al., “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *CVPR*, 2019.
- [76] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [77] K. Xu, J. Ba, R. Kiros, et al., “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention,” in *ICML*, 2015.
- [78] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *ICLR*, 2015.
- [79] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [80] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [81] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [82] K. Simonyan and A. Zisserman, “Two-Stream Convolutional Networks for Action Recognition in Videos,” in *NeurIPS*, 2014.
- [83] J. Carreira and A. Zisserman, “Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset,” in *CVPR*, 2017.
- [84] D. Tran, H. Wang, L. Torresani, et al., “A Closer Look at Spatiotemporal Convolutions for Action Recognition,” in *CVPR*, 2018.
- [85] C. Feichtenhofer, H. Fan, J. Malik, et al., “SlowFast Networks for Video Recognition,” in *ICCV*, 2019.
- [86] A. Dosovitskiy, P. Fischer, E. Ilg, et al., “FlowNet: Learning Optical Flow with Convolutional Networks,” in *ICCV*, 2015.
- [87] D. Sun, X. Yang, M.-Y. Liu, et al., “PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume,” in *CVPR*, 2018.
- [88] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.
- [89] A. Bewley, Z. Ge, L. Ott, et al., “Simple Online and Realtime Tracking,” in *ICIP*, 2016.
- [90] Y. Zhang, P. Sun, Y. Jiang, et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *ECCV*, 2022.
- [91] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [92] K. Bernardin and R. Stiefelwagen, “Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics,” *EURASIP J. Image and Video Processing*, 2008.
- [93] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [94] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [95] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.

- [96] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *CVPR*, 2024.
- [97] D. Eigen, C. Puhrsch, and R. Fergus, “Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network,” in *NeurIPS*, 2014.
- [98] R. Ranftl, K. Lasinger, D. Hafner, et al., “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer,” *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [99] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IROS*, 2012.
- [100] C. R. Qi, H. Su, K. Mo, et al., “PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation,” in *CVPR*, 2017.
- [101] C. R. Qi, L. Yi, H. Su, et al., “PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space,” in *NeurIPS*, 2017.
- [102] A. H. Lang, S. Vora, H. Caesar, et al., “PointPillars: Fast Encoders for Object Detection from Point Clouds,” in *CVPR*, 2019.
- [103] T. Müller, A. Evans, C. Schied, et al., “Instant Neural Graphics Primitives with a Multiresolution Hash Encoding,” *ACM TOG*, vol. 41, no. 4, 2022.
- [104] B. Kerbl, G. Kopanas, T. Leimkühler, et al., “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *ACM TOG*, vol. 42, no. 4, 2023.
- [105] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.
- [106] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” in *ICLR*, 2014.
- [107] I. Goodfellow, J. Pouget-Abadie, M. Mirza, et al., “Generative Adversarial Nets,” in *NeurIPS*, 2014.
- [108] T. Karras, S. Laine, and T. Aila, “A Style-Based Generator Architecture for Generative Adversarial Networks,” in *CVPR*, 2019.
- [109] J. Ho, A. Jain, and P. Abbeel, “Denoising Diffusion Probabilistic Models,” in *NeurIPS*, 2020.
- [110] Y. Song, J. Sohl-Dickstein, D. P. Kingma, et al., “Score-Based Generative Modeling Through Stochastic Differential Equations,” in *ICLR*, 2021.
- [111] J. Johnson, A. Alahi, and L. Fei-Fei, “Perceptual Losses for Real-Time Style Transfer and Super-Resolution,” in *ECCV*, 2016.
- [112] P. Isola, J.-Y. Zhu, T. Zhou, et al., “Image-to-Image Translation with Conditional Adversarial Networks,” in *CVPR*, 2017.
- [113] L. A. Gatys, A. S. Ecker, and M. Bethge, “Image Style Transfer Using Convolutional Neural Networks,” in *CVPR*, 2016.
- [114] X. Huang and S. Belongie, “Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization,” in *ICCV*, 2017.
- [115] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.
- [116] J. Song, C. Meng, and S. Ermon, “Denoising Diffusion Implicit Models,” in *ICLR*, 2021.
- [117] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, et al., “Flow Matching for Generative Modeling,” in *ICLR*, 2023.
- [118] J. Ho and T. Salimans, “Classifier-Free Diffusion Guidance,” *arXiv:2207.12598*, 2022.
- [119] R. Rombach, A. Blattmann, D. Lorenz, et al., “High-Resolution Image Synthesis with Latent Diffusion Models,” in *CVPR*, 2022.
- [120] W. Peebles and S. Xie, “Scalable Diffusion Models with Transformers,” in *ICCV*, 2023.



- [121] L. Zhang, A. Rao, and M. Agrawala, "Adding Conditional Control to Text-to-Image Diffusion Models," in *ICCV*, 2023.
- [122] Y. Blau and T. Michaeli, "The Perception-Distortion Tradeoff," in *CVPR*, 2018.
- [123] C. Ledig, L. Theis, F. Huszár, et al., "Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network," in *CVPR*, 2017.
- [124] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., "Image Quality Assessment: From Error Visibility to Structural Similarity," *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [125] M. Heusel, H. Ramsauer, T. Unterthiner, et al., "GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium," in *NeurIPS*, 2017.
- [126] T. Brooks, B. Mildenhall, T. Xue, et al., "Unprocessing Images for Learned Raw Denoising," in *CVPR*, 2019.
- [127] A. Abdelhamed, S. Lin, and M. S. Brown, "A High-Quality Denoising Dataset for Smartphone Cameras," in *CVPR*, 2018.
- [128] J. Deng, W. Dong, R. Socher, et al., "ImageNet: A Large-Scale Hierarchical Image Database," in *CVPR*, 2009.
- [129] A. Radford, J. W. Kim, C. Hallacy, et al., "Learning Transferable Visual Models from Natural Language Supervision," in *ICML*, 2021.
- [130] J. Li, D. Li, S. Savarese, et al., "BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models," in *ICML*, 2023.
- [131] H. Liu, C. Li, Q. Wu, et al., "Visual Instruction Tuning," in *NeurIPS*, 2023.
- [132] S. Kornblith, J. Shlens, and Q. V. Le, "Do Better ImageNet Models Transfer Better?" In *CVPR*, 2019.
- [133] T. Chen, S. Kornblith, M. Norouzi, et al., "A Simple Framework for Contrastive Learning of Visual Representations," in *ICML*, 2020.
- [134] K. He, H. Fan, Y. Wu, et al., "Momentum Contrast for Unsupervised Visual Representation Learning," in *CVPR*, 2020.
- [135] J.-B. Grill, F. Strub, F. Altché, et al., "Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning," in *NeurIPS*, 2020.
- [136] M. Caron, H. Touvron, I. Misra, et al., "Emerging Properties in Self-Supervised Vision Transformers," in *ICCV*, 2021.
- [137] K. He, X. Chen, S. Xie, et al., "Masked Autoencoders Are Scalable Vision Learners," in *CVPR*, 2022.
- [138] A. van den Oord, Y. Li, and O. Vinyals, "Representation Learning with Contrastive Predictive Coding," in *arXiv:1807.03748*, 2018.
- [139] J. Kaplan, S. McCandlish, T. Henighan, et al., "Scaling Laws for Neural Language Models," *arXiv:2001.08361*, 2020.
- [140] X. Zhai, A. Kolesnikov, N. Houlsby, et al., "Scaling Vision Transformers," in *CVPR*, 2022.
- [141] J. Hoffmann, S. Borgeaud, A. Mensch, et al., "Training Compute-Optimal Large Language Models," in *NeurIPS*, 2022.
- [142] C. G. Northcutt, A. Athalye, and J. Mueller, "Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks," in *NeurIPS Datasets and Benchmarks*, 2021.
- [143] B. Recht, R. Roelofs, L. Schmidt, et al., "Do ImageNet Classifiers Generalize to ImageNet?" In *ICML*, 2019.
- [144] N. Houlsby, A. Giurgiu, S. Jastrzebski, et al., "Parameter-Efficient Transfer Learning for NLP," in *ICML*, 2019.



- [145] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [146] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *RecSys*, 2019.

## 5.2 Chọn biểu diễn 3D, độ sâu và SLAM (3D Representations, Depth and SLAM)

**PHỤ THUỘC** C/16-thi-giac-3d/01-bon-the-he-3d (bốn thế hệ); B/10-thiet-ke-kien-truc (thiết kế theo bất biến); B/07-chia-du-lieu-va-danh-gia (ATE/RPE và giao thức đánh giá); A/01-toan-toi-thieu/03-xac-suat-va-thong-tin (tính xác định được của tham số).

### TÓM TẮT

Mục này nói về ba quyết định con nằm bên trong mọi hệ thống 3D: lưu cảnh bằng biểu diễn nào, lấy độ sâu từ đâu, và học sâu làm gì được trên dữ liệu không phải lưới. Đọc xong bạn làm được ba việc. Chọn biểu diễn cho một pipeline mới mà không phải viết lại nó sáu tháng sau. Nhìn một mô-đun độ sâu là biết nó có quyền xuất hiện trong phần dư metric hay không. Và đọc một mạng trên point cloud để biết nó mù với cái gì. Nếu chỉ nhớ hai điều: biểu diễn quyết định *phép toán nào rẻ*, nên phải chọn nó theo khâu cuối cùng của pipeline chứ không theo khâu đầu tiên. Và độ sâu đơn ảnh không phải phép đo mà là suy đoán có học, với một tỉ lệ trôi giữa các khung hình — đó chính là chỗ nó phá hỏng một bộ ước lượng.

### KIẾN THỨC CẦN CÓ TRƯỚC

- **voxel** — ô lập phương trong một lưới đều 3D — “pixel của không gian ba chiều”. Đơn giản nhưng số ô tăng theo lũy thừa ba của độ phân giải.
- **point cloud (point cloud)** — một tập tọa độ 3D không có thứ tự và không có kết nối giữa các điểm; đây là dạng dữ liệu thô của LiDAR và của stereo dày đặc.
- **mesh (mesh)** — biểu diễn bề mặt bằng các đỉnh nối thành tam giác; chuẩn của đồ họa máy tính, vì nó cho biết *đâu là mặt* chứ không chỉ *đâu có vật chất*.
- **SDF (signed distance function)** — một hàm gán cho mỗi điểm trong không gian khoảng cách có dấu tới bề mặt gần nhất — âm ở trong, dương ở ngoài. Bề mặt chính là tập điểm có giá trị 0, nên nó được định nghĩa rất sạch.
- **trường ẩn (implicit field)** — cách mô tả hình dạng bằng một *hàm* (thường là một mạng nhỏ) thay vì bằng một danh sách phần tử; SDF là ví dụ điển hình.
- **đường cơ sở (baseline)** — khoảng cách giữa hai tâm camera trong một hệ stereo; nó là thứ duy nhất cấp đơn vị mét cho phép đo độ sâu.
- **ATE và RPE** — hai chỉ số đánh giá quỹ đạo: ATE đo sai lệch tuyệt đối sau khi căn chỉnh toàn cục hai quỹ đạo, RPE đo sai lệch trên các đoạn ngắn nên không cần căn chỉnh. Bạn đã quen hai chỉ số này; ở đây chỉ dùng chúng làm ví dụ về việc “phép căn chỉnh che giấu điều gì”.
- **bất biến hoán vị (permutation invariance)** — tính chất của một hàm mà kết quả không đổi khi ta xáo trộn thứ tự các phần tử đầu vào; bắt buộc phải có nếu đầu vào là một *tập*.

### A. Đặt vấn đề

Trong ảnh 2D không ai phải chọn biểu diễn: ảnh là một lưới pixel, hết chuyện. Trong 3D thì không có biểu diễn mặc định nào cả, và lựa chọn đó quyết định nhiều hơn cả việc chọn kiến trúc mạng. Lý do rất cụ thể: **biểu diễn quyết định phép toán nào rẻ**. Trên lưới *voxel*, tích chập là rẻ nhưng bộ nhớ tăng theo lũy thừa ba. Trên *point cloud*, bộ nhớ rẻ nhưng “lân cận” không có sẵn nên mọi

phép cục bộ phải trả tiền tìm kiếm. Trên *mesh*, bề mặt là tường minh nên render và mô phỏng vật lý rẻ, nhưng thay đổi *topology* thì gần như không khả vi. Một khi đã chọn biểu diễn, bạn đã chọn luôn tập bài toán còn giải được hiệu quả — và đây là quyết định khó đảo ngược nhất trong một pipeline 3D.

Câu chuyện thứ hai là nguồn của độ sâu. Bốn thế hệ ở mục trước đều cần độ sâu từ đâu đó. Có ba cách lấy nó: một ảnh, hai ảnh, hoặc cảm biến chủ động. Chúng khác nhau không phải ở độ chính xác mà ở chỗ *cái nào xác định được tỉ lệ tuyệt đối*. Cuối cùng, ta đặt SLAM và VIO vào đúng vị trí: không phải một chủ đề riêng mà là hệ thống duy nhất bắt buộc giải cả ba bài toán — biểu diễn, độ sâu, tối ưu — dưới ràng buộc thời gian thực.

**Học mục này thì làm được gì mà trước đó không làm được?** Ba việc: chọn được biểu diễn cho một pipeline mới mà không phải viết lại nó sáu tháng sau; nhìn một mô-đun độ sâu và nói ngay nó có quyền xuất hiện trong phần dư metric hay không; và đọc được một mô hình học trên point cloud để biết nó mù với cái gì.

#### TRỰC GIÁC

Chọn biểu diễn 3D giống chọn hệ tọa độ để viết một bài toán cơ học. Bài toán không đổi, nhưng viết trong hệ Descartes thì trọng lực đơn giản còn chuyển động tròn phức tạp; trong hệ cực thì ngược lại. Voxel là hệ Descartes của 3D: đều đặn, dễ lập trình, và lãng phí khủng khiếp vì phần lớn không gian là rỗng. Point cloud là “danh sách các chỗ có vật chất”: gọn đúng bằng lượng thông tin thật, nhưng bạn phải tự dựng lại khái niệm hàng xóm mỗi lần cần tới nó.

#### B. Năm biểu diễn và cái giá của mỗi cái

Năm biểu diễn dưới đây không ngang hàng nhau. Chúng chia làm ba nhóm theo *nguồn gốc*, và biết nhóm nào giúp đoán ngay chỗ hỏng:

- **Biểu diễn đo được** — voxel và point cloud, ra thẳng từ cảm biến. Chỗ hỏng của chúng là chỗ hỏng của cảm biến: mật độ không đều, vùng khuất, nhiễu phụ thuộc khoảng cách.
- **Biểu diễn đồ họa** — mesh. Chỗ hỏng là bước tái dựng bề mặt đứng trước nó.
- **Biểu diễn được tối ưu** — **trường ẩn** (*implicit field*) và *Gaussian splatting*. Chúng không tồn tại cho tới khi chạy xong một vòng tối ưu trên chính cảnh đó, nên chỗ hỏng của chúng là chỗ hỏng của hàm mất mát đã sinh ra chúng.

**Bảng 5.2.** Năm biểu diễn 3D. Cột cuối là cột quyết định: mỗi biểu diễn hồng ở một chỗ khác nhau, và chỗ đó thường lộ ra muộn.

| Biểu diễn                       | Cơ chế lưu                                                    | Được                                                        | Mất / hồng khi                                                      |
|---------------------------------|---------------------------------------------------------------|-------------------------------------------------------------|---------------------------------------------------------------------|
| Voxel                           | Lưới đều 3D, mỗi ô một giá trị                                | Conv 3D dùng được nguyên xi; lân cận cho sẵn                | Bộ nhớ theo lũy thừa ba; hồng ngay khi cần độ phân giải cao         |
| Point cloud                     | Tập điểm không thứ tự                                         | Gọn, hợp với LiDAR và stereo                                | Không có cấu trúc lân cận; phải bất biến hoán vị; mật độ không đều  |
| Mesh                            | Đỉnh + mặt                                                    | Chuẩn của đồ hoạ và mô phỏng; bề mặt tường minh             | Tối ưu topology gần như không khả vi; mặt tự cắt nhau               |
| Implicit field (SDF, occupancy) | Một hàm $\mathbb{R}^3 \rightarrow \mathbb{R}$ , thường là MLP | Độ phân giải liên tục; bề mặt là tập mức 0, định nghĩa sạch | Render chậm vì phải lấy mẫu nhiều lần dọc tia; khó chỉnh sửa cục bộ |
| Gaussian splatting              | Tập Gaussian dị hướng có màu và độ đục                        | Rasterize được nên render rất nhanh                         | Dung lượng lớn; hình học không tường minh nên trích mesh khó        |

Hình 5.6 vẽ cùng một mặt cong bằng cả năm biểu diễn, đặt cạnh nhau — đây là cách nhanh nhất để thấy mỗi cái giữ lại gì và vứt đi gì.

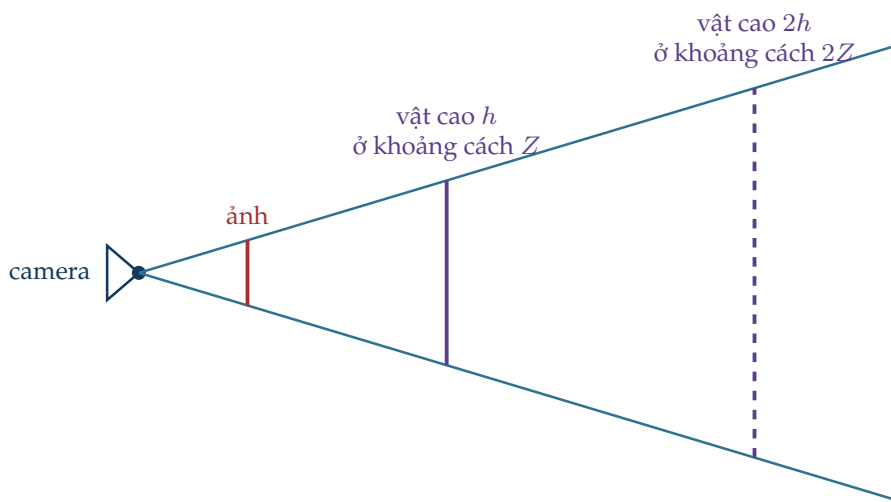


**Hình 5.6.** Cùng một mặt cong, năm biểu diễn. Nhìn hàng chú thích dưới cùng thì thấy ngay điều mà bảng không nói được: mỗi biểu diễn giữ lại một loại thông tin và vứt đi loại khác. Point cloud biết ở đâu có vật chất nhưng không biết đâu là mặt; voxel biết cả không gian rỗng nhưng trả tiền cho nó; mesh biết mặt nhưng khoá cứng topology; SDF định nghĩa mặt sạch nhất nhưng phải dò dọc tia mới render được; Gaussian render nhanh nhất nhưng không có mặt để mà trích ra.

Con số làm cho dòng đầu bảng thành cụ thể. Một lưới  $512^3$  chứa khoảng  $1,34 \times 10^8$  ô; lưu một số thực 32-bit cho mỗi ô đã tốn 512 MiB, và một tầng conv 3D với 32 kênh ở cùng độ phân giải cần 16 GiB chỉ để giữ *activation*. Nâng lên  $1024^3$  thì nhân tám. Trong khi đó một quét LiDAR dày đặc của cùng cảnh có chừng  $2 \times 10^5$  điểm, tức 2,4 MB nếu lưu ba toạ độ float — ít hơn hai bậc độ lớn. Toàn bộ dòng **tích chập thưa** (*sparse convolution*) tồn tại chỉ vì khoảng cách này: hơn 99 % ô của lưới đều là rỗng, nên trả tiền cho chúng là trả tiền cho hư không.

### C. Độ sâu đến từ đâu, và cái gì cố định tỉ lệ

Với ảnh đơn, bài toán độ sâu không phải khó — nó *không xác định được*. Nhân toàn bộ cảnh với hệ số  $s$  và nhân luôn mọi vị trí camera với  $s$  thì mọi ảnh thu được giống hệt nhau. Hình 5.7 là toàn bộ chứng minh, vẽ ra trong một hình.



Hai cảnh khác nhau, **một ảnh duy nhất**. Không phép đo nào trong ảnh phân biệt được chúng.

**Hình 5.7.** Tính không xác định được của tỉ lệ trong ảnh đơn. Nhân cảnh với  $s$  và nhân quãng đường camera với cùng  $s$  cho ra đúng bộ ảnh cũ; vì vậy *không lượng dữ liệu nào* chứa được, chỉ prior mới chứa được. Đây cũng chính là lý do SLAM đơn mắt có một bậc tự do tỉ lệ, và vì sao chỉ cần thêm một gia tốc kế — một cảm biến biết đơn vị mét — là bậc tự do đó biến mất.

Đây đúng là hiện tượng **không xác định được** (*non-identifiability*) ở chương 4: dữ liệu không chứa thông tin để phân biệt hai giả thuyết.

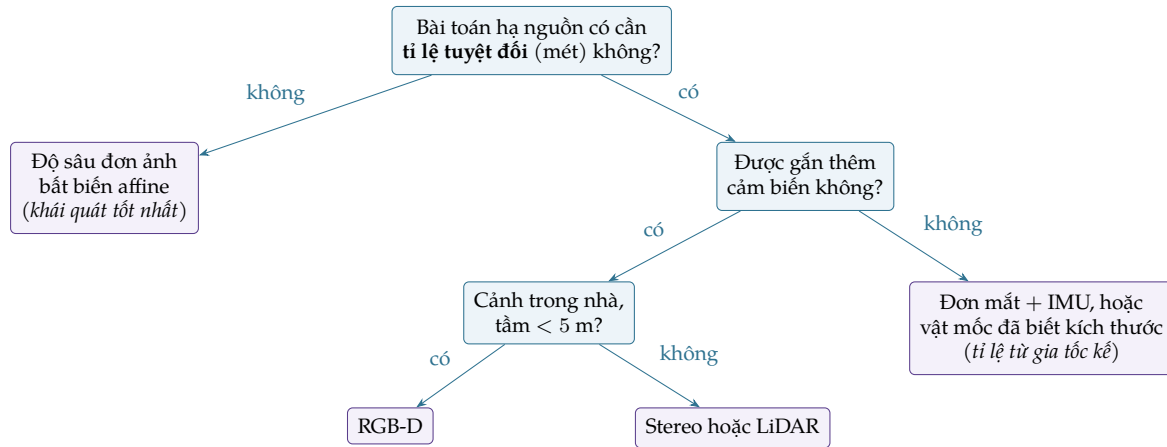
↔ **trường hợp riêng của** B/04 — scale ambiguity là một ca sạch của tính không xác định được: hai tham số khác nhau sinh ra cùng một phân phối quan sát, nên thêm dữ liệu không giúp gì.

Một mạng độ sâu đơn ảnh phá thế bế tắc bằng cách học kích thước điển hình của vật thể và mối liên hệ với tiêu cự; nghĩa là con số nó trả về là một *suy đoán có học*, không phải một phép đo. Nó vì thế sai theo kiểu rất đặc trưng ở đúng những cảnh phá vỡ prior kích thước: mô hình thu nhỏ, ảnh chụp từ trên cao, nội soi, ảnh macro.

Bốn nguồn độ sâu, đọc theo cái mà mỗi nguồn dùng để phá thế bế tắc:

- **Đơn ảnh, metric** — phá bằng prior kích thước. Cho ra mét ngay, nhưng chỉ đúng trong miền đã học.
- **Đơn ảnh, bất biến affine** — không phá, mà *bỏ* tỉ lệ đi: huấn luyện trên nhiều tập dữ liệu hỗn tạp với hàm mất mát bất biến với phép biến đổi affine của độ sâu, nên mô hình chỉ phải học thứ tự và hình dạng tương đối [97], [98]. Khái quát rất tốt; đổi lại mỗi khung hình còn dư hai ẩn (tỉ lệ và độ dịch) và hai ẩn đó *không nhất quán giữa các khung hình*.
- **Stereo** — phá bằng vật lý: đường cơ sở đã biết cho tỉ lệ. Cần hiệu chỉnh chính xác, cần texture để khớp, và có chân trời độ sâu tính được từ  $\Delta Z = Z^2 \Delta d / (fb)$ .
- **RGB-D** — phá bằng cảm biến chủ động: chiếu ánh sáng ra và đo phản hồi, nên có tỉ lệ tuyệt đối ngay cả trên tường trắng. Chết dưới nắng, trên mặt gương, trên vật hấp thụ hồng ngoại; có sai lệch phụ thuộc khoảng cách cần hiệu chuẩn riêng.

Hình 5.8 gói bốn lựa chọn đó thành một cây quyết định dùng được ngay.



**Hình 5.8.** Cây quyết định chọn nguồn độ sâu. Câu hỏi đầu tiên không phải “mô hình nào chính xác nhất” mà “bài toán của tôi có thật sự cần mét không”. Rất nhiều bài toán chỉ cần độ sâu tương đối: tránh vật cản gần, sắp thứ tự lớp, tách nền. Ở đó lựa chọn bất biến affine vừa rẻ vừa bền hơn hẳn.

#### D. SLAM và VIO đứng ở đâu

Bạn đọc chương này đã sống trong SLAM nên mục này không nhắc lại cơ chế; nó chỉ đặt SLAM vào bức tranh bốn thể hệ. SLAM là thể hệ một chạy dưới ràng buộc thời gian thực, và toàn bộ kiến trúc *frontend/backend* là hệ quả trực tiếp của ràng buộc đó: frontend phải quyết định tham lam trong vài mili-giây mỗi khung hình nên chỉ dùng ràng buộc cục bộ, còn backend là nơi hình học thực sự được ép đúng trên một cửa sổ hoặc trên toàn đồ thị [79], [80]. **Đóng vòng** (*loop closure*) không phải một tính năng phụ mà là thứ duy nhất đối *bậc* của sai số: không có nó, sai số tích lũy tăng đơn điệu theo quãng đường; có nó, sai số bị chặn bởi độ chính xác của phép đóng vòng.

Bốn thể hệ chạ vào SLAM ở ba chỗ khác nhau:

- **Thể hệ lai thay frontend** — đặc trưng và khớp học được, backend giữ nguyên. Đây là hướng đã thành thực dụng.
- **Thể hệ lai, bản triệt để hơn** — dòng DROID-SLAM dùng **dòng quang** (*optical flow*) dày đặc học được nhưng vẫn nhét một tầng hiệu chỉnh chùm tia khả vi vào giữa, nên nó vẫn là thể hệ hai chứ không phải thể hệ bốn.
- **Thể hệ ba thay bản đồ** — giữ tracker cổ điển nhưng lưu bản đồ bằng trường bức xạ hoặc Gaussian. Đánh đổi bản đồ thừa lấy khả năng render: đáng giá cho robot cần nhìn lại cảnh, đáng ngờ cho robot chỉ cần định vị.

Về đánh giá chỉ cần một câu nhắc: *ATE* cần một phép căn chỉnh toàn cục trước khi đo, vì quỹ đạo ước lượng sống trong một hệ tọa độ tự do — với đơn mắt thì tự do cả tỉ lệ. Chính phép căn chỉnh đó hấp thụ một phần *drift* toàn cục. *RPE* thì đo trên các đoạn ngắn nên không cần căn chỉnh, và vì thể nhảy với lỗi cục bộ mà *ATE* làm phẳng đi [99]. Hai chỉ số phải đọc cùng nhau, và mọi thảo luận sâu hơn về giao thức — chia dữ liệu, số lần lặp, khoảng tin cậy — nằm ở B/07-chia-du-lieu-va-danh-gia.

↪ **ràng buộc bởi B/07** — phép căn chỉnh của *ATE* là một bước tiền xử lý của giao thức đánh giá, nên nó quyết định lỗi nào còn nhìn thấy được.

#### E. Học sâu trên point cloud

Muốn đưa mạng nơ-ron lên point cloud thì vấp ngay vào một trở ngại cấu trúc: đầu vào là một *tập*, không phải chuỗi hay lưới. Với  $n$  điểm có  $n!$  cách đánh số cùng mô tả một hình dạng, nên bất kỳ hàm nào không bất biến với hoán vị đều đang học lại cùng một hình dạng hàng giai thừa lần.

## Mạch 2 – từ tập điểm tới mạng dùng được

### 1 Vấn đề

#### Giải pháp

#### Giả định

#### Hệ quả

đầu vào là tập không thứ tự, mà MLP thường thì không bất biến hoán vị. ép mọi điểm qua cùng một MLP chia sẻ trọng số rồi gộp bằng một **hàm đối xứng** (*symmetric function*) — lấy max theo từng chiều đặc trưng [100].  
hình dạng mô tả được đủ tốt bằng một vectơ đặc trưng toàn cục.  
bất biến hoán vị ở đây là bất biến *theo cấu trúc*, không phải học được từ **tăng cường dữ liệu** (*data augmentation*), nên nó đúng tuyệt đối chứ không đúng gần đúng.

### 2 Vấn đề

#### Giá phải trả

#### Giải pháp

#### Vấn đề mới

max toàn cục vứt bỏ mọi cấu trúc cục bộ: đầu ra chỉ phụ thuộc một tập điểm “tới hạn” rất nhỏ — những điểm đang giữ giá trị lớn nhất ở mỗi chiều.  
thêm hay bớt điểm bên trong bao lồi của tập tới hạn không đổi đầu ra: rất bền với thay đổi mật độ, nhưng mù với chi tiết bề mặt mịn.  
lồng PointNet vào chính nó theo phân cấp không gian — nhóm điểm theo bán kính, chạy PointNet trên từng nhóm, rồi lặp lại ở tỉ lệ lớn hơn [101].  
chi phí truy vấn lân cận thành nút cổ chai, và kết quả nhạy với việc chọn bán kính.

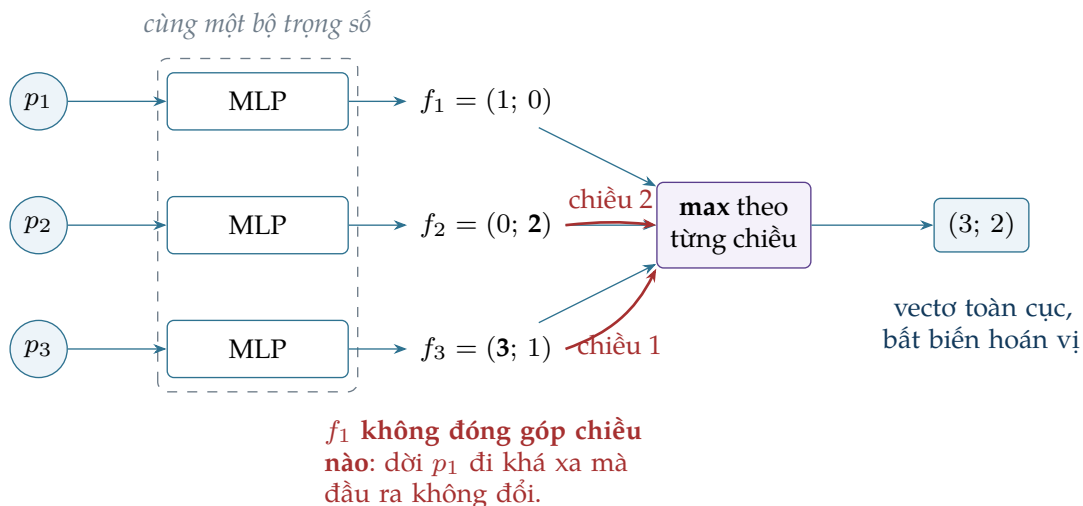
### 3 Vấn đề

#### Giải pháp

#### Giả định

#### Giá phải trả

ở quy mô một cảnh đường phố với hàng trăm nghìn điểm, truy vấn lân cận là quá đắt.  
quay lại lưới nhưng chỉ giữ ô không rỗng (*sparse convolution*), hoặc gộp điểm theo cột dọc thành *pillar* rồi rơi về một bài toán 2D quen thuộc [102].  
cảnh ngoài trời gần như phẳng theo phương thẳng đứng nên nén trục  $z$  mất ít thông tin.  
giả định đó sai ngay trong nhà, nơi vật thể chồng lên nhau theo chiều cao.



**Hình 5.9.** Hàm đối xứng của PointNet, vẽ trên đúng ba điểm của ví dụ tính tay. Mọi điểm đi qua cùng một MLP rồi được gộp bằng phép max theo từng chiều, nên xáo trộn thứ tự ba điểm cho ra cùng một vectơ  $(3; 2)$  — tính bất biến ở đây là bất biến theo cấu trúc, không phải học được. Nhưng hình cũng phơi ra cái giá: đầu ra chỉ do *tập điểm tới hạn* quyết định (ở đây là  $f_2$  và  $f_3$ ), còn  $f_1$  hoàn toàn vô hình. Đúng cái mẹo giữ cho mạng bất biến cũng là đúng cái mẹo làm nó mù.

Hình 5.9 vẽ đúng cơ chế đó trên ba điểm.

**Tính tay để thấy hàm đối xứng thật sự làm gì.** Lấy ba điểm đã qua MLP thành các đặc trưng hai chiều  $f_1 = (1; 0)$ ,  $f_2 = (0; 2)$ ,  $f_3 = (3; 1)$ . Gộp bằng max theo từng chiều cho  $(3; 2)$ . Đảo thứ



tự ba điểm bằng bất kỳ hoán vị nào vẫn cho (3; 2) — đó là toàn bộ mẹo của PointNet. Nhưng phép tính đó cũng phơi ra ngay giới hạn: chiều thứ nhất chỉ do  $f_3$  đóng góp, chiều thứ hai chỉ do  $f_2$ ; điểm  $f_1$  hoàn toàn không ảnh hưởng, và bạn có thể dời nó đi khá xa mà vectơ toàn cục không nhúc nhích. *Đúng cái mẹo giữ cho mạng bất biến cũng là đúng cái mẹo làm nó mù.* Cùng cơ chế này sẽ xuất hiện lại ở [C/17-mo-hinh-sinh/02-toi-uu-tren-pixel](#), nơi ma trận Gram vứt bỏ vị trí không gian để giữ lại “phong cách” — hai kỹ thuật trông không liên quan, một nguyên lý.

→ cùng nguyên lý với [C/17/02](#) — ma trận Gram gộp trên vị trí đúng như PointNet gộp trên điểm: một hàm đối xứng mua tính bất biến bằng chính thông tin bị gộp đi.

→ trường hợp riêng của [B/10](#) — bất biến hoán vị là một ví dụ của việc cài thiên lệch quy nạp thẳng vào kiến trúc thay vì học nó từ dữ liệu.

## F. Giới hạn và trường hợp hỏng

Điểm yếu chung của cả mục này là các biểu diễn không chuyển đổi qua lại mà không mất mát. Điểm sang mesh cần một bước tái dựng bề mặt vốn tự nó là bài toán ước lượng có prior riêng; mesh sang SDF cần mesh kín, điều gần như không đúng với mesh quét thực tế; Gaussian sang mesh là bài toán mở. Vì vậy lỗi thiết kế đắt nhất trong một pipeline 3D là chọn biểu diễn theo khâu đầu tiên thay vì theo khâu cuối cùng cần dùng kết quả.

Hai chế độ hỏng đáng nhớ riêng:

- **Độ sâu học được:** nguy hiểm nhất không phải sai số lớn mà là sai số *không nhất quán theo thời gian* — mỗi khung hình một tỉ lệ khác nhau trong khi từng khung hình đều trông hợp lý.
- **Point cloud:** thay đổi mật độ giữa lúc huấn luyện và lúc chạy. Mô hình huấn luyện trên LiDAR 64 tia thường tụt mạnh trên 32 tia, và triệu chứng là *mất vật ở xa trước tiên* — đúng nơi mật độ giảm nhanh nhất.

### CẠM BẦY

Cạm bẫy quen thuộc với người làm SLAM: lấy độ sâu đơn ảnh làm phép đo trong một bộ ước lượng có trọng số. Nó không những sai tỉ lệ, mà tỉ lệ đó còn *trôi giữa các khung hình*. Sai số của nó vì thế có tương quan mạnh theo thời gian. Đó là vi phạm đúng giả định nhiễu trắng độc lập mà bộ lọc dựa vào, và hệ quả là hiệp phương sai bị đánh giá quá lạc quan. Có hai cách dùng an toàn. Cách thứ nhất: coi nó như ràng buộc *tương đối* bên trong một khung hình, ví dụ thứ tự độ sâu hay tính phẳng của mặt. Cách thứ hai: ước lượng tường minh một tỉ lệ và một độ dịch cho mỗi khung hình, rồi đưa hai ẩn đó vào bài toán tối ưu. Cách sai là nhét mét thẳng vào phần dư.

### CẦN NHỚ

Biểu diễn quyết định phép toán nào rẻ, và vì thế quyết định bài toán nào bạn còn giải được về sau: hãy chọn nó theo khâu *cuối cùng* của pipeline, không theo khâu đầu tiên. Còn về độ sâu: chỉ có baseline và cảm biến chủ động cho ra mét; mọi thứ khác cho ra một suy đoán có học, và suy đoán đó không được phép xuất hiện trong phần dư metric mà không kèm hai ẩn tỉ lệ và độ dịch. [T4]

## G. Kết nối

Mục này cung cấp vốn từ cho [C/16-thi-giac-3d/03-nerf-3dgs-va-hop-nhat](#), nơi hai biểu diễn cuối bảng được đào sâu như hai câu trả lời cho cùng một bài toán render. Bất biến hoán vị của PointNet là trường hợp riêng của nguyên tắc thiết kế theo bất biến ở [B/10-thiet-ke-kien-truc](#);

tính không xác định được của tỉ lệ là trường hợp riêng của B/04-nen-tang-hoc-tu-du-lieu. Phần ATE/RPE nổi thẳng sang B/07-chia-du-lieu-va-danh-gia, còn tụt hiệu năng khi đổi mật độ cảm biến là một ca của B/08-dich-chuyen-phan-phoi.

#### ■ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao bộ nhớ voxel không phải vấn đề kỹ thuật có thể tối ưu đi mà là hệ quả cấu trúc của biểu diễn?
2. Độ sâu đơn ảnh bất biến affine tổng quát tốt hơn độ sâu metric. Cái giá của tính tổng quát đó hiện ra ở đâu khi ghép nhiều khung hình?
3. PointNet bất biến hoán vị “theo cấu trúc”. Vì sao điều đó mạnh hơn hẳn việc học tính bất biến bằng augmentation, và nó khiến mạng mù với cái gì?
4. Nếu loop closure bị tắt, sai số của SLAM đổi bậc như thế nào, và chỉ số nào trong hai chỉ số ATE/RPE phản ánh sự thay đổi đó rõ hơn?
5. Một mô hình phát hiện vật thể trên LiDAR 64 tia được đưa sang cảm biến 32 tia. Bạn dự đoán triệu chứng đầu tiên là gì, và vì sao?
6. Hình 5.7 nói rằng không lượng dữ liệu nào phá được tính không xác định của tỉ lệ. Vậy vì sao một mạng “metric depth” vẫn cho ra mét, và điều đó ngầm giả định gì?

- [1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [2] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [3] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *AISTATS*, 2010.
- [4] K. He, X. Zhang, S. Ren, et al., “Delving Deep into Rectifiers,” in *ICCV*, 2015.
- [5] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [6] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [7] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [8] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [9] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [10] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [11] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” in *ICLR*, 2017.
- [12] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [13] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [14] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [15] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [16] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [17] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [18] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [19] G. Huang, Y. Sun, Z. Liu, et al., “Deep Networks with Stochastic Depth,” in *ECCV*, 2016.
- [20] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [21] H. Zhang, M. Cisse, Y. N. Dauphin, et al., “mixup: Beyond Empirical Risk Minimization,” in *ICLR*, 2018.
- [22] S. Yun, D. Han, S. J. Oh, et al., “CutMix: Regularization Strategy to Train Strong Classifiers,” in *ICCV*, 2019.
- [23] E. D. Cubuk, B. Zoph, J. Shlens, et al., “RandAugment: Practical Automated Data Augmentation,” in *CVPR Workshops*, 2020.

- [24] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [25] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [26] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *NeurIPS*, 2012.
- [27] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in *ICLR*, 2015.
- [28] C. Szegedy, W. Liu, et al., “Going Deeper with Convolutions,” in *CVPR*, 2015.
- [29] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [30] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [31] G. Huang, Z. Liu, L. van der Maaten, et al., “Densely Connected Convolutional Networks,” in *CVPR*, 2017.
- [32] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [33] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [34] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [35] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [36] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [37] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.
- [38] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [39] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [40] J. Su, M. Ahmed, Y. Lu, et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, vol. 568, 2024.
- [41] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” *arXiv:2312.00752*, 2023.
- [42] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [43] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.
- [44] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [45] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [46] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [47] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [48] R. Girshick, J. Donahue, T. Darrell, et al., “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation,” in *CVPR*, 2014.

- [49] R. Girshick, "Fast R-CNN," in *ICCV*, 2015.
- [50] S. Ren, K. He, R. Girshick, et al., "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in *NeurIPS*, 2015.
- [51] T.-Y. Lin, P. Dollár, R. Girshick, et al., "Feature Pyramid Networks for Object Detection," in *CVPR*, 2017.
- [52] K. He, G. Gkioxari, P. Dollár, et al., "Mask R-CNN," in *ICCV*, 2017.
- [53] D. Bolya, S. Foley, J. Hays, et al., "TIDE: A General Toolbox for Identifying Object Detection Errors," in *ECCV*, 2020.
- [54] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [55] J. Redmon, S. Divvala, R. Girshick, et al., "You Only Look Once: Unified, Real-Time Object Detection," in *CVPR*, 2016.
- [56] W. Liu, D. Anguelov, D. Erhan, et al., "SSD: Single Shot MultiBox Detector," in *ECCV*, 2016.
- [57] T.-Y. Lin, P. Goyal, R. Girshick, et al., "Focal Loss for Dense Object Detection," in *ICCV*, 2017.
- [58] Z. Tian, C. Shen, H. Chen, et al., "FCOS: Fully Convolutional One-Stage Object Detection," in *ICCV*, 2019.
- [59] X. Zhou, D. Wang, and P. Krähenbühl, "Objects as Points," *arXiv:1904.07850*, 2019.
- [60] S. Zhang, C. Chi, Y. Yao, et al., "Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection," in *CVPR*, 2020.
- [61] Z. Ge, S. Liu, Z. Li, et al., "OTA: Optimal Transport Assignment for Object Detection," in *CVPR*, 2021.
- [62] N. Bodla, B. Singh, R. Chellappa, et al., "Soft-NMS: Improving Object Detection with One Line of Code," in *ICCV*, 2017.
- [63] X. Zhu, W. Su, L. Lu, et al., "Deformable DETR: Deformable Transformers for End-to-End Object Detection," in *ICLR*, 2021.
- [64] H. Zhang, F. Li, S. Liu, et al., "DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection," in *ICLR*, 2023.
- [65] J. Long, E. Shelhamer, and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," in *CVPR*, 2015.
- [66] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *MICCAI*, 2015.
- [67] L.-C. Chen, G. Papandreou, I. Kokkinos, et al., "DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs," *IEEE TPAMI*, vol. 40, no. 4, pp. 834–848, 2018.
- [68] E. Xie, W. Wang, Z. Yu, et al., "SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers," in *NeurIPS*, 2021.
- [69] B. Cheng, I. Misra, A. G. Schwing, et al., "Masked-Attention Mask Transformer for Universal Image Segmentation," in *CVPR*, 2022.
- [70] A. Kirillov, K. He, R. Girshick, et al., "Panoptic Segmentation," in *CVPR*, 2019.
- [71] A. Kirillov, E. Mintun, N. Ravi, et al., "Segment Anything," in *ICCV*, 2023.
- [72] K. Sun, B. Xiao, D. Liu, et al., "Deep High-Resolution Representation Learning for Human Pose Estimation," in *CVPR*, 2019.

- [73] Z. Cao, T. Simon, S.-E. Wei, et al., “Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields,” in *CVPR*, 2017.
- [74] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [75] J. Deng, J. Guo, N. Xue, et al., “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *CVPR*, 2019.
- [76] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [77] K. Xu, J. Ba, R. Kiros, et al., “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention,” in *ICML*, 2015.
- [78] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *ICLR*, 2015.
- [79] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [80] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [81] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [82] K. Simonyan and A. Zisserman, “Two-Stream Convolutional Networks for Action Recognition in Videos,” in *NeurIPS*, 2014.
- [83] J. Carreira and A. Zisserman, “Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset,” in *CVPR*, 2017.
- [84] D. Tran, H. Wang, L. Torresani, et al., “A Closer Look at Spatiotemporal Convolutions for Action Recognition,” in *CVPR*, 2018.
- [85] C. Feichtenhofer, H. Fan, J. Malik, et al., “SlowFast Networks for Video Recognition,” in *ICCV*, 2019.
- [86] A. Dosovitskiy, P. Fischer, E. Ilg, et al., “FlowNet: Learning Optical Flow with Convolutional Networks,” in *ICCV*, 2015.
- [87] D. Sun, X. Yang, M.-Y. Liu, et al., “PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume,” in *CVPR*, 2018.
- [88] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.
- [89] A. Bewley, Z. Ge, L. Ott, et al., “Simple Online and Realtime Tracking,” in *ICIP*, 2016.
- [90] Y. Zhang, P. Sun, Y. Jiang, et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *ECCV*, 2022.
- [91] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [92] K. Bernardin and R. Stiefelwagen, “Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics,” *EURASIP J. Image and Video Processing*, 2008.
- [93] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [94] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [95] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.



- [96] S. Wang, V. Leroy, Y. Cabon, et al., "DUST3R: Geometric 3D Vision Made Easy," in *CVPR*, 2024.
- [97] D. Eigen, C. Puhrsch, and R. Fergus, "Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network," in *NeurIPS*, 2014.
- [98] R. Ranftl, K. Lasinger, D. Hafner, et al., "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer," *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [99] J. Sturm, N. Engelhard, F. Endres, et al., "A Benchmark for the Evaluation of RGB-D SLAM Systems," in *IROS*, 2012.
- [100] C. R. Qi, H. Su, K. Mo, et al., "PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation," in *CVPR*, 2017.
- [101] C. R. Qi, L. Yi, H. Su, et al., "PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space," in *NeurIPS*, 2017.
- [102] A. H. Lang, S. Vora, H. Caesar, et al., "PointPillars: Fast Encoders for Object Detection from Point Clouds," in *CVPR*, 2019.
- [103] T. Müller, A. Evans, C. Schied, et al., "Instant Neural Graphics Primitives with a Multiresolution Hash Encoding," *ACM TOG*, vol. 41, no. 4, 2022.
- [104] B. Kerbl, G. Kopanas, T. Leimkühler, et al., "3D Gaussian Splatting for Real-Time Radiance Field Rendering," *ACM TOG*, vol. 42, no. 4, 2023.
- [105] R. Zhang, P. Isola, A. A. Efros, et al., "The Unreasonable Effectiveness of Deep Features as a Perceptual Metric," in *CVPR*, 2018.
- [106] D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," in *ICLR*, 2014.
- [107] I. Goodfellow, J. Pouget-Abadie, M. Mirza, et al., "Generative Adversarial Nets," in *NeurIPS*, 2014.
- [108] T. Karras, S. Laine, and T. Aila, "A Style-Based Generator Architecture for Generative Adversarial Networks," in *CVPR*, 2019.
- [109] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," in *NeurIPS*, 2020.
- [110] Y. Song, J. Sohl-Dickstein, D. P. Kingma, et al., "Score-Based Generative Modeling Through Stochastic Differential Equations," in *ICLR*, 2021.
- [111] J. Johnson, A. Alahi, and L. Fei-Fei, "Perceptual Losses for Real-Time Style Transfer and Super-Resolution," in *ECCV*, 2016.
- [112] P. Isola, J.-Y. Zhu, T. Zhou, et al., "Image-to-Image Translation with Conditional Adversarial Networks," in *CVPR*, 2017.
- [113] L. A. Gatys, A. S. Ecker, and M. Bethge, "Image Style Transfer Using Convolutional Neural Networks," in *CVPR*, 2016.
- [114] X. Huang and S. Belongie, "Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization," in *ICCV*, 2017.
- [115] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," in *NeurIPS Deep Learning Workshop*, 2015.
- [116] J. Song, C. Meng, and S. Ermon, "Denoising Diffusion Implicit Models," in *ICLR*, 2021.
- [117] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, et al., "Flow Matching for Generative Modeling," in *ICLR*, 2023.
- [118] J. Ho and T. Salimans, "Classifier-Free Diffusion Guidance," *arXiv:2207.12598*, 2022.
- [119] R. Rombach, A. Blattmann, D. Lorenz, et al., "High-Resolution Image Synthesis with Latent Diffusion Models," in *CVPR*, 2022.
- [120] W. Peebles and S. Xie, "Scalable Diffusion Models with Transformers," in *ICCV*, 2023.

- [121] L. Zhang, A. Rao, and M. Agrawala, "Adding Conditional Control to Text-to-Image Diffusion Models," in *ICCV*, 2023.
- [122] Y. Blau and T. Michaeli, "The Perception-Distortion Tradeoff," in *CVPR*, 2018.
- [123] C. Ledig, L. Theis, F. Huszár, et al., "Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network," in *CVPR*, 2017.
- [124] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., "Image Quality Assessment: From Error Visibility to Structural Similarity," *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [125] M. Heusel, H. Ramsauer, T. Unterthiner, et al., "GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium," in *NeurIPS*, 2017.
- [126] T. Brooks, B. Mildenhall, T. Xue, et al., "Unprocessing Images for Learned Raw Denoising," in *CVPR*, 2019.
- [127] A. Abdelhamed, S. Lin, and M. S. Brown, "A High-Quality Denoising Dataset for Smartphone Cameras," in *CVPR*, 2018.
- [128] J. Deng, W. Dong, R. Socher, et al., "ImageNet: A Large-Scale Hierarchical Image Database," in *CVPR*, 2009.
- [129] A. Radford, J. W. Kim, C. Hallacy, et al., "Learning Transferable Visual Models from Natural Language Supervision," in *ICML*, 2021.
- [130] J. Li, D. Li, S. Savarese, et al., "BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models," in *ICML*, 2023.
- [131] H. Liu, C. Li, Q. Wu, et al., "Visual Instruction Tuning," in *NeurIPS*, 2023.
- [132] S. Kornblith, J. Shlens, and Q. V. Le, "Do Better ImageNet Models Transfer Better?" In *CVPR*, 2019.
- [133] T. Chen, S. Kornblith, M. Norouzi, et al., "A Simple Framework for Contrastive Learning of Visual Representations," in *ICML*, 2020.
- [134] K. He, H. Fan, Y. Wu, et al., "Momentum Contrast for Unsupervised Visual Representation Learning," in *CVPR*, 2020.
- [135] J.-B. Grill, F. Strub, F. Altché, et al., "Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning," in *NeurIPS*, 2020.
- [136] M. Caron, H. Touvron, I. Misra, et al., "Emerging Properties in Self-Supervised Vision Transformers," in *ICCV*, 2021.
- [137] K. He, X. Chen, S. Xie, et al., "Masked Autoencoders Are Scalable Vision Learners," in *CVPR*, 2022.
- [138] A. van den Oord, Y. Li, and O. Vinyals, "Representation Learning with Contrastive Predictive Coding," in *arXiv:1807.03748*, 2018.
- [139] J. Kaplan, S. McCandlish, T. Henighan, et al., "Scaling Laws for Neural Language Models," *arXiv:2001.08361*, 2020.
- [140] X. Zhai, A. Kolesnikov, N. Houlsby, et al., "Scaling Vision Transformers," in *CVPR*, 2022.
- [141] J. Hoffmann, S. Borgeaud, A. Mensch, et al., "Training Compute-Optimal Large Language Models," in *NeurIPS*, 2022.
- [142] C. G. Northcutt, A. Athalye, and J. Mueller, "Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks," in *NeurIPS Datasets and Benchmarks*, 2021.
- [143] B. Recht, R. Roelofs, L. Schmidt, et al., "Do ImageNet Classifiers Generalize to ImageNet?" In *ICML*, 2019.
- [144] N. Houlsby, A. Giurgiu, S. Jastrzebski, et al., "Parameter-Efficient Transfer Learning for NLP," in *ICML*, 2019.

- [145] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [146] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *RecSys*, 2019.

### 5.3 NeRF, 3DGS và hợp nhất cảm biến (Neural Rendering and Sensor Fusion)

**PHỤ THUỘC** C/16-thi-giac-3d/01-bon-the-he-3d (thế hệ ba nằm ở đầu trong pipeline); C/16-thi-giac-3d/02-bieu-dien-3d-va-depth (trường ẩn và Gaussian trong bảng biểu diễn); A/02-vat-ly-tao-anh (phơi sáng, cân bằng trắng, hiệu chỉnh ngoại).

#### TÓM TẮT

Mục này giải thích cơ chế thật của *neural rendering*: vì sao NeRF phải biểu diễn cảnh bằng mật độ liên tục chứ không bằng bề mặt, và 3DGS đổi cái gì lấy tốc độ. Nửa sau chuyển sang bài toán hợp nhất camera–LiDAR–IMU. Đọc xong bạn quyết định được khi nào một bản đồ render được là đáng giá cho robot, và khi nào bản đồ thừa cổ điển vẫn đúng hơn. Bạn cũng tính được bằng con số vì sao *early fusion* nhạy với lệch hiệu chỉnh còn *late fusion* thì không. Nếu chỉ nhớ một điều: hàm mất mát của neural rendering chỉ ràng buộc *ảnh render*, không ràng buộc *hình học*. Nên PSNR cao không chứng minh bản đồ đúng. Nếu robot sắp va chạm dựa trên bản đồ đó, phải đánh giá bằng độ chính xác hình học so với một quét chân trị.

#### KIẾN THỨC CẦN CÓ TRƯỚC

- **novel view synthesis ( dựng ảnh ở góc nhìn mới )** — bài toán: cho một số ảnh đã chụp của một cảnh, hãy tạo ra ảnh của cảnh đó từ một vị trí camera chưa từng chụp.
- **radiance field (trường bức xạ)** — một hàm gán cho mỗi điểm trong không gian và mỗi hướng nhìn một màu và một độ đặc; đây là “cảnh” mà NeRF học, thay cho bề mặt.
- **volume rendering ( dựng ảnh thể tích )** — cách tổng hợp một pixel bằng cách lấy mẫu nhiều điểm dọc theo tia nhìn rồi cộng dồn màu có trọng số theo độ đục tích lũy — thay vì tìm giao điểm với một bề mặt cứng.
- **splat / Gaussian splatting** — “splat” là vết loang một hạt để lại khi bị chiếu xuống mặt ảnh. 3DGS mô tả cảnh bằng hàng triệu hạt Gaussian mờ, và render là chiếu chúng xuống rồi trộn theo thứ tự độ sâu.
- **rasterize** — chuyển một hình học đã chiếu thành pixel bằng cách quét trực tiếp trên lưới ảnh; nhanh hơn nhiều so với đi dò dọc từng tia, và là thứ GPU sinh ra để làm.
- **PSNR** — chỉ số đo sai lệch ảnh dựa trên bình phương trung bình theo pixel, tính bằng decibel; cao là “gần ảnh gốc theo nghĩa số học”, không phải “trông giống theo nghĩa thị giác”.
- **hiệu chỉnh ngoại (extrinsic calibration)** — phép xác định vị trí và hướng tương đối giữa hai cảm biến; sai một độ ở đây biến thành sai vài chục centimet ở tầm hai chục mét.
- **early / late fusion** — hai tầng hợp nhất cảm biến: gộp ở mức đặc trưng (sớm, giàu thông tin) hoặc gộp ở mức kết quả từng nhánh (muộn, bền hơn).

#### A. Đặt vấn đề

Giả sử SfM đã chạy xong: bạn có pose của bốn mươi tám ảnh và một point cloud thừa. Bây giờ hãy render cảnh đó từ một góc nhìn chưa từng chụp. Kết quả sẽ tệ, và tệ theo cách rất đặc trưng: thủng lỗ ở chỗ không có điểm, viền răng cưa ở biên vật thể, mặt kính và mặt kim loại sai hoàn toàn vì màu của chúng phụ thuộc góc nhìn còn point cloud chỉ lưu được một màu cho mỗi điểm.

Vấn đề nằm ở chỗ pipeline cổ điển tối ưu cho *độ chính xác hình học* rồi hy vọng ảnh đẹp theo sau, trong khi cái ta cần ở đây là ngược lại.

Neural rendering đảo ngược thứ tự đó bằng một ý tưởng đơn giản tới mức khó chịu: *nếu thứ ta muốn là ảnh, hãy đặt ảnh vào hàm mất mát*. Tham số hoá cảnh bằng một biểu diễn nào đó, viết một bộ render **khả vi** (*differentiable*) từ biểu diễn đó ra ảnh, rồi hạ **gradient** trên tham số của biểu diễn cho tới khi ảnh render trùng khít mọi ảnh quan sát. Không có bước tìm tương ứng, không tam giác hoá, không ràng buộc epipolar nào được viết ra — hình học nếu có xuất hiện thì xuất hiện như một *tác dụng phụ* của việc khớp ảnh. Câu “nếu có” đó là toàn bộ nội dung mục này.

**Học mục này thì làm được gì mà trước đó không làm được?** Ba việc rất cụ thể: đánh giá được một đề xuất kiểu “thay bản đồ SLAM bằng Gaussian” thay vì phải tin hay không tin; đọc được một bài neural rendering và chỉ ra ngay giả định nào trong ba giả định nền của nó bị dữ liệu robot vi phạm; và biến câu “hiệu chỉnh phải chính xác” thành một ngân sách sai số tính bằng centimét ở một tầm xa cụ thể.

#### TRỰC GIÁC

Vì sao NeRF dựng khối chứ không dựng tường? Vì đạo hàm. Nếu cảnh gồm các bề mặt cứng, dịch một bề mặt đi một phần nghìn mét thì một pixel hoặc đổi màu hẳn, hoặc không đổi gì cả. Hàm mất mát khi đó là bậc thang: gradient bằng không gần như mọi nơi, và hạ **gradient** (*gradient descent*) không biết vận về hướng nào. Thay bề mặt cứng bằng một trường mật độ liên tục thì mỗi pixel trở thành tổng có trọng số của hàng chục điểm dọc theo tia, và *mọi* điểm đều nhận được một phần gradient. Đây là phép đổi một công tắc bật/tắt thành một chiết áp: chỉ chiết áp mới nói được nên vận theo chiều nào.

#### B. NeRF: cảnh là một hàm, ảnh là tích phân dọc tia

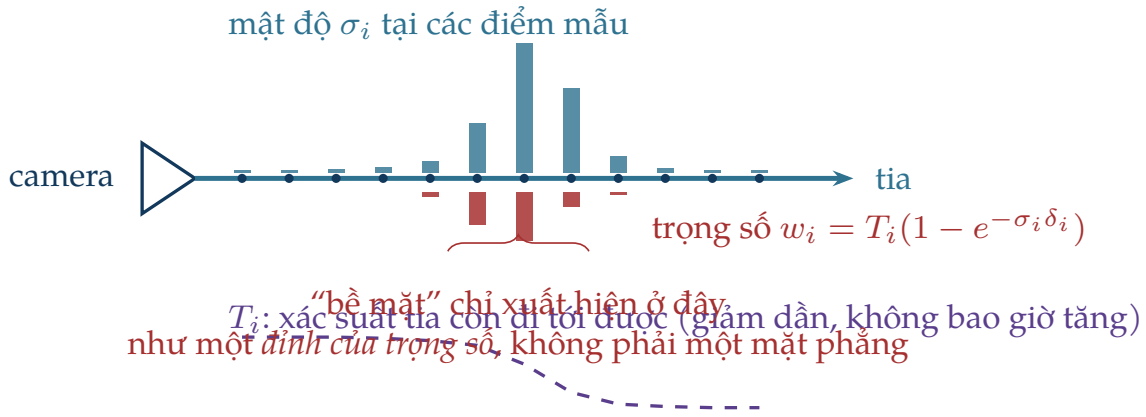
NeRF biểu diễn cảnh bằng một MLP nhận toạ độ 3D và hướng nhìn, trả về mật độ  $\sigma$  và màu  $c$  [95]. Để render một pixel, lấy mẫu  $N$  điểm dọc tia đi qua pixel đó rồi tổng hợp theo công thức dựng ảnh thể tích rồi raster hoá:

$$C(\mathbf{r}) = \sum_{i=1}^N T_i (1 - e^{-\sigma_i \delta_i}) c_i, \quad T_i = \exp\left(-\sum_{j<i} \sigma_j \delta_j\right).$$

*Công thức này thực sự nói gì:  $T_i$  là xác suất tia đi tới được điểm thứ  $i$  mà chưa bị chặn, còn  $1 - e^{-\sigma_i \delta_i}$  là xác suất nó dừng lại đúng ở đó; vậy màu pixel là kỳ vọng của màu tại điểm tia dừng lại.* Điều đáng nhớ là mọi số hạng đều khả vi theo  $\sigma$  và  $c$ , nên gradient của sai số ảnh chảy ngược được về mọi điểm mẫu — đó là toàn bộ lý do NeRF hoạt động; phần MLP chỉ là một lựa chọn triển khai, và mục sau sẽ thay nó.

↪ **tiền đề C/12/01** — toàn bộ cơ chế đứng trên một điều duy nhất: mọi nút trên đường từ tham số tới ảnh phải khả vi.

Hình 5.10 vẽ đúng ba đại lượng trong công thức trên một tia duy nhất.

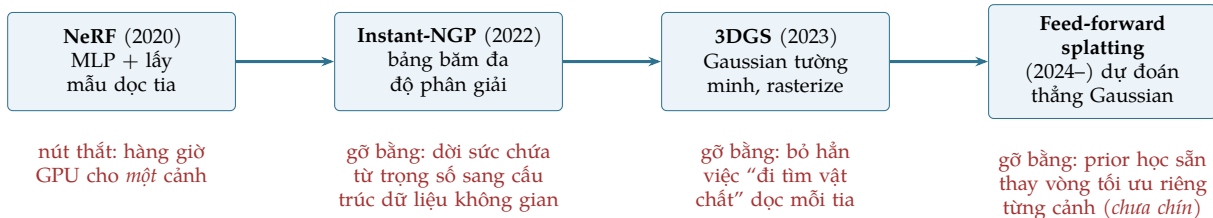


**Hình 5.10.** Dựng ảnh thể tích, vẽ trên một tia. Mạng không trả lời câu hỏi “mặt ở đâu” mà trả lời “mật độ tại điểm này bao nhiêu” ở hàng chục điểm dọc tia; màu pixel là tổng có trọng số của màu tại các điểm đó. Ba hàng trong hình giải thích công thức: mật độ  $\sigma_i$  quyết định tia dừng lại bao nhiêu ở mỗi điểm,  $T_i$  là phần tia còn sống sót tới đó, và tích của hai thứ cho trọng số. Vì mọi số hạng đều liên tục, mọi điểm mẫu đều nhận được gradient — đó chính là điều mà một bề mặt cứng không làm được.

Một chi tiết dễ bỏ qua nhưng quyết định chất lượng là **mã hoá vị trí** (*positional encoding*): nhồi thẳng toạ độ vào MLP cho ra ảnh mờ nhoè, vì MLP có thiên lệch mạnh về hàm tần số thấp. Ánh xạ toạ độ qua một dãy sin/cos nhiều tần số trước khi đưa vào mạng là cách trả lại cho mạng khả năng biểu diễn chi tiết sắc nét — cùng câu chuyện *spectral bias* đã gặp ở B/10-thiet-ke-kien-truc.

↪ **hệ quả của B/10** — mã hoá vị trí là cách bù cho một thiên lệch quy nạp có sẵn của MLP, chứ không phải một mẹo kỹ thuật rời rạc.

Cái giá là chi phí tính toán, và nó lớn hơn trực giác nhiều. Một ảnh  $800 \times 600$  có 480 000 tia; với 64 điểm mẫu thô cộng 128 điểm mẫu tinh, ta cần khoảng  $9,2 \times 10^7$  lần chạy MLP cho *một* khung hình. Nhân với hàng chục nghìn bước huấn luyện thì thành nhiều giờ GPU cho một cảnh duy nhất — và đây là nút thắt sinh ra cả dòng công trình tiếp theo, tóm tắt ở Hình 5.11.



**Hình 5.11.** Dòng thời gian của neural rendering, đọc theo nút thắt. Mỗi bước không phải “phiên bản tốt hơn” mà là câu trả lời cho đúng một nút cổ chai của bước trước; biết nút thắt nào đang cản trong bài toán của mình thì biết nên dừng ở mốc nào.

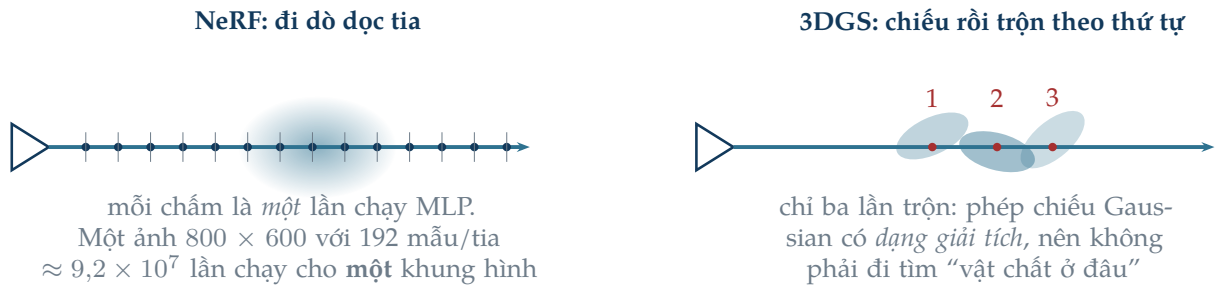
Instant-NGP tấn công đúng chi phí đó. Nó chuyển phần lớn sức chứa của mô hình từ trọng số MLP sang một bảng băm đa độ phân giải đặt trong không gian, để MLP chỉ còn nhỏ xíu [103]. Thời gian huấn luyện tụt từ hàng giờ xuống hàng chục giây. Đây là một mẫu hình sẽ gặp lại nhiều lần: khi mạng phải học một hàm có cấu trúc không gian rõ ràng, đưa cấu trúc đó vào *cấu trúc dữ liệu* rẻ hơn nhiều so với bất trọng số học lại nó.

### C. 3DGS: đối biểu diễn ẩn lấy biểu diễn tường minh

Ngay cả với bảng băm, NeRF vẫn phải đi dọc từng tia và lấy mẫu — vẫn “đi tìm vật chất ở đâu” cho mỗi pixel mỗi khung hình. 3D Gaussian splatting bỏ hẳn bước tìm kiếm đó: cảnh là một tập hữu hạn Gaussian dị hướng, mỗi cái có vị trí, ma trận hiệp phương sai, độ đục và màu phụ thuộc góc nhìn; render là chiếu từng Gaussian xuống mặt phẳng ảnh, sắp xếp theo độ sâu, rồi trộn theo

thứ tự [104]. Toán tổng hợp màu vẫn đúng công thức trên, nhưng phép chiếu có dạng giải tích nên bộ rasterize chạy được ở tốc độ thời gian thực.

Hình 5.12 đặt hai cách tính cùng một pixel cạnh nhau, và cho thấy khoản tiết kiệm đến từ đâu.



**Hình 5.12.** Cùng một tia, hai cách tính màu pixel. Bên trái, NeRF phải lấy mẫu dày dọc tia vì nó *không biết* trước vật chất nằm ở đâu — toàn bộ chi phí đến từ việc đi tìm. Bên phải, 3DGS đã lưu sẵn vật chất thành các nguyên thể tường minh, nên chỉ cần chiếu chúng xuống mặt ảnh, sắp xếp theo độ sâu và trộn ba lần. Toán tổng hợp màu ở hai bên là một; thứ thay đổi là ai đi tìm — và đó là toàn bộ khoản chênh hai tới ba bậc độ lớn về tốc độ render.

**Bảng 5.3.** Ba mốc của neural rendering. Cả ba chia chung một tập giả định — pose đã biết, cảnh tĩnh, phơi sáng nhất quán — nên hồng chung ở những chỗ đó, và hồng riêng ở cột cuối.

| Phương pháp           | Cơ chế                                          | Chi phí                                             | Hồng khi / mất gì                                                  |
|-----------------------|-------------------------------------------------|-----------------------------------------------------|--------------------------------------------------------------------|
| NeRF (MLP)            | MLP + lấy mẫu dọc tia + tích phân thể tích      | Huấn luyện hàng giờ; render < 1 fps; mô hình vài MB | Chậm cả hai đầu; không chỉnh sửa được cục bộ                       |
| Instant-NGP           | Bảng băm đa độ phân giải thay phần lớn trọng số | Huấn luyện hàng chục giây; tốn VRAM                 | Va chạm băm gây nhiễu ở vùng chi tiết                              |
| 3D Gaussian splatting | Tập Gaussian tường minh, rasterize có sắp xếp   | Render thời gian thực; mô hình vài trăm MB          | Dung lượng lớn; bề mặt không tường minh; nhạy với nhiễu tạo từ SfM |

**Tính tay dung lượng một lần cho nhớ.** Mỗi Gaussian lưu 3 số vị trí, 3 số tỉ lệ, 4 số quaternion, 1 số độ đục và 48 hệ số **hàm điều hoà cầu** (*spherical harmonics*) bậc ba cho màu: tổng 59 số thực 32-bit, tức 236 byte. Một cảnh phòng khách dựng tốt thường cần cỡ một triệu Gaussian, tức khoảng 236 MB — so với vài megabyte cho trọng số của một NeRF MLP. Đó chính là “giá phải trả” hay được nhắc chung chung: đổi khoảng hai bậc độ lớn dung lượng lấy khoảng hai tới ba bậc độ lớn tốc độ render.

Cái mất thứ hai tinh tế hơn và quan trọng hơn với người làm robot: **hình học kém tường minh**. Trong một trường *SDF*, bề mặt là tập mức 0 — định nghĩa sạch, trích mesh bằng *marching cubes* là chuyện cơ học. Trong 3DGS, “bề mặt” không được định nghĩa ở đâu cả: tối ưu chỉ đòi ảnh render đúng, nên nó thoải mái đặt các Gaussian dẹt lơ lửng trước một bức tường nếu làm vậy khớp ảnh tốt hơn. Muốn trích mesh dùng được, người ta phải thêm ràng buộc ép Gaussian bám bề mặt (dẹt hoá, gán pháp tuyến) — tức là trả lại đúng phần ràng buộc hình học mà bước đầu đã cố tình bỏ đi.



#### D. Ba giả định chung, và điều gì xảy ra khi chúng sai

Cả họ neural rendering đứng trên ba giả định, và với dữ liệu quay bằng camera robot thì cả ba đều dễ vỡ:

- **Pose đã biết và chính xác.** Sai lệch vài phần mười độ làm các ảnh mâu thuẫn nhau; vì tối ưu luôn tìm cách giảm loss, nó bù bằng cách sinh ra vật chất giả lơ lửng — hiện tượng *floater*. Đây là lý do 3DGS “nhảy với khối tạo từ SfM”: rác vào thì rác ra, chỉ khác là rác ra trông rất đẹp.
- **Cảnh tĩnh.** Người đi qua khung hình để lại vệt mờ hoặc khối đục lơ lửng ở giữa phòng.
- **Quá trình tạo ảnh nhất quán.** Camera bật phơi sáng và cân bằng trắng tự động vì phạm giả định này liên tục, và mô hình buộc phải giải thích thay đổi độ sáng bằng cách nhét thêm hình học sai. **Ảnh mờ do chuyển động** (*motion blur*) cũng thuộc nhóm này: một khung hình mờ không phải quan sát nhiều của cảnh tĩnh mà là tích phân của cảnh dọc một quãng chuyển động, và mô hình render không có chỗ nào biểu diễn điều đó.

#### CẠM BẮY

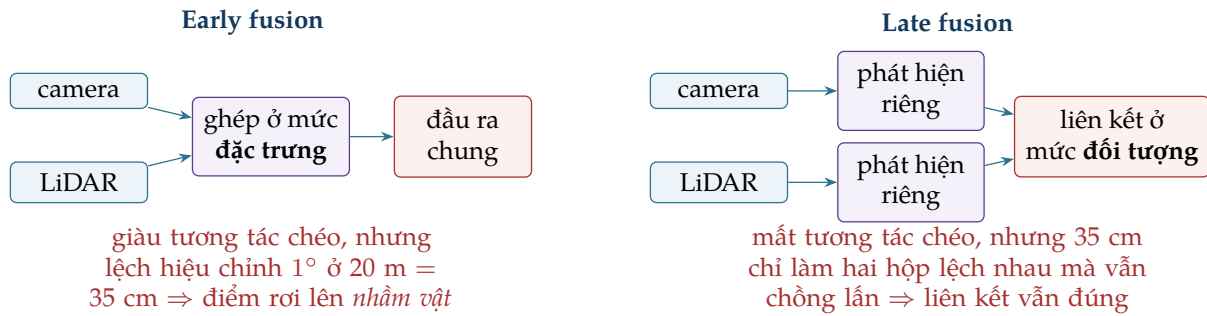
PSNR cao trên các view giữ lại *không* chứng minh hình học đúng. Hàm mất mát chỉ ràng buộc ảnh render; hai cấu hình mật độ rất khác nhau có thể sinh ra cùng bộ ảnh, đặc biệt khi màu được phép phụ thuộc hướng nhìn — mô hình có thể “vẽ” thứ đáng lẽ phải “dựng”. Triệu chứng chẩn đoán: chất lượng đẹp ở các view nội suy gần quỹ đạo chụp và sập ngay khi ngoại suy ra ngoài. Nếu bạn định dùng đầu ra làm bản đồ để robot va chạm hay bám bề mặt, hãy đánh giá bằng độ chính xác hình học so với một quét chân trị, không phải bằng PSNR hay *LPIPS* [105].

Với người làm SLAM, hệ quả thực hành gọn lại thành một câu hỏi: *bạn có thực sự cần render không?* Nếu chỉ cần định vị, bản đồ thừa của SLAM cổ điển vẫn là lựa chọn đúng — rẻ hơn nhiều bậc và có sai số kiểm chứng được. Neural rendering đáng đưa vào khi bài toán hạ nguồn thật sự cần ảnh: kiểm tra trực quan từ xa, sinh dữ liệu huấn luyện từ góc nhìn mới, mô phỏng cảm biến, hoặc định vị lại bằng cách so ảnh render với ảnh thật.

→ cùng nguyên lý với C/17/02 — cùng một ý tưởng “đặt thứ ta muốn vào hàm mất mát rồi tối ưu trên chính đối tượng cần tạo ra”, chỉ khác biến tối ưu là ảnh thay vì cảnh.

#### E. Hợp nhất camera, LiDAR và IMU

Ba cảm biến này bù trừ cho nhau gần như hoàn hảo: camera cho ngữ nghĩa và độ phân giải góc cao nhưng không có tỉ lệ; LiDAR cho hình học metric thừa và bền với ánh sáng; IMU cho chuyển động tần số cao và quan sát được phương trọng lực nhưng trôi rất nhanh khi tích phân. Câu hỏi thiết kế là hợp nhất ở tầng nào (Hình 5.13).



**Hình 5.13.** Cùng một sai số hiệu chỉnh, hai hậu quả khác hẳn nhau. Early fusion liên kết ở mức pixel nên sai số biến thành *đầu vào rác*; late fusion liên kết ở mức đối tượng, nơi vài chục centimet vẫn nằm trong kích thước của vật, nên sai số chỉ thành nhiễu đầu ra. Đó là toàn bộ nội dung của cuộc đánh đổi “giàu thông tin” đổi lại “bền vững”.

Con số trong hình tính ra như sau: sai số hiệu chỉnh ngoại xoay  $1^\circ$  chuyển thành sai số ngang  $\tan 1^\circ \approx 17,5$  mm trên mỗi mét tầm xa, nên ở 20 m là 35 cm. Lệch đồng bộ thời gian sinh ra đúng loại lỗi đó, nhưng âm thầm hơn vì nó chỉ xuất hiện khi có chuyển động. Robot quay  $100^\circ/\text{s}$  với lệch nhãn thời gian 5 ms cho sai lệch góc  $0,5^\circ$ . Con số này tỉ lệ thuận với tốc độ quay. Nghĩa là hệ thống chạy hoàn hảo khi đứng yên, rồi hỏng đúng lúc đang xoay nhanh — đúng lúc bạn cần nó nhất. Đây là lý do một nhận định lặp đi lặp lại trong thực hành: ở các hệ đa cảm biến, phần lớn thời gian sửa lỗi nằm ở đồng bộ thời gian và hiệu chỉnh ngoại chứ không ở thuật toán hợp nhất.

#### CẦN NHỚ

Sai số hiệu chỉnh và sai số đồng bộ không phải nhiễu — chúng là sai số *có hệ thống*, phụ thuộc tầm xa và tốc độ, nên không trung bình về không khi gom nhiều quan sát. Hệ quả bất buộc: chúng phải được đưa vào mô hình như biến cần ước lượng (hiệu chỉnh trực tuyến, ước lượng độ trễ  $t_d$ ) chứ không phải được hy vọng là nhỏ. Và với neural rendering, điều tương đương là: hàm mất mát ràng buộc ảnh, nên hình học phải được kiểm tra bằng một phép đo bên ngoài. [T4]

## F. Giới hạn và trường hợp hỏng

Giới hạn chung của cả họ neural rendering là tính riêng cho từng cảnh: một mô hình đã tối ưu xong không dùng lại được cho cảnh khác, nên chi phí là chi phí *lặp lại* chứ không phải chi phí một lần như huấn luyện một *backbone*. Các dòng công trình lai — mô hình tiến thẳng dự đoán thẳng tập Gaussian — đang xoá dần ranh giới này, nhưng tại thời điểm viết, chất lượng của chúng vẫn dưới bản tối ưu riêng từng cảnh; đây là quan sát về xu hướng, chưa phải một kết quả đã ổn định.

Về hợp nhất, chế độ hỏng đáng sợ nhất là hỏng *im lặng* của một nhánh. Mưa làm LiDAR sinh điểm giả; đường hầm làm camera mất hết đặc trưng; rung động làm IMU bão hoà. Một hệ hợp nhất không kiểm tra tính nhất quán giữa các nhánh sẽ vẫn cho ra ước lượng trông bình thường, với hiệp phương sai cũng trông bình thường. Vì vậy phần đáng đầu tư trong một hệ thật thường không phải bộ hợp nhất mà là bộ chẩn đoán từng nhánh: phần dư theo từng cảm biến, kiểm tra chi bình phương, và một cơ chế hạ cấp có kiểm soát khi một nhánh bị loại.

↔ **hệ quả của** E/25 — hỏng im lặng của một nhánh cảm biến là trường hợp cụ thể của yêu cầu chung: hệ thống phải biết lúc nào nó không đáng tin.

## G. Kết nối

Mục này khép lại chương 16 và nối ngược lên [C/16-thi-giac-3d/01-bon-the-he-3d](#): neural rendering là thể hệ ba, đứng *sau* khâu pose, nên nó tiêu thụ đầu ra của thể hệ một chứ không thay thế. Ý tưởng “đặt cái ta muốn vào hàm mất mát rồi hạ gradient trên chính đối tượng cần tạo ra” sẽ trở lại nguyên vẹn ở [C/17-mo-hinh-sinh/02-toi-uu-tren-pixel](#), chỉ khác là biến tối ưu ở đó là ảnh chứ không phải cảnh. Các giả định về quá trình tạo ảnh thuộc [A/02-vat-ly-tao-anh](#); việc chẩn đoán từng nhánh cảm biến và hạ cấp có kiểm soát thuộc [E/25-do-tin-cay-va-van-hanh](#).

### ■ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao biểu diễn cảnh bằng mật độ liên tục là điều kiện cần để gradient descent hoạt động, trong khi bề mặt cứng thì không?
2. 3DGS nhanh hơn NeRF nhờ đổi cái gì lấy cái gì, và vì sao cái mất đó lại đặc biệt bất tiện cho robot?
3. Một cảnh có PSNR rất cao trên các view giữ lại nhưng bản đồ hình học sai. Cơ chế nào cho phép hai điều đó cùng đúng, và bạn thiết kế phép thử nào để phát hiện?
4. Vì sao một hệ early fusion có thể chạy hoàn hảo khi robot đứng yên và hỏng khi robot xoay nhanh, dù không tham số nào thay đổi?
5. Nếu buộc phải chọn giữa một tuần cải tiến thuật toán hợp nhất và một tuần dựng lại quy trình hiệu chỉnh cùng đồng bộ thời gian, bạn chọn cái nào và dựa trên bằng chứng gì?
6. Hình [5.11](#) đọc theo nút thất. Trong bài toán của bạn, nút thất đang cần là thời gian dựng, tốc độ render, hay dung lượng bản đồ — và điều đó chỉ về mốc nào?

- [1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [2] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [3] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *AISTATS*, 2010.
- [4] K. He, X. Zhang, S. Ren, et al., “Delving Deep into Rectifiers,” in *ICCV*, 2015.
- [5] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [6] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [7] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [8] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [9] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [10] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [11] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” in *ICLR*, 2017.
- [12] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [13] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [14] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [15] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [16] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [17] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [18] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [19] G. Huang, Y. Sun, Z. Liu, et al., “Deep Networks with Stochastic Depth,” in *ECCV*, 2016.
- [20] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [21] H. Zhang, M. Cisse, Y. N. Dauphin, et al., “mixup: Beyond Empirical Risk Minimization,” in *ICLR*, 2018.
- [22] S. Yun, D. Han, S. J. Oh, et al., “CutMix: Regularization Strategy to Train Strong Classifiers,” in *ICCV*, 2019.
- [23] E. D. Cubuk, B. Zoph, J. Shlens, et al., “RandAugment: Practical Automated Data Augmentation,” in *CVPR Workshops*, 2020.

- [24] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [25] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [26] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *NeurIPS*, 2012.
- [27] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in *ICLR*, 2015.
- [28] C. Szegedy, W. Liu, et al., “Going Deeper with Convolutions,” in *CVPR*, 2015.
- [29] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [30] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [31] G. Huang, Z. Liu, L. van der Maaten, et al., “Densely Connected Convolutional Networks,” in *CVPR*, 2017.
- [32] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [33] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [34] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [35] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [36] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [37] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.
- [38] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [39] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [40] J. Su, M. Ahmed, Y. Lu, et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, vol. 568, 2024.
- [41] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” *arXiv:2312.00752*, 2023.
- [42] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [43] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.
- [44] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [45] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [46] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [47] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [48] R. Girshick, J. Donahue, T. Darrell, et al., “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation,” in *CVPR*, 2014.

- [49] R. Girshick, "Fast R-CNN," in *ICCV*, 2015.
- [50] S. Ren, K. He, R. Girshick, et al., "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in *NeurIPS*, 2015.
- [51] T.-Y. Lin, P. Dollár, R. Girshick, et al., "Feature Pyramid Networks for Object Detection," in *CVPR*, 2017.
- [52] K. He, G. Gkioxari, P. Dollár, et al., "Mask R-CNN," in *ICCV*, 2017.
- [53] D. Bolya, S. Foley, J. Hays, et al., "TIDE: A General Toolbox for Identifying Object Detection Errors," in *ECCV*, 2020.
- [54] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [55] J. Redmon, S. Divvala, R. Girshick, et al., "You Only Look Once: Unified, Real-Time Object Detection," in *CVPR*, 2016.
- [56] W. Liu, D. Anguelov, D. Erhan, et al., "SSD: Single Shot MultiBox Detector," in *ECCV*, 2016.
- [57] T.-Y. Lin, P. Goyal, R. Girshick, et al., "Focal Loss for Dense Object Detection," in *ICCV*, 2017.
- [58] Z. Tian, C. Shen, H. Chen, et al., "FCOS: Fully Convolutional One-Stage Object Detection," in *ICCV*, 2019.
- [59] X. Zhou, D. Wang, and P. Krähenbühl, "Objects as Points," *arXiv:1904.07850*, 2019.
- [60] S. Zhang, C. Chi, Y. Yao, et al., "Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection," in *CVPR*, 2020.
- [61] Z. Ge, S. Liu, Z. Li, et al., "OTA: Optimal Transport Assignment for Object Detection," in *CVPR*, 2021.
- [62] N. Bodla, B. Singh, R. Chellappa, et al., "Soft-NMS: Improving Object Detection with One Line of Code," in *ICCV*, 2017.
- [63] X. Zhu, W. Su, L. Lu, et al., "Deformable DETR: Deformable Transformers for End-to-End Object Detection," in *ICLR*, 2021.
- [64] H. Zhang, F. Li, S. Liu, et al., "DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection," in *ICLR*, 2023.
- [65] J. Long, E. Shelhamer, and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," in *CVPR*, 2015.
- [66] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *MICCAI*, 2015.
- [67] L.-C. Chen, G. Papandreou, I. Kokkinos, et al., "DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs," *IEEE TPAMI*, vol. 40, no. 4, pp. 834–848, 2018.
- [68] E. Xie, W. Wang, Z. Yu, et al., "SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers," in *NeurIPS*, 2021.
- [69] B. Cheng, I. Misra, A. G. Schwing, et al., "Masked-Attention Mask Transformer for Universal Image Segmentation," in *CVPR*, 2022.
- [70] A. Kirillov, K. He, R. Girshick, et al., "Panoptic Segmentation," in *CVPR*, 2019.
- [71] A. Kirillov, E. Mintun, N. Ravi, et al., "Segment Anything," in *ICCV*, 2023.
- [72] K. Sun, B. Xiao, D. Liu, et al., "Deep High-Resolution Representation Learning for Human Pose Estimation," in *CVPR*, 2019.



- [73] Z. Cao, T. Simon, S.-E. Wei, et al., “Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields,” in *CVPR*, 2017.
- [74] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [75] J. Deng, J. Guo, N. Xue, et al., “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *CVPR*, 2019.
- [76] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [77] K. Xu, J. Ba, R. Kiros, et al., “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention,” in *ICML*, 2015.
- [78] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *ICLR*, 2015.
- [79] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [80] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [81] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [82] K. Simonyan and A. Zisserman, “Two-Stream Convolutional Networks for Action Recognition in Videos,” in *NeurIPS*, 2014.
- [83] J. Carreira and A. Zisserman, “Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset,” in *CVPR*, 2017.
- [84] D. Tran, H. Wang, L. Torresani, et al., “A Closer Look at Spatiotemporal Convolutions for Action Recognition,” in *CVPR*, 2018.
- [85] C. Feichtenhofer, H. Fan, J. Malik, et al., “SlowFast Networks for Video Recognition,” in *ICCV*, 2019.
- [86] A. Dosovitskiy, P. Fischer, E. Ilg, et al., “FlowNet: Learning Optical Flow with Convolutional Networks,” in *ICCV*, 2015.
- [87] D. Sun, X. Yang, M.-Y. Liu, et al., “PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume,” in *CVPR*, 2018.
- [88] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.
- [89] A. Bewley, Z. Ge, L. Ott, et al., “Simple Online and Realtime Tracking,” in *ICIP*, 2016.
- [90] Y. Zhang, P. Sun, Y. Jiang, et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *ECCV*, 2022.
- [91] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [92] K. Bernardin and R. Stiefelhagen, “Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics,” *EURASIP J. Image and Video Processing*, 2008.
- [93] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [94] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [95] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.

- [96] S. Wang, V. Leroy, Y. Cabon, et al., "DUST3R: Geometric 3D Vision Made Easy," in *CVPR*, 2024.
- [97] D. Eigen, C. Puhrsch, and R. Fergus, "Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network," in *NeurIPS*, 2014.
- [98] R. Ranftl, K. Lasinger, D. Hafner, et al., "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer," *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [99] J. Sturm, N. Engelhard, F. Endres, et al., "A Benchmark for the Evaluation of RGB-D SLAM Systems," in *IROS*, 2012.
- [100] C. R. Qi, H. Su, K. Mo, et al., "PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation," in *CVPR*, 2017.
- [101] C. R. Qi, L. Yi, H. Su, et al., "PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space," in *NeurIPS*, 2017.
- [102] A. H. Lang, S. Vora, H. Caesar, et al., "PointPillars: Fast Encoders for Object Detection from Point Clouds," in *CVPR*, 2019.
- [103] T. Müller, A. Evans, C. Schied, et al., "Instant Neural Graphics Primitives with a Multiresolution Hash Encoding," *ACM TOG*, vol. 41, no. 4, 2022.
- [104] B. Kerbl, G. Kopanas, T. Leimkühler, et al., "3D Gaussian Splatting for Real-Time Radiance Field Rendering," *ACM TOG*, vol. 42, no. 4, 2023.
- [105] R. Zhang, P. Isola, A. A. Efros, et al., "The Unreasonable Effectiveness of Deep Features as a Perceptual Metric," in *CVPR*, 2018.
- [106] D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," in *ICLR*, 2014.
- [107] I. Goodfellow, J. Pouget-Abadie, M. Mirza, et al., "Generative Adversarial Nets," in *NeurIPS*, 2014.
- [108] T. Karras, S. Laine, and T. Aila, "A Style-Based Generator Architecture for Generative Adversarial Networks," in *CVPR*, 2019.
- [109] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," in *NeurIPS*, 2020.
- [110] Y. Song, J. Sohl-Dickstein, D. P. Kingma, et al., "Score-Based Generative Modeling Through Stochastic Differential Equations," in *ICLR*, 2021.
- [111] J. Johnson, A. Alahi, and L. Fei-Fei, "Perceptual Losses for Real-Time Style Transfer and Super-Resolution," in *ECCV*, 2016.
- [112] P. Isola, J.-Y. Zhu, T. Zhou, et al., "Image-to-Image Translation with Conditional Adversarial Networks," in *CVPR*, 2017.
- [113] L. A. Gatys, A. S. Ecker, and M. Bethge, "Image Style Transfer Using Convolutional Neural Networks," in *CVPR*, 2016.
- [114] X. Huang and S. Belongie, "Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization," in *ICCV*, 2017.
- [115] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," in *NeurIPS Deep Learning Workshop*, 2015.
- [116] J. Song, C. Meng, and S. Ermon, "Denoising Diffusion Implicit Models," in *ICLR*, 2021.
- [117] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, et al., "Flow Matching for Generative Modeling," in *ICLR*, 2023.
- [118] J. Ho and T. Salimans, "Classifier-Free Diffusion Guidance," *arXiv:2207.12598*, 2022.
- [119] R. Rombach, A. Blattmann, D. Lorenz, et al., "High-Resolution Image Synthesis with Latent Diffusion Models," in *CVPR*, 2022.
- [120] W. Peebles and S. Xie, "Scalable Diffusion Models with Transformers," in *ICCV*, 2023.

- [121] L. Zhang, A. Rao, and M. Agrawala, "Adding Conditional Control to Text-to-Image Diffusion Models," in *ICCV*, 2023.
- [122] Y. Blau and T. Michaeli, "The Perception-Distortion Tradeoff," in *CVPR*, 2018.
- [123] C. Ledig, L. Theis, F. Huszár, et al., "Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network," in *CVPR*, 2017.
- [124] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., "Image Quality Assessment: From Error Visibility to Structural Similarity," *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [125] M. Heusel, H. Ramsauer, T. Unterthiner, et al., "GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium," in *NeurIPS*, 2017.
- [126] T. Brooks, B. Mildenhall, T. Xue, et al., "Unprocessing Images for Learned Raw Denoising," in *CVPR*, 2019.
- [127] A. Abdelhamed, S. Lin, and M. S. Brown, "A High-Quality Denoising Dataset for Smartphone Cameras," in *CVPR*, 2018.
- [128] J. Deng, W. Dong, R. Socher, et al., "ImageNet: A Large-Scale Hierarchical Image Database," in *CVPR*, 2009.
- [129] A. Radford, J. W. Kim, C. Hallacy, et al., "Learning Transferable Visual Models from Natural Language Supervision," in *ICML*, 2021.
- [130] J. Li, D. Li, S. Savarese, et al., "BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models," in *ICML*, 2023.
- [131] H. Liu, C. Li, Q. Wu, et al., "Visual Instruction Tuning," in *NeurIPS*, 2023.
- [132] S. Kornblith, J. Shlens, and Q. V. Le, "Do Better ImageNet Models Transfer Better?" In *CVPR*, 2019.
- [133] T. Chen, S. Kornblith, M. Norouzi, et al., "A Simple Framework for Contrastive Learning of Visual Representations," in *ICML*, 2020.
- [134] K. He, H. Fan, Y. Wu, et al., "Momentum Contrast for Unsupervised Visual Representation Learning," in *CVPR*, 2020.
- [135] J.-B. Grill, F. Strub, F. Altché, et al., "Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning," in *NeurIPS*, 2020.
- [136] M. Caron, H. Touvron, I. Misra, et al., "Emerging Properties in Self-Supervised Vision Transformers," in *ICCV*, 2021.
- [137] K. He, X. Chen, S. Xie, et al., "Masked Autoencoders Are Scalable Vision Learners," in *CVPR*, 2022.
- [138] A. van den Oord, Y. Li, and O. Vinyals, "Representation Learning with Contrastive Predictive Coding," in *arXiv:1807.03748*, 2018.
- [139] J. Kaplan, S. McCandlish, T. Henighan, et al., "Scaling Laws for Neural Language Models," *arXiv:2001.08361*, 2020.
- [140] X. Zhai, A. Kolesnikov, N. Houlsby, et al., "Scaling Vision Transformers," in *CVPR*, 2022.
- [141] J. Hoffmann, S. Borgeaud, A. Mensch, et al., "Training Compute-Optimal Large Language Models," in *NeurIPS*, 2022.
- [142] C. G. Northcutt, A. Athalye, and J. Mueller, "Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks," in *NeurIPS Datasets and Benchmarks*, 2021.
- [143] B. Recht, R. Roelofs, L. Schmidt, et al., "Do ImageNet Classifiers Generalize to ImageNet?" In *ICML*, 2019.
- [144] N. Houlsby, A. Giurgiu, S. Jastrzebski, et al., "Parameter-Efficient Transfer Learning for NLP," in *ICML*, 2019.

- [145] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [146] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *RecSys*, 2019.

## 5.4 Thực hành

Chương này là chương dễ thực hành nhất của Phần C, vì hạ tầng mã nguồn mở đã chín và một chiếc điện thoại là đủ để tạo dữ liệu. Một lưu ý về bán rã trước khi vào danh sách: danh sách dưới đây chốt tại **tháng 9/2026**, và các tên mô hình dẫn đầu bảng sẽ cũ trong 6–12 tháng. Các repo hạ tầng thì bền hơn nhiều: COLMAP, Open3D, evo, gsplat. Hãy đầu tư kỹ năng vào nhóm sau.

**Repo và tài nguyên.** SfM tham chiếu: [colmap/colmap](#) (giấy phép BSD; lưu ý bản 4.x đổi backend đọc ảnh nên không tái lập bit-for-bit với 3.x) [94]. *Feed-forward geometry*: [facebookresearch/vggt](#) (CVPR 2025 best paper; nhiều pipeline không cần COLMAP dựng trên nó), DUST3R/MASt3R [96]. *Splatting và rendering*: [nerfstudio-project/gsplat](#) (Apache 2.0, mặc định hợp lý cho nghiên cứu), [graphdeco-inria/gaussian-splatting](#) (bản gốc, giấy phép hạn chế nhất — đọc kỹ trước khi dùng cho sản phẩm), [nerfstudio-project/nerfstudio](#), NVIDIA vkSplatting. *Độ sâu đơn ảnh*: Depth Anything V2, Apple MoGe. *SLAM*: ORB-SLAM3 [80], DROID-SLAM; đánh giá bằng [MichaelGrupp/evo](#). *Point cloud*: [isl-org/Open3D](#), [open-mmlab/mmdetection3d](#).

**Câu hỏi kiểm tra hiểu.** Vì sao ATE cần bước căn chỉnh còn RPE thì không, và mỗi cái nhảy với loại lỗi nào? 3DGS nhanh hơn NeRF nhờ đổi cái gì lấy cái gì? Feed-forward geometry tốt hơn COLMAP ở tình huống nào và tệ hơn ở tình huống nào — và bạn nhận ra mình đang ở tình huống nào bằng dấu hiệu gì *trước khi* chạy?

**Mini project (vài giờ đến hai ngày).** Quay một video 60 giây quanh một vật thể. Chạy hai đường: (a) COLMAP → gsplat, (b) một mô hình tiến thẳng → gsplat. So ba đại lượng: thời gian tường, PSNR trên các view giữ lại, và độ lệch pose giữa hai đường. Giá trị của bài này không nằm ở con số mà ở chỗ bạn buộc phải quyết định *đường nào là tham chiếu* — và phát hiện ra rằng câu hỏi đó không có câu trả lời hiển nhiên.

### Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

| Thuật ngữ / câu hỏi                           | Trả lời                                                                                                                                                                                                                  |
|-----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ••• SfM                                       | Quy trình cổ điển suy đồng thời pose camera và cấu trúc 3D thừa từ một tập ảnh: tìm tương ứng, ước lượng pose, tam giác hoá, tinh chỉnh. Nó là thể hệ một và vẫn là chuẩn đối chiếu của mọi phương pháp mới.             |
| ••• Hiệu chỉnh chùm tia                       | Bài toán bình phương tối thiểu phi tuyến, tinh chỉnh đồng thời mọi pose và mọi điểm 3D sao cho sai số chiếu lại nhỏ nhất. Điều đáng nhớ nhất: nó trả về phần dư và hiệp phương sai, nên hệ thống tự biết lúc nào nó sai. |
| •• Point map                                  | Ảnh mà mỗi pixel chứa một toạ độ 3D thay vì một màu. Đây là dạng đầu ra của các mô hình tiến thẳng như DUST3R hay VGGT, thay cho đám mây điểm rời rạc.                                                                   |
| ••• Vì sao mô hình tiến thẳng hồng “im lặng”? | Vì nó là một bài hồi quy, không phải một bài ước lượng có ràng buộc. Không còn phần dư hình học nào để mâu thuẫn với kết quả, nên cảnh trong hay ngoài phân phối nó đều trả lời với cùng vẻ tự tin.                      |

| Thuật ngữ / câu hỏi                                                                | Trả lời (tiếp)                                                                                                                                                                                                                                   |
|------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ●● Thị sai                                                                         | Độ lệch ngang của cùng một điểm giữa ảnh trái và ảnh phải trong hệ stereo. Nó tỉ lệ nghịch với độ sâu: $Z = fb/d$ .                                                                                                                              |
| ●● Vì sao sai số độ sâu stereo tăng theo $Z^2$ ?                                   | Vì $Z = fb/d$ nên $\Delta Z = Z^2 \Delta d / (fb)$ . Hệ quả thực hành: mọi hệ stereo có một “chân trời”, và cách duy nhất đẩy nó ra xa là tăng tiêu cự hoặc đường cơ sở.                                                                         |
| ●●● Scale ambiguity                                                                | Phóng to cả cảnh lẫn quãng đường camera cùng một hệ số cho ra đúng bộ ảnh cũ. Vì vậy tỉ lệ tuyệt đối không suy ra được từ ảnh, dù thêm bao nhiêu dữ liệu; chỉ prior hoặc một cảm biến biết mét mới phá được.                                     |
| ●● Voxel                                                                           | Ô lập phương trong một lưới đều 3D. Ưu điểm là lân cận cho sẵn nên conv 3D dùng được ngay; nhược điểm là số ô tăng theo lũy thừa ba, và hơn 99% ô là rỗng.                                                                                       |
| ●● SDF                                                                             | Hàm gán cho mỗi điểm khoảng cách có dấu tới bề mặt gần nhất — âm ở trong, dương ở ngoài. Bề mặt là tập điểm bằng 0, nên trích lưới tam giác là chuyện cơ học; đổi lại render phải dò dọc tia.                                                    |
| ●● Point cloud                                                                     | Tập tọa độ 3D không có thứ tự và không có kết nối. Gọn và hợp với LiDAR, nhưng không có sẵn khái niệm lân cận, nên mọi phép cục bộ đều phải trả tiền tìm kiếm.                                                                                   |
| ●● PointNet                                                                        | Mạng cho tập điểm: mọi điểm qua cùng một MLP rồi gộp bằng phép max theo từng chiều. Phép max không quan tâm thứ tự nên tính bất biến hoán vị là đúng tuyệt đối, không phải học được.                                                             |
| ● Bản đồ dựng ra trông đẹp nhưng mọi khoảng cách sai cùng một hệ số — nguyên nhân? | Scale ambiguity chưa được phá: pipeline đơn mắt không có cảm biến nào biết đơn vị mét. Phép phân biệt với lỗi hiệu chỉnh: đo vài vật ở các tầm khác nhau — sai một hệ số <i>không đổi</i> là lỗi tỉ lệ, sai tăng dần theo tầm là lỗi hiệu chỉnh. |
| ● Mô hình LiDAR 64 tia tụt mạnh trên 32 tia, mất vật ở xa trước — nguyên nhân?     | Mật độ điểm lúc chạy khác lúc huấn luyện, và vùng xa là nơi mật độ giảm nhanh nhất. Phân biệt với lỗi hiệu chỉnh: lỗi hiệu chỉnh làm lệch vị trí hộp ở mọi tầm, không làm mất hẳn vật ở xa.                                                      |
| ●●● Volume rendering                                                               | Cách tính màu một pixel bằng cách lấy mẫu nhiều điểm dọc tia rồi cộng dồn màu có trọng số theo độ đục tích lũy. Nó thay câu hỏi “bề mặt ở đâu” bằng câu hỏi “mật độ tại điểm này bao nhiêu”.                                                     |
| ●●● Vì sao NeRF dùng mật độ liên tục, không dùng bề mặt?                           | Vì đạo hàm. Bề mặt cứng làm hàm mất mát thành bậc thang, gradient bằng không gần như mọi nơi. Mật độ liên tục biến mỗi pixel thành tổng có trọng số, nên mọi điểm mẫu đều nhận được gradient.                                                    |
| ●● Positional encoding                                                             | Ánh xạ tọa độ qua một dãy sin/cos nhiều tần số trước khi đưa vào MLP. Không có nó thì ảnh mờ nhòe, vì MLP thiên lệch mạnh về hàm tần số thấp.                                                                                                    |
| ●●● 3D Gaussian splatting                                                          | Biểu diễn cảnh bằng hàng triệu Gaussian dị hướng có màu và độ đục, render bằng cách chiếu rồi trộn theo thứ tự độ sâu. Được tốc độ thời gian thực; mất dung lượng (236 byte mỗi Gaussian) và mất bề mặt tường minh.                              |
| ●● Vì sao early fusion nhạy với lệch hiệu chỉnh hơn late fusion?                   | Vì nó liên kết ở mức pixel. Lệch $1^\circ$ ở 20 m là 35 cm, đủ để điểm LiDAR rơi lên nhầm vật và biến đặc trưng ghép thành rác. Late fusion liên kết ở mức đối tượng, nơi 35 cm vẫn nằm trong kích thước vật.                                    |
| ● PSNR trên view giữ lại rất cao nhưng bản đồ hình học sai — nghĩa là gì?          | Mô hình đã “vẽ” thay vì “dựng”: nó khớp ảnh bằng màu phụ thuộc hướng nhìn. Phép phân biệt: ngoại suy ra ngoài quỹ đạo chụp, hoặc so với một quét chân trị.                                                                                       |
| ● Hệ hợp nhất chạy hoàn hảo khi robot đứng yên, hỏng khi xoay nhanh — nguyên nhân? | Lệch đồng bộ thời gian, vì sai số của nó tỉ lệ với tốc độ quay: 5 ms ở $100^\circ/\text{s}$ là $0,5^\circ$ . Phân biệt với lệch hiệu chỉnh ngoại: lệch hiệu chỉnh sai cả khi đứng yên.                                                           |



**Kiến thức này đang sống ở đâu** (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

| Công trình / hệ thống   | Dùng ý tưởng nào của chương                                                                                                            | Ghi chú                                                                                        |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| COLMAP                  | Hiện thực tham chiếu của thể hệ một: tìm tương ứng, ước lượng pose, hiệu chỉnh chùm tia. Vẫn là chuẩn để đối chiếu mọi phương pháp mới | Hạ tầng bền; phần lớn dữ liệu huấn luyện của thể hệ bốn cũng do nó tạo ra                      |
| ORB-SLAM3               | Hiệu chỉnh chùm tia dưới ràng buộc thời gian thực, với tách frontend/backend và đóng vòng để chặn sai số                               | Chuẩn đối chứng của SLAM đặc trưng thưa                                                        |
| DUST3R / MAST3R / VGGT  | Thể hệ bốn: point map trong một hệ tọa độ chung, không cần pose đầu vào                                                                | Mạnh ở khởi tạo và ảnh thưa; vẫn thường được nối tiếp bằng một vòng hiệu chỉnh chùm tia        |
| gsplat, nerfstudio      | Neural rendering thực dụng: tối ưu biểu diễn cảnh bằng loss render khả vi, tiêu thụ pose từ SfM                                        | Hạ tầng; phần lớn pipeline dựng cảnh hiện nay đi qua đây                                       |
| Depth Anything V2, MoGe | Độ sâu đơn ảnh bất biến affine — bỏ tỉ lệ để đổi lấy khả năng khái quát                                                                | Dùng làm prior tương đối thì tốt; dùng làm phép đo metric thì phải kèm hai ẩn tỉ lệ và độ dịch |
| Open3D, mmdetection3d   | Học sâu trên point cloud: bất biến hoán vị, phân cấp không gian, tích chập thưa                                                        | Hạ tầng cho mảng LiDAR                                                                         |

**MINI PROJECT SLAM** · Hai đường dựng cảnh 3D và điểm giao của chúng · 1–2 ngày

|                 |                                                                                                                                                                                                                                                                                                                   |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Mục tiêu</b> | tìm bằng số cái ngưỡng mà ở dưới nó thể hệ tiến thẳng thắng hình học cổ điển, và ở trên nó thì thua. Đo trên chính camera và chính loại cảnh bạn đang chạy SLAM, không phải trên benchmark của paper.                                                                                                             |
| <b>Dữ liệu</b>  | một video 60 giây quay quanh một vật thể hoặc một góc phòng, quay bằng đúng camera bạn dùng cho SLAM (giữ nguyên phơi sáng thủ công nếu có). Trích ra bốn tập ảnh với $N = 8, 15, 30, 60$ khung hình phân bố đều, và giữ riêng 10 % khung hình làm view kiểm.                                                     |
| <b>Các bước</b> | với mỗi $N$ : (1) chạy đường A — COLMAP → gsplat — và đường B — một mô hình 3D tiến thẳng (DUST3R/MAST3R/VGGT) → gsplat; (2) ghi thời gian tường của từng khâu; (3) ghi PSNR trên view kiểm; (4) ghi độ lệch pose giữa hai đường sau khi căn chỉnh Sim3; (5) ghi lại $N$ nhỏ nhất mà mỗi đường còn khởi tạo được. |
| <b>Xong khi</b> | bạn vẽ được hai đường PSNR theo $N$ trên cùng một trục và chỉ ra điểm giao bằng con số, kèm bảng thời gian tường và $N$ tối thiểu của từng đường. Câu trả lời phải có dạng “dưới $N = \dots$ đường tiến thẳng thắng, trên đó COLMAP thắng, và nó tốn thêm ... phút”.                                              |
| <b>Nối với</b>  | <a href="#">C/16/01</a> giải thích vì sao điểm giao đó <i>phải</i> tồn tại; <a href="#">B/07</a> cho cách tách view kiểm cho đúng; <a href="#">E/24</a> cho việc lặp lại nhiều lần trước khi tin một chênh lệch. Bảng số bạn tạo ra ở đây là baseline cho mini project của chương sau.                            |

**TRẢ LỜI CÂU HỎI MỞ ĐẦU**

- A1. Vì phóng to cả cảnh lẫn quãng đường camera cùng một hệ số cho ra chuỗi ảnh y hệt — đây là tính không xác định được, dữ liệu không mang thông tin đó. Chỉ prior kích thước, hoặc một cảm biến biết đơn vị mét (đường cơ sở stereo, LiDAR, gia tốc kế), mới phá được.
- A2. Vì hai bên giải hai bài toán khác nhau. Mô hình tiến thẳng mạnh ở *khởi tạo*: ít ảnh, chồng lấn kém, không pose. Hiệu chỉnh chùm tia mạnh ở *tinh chỉnh*. Quan trọng hơn, chỉ nó trả về phần dư và hiệp phương sai, nên chỉ nó cho ta biết lúc nào kết quả không đáng tin.
- A3. Vì đạo hàm. Bề mặt cứng làm hàm mất mát thành bậc thang, gradient bằng không gần như mọi nơi nên không có hướng để đi. Mật độ liên tục biến mỗi pixel thành tổng có trọng số của hàng chục điểm dọc tia, và mọi điểm đều nhận được một phần gradient.



- A4. Vì lệch đồng bộ thời gian chỉ hiện ra khi có chuyển động và tỉ lệ thuận với tốc độ quay: 5 ms ở  $100^\circ/\text{s}$  là 0,5°, ở tầm 20 m thành khoảng 17 cm. Early fusion liên kết ở mức pixel nên độ lệch đó biến điểm LiDAR thành rơi lên nhầm vật, tức là đầu vào rác.

### NÓI LẠI THẬT ĐƠN GIẢN

Hãy nghĩ tới cái bóng của một vật in trên tường. Nhìn cái bóng, bạn không có cách nào biết vật đó to bằng nào và cách tường bao xa: một cái cốc nhỏ để gần và một cái thùng lớn để xa đổ ra đúng cùng một cái bóng. Đó chính xác là chuyện xảy ra khi máy ảnh chụp một cảnh — ảnh là cái bóng, và chiều sâu bị mất khi chụp.

Cả chương này là các cách lấy lại chiều đã mất, và chỉ có ba cách. Cách thứ nhất là đi vòng sang chỗ khác chụp thêm vài cái bóng nữa rồi ghép chúng lại; chỗ nào các cái bóng chỉ về cùng một điểm thì đó là vật thật. Cách này chính xác tới từng centimét, và quan trọng hơn, nếu các cái bóng mâu thuẫn nhau thì bạn biết ngay là mình đã sai ở đâu đó. Cách thứ hai là nặn một mô hình bằng đất sét trong bóng tối, rồi soi đèn từ đúng những hướng đã chụp và chỉnh đất sét cho tới khi mọi cái bóng khớp. Cách thứ ba là đưa ba tấm ảnh cho một người đã nhìn hàng triệu cảnh và hỏi luôn.

Cái giá thì rất rõ. Cách thứ nhất cần bạn đi thêm một vòng và cần cảnh đứng yên. Cách thứ hai cho mô hình đổ bóng đúng ở mọi hướng đã thử. Nhưng nó chưa chắc đúng khi soi từ một hướng mới, vì bạn chỉ ép nó khớp *cái bóng*, không ép nó khớp *hình dạng*. Cách thứ ba trả lời trong một giây và không cần đi đâu cả, nhưng khi gặp một cảnh lạ nó vẫn trả lời tự tin y hệt, và không có cái bóng nào mâu thuẫn để tố cáo nó.

**Chỗ không rút gọn được:** chỗ nói về sai lệch khi lắp camera và cảm biến với nhau. Cách nói đơn giản tự nhiên là “phải lắp cho thật chuẩn”, nhưng nói vậy là sai trọng tâm: vấn đề không phải sai số lớn hay nhỏ, mà là nó *lắp lại y hệt* ở mọi phép đo, nên gom thêm bao nhiêu phép đo cũng không triệt tiêu được nó. Đây là chỗ phải nói đủ dài mới không sai.

**Một câu:** *Ảnh làm mất chiều sâu, và bạn chỉ có hai cách lấy lại: đi thêm một vòng để nhìn từ chỗ khác, hoặc đoán bằng kinh nghiệm — chỉ cách thứ nhất mới tự nói cho bạn biết lúc nào nó sai.*

### Kết nối

Chương này khép lại phần hình học của Phần C. [C/17-mo-hinh-sinh](#) lấy lại nguyên vẹn ý tưởng “đặt cái ta muốn vào hàm mất mát rồi tối ưu trên chính đối tượng cần tạo ra”, lần này trên pixel; [B/07-chia-du-lieu-va-danh-gia](#) hoàn thiện phần giao thức đánh giá mà chương này chỉ nhắc qua; [E/25-do-tin-cay-va-van-hanh-bien](#) “cơ chế phát hiện sai từ bên ngoài” thành thiết kế cụ thể.

### Tổng kết chương

Ta đã đi từ một bài toán duy nhất — lấy lại chiều đã mất khi chiếu — qua bốn cách giải nó, và thấy chúng không xếp thành một chuỗi tiến bộ mà thành một trục đánh đổi. Ở một đầu trục là hình học cổ điển: giả định nhiều, dữ liệu ít, độ chính xác metric cao, và quan trọng nhất là tự biết lúc nào mình sai. Ở đầu kia là mô hình tiến thẳng: giả định ít, dữ liệu nhiều, chạy được ở chỗ hình học bó tay, và im lặng khi sai. Neural rendering không nằm trên trục đó mà nằm ở một tầng khác — nó không cạnh tranh với SfM mà tiêu thụ đầu ra của SfM, và nó tối ưu cho một mục tiêu khác hẳn là chất lượng ảnh.

Ba chủ đề còn bỏ ngỏ, và cả ba đều là chỗ nên theo dõi trong một hai năm tới: thứ nhất, các mô hình tiến thẳng dự đoán thẳng biểu diễn render được, tức là gộp thể hệ ba và bốn làm một; thứ hai, neural rendering cho cảnh động, nơi giả định cảnh tĩnh bị gỡ bỏ; thứ ba, và thực dụng nhất với robot, cách gắn một ước lượng bất định tin cậy được vào đầu ra của các mô hình học. Chừng nào chưa có nó, thể hệ bốn vẫn chỉ dùng được ở vai trò khởi tạo, không phải vai trò quyết định.

### ❏ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Luận điểm gộp là mỗi lần bỏ một giả định, bạn bù bằng dữ liệu *và* mất một phần khả năng tự chẩn đoán. Theo dấu khoản mất thứ hai qua bốn thể hệ: ở mỗi thể hệ, tín hiệu tự chẩn đoán nào biến mất?
2. Tính không xác định của tỉ lệ ở mục 1 và mục 2, so với việc mô hình ở mục 3 vẫn dựng được cảnh trông rất đúng — hai chỗ này có mâu thuẫn không? Chỉ ra thông tin nào đã lén vào thay cho ràng buộc còn thiếu.
3. Bạn có ba nguồn độ sâu cho cùng một pixel: stereo, độ sâu đơn ảnh học được, và LiDAR thừa. Ba luật sai số ở đây khác hẳn nhau; luật nào khiến việc gán trọng số theo nghịch đảo phương sai trở nên không hợp lệ, và bạn xử lý thế nào?
4. Hình học cổ điển hồng ồn ào còn mô hình học được hồng im lặng. Với một hệ dùng cả hai, bạn dựng cơ chế nào để buộc phần im lặng phát ra tiếng, và cơ chế đó tiêu tốn cái gì?
5. Loop closure ở mục 2 và tinh chỉnh toàn cục sau khâu pose ở mục 1 đều là ràng buộc toàn cục. Vì sao neural rendering ở mục 3 hầu như không có thứ tương đương, và hệ quả với một bản đồ dựng suốt nhiều giờ là gì?
6. Bạn nghi hệ hợp nhất hồng vì hiệu chỉnh ngoại tại hoặc lệch đồng bộ thời gian chứ không vì thuật toán. Phép thử nào tách hai nguyên nhân đó, và vì sao chuyển động xoay là điều kiện thử tốt hơn chuyển động thẳng?

- [1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [2] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [3] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *AISTATS*, 2010.
- [4] K. He, X. Zhang, S. Ren, et al., “Delving Deep into Rectifiers,” in *ICCV*, 2015.
- [5] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [6] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [7] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [8] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [9] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [10] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [11] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” in *ICLR*, 2017.
- [12] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [13] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [14] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [15] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [16] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [17] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [18] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [19] G. Huang, Y. Sun, Z. Liu, et al., “Deep Networks with Stochastic Depth,” in *ECCV*, 2016.
- [20] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [21] H. Zhang, M. Cisse, Y. N. Dauphin, et al., “mixup: Beyond Empirical Risk Minimization,” in *ICLR*, 2018.
- [22] S. Yun, D. Han, S. J. Oh, et al., “CutMix: Regularization Strategy to Train Strong Classifiers,” in *ICCV*, 2019.
- [23] E. D. Cubuk, B. Zoph, J. Shlens, et al., “RandAugment: Practical Automated Data Augmentation,” in *CVPR Workshops*, 2020.

- [24] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [25] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [26] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *NeurIPS*, 2012.
- [27] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in *ICLR*, 2015.
- [28] C. Szegedy, W. Liu, et al., “Going Deeper with Convolutions,” in *CVPR*, 2015.
- [29] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [30] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [31] G. Huang, Z. Liu, L. van der Maaten, et al., “Densely Connected Convolutional Networks,” in *CVPR*, 2017.
- [32] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [33] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [34] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [35] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [36] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [37] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.
- [38] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [39] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [40] J. Su, M. Ahmed, Y. Lu, et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, vol. 568, 2024.
- [41] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” *arXiv:2312.00752*, 2023.
- [42] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [43] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.
- [44] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [45] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [46] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [47] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [48] R. Girshick, J. Donahue, T. Darrell, et al., “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation,” in *CVPR*, 2014.

- [49] R. Girshick, "Fast R-CNN," in *ICCV*, 2015.
- [50] S. Ren, K. He, R. Girshick, et al., "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in *NeurIPS*, 2015.
- [51] T.-Y. Lin, P. Dollár, R. Girshick, et al., "Feature Pyramid Networks for Object Detection," in *CVPR*, 2017.
- [52] K. He, G. Gkioxari, P. Dollár, et al., "Mask R-CNN," in *ICCV*, 2017.
- [53] D. Bolya, S. Foley, J. Hays, et al., "TIDE: A General Toolbox for Identifying Object Detection Errors," in *ECCV*, 2020.
- [54] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [55] J. Redmon, S. Divvala, R. Girshick, et al., "You Only Look Once: Unified, Real-Time Object Detection," in *CVPR*, 2016.
- [56] W. Liu, D. Anguelov, D. Erhan, et al., "SSD: Single Shot MultiBox Detector," in *ECCV*, 2016.
- [57] T.-Y. Lin, P. Goyal, R. Girshick, et al., "Focal Loss for Dense Object Detection," in *ICCV*, 2017.
- [58] Z. Tian, C. Shen, H. Chen, et al., "FCOS: Fully Convolutional One-Stage Object Detection," in *ICCV*, 2019.
- [59] X. Zhou, D. Wang, and P. Krähenbühl, "Objects as Points," *arXiv:1904.07850*, 2019.
- [60] S. Zhang, C. Chi, Y. Yao, et al., "Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection," in *CVPR*, 2020.
- [61] Z. Ge, S. Liu, Z. Li, et al., "OTA: Optimal Transport Assignment for Object Detection," in *CVPR*, 2021.
- [62] N. Bodla, B. Singh, R. Chellappa, et al., "Soft-NMS: Improving Object Detection with One Line of Code," in *ICCV*, 2017.
- [63] X. Zhu, W. Su, L. Lu, et al., "Deformable DETR: Deformable Transformers for End-to-End Object Detection," in *ICLR*, 2021.
- [64] H. Zhang, F. Li, S. Liu, et al., "DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection," in *ICLR*, 2023.
- [65] J. Long, E. Shelhamer, and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," in *CVPR*, 2015.
- [66] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *MICCAI*, 2015.
- [67] L.-C. Chen, G. Papandreou, I. Kokkinos, et al., "DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs," *IEEE TPAMI*, vol. 40, no. 4, pp. 834–848, 2018.
- [68] E. Xie, W. Wang, Z. Yu, et al., "SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers," in *NeurIPS*, 2021.
- [69] B. Cheng, I. Misra, A. G. Schwing, et al., "Masked-Attention Mask Transformer for Universal Image Segmentation," in *CVPR*, 2022.
- [70] A. Kirillov, K. He, R. Girshick, et al., "Panoptic Segmentation," in *CVPR*, 2019.
- [71] A. Kirillov, E. Mintun, N. Ravi, et al., "Segment Anything," in *ICCV*, 2023.
- [72] K. Sun, B. Xiao, D. Liu, et al., "Deep High-Resolution Representation Learning for Human Pose Estimation," in *CVPR*, 2019.

- [73] Z. Cao, T. Simon, S.-E. Wei, et al., “Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields,” in *CVPR*, 2017.
- [74] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [75] J. Deng, J. Guo, N. Xue, et al., “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *CVPR*, 2019.
- [76] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [77] K. Xu, J. Ba, R. Kiros, et al., “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention,” in *ICML*, 2015.
- [78] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *ICLR*, 2015.
- [79] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [80] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [81] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [82] K. Simonyan and A. Zisserman, “Two-Stream Convolutional Networks for Action Recognition in Videos,” in *NeurIPS*, 2014.
- [83] J. Carreira and A. Zisserman, “Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset,” in *CVPR*, 2017.
- [84] D. Tran, H. Wang, L. Torresani, et al., “A Closer Look at Spatiotemporal Convolutions for Action Recognition,” in *CVPR*, 2018.
- [85] C. Feichtenhofer, H. Fan, J. Malik, et al., “SlowFast Networks for Video Recognition,” in *ICCV*, 2019.
- [86] A. Dosovitskiy, P. Fischer, E. Ilg, et al., “FlowNet: Learning Optical Flow with Convolutional Networks,” in *ICCV*, 2015.
- [87] D. Sun, X. Yang, M.-Y. Liu, et al., “PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume,” in *CVPR*, 2018.
- [88] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.
- [89] A. Bewley, Z. Ge, L. Ott, et al., “Simple Online and Realtime Tracking,” in *ICIP*, 2016.
- [90] Y. Zhang, P. Sun, Y. Jiang, et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *ECCV*, 2022.
- [91] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [92] K. Bernardin and R. Stiefelhagen, “Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics,” *EURASIP J. Image and Video Processing*, 2008.
- [93] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [94] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [95] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.



- [96] S. Wang, V. Leroy, Y. Cabon, et al., "DUST3R: Geometric 3D Vision Made Easy," in *CVPR*, 2024.
- [97] D. Eigen, C. Puhrsch, and R. Fergus, "Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network," in *NeurIPS*, 2014.
- [98] R. Ranftl, K. Lasinger, D. Hafner, et al., "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer," *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [99] J. Sturm, N. Engelhard, F. Endres, et al., "A Benchmark for the Evaluation of RGB-D SLAM Systems," in *IROS*, 2012.
- [100] C. R. Qi, H. Su, K. Mo, et al., "PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation," in *CVPR*, 2017.
- [101] C. R. Qi, L. Yi, H. Su, et al., "PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space," in *NeurIPS*, 2017.
- [102] A. H. Lang, S. Vora, H. Caesar, et al., "PointPillars: Fast Encoders for Object Detection from Point Clouds," in *CVPR*, 2019.
- [103] T. Müller, A. Evans, C. Schied, et al., "Instant Neural Graphics Primitives with a Multiresolution Hash Encoding," *ACM TOG*, vol. 41, no. 4, 2022.
- [104] B. Kerbl, G. Kopanas, T. Leimkühler, et al., "3D Gaussian Splatting for Real-Time Radiance Field Rendering," *ACM TOG*, vol. 42, no. 4, 2023.
- [105] R. Zhang, P. Isola, A. A. Efros, et al., "The Unreasonable Effectiveness of Deep Features as a Perceptual Metric," in *CVPR*, 2018.
- [106] D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," in *ICLR*, 2014.
- [107] I. Goodfellow, J. Pouget-Abadie, M. Mirza, et al., "Generative Adversarial Nets," in *NeurIPS*, 2014.
- [108] T. Karras, S. Laine, and T. Aila, "A Style-Based Generator Architecture for Generative Adversarial Networks," in *CVPR*, 2019.
- [109] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," in *NeurIPS*, 2020.
- [110] Y. Song, J. Sohl-Dickstein, D. P. Kingma, et al., "Score-Based Generative Modeling Through Stochastic Differential Equations," in *ICLR*, 2021.
- [111] J. Johnson, A. Alahi, and L. Fei-Fei, "Perceptual Losses for Real-Time Style Transfer and Super-Resolution," in *ECCV*, 2016.
- [112] P. Isola, J.-Y. Zhu, T. Zhou, et al., "Image-to-Image Translation with Conditional Adversarial Networks," in *CVPR*, 2017.
- [113] L. A. Gatys, A. S. Ecker, and M. Bethge, "Image Style Transfer Using Convolutional Neural Networks," in *CVPR*, 2016.
- [114] X. Huang and S. Belongie, "Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization," in *ICCV*, 2017.
- [115] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," in *NeurIPS Deep Learning Workshop*, 2015.
- [116] J. Song, C. Meng, and S. Ermon, "Denoising Diffusion Implicit Models," in *ICLR*, 2021.
- [117] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, et al., "Flow Matching for Generative Modeling," in *ICLR*, 2023.
- [118] J. Ho and T. Salimans, "Classifier-Free Diffusion Guidance," *arXiv:2207.12598*, 2022.
- [119] R. Rombach, A. Blattmann, D. Lorenz, et al., "High-Resolution Image Synthesis with Latent Diffusion Models," in *CVPR*, 2022.
- [120] W. Peebles and S. Xie, "Scalable Diffusion Models with Transformers," in *ICCV*, 2023.

- [121] L. Zhang, A. Rao, and M. Agrawala, "Adding Conditional Control to Text-to-Image Diffusion Models," in *ICCV*, 2023.
- [122] Y. Blau and T. Michaeli, "The Perception-Distortion Tradeoff," in *CVPR*, 2018.
- [123] C. Ledig, L. Theis, F. Huszár, et al., "Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network," in *CVPR*, 2017.
- [124] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., "Image Quality Assessment: From Error Visibility to Structural Similarity," *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [125] M. Heusel, H. Ramsauer, T. Unterthiner, et al., "GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium," in *NeurIPS*, 2017.
- [126] T. Brooks, B. Mildenhall, T. Xue, et al., "Unprocessing Images for Learned Raw Denoising," in *CVPR*, 2019.
- [127] A. Abdelhamed, S. Lin, and M. S. Brown, "A High-Quality Denoising Dataset for Smartphone Cameras," in *CVPR*, 2018.
- [128] J. Deng, W. Dong, R. Socher, et al., "ImageNet: A Large-Scale Hierarchical Image Database," in *CVPR*, 2009.
- [129] A. Radford, J. W. Kim, C. Hallacy, et al., "Learning Transferable Visual Models from Natural Language Supervision," in *ICML*, 2021.
- [130] J. Li, D. Li, S. Savarese, et al., "BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models," in *ICML*, 2023.
- [131] H. Liu, C. Li, Q. Wu, et al., "Visual Instruction Tuning," in *NeurIPS*, 2023.
- [132] S. Kornblith, J. Shlens, and Q. V. Le, "Do Better ImageNet Models Transfer Better?" In *CVPR*, 2019.
- [133] T. Chen, S. Kornblith, M. Norouzi, et al., "A Simple Framework for Contrastive Learning of Visual Representations," in *ICML*, 2020.
- [134] K. He, H. Fan, Y. Wu, et al., "Momentum Contrast for Unsupervised Visual Representation Learning," in *CVPR*, 2020.
- [135] J.-B. Grill, F. Strub, F. Altché, et al., "Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning," in *NeurIPS*, 2020.
- [136] M. Caron, H. Touvron, I. Misra, et al., "Emerging Properties in Self-Supervised Vision Transformers," in *ICCV*, 2021.
- [137] K. He, X. Chen, S. Xie, et al., "Masked Autoencoders Are Scalable Vision Learners," in *CVPR*, 2022.
- [138] A. van den Oord, Y. Li, and O. Vinyals, "Representation Learning with Contrastive Predictive Coding," in *arXiv:1807.03748*, 2018.
- [139] J. Kaplan, S. McCandlish, T. Henighan, et al., "Scaling Laws for Neural Language Models," *arXiv:2001.08361*, 2020.
- [140] X. Zhai, A. Kolesnikov, N. Houlsby, et al., "Scaling Vision Transformers," in *CVPR*, 2022.
- [141] J. Hoffmann, S. Borgeaud, A. Mensch, et al., "Training Compute-Optimal Large Language Models," in *NeurIPS*, 2022.
- [142] C. G. Northcutt, A. Athalye, and J. Mueller, "Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks," in *NeurIPS Datasets and Benchmarks*, 2021.
- [143] B. Recht, R. Roelofs, L. Schmidt, et al., "Do ImageNet Classifiers Generalize to ImageNet?" In *ICML*, 2019.
- [144] N. Houlsby, A. Giurgiu, S. Jastrzebski, et al., "Parameter-Efficient Transfer Learning for NLP," in *ICML*, 2019.

- [145] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [146] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *RecSys*, 2019.

## Mô hình sinh, style transfer và low-level vision (Generative Models and Low-Level Vision)

**PHỤ THUỘC** B/09-thiet-ke-ham-mat-mat (hướng đo khoảng cách phân phối — điều kiện tiên quyết thật sự của chương này); C/12-co-che-huan-luyen (huấn luyện ổn định, quy mô); C/13-kien-truc-backbone (U-Net, transformer); A/02-vat-ly-tao-anh (nhiều thật, dài sáng động).

### TÓM TẮT

Chương này gom bốn chủ đề thường bị dạy rời nhau — mô hình sinh, chuyển phong cách, diffusion, và khôi phục ảnh — vì chúng chia chung đúng một câu hỏi: khi nhiều đầu ra cùng hợp lệ, cái gì quyết định chọn cái nào? Bốn họ mô hình sinh là bốn câu trả lời, chuyển phong cách là câu trả lời viết thẳng lên pixel, diffusion là câu trả lời hiện đang thắng, và low-level vision là nơi câu hỏi trở thành vấn đề đúng sai chứ không còn là thẩm mỹ. Đọc xong bạn nhìn một mô hình sinh là đọc ra kiểu thất bại của nó trước khi chạy. Nếu chỉ nhớ một điều: chất lượng đầu ra khác nhau là vì *định nghĩa “giống dữ liệu thật”* khác nhau, không phải vì mô hình này thông minh hơn mô hình kia — và mọi cơ chế làm ảnh “đẹp hơn” đều mua độ trung thành bằng độ đa dạng.

### KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình sinh (generative model)** — mô hình học *phân phối* của dữ liệu, để có thể lấy mẫu ra dữ liệu mới; khác với mô hình phân biệt, vốn chỉ học ánh xạ từ đầu vào sang nhãn.
- **phân phối dữ liệu (data distribution)** — cách nghĩ về tập ảnh thật như các mẫu rút ra từ một phân phối xác suất trên không gian ảnh; “sinh ảnh mới” nghĩa là rút thêm một mẫu từ phân phối đó.
- **hàm mất mát (loss function)** — đại lượng số mà quá trình huấn luyện cực tiểu hoá. Toàn chương này là một minh hoạ kéo dài cho nguyên tắc: chọn hàm mất mát là chọn trước kiểu sai lầm mà mô hình sẽ mắc.
- **không gian ẩn (latent space)** — không gian chiều thấp mà mô hình dùng để mã hoá một ảnh; nó vừa là nơi “ý nghĩa” được sắp xếp, vừa là mẹo giúp diffusion chạy được trên GPU thường.
- **style transfer (chuyển phong cách)** — bài toán giữ nội dung của một ảnh nhưng khoác lên nó “chất” của một ảnh khác — nét cọ, bảng màu, kết cấu.
- **ill-posed (đặt không chính)** — bài toán có nhiều nghiệm cùng hợp lệ vì thông tin đã bị xoá; mọi bài toán khôi phục ảnh đều thuộc loại này.
- **FID / LPIPS / PSNR** — ba chỉ số đánh giá ở ba tầng khác nhau: FID so hai *phân phối* ảnh, LPIPS so hai ảnh trong không gian đặc trưng học được, PSNR so hai ảnh theo từng pixel. Mục 17.4 nói rõ mỗi cái mù với gì.

### CÂU HỎI MỞ ĐẦU

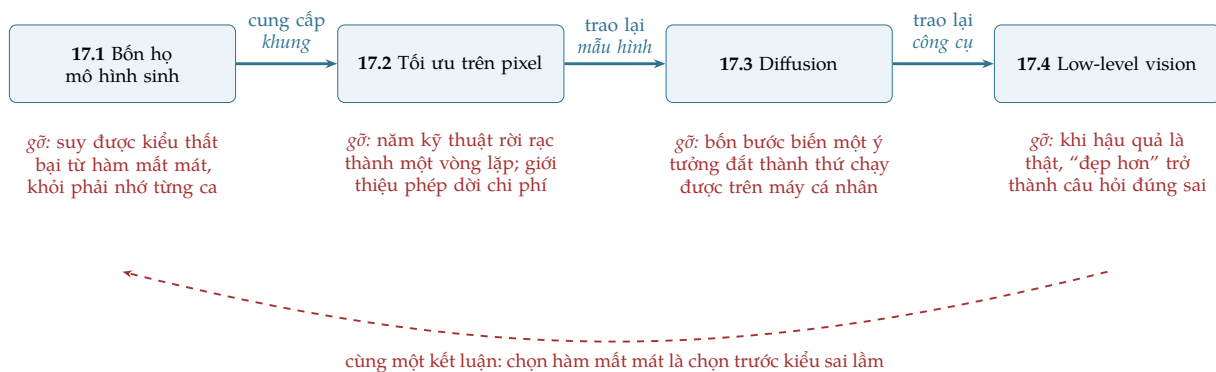
- Q1. Vì sao mô hình tối ưu MSE *luôn* cho ảnh mờ, và vì sao đó là một kết quả toán học chứ không phải lỗi cài đặt?
- Q2. VAE hỏng vì mờ, GAN hỏng vì nghèo — hai triệu chứng trái ngược nhau. Vì sao chúng lại là hai mặt của cùng một quyết định thiết kế?

- Q3.** Vì sao diffusion, thứ phải chạy mạng hàng chục tới hàng trăm lần cho một ảnh, lại đánh bại GAN vốn chỉ cần một lượt xuôi?
- Q4.** Vì sao chỉnh một pipeline sinh ảnh tới lúc trông “đẹp nhất” lại là cách chắc chắn nhất để làm hỏng nó khi dùng đầu ra làm dữ liệu huấn luyện?

### Vì sao chương này chia làm bốn mục

Bốn mục là bốn tầng của cùng một câu hỏi, và thứ tự là thứ tự phụ thuộc.

Hình 6.1 vẽ bốn mục cùng nút thắt mà mỗi mục gỡ.



**Hình 6.1.** Bốn mục của chương, xếp theo thứ tự phụ thuộc chứ không theo mức độ phổ biến. Mục 17.2 nằm giữa lý thuyết và diffusion không phải ngẫu nhiên: nó là ví dụ sạch nhất cho việc hàm mục tiêu quyết định tất cả, và nó giới thiệu mẫu hình “dời chi phí sang lúc huấn luyện” mà mục 17.3 dùng lại nguyên vẹn. Mũi tên đứt là kết luận chung mà cả bốn mục đều dẫn tới.

Mục thứ nhất dựng khung: bốn họ mô hình sinh là bốn định nghĩa khác nhau của cụm từ “giống dữ liệu thật”. Khung này quan trọng hơn bất kỳ chi tiết kiến trúc nào, vì nó cho phép suy ra kiểu thất bại từ hàm mất mát — ảnh mờ, mất mode, hay chậm — thay vì phải ghi nhớ từng trường hợp.

Mục thứ hai tách riêng một ý tưởng cắt ngang: đóng băng mạng, coi ảnh là biến, hạ gradient trên chính pixel. Nó được đặt ở đây, giữa lý thuyết và diffusion, vì hai lý do: nó là ví dụ sạch nhất về việc hàm mục tiêu quyết định tất cả (đổi một dòng là được một kỹ thuật khác), và nó giới thiệu mẫu hình “dời chi phí từ lúc chạy sang lúc huấn luyện” mà mục ba sẽ dùng lại nguyên vẹn.

Mục thứ ba là họ đang thắng. Nó không chỉ trình bày nguyên lý mà đi qua bốn bước kỹ thuật đã biến diffusion từ một ý tưởng thành sản phẩm chạy được trên máy cá nhân — và mỗi bước là một đánh đổi cụ thể mà người dùng phải chỉnh bằng tay.

Mục thứ tư đưa cùng bộ khái niệm sang một bối cảnh mà hậu quả là thật: khôi phục ảnh. Ở đây “sắc nét hơn” đồng nghĩa với “bị nhiễu hơn”, và điều đó không phải một lời than phiền mà là một kết quả có chứng minh. Mục này đặt cuối vì nó cần cả ba mục trước để nói cho đủ.

### Bản đồ chương 17 — Mô hình sinh, style transfer và low-level vision

**TÓM TẮT**

Chương 17 gồm bốn mục, xếp theo thứ tự phụ thuộc chứ không theo mức độ phổ biến: khung bốn họ mô hình sinh, ý tưởng tối ưu trên pixel, diffusion, rồi khôi phục ảnh. Bản đồ này nói mục nào cốt lõi, mục nào tra khi cần, và trình tự đọc tùy mục tiêu của bạn. Nếu chỉ nhớ một điều: đọc mục 17.1 trước — nó cho phép *suy ra* kiểu thất bại của một mô hình từ hàm mất mát, và ba mục còn lại đều dùng lại khả năng đó.

**KIẾN THỨC CẦN CÓ TRƯỚC**

- **mô hình sinh (generative model)** — mô hình học phân phối dữ liệu để lấy mẫu ra dữ liệu mới, khác với mô hình dự đoán nhãn.
- **diffusion** — họ mô hình sinh bằng cách học khử nhiễu dần, từ nhiễu trắng về ảnh; hiện là họ thống trị.
- **style transfer** — giữ nội dung của ảnh này, khoác “chất” của ảnh kia.
- **low-level vision** — nhóm bài toán ảnh sang ảnh: khử nhiễu, siêu phân giải, khử nhoè, HDR.

**Bốn mục và vai trò của từng mục**

- **17.1 — Bốn họ và cuộc đánh đổi ba chiều (cốt lõi).** Autoregressive, VAE, GAN, diffusion là bốn định nghĩa của “giống dữ liệu thật”. Đây là khung để suy ra kiểu thất bại, và là mục đắt giá nhất của cả chương.
- **17.2 — Tối ưu trên pixel thay vì trên trọng số (cốt lõi về ý tưởng, tham khảo về chi tiết style transfer).** Một vòng lặp mười dòng cho ra năm kỹ thuật; ma trận Gram; mẫu hình dờ chi phí sang lúc huấn luyện.
- **17.3 — Diffusion và các bước làm cho nó khả thi (cốt lõi).** Quá trình thuận/ngược, score function, bộ lấy mẫu, dẫn hướng, khuếch tán trong không gian ẩn, ControlNet.
- **17.4 — Low-level vision và trade-off không thể tránh (cốt lõi với người làm hệ thống thật).** Ill-posed, đánh đổi perceptual-distortion, các chỉ số và chỗ mù của từng cái, dữ liệu sinh dùng để huấn luyện.

**Trình tự đọc**

1. **Mục tiêu là hiểu nguyên lý:** 17.1 → 17.3 → 17.4, bỏ qua 17.2 ở lần đọc đầu.
2. **Mục tiêu là dùng được công cụ sinh ảnh:** 17.1 (chỉ phần bảng và tam giác) → 17.3 toàn bộ → 17.4 phần đánh giá.
3. **Mục tiêu là làm hệ thống thật có khôi phục ảnh:** 17.4 trước, rồi quay lên 17.1 để hiểu vì sao “mờ” là nghiệm đúng.
4. Khối *Thực hành* nằm ở file chương, sau bốn mục.

**Chương này phụ thuộc gì và được dùng lại ở đâu**

- **Phụ thuộc lên trên:** B/09-thiet-ke-ham-mat-mat (hai hướng của KL — điều kiện tiên quyết thật sự); C/12-co-che-huan-luyen; C/13-kien-truc-backbone (U-Net, transformer); A/02-vat-ly-tao-anh (nhiều thật); C/16-thi-giac-3d (vai trò của prior trong bài toán thiếu ràng buộc).
- **Được dùng lại ở dưới:** D/20-toi-uu-va-nen-mo-hinh (dời chi phí sang lúc huấn luyện, chung cất); C/18-pretraining-va-foundation (nguồn giám sát và dữ liệu sinh);



E/25-do-tin-cay-va-van-hanh (hồng im lặng, trực quan hoá và các phép kiểm tra tính đúng đắn).

## 6.1 Bốn họ và cuộc đánh đổi ba chiều (Four Families of Generative Models)

**PHỤ THUỘC** B/09-thiet-ke-ham-mat-mat (hướng đo khoảng cách phân phối, forward đối lại reverse KL); A/01-toan-toi-thieu/03-xac-suat-va-thong-tin (likelihood, entropy tương đối); C/13-kien-truc-backbone (U-Net, transformer).

### TÓM TẮT

Bốn họ mô hình sinh — *autoregressive*, VAE, GAN, *diffusion* — là bốn định nghĩa khác nhau của cụm từ “giống dữ liệu thật”. Chất lượng đầu ra khác nhau chính vì định nghĩa khác nhau, không phải vì mô hình này thông minh hơn mô hình kia. Đọc xong bạn nhìn một mô hình sinh là đọc ra ngay nó đang tối ưu đại lượng gì. Từ đó suy được nó sẽ mờ, sẽ thiếu đa dạng, hay sẽ chậm. Và bạn biết mình đang mua cái gì bằng cái gì. Nếu chỉ nhớ một điều: ảnh mờ của VAE và *mode collapse* của GAN không phải hai lỗi kỹ thuật khác nhau mà là hai mặt của cùng một lựa chọn — đo khoảng cách phân phối theo hướng nào.

### KIẾN THỨC CẦN CÓ TRƯỚC

- **likelihood (độ hợp lý)** — xác suất mà một mô hình gán cho dữ liệu đã quan sát; “cực đại hoá likelihood” là chỉnh tham số sao cho mô hình thấy dữ liệu thật là chuyện bình thường chứ không phải chuyện lạ.
- **KL divergence (phân kỳ Kullback–Leibler)** — một cách đo khoảng cách giữa hai phân phối; nó *không đối xứng*, và chính sự không đối xứng đó sinh ra khác biệt giữa “mờ” và “mất mode”.
- **mode (mode của phân phối)** — một vùng tập trung nhiều xác suất — ví dụ “ảnh mèo” và “ảnh chó” là hai mode. Phủ hết mode nghĩa là sinh ra được cả hai loại.
- **mode collapse** — hiện tượng mô hình chỉ sinh ra một phần nhỏ của phân phối (chỉ vài kiểu ảnh) trong khi vẫn đạt điểm cao theo tiêu chí huấn luyện.
- **biến ẩn (latent variable)** — vectơ chiều thấp mà mô hình dùng làm “bản tóm tắt” của một mẫu; sinh ảnh là lấy ngẫu nhiên một vectơ như vậy rồi giải mã.
- **discriminator (bộ phân biệt)** — mạng thứ hai trong GAN, có việc duy nhất là phân biệt mẫu thật với mẫu do mô hình sinh; nó đóng vai một hàm mất mát *học được*.
- **score matching** — cách huấn luyện bằng việc khớp gradient của log mật độ thay vì khớp mật độ; đây là nền toán của diffusion.

### A. Đặt vấn đề

Một mô hình sinh phải làm được một việc nghe rất mơ hồ: tạo ra mẫu mới “giống” dữ liệu huấn luyện nhưng không phải bản sao của nó. Muốn tối ưu bất cứ thứ gì thì phải viết “giống” thành một con số, và đây là chỗ bốn họ mô hình rẽ nhánh — chúng viết ra bốn con số khác nhau. Autoregressive đo bằng **độ hợp lý** (*likelihood*) chính xác của từng pixel theo thứ tự; VAE đo bằng một cận dưới của likelihood; GAN đo bằng “một bộ phân biệt học được không tách nổi mẫu giả khỏi mẫu thật”; diffusion đo bằng mức khớp của *score* ở mọi mức nhiễu. Bốn định nghĩa, bốn hình dạng lỗi.

Điều làm chủ đề này đáng học kỹ là các chế độ lỗi có thể *suy ra* chứ không phải ghi nhớ. Khi biết một mô hình tối ưu trung bình bình phương theo pixel, bạn dự đoán được nó cho ảnh mờ trước khi chạy nó; khi biết một mô hình tối ưu một hàm mục tiêu dạng trò chơi hai người, bạn dự đoán được nó không hội tụ theo nghĩa thông thường.

**Học mục này thì làm được gì mà trước đó không làm được?** Đọc một bài báo mô hình sinh và trả lời trong ba mươi giây: mô hình này sẽ mờ, thiếu đa dạng, hay chậm — và vì sao bắt buộc phải như vậy. Sau đó chọn được họ mô hình cho một bài toán cụ thể thay vì chọn theo cái đang nổi.

#### TRỰC GIÁC

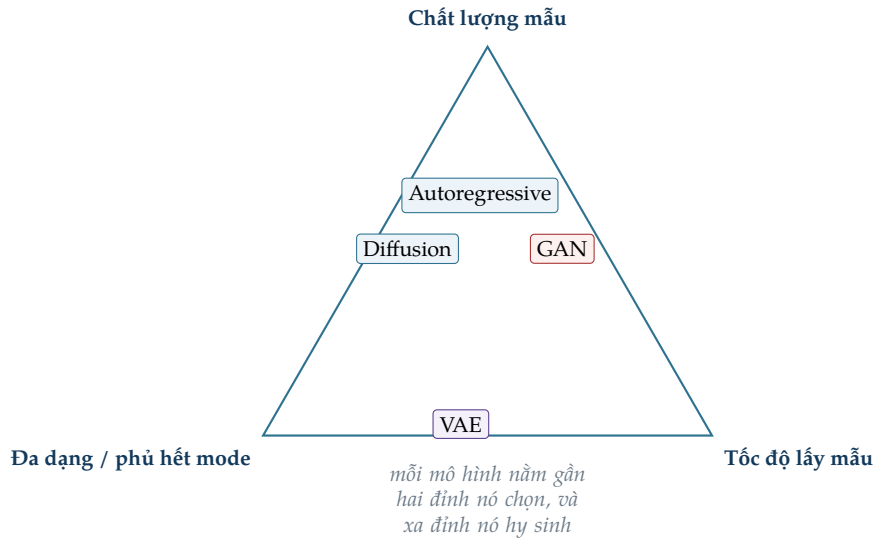
Bốn họ giống bốn cách chấm điểm một người làm tranh giả. *Autoregressive* bắt vẽ từng nét theo đúng xác suất của nét tiếp theo: chấm được điểm rất chính xác, nhưng vẽ chậm khủng khiếp vì mỗi nét phải chờ nét trước. *VAE* bắt nén bức tranh thành vài chục con số rồi giải nén: nhanh, nhưng phần giải nén trả về *trung bình* của mọi bức tranh khớp với mấy con số đó, nên nét bị nhoè. *GAN* thuê một giám định viên và chỉ đòi lừa được ông ta: nét rất sắc vì giám định viên soi nét — nhưng nếu ông ta chỉ quen soi hoa văn ở góc trái thì kẻ làm giả sẽ chỉ học vẽ giỏi đúng cái góc đó. *Diffusion* bắt đầu từ một tờ giấy đầy nhiễu và tẩy dần từng nấc, mỗi nấc chỉ phải đoán lùi một chút. Cách này chậm vì nhiều nấc. Bù lại mỗi nấc là một bài dễ, nên nó không bao giờ “vẽ hỏng”.

#### B. Bốn họ đặt cạnh nhau

**Bảng 6.1.** Bốn họ mô hình sinh, đọc theo định nghĩa của “giống”. Cột cuối suy ra được từ cột thứ hai — đó là điểm của cả bảng.

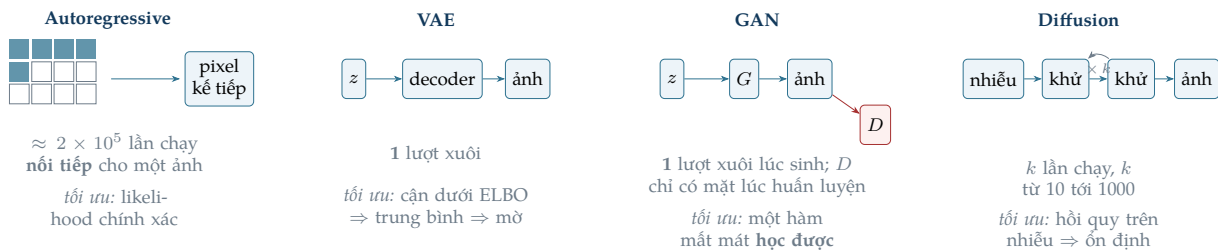
| Họ                              | Định nghĩa “giống”                               | Được                                                      | Mất / hỏng khi                                                            |
|---------------------------------|--------------------------------------------------|-----------------------------------------------------------|---------------------------------------------------------------------------|
| Auto-regressive (PixelRN-N/CNN) | Likelihood chính xác, phân rã theo quy tắc chuỗi | Tính được likelihood; huấn luyện ổn định                  | Sinh cực chậm vì tuần tự theo pixel; lệch giữa lúc huấn luyện và lúc sinh |
| VAE                             | Cận dưới của likelihood (ELBO)                   | Nhanh; có không gian ẩn có nghĩa, nội suy được            | Ảnh mờ — hệ quả trực tiếp của loss trung bình; <i>posterior collapse</i>  |
| GAN                             | “Một discriminator học được không phân biệt nổi” | Ảnh sắc nét ở cùng ngân sách tính toán                    | Huấn luyện bất ổn; mode collapse; không có likelihood để chẩn đoán        |
| Diffusion                       | Khớp score của phân phối ở mọi mức nhiễu         | Chất lượng và đa dạng cao; huấn luyện ổn định như hồi quy | Sinh chậm vì nhiều bước; nhạy với lựa chọn bộ lấy mẫu                     |

Ba đại lượng trong cột ba và bốn không độc lập nhau. Trong tài liệu, người ta hay gọi tình thế này là *thế lưỡng nan ba chiều* của mô hình sinh: chất lượng mẫu, độ phủ hết các mode, và tốc độ lấy mẫu — ba thứ mà cho tới nay chưa họ nào đạt đồng thời cả ba ở mức tốt nhất. Cần nói rõ để trung thực: đây là một *cách gọi tên* một quan sát thực nghiệm lặp lại, không phải một định lý; nó khác hẳn về địa vị với đánh đổi perceptual-distortion ở mục 17.4, vốn là một kết quả có chứng minh. Hình 6.2 đặt bốn họ lên tam giác đó.



**Hình 6.2.** Thế lưỡng nan ba chiều. GAN mua chất lượng và tốc độ bằng độ phủ mode; VAE mua tốc độ và độ phủ bằng chất lượng; diffusion và autoregressive mua chất lượng cùng độ phủ bằng tốc độ. Toàn bộ tiến bộ kỹ thuật của mảng này — *sampler distillation, flow matching, latent diffusion* — là các nỗ lực kéo một điểm về gần tâm hơn.

Hình 6.3 bổ sung cho tam giác một thứ mà tam giác không nói được: *cơ chế* sinh ra mỗi vị trí trên đó.



**Hình 6.3.** Bốn cơ chế lấy mẫu, đặt cạnh nhau để thấy chi phí nằm ở đâu. Hai họ ở giữa sinh ảnh trong một lượt xuôi duy nhất; hai họ ở hai đầu phải chạy mạng nhiều lần, và đó chính là đỉnh “tốc độ lấy mẫu” mà chúng hy sinh trong Hình 6.2. Điểm đáng chú ý nhất là bộ phân biệt  $D$  của GAN: nó *không* tham gia lúc sinh, nó chỉ tồn tại lúc huấn luyện — vai trò của nó là một hàm mất mát học được, không phải một khối tính toán.

### C. Vì sao họ sau ra đời

#### I MẠCH 1 — bốn định nghĩa của “giống dữ liệu thật”

- 1 Vấn đề** muốn mô hình hoá một phân phối trên không gian ảnh, mà không gian đó quá lớn để đếm.
- Giải pháp** phân rã theo quy tắc chuỗi:  $p(x) = \prod_i p(x_i \mid x_{<i})$ , rồi học từng thừa số bằng một mạng.
- Giả định** có một thứ tự tự nhiên trên pixel (thường là quét từ trái sang phải, trên xuống dưới — một giả định đã sai ngay từ đầu với ảnh).
- Hệ quả** likelihood tính được chính xác, nên có thể so sánh mô hình bằng một con số có ý nghĩa.
- Giá phải trả** lấy mẫu là tuần tự: một ảnh  $256 \times 256$  màu cần 196 608 lần chạy mạng nối tiếp nhau; ở 1 ms mỗi lần thì hơn ba phút cho *một* ảnh.

- 2 Vấn đề** sinh tuần tự quá chậm để dùng được.
- Giải pháp** đưa vào một biến ẩn  $z$  chiều thấp, học đồng thời bộ mã hoá và bộ giải mã, tối ưu cận dưới ELBO [106]; lấy mẫu chỉ còn một lượt xuôi.
- Hệ quả** thêm một phần thưởng ngoài dự tính: không gian ẩn có cấu trúc, nội suy giữa hai điểm cho ra biến đổi mượt.
- Giá phải trả** bộ giải mã Gauss làm cho phần tái tạo của ELBO trở thành sai số bình phương trung bình theo pixel. Mà nghiệm tối ưu của bình phương trung bình là *kỳ vọng có điều kiện*, tức trung bình của mọi ảnh hợp lệ. Trung bình của nhiều ảnh sắc nét lệch nhau vài pixel là một ảnh mờ.
- **hệ quả của C/17/04** — cùng phép tính này quay lại ở khôi phục ảnh, nơi “mờ” trở thành nghiệm tối ưu PSNR và là một nửa của đánh đổi perceptual-distortion.
- Vấn đề mới** ảnh mờ không sửa được bằng cách huấn luyện lâu hơn; nó là nghiệm đúng của bài toán đã đặt ra.
- 3 Vấn đề** muốn ảnh sắc thì phải bỏ hàm mất mát trung bình, nhưng ta không biết viết công thức nào cho “trông thật”.
- Giải pháp** đừng viết — hãy *học* nó: cho một mạng thứ hai làm bộ phân biệt và huấn luyện bộ sinh để đánh lừa nó [107].
- Hệ quả** bộ phân biệt phạt bất cứ dấu hiệu nào lộ ra chất giả, kể cả dấu hiệu ta không nghĩ ra được, nên ảnh sắc nét [108].
- **cùng nguyên lý với C/17/02** — “không viết được hàm mất mát thì học nó” cũng chính là ý tưởng của mất mát tri giác, chỉ khác ở chỗ mạng chấm điểm được đóng băng thay vì huấn luyện đối kháng.
- Giá phải trả** mục tiêu không còn là cực tiểu hoá một hàm cố định mà là điểm cân bằng của một trò chơi hai người; “hội tụ” theo nghĩa thông thường không tồn tại, và mô hình có thể đạt mục tiêu bằng cách chỉ sinh ra một phần nhỏ của phân phối — *mode collapse*.
- Vấn đề mới** không có likelihood nên mất luôn công cụ chẩn đoán tự nhiên.
- 4 Vấn đề** muốn sắc nét như GAN nhưng ổn định như một bài hồi quy.
- Giải pháp** chẻ bài toán khó thành một chuỗi bài toán dễ: thêm nhiều dần cho tới khi ảnh thành nhiễu trắng, rồi học khử nhiễu ngược lại từng nấc [109], [110].
- Giả định** chuỗi đủ mịn để mỗi nấc chỉ cần một bước nhỏ.
- Hệ quả** hàm mất mát trở thành bình phương trung bình trên *nhiều* chứ không trên ảnh. Huấn luyện vì thế ổn định như mọi bài hồi quy. Đó là lý do diffusion đánh bại GAN: không phải bằng ý tưởng đẹp hơn, mà bằng việc dễ huấn luyện hơn ở quy mô lớn.
- Giá phải trả** sinh chậm vì phải chạy hàng chục tới hàng trăm nấc. Mục 17.3 dành trọn cho việc gỡ đúng cái giá này.

#### D. Ví dụ nhỏ nhất: vì sao mờ và mode collapse là hai mặt của một đồng xu

Lấy phân phối thật là hỗn hợp hai Gauss cách nhau:  $p = \frac{1}{2}\mathcal{N}(-1, 0,25) + \frac{1}{2}\mathcal{N}(+1, 0,25)$ . Bây giờ ép mô hình phải là *một* Gauss duy nhất  $q = \mathcal{N}(\mu, \sigma^2)$ . Mô hình quá nghèo để khớp — đúng tình huống thực tế, chỉ là ở chiều thấp.

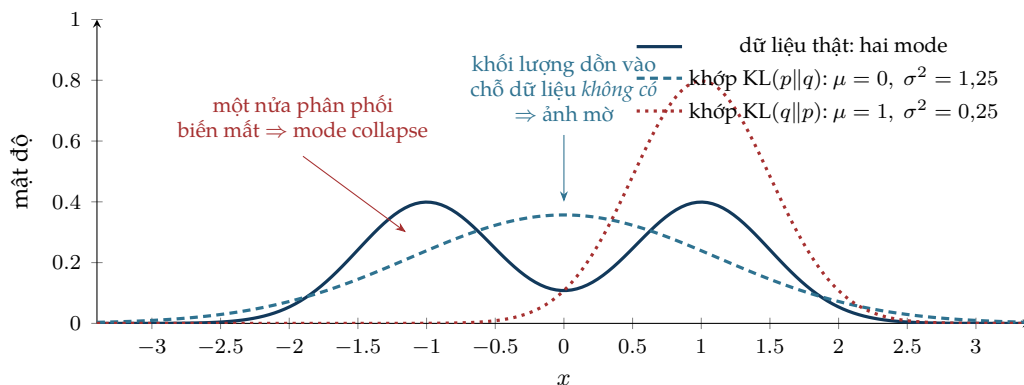
- **Nếu tối ưu  $KL(p \parallel q)$**  (hướng “thuận”, hướng mà cực đại hoá likelihood dùng): nghiệm là Gauss khớp mô-men của  $p$ , tức  $\mu = 0$  và  $\sigma^2 = 0,25 + 1 = 1,25$ . Mô hình *trải rộng phủ cả hai mode*, và đặt nhiều khối lượng nhất ở  $x = 0$  — đúng chỗ dữ liệu thật gần như không có gì. Dịch sang ảnh: đây chính là ảnh mờ.

- **Nếu tối ưu  $KL(q \parallel p)$**  (hướng “ngược”): phạt rất nặng việc đặt khối lượng ở nơi  $p$  nhỏ, nên nghiệm co về đúng một mode,  $\mu \approx \pm 1, \sigma^2 \approx 0,25$ . Mẫu sinh ra đều hợp lệ và sắc nét — nhưng một nửa phân phối biến mất. Dịch sang ảnh: đây chính là mode collapse.

Hai dòng trên là toàn bộ lời giải thích. VAE mờ và GAN thiếu đa dạng không phải hai lỗi cần sửa bằng hai mẹo khác nhau; chúng là hai nghiệm của cùng một bài toán, khác nhau ở hướng đo khoảng cách. Chi tiết về hai hướng KL nằm ở B/09-thiet-ke-ham-mat-mat; ở đây chỉ cần nhớ rằng chọn hàm mất mát là chọn kiểu thất bại.

↪ **tiền đề** B/09 — hai hướng của KL là điều kiện tiên quyết thật sự của cả chương: không nắm nó thì “mờ” và “mất mode” trông như hai lỗi rời rạc.

Hình 6.4 vẽ đúng ba phân phối của ví dụ trên.



**Hình 6.4.** Toàn bộ chương gói trong một hình. Cùng một phân phối thật hai mode, cùng một họ mô hình nghèo (một Gauss duy nhất), chỉ đổi *hướng* đo khoảng cách thì ra hai nghiệm trái ngược. Hướng thuận phạt việc bỏ sót mode nên nghiệm trải rộng và đặt nhiều khối lượng nhất ở  $x = 0$ , đúng chỗ dữ liệu thật gần như trống — dịch sang ảnh, đó là ảnh mờ của VAE. Hướng ngược phạt việc đặt khối lượng ở nơi mật độ thật thấp nên nghiệm co về đúng một mode — dịch sang ảnh, đó là mode collapse của GAN. Số trong hình tính tay được từ ví dụ ở mục D.

## E. Giới hạn và trường hợp hỏng

Bảng và tam giác ở trên đơn giản hoá một bức tranh thật ra phức tạp hơn, và có ba chỗ cần nói thẳng:

- **Ranh giới giữa các họ đang mờ đi.** Diffusion trong không gian ẩn dùng một autoencoder kiểu VAE; *flow matching* là diffusion viết lại dưới dạng phương trình vi phân thường; mô hình sinh ảnh bằng token tự hồi quy đang quay lại mạnh mẽ. Bốn họ là bốn *nguyên lý*, không phải bốn hộp kín.
- **Likelihood cao không đồng nghĩa mẫu đẹp.** Một mô hình có thể đạt likelihood tốt nhờ mô hình hoá tốt các chi tiết tần số cao vô nghĩa mà vẫn sinh ra ảnh xấu; ngược lại GAN không có likelihood nhưng mẫu đẹp. Hai đại lượng đo hai thứ khác nhau.
- **Đánh giá là chỗ yếu nhất của cả mảng.** Nhìn vài chục mẫu được chọn lọc không nói được gì về độ phủ mode, mà độ phủ mới là thứ hay hỏng. Mục 17.4 nói kỹ về chỗ này.

### CẠM BÃY

Đừng đánh giá một mô hình sinh bằng những mẫu đẹp nhất của nó. Chế độ hỏng đắt nhất — mode collapse và thu hẹp phân phối — *không nhìn thấy được* trong mẫu chọn lọc, vì các mẫu còn lại vẫn đẹp, chỉ là chúng giống nhau. Phép thử tối thiểu: sinh vài nghìn mẫu với cùng điều kiện rồi đo độ đa dạng (khoảng cách trung bình theo cặp trong không gian đặc

trưng, số cụm sau khi gom nhóm), và đối chiếu với cùng số lượng mẫu thật. Nếu bạn định dùng mẫu sinh làm dữ liệu huấn luyện thì phép thử này là bắt buộc chứ không phải tùy chọn — xem [C/17-mo-hinh-sinh/04-low-level-vision](#).

#### CẦN NHỚ

Mỗi họ mô hình sinh là một định nghĩa của “giống dữ liệu thật”, và *kiểu thất bại của mô hình được quyết định bởi định nghĩa đó, không phải bởi kiến trúc*. Trung bình theo pixel cho ảnh mờ; hướng ngược của KL cho mode collapse; chuỗi bài toán dễ cho huấn luyện ổn định nhưng lấy mẫu chậm. Biết mô hình tối ưu đại lượng gì là biết trước nó sẽ hỏng thế nào. [T2]

#### F. Kết nối

Mục này là khung cho cả chương 17: [C/17-mo-hinh-sinh/02-toi-uu-tren-pixel](#) tách riêng một ý tưởng cắt ngang cả bốn họ (đóng băng mạng, tối ưu trên chính ảnh); [C/17-mo-hinh-sinh/03-diffusion](#) đào sâu họ thứ tư và các bước biến nó thành sản phẩm; [C/17-mo-hinh-sinh/04-low-level-vision](#) cho thấy cùng bộ khái niệm này quay lại ở bài toán khôi phục ảnh. Hướng KL và hệ quả “mờ đổi lại thu hẹp” thuộc B/09-thiet-ke-ham-mat-mat. Ý tưởng “không viết được hàm mất mát thì học nó” cũng chính là ý tưởng của **mất mát tri giác** (*perceptual loss*) [111]. Nó cũng là ý tưởng của bộ phân biệt trong *pix2pix* [112], một cầu nối trực tiếp sang bài toán ảnh sang ảnh.

#### Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao ảnh mờ của VAE là *nghiệm đúng* của bài toán đã đặt ra chứ không phải dấu hiệu huấn luyện chưa đủ?
2. GAN không có likelihood. Điều đó lấy đi của bạn khả năng chẩn đoán cụ thể nào khi mô hình hoạt động kém?
3. Vì sao diffusion huấn luyện ổn định hơn GAN, và lý do đó có liên quan gì tới việc mục tiêu của nó là một bài hồi quy?
4. Bạn có một mô hình sinh và một trăm mẫu trông rất đẹp; phép thử ba mươi phút nào cho biết nó có bị thu hẹp phân phối hay không?
5. Nếu ràng buộc cứng của bạn là sinh 30 ảnh mỗi giây trên một GPU nhúng, tam giác ở Hình 6.2 chỉ về đâu, và bạn chấp nhận mất gì?



- [1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [2] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [3] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *AISTATS*, 2010.
- [4] K. He, X. Zhang, S. Ren, et al., “Delving Deep into Rectifiers,” in *ICCV*, 2015.
- [5] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [6] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [7] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [8] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [9] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [10] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [11] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” in *ICLR*, 2017.
- [12] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [13] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [14] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [15] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [16] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [17] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [18] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [19] G. Huang, Y. Sun, Z. Liu, et al., “Deep Networks with Stochastic Depth,” in *ECCV*, 2016.
- [20] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [21] H. Zhang, M. Cisse, Y. N. Dauphin, et al., “mixup: Beyond Empirical Risk Minimization,” in *ICLR*, 2018.
- [22] S. Yun, D. Han, S. J. Oh, et al., “CutMix: Regularization Strategy to Train Strong Classifiers,” in *ICCV*, 2019.
- [23] E. D. Cubuk, B. Zoph, J. Shlens, et al., “RandAugment: Practical Automated Data Augmentation,” in *CVPR Workshops*, 2020.

- [24] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [25] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [26] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *NeurIPS*, 2012.
- [27] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in *ICLR*, 2015.
- [28] C. Szegedy, W. Liu, et al., “Going Deeper with Convolutions,” in *CVPR*, 2015.
- [29] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [30] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [31] G. Huang, Z. Liu, L. van der Maaten, et al., “Densely Connected Convolutional Networks,” in *CVPR*, 2017.
- [32] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [33] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [34] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [35] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [36] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [37] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.
- [38] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [39] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [40] J. Su, M. Ahmed, Y. Lu, et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, vol. 568, 2024.
- [41] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” *arXiv:2312.00752*, 2023.
- [42] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [43] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.
- [44] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [45] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [46] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [47] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [48] R. Girshick, J. Donahue, T. Darrell, et al., “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation,” in *CVPR*, 2014.

- [49] R. Girshick, "Fast R-CNN," in *ICCV*, 2015.
- [50] S. Ren, K. He, R. Girshick, et al., "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in *NeurIPS*, 2015.
- [51] T.-Y. Lin, P. Dollár, R. Girshick, et al., "Feature Pyramid Networks for Object Detection," in *CVPR*, 2017.
- [52] K. He, G. Gkioxari, P. Dollár, et al., "Mask R-CNN," in *ICCV*, 2017.
- [53] D. Bolya, S. Foley, J. Hays, et al., "TIDE: A General Toolbox for Identifying Object Detection Errors," in *ECCV*, 2020.
- [54] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [55] J. Redmon, S. Divvala, R. Girshick, et al., "You Only Look Once: Unified, Real-Time Object Detection," in *CVPR*, 2016.
- [56] W. Liu, D. Anguelov, D. Erhan, et al., "SSD: Single Shot MultiBox Detector," in *ECCV*, 2016.
- [57] T.-Y. Lin, P. Goyal, R. Girshick, et al., "Focal Loss for Dense Object Detection," in *ICCV*, 2017.
- [58] Z. Tian, C. Shen, H. Chen, et al., "FCOS: Fully Convolutional One-Stage Object Detection," in *ICCV*, 2019.
- [59] X. Zhou, D. Wang, and P. Krähenbühl, "Objects as Points," *arXiv:1904.07850*, 2019.
- [60] S. Zhang, C. Chi, Y. Yao, et al., "Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection," in *CVPR*, 2020.
- [61] Z. Ge, S. Liu, Z. Li, et al., "OTA: Optimal Transport Assignment for Object Detection," in *CVPR*, 2021.
- [62] N. Bodla, B. Singh, R. Chellappa, et al., "Soft-NMS: Improving Object Detection with One Line of Code," in *ICCV*, 2017.
- [63] X. Zhu, W. Su, L. Lu, et al., "Deformable DETR: Deformable Transformers for End-to-End Object Detection," in *ICLR*, 2021.
- [64] H. Zhang, F. Li, S. Liu, et al., "DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection," in *ICLR*, 2023.
- [65] J. Long, E. Shelhamer, and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," in *CVPR*, 2015.
- [66] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *MICCAI*, 2015.
- [67] L.-C. Chen, G. Papandreou, I. Kokkinos, et al., "DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs," *IEEE TPAMI*, vol. 40, no. 4, pp. 834–848, 2018.
- [68] E. Xie, W. Wang, Z. Yu, et al., "SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers," in *NeurIPS*, 2021.
- [69] B. Cheng, I. Misra, A. G. Schwing, et al., "Masked-Attention Mask Transformer for Universal Image Segmentation," in *CVPR*, 2022.
- [70] A. Kirillov, K. He, R. Girshick, et al., "Panoptic Segmentation," in *CVPR*, 2019.
- [71] A. Kirillov, E. Mintun, N. Ravi, et al., "Segment Anything," in *ICCV*, 2023.
- [72] K. Sun, B. Xiao, D. Liu, et al., "Deep High-Resolution Representation Learning for Human Pose Estimation," in *CVPR*, 2019.

- [73] Z. Cao, T. Simon, S.-E. Wei, et al., “Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields,” in *CVPR*, 2017.
- [74] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [75] J. Deng, J. Guo, N. Xue, et al., “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *CVPR*, 2019.
- [76] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [77] K. Xu, J. Ba, R. Kiros, et al., “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention,” in *ICML*, 2015.
- [78] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *ICLR*, 2015.
- [79] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [80] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [81] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [82] K. Simonyan and A. Zisserman, “Two-Stream Convolutional Networks for Action Recognition in Videos,” in *NeurIPS*, 2014.
- [83] J. Carreira and A. Zisserman, “Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset,” in *CVPR*, 2017.
- [84] D. Tran, H. Wang, L. Torresani, et al., “A Closer Look at Spatiotemporal Convolutions for Action Recognition,” in *CVPR*, 2018.
- [85] C. Feichtenhofer, H. Fan, J. Malik, et al., “SlowFast Networks for Video Recognition,” in *ICCV*, 2019.
- [86] A. Dosovitskiy, P. Fischer, E. Ilg, et al., “FlowNet: Learning Optical Flow with Convolutional Networks,” in *ICCV*, 2015.
- [87] D. Sun, X. Yang, M.-Y. Liu, et al., “PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume,” in *CVPR*, 2018.
- [88] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.
- [89] A. Bewley, Z. Ge, L. Ott, et al., “Simple Online and Realtime Tracking,” in *ICIP*, 2016.
- [90] Y. Zhang, P. Sun, Y. Jiang, et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *ECCV*, 2022.
- [91] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [92] K. Bernardin and R. Stiefelhagen, “Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics,” *EURASIP J. Image and Video Processing*, 2008.
- [93] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [94] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [95] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.

- [96] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *CVPR*, 2024.
- [97] D. Eigen, C. Puhrsch, and R. Fergus, “Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network,” in *NeurIPS*, 2014.
- [98] R. Ranftl, K. Lasinger, D. Hafner, et al., “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer,” *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [99] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IROS*, 2012.
- [100] C. R. Qi, H. Su, K. Mo, et al., “PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation,” in *CVPR*, 2017.
- [101] C. R. Qi, L. Yi, H. Su, et al., “PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space,” in *NeurIPS*, 2017.
- [102] A. H. Lang, S. Vora, H. Caesar, et al., “PointPillars: Fast Encoders for Object Detection from Point Clouds,” in *CVPR*, 2019.
- [103] T. Müller, A. Evans, C. Schied, et al., “Instant Neural Graphics Primitives with a Multiresolution Hash Encoding,” *ACM TOG*, vol. 41, no. 4, 2022.
- [104] B. Kerbl, G. Kopanas, T. Leimkühler, et al., “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *ACM TOG*, vol. 42, no. 4, 2023.
- [105] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.
- [106] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” in *ICLR*, 2014.
- [107] I. Goodfellow, J. Pouget-Abadie, M. Mirza, et al., “Generative Adversarial Nets,” in *NeurIPS*, 2014.
- [108] T. Karras, S. Laine, and T. Aila, “A Style-Based Generator Architecture for Generative Adversarial Networks,” in *CVPR*, 2019.
- [109] J. Ho, A. Jain, and P. Abbeel, “Denoising Diffusion Probabilistic Models,” in *NeurIPS*, 2020.
- [110] Y. Song, J. Sohl-Dickstein, D. P. Kingma, et al., “Score-Based Generative Modeling Through Stochastic Differential Equations,” in *ICLR*, 2021.
- [111] J. Johnson, A. Alahi, and L. Fei-Fei, “Perceptual Losses for Real-Time Style Transfer and Super-Resolution,” in *ECCV*, 2016.
- [112] P. Isola, J.-Y. Zhu, T. Zhou, et al., “Image-to-Image Translation with Conditional Adversarial Networks,” in *CVPR*, 2017.
- [113] L. A. Gatys, A. S. Ecker, and M. Bethge, “Image Style Transfer Using Convolutional Neural Networks,” in *CVPR*, 2016.
- [114] X. Huang and S. Belongie, “Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization,” in *ICCV*, 2017.
- [115] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.
- [116] J. Song, C. Meng, and S. Ermon, “Denoising Diffusion Implicit Models,” in *ICLR*, 2021.
- [117] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, et al., “Flow Matching for Generative Modeling,” in *ICLR*, 2023.
- [118] J. Ho and T. Salimans, “Classifier-Free Diffusion Guidance,” *arXiv:2207.12598*, 2022.
- [119] R. Rombach, A. Blattmann, D. Lorenz, et al., “High-Resolution Image Synthesis with Latent Diffusion Models,” in *CVPR*, 2022.
- [120] W. Peebles and S. Xie, “Scalable Diffusion Models with Transformers,” in *ICCV*, 2023.



- [121] L. Zhang, A. Rao, and M. Agrawala, “Adding Conditional Control to Text-to-Image Diffusion Models,” in *ICCV*, 2023.
- [122] Y. Blau and T. Michaeli, “The Perception-Distortion Tradeoff,” in *CVPR*, 2018.
- [123] C. Ledig, L. Theis, F. Huszár, et al., “Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network,” in *CVPR*, 2017.
- [124] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., “Image Quality Assessment: From Error Visibility to Structural Similarity,” *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [125] M. Heusel, H. Ramsauer, T. Unterthiner, et al., “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium,” in *NeurIPS*, 2017.
- [126] T. Brooks, B. Mildenhall, T. Xue, et al., “Unprocessing Images for Learned Raw Denoising,” in *CVPR*, 2019.
- [127] A. Abdelhamed, S. Lin, and M. S. Brown, “A High-Quality Denoising Dataset for Smartphone Cameras,” in *CVPR*, 2018.
- [128] J. Deng, W. Dong, R. Socher, et al., “ImageNet: A Large-Scale Hierarchical Image Database,” in *CVPR*, 2009.
- [129] A. Radford, J. W. Kim, C. Hallacy, et al., “Learning Transferable Visual Models from Natural Language Supervision,” in *ICML*, 2021.
- [130] J. Li, D. Li, S. Savarese, et al., “BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models,” in *ICML*, 2023.
- [131] H. Liu, C. Li, Q. Wu, et al., “Visual Instruction Tuning,” in *NeurIPS*, 2023.
- [132] S. Kornblith, J. Shlens, and Q. V. Le, “Do Better ImageNet Models Transfer Better?” In *CVPR*, 2019.
- [133] T. Chen, S. Kornblith, M. Norouzi, et al., “A Simple Framework for Contrastive Learning of Visual Representations,” in *ICML*, 2020.
- [134] K. He, H. Fan, Y. Wu, et al., “Momentum Contrast for Unsupervised Visual Representation Learning,” in *CVPR*, 2020.
- [135] J.-B. Grill, F. Strub, F. Altché, et al., “Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning,” in *NeurIPS*, 2020.
- [136] M. Caron, H. Touvron, I. Misra, et al., “Emerging Properties in Self-Supervised Vision Transformers,” in *ICCV*, 2021.
- [137] K. He, X. Chen, S. Xie, et al., “Masked Autoencoders Are Scalable Vision Learners,” in *CVPR*, 2022.
- [138] A. van den Oord, Y. Li, and O. Vinyals, “Representation Learning with Contrastive Predictive Coding,” in *arXiv:1807.03748*, 2018.
- [139] J. Kaplan, S. McCandlish, T. Henighan, et al., “Scaling Laws for Neural Language Models,” *arXiv:2001.08361*, 2020.
- [140] X. Zhai, A. Kolesnikov, N. Houlsby, et al., “Scaling Vision Transformers,” in *CVPR*, 2022.
- [141] J. Hoffmann, S. Borgeaud, A. Mensch, et al., “Training Compute-Optimal Large Language Models,” in *NeurIPS*, 2022.
- [142] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [143] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [144] N. Houlsby, A. Giurgiu, S. Jastrzebski, et al., “Parameter-Efficient Transfer Learning for NLP,” in *ICML*, 2019.



- [145] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [146] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *RecSys*, 2019.

## 6.2 Tối ưu trên pixel thay vì trên trọng số (Optimizing Pixels, Not Weights)

**PHỤ THUỘC** [C/12-co-che-huan-luyen/01-co-hoc-cua-gradient](#) (gradient chảy về đâu là do ta chọn); [C/17-mo-hinh-sinh/01-bon-ho-mo-hinh-sinh](#) (hàm mất mát quyết định kiểu thất bại); [C/13-kien-truc-backbone/01-dong-cnn](#) (bản đồ đặc trưng theo tầng).

### TÓM TẮT

Có một ý tưởng đơn giản nối liền năm kỹ thuật trông chẳng liên quan gì tới nhau: đóng băng một mạng đã huấn luyện, coi *ảnh* là biến cần tối ưu, rồi hạ gradient trên chính pixel. Đối hàm mục tiêu là được một kỹ thuật khác — đảo ngược đặc trưng, DeepDream, trực quan hoá lớp, mẫu đối kháng, và chuyển phong cách. Đọc xong bạn viết được cả năm bằng cùng một vòng lặp mười dòng, và hiểu vì sao chuyển phong cách phiên bản tiến thẳng là ví dụ sạch nhất của mẫu hình “đẩy chi phí từ lúc chạy sang lúc huấn luyện”. Nếu chỉ nhớ một điều: tối ưu trên pixel tìm ra đúng những hướng mà mạng và mắt người *bất đồng*; khi hai bên tình cờ đồng ý ta gọi kết quả là nghệ thuật, khi bất đồng ta gọi nó là mẫu đối kháng — cùng một cơ chế.

### KIẾN THỨC CẦN CÓ TRƯỚC

- **lan truyền ngược (backpropagation)** — thuật toán tính đạo hàm của một đại lượng đầu ra theo mọi nút của một đồ thị tính toán khả vi. Nó không phân biệt “tham số” với “dữ liệu” — đó là tiền đề của cả mục này.
- **bản đồ đặc trưng (feature map)** — tensor đầu ra của một tầng tích chập: một chồng “ảnh” nhỏ hơn, mỗi kênh phản ứng với một loại mẫu hình thị giác.
- **đóng băng (freeze)** — đặt cờ không tính gradient cho trọng số của một mạng, để nó chỉ còn đóng vai một hàm chấm điểm cố định.
- **ma trận Gram** — ma trận tích trong giữa các kênh của một bản đồ đặc trưng, cộng dồn trên mọi vị trí không gian; nó mô tả “kênh nào hay bật cùng kênh nào” mà bỏ hẳn thông tin vị trí.
- **mẫu đối kháng (adversarial example)** — ảnh bị thêm một nhiễu rất nhỏ, mắt người không thấy, nhưng đủ để mạng phân loại sai hoàn toàn.
- **hàm mục tiêu (objective function)** — toàn bộ đại lượng được tối ưu, gồm phần đo sai lệch (**hàm mất mát (loss function)**) cộng các số hạng chính quy hoá. Mục này dùng đúng từ “hàm mục tiêu” vì phần chính quy hoá ở đây quyết định kết quả không kém phần đo sai lệch.
- **chính quy hoá (regularization)** — phần cộng thêm vào hàm mục tiêu để loại bỏ các nghiệm “đúng về số nhưng vô lý về mặt ảnh”, ví dụ phạt biến thiên toàn phần để ép ảnh mượt.
- **feed-forward (tiến thẳng)** — chạy dữ liệu qua mạng đúng một lần từ đầu tới cuối, không lặp — đối lập với một vòng tối ưu nhiều trăm bước.

### A. Đặt vấn đề

Trong huấn luyện thông thường, dữ liệu là hằng số và trọng số là ẩn số: ta cố định ảnh, tính gradient của hàm mất mát theo trọng số, rồi cập nhật trọng số. Nhưng **lan truyền ngược (backpropagation)** không quan tâm ta gọi cái gì là ẩn số — nó tính đạo hàm dọc theo một đồ thị khả vi,

và đồ thị đó có đầu vào cũng là một nút. Vậy hoàn toàn có thể làm ngược lại: cố định trọng số, tính gradient theo *pixel đầu vào*, rồi cập nhật ảnh.

Câu hỏi thú vị không phải “làm được không” mà “để làm gì”. Câu trả lời: một mạng đã huấn luyện là một hàm chấm điểm ngầm định của thế giới thị giác — nó đã học được cái gì là mèo, cái gì là hoa văn, cái gì là biên. Tối ưu trên pixel là cách ép hàm chấm điểm đó *nói ra* thứ nó biết, dưới dạng một bức ảnh. Hình 6.5 là toàn bộ sự khác biệt, vẽ trong một hình.

**Học mục này thì làm được gì mà trước đó không làm được?** Bốn việc: cài chuyển phong cách, DeepDream, trực quan hoá lớp và tấn công đối kháng bằng cùng một đoạn code; giải thích được vì sao ma trận Gram bắt được “phong cách”; nhận ra mẫu hình đời chi phí sang lúc huấn luyện khi gặp lại nó ở **chưng cất tri thức** (*knowledge distillation*); và có một công cụ chẩn đoán mạnh — muốn biết mạng của bạn thật sự nhìn cái gì, hãy tối ưu một ảnh làm nó phát điên rồi nhìn ảnh đó.



**Hình 6.5.** Cùng một đồ thị khả vi, hai cách dùng. Lan truyền ngược không phân biệt “dữ liệu” với “tham số” — nó chỉ tính đạo hàm dọc đồ thị; việc chọn nút nào là biến tối ưu hoàn toàn nằm ở người viết. Đây là lý do năm kỹ thuật ở Bảng 6.2 dùng chung một đoạn code.

→ **trường hợp riêng của C/12/01** — tối ưu trên pixel chỉ là lan truyền ngược chạy tới tận nút đầu vào thay vì dừng ở nút tham số.

## TRỰC GIÁC

Hãy hình dung mạng đã huấn luyện là một giám khảo hội hoạ rất nhất quán nhưng không biết nói — ông ta chỉ chấm điểm, không giải thích. Tối ưu trên pixel là điều khắc một bức tượng bằng cách liên tục hỏi “sửa chỗ nào thì điểm tăng” rồi sửa đúng chỗ đó. Bức tượng cuối cùng phản ánh chính xác thứ mà giám khảo quan tâm — kể cả những thứ vô lý mà ông ta quan tâm nhưng con người thì không. Khi những thứ đó trùng với thẩm mỹ của ta, ta gọi kết quả là chuyển phong cách; khi chúng lệch hẳn, ta gọi kết quả là mẫu đối kháng.

## B. Một vòng lặp, năm kỹ thuật

Vòng lặp chung ngắn tới mức viết ra được trọn vẹn:

```
# Dong bang mang; bien toi uu la anh, khong phai net.parameters()
for p in net.parameters():
    p.requires_grad_(False)
x = init_image.clone().requires_grad_(True)
opt = torch.optim.LBFGS([x])
def step():
    opt.zero_grad()
    loss = objective(net, x) # doi objective -> doi ky thuat
    loss.backward() # gradient chay ve tan x
    return loss
for _ in range(300):
    opt.step(step)
```

Chỉ dòng `objective` thay đổi giữa năm kỹ thuật ở Bảng 6.2. Đây không phải một sự trùng hợp thú vị mà là một bài học về cách đọc tài liệu: khi năm phương pháp chia chung một khung, học khung rẻ hơn học năm phương pháp gấp nhiều lần.

**Bảng 6.2.** Năm kỹ thuật cùng một vòng lặp, khác nhau đúng ở hàm mục tiêu.

| Kỹ thuật            | Hàm mục tiêu                                                | Nó tiết lộ điều gì / hỏng khi nào                                                                          |
|---------------------|-------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| Đảo ngược đặc trưng | Khớp bản đồ đặc trưng của một ảnh mục tiêu ở một tầng       | Tầng càng sâu càng mất thông tin vị trí; ảnh khôi phục ngày càng “đúng nội dung, sai bố cục”               |
| DeepDream           | Cực đại hoá chuẩn của activation ở một tầng                 | Cho thấy tầng đó thích hoa văn gì; kết quả lặp lại một mô-típ tới mức bệnh hoạn                            |
| Trực quan hoá lớp   | Cực đại hoá điểm số của một lớp + phạt chính quy hoá        | Không có phạt thì ra nhiều có điểm số cao — bằng chứng trực tiếp rằng mạng không nhìn như người            |
| Mẫu đối kháng       | Giảm điểm số lớp đúng với ràng buộc $\ \delta\ $ nhỏ        | Tồn tại nhiều vô hình đối hản dự đoán; đây là chế độ hỏng của <i>mọi</i> mạng, không phải của một mạng kém |
| Chuyển phong cách   | Nội dung ở tầng sâu + thống kê phong cách ở nhiều tầng nông | Phong cách nắm được là <i>kết cấu</i> , không phải bố cục hay ngữ nghĩa                                    |

### C. Neural Style Transfer: vì sao ma trận Gram lại là “phong cách”

Bài toán của Gatys: giữ *nội dung* của ảnh A và *phong cách* của ảnh B [113]. Nội dung dễ định nghĩa — bản đồ đặc trưng ở một tầng sâu, vì tầng sâu giữ “có gì ở đâu” mà bỏ bớt chi tiết bề mặt. Phong cách mới là chỗ khó: nó phải là thứ mô tả “chất” của bức tranh mà không mô tả nội dung, tức là phải *độc lập với vị trí*.

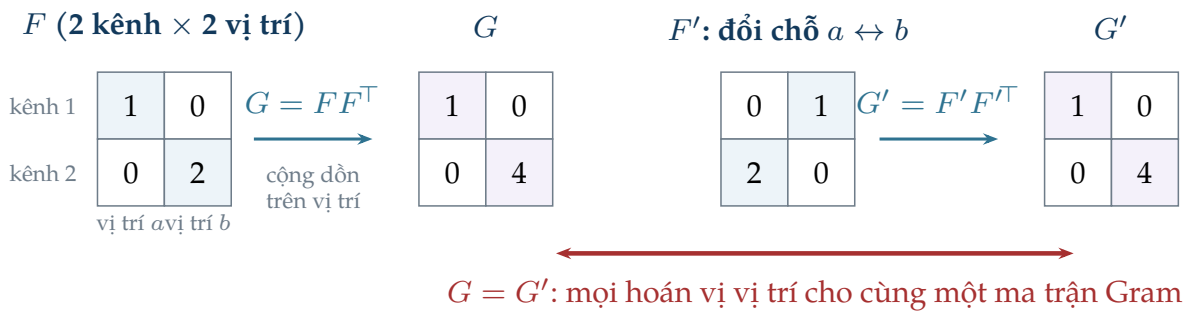
Lời giải là **ma trận Gram** (*Gram matrix*). Gọi  $F \in \mathbb{R}^{C \times N}$  là bản đồ đặc trưng của một tầng, với  $C$  kênh và  $N$  vị trí không gian; đặt  $G = FF^\top$ , tức  $G_{ij} = \sum_p F_{ip}F_{jp}$ . Công thức này nói gì:  $G_{ij}$  đo mức độ hai kênh  $i$  và  $j$  cùng bật lên tại cùng một vị trí, cộng dồn trên toàn ảnh — nó là thống kê *đồng xuất hiện* của các đặc trưng, và phép cộng dồn trên  $p$  đã vứt bỏ hoàn toàn thông tin vị trí.

**Tính tay trên ma trận  $2 \times 2$ .** Hai kênh, hai vị trí:

$$F = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix} \Rightarrow G = FF^\top = \begin{pmatrix} 1 & 0 \\ 0 & 4 \end{pmatrix}, \quad F' = \begin{pmatrix} 0 & 1 \\ 2 & 0 \end{pmatrix} \Rightarrow G' = \begin{pmatrix} 1 & 0 \\ 0 & 4 \end{pmatrix}.$$

Đổi chỗ hai vị trí không gian mà  $G$  không đổi. Đó vừa là sức mạnh — “phong cách” đúng là thứ không phụ thuộc chỗ đặt — vừa là giới hạn: *Gram không phân biệt được bố cục*. Nó nắm được kết cấu nét cọ, bảng màu, độ thô của hạt; nó không nắm được “chân dung thì mất ở trên miệng”. Mọi thất bại quen thuộc của chuyển phong cách đều suy ra từ đây: khuôn mặt bị loang màu, bầu trời mọc nét cọ ở giữa mặt người, cấu trúc lớn của tranh gốc biến mất.

Hình 6.6 vẽ đúng phép tính đó.



**Hình 6.6.** Vì sao ma trận Gram bắt được “phong cách” — và vì sao nó không bao giờ bắt được bố cục. Phép cộng dồn trên các vị trí không gian biến  $F$  thành một thống kê *đồng xuất hiện giữa các kênh*, và phép cộng thì không quan tâm thứ tự. Hai bản đồ đặc trưng khác hẳn nhau về chỗ đặt cho ra cùng một  $G$ . Đó vừa là định nghĩa vừa là giới hạn của “phong cách”: nét cọ và bảng màu thì giữ được, “mắt nằm trên miệng” thì không.

Đáng dừng lại một nhịp ở chỗ này, vì đây là lần thứ hai ta gặp cùng một mẹo:  $G$  là một *tổng* trên các vị trí, tức một hàm đối xứng — đúng cấu trúc mà PointNet dùng khi lấy max trên các điểm (C/16-thi-giac-3d/02-bieu-dien-3d-va-depth). Hai lĩnh vực không liên quan, một nguyên lý: muốn bất biến với thứ tự hay vị trí thì gộp bằng một phép toán đối xứng, và cái giá luôn luôn là mất chính cái thông tin thứ tự đó.

↪ cùng nguyên lý với C/16/02 — PointNet lấy max trên tập điểm, Gram lấy tổng trên vị trí: cùng một hàm đối xứng, cùng một cái giá.

#### D. Dời chi phí từ lúc chạy sang lúc huấn luyện

Bản gốc của Gatys phải chạy vài trăm bước tối ưu cho *mỗi* ảnh, mất từ vài chục giây tới vài phút trên GPU — không dùng được cho video hay cho ứng dụng tương tác. Cách gỡ là một mẫu hình tổng quát, không riêng gì chuyển phong cách.

#### I. Mạch 2 — ba nấc của cùng một bài toán

##### 1 Vấn đề

mỗi ảnh cần một vòng tối ưu riêng.

##### Giải pháp

huấn luyện một mạng **tiền thẳng** (*feed-forward*) để làm đúng việc mà vòng tối ưu làm, dùng chính hàm mất mát của Gatys làm tín hiệu giám sát — tức là **mất mát tri giác** (*perceptual loss*) [111].

##### Hệ quả

lúc chạy chỉ còn một lượt xuôi, nhanh hơn ba bậc độ lớn.

##### Giá phải trả

phải huấn luyện một mạng cho *mỗi* phong cách, và mất khả năng tinh chỉnh trọng số nội dung/phong cách sau khi huấn luyện.

##### 2 Vấn đề

một mạng cho một phong cách là không mở rộng được.

##### Giải pháp

**AdaIN**: chuẩn hoá đặc trưng của ảnh nội dung về trung bình–độ lệch chuẩn 0–1 theo từng kênh, rồi *gán lại* trung bình và độ lệch chuẩn theo từng kênh của ảnh phong cách [114].

##### Giả định

điều đáng nói nằm ở giả định ngầm: nếu chỉ cần khớp thống kê bậc nhất theo kênh đã ra phong cách, thì phần lớn nội dung của ma trận Gram (thống kê bậc hai) là dư thừa — một phát hiện thực nghiệm khá bất ngờ.

##### Hệ quả

một mạng duy nhất làm được phong cách tùy ý, thời gian thực.

##### Giá phải trả

chất lượng thấp hơn bản tối ưu; các phong cách có cấu trúc lớn (tranh có hình khối rõ) bị mất nhiều nhất.

##### 3 Vấn đề

ta vẫn chỉ có một đánh đổi chứ chưa có bữa trưa miễn phí.

Hệ quả

mẫu hình chung: độ khó không biến mất, nó chuyển từ lúc chạy sang lúc huấn luyện — và cùng mẫu hình đó sẽ xuất hiện ở chừng cất tri thức (D/20-toi-uu-va-nen-mo-hinh) và ở việc chừng cất bộ lấy mẫu diffusion (C/17-mo-hinh-sinh/03-diffusion).

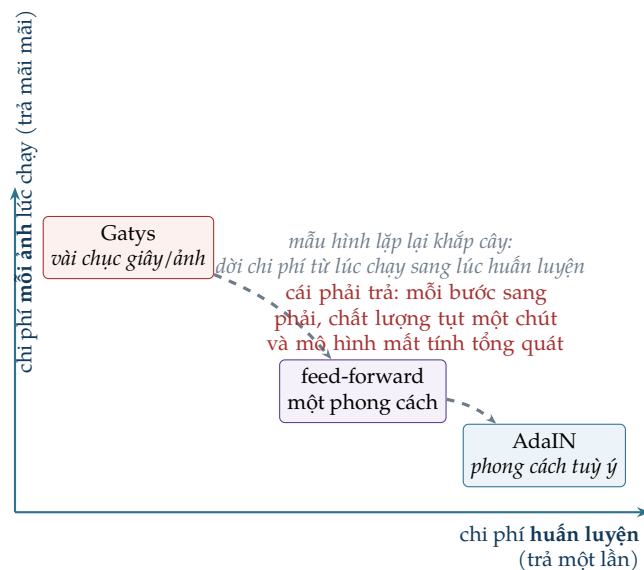
→ cùng nguyên lý với D/20 — chừng cất tri thức là chính mẫu hình này ở tầng mô hình: trả một lần lúc huấn luyện để khỏi trả mãi lúc suy luận.

→ cùng nguyên lý với C/16/03 — neural rendering làm đúng việc này trên một cảnh 3D thay vì trên một ảnh.

**Bảng 6.3.** Ba nấc của chuyển phong cách, đọc theo chỗ chi phí nằm ở đâu.

| Cách làm                  | Chi phí mỗi ảnh            | Chi phí huấn luyện                    | Hỏng khi nào                                            |
|---------------------------|----------------------------|---------------------------------------|---------------------------------------------------------|
| Gatys (tối ưu trên pixel) | Vài chục giây tới vài phút | Không có                              | Không dùng được cho video hay tương tác                 |
| Tiến thẳng một phong cách | Một lượt xuôi              | Một lần huấn luyện cho mỗi phong cách | Số phong cách lớn; muốn đổi trọng số sau khi huấn luyện |
| AdaIN (phong cách tùy ý)  | Một lượt xuôi              | Một lần huấn luyện duy nhất           | Phong cách có cấu trúc lớn; chất lượng dưới bản tối ưu  |

Hình 6.7 đặt ba nấc lên một mặt phẳng để thấy chi phí đã đi đâu.



**Hình 6.7.** Ba nấc của chuyển phong cách, vẽ trên mặt phẳng “ai trả chi phí”. Không nấc nào rẻ hơn nấc trước một cách miễn phí: cái được lấy đi ở trục dọc bị trả lại ở trục ngang, và một phần nhỏ nữa bị trả bằng chất lượng. Đây là cùng một hình sẽ đúng cho chừng cất tri thức và cho chừng cất bộ lấy mẫu diffusion — chỉ cần đổi nhãn ba hộp.

## E. Giới hạn và trường hợp hỏng

Giới hạn nền tảng của cả mục này là ta đang tin vào một hàm chấm điểm mà chính ta không kiểm soát. Mạng đã huấn luyện có vô số hướng trong không gian ảnh mà nó nhạy còn mất người thì không — tối ưu trên pixel đi thẳng vào những hướng đó, vì gradient bao giờ cũng chỉ về hướng dốc nhất. Đó là lý do trực quan hoá lớp không có phạt chính quy hoá cho ra nhiễu, và là lý do mẫu đối kháng tồn tại ở mọi mạng.

Hai chế độ hỏng cụ thể đáng nhớ:

- **Kết quả phụ thuộc mạnh vào khởi tạo và bộ tối ưu.** Bài toán không lỗi trên hàng trăm nghìn biến; khởi tạo từ ảnh nội dung, từ nhiễu, hay từ ảnh trắng cho ba kết quả khác nhau. Đây là tính chất của bài toán chứ không phải lỗi cài đặt — báo cáo kết quả mà không nói khởi tạo là báo cáo thiếu.
- **Chuyển phong cách mất tính nhất quán theo thời gian.** Áp cho từng khung hình video thì ảnh nhấp nháy, vì hai khung hình gần nhau rơi vào hai cực tiểu địa phương khác nhau. Cách chữa là thêm ràng buộc thời gian dựa trên dòng quang — và ta lại quay về đúng chủ đề của [C/15-thi-giac-khong-thoi-gian](#).

#### CẠM BẤY

Đừng đọc một ảnh trực quan hoá lớp như bằng chứng “mạng hiểu khái niệm này”. Ảnh đó là nghiệm của một bài tối ưu có phạt chính quy hoá do bạn chọn, và phần lớn cấu trúc đẹp mắt trong đó đến từ chính phần phạt (biến thiên toàn phần, jitter, làm mượt) chứ không từ mạng. Bằng chứng: bỏ phạt đi thì ra nhiễu, dù điểm số của lớp còn cao hơn. Muốn dùng công cụ này để chẩn đoán, hãy luôn báo cáo phần phạt cùng với ảnh — xem thêm [E/25-do-tin-cay-va-van-hanh](#) về các phép kiểm tra tính đúng đắn của phương pháp giải thích [81].

#### CẦN NHỚ

Lan truyền ngược không phân biệt tham số với dữ liệu; chọn nút nào làm biến tối ưu là quyền của bạn. Đóng băng mạng và tối ưu trên pixel cho ra năm kỹ thuật khác nhau chỉ bằng cách đổi hàm mục tiêu — và bất cứ khi nào một vòng tối ưu lúc chạy quá đắt, hãy hỏi liệu có huấn luyện được một mạng làm thay nó hay không. Đó là mẫu hình dời chi phí sang lúc huấn luyện, gặp lại ở distillation. [T1]

### F. Kết nối

Ý tưởng của mục này là bản sao chính xác của điều đã gặp ở [C/16-thi-giac-3d/03-nerf-3dgs-va-hop-nhat](#): đặt thứ ta muốn vào hàm mất mát rồi hạ gradient trên chính đối tượng cần tạo ra — ở đó đối tượng là một cảnh 3D, ở đây là một ảnh. Ma trận Gram dùng chung nguyên lý hàm đối xứng với PointNet ở [C/16-thi-giac-3d/02-bieu-dien-3d-va-depth](#). Mẫu đối kháng và trực quan hoá được nói kỹ ở [E/25-do-tin-cay-va-van-hanh](#); mẫu hình dời chi phí sang lúc huấn luyện quay lại ở [D/20-toi-uu-va-nen-mo-hinh](#) [115] và ở phần chứng cất bộ lấy mẫu của [C/17-mo-hinh-sinh/03-diffusion](#). Perceptual loss cũng chính là cầu nối sang [C/17-mo-hinh-sinh/04-low-level-vision](#).

#### ❏ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao cùng một đoạn code cho ra được cả một tác phẩm nghệ thuật lẫn một mẫu đối kháng — điều gì thực sự khác nhau giữa hai trường hợp?
2. Ma trận Gram bất biến với vị trí; hai thất bại nào của chuyển phong cách suy ra trực tiếp từ tính chất đó?
3. AdaIN chỉ khớp trung bình và độ lệch chuẩn theo kênh mà vẫn ra phong cách. Điều đó gợi ý gì về lượng thông tin thực sự cần thiết trong ma trận Gram?
4. Bạn thấy một ảnh trực quan hoá lớp rất đẹp trong một bài báo. Ba câu hỏi bạn phải hỏi trước khi tin nó là gì?



5. Mẫu hình “dời chi phí sang lúc huấn luyện” được mua bằng cái gì, và trong trường hợp cụ thể nào thì cái giá đó là không chấp nhận được?

- [1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [2] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [3] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *AISTATS*, 2010.
- [4] K. He, X. Zhang, S. Ren, et al., “Delving Deep into Rectifiers,” in *ICCV*, 2015.
- [5] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [6] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [7] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [8] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [9] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [10] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [11] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” in *ICLR*, 2017.
- [12] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [13] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [14] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [15] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [16] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [17] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [18] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [19] G. Huang, Y. Sun, Z. Liu, et al., “Deep Networks with Stochastic Depth,” in *ECCV*, 2016.
- [20] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [21] H. Zhang, M. Cisse, Y. N. Dauphin, et al., “mixup: Beyond Empirical Risk Minimization,” in *ICLR*, 2018.
- [22] S. Yun, D. Han, S. J. Oh, et al., “CutMix: Regularization Strategy to Train Strong Classifiers,” in *ICCV*, 2019.
- [23] E. D. Cubuk, B. Zoph, J. Shlens, et al., “RandAugment: Practical Automated Data Augmentation,” in *CVPR Workshops*, 2020.

- [24] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [25] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [26] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *NeurIPS*, 2012.
- [27] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in *ICLR*, 2015.
- [28] C. Szegedy, W. Liu, et al., “Going Deeper with Convolutions,” in *CVPR*, 2015.
- [29] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [30] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [31] G. Huang, Z. Liu, L. van der Maaten, et al., “Densely Connected Convolutional Networks,” in *CVPR*, 2017.
- [32] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [33] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [34] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [35] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [36] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [37] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.
- [38] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [39] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [40] J. Su, M. Ahmed, Y. Lu, et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, vol. 568, 2024.
- [41] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” *arXiv:2312.00752*, 2023.
- [42] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [43] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.
- [44] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [45] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [46] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [47] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [48] R. Girshick, J. Donahue, T. Darrell, et al., “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation,” in *CVPR*, 2014.

- [49] R. Girshick, "Fast R-CNN," in *ICCV*, 2015.
- [50] S. Ren, K. He, R. Girshick, et al., "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in *NeurIPS*, 2015.
- [51] T.-Y. Lin, P. Dollár, R. Girshick, et al., "Feature Pyramid Networks for Object Detection," in *CVPR*, 2017.
- [52] K. He, G. Gkioxari, P. Dollár, et al., "Mask R-CNN," in *ICCV*, 2017.
- [53] D. Bolya, S. Foley, J. Hays, et al., "TIDE: A General Toolbox for Identifying Object Detection Errors," in *ECCV*, 2020.
- [54] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [55] J. Redmon, S. Divvala, R. Girshick, et al., "You Only Look Once: Unified, Real-Time Object Detection," in *CVPR*, 2016.
- [56] W. Liu, D. Anguelov, D. Erhan, et al., "SSD: Single Shot MultiBox Detector," in *ECCV*, 2016.
- [57] T.-Y. Lin, P. Goyal, R. Girshick, et al., "Focal Loss for Dense Object Detection," in *ICCV*, 2017.
- [58] Z. Tian, C. Shen, H. Chen, et al., "FCOS: Fully Convolutional One-Stage Object Detection," in *ICCV*, 2019.
- [59] X. Zhou, D. Wang, and P. Krähenbühl, "Objects as Points," *arXiv:1904.07850*, 2019.
- [60] S. Zhang, C. Chi, Y. Yao, et al., "Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection," in *CVPR*, 2020.
- [61] Z. Ge, S. Liu, Z. Li, et al., "OTA: Optimal Transport Assignment for Object Detection," in *CVPR*, 2021.
- [62] N. Bodla, B. Singh, R. Chellappa, et al., "Soft-NMS: Improving Object Detection with One Line of Code," in *ICCV*, 2017.
- [63] X. Zhu, W. Su, L. Lu, et al., "Deformable DETR: Deformable Transformers for End-to-End Object Detection," in *ICLR*, 2021.
- [64] H. Zhang, F. Li, S. Liu, et al., "DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection," in *ICLR*, 2023.
- [65] J. Long, E. Shelhamer, and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," in *CVPR*, 2015.
- [66] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *MICCAI*, 2015.
- [67] L.-C. Chen, G. Papandreou, I. Kokkinos, et al., "DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs," *IEEE TPAMI*, vol. 40, no. 4, pp. 834–848, 2018.
- [68] E. Xie, W. Wang, Z. Yu, et al., "SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers," in *NeurIPS*, 2021.
- [69] B. Cheng, I. Misra, A. G. Schwing, et al., "Masked-Attention Mask Transformer for Universal Image Segmentation," in *CVPR*, 2022.
- [70] A. Kirillov, K. He, R. Girshick, et al., "Panoptic Segmentation," in *CVPR*, 2019.
- [71] A. Kirillov, E. Mintun, N. Ravi, et al., "Segment Anything," in *ICCV*, 2023.
- [72] K. Sun, B. Xiao, D. Liu, et al., "Deep High-Resolution Representation Learning for Human Pose Estimation," in *CVPR*, 2019.

- [73] Z. Cao, T. Simon, S.-E. Wei, et al., “Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields,” in *CVPR*, 2017.
- [74] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [75] J. Deng, J. Guo, N. Xue, et al., “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *CVPR*, 2019.
- [76] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [77] K. Xu, J. Ba, R. Kiros, et al., “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention,” in *ICML*, 2015.
- [78] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *ICLR*, 2015.
- [79] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [80] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [81] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [82] K. Simonyan and A. Zisserman, “Two-Stream Convolutional Networks for Action Recognition in Videos,” in *NeurIPS*, 2014.
- [83] J. Carreira and A. Zisserman, “Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset,” in *CVPR*, 2017.
- [84] D. Tran, H. Wang, L. Torresani, et al., “A Closer Look at Spatiotemporal Convolutions for Action Recognition,” in *CVPR*, 2018.
- [85] C. Feichtenhofer, H. Fan, J. Malik, et al., “SlowFast Networks for Video Recognition,” in *ICCV*, 2019.
- [86] A. Dosovitskiy, P. Fischer, E. Ilg, et al., “FlowNet: Learning Optical Flow with Convolutional Networks,” in *ICCV*, 2015.
- [87] D. Sun, X. Yang, M.-Y. Liu, et al., “PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume,” in *CVPR*, 2018.
- [88] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.
- [89] A. Bewley, Z. Ge, L. Ott, et al., “Simple Online and Realtime Tracking,” in *ICIP*, 2016.
- [90] Y. Zhang, P. Sun, Y. Jiang, et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *ECCV*, 2022.
- [91] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [92] K. Bernardin and R. Stiefelwagen, “Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics,” *EURASIP J. Image and Video Processing*, 2008.
- [93] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [94] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [95] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.

- [96] S. Wang, V. Leroy, Y. Cabon, et al., "DUST3R: Geometric 3D Vision Made Easy," in *CVPR*, 2024.
- [97] D. Eigen, C. Puhrsch, and R. Fergus, "Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network," in *NeurIPS*, 2014.
- [98] R. Ranftl, K. Lasinger, D. Hafner, et al., "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer," *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [99] J. Sturm, N. Engelhard, F. Endres, et al., "A Benchmark for the Evaluation of RGB-D SLAM Systems," in *IROS*, 2012.
- [100] C. R. Qi, H. Su, K. Mo, et al., "PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation," in *CVPR*, 2017.
- [101] C. R. Qi, L. Yi, H. Su, et al., "PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space," in *NeurIPS*, 2017.
- [102] A. H. Lang, S. Vora, H. Caesar, et al., "PointPillars: Fast Encoders for Object Detection from Point Clouds," in *CVPR*, 2019.
- [103] T. Müller, A. Evans, C. Schied, et al., "Instant Neural Graphics Primitives with a Multiresolution Hash Encoding," *ACM TOG*, vol. 41, no. 4, 2022.
- [104] B. Kerbl, G. Kopanas, T. Leimkühler, et al., "3D Gaussian Splatting for Real-Time Radiance Field Rendering," *ACM TOG*, vol. 42, no. 4, 2023.
- [105] R. Zhang, P. Isola, A. A. Efros, et al., "The Unreasonable Effectiveness of Deep Features as a Perceptual Metric," in *CVPR*, 2018.
- [106] D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," in *ICLR*, 2014.
- [107] I. Goodfellow, J. Pouget-Abadie, M. Mirza, et al., "Generative Adversarial Nets," in *NeurIPS*, 2014.
- [108] T. Karras, S. Laine, and T. Aila, "A Style-Based Generator Architecture for Generative Adversarial Networks," in *CVPR*, 2019.
- [109] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," in *NeurIPS*, 2020.
- [110] Y. Song, J. Sohl-Dickstein, D. P. Kingma, et al., "Score-Based Generative Modeling Through Stochastic Differential Equations," in *ICLR*, 2021.
- [111] J. Johnson, A. Alahi, and L. Fei-Fei, "Perceptual Losses for Real-Time Style Transfer and Super-Resolution," in *ECCV*, 2016.
- [112] P. Isola, J.-Y. Zhu, T. Zhou, et al., "Image-to-Image Translation with Conditional Adversarial Networks," in *CVPR*, 2017.
- [113] L. A. Gatys, A. S. Ecker, and M. Bethge, "Image Style Transfer Using Convolutional Neural Networks," in *CVPR*, 2016.
- [114] X. Huang and S. Belongie, "Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization," in *ICCV*, 2017.
- [115] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," in *NeurIPS Deep Learning Workshop*, 2015.
- [116] J. Song, C. Meng, and S. Ermon, "Denoising Diffusion Implicit Models," in *ICLR*, 2021.
- [117] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, et al., "Flow Matching for Generative Modeling," in *ICLR*, 2023.
- [118] J. Ho and T. Salimans, "Classifier-Free Diffusion Guidance," *arXiv:2207.12598*, 2022.
- [119] R. Rombach, A. Blattmann, D. Lorenz, et al., "High-Resolution Image Synthesis with Latent Diffusion Models," in *CVPR*, 2022.
- [120] W. Peebles and S. Xie, "Scalable Diffusion Models with Transformers," in *ICCV*, 2023.



- [121] L. Zhang, A. Rao, and M. Agrawala, “Adding Conditional Control to Text-to-Image Diffusion Models,” in *ICCV*, 2023.
- [122] Y. Blau and T. Michaeli, “The Perception-Distortion Tradeoff,” in *CVPR*, 2018.
- [123] C. Ledig, L. Theis, F. Huszár, et al., “Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network,” in *CVPR*, 2017.
- [124] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., “Image Quality Assessment: From Error Visibility to Structural Similarity,” *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [125] M. Heusel, H. Ramsauer, T. Unterthiner, et al., “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium,” in *NeurIPS*, 2017.
- [126] T. Brooks, B. Mildenhall, T. Xue, et al., “Unprocessing Images for Learned Raw Denoising,” in *CVPR*, 2019.
- [127] A. Abdelhamed, S. Lin, and M. S. Brown, “A High-Quality Denoising Dataset for Smartphone Cameras,” in *CVPR*, 2018.
- [128] J. Deng, W. Dong, R. Socher, et al., “ImageNet: A Large-Scale Hierarchical Image Database,” in *CVPR*, 2009.
- [129] A. Radford, J. W. Kim, C. Hallacy, et al., “Learning Transferable Visual Models from Natural Language Supervision,” in *ICML*, 2021.
- [130] J. Li, D. Li, S. Savarese, et al., “BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models,” in *ICML*, 2023.
- [131] H. Liu, C. Li, Q. Wu, et al., “Visual Instruction Tuning,” in *NeurIPS*, 2023.
- [132] S. Kornblith, J. Shlens, and Q. V. Le, “Do Better ImageNet Models Transfer Better?” In *CVPR*, 2019.
- [133] T. Chen, S. Kornblith, M. Norouzi, et al., “A Simple Framework for Contrastive Learning of Visual Representations,” in *ICML*, 2020.
- [134] K. He, H. Fan, Y. Wu, et al., “Momentum Contrast for Unsupervised Visual Representation Learning,” in *CVPR*, 2020.
- [135] J.-B. Grill, F. Strub, F. Altché, et al., “Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning,” in *NeurIPS*, 2020.
- [136] M. Caron, H. Touvron, I. Misra, et al., “Emerging Properties in Self-Supervised Vision Transformers,” in *ICCV*, 2021.
- [137] K. He, X. Chen, S. Xie, et al., “Masked Autoencoders Are Scalable Vision Learners,” in *CVPR*, 2022.
- [138] A. van den Oord, Y. Li, and O. Vinyals, “Representation Learning with Contrastive Predictive Coding,” in *arXiv:1807.03748*, 2018.
- [139] J. Kaplan, S. McCandlish, T. Henighan, et al., “Scaling Laws for Neural Language Models,” *arXiv:2001.08361*, 2020.
- [140] X. Zhai, A. Kolesnikov, N. Houlsby, et al., “Scaling Vision Transformers,” in *CVPR*, 2022.
- [141] J. Hoffmann, S. Borgeaud, A. Mensch, et al., “Training Compute-Optimal Large Language Models,” in *NeurIPS*, 2022.
- [142] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [143] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [144] N. Houlsby, A. Giurgiu, S. Jastrzebski, et al., “Parameter-Efficient Transfer Learning for NLP,” in *ICML*, 2019.

- [145] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [146] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *RecSys*, 2019.

## 6.3 Diffusion và các bước làm cho nó khả thi (Diffusion Models)

**PHỤ THUỘC** C/17-mo-hình-sinh/01-bon-ho-mo-hình-sinh (vì sao diffusion ổn định hơn GAN); B/09-thiet-ke-ham-mat-mat (hồi quy trên nhiễu là bình phương trung bình); C/13-kien-truc-backbone (U-Net và transformer); C/12-co-che-huan-luyen (huấn luyện quy mô lớn).

### TÓM TẮT

Diffusion biến một bài toán sinh rất khó thành một chuỗi bài toán khử nhiễu rất dễ, và đó là toàn bộ lý do nó thắng: hàm mất mát của nó là bình phương trung bình trên nhiễu, ổn định như mọi bài hồi quy. Mục này đi từ nguyên lý (quá trình thuận cố định, quá trình ngược học được, liên hệ với score function) tới bốn bước biến nó thành sản phẩm dùng được: bộ lấy mẫu ít bước, dẫn hướng không cần bộ phân lớp, khuếch tán trong không gian ẩn, và cơ chế điều khiển đầu ra. Đọc xong bạn biết mỗi tham số trong một pipeline sinh ảnh đang mua gì bằng gì — số bước, hệ số dẫn hướng, độ mạnh ảnh sang ảnh — và biết *latent diffusion* phá hỏng cái gì khi nó tiết kiệm tính toán. Nếu chỉ nhớ một điều: mọi cơ chế điều khiển trong diffusion đều mua độ trung thành bằng *độ đa dạng*, nên một pipeline chính tới mức đẹp nhất cũng là pipeline có phân phối hẹp nhất — điều nguy hiểm nếu bạn định dùng đầu ra làm dữ liệu huấn luyện.

### KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình sinh (generative model)** — mô hình học phân phối của dữ liệu chứ không học một ảnh xạ đầu vào → nhãn; dùng nó là lấy mẫu ra dữ liệu mới, không phải dự đoán.
- **nhiều Gauss (Gaussian noise)** — nhiễu cộng lấy từ phân phối chuẩn; “pha nhiễu” nghĩa là cộng thêm một lượng nhiễu như vậy vào ảnh, làm ảnh mờ dần thành hạt.
- **khử nhiễu (denoising)** — việc ước lượng lại ảnh sạch (hoặc lượng nhiễu đã cộng vào) từ ảnh đã bị pha nhiễu. Diffusion chỉ làm đúng việc này, lặp lại nhiều lần.
- **bộ lấy mẫu (sampler)** — đoạn mã quyết định đi từ nhiễu về ảnh theo mấy bước và mỗi bước dài bao nhiêu. Nó tách rời khỏi mô hình: cùng một mô hình đổi sampler cho ra ảnh khác nhau.
- **không gian ẩn (latent space)** — không gian chiều thấp mà một autoencoder nén ảnh vào; một ảnh  $512 \times 512$  thành một tensor  $64 \times 64$ , nên mọi phép tính sau đó rẻ đi hàng chục lần.
- **dẫn hướng (guidance)** — kỹ thuật kéo mẫu sinh ra về phía thoả mãn điều kiện (câu mô tả, ảnh cạnh...) mạnh hơn mức tự nhiên, đổi lại phân phối hẹp hơn.
- **score function (score function)** — gradient của log mật độ xác suất theo dữ liệu,  $\nabla_x \log p(x)$ ; nó là một trường vector chỉ về hướng “đi đâu thì hợp lý hơn” tại mỗi điểm trong không gian ảnh.
- **SNR (signal-to-noise ratio)** — tỉ số giữa phần tín hiệu gốc còn lại và phần nhiễu ở một bước; SNR cao nghĩa là ảnh còn gần như nguyên vẹn.

### A. Đặt vấn đề

Sau mục 17.1 ta có một yêu cầu rõ ràng: cần chất lượng của GAN mà không có sự bất ổn của một bài toán trò chơi. Ý tưởng của diffusion đơn giản một cách bất ngờ — nếu bài toán “sinh một ảnh từ hư không” quá khó, hãy chẻ nó thành một nghìn bài toán “làm ảnh này bớt nhiễu đi một chút”, mỗi bài dễ tới mức một mạng hồi quy bình thường giải được. Cộng đủ nhiều bước dễ thì ra một bước khó.

Cái phải trả cho sự chẻ nhỏ đó là thời gian: sinh một ảnh cần chạy mạng hàng chục tới hàng nghìn lần. Toàn bộ lịch sử kỹ thuật của mảng này, từ 2020 tới nay, là chuỗi các nỗ lực trả bớt cái giá đó mà không mất chất lượng — và đó cũng là cấu trúc của mục này.

**Học mục này thì làm được gì mà trước đó không làm được?** Đọc một tệp cấu hình `diffusers` và biết mỗi tham số đang đánh đổi gì; chẩn đoán được ảnh ra xấu là do bộ lấy mẫu, do hệ số dẫn hướng, hay do bộ mã hoá ẩn; và biết khi nào một mô hình sinh ảnh *không* nên được dùng để tạo dữ liệu huấn luyện.

#### TRỰC GIÁC

Hãy nghĩ tới một bức ảnh bị nhoè dần trong một chuỗi bản sao chép: bản 1 gần như nguyên vẹn, bản 500 lem nhem, bản 1000 chỉ còn là hạt. Việc đoán bản 1000 từ hư không là bất khả. Nhưng việc đoán bản 499 khi đã có bản 500 thì gần như tầm thường — chỉ phải lùi một nấc. Diffusion là phép nhận ra rằng nếu bạn làm được cái tầm thường đó một nghìn lần, bạn đã làm được cái bất khả. Và điều tinh tế: mấy nấc *cuối* (nhiều nhiều) là nơi mô hình quyết định bức ảnh sẽ vẽ *cái gì*, còn mấy nấc *đầu* (nhiều ít) chỉ tô nốt chi tiết.

### B. Quá trình thuận, quá trình ngược

Quá trình thuận không có gì để học: nó chỉ là công thức pha nhiều Gauss dần dần, và nhờ tính chất cộng của Gauss, ta nhảy thẳng từ ảnh gốc tới bước bất kỳ mà không cần lặp:

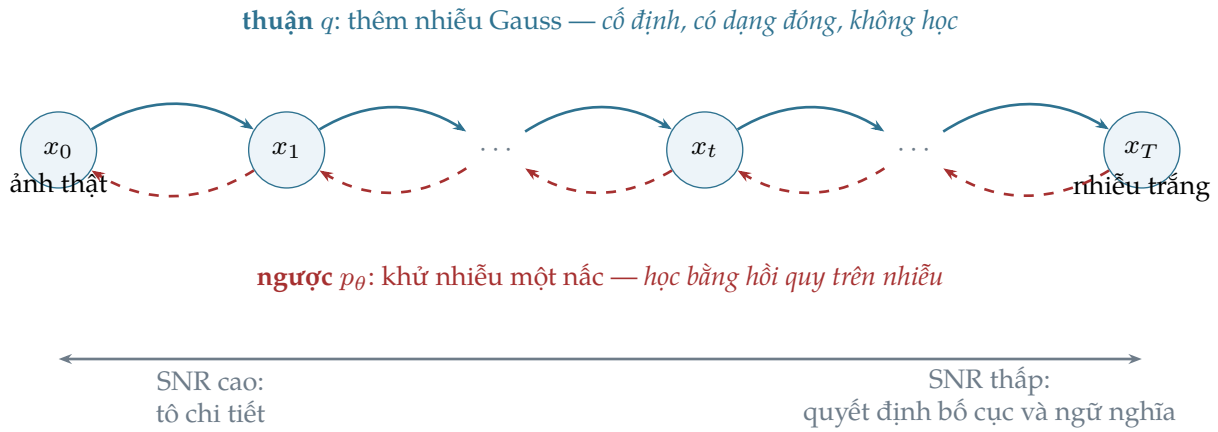
$$q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) I), \quad \bar{\alpha}_t = \prod_{s \leq t} (1 - \beta_s).$$

*Công thức này nói gì:* ở bước  $t$ , ảnh còn lại đúng một phần  $\sqrt{\bar{\alpha}_t}$  tín hiệu gốc, phần còn lại là nhiễu trắng — và vì có dạng đóng, ta lấy mẫu một  $t$  ngẫu nhiên rồi huấn luyện ngay, không phải chạy chuỗi. Đây là lý do kỹ thuật khiến diffusion huấn luyện được ở quy mô lớn.

**Tính tay để thấy  $T = 1000$  từ đâu ra.** Với lịch  $\beta$  tuyến tính từ  $10^{-4}$  tới 0,02, trung bình  $\beta \approx 0,0101$ , nên  $\bar{\alpha}_T \approx \exp(-\sum_t \beta_t) = \exp(-10,1) \approx 4 \times 10^{-5}$ , tức  $\sqrt{\bar{\alpha}_T} \approx 0,006$ . Nói cách khác, sau một nghìn bước thì chưa tới 1 % biên độ tín hiệu gốc còn sót lại — vừa đủ để coi  $x_T$  là nhiễu trắng thuần túy và bắt đầu quá trình ngược từ đó.

Quá trình ngược mới là phần học. Mạng nhận  $x_t$  và  $t$ , dự đoán *nhiều* đã thêm vào, và hàm mất mát là bình phương trung bình  $\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2$  [109]. Đây là điểm mấu chốt của cả họ mô hình: mục tiêu là một bài hồi quy tầm thường, không phải một trò chơi hai người, nên không có bất ổn, không có *mode collapse*, và tăng quy mô thì tốt lên đều đặn.

↪ **hệ quả của** B/09 — ổn định của diffusion đến thẳng từ dạng của hàm mất mát — bình phương trung bình trên nhiễu — chứ không từ kiến trúc nào.



**Hình 6.8.** Hai chiều của cùng một chuỗi. Chiều thuận không có tham số nào, nên toàn bộ khả năng của mô hình dồn vào chiều ngược. Trục dưới giải thích một điều thực hành quan trọng: các bước ở phía nhiễu nhiều quyết định *nội dung* ảnh, nên cắt bớt bước ở phía đó thay đổi ảnh nhiều hơn hẳn cắt bớt ở phía nhiễu ít — đó là lý do các lịch lấy mẫu ít bước không chia đều khoảng cách.

### C. Vì sao gọi là “khớp score function”, và vì sao cần nhiều bước

Có một cách nhìn thứ hai cho cùng một mô hình, và nó giải thích được nhiều thứ hơn. Với phân phối  $p_t$  của dữ liệu đã pha nhiễu ở mức  $t$ , **score function** là  $\nabla_x \log p_t(x)$  — vector chỉ về hướng “đi đâu thì hợp lý hơn”. Có thể chứng minh rằng mạng dự đoán nhiễu ở trên chính là ước lượng của score function sai khác một hệ số:  $\varepsilon_\theta(x_t, t) \approx -\sqrt{1 - \bar{\alpha}_t} \nabla_x \log p_t(x_t)$  [110]. *Nói gọn thì*: mô hình học một trường vector phủ khắp không gian ảnh, và lấy mẫu là thả một điểm nhiễu vào trường đó rồi để nó trôi về vùng có mật độ cao.

Cách nhìn này trả lời ngay câu hỏi “vì sao cần nhiều bước”: trôi theo một trường vector là giải một phương trình vi phân, và mỗi bước rời rạc là một xấp xỉ tuyến tính chỉ đúng trong lân cận nhỏ. Bước dài thì sai số rời rạc hoá lớn, ảnh ra méo. Từ đó suy ra hai hướng cải thiện, và cả hai đều đã thành chuẩn:

- **Đổi bộ giải.** DDIM biến quá trình lấy mẫu ngẫu nhiên thành một phương trình vi phân thường tất định, cho phép nhảy bước dài hơn với cùng mô hình đã huấn luyện [116]. Không huấn luyện lại gì cả — đây thuần tuý là giải số tốt hơn.
- **Làm thẳng quỹ đạo.** *Flow matching* huấn luyện thẳng một trường vận tốc nổi nhiễu với dữ liệu theo đường càng gần đường thẳng càng tốt [117]; quỹ đạo thẳng thì bộ giải cần ít bước hơn. Đây là hướng thiết kế thống trị các mô hình sinh lớn hiện nay.

### D. Điều khiển đầu ra: dẫn hướng, không gian ẩn, ràng buộc cấu trúc

#### I Mạch 3 — bốn bước biến diffusion thành sản phẩm

##### 1 Vấn đề

##### Giải pháp

mô hình sinh ra ảnh hợp lý nhưng không theo ý ta.

**classifier-free guidance**: trong lúc huấn luyện, thỉnh thoảng bỏ điều kiện đi để một mạng duy nhất học cả bản có điều kiện lẫn bản không; lúc lấy mẫu, ngoại suy  $\tilde{\varepsilon} = \varepsilon_{\text{vô đk}} + w(\varepsilon_{\text{có đk}} - \varepsilon_{\text{vô đk}})$  [118]. *Công thức này nói gì*: lấy hiệu giữa “có điều kiện” và “không điều kiện” làm hướng của điều kiện, rồi đi quá lên  $w$  lần theo hướng đó.

##### Giá phải trả

$w > 1$  không phải là “làm đúng hơn” mà là *làm phân phối hẹp lại*: bám prompt tốt hơn, đa dạng kém đi, và ở  $w$  lớn thì màu bị cháy.

##### 2 Vấn đề

chạy khuếch tán trên không gian pixel quá đắt.

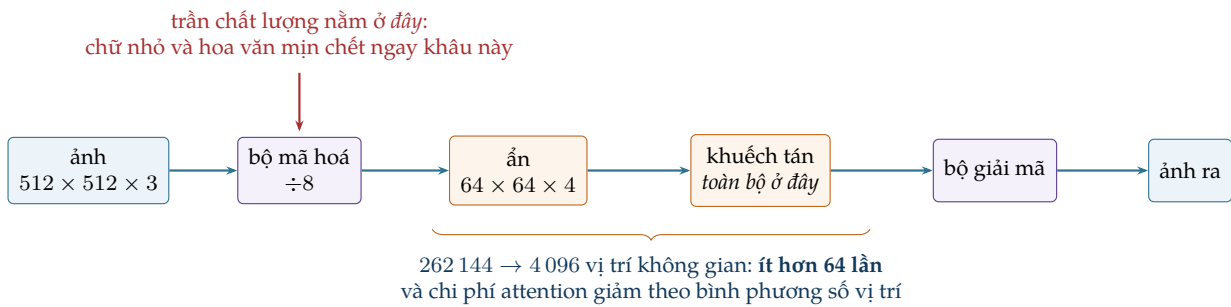
- Giải pháp** nén ảnh bằng một autoencoder rồi khuếch tán trong không gian ẩn [119]. Với hệ số nén 8, ảnh  $512 \times 512$  còn  $64 \times 64$  vị trí — ít hơn 64 lần, và vì chi phí *attention* tăng theo bình phương số vị trí, phần đó rẻ đi hàng nghìn lần.
- Hệ quả** đây chính là bước biến diffusion từ nghiên cứu thành sản phẩm chạy được trên GPU tiêu dùng.
- ↔ **ràng buộc bởi** B/11 — lợi ích của không gian ẩn tính trực tiếp từ ngân sách bộ nhớ và chi phí attention theo số vị trí.
- Giá phải trả** chất lượng bị chặn trên bởi chất lượng autoencoder; chi tiết rất nhỏ — chữ nhỏ, mắt người ở xa, hoa văn mịn — bị phá ngay ở khâu nén và không mô hình khuếch tán nào cứu lại được.
- 3 Vấn đề** U-Net khó mở rộng quy mô một cách có quy luật.
- Giải pháp** thay bằng *transformer* thao tác trên các *token* của không gian ẩn — *diffusion transformer* [120].
- Hệ quả** quy luật mở rộng quy mô trở nên dự đoán được, giống như ở mô hình ngôn ngữ.
- 4 Vấn đề** prompt bằng chữ không đủ để ràng buộc cấu trúc hình học.
- Giải pháp** *ControlNet*: nhân bản phần mã hoá của mô hình gốc, cho nhánh sao chép nhận điều kiện cấu trúc (cạnh, tư thế người, bản đồ độ sâu), và nối nó vào mô hình gốc qua các tầng tích chập *khởi tạo bằng 0* [121]. *Chi tiết khởi tạo 0 mới là chỗ hay*: ở bước huấn luyện đầu tiên nhánh mới đóng góp đúng bằng không, nên mô hình gốc không bị phá; huấn luyện chỉ *thêm* khả năng chứ không làm hỏng prior đã có.
- Giá phải trả** càng nhiều ràng buộc thì càng ít đa dạng — cùng một quy luật với dẫn hướng ở bước 1.

**Bảng 6.4.** Ba cách điều khiển đầu ra. Cả ba mua độ trung thành bằng cùng một đồng tiền là độ đa dạng.

| Cách                      | Cơ chế                                      | Ràng buộc được gì                       | Mất gì                               |
|---------------------------|---------------------------------------------|-----------------------------------------|--------------------------------------|
| Dẫn hướng (CFG)           | Ngoại suy giữa nhánh có và không điều kiện  | Ngữ nghĩa, độ bám prompt                | Đa dạng; $w$ lớn gây cháy màu        |
| ControlNet                | Nhánh sao chép, nối bằng tầng khởi tạo 0    | Cấu trúc hình học: cạnh, tư thế, độ sâu | Tự do bố cục; thêm chi phí tính toán |
| Ảnh sang ảnh / inpainting | Bắt đầu từ ảnh đã pha nhiễu tới mức $t < T$ | Bố cục và màu của ảnh gốc               | Mức thay đổi bị chặn bởi $t$ đã chọn |

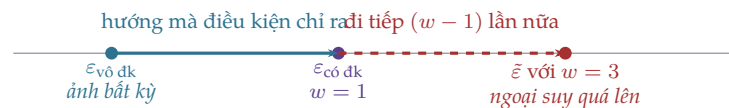
Riêng dòng cuối bảng đáng một câu giải thích, vì nó là hệ quả trực tiếp của Hình 6.8: tham số “độ mạnh” trong ảnh sang ảnh thực chất chỉ là *chọn bắt đầu từ nấc nào*. Bắt đầu ở nấc nhiễu ít thì ảnh gốc còn nguyên và mô hình chỉ tô lại bề mặt; bắt đầu ở nấc nhiễu nhiều thì bố cục cũng bị viết lại. Không có gì bí ẩn ở đây — chỉ là một chỉ số trên trục thời gian.

Hình 6.9 và Hình 6.10 vẽ hai bước quan trọng nhất trong mạch trên.



**Hình 6.9.** Latent diffusion, và chỗ nó lấy tiền về cũng như chỗ nó trả tiền ra. Toàn bộ phần đất — hàng chục tới hàng trăm lượt chạy mạng — được dời vào không gian ẩn nhỏ hơn 64 lần về số vị trí, và đó là lý do một mô hình sinh ảnh chạy được trên GPU tiêu dùng. Cái giá nằm ở mũi tên đỏ: mọi chi tiết mà bộ mã hoá không giữ được thì không mô hình khuếch tán nào ở phía sau dựng lại được. Vì vậy phép chẩn đoán đầu tiên khi ảnh mất chi tiết là mã hoá rồi giải mã ngay ảnh gốc, bỏ qua khuếch tán.

càng sang phải: bám prompt càng chặt, phân phối càng **hẹp**, màu càng dễ cháy



**Hình 6.10.** Dẫn hướng không cần bộ phân lớp, nhìn như một phép ngoại suy trên đường thẳng. Hiệu giữa dự đoán có điều kiện và dự đoán không điều kiện chính là “hướng của điều kiện”; hệ số  $w$  quyết định đi bao xa theo hướng đó.  $w = 1$  là dừng đúng dự đoán có điều kiện;  $w > 1$  là đi *quá* lên. Điều quan trọng cần đọc ra từ hình: không có chỗ nào trong phép này làm mô hình “đúng hơn” — nó chỉ đẩy mẫu về vùng điển hình và cắt bớt đuôi, nên ảnh riêng lẻ đẹp hơn còn tập ảnh thì nghèo đi.

## E. Giới hạn và trường hợp hỏng

- **Chậm là tính chất, không phải lỗi.** Mọi cách tăng tốc — ít bước, chưng cất bộ lấy mẫu, quỹ đạo thẳng — đều đang xấp xỉ cùng một quỹ đạo, và ở số bước rất nhỏ thì chất lượng và đa dạng đều tụt. Chưng cất bộ lấy mẫu là đúng mẫu hình “dời chi phí sang lúc huấn luyện” của mục 17.2.

↔ cùng nguyên lý với D/20 — cùng phép đổi chi phí suy luận lấy chi phí huấn luyện đã gặp ở chưng cất mô hình.

- **Trần chất lượng nằm ở autoencoder.** Nếu ảnh ra mất chữ nhỏ hoặc chi tiết mịn, đừng đổ cho mô hình khuếch tán trước khi kiểm tra: mã hoá rồi giải mã ngay ảnh gốc, không chạy khuếch tán, và xem chi tiết đó có sống sót không. Phép thử này mất ba mươi giây và loại bỏ được một nửa số giả thuyết.
- **Đếm, chữ, và quan hệ không gian vẫn yếu.** Đây là hệ quả của việc điều kiện đi vào mô hình qua *cross-attention* trên biểu diễn văn bản, chứ không qua một cơ chế suy luận rời rạc nào.
- **Lệch giữa mô hình và bộ lấy mẫu.** Một mô hình huấn luyện với lịch nhiễu này, chạy bằng bộ lấy mẫu giả định lịch khác, cho ra ảnh xấu mà không có thông báo lỗi nào. Đây là lỗi cấu hình phổ biến nhất trong thực hành.

### CẠM BẦY

Tăng hệ số dẫn hướng cho tới khi ảnh “đẹp nhất” là một cái bẫy có hậu quả xa. Dẫn hướng mạnh không làm mô hình đúng hơn — nó *cắt bớt đuôi phân phối*, giữ lại phần lõi điển hình nhất. Ảnh riêng lẻ đẹp hơn, nhưng tập ảnh sinh ra nghèo hơn hẳn tập ảnh thật. Nếu bạn định dùng đầu ra làm dữ liệu huấn luyện, đây chính là cơ chế gây thu hẹp



phân phối qua từng vòng lặp: mỗi thể hệ học từ đuôi đã bị cắt của thể hệ trước. Xem [C/17-mo-hinh-sinh/04-low-level-vision](#).

↔ **tiền đề** [C/17/04](#) — muốn hiểu vì sao dữ liệu sinh có thể đầu độc mô hình sau, phải hiểu trước rằng mọi nóm đều khiến ở đây đều cắt đuôi phân phối.

### CẦN NHỚ

Diffusion thắng không phải vì ý tưởng đẹp hơn GAN mà vì nó biến bài toán sinh thành một bài *hồi quy*, và hồi quy thì mở rộng quy mô được. Ba con số đáng nhớ khi chỉnh một pipeline: số bước quyết định sai số rời rạc hoá, hệ số dẫn hướng quyết định đánh đổi trung thành ↔ đa dạng, và hệ số nén của autoencoder quyết định trần chi tiết. Ba tham số đó độc lập nhau, và chẩn đoán sai lầm hay gặp nhất là đổ lỗi cho tham số thứ nhất khi thủ phạm là tham số thứ ba. [T2]

## F. Kết nối

Mục này là phần chi tiết của họ thứ tư trong [C/17-mo-hinh-sinh/01-bon-ho-mo-hinh-sinh](#); “chưng cất bộ lấy mẫu” là bản sao của mẫu hình ở [C/17-mo-hinh-sinh/02-toi-uu-tren-pixel](#) và ở [D/20-toi-uu-va-nen-mo-hinh](#). Việc dùng đầu ra làm dữ liệu huấn luyện và hậu quả thu hẹp phân phối nằm ở [C/17-mo-hinh-sinh/04-low-level-vision](#); phần kiến trúc transformer thay U-Net nổi về [C/13-kien-truc-backbone/03-vit-va-inductive-bias](#); chi phí bộ nhớ và tính toán của attention theo số vị trí nằm ở [B/11-gioi-han-tinh-toan](#).

### Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao quá trình thuận *không có tham số* nào lại là điều kiện kỹ thuật khiến diffusion huấn luyện được ở quy mô lớn?
2. Cách nhìn “score function” giải thích được điều gì mà cách nhìn “khử nhiễu từng nấc” không giải thích được?
3. Latent diffusion tiết kiệm tính toán bằng cách nào, và bạn thiết kế phép thử ba mươi giây nào để biết chi tiết bị mất là do khâu nén hay do khâu khuếch tán?
4. Vì sao tăng hệ số dẫn hướng làm ảnh đẹp hơn nhưng làm tập ảnh nghèo hơn — và khi nào điều đó là nguy hiểm chứ không chỉ là đáng tiếc?
5. Tầng tích chập nổi của ControlNet được khởi tạo bằng 0. Bỏ chi tiết đó đi thì hỏng ở đâu, trong bước huấn luyện thứ mấy?
6. Theo Hình 6.8, nếu chỉ được giữ 10 trong 1000 bước, bạn đặt chúng ở đâu và vì sao?

- [1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [2] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [3] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *AISTATS*, 2010.
- [4] K. He, X. Zhang, S. Ren, et al., “Delving Deep into Rectifiers,” in *ICCV*, 2015.
- [5] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [6] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [7] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [8] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [9] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [10] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [11] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” in *ICLR*, 2017.
- [12] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [13] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [14] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [15] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [16] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [17] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [18] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [19] G. Huang, Y. Sun, Z. Liu, et al., “Deep Networks with Stochastic Depth,” in *ECCV*, 2016.
- [20] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [21] H. Zhang, M. Cisse, Y. N. Dauphin, et al., “mixup: Beyond Empirical Risk Minimization,” in *ICLR*, 2018.
- [22] S. Yun, D. Han, S. J. Oh, et al., “CutMix: Regularization Strategy to Train Strong Classifiers,” in *ICCV*, 2019.
- [23] E. D. Cubuk, B. Zoph, J. Shlens, et al., “RandAugment: Practical Automated Data Augmentation,” in *CVPR Workshops*, 2020.

- [24] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [25] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [26] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *NeurIPS*, 2012.
- [27] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in *ICLR*, 2015.
- [28] C. Szegedy, W. Liu, et al., “Going Deeper with Convolutions,” in *CVPR*, 2015.
- [29] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [30] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [31] G. Huang, Z. Liu, L. van der Maaten, et al., “Densely Connected Convolutional Networks,” in *CVPR*, 2017.
- [32] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [33] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [34] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [35] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [36] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [37] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.
- [38] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [39] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [40] J. Su, M. Ahmed, Y. Lu, et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, vol. 568, 2024.
- [41] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” *arXiv:2312.00752*, 2023.
- [42] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [43] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.
- [44] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [45] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [46] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [47] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [48] R. Girshick, J. Donahue, T. Darrell, et al., “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation,” in *CVPR*, 2014.

- [49] R. Girshick, "Fast R-CNN," in *ICCV*, 2015.
- [50] S. Ren, K. He, R. Girshick, et al., "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in *NeurIPS*, 2015.
- [51] T.-Y. Lin, P. Dollár, R. Girshick, et al., "Feature Pyramid Networks for Object Detection," in *CVPR*, 2017.
- [52] K. He, G. Gkioxari, P. Dollár, et al., "Mask R-CNN," in *ICCV*, 2017.
- [53] D. Bolya, S. Foley, J. Hays, et al., "TIDE: A General Toolbox for Identifying Object Detection Errors," in *ECCV*, 2020.
- [54] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [55] J. Redmon, S. Divvala, R. Girshick, et al., "You Only Look Once: Unified, Real-Time Object Detection," in *CVPR*, 2016.
- [56] W. Liu, D. Anguelov, D. Erhan, et al., "SSD: Single Shot MultiBox Detector," in *ECCV*, 2016.
- [57] T.-Y. Lin, P. Goyal, R. Girshick, et al., "Focal Loss for Dense Object Detection," in *ICCV*, 2017.
- [58] Z. Tian, C. Shen, H. Chen, et al., "FCOS: Fully Convolutional One-Stage Object Detection," in *ICCV*, 2019.
- [59] X. Zhou, D. Wang, and P. Krähenbühl, "Objects as Points," *arXiv:1904.07850*, 2019.
- [60] S. Zhang, C. Chi, Y. Yao, et al., "Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection," in *CVPR*, 2020.
- [61] Z. Ge, S. Liu, Z. Li, et al., "OTA: Optimal Transport Assignment for Object Detection," in *CVPR*, 2021.
- [62] N. Bodla, B. Singh, R. Chellappa, et al., "Soft-NMS: Improving Object Detection with One Line of Code," in *ICCV*, 2017.
- [63] X. Zhu, W. Su, L. Lu, et al., "Deformable DETR: Deformable Transformers for End-to-End Object Detection," in *ICLR*, 2021.
- [64] H. Zhang, F. Li, S. Liu, et al., "DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection," in *ICLR*, 2023.
- [65] J. Long, E. Shelhamer, and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," in *CVPR*, 2015.
- [66] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *MICCAI*, 2015.
- [67] L.-C. Chen, G. Papandreou, I. Kokkinos, et al., "DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs," *IEEE TPAMI*, vol. 40, no. 4, pp. 834–848, 2018.
- [68] E. Xie, W. Wang, Z. Yu, et al., "SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers," in *NeurIPS*, 2021.
- [69] B. Cheng, I. Misra, A. G. Schwing, et al., "Masked-Attention Mask Transformer for Universal Image Segmentation," in *CVPR*, 2022.
- [70] A. Kirillov, K. He, R. Girshick, et al., "Panoptic Segmentation," in *CVPR*, 2019.
- [71] A. Kirillov, E. Mintun, N. Ravi, et al., "Segment Anything," in *ICCV*, 2023.
- [72] K. Sun, B. Xiao, D. Liu, et al., "Deep High-Resolution Representation Learning for Human Pose Estimation," in *CVPR*, 2019.

- [73] Z. Cao, T. Simon, S.-E. Wei, et al., “Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields,” in *CVPR*, 2017.
- [74] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [75] J. Deng, J. Guo, N. Xue, et al., “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *CVPR*, 2019.
- [76] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [77] K. Xu, J. Ba, R. Kiros, et al., “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention,” in *ICML*, 2015.
- [78] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *ICLR*, 2015.
- [79] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [80] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [81] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [82] K. Simonyan and A. Zisserman, “Two-Stream Convolutional Networks for Action Recognition in Videos,” in *NeurIPS*, 2014.
- [83] J. Carreira and A. Zisserman, “Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset,” in *CVPR*, 2017.
- [84] D. Tran, H. Wang, L. Torresani, et al., “A Closer Look at Spatiotemporal Convolutions for Action Recognition,” in *CVPR*, 2018.
- [85] C. Feichtenhofer, H. Fan, J. Malik, et al., “SlowFast Networks for Video Recognition,” in *ICCV*, 2019.
- [86] A. Dosovitskiy, P. Fischer, E. Ilg, et al., “FlowNet: Learning Optical Flow with Convolutional Networks,” in *ICCV*, 2015.
- [87] D. Sun, X. Yang, M.-Y. Liu, et al., “PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume,” in *CVPR*, 2018.
- [88] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.
- [89] A. Bewley, Z. Ge, L. Ott, et al., “Simple Online and Realtime Tracking,” in *ICIP*, 2016.
- [90] Y. Zhang, P. Sun, Y. Jiang, et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *ECCV*, 2022.
- [91] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [92] K. Bernardin and R. Stiefelhagen, “Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics,” *EURASIP J. Image and Video Processing*, 2008.
- [93] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [94] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [95] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.

- [96] S. Wang, V. Leroy, Y. Cabon, et al., "DUST3R: Geometric 3D Vision Made Easy," in *CVPR*, 2024.
- [97] D. Eigen, C. Puhrsch, and R. Fergus, "Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network," in *NeurIPS*, 2014.
- [98] R. Ranftl, K. Lasinger, D. Hafner, et al., "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer," *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [99] J. Sturm, N. Engelhard, F. Endres, et al., "A Benchmark for the Evaluation of RGB-D SLAM Systems," in *IROS*, 2012.
- [100] C. R. Qi, H. Su, K. Mo, et al., "PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation," in *CVPR*, 2017.
- [101] C. R. Qi, L. Yi, H. Su, et al., "PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space," in *NeurIPS*, 2017.
- [102] A. H. Lang, S. Vora, H. Caesar, et al., "PointPillars: Fast Encoders for Object Detection from Point Clouds," in *CVPR*, 2019.
- [103] T. Müller, A. Evans, C. Schied, et al., "Instant Neural Graphics Primitives with a Multiresolution Hash Encoding," *ACM TOG*, vol. 41, no. 4, 2022.
- [104] B. Kerbl, G. Kopanas, T. Leimkühler, et al., "3D Gaussian Splatting for Real-Time Radiance Field Rendering," *ACM TOG*, vol. 42, no. 4, 2023.
- [105] R. Zhang, P. Isola, A. A. Efros, et al., "The Unreasonable Effectiveness of Deep Features as a Perceptual Metric," in *CVPR*, 2018.
- [106] D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," in *ICLR*, 2014.
- [107] I. Goodfellow, J. Pouget-Abadie, M. Mirza, et al., "Generative Adversarial Nets," in *NeurIPS*, 2014.
- [108] T. Karras, S. Laine, and T. Aila, "A Style-Based Generator Architecture for Generative Adversarial Networks," in *CVPR*, 2019.
- [109] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," in *NeurIPS*, 2020.
- [110] Y. Song, J. Sohl-Dickstein, D. P. Kingma, et al., "Score-Based Generative Modeling Through Stochastic Differential Equations," in *ICLR*, 2021.
- [111] J. Johnson, A. Alahi, and L. Fei-Fei, "Perceptual Losses for Real-Time Style Transfer and Super-Resolution," in *ECCV*, 2016.
- [112] P. Isola, J.-Y. Zhu, T. Zhou, et al., "Image-to-Image Translation with Conditional Adversarial Networks," in *CVPR*, 2017.
- [113] L. A. Gatys, A. S. Ecker, and M. Bethge, "Image Style Transfer Using Convolutional Neural Networks," in *CVPR*, 2016.
- [114] X. Huang and S. Belongie, "Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization," in *ICCV*, 2017.
- [115] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," in *NeurIPS Deep Learning Workshop*, 2015.
- [116] J. Song, C. Meng, and S. Ermon, "Denoising Diffusion Implicit Models," in *ICLR*, 2021.
- [117] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, et al., "Flow Matching for Generative Modeling," in *ICLR*, 2023.
- [118] J. Ho and T. Salimans, "Classifier-Free Diffusion Guidance," *arXiv:2207.12598*, 2022.
- [119] R. Rombach, A. Blattmann, D. Lorenz, et al., "High-Resolution Image Synthesis with Latent Diffusion Models," in *CVPR*, 2022.
- [120] W. Peebles and S. Xie, "Scalable Diffusion Models with Transformers," in *ICCV*, 2023.



- [121] L. Zhang, A. Rao, and M. Agrawala, “Adding Conditional Control to Text-to-Image Diffusion Models,” in *ICCV*, 2023.
- [122] Y. Blau and T. Michaeli, “The Perception-Distortion Tradeoff,” in *CVPR*, 2018.
- [123] C. Ledig, L. Theis, F. Huszár, et al., “Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network,” in *CVPR*, 2017.
- [124] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., “Image Quality Assessment: From Error Visibility to Structural Similarity,” *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [125] M. Heusel, H. Ramsauer, T. Unterthiner, et al., “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium,” in *NeurIPS*, 2017.
- [126] T. Brooks, B. Mildenhall, T. Xue, et al., “Unprocessing Images for Learned Raw Denoising,” in *CVPR*, 2019.
- [127] A. Abdelhamed, S. Lin, and M. S. Brown, “A High-Quality Denoising Dataset for Smartphone Cameras,” in *CVPR*, 2018.
- [128] J. Deng, W. Dong, R. Socher, et al., “ImageNet: A Large-Scale Hierarchical Image Database,” in *CVPR*, 2009.
- [129] A. Radford, J. W. Kim, C. Hallacy, et al., “Learning Transferable Visual Models from Natural Language Supervision,” in *ICML*, 2021.
- [130] J. Li, D. Li, S. Savarese, et al., “BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models,” in *ICML*, 2023.
- [131] H. Liu, C. Li, Q. Wu, et al., “Visual Instruction Tuning,” in *NeurIPS*, 2023.
- [132] S. Kornblith, J. Shlens, and Q. V. Le, “Do Better ImageNet Models Transfer Better?” In *CVPR*, 2019.
- [133] T. Chen, S. Kornblith, M. Norouzi, et al., “A Simple Framework for Contrastive Learning of Visual Representations,” in *ICML*, 2020.
- [134] K. He, H. Fan, Y. Wu, et al., “Momentum Contrast for Unsupervised Visual Representation Learning,” in *CVPR*, 2020.
- [135] J.-B. Grill, F. Strub, F. Altché, et al., “Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning,” in *NeurIPS*, 2020.
- [136] M. Caron, H. Touvron, I. Misra, et al., “Emerging Properties in Self-Supervised Vision Transformers,” in *ICCV*, 2021.
- [137] K. He, X. Chen, S. Xie, et al., “Masked Autoencoders Are Scalable Vision Learners,” in *CVPR*, 2022.
- [138] A. van den Oord, Y. Li, and O. Vinyals, “Representation Learning with Contrastive Predictive Coding,” in *arXiv:1807.03748*, 2018.
- [139] J. Kaplan, S. McCandlish, T. Henighan, et al., “Scaling Laws for Neural Language Models,” *arXiv:2001.08361*, 2020.
- [140] X. Zhai, A. Kolesnikov, N. Houlsby, et al., “Scaling Vision Transformers,” in *CVPR*, 2022.
- [141] J. Hoffmann, S. Borgeaud, A. Mensch, et al., “Training Compute-Optimal Large Language Models,” in *NeurIPS*, 2022.
- [142] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [143] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [144] N. Houlsby, A. Giurgiu, S. Jastrzebski, et al., “Parameter-Efficient Transfer Learning for NLP,” in *ICML*, 2019.

- [145] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [146] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *RecSys*, 2019.

## 6.4 Low-level vision và trade-off không thể tránh (Low-Level Vision)

**PHỤ THUỘC** C/17-mo-hinh-sinh/01-bon-ho-mo-hinh-sinh (vì sao loss trung bình cho ảnh mờ); A/02-vat-ly-tao-anh/01-anh-sang-va-nhieu (nhiều cảm biến thật khác nhiều Gauss thế nào); B/07-chia-du-lieu-va-danh-gia (chọn chỉ số là chọn kết luận).

### TÓM TẮT

Khử nhiễu, siêu phân giải, khử nhoè và HDR đều là bài toán *ill-posed*: nhiều ảnh đầu ra khác nhau cùng khớp với đầu vào, nên vai trò thật của mô hình là *chọn một* chứ không phải *tìm ra đáp án*. Từ đó suy ra một kết quả có chứng minh: không thể đồng thời tối ưu sai lệch theo pixel và độ chân thực thị giác. Vậy nên mọi tranh cãi “phương pháp nào tốt hơn” thật ra là tranh cãi về việc chọn điểm nào trên đường đánh đổi. Đọc xong bạn đọc được một bảng kết quả khôi phục ảnh mà không bị con số dẫn dắt, và biết khi nào một mô hình “đẹp hơn” thực chất đang bịa. Nếu chỉ nhớ một điều: tối ưu PSNR cho ảnh mờ nhưng an toàn, còn tối ưu độ chân thực cho ảnh sắc nét nhưng có thể thêm chi tiết không hề có trong đầu vào. Trong ảnh y tế, giám định hay đo đạc, đó là lỗi đúng sai chứ không phải chuyện thẩm mỹ.

### KIẾN THỨC CẦN CÓ TRƯỚC

- **low-level vision** — nhóm bài toán biến ảnh thành ảnh mà không cần hiểu nội dung: khử nhiễu, tăng độ phân giải, khử nhoè, ghép dải sáng động. Đối lập với high-level vision (phân loại, phát hiện) vốn biến ảnh thành nhân.
- **ill-posed (đặt không chỉnh)** — bài toán mà nghiệm không tồn tại duy nhất hoặc không phụ thuộc liên tục vào dữ liệu; ở đây luôn là trường hợp “nhiều nghiệm cùng hợp lệ”, vì phép làm hỏng ảnh đã xoá mất thông tin.
- **toán tử thuận (forward operator)** — mô hình toán của quá trình làm hỏng ảnh — lấy trung bình rồi lấy mẫu thưa (siêu phân giải), chụp với nhân nhoè (khử nhoè), cộng nhiễu (khử nhiễu). Biết nó là biết mình đang phải đảo ngược cái gì.
- **prior (prior)** — giả định về “ảnh thật trông thế nào” dùng để chọn một nghiệm trong vô số nghiệm hợp lệ; ngày nay prior thường nằm ẩn trong trọng số của một mạng đã học.
- **PSNR** — sai lệch theo pixel quy ra decibel, suy ra trực tiếp từ bình phương trung bình; đo “gần theo nghĩa số học”.
- **SSIM** — chỉ số so sánh độ sáng, tương phản và cấu trúc cục bộ giữa hai ảnh; gần thị giác hơn PSNR nhưng vẫn thuộc nhóm đo sai lệch theo cặp.
- **LPIPS** — khoảng cách giữa hai ảnh đo trong không gian đặc trưng của một mạng đã huấn luyện; bám thị giác người tốt hơn, nhưng cần có ảnh gốc để so cặp.
- **FID** — khoảng cách giữa *phân phối* ảnh sinh và phân phối ảnh thật. Nó khớp một Gauss trên đặc trưng của một mạng, cần hàng vạn mẫu, và không nói gì về từng ảnh riêng lẻ.

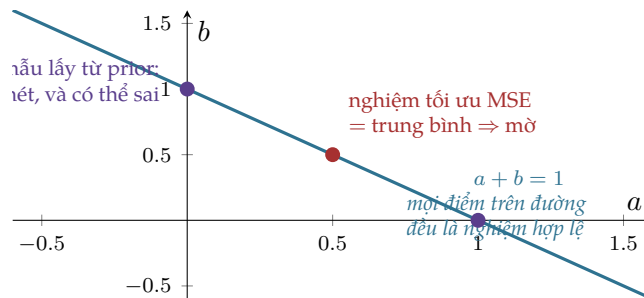
### A. Đặt vấn đề

Mọi bài toán trong mục này có cùng một hình dạng: một toán tử thuận đã xoá bớt thông tin, và ta phải đi ngược. Điều quan trọng là thông tin đã mất thì *mất thật* — không thuật toán nào lấy lại được, chỉ có thể đoán.

**Ví dụ nhỏ nhất, tính tay được.** Lấy tín hiệu một chiều và giảm độ phân giải hai lần bằng cách lấy trung bình từng cặp: hai giá trị  $a, b$  cho ra một giá trị  $m = (a + b)/2$ . Bây giờ đi ngược: biết  $m = 0,5$ , hãy tìm  $a$  và  $b$ . Một phương trình, hai ẩn — nghiệm là cả một đường thẳng  $\{(a, 1 - a)\}$ . Cặp  $(0,5; 0,5)$  hợp lệ, mà  $(0; 1)$  cũng hợp lệ, và  $(-3; 4)$  cũng vậy. Với siêu phân giải  $\times 4$  trên ảnh, mỗi pixel thấp cấp ứng với 16 pixel cao cấp: một ràng buộc, mười sáu ẩn. Mô hình không giải bài toán này; nó chọn một điểm trên không gian nghiệm, và cái quyết định điểm nào là prior.

→ cùng nguyên lý với C/16/01 — thị giác 3D đối mặt đúng tình thế này: khi ràng buộc hình học không đủ, prior học được chọn nghiệm — và cũng mất luôn khả năng tự phát hiện sai.

Hình 6.11 vẽ đúng không gian nghiệm đó.



**Hình 6.11.** Bài toán ill-posed nhỏ nhất có thể vẽ được. Giảm độ phân giải hai lần bằng phép lấy trung bình biến  $(a, b)$  thành một số  $m = 0,5$ ; đi ngược thì một phương trình phải xác định hai ẩn, nên nghiệm là cả một đường thẳng. Mô hình không “giải” bài toán này — nó chọn một điểm trên đường. Chọn trọng tâm thì được điểm số sai lệch tốt nhất và được ảnh mờ; chọn một điểm lấy từ phân phối ảnh thật thì được ảnh sắc nét và chấp nhận rủi ro chọn nhầm. Siêu phân giải  $\times 4$  trên ảnh chỉ là hình này với một ràng buộc và mười sáu ẩn.

Điều đó dẫn thẳng tới câu hỏi trung tâm của cả mục: chọn điểm nào? Nếu chọn trung bình của mọi nghiệm hợp lệ thì được điểm số sai lệch tốt nhất, nhưng trung bình của nhiều ảnh sắc nét lệch nhau là một ảnh mờ — đúng cơ chế đã gặp ở mục 17.1.

→ hệ quả của C/17/01 — ảnh mờ ở đây và ảnh mờ của VAE là cùng một phép tính: nghiệm tối ưu của bình phương trung bình là kỳ vọng có điều kiện.

Nếu chọn một nghiệm cụ thể lấy từ phân phối ảnh thật thì được ảnh sắc nét trông rất thật, nhưng nó có thể khác hẳn ảnh gốc ở chi tiết. Đây không phải một lựa chọn kỹ thuật mà là một đánh đổi có chứng minh.

**Học mục này thì làm được gì mà trước đó không làm được?** Đọc một bảng so sánh phương pháp khôi phục ảnh và biết ngay vì sao phương pháp có PSNR cao nhất lại trông tệ nhất; chọn chỉ số đánh giá phù hợp với việc mình định làm; và quyết định có được phép dùng ảnh sinh làm dữ liệu huấn luyện hay không, dựa trên lý do chứ không dựa trên cảm giác.

#### TRỰC GIÁC

Khôi phục ảnh giống việc đọc một lá thư bị nhoè mực. Chỗ nhoè có thể là chữ “hoa” hoặc chữ “hòa” — nét mực không còn phân biệt được nữa. Bạn có hai cách. Cách thứ nhất: viết chồng cả hai khả năng lên nhau thành một vệt mờ; ai đọc cũng thấy chỗ đó không rõ, nên không ai bị lừa. Cách thứ hai: chọn dứt khoát một chữ và viết thật đẹp; lá thư đọc trôi chảy, trông như nguyên bản — và người đọc không có cách nào biết chữ đó là bạn đoán. Tối ưu PSNR là cách thứ nhất; tối ưu độ chân thực là cách thứ hai. Không có cách thứ ba.

## B. Bốn bài toán, một hình dạng

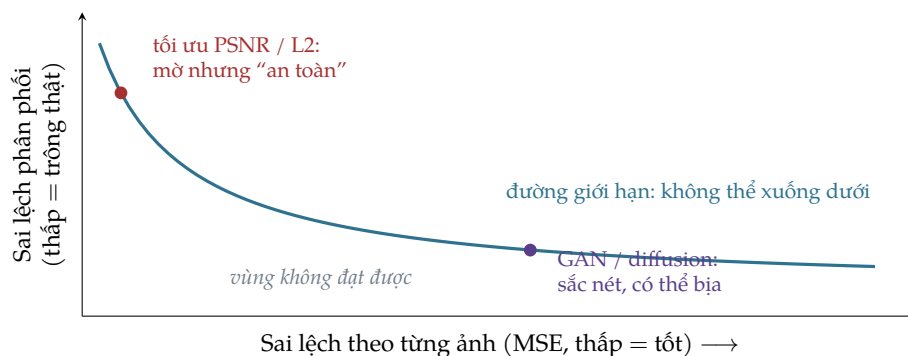
**Bảng 6.5.** Bốn bài toán low-level vision, đọc theo cái mà toán tử thuận đã xóa đi. Cột cuối cho biết prior phải bù vào chỗ nào — và do đó mô hình sẽ bịa ra loại nội dung gì khi nó sai.

| Bài toán       | Toán tử thuận                                                      | Cái bị xóa                                 | Prior phải bù gì / hỏng khi                                                               |
|----------------|--------------------------------------------------------------------|--------------------------------------------|-------------------------------------------------------------------------------------------|
| Khử nhiễu      | Cộng nhiễu (thực tế phụ thuộc cường độ sáng, không phải Gauss đều) | Chi tiết biên độ nhỏ, lẫn vào nhiễu        | Kết cấu tự nhiên; hỏng khi mô hình nhiễu lúc huấn luyện khác nhiễu thật                   |
| Siêu phân giải | Làm mờ rồi lấy mẫu thưa                                            | Toàn bộ tần số cao trên ngưỡng Nyquist     | Chi tiết tần số cao; hỏng ở chữ, khuôn mặt, biển số — những chỗ sai một chút là sai nghĩa |
| Khử nhoè       | Chập với nhân nhoè (do chuyển động hoặc lệch nét)                  | Các tần số bị nhân nhoè triệt tiêu         | Cấu trúc biên; hỏng khi nhân nhoè không đều trên ảnh hoặc không biết trước                |
| HDR            | Cắt ngưỡng trên và dưới của dải sáng                               | Thông tin ở vùng cháy sáng và vùng tối kịt | Nội dung vùng đã cháy; hỏng vì ở đó dữ liệu <i>bằng không</i> , không phải “nhiều”        |

Dòng cuối đáng chú ý vì nó là trường hợp cực đoan giúp nhìn rõ bản chất: ở vùng cháy sáng, cảm biến đã bão hoà nên mọi giá trị thật đều cho ra cùng một con số. Cái mô hình điền vào đó *hoàn toàn* là prior, không có một chút thông tin đo đạc nào. Ba bài toán còn lại chỉ là phiên bản nhẹ hơn của cùng tình huống.

## C. Đánh đổi perceptual-distortion: một kết quả, không phải một kinh nghiệm

Đây là chỗ cần phân biệt rõ địa vị của các phát biểu. Trong mục 17.1, “thể lưỡng nan ba chiều” là một cách gọi tên quan sát thực nghiệm. Ở đây thì khác. Trước hết phải hình thức hoá hai đại lượng: *sai lệch* là kỳ vọng của một khoảng cách tính theo từng ảnh, còn *độ chân thực* là khoảng cách giữa phân phối ảnh khôi phục và phân phối ảnh thật. Với hai định nghĩa đó, người ta *chứng minh được* rằng hàm biểu diễn giới hạn giữa chúng là không tăng [122]. Nói bằng lời: muốn giảm sai lệch xuống dưới một mức nào đó thì buộc phải chấp nhận độ chân thực kém đi. Hình 6.12 vẽ tình thế đó.



**Hình 6.12.** Đánh đổi perceptual-distortion. Mọi phương pháp khôi phục ảnh đều nằm *trên* đường cong này; tiến bộ thuật toán đẩy đường cong xuống, còn việc chọn hàm mất mát chỉ là chọn *trượt dọc* theo nó. Hệ quả trực tiếp: không xếp hạng được các phương pháp bằng một con số duy nhất, và hai phương pháp ở hai đầu đường cong không so sánh được với nhau nếu chưa nói rõ mục đích sử dụng.

Hệ quả thực hành nghiêm túc hơn về ngoài của nó. Một mô hình siêu phân giải kiểu GAN hay diffusion [123] có thể dựng ra một khuôn mặt sắc nét, hợp lý và *sai người*, hoặc một biển số đọc

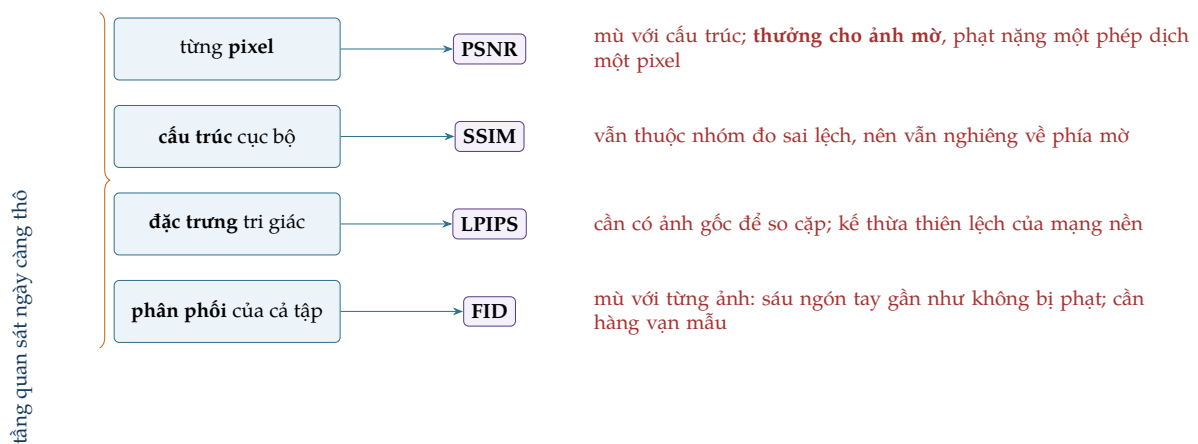
được và sai số. Trong ảnh giải trí, đó là tính năng. Trong ảnh y tế, giám định pháp y, đo đạc từ ảnh vệ tinh hay kiểm tra khuyết tật công nghiệp, đó là lỗi đúng sai. Nó còn nguy hiểm gấp bội vì đầu ra trông *đáng tin hơn* đầu vào.

#### D. Đánh giá: mỗi chỉ số nhạy với một thứ, và mù với thứ khác

**Bảng 6.6.** Bốn chỉ số hay dùng. Không chỉ số nào đủ một mình; chọn chỉ số là chọn trước loại lỗi mà bạn sẽ không nhìn thấy.

| Chỉ số | Đo gì                                                 | Nhạy với                                           | Mù với                                                                      |
|--------|-------------------------------------------------------|----------------------------------------------------|-----------------------------------------------------------------------------|
| PSNR   | Bình phương trung bình theo pixel                     | Lệch sáng, lệch màu toàn cục, sai lệch biên độ lớn | Cấu trúc; nó <i>thường</i> cho ảnh mờ, và phạt nặng một phép dịch một pixel |
| SSIM   | Tương quan cục bộ về sáng, tương phản, cấu trúc [124] | Mất cấu trúc cục bộ, nhoè                          | Sai lệch ngữ nghĩa; vẫn thuộc nhóm đo sai lệch nên vẫn nghiêng về ảnh mờ    |
| LPIPS  | Khoảng cách trong không gian đặc trưng học được [105] | Khác biệt mà mắt người thấy: kết cấu, nét          | Cần ảnh gốc; kế thừa thiên lệch của mạng nền                                |
| FID    | Khoảng cách giữa hai <i>phân phối</i> đặc trưng [125] | Lệch thống kê tổng thể, thu hẹp phân phối          | Từng ảnh riêng lẻ; lỗi ngữ nghĩa (sáu ngón tay) gần như không bị phạt       |

Hình 6.13 xếp bốn chỉ số theo tầng mà chúng quan sát, kèm chỗ mù của từng cái.



**Hình 6.13.** Bốn chỉ số, bốn *tầng quan sát* khác nhau — và đó là lý do không chỉ số nào thay thế được chỉ số khác. Đi từ trên xuống, thước đo ngày càng ít quan tâm tới từng pixel và ngày càng quan tâm tới thống kê tổng thể; cột phải là cái mà mỗi tầng *không nhìn thấy*. Chọn chỉ số vì thế là chọn trước loại lỗi mà bạn sẽ để lọt, nên báo cáo tối thiểu phải có một chỉ số ở nhóm trên và một ở nhóm dưới.

Hai điểm thực hành ít được nói. Thứ nhất, FID cần cỡ hàng vạn mẫu mới ổn định. Giá trị của nó còn phụ thuộc mạnh vào chi tiết tiền xử lý: thư viện dùng để đổi kích thước ảnh, và chất lượng nén JPEG. Hệ quả là hai con số FID từ hai bài báo thường không so sánh trực tiếp được. Đó là lý do tồn tại các bản cài đặt chuẩn hoá như [GaParmar/clean-fid](#). Thứ hai, và quan trọng hơn: FID nhạy với việc *phân phối bị thu hẹp* — đúng thứ mà mắt người không phát hiện được khi nhìn từng ảnh. Đó là lý do nó vẫn là chỉ số chính dù mọi người đều biết nó có nhiều khuyết điểm.

### E. Nhiều thật không phải nhiều Gauss

Một chế độ hỏng rất phổ biến, và hoàn toàn tránh được nếu biết trước. Gần như mọi mô hình khử nhiễu trong tài liệu được huấn luyện bằng cách cộng nhiễu Gauss trắng vào ảnh sạch, vì làm vậy thì có dữ liệu vô hạn miễn phí. Nhưng nhiễu của cảm biến thật là hỗn hợp của nhiễu Poisson phụ thuộc cường độ sáng và nhiễu đọc. Bộ xử lý ảnh trong máy còn biến đổi nó một cách phi tuyến qua khử khảm, cân bằng trắng, gamma và nén. Sau chuỗi đó, nhiễu trở nên có tương quan không gian và phụ thuộc màu [126]. Mô hình huấn luyện trên nhiễu Gauss đạt điểm rất cao trên bộ kiểm thử tổng hợp và tụt hẳn trên ảnh chụp thật [127].

Đây là một ca sách giáo khoa của lệch phân phối (B/08-dịch-chuyen-phan-phoi), và cách chữa cũng là cách chữa chuẩn: hoặc thu dữ liệu ghép cặp thật, hoặc mô phỏng đúng chuỗi vật lý bằng cách “gỡ ngược” quá trình xử lý ảnh về miền raw, thêm nhiễu đúng mô hình ở đó, rồi chạy xuôi trở lại. Chi tiết vật lý nằm ở A/02-vat-ly-tao-anh.

→ **ràng buộc bởi** A/02 — mô hình nhiễu dùng lúc huấn luyện phải khớp chuỗi vật lý thật của cảm biến, nếu không thì mọi con số đo được đều vô nghĩa.

→ **trường hợp riêng của** B/08 — nhiễu Gauss tổng hợp đối lại nhiễu cảm biến thật là một ca sách giáo khoa của lệch phân phối.

### F. Dùng ảnh sinh làm dữ liệu huấn luyện

Đây là vấn đề mới của thời đại này, và câu trả lời không phải “có” hay “không” mà là một điều kiện.

- **Có ích khi** bạn dùng mô hình sinh để bù vào các vùng *hiếm* mà bạn biết trước là hiếm: tư thế lạ, thời tiết hiếm, khuyết tật ít gặp. Điều kiện đi kèm là bạn giữ được một tập kiểm thử *thật* hoàn toàn tách biệt để đo. Ở đây ảnh sinh đóng vai một dạng tăng cường dữ liệu có kiểm soát, và tín hiệu đánh giá vẫn đến từ thế giới thật.
- **Tự đầu độc khi** vòng lặp được lặp lại: mô hình sinh dữ liệu, dữ liệu huấn luyện mô hình sau, mô hình sau lại sinh dữ liệu. Cơ chế thu hẹp rất cụ thể. Mọi thao tác làm ảnh “đẹp hơn” — dẫn hướng mạnh, cắt ngưỡng, chọn lọc thủ công — đều cắt bớt đuôi của phân phối. Mỗi vòng lặp vì thế co phương sai lại một chút, và các mode hiếm biến mất hẳn. Hiện tượng này đã được chứng minh bằng thực nghiệm trong nhiều công trình gần đây; mức độ nghiêm trọng phụ thuộc rất mạnh vào một điều kiện: mỗi vòng có giữ lại dữ liệu thật hay không. Điểm này đáng nói rõ, vì các phát biểu kiểu “mô hình sẽ tự sụp đổ” thường bỏ qua nó.

### G. Giới hạn và trường hợp hỏng

Giới hạn lớn nhất của cả mảng này là ta không có cách đánh giá tốt. Đường cong ở Hình 6.12 nói rằng một con số không đủ về mặt nguyên tắc, chứ không phải vì ta chưa nghĩ ra chỉ số hay hơn. Hệ quả là mọi bảng xếp hạng khôi phục ảnh đều mang một giả định ngầm về mục đích sử dụng, và giả định đó hầu như không bao giờ được viết ra.

Ba chế độ hỏng còn lại đáng nhớ: mô hình khử nhiễu huấn luyện ở một mức nhiễu cố định hỏng ở mức khác; mô hình siêu phân giải huấn luyện với một toán tử làm mờ cố định (thường là bicubic) hỏng ngay khi ảnh thật bị làm hỏng theo cách khác; và mọi mô hình trong nhóm này đều hỏng *im lặng* — đầu ra luôn là một ảnh trông hợp lý, không bao giờ là một thông báo “tôi không biết”.

#### CẠM BÃY

Cạm bẫy phổ biến nhất là dùng PSNR để chọn mô hình rồi ngạc nhiên vì mô hình được chọn trông tệ. Đây không phải trực giác mà là hành vi đúng của chỉ số: PSNR suy ra từ bình



phương trung bình, và nghiệm tối ưu của bình phương trung bình là trung bình của mọi ảnh hợp lệ, tức là ảnh mờ. *Chọn chỉ số là chọn trước kết luận.* Cách làm tối thiểu chấp nhận được có hai phần. Thứ nhất, báo cáo một chỉ số sai lệch và một chỉ số chân thực, tức là một ở mỗi đầu đường cong. Thứ hai, với ứng dụng có rủi ro, thêm một phép kiểm tra rằng mô hình *không* thêm nội dung. Phép thử rẻ nhất: cho đầu ra chạy lại qua toán tử thuận rồi so với đầu vào. Chỗ nào lệch là chỗ mô hình đã bịa.

#### CẦN NHỚ

Mọi bài toán low-level vision đều ill-posed, nên mô hình không “khôi phục” mà *chọn một nghiệm*, và prior là thứ quyết định chọn cái nào. Từ đó suy ra đánh đổi perceptual-distortion, vốn là một kết quả có chứng minh chứ không phải kinh nghiệm: sắc nét hơn đồng nghĩa với bịa nhiều hơn. Câu hỏi đúng khi chọn mô hình không phải “cái nào tốt hơn” mà “sai kiểu nào thì tôi chịu được”. [T1]

## H. Kết nối

Cơ chế “loss trung bình cho ảnh mờ” ở đây là đúng cơ chế của VAE trong [C/17-mo-hinh-sinh/01-bon-ho-mo-hinh-sinh](#); các mô hình khôi phục hiện đại nhất chính là mô hình diffusion có điều kiện của [C/17-mo-hinh-sinh/03-diffusion](#), nên hệ số dẫn hướng ở đó cũng chính là nùm vặn trượt dọc đường cong ở đây. Vai trò của prior trong bài toán ill-posed là bản sao chính xác của vai trò prior trong [C/16-thi-giac-3d/01-bon-the-he-3d](#): cả hai lĩnh vực đều đòi ràng buộc cứng lấy tri thức học được, và đều mất khả năng tự phát hiện sai. Mô hình nhiễu thật nằm ở [A/02-vat-ly-tao-anh/01-anh-sang-va-nhieu](#); việc chọn chỉ số và giao thức đo nằm ở [B/07-chia-du-lieu-va-danh-gia](#); hậu quả vận hành của “hông im lặng” nằm ở [E/25-do-tin-cay-va-van-hanh](#).

### Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao một mô hình tối ưu MSE cho ảnh mờ, và vì sao đó là *ng nghiệm đúng* chứ không phải mô hình kém?
2. Đánh đổi perceptual-distortion khác về địa vị thế nào so với “thể lưỡng nan ba chiều” ở mục 17.1, và vì sao sự khác biệt đó quan trọng khi đọc paper?
3. FID nhạy với thứ gì mà mắt người không để ý, và mù với thứ gì mà mắt người nhận ra ngay?
4. Một mô hình khử nhiễu đạt kết quả xuất sắc trên bộ kiểm thử tổng hợp và kém hẳn trên ảnh điện thoại; cơ chế nào giải thích điều đó và bạn sửa bằng cách nào?
5. Trong trường hợp nào thì việc dùng ảnh sinh làm dữ liệu huấn luyện là an toàn, và điều kiện nào bị vi phạm thì nó trở thành tự đầu độc?
6. Nếu phải giao một mô hình siêu phân giải cho một hệ thống đọc biển số, bạn chọn điểm nào trên đường cong ở Hình 6.12, và bạn thêm phép kiểm tra nào để bảo vệ mình?

- [1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [2] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [3] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *AISTATS*, 2010.
- [4] K. He, X. Zhang, S. Ren, et al., “Delving Deep into Rectifiers,” in *ICCV*, 2015.
- [5] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [6] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [7] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [8] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [9] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [10] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [11] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” in *ICLR*, 2017.
- [12] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [13] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [14] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [15] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [16] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [17] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [18] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [19] G. Huang, Y. Sun, Z. Liu, et al., “Deep Networks with Stochastic Depth,” in *ECCV*, 2016.
- [20] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [21] H. Zhang, M. Cisse, Y. N. Dauphin, et al., “mixup: Beyond Empirical Risk Minimization,” in *ICLR*, 2018.
- [22] S. Yun, D. Han, S. J. Oh, et al., “CutMix: Regularization Strategy to Train Strong Classifiers,” in *ICCV*, 2019.
- [23] E. D. Cubuk, B. Zoph, J. Shlens, et al., “RandAugment: Practical Automated Data Augmentation,” in *CVPR Workshops*, 2020.

- [24] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [25] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [26] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *NeurIPS*, 2012.
- [27] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in *ICLR*, 2015.
- [28] C. Szegedy, W. Liu, et al., “Going Deeper with Convolutions,” in *CVPR*, 2015.
- [29] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [30] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [31] G. Huang, Z. Liu, L. van der Maaten, et al., “Densely Connected Convolutional Networks,” in *CVPR*, 2017.
- [32] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [33] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [34] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [35] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [36] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [37] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.
- [38] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [39] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [40] J. Su, M. Ahmed, Y. Lu, et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, vol. 568, 2024.
- [41] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” *arXiv:2312.00752*, 2023.
- [42] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [43] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.
- [44] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [45] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [46] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [47] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [48] R. Girshick, J. Donahue, T. Darrell, et al., “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation,” in *CVPR*, 2014.

- [49] R. Girshick, "Fast R-CNN," in *ICCV*, 2015.
- [50] S. Ren, K. He, R. Girshick, et al., "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in *NeurIPS*, 2015.
- [51] T.-Y. Lin, P. Dollár, R. Girshick, et al., "Feature Pyramid Networks for Object Detection," in *CVPR*, 2017.
- [52] K. He, G. Gkioxari, P. Dollár, et al., "Mask R-CNN," in *ICCV*, 2017.
- [53] D. Bolya, S. Foley, J. Hays, et al., "TIDE: A General Toolbox for Identifying Object Detection Errors," in *ECCV*, 2020.
- [54] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [55] J. Redmon, S. Divvala, R. Girshick, et al., "You Only Look Once: Unified, Real-Time Object Detection," in *CVPR*, 2016.
- [56] W. Liu, D. Anguelov, D. Erhan, et al., "SSD: Single Shot MultiBox Detector," in *ECCV*, 2016.
- [57] T.-Y. Lin, P. Goyal, R. Girshick, et al., "Focal Loss for Dense Object Detection," in *ICCV*, 2017.
- [58] Z. Tian, C. Shen, H. Chen, et al., "FCOS: Fully Convolutional One-Stage Object Detection," in *ICCV*, 2019.
- [59] X. Zhou, D. Wang, and P. Krähenbühl, "Objects as Points," *arXiv:1904.07850*, 2019.
- [60] S. Zhang, C. Chi, Y. Yao, et al., "Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection," in *CVPR*, 2020.
- [61] Z. Ge, S. Liu, Z. Li, et al., "OTA: Optimal Transport Assignment for Object Detection," in *CVPR*, 2021.
- [62] N. Bodla, B. Singh, R. Chellappa, et al., "Soft-NMS: Improving Object Detection with One Line of Code," in *ICCV*, 2017.
- [63] X. Zhu, W. Su, L. Lu, et al., "Deformable DETR: Deformable Transformers for End-to-End Object Detection," in *ICLR*, 2021.
- [64] H. Zhang, F. Li, S. Liu, et al., "DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection," in *ICLR*, 2023.
- [65] J. Long, E. Shelhamer, and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," in *CVPR*, 2015.
- [66] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *MICCAI*, 2015.
- [67] L.-C. Chen, G. Papandreou, I. Kokkinos, et al., "DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs," *IEEE TPAMI*, vol. 40, no. 4, pp. 834–848, 2018.
- [68] E. Xie, W. Wang, Z. Yu, et al., "SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers," in *NeurIPS*, 2021.
- [69] B. Cheng, I. Misra, A. G. Schwing, et al., "Masked-Attention Mask Transformer for Universal Image Segmentation," in *CVPR*, 2022.
- [70] A. Kirillov, K. He, R. Girshick, et al., "Panoptic Segmentation," in *CVPR*, 2019.
- [71] A. Kirillov, E. Mintun, N. Ravi, et al., "Segment Anything," in *ICCV*, 2023.
- [72] K. Sun, B. Xiao, D. Liu, et al., "Deep High-Resolution Representation Learning for Human Pose Estimation," in *CVPR*, 2019.

- [73] Z. Cao, T. Simon, S.-E. Wei, et al., “Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields,” in *CVPR*, 2017.
- [74] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [75] J. Deng, J. Guo, N. Xue, et al., “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *CVPR*, 2019.
- [76] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [77] K. Xu, J. Ba, R. Kiros, et al., “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention,” in *ICML*, 2015.
- [78] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *ICLR*, 2015.
- [79] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [80] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [81] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [82] K. Simonyan and A. Zisserman, “Two-Stream Convolutional Networks for Action Recognition in Videos,” in *NeurIPS*, 2014.
- [83] J. Carreira and A. Zisserman, “Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset,” in *CVPR*, 2017.
- [84] D. Tran, H. Wang, L. Torresani, et al., “A Closer Look at Spatiotemporal Convolutions for Action Recognition,” in *CVPR*, 2018.
- [85] C. Feichtenhofer, H. Fan, J. Malik, et al., “SlowFast Networks for Video Recognition,” in *ICCV*, 2019.
- [86] A. Dosovitskiy, P. Fischer, E. Ilg, et al., “FlowNet: Learning Optical Flow with Convolutional Networks,” in *ICCV*, 2015.
- [87] D. Sun, X. Yang, M.-Y. Liu, et al., “PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume,” in *CVPR*, 2018.
- [88] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.
- [89] A. Bewley, Z. Ge, L. Ott, et al., “Simple Online and Realtime Tracking,” in *ICIP*, 2016.
- [90] Y. Zhang, P. Sun, Y. Jiang, et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *ECCV*, 2022.
- [91] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [92] K. Bernardin and R. Stiefelwagen, “Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics,” *EURASIP J. Image and Video Processing*, 2008.
- [93] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [94] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [95] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.



- [96] S. Wang, V. Leroy, Y. Cabon, et al., "DUST3R: Geometric 3D Vision Made Easy," in *CVPR*, 2024.
- [97] D. Eigen, C. Puhrsch, and R. Fergus, "Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network," in *NeurIPS*, 2014.
- [98] R. Ranftl, K. Lasinger, D. Hafner, et al., "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer," *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [99] J. Sturm, N. Engelhard, F. Endres, et al., "A Benchmark for the Evaluation of RGB-D SLAM Systems," in *IROS*, 2012.
- [100] C. R. Qi, H. Su, K. Mo, et al., "PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation," in *CVPR*, 2017.
- [101] C. R. Qi, L. Yi, H. Su, et al., "PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space," in *NeurIPS*, 2017.
- [102] A. H. Lang, S. Vora, H. Caesar, et al., "PointPillars: Fast Encoders for Object Detection from Point Clouds," in *CVPR*, 2019.
- [103] T. Müller, A. Evans, C. Schied, et al., "Instant Neural Graphics Primitives with a Multiresolution Hash Encoding," *ACM TOG*, vol. 41, no. 4, 2022.
- [104] B. Kerbl, G. Kopanas, T. Leimkühler, et al., "3D Gaussian Splatting for Real-Time Radiance Field Rendering," *ACM TOG*, vol. 42, no. 4, 2023.
- [105] R. Zhang, P. Isola, A. A. Efros, et al., "The Unreasonable Effectiveness of Deep Features as a Perceptual Metric," in *CVPR*, 2018.
- [106] D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," in *ICLR*, 2014.
- [107] I. Goodfellow, J. Pouget-Abadie, M. Mirza, et al., "Generative Adversarial Nets," in *NeurIPS*, 2014.
- [108] T. Karras, S. Laine, and T. Aila, "A Style-Based Generator Architecture for Generative Adversarial Networks," in *CVPR*, 2019.
- [109] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," in *NeurIPS*, 2020.
- [110] Y. Song, J. Sohl-Dickstein, D. P. Kingma, et al., "Score-Based Generative Modeling Through Stochastic Differential Equations," in *ICLR*, 2021.
- [111] J. Johnson, A. Alahi, and L. Fei-Fei, "Perceptual Losses for Real-Time Style Transfer and Super-Resolution," in *ECCV*, 2016.
- [112] P. Isola, J.-Y. Zhu, T. Zhou, et al., "Image-to-Image Translation with Conditional Adversarial Networks," in *CVPR*, 2017.
- [113] L. A. Gatys, A. S. Ecker, and M. Bethge, "Image Style Transfer Using Convolutional Neural Networks," in *CVPR*, 2016.
- [114] X. Huang and S. Belongie, "Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization," in *ICCV*, 2017.
- [115] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," in *NeurIPS Deep Learning Workshop*, 2015.
- [116] J. Song, C. Meng, and S. Ermon, "Denoising Diffusion Implicit Models," in *ICLR*, 2021.
- [117] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, et al., "Flow Matching for Generative Modeling," in *ICLR*, 2023.
- [118] J. Ho and T. Salimans, "Classifier-Free Diffusion Guidance," *arXiv:2207.12598*, 2022.
- [119] R. Rombach, A. Blattmann, D. Lorenz, et al., "High-Resolution Image Synthesis with Latent Diffusion Models," in *CVPR*, 2022.
- [120] W. Peebles and S. Xie, "Scalable Diffusion Models with Transformers," in *ICCV*, 2023.



- [121] L. Zhang, A. Rao, and M. Agrawala, "Adding Conditional Control to Text-to-Image Diffusion Models," in *ICCV*, 2023.
- [122] Y. Blau and T. Michaeli, "The Perception-Distortion Tradeoff," in *CVPR*, 2018.
- [123] C. Ledig, L. Theis, F. Huszár, et al., "Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network," in *CVPR*, 2017.
- [124] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., "Image Quality Assessment: From Error Visibility to Structural Similarity," *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [125] M. Heusel, H. Ramsauer, T. Unterthiner, et al., "GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium," in *NeurIPS*, 2017.
- [126] T. Brooks, B. Mildenhall, T. Xue, et al., "Unprocessing Images for Learned Raw Denoising," in *CVPR*, 2019.
- [127] A. Abdelhamed, S. Lin, and M. S. Brown, "A High-Quality Denoising Dataset for Smartphone Cameras," in *CVPR*, 2018.
- [128] J. Deng, W. Dong, R. Socher, et al., "ImageNet: A Large-Scale Hierarchical Image Database," in *CVPR*, 2009.
- [129] A. Radford, J. W. Kim, C. Hallacy, et al., "Learning Transferable Visual Models from Natural Language Supervision," in *ICML*, 2021.
- [130] J. Li, D. Li, S. Savarese, et al., "BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models," in *ICML*, 2023.
- [131] H. Liu, C. Li, Q. Wu, et al., "Visual Instruction Tuning," in *NeurIPS*, 2023.
- [132] S. Kornblith, J. Shlens, and Q. V. Le, "Do Better ImageNet Models Transfer Better?" In *CVPR*, 2019.
- [133] T. Chen, S. Kornblith, M. Norouzi, et al., "A Simple Framework for Contrastive Learning of Visual Representations," in *ICML*, 2020.
- [134] K. He, H. Fan, Y. Wu, et al., "Momentum Contrast for Unsupervised Visual Representation Learning," in *CVPR*, 2020.
- [135] J.-B. Grill, F. Strub, F. Altché, et al., "Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning," in *NeurIPS*, 2020.
- [136] M. Caron, H. Touvron, I. Misra, et al., "Emerging Properties in Self-Supervised Vision Transformers," in *ICCV*, 2021.
- [137] K. He, X. Chen, S. Xie, et al., "Masked Autoencoders Are Scalable Vision Learners," in *CVPR*, 2022.
- [138] A. van den Oord, Y. Li, and O. Vinyals, "Representation Learning with Contrastive Predictive Coding," in *arXiv:1807.03748*, 2018.
- [139] J. Kaplan, S. McCandlish, T. Henighan, et al., "Scaling Laws for Neural Language Models," *arXiv:2001.08361*, 2020.
- [140] X. Zhai, A. Kolesnikov, N. Houlsby, et al., "Scaling Vision Transformers," in *CVPR*, 2022.
- [141] J. Hoffmann, S. Borgeaud, A. Mensch, et al., "Training Compute-Optimal Large Language Models," in *NeurIPS*, 2022.
- [142] C. G. Northcutt, A. Athalye, and J. Mueller, "Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks," in *NeurIPS Datasets and Benchmarks*, 2021.
- [143] B. Recht, R. Roelofs, L. Schmidt, et al., "Do ImageNet Classifiers Generalize to ImageNet?" In *ICML*, 2019.
- [144] N. Houlsby, A. Giurgiu, S. Jastrzebski, et al., "Parameter-Efficient Transfer Learning for NLP," in *ICML*, 2019.

- [145] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [146] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *RecSys*, 2019.

## 6.5 Thực hành

Chương này có một đặc điểm hiếm: bài thực hành nhỏ nhất của nó — vẽ đường cong chất lượng theo số bước lấy mẫu — dạy được nhiều hơn cả chục trang lý thuyết, vì nó biến một đánh đổi trừu tượng thành một hình vẽ bạn tự tay tạo ra. Một lưu ý về bản rã trước khi vào danh sách: danh sách dưới đây chốt tại **tháng 9/2026**; tên các mô hình dẫn đầu sẽ cũ trong 6–12 tháng, trong khi phần hạ tầng — [diffusers](#), [k-diffusion](#), [clean-fid](#) — thì bền hơn nhiều — hãy đầu tư kỹ năng vào nhóm sau.

**Repo và tài nguyên.** *Hạ tầng:* [huggingface/diffusers](#), [comfyanonymous/ComfyUI](#), [crowsonkb/k-diffusion](#). *Ảnh:* FLUX.2 (dẫn về ảnh chân thực; bản **klein 4B** chạy được trong khoảng 8 GB VRAM), Stable Diffusion 3.5 (hệ sinh thái tinh chỉnh sâu nhất), Qwen-Image (dựng chữ trong ảnh), Z-Image Turbo (nhanh). *Video:* Wan 2.2 (Apache 2.0, lựa chọn an toàn nhất về giấy phép, bản 5B từ 8 GB), LTX-2.3 (sinh âm thanh đồng bộ trong cùng lượt khuếch tán), HunyuanVideo 1.5. *Học nguyên lý:* [lucidrains/denoising-diffusion-pytorch](#) — đọc mã nguồn này một lần đáng giá hơn đọc ba bài báo. *Đánh giá:* [GaParmar/clean-fid](#), [richzhang/PerceptualSimilarity](#).

**Câu hỏi kiểm tra hiểu.** Vì sao mô hình tối ưu MSE cho ảnh mờ, và perceptual loss “sửa” điều đó bằng cách đánh đổi gì? FID nhảy với thứ gì mà mắt người không quan tâm, và ngược lại? Latent diffusion tiết kiệm tính toán bằng cách nào, và mắt gì ở đúng chỗ đó?

**Mini project (vài giờ đến hai ngày).** Cài một DDPM tối giản khoảng 200 dòng, huấn luyện trên MNIST hoặc CIFAR-10. Cố định mô hình, chỉ đổi bộ lấy mẫu và số bước (10, 25, 50, 250), rồi vẽ FID theo số bước *và* theo thời gian tường. Đường cong đó là bài học đánh đổi rõ nhất của cả mảng diffusion — và nó cũng cho bạn thấy tận mắt rằng chất lượng bão hòa trước khi thời gian bão hòa.

### Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

| Thuật ngữ / câu hỏi | Trả lời                                                                                                                                                                                                                    |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ••• Độ hợp lý       | Xác suất mà mô hình gán cho dữ liệu đã quan sát. Chỉ họ autoregressive tính được nó chính xác; GAN không có nó, và đó là lý do GAN mất công cụ chẩn đoán tự nhiên.                                                         |
| •• ELBO             | Cận dưới của log độ hợp lý, thứ mà VAE thực sự tối ưu. Phần tái tạo của nó là bình phương trung bình theo pixel, nên ảnh mờ là hệ quả trực tiếp của chính công thức này.                                                   |
| ••• Mode collapse   | Mô hình chỉ sinh ra một phần nhỏ của phân phối trong khi vẫn đạt điểm cao theo tiêu chí huấn luyện. Nhìn mẫu chọn lọc không phát hiện được, vì các mẫu vẫn đẹp — chúng chỉ giống nhau.                                     |
| ••• Score matching  | Huấn luyện bằng cách khớp gradient của log mật độ, $\nabla_x \log p(x)$ , thay vì khớp mật độ. Đây là nền toán của diffusion: mô hình học một trường vectơ, và lấy mẫu là thả một điểm nhiều vào trường đó rồi để nó trôi. |

| Thuật ngữ / câu hỏi                                                              | Trả lời (tiếp)                                                                                                                                                                                                                                         |
|----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ••• Vì sao mô hình tối ưu MSE luôn cho ảnh mờ?                                   | Vì nghiệm cực tiểu của bình phương trung bình là kỳ vọng có điều kiện, tức trung bình của mọi ảnh hợp lệ. Các ảnh đó lệch nhau vài pixel ở biên, nên trung bình của chúng nhoè.                                                                        |
| ••• Vì sao diffusion huấn luyện ổn định hơn GAN?                                 | Vì hàm mất mát của nó là bình phương trung bình trên nhiều, tức một bài hồi quy tầm thường. GAN đi tìm điểm cân bằng của một trò chơi hai người, nên không có “hội tụ” theo nghĩa thông thường.                                                        |
| •• Ma trận Gram                                                                  | Tích trong giữa các kênh của một bản đồ đặc trưng, cộng dồn trên mọi vị trí không gian. Phép cộng dồn xoá hết thông tin vị trí, nên nó giữ được kết cấu và bảng màu nhưng không giữ được bố cục.                                                       |
| •• AdaIN                                                                         | Chuẩn hoá đặc trưng của ảnh nội dung về trung bình–độ lệch chuẩn 0–1 theo từng kênh, rồi gán lại thống kê của ảnh phong cách. Cho phép một mạng duy nhất làm phong cách tuỳ ý, đổi lại chất lượng thấp hơn bản tối ưu.                                 |
| •• Vì sao tối ưu trên pixel lại tìm ra mẫu đối kháng?                            | Vì gradient luôn chỉ về hướng dốc nhất của hàm chấm điểm, mà mạng có vô số hướng nó nhảy còn mất người thì không. Tối ưu đi thẳng vào đúng những hướng bất đồng đó.                                                                                    |
| ••• Quá trình thuận                                                              | Phép pha nhiễu Gauss dần dần, không có tham số nào để học. Nhờ tính cộng của Gauss, ta nhảy thẳng từ ảnh gốc tới bước bất kỳ — đó là lý do kỹ thuật khiến diffusion huấn luyện được ở quy mô lớn.                                                      |
| •• Latent diffusion                                                              | Nén ảnh bằng một autoencoder rồi mới khuếch tán trong không gian ẩn. Với hệ số nén 8, ảnh $512 \times 512$ còn $64 \times 64$ vị trí — ít hơn 64 lần. Trần chất lượng vì thế nằm ở autoencoder.                                                        |
| •• Classifier-free guidance                                                      | Ngoại suy giữa dự đoán có điều kiện và không điều kiện: $\tilde{\epsilon} = \epsilon_{\text{vô dk}} + w(\epsilon_{\text{có dk}} - \epsilon_{\text{vô dk}})$ . Hệ số lớn bám câu mô tả chặt hơn, nhưng cắt bớt đuôi phân phối nên đa dạng giảm.         |
| •• ControlNet                                                                    | Nhánh sao chép phần mã hoá của mô hình gốc, nhận điều kiện cấu trúc (cạnh, tư thế, độ sâu) và nối vào qua các tầng khởi tạo bằng 0. Chi tiết khởi tạo 0 mới là phần đáng nhớ: nó khiến huấn luyện chỉ thêm khả năng chứ không phá prior đã có.         |
| •• Vì sao “độ mạnh” chỉ là một chỉ số thời gian?                                 | Vì nó quyết định bắt đầu khử nhiễu từ nấc nào. Bắt đầu ở nấc nhiễu ít thì mô hình chỉ tô lại bề mặt; bắt đầu ở nấc nhiễu nhiều thì bố cục cũng bị viết lại.                                                                                            |
| ••• Ill-posed                                                                    | Bài toán có nhiều nghiệm cùng hợp lệ vì toán tử thuận đã xoá thông tin. Siêu phân giải $\times 4$ là một ràng buộc cho mười sáu ẩn; mô hình không giải mà <i>chọn</i> một nghiệm, và prior quyết định chọn cái nào.                                    |
| ••• Vì sao không thể vừa sắc nét vừa đúng từng pixel?                            | Đó là đánh đổi perceptual–distortion: giảm sai lệch theo từng ảnh xuống dưới một mức thì buộc phải chấp nhận độ chân thực kém đi. Đây là kết quả <i>có chứng minh</i> dưới một cách hình thức hoá cụ thể, không phải lời khuyên rút ra từ kinh nghiệm. |
| •• FID                                                                           | Khoảng cách giữa phân phối ảnh sinh và phân phối ảnh thật, ước lượng bằng cách khớp Gauss trên đặc trưng của một mạng. Nhảy với việc phân phối bị thu hẹp, mù với lỗi ngữ nghĩa của từng ảnh, và cần hàng vạn mẫu.                                     |
| • Bản GAN sắc nét hơn hẳn với mắt người nhưng PSNR thua bản mờ — vì sao?         | Vì hai bên nằm ở hai đầu đường cong đánh đổi. Bản mờ là trung bình của mọi ảnh hợp lệ nên tối ưu sai lệch theo pixel; bản GAN chọn dứt khoát một nghiệm nên sắc nét mà lệch từng pixel. Muốn thấy khác biệt thì phải đo thêm bằng LPIPS hoặc FID.      |
| • Ảnh sinh mất chữ nhỏ và hoa văn mịn — chẩn đoán đầu tiên?                      | Mã hoá rồi giải mã ngay ảnh gốc, bỏ qua khuếch tán. Nếu chi tiết đã chết ở đó thì thủ phạm là autoencoder, không phải mô hình khuếch tán hay số bước lấy mẫu.                                                                                          |
| • Khử nhiễu rất tốt trên bộ tổng hợp, kém hẳn trên ảnh điện thoại — nguyên nhân? | Nó được huấn luyện trên nhiễu Gauss trắng, còn nhiễu thật là Poisson phụ thuộc cường độ sáng cộng nhiễu đọc, rồi bị bộ xử lý ảnh làm cho có tương quan không gian. Cách chữa là mô phỏng đúng chuỗi vật lý ở miền raw.                                 |

● ● ● phải thuộc    ● ● nên nắm    ● biết là có

**I Kiến thức này đang sống ở đâu** (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

| Công trình / hệ thống                          | Dùng ý tưởng nào của chương                                                                                      | Ghi chú                                                                     |
|------------------------------------------------|------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| diffusers, k-diffusion                         | Tách rời mô hình khỏi bộ lấy mẫu — đúng phân tách của mục 17.3, và là lý do đối sampler không cần huấn luyện lại | Hạ tầng bền; cũng là nơi lỗi lệch lịch nhiều hay xảy ra                     |
| Latent diffusion (dòng Stable Diffusion, FLUX) | Khuếch tán trong không gian ẩn để giảm số vị trí không gian; kiến trúc transformer thay U-Net                    | Trần chi tiết vẫn nằm ở autoencoder — kiểm tra bằng phép mã hoá rồi giải mã |
| ControlNet                                     | Ràng buộc cấu trúc bằng nhánh sao chép nối qua tầng khởi tạo 0, để không phá prior đã học                        | Chi tiết khởi tạo 0 là phần đáng học nhất, không phải phần kiến trúc        |
| Classifier-free guidance                       | Ngoại suy giữa nhánh có và không điều kiện; có mặt trong gần như mọi mô hình sinh ảnh hiện nay                   | Cũng là cơ chế cắt đuôi phân phối được nói ở mục 17.4                       |
| AdaIN                                          | Chuẩn hoá rồi gán lại thống kê theo kênh — vẫn được dùng lại trong nhiều mạng sinh và mạng điều kiện hoá         | Ý tưởng sống lâu hơn ứng dụng style transfer ban đầu của nó                 |
| clean-fid, LPIPS                               | Đánh giá ở hai tầng khác nhau (phân phối và theo cặp), đúng như bảng chỉ số của mục 17.4                         | Hạ tầng; tồn tại chính vì FID nhạy với chi tiết tiền xử lý                  |

**MINI PROJECT SLAM** · Khử nhiễu trước frontend: “đẹp hơn” có nghĩa là “bám tốt hơn” không? · 1–2 ngày

|          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Mục tiêu | nói thẳng: đây là liên hệ <i>gián tiếp</i> . Chương này về mô hình sinh, còn chỗ nó thật sự chạm vào visual SLAM là khâu tiền xử lý ảnh thiếu sáng đặt trước frontend. Mục tiêu là kiểm chứng bằng số một giả định mà hầu như ai cũng mặc nhiên tin: ảnh sau khử nhiễu trông sạch hơn thì frontend bám đặc trưng tốt hơn.                                                                                                                                                |
| Dữ liệu  | một chuỗi EuRoC thiếu sáng và nhiễu nhoè (V2_03 hoặc MH_04), cộng ba bản sao được làm tối và thêm nhiễu có kiểm soát để tạo một thang bốn mức nhiễu. Giữ nguyên <b>chân trị</b> ( <i>ground truth</i> ) của chuỗi gốc cho cả bốn mức.                                                                                                                                                                                                                                    |
| Các bước | (1) chạy ORB-SLAM3 trên từng mức nhiễu <i>chưa</i> khử để lấy đường cơ sở: ATE, và số đặc trưng bám được trung bình mỗi khung hình; (2) chèn một bộ khử nhiễu nhẹ trước frontend — BM3D hoặc một CNN khử nhiễu nhỏ chạy đủ nhanh; (3) chạy lại với cấu hình y hệt, mỗi cấu hình ít nhất năm lần vì harness không tắt định; (4) với mỗi mức nhiễu ghi ba đại lượng: PSNR (hoặc LPIPS) của ảnh sau khử nhiễu, số đặc trưng bám được, và ATE trung bình cùng độ lệch chuẩn. |
| Xong khi | bạn có một bảng “mức nhiễu × {PSNR, số đặc trưng bám được, ATE}” và trả lời được bằng số: mức nhiễu nào thì khử nhiễu cải thiện ATE vượt độ lệch chuẩn của harness, mức nào thì không, và có tồn tại một mức mà PSNR <i>tăng</i> trong khi số đặc trưng bám được <i>giảm</i> hay không. Nếu có, hãy chỉ đúng chỗ đó — đó là đánh đổi perceptual-distortion hiện ra ở dạng cụ thể nhất.                                                                                   |
| Nối với  | <a href="#">C/17/04</a> cho cơ chế: làm mượt để tối ưu sai lệch theo pixel cũng chính là xoá đi các góc mà detector cần; <a href="#">A/02</a> cho mô hình nhiễu cảm biến thật; <a href="#">E/24</a> và <a href="#">B/07</a> cho cách quyết định một chênh lệch có thật hay chỉ là nhiễu của harness. Dùng lại đúng harness lặp nhiều lần mà mini project chương 16 đã buộc bạn dựng.                                                                                     |

**TRẢ LỜI CÂU HỎI MỞ ĐẦU**

- A1. Vì nghiệm cực tiểu của bình phương trung bình là kỳ vọng có điều kiện — trung bình của *mọi* ảnh sắc nét khớp với đầu vào. Các ảnh đó lệch nhau vài pixel ở biên, nên trung bình của chúng là một ảnh mờ. Đây là nghiệm đúng của bài toán đã đặt ra, không phải dấu hiệu huấn luyện chưa đủ.

- A2. Vì cả hai chỉ là hai hướng đo khoảng cách giữa phân phối mô hình và phân phối dữ liệu. Hướng thuận phạt việc bỏ sót mode nên mô hình trải rộng phủ cả hai mode và đặt khối lượng vào vùng trống — ra ảnh mờ. Hướng ngược phạt việc đặt khối lượng ở nơi dữ liệu hiếm nên mô hình co về một mode — ra mode collapse. Một lựa chọn, hai triệu chứng.
- A3. Vì hàm mất mát của nó là bình phương trung bình trên nhiều, tức một bài hồi quy tầm thường, trong khi GAN tối ưu điểm cân bằng của một trò chơi hai người. Hồi quy thì tăng quy mô là tốt lên đều đặn; trò chơi thì không có “hội tụ” theo nghĩa thông thường. Diffusion mua sự ổn định đó bằng số bước lấy mẫu.
- A4. Vì mọi nôm làm ảnh đẹp hơn — dẫn hướng mạnh, cắt ngưỡng, chọn lọc thủ công — đều hoạt động bằng cách cắt bớt đuôi phân phối. Từng ảnh đẹp hơn nhưng tập ảnh nghèo hơn, và nếu vòng lặp lặp lại thì mỗi thế hệ học từ đuôi đã bị cắt của thế hệ trước, phương sai co dần và các mode hiếm biến mất.

### NÓI LẠI THẬT ĐƠN GIẢN

Tưởng tượng một cuộc thi vẽ, và ba giám khảo chấm cùng một bài. Người thứ nhất đặt bài thi chồng lên bản mẫu rồi đo xem mỗi nét lệch bao nhiêu milimét. Người thứ hai lùi ra xa ba mét, nheo mắt, và hỏi “nhìn tổng thể nó có ra dáng không”. Người thứ ba trộn bài đó vào một chồng năm trăm bài thật rồi xem có rút nó ra được không.

Ba cách chấm đó cho ba thí sinh thủ khoa hoàn toàn khác nhau, và đây mới là điều đáng nhớ: thí sinh sẽ luyện đúng theo cách mình bị chấm. Nếu bị chấm bằng thước, cách an toàn nhất là vẽ nhòe — một nét nhòe nằm giữa mọi khả năng thì không bị trừ nhiều ở bất cứ chỗ nào. Nếu bị chấm bằng mắt từ ba mét, cách thắng là vẽ mạnh tay và tự bịa thêm chi tiết cho sống động, kể cả những chi tiết không có trong bản mẫu.

Toàn bộ chương này nói rằng máy cũng vậy. Người ta không dạy máy “vẽ đẹp hơn”; người ta chỉ chọn một cách chấm, rồi máy tự tìm cách thắng cách chấm đó. Ảnh mờ không phải vì máy kém mà vì nó đang bị chấm bằng thước. Ảnh sắc nét nhưng bịa chi tiết không phải vì máy nói dối mà vì nó đang bị chấm bằng mắt.

Cái giá là bạn không thể chọn cả hai. Muốn không bao giờ bịa thì phải chấp nhận nhòe; muốn sắc nét thì phải chấp nhận rằng một phần những gì bạn thấy là do máy tự thêm vào. Trong ảnh trang trí, đó là ưu điểm. Trong ảnh y tế hay ảnh dùng làm bằng chứng, đó là sai lầm nghiêm trọng.

**Chỗ không rút gọn được:** câu “không thể chọn cả hai” ở trên. Nói bằng lời thường thì nó nghe như một lời khuyên rút ra từ kinh nghiệm, nhưng thực chất nó là một kết quả *chứng minh được* dưới một cách phát biểu chặt chẽ. Sự khác biệt đó rất quan trọng khi đọc một bài báo tuyên bố “chúng tôi vừa chính xác hơn vừa sắc nét hơn”: với một lời khuyên kinh nghiệm thì tuyên bố ấy có thể đúng, với một định lý thì nó chỉ có thể đúng nếu bài báo đang đo hai thứ khác nhau.

**Một câu:** Mọi mô hình sinh đều là một thí sinh luyện theo cách chấm điểm, nên đổi thước đo là đổi kiểu sai lầm, không phải đổi mức thông minh.

### Kết nối

[C/18-pretraining-va-foundation](#) tiếp tục câu chuyện “tín hiệu giám sát đến từ đâu” mà chương này chạm vào khi nói về dữ liệu sinh; [D/20-toi-uu-va-nen-mo-hinh](#) trình bày đầy đủ mẫu hình dời chi phí sang lúc huấn luyện; [E/25-do-tin-cay-va-van-hanh-bien](#) “hông im lặng” của các mô hình khôi phục thành thiết kế phòng thủ cụ thể; và [C/16-thi-giac-3d](#) là chương song song, nơi cùng nguyên lý “prior chọn nghiệm” xuất hiện trong bài toán hình học.

### Tổng kết chương

Ta đã đi qua bốn tầng của một câu hỏi duy nhất — khi nhiều đầu ra cùng hợp lệ thì cái gì chọn? — và câu trả lời lặp đi lặp lại là: hàm mất mát chọn, chứ không phải kiến trúc chọn. Bốn họ mô hình sinh khác nhau vì bốn cách viết “giống dữ liệu thật” thành một con số, và kiểu thất bại của mỗi họ suy ra được từ chính con số đó. Tối ưu trên pixel cho thấy điều này ở dạng thuần khiết

nhất: cùng một vòng lặp, đối hàm mục tiêu là được một kỹ thuật khác hẳn, từ tác phẩm nghệ thuật tới mẫu đối kháng. Diffusion thắng không nhờ ý tưởng đẹp hơn mà nhờ biến bài toán sinh thành một bài hồi quy có thể mở rộng quy mô. Và low-level vision cho thấy khi hậu quả là thật, cùng một đánh đổi đó trở thành câu hỏi “tôi chịu được kiểu sai nào”.

Ba chỗ còn bỏ ngỏ, đáng theo dõi: thứ nhất, việc rút số bước lấy mẫu xuống một chữ số mà không mất đa dạng — flow matching và chưng cất bộ lấy mẫu đang tiến nhanh nhưng chưa có lời giải sạch; thứ hai, đánh giá mô hình sinh, nơi mọi chỉ số hiện có đều mù ở một chỗ quan trọng và chưa ai có phương án thay thế thuyết phục; thứ ba, hệ quả dài hạn của việc dữ liệu sinh tràn vào tập huấn luyện của thể hệ mô hình sau — cơ chế thu hẹp phân phối đã rõ, nhưng độ nghiêm trọng trong thực tế còn phụ thuộc nhiều điều kiện chưa được đo đủ.

### **Câu hỏi tự kiểm** (trả lời không nhìn bài)

1. Câu hỏi chung của cả chương là “khi nhiều đầu ra cùng hợp lệ thì cái gì quyết định chọn cái nào”. Với VAE, GAN, diffusion và một mạng khôi phục ảnh huấn luyện bằng MSE, mỗi cái trả lời bằng cơ chế nào, và hai cái nào trong bốn thực ra đang đưa cùng một câu trả lời?
2. Mọi cơ chế làm ảnh đẹp hơn đều mua độ trung thành bằng độ đa dạng — hệ số dẫn hướng, loss cảm nhận và ma trận Gram đều thuộc họ đó. Ba cái này thu hẹp phân phối ở ba chỗ khác nhau nào trong pipeline?
3. Đánh đổi perceptual-distortion ở mục 4 và thể lưỡng nan ba chiều ở mục 1 nghe rất giống nhau. Chúng khác nhau ở địa vị lý thuyết thế nào, và vì sao gộp nhầm hai thứ đó lại dẫn tới kết luận sai khi đọc paper?
4. Mẫu đối kháng ở mục 2 và ảnh có FID rất tốt nhưng sai chi tiết ở mục 4 đều là “tối ưu đúng thước đo, sai mục đích”. Điểm chung về cơ chế là gì, và cùng một cách phòng vệ có dùng được cho cả hai không?
5. Bạn dùng đầu ra của một mô hình sinh làm dữ liệu huấn luyện cho mô hình khác. Điều kiện nào phải giữ để việc đó an toàn, và triệu chứng đầu tiên khi điều kiện bị vi phạm hiện ra ở đâu?
6. Latent diffusion, AdaIN và mẫu hình “dời chi phí sang lúc huấn luyện” đều đổi chất lượng lấy tốc độ. Với ràng buộc thời gian thực trên thiết bị nhúng, cái nào bị loại đầu tiên và vì sao?
7. Cả bốn mục đều cảnh báo về việc tin vào thứ nhìn thấy được — ảnh mẫu đẹp, bản đồ attention, ảnh trực quan hoá lớp, ảnh khử nhiễu sắc nét. Có quy tắc chung nào rút ra được không, và nó buộc bạn đo thêm cái gì?



- [1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [2] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [3] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *AISTATS*, 2010.
- [4] K. He, X. Zhang, S. Ren, et al., “Delving Deep into Rectifiers,” in *ICCV*, 2015.
- [5] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [6] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [7] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [8] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [9] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [10] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [11] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” in *ICLR*, 2017.
- [12] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [13] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [14] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [15] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [16] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [17] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [18] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [19] G. Huang, Y. Sun, Z. Liu, et al., “Deep Networks with Stochastic Depth,” in *ECCV*, 2016.
- [20] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [21] H. Zhang, M. Cisse, Y. N. Dauphin, et al., “mixup: Beyond Empirical Risk Minimization,” in *ICLR*, 2018.
- [22] S. Yun, D. Han, S. J. Oh, et al., “CutMix: Regularization Strategy to Train Strong Classifiers,” in *ICCV*, 2019.
- [23] E. D. Cubuk, B. Zoph, J. Shlens, et al., “RandAugment: Practical Automated Data Augmentation,” in *CVPR Workshops*, 2020.

- [24] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [25] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [26] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *NeurIPS*, 2012.
- [27] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in *ICLR*, 2015.
- [28] C. Szegedy, W. Liu, et al., “Going Deeper with Convolutions,” in *CVPR*, 2015.
- [29] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [30] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [31] G. Huang, Z. Liu, L. van der Maaten, et al., “Densely Connected Convolutional Networks,” in *CVPR*, 2017.
- [32] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [33] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [34] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [35] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [36] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [37] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.
- [38] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [39] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [40] J. Su, M. Ahmed, Y. Lu, et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, vol. 568, 2024.
- [41] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” *arXiv:2312.00752*, 2023.
- [42] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [43] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.
- [44] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [45] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [46] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [47] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [48] R. Girshick, J. Donahue, T. Darrell, et al., “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation,” in *CVPR*, 2014.

- [49] R. Girshick, "Fast R-CNN," in *ICCV*, 2015.
- [50] S. Ren, K. He, R. Girshick, et al., "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in *NeurIPS*, 2015.
- [51] T.-Y. Lin, P. Dollár, R. Girshick, et al., "Feature Pyramid Networks for Object Detection," in *CVPR*, 2017.
- [52] K. He, G. Gkioxari, P. Dollár, et al., "Mask R-CNN," in *ICCV*, 2017.
- [53] D. Bolya, S. Foley, J. Hays, et al., "TIDE: A General Toolbox for Identifying Object Detection Errors," in *ECCV*, 2020.
- [54] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [55] J. Redmon, S. Divvala, R. Girshick, et al., "You Only Look Once: Unified, Real-Time Object Detection," in *CVPR*, 2016.
- [56] W. Liu, D. Anguelov, D. Erhan, et al., "SSD: Single Shot MultiBox Detector," in *ECCV*, 2016.
- [57] T.-Y. Lin, P. Goyal, R. Girshick, et al., "Focal Loss for Dense Object Detection," in *ICCV*, 2017.
- [58] Z. Tian, C. Shen, H. Chen, et al., "FCOS: Fully Convolutional One-Stage Object Detection," in *ICCV*, 2019.
- [59] X. Zhou, D. Wang, and P. Krähenbühl, "Objects as Points," *arXiv:1904.07850*, 2019.
- [60] S. Zhang, C. Chi, Y. Yao, et al., "Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection," in *CVPR*, 2020.
- [61] Z. Ge, S. Liu, Z. Li, et al., "OTA: Optimal Transport Assignment for Object Detection," in *CVPR*, 2021.
- [62] N. Bodla, B. Singh, R. Chellappa, et al., "Soft-NMS: Improving Object Detection with One Line of Code," in *ICCV*, 2017.
- [63] X. Zhu, W. Su, L. Lu, et al., "Deformable DETR: Deformable Transformers for End-to-End Object Detection," in *ICLR*, 2021.
- [64] H. Zhang, F. Li, S. Liu, et al., "DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection," in *ICLR*, 2023.
- [65] J. Long, E. Shelhamer, and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," in *CVPR*, 2015.
- [66] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *MICCAI*, 2015.
- [67] L.-C. Chen, G. Papandreou, I. Kokkinos, et al., "DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs," *IEEE TPAMI*, vol. 40, no. 4, pp. 834–848, 2018.
- [68] E. Xie, W. Wang, Z. Yu, et al., "SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers," in *NeurIPS*, 2021.
- [69] B. Cheng, I. Misra, A. G. Schwing, et al., "Masked-Attention Mask Transformer for Universal Image Segmentation," in *CVPR*, 2022.
- [70] A. Kirillov, K. He, R. Girshick, et al., "Panoptic Segmentation," in *CVPR*, 2019.
- [71] A. Kirillov, E. Mintun, N. Ravi, et al., "Segment Anything," in *ICCV*, 2023.
- [72] K. Sun, B. Xiao, D. Liu, et al., "Deep High-Resolution Representation Learning for Human Pose Estimation," in *CVPR*, 2019.

- [73] Z. Cao, T. Simon, S.-E. Wei, et al., “Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields,” in *CVPR*, 2017.
- [74] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [75] J. Deng, J. Guo, N. Xue, et al., “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *CVPR*, 2019.
- [76] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [77] K. Xu, J. Ba, R. Kiros, et al., “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention,” in *ICML*, 2015.
- [78] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *ICLR*, 2015.
- [79] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [80] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [81] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [82] K. Simonyan and A. Zisserman, “Two-Stream Convolutional Networks for Action Recognition in Videos,” in *NeurIPS*, 2014.
- [83] J. Carreira and A. Zisserman, “Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset,” in *CVPR*, 2017.
- [84] D. Tran, H. Wang, L. Torresani, et al., “A Closer Look at Spatiotemporal Convolutions for Action Recognition,” in *CVPR*, 2018.
- [85] C. Feichtenhofer, H. Fan, J. Malik, et al., “SlowFast Networks for Video Recognition,” in *ICCV*, 2019.
- [86] A. Dosovitskiy, P. Fischer, E. Ilg, et al., “FlowNet: Learning Optical Flow with Convolutional Networks,” in *ICCV*, 2015.
- [87] D. Sun, X. Yang, M.-Y. Liu, et al., “PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume,” in *CVPR*, 2018.
- [88] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.
- [89] A. Bewley, Z. Ge, L. Ott, et al., “Simple Online and Realtime Tracking,” in *ICIP*, 2016.
- [90] Y. Zhang, P. Sun, Y. Jiang, et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *ECCV*, 2022.
- [91] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [92] K. Bernardin and R. Stiefelhagen, “Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics,” *EURASIP J. Image and Video Processing*, 2008.
- [93] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [94] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [95] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.

- [96] S. Wang, V. Leroy, Y. Cabon, et al., "DUST3R: Geometric 3D Vision Made Easy," in *CVPR*, 2024.
- [97] D. Eigen, C. Puhrsch, and R. Fergus, "Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network," in *NeurIPS*, 2014.
- [98] R. Ranftl, K. Lasinger, D. Hafner, et al., "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer," *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [99] J. Sturm, N. Engelhard, F. Endres, et al., "A Benchmark for the Evaluation of RGB-D SLAM Systems," in *IROS*, 2012.
- [100] C. R. Qi, H. Su, K. Mo, et al., "PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation," in *CVPR*, 2017.
- [101] C. R. Qi, L. Yi, H. Su, et al., "PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space," in *NeurIPS*, 2017.
- [102] A. H. Lang, S. Vora, H. Caesar, et al., "PointPillars: Fast Encoders for Object Detection from Point Clouds," in *CVPR*, 2019.
- [103] T. Müller, A. Evans, C. Schied, et al., "Instant Neural Graphics Primitives with a Multiresolution Hash Encoding," *ACM TOG*, vol. 41, no. 4, 2022.
- [104] B. Kerbl, G. Kopanas, T. Leimkühler, et al., "3D Gaussian Splatting for Real-Time Radiance Field Rendering," *ACM TOG*, vol. 42, no. 4, 2023.
- [105] R. Zhang, P. Isola, A. A. Efros, et al., "The Unreasonable Effectiveness of Deep Features as a Perceptual Metric," in *CVPR*, 2018.
- [106] D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," in *ICLR*, 2014.
- [107] I. Goodfellow, J. Pouget-Abadie, M. Mirza, et al., "Generative Adversarial Nets," in *NeurIPS*, 2014.
- [108] T. Karras, S. Laine, and T. Aila, "A Style-Based Generator Architecture for Generative Adversarial Networks," in *CVPR*, 2019.
- [109] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," in *NeurIPS*, 2020.
- [110] Y. Song, J. Sohl-Dickstein, D. P. Kingma, et al., "Score-Based Generative Modeling Through Stochastic Differential Equations," in *ICLR*, 2021.
- [111] J. Johnson, A. Alahi, and L. Fei-Fei, "Perceptual Losses for Real-Time Style Transfer and Super-Resolution," in *ECCV*, 2016.
- [112] P. Isola, J.-Y. Zhu, T. Zhou, et al., "Image-to-Image Translation with Conditional Adversarial Networks," in *CVPR*, 2017.
- [113] L. A. Gatys, A. S. Ecker, and M. Bethge, "Image Style Transfer Using Convolutional Neural Networks," in *CVPR*, 2016.
- [114] X. Huang and S. Belongie, "Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization," in *ICCV*, 2017.
- [115] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," in *NeurIPS Deep Learning Workshop*, 2015.
- [116] J. Song, C. Meng, and S. Ermon, "Denoising Diffusion Implicit Models," in *ICLR*, 2021.
- [117] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, et al., "Flow Matching for Generative Modeling," in *ICLR*, 2023.
- [118] J. Ho and T. Salimans, "Classifier-Free Diffusion Guidance," *arXiv:2207.12598*, 2022.
- [119] R. Rombach, A. Blattmann, D. Lorenz, et al., "High-Resolution Image Synthesis with Latent Diffusion Models," in *CVPR*, 2022.
- [120] W. Peebles and S. Xie, "Scalable Diffusion Models with Transformers," in *ICCV*, 2023.



- [121] L. Zhang, A. Rao, and M. Agrawala, “Adding Conditional Control to Text-to-Image Diffusion Models,” in *ICCV*, 2023.
- [122] Y. Blau and T. Michaeli, “The Perception-Distortion Tradeoff,” in *CVPR*, 2018.
- [123] C. Ledig, L. Theis, F. Huszár, et al., “Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network,” in *CVPR*, 2017.
- [124] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., “Image Quality Assessment: From Error Visibility to Structural Similarity,” *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [125] M. Heusel, H. Ramsauer, T. Unterthiner, et al., “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium,” in *NeurIPS*, 2017.
- [126] T. Brooks, B. Mildenhall, T. Xue, et al., “Unprocessing Images for Learned Raw Denoising,” in *CVPR*, 2019.
- [127] A. Abdelhamed, S. Lin, and M. S. Brown, “A High-Quality Denoising Dataset for Smartphone Cameras,” in *CVPR*, 2018.
- [128] J. Deng, W. Dong, R. Socher, et al., “ImageNet: A Large-Scale Hierarchical Image Database,” in *CVPR*, 2009.
- [129] A. Radford, J. W. Kim, C. Hallacy, et al., “Learning Transferable Visual Models from Natural Language Supervision,” in *ICML*, 2021.
- [130] J. Li, D. Li, S. Savarese, et al., “BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models,” in *ICML*, 2023.
- [131] H. Liu, C. Li, Q. Wu, et al., “Visual Instruction Tuning,” in *NeurIPS*, 2023.
- [132] S. Kornblith, J. Shlens, and Q. V. Le, “Do Better ImageNet Models Transfer Better?” In *CVPR*, 2019.
- [133] T. Chen, S. Kornblith, M. Norouzi, et al., “A Simple Framework for Contrastive Learning of Visual Representations,” in *ICML*, 2020.
- [134] K. He, H. Fan, Y. Wu, et al., “Momentum Contrast for Unsupervised Visual Representation Learning,” in *CVPR*, 2020.
- [135] J.-B. Grill, F. Strub, F. Altché, et al., “Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning,” in *NeurIPS*, 2020.
- [136] M. Caron, H. Touvron, I. Misra, et al., “Emerging Properties in Self-Supervised Vision Transformers,” in *ICCV*, 2021.
- [137] K. He, X. Chen, S. Xie, et al., “Masked Autoencoders Are Scalable Vision Learners,” in *CVPR*, 2022.
- [138] A. van den Oord, Y. Li, and O. Vinyals, “Representation Learning with Contrastive Predictive Coding,” in *arXiv:1807.03748*, 2018.
- [139] J. Kaplan, S. McCandlish, T. Henighan, et al., “Scaling Laws for Neural Language Models,” *arXiv:2001.08361*, 2020.
- [140] X. Zhai, A. Kolesnikov, N. Houlsby, et al., “Scaling Vision Transformers,” in *CVPR*, 2022.
- [141] J. Hoffmann, S. Borgeaud, A. Mensch, et al., “Training Compute-Optimal Large Language Models,” in *NeurIPS*, 2022.
- [142] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [143] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [144] N. Houlsby, A. Giurgiu, S. Jastrzebski, et al., “Parameter-Efficient Transfer Learning for NLP,” in *ICML*, 2019.



- [145] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [146] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *RecSys*, 2019.

## Pretraining, transfer và mô hình nền tảng (Pretraining, Transfer and Foundation Models)

**PHỤ THUỘC** B/06-chat-luong-du-lieu và B/07-chia-du-lieu-va-danh-gia (chi phí nhân và cách đánh giá không tự lừa mình); B/09-thiet-ke-ham-mat-mat (*contrastive loss*, *cross-entropy*); C/13-kien-truc-backbone (ViT và *inductive bias* — chương này giải thích vì sao ViT chỉ toả sáng khi có tiền huấn luyện quy mô lớn).

### TÓM TẮT

Chương này trả lời một câu hỏi: nhân thì đắt, dữ liệu không nhân thì vô hạn — vậy tín hiệu giám sát miễn phí lấy từ đâu, và mua bao nhiêu thì đủ. Mục 18.1 đi qua bốn nguồn giám sát rẻ dần và nói mỗi nguồn hỏng ở đâu; mục 18.2 đưa ra mặt định lượng: scaling laws, bài học Chinchilla, nhiệm vụ kiểm tra, và LoRA. Đọc xong bạn chọn được *backbone*, chọn được chế độ thích nghi, và ước lượng được hoá đơn compute — thay vì hỏi người khác hoặc thử mù. Nếu chỉ nhớ một điều: mô hình nền tảng đã biến bạn từ người huấn luyện thành người tích hợp, nên câu hỏi trung tâm đổi từ “mô hình này có đúng không” sang “tôi có biết nó giả định gì và hỏng ở đâu không”.

### KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình (model)** — một hàm có tham số, ánh xạ đầu vào (ảnh) sang đầu ra (nhân, hộp, vector đặc trưng). “Huấn luyện” nghĩa là tìm bộ tham số làm hàm mất mát nhỏ nhất trên dữ liệu.
- **tiền huấn luyện (pretraining)** — huấn luyện một mô hình trên một tập dữ liệu lớn, tổng quát, trước khi biết bài toán cuối là gì. Mục đích là để mô hình học sẵn những đặc trưng chung của ảnh, thứ mà dữ liệu ít ỏi của bài toán cuối không đủ để học lại.
- **học chuyển giao (transfer learning)** — lấy mô hình đã tiền huấn luyện rồi dùng lại cho bài toán khác, hoặc bằng cách đóng băng nó, hoặc bằng cách huấn luyện tiếp trên dữ liệu của mình.
- **backbone** — phần thân của mạng chịu trách nhiệm biến ảnh thành vector đặc trưng; phần “đầu” (head) gắn sau nó mới sinh ra đầu ra cụ thể của từng bài toán. Tách hai phần ra là điều kiện để tái sử dụng.
- **học tự giám sát (self-supervised learning)** — huấn luyện mà không dùng nhãn người: đáp án của bài toán phụ được suy ra từ chính dữ liệu, chẳng hạn “hai lát cắt của cùng bức ảnh phải cho vector giống nhau”.
- **mô hình nền tảng (foundation model)** — một mô hình lớn được tiền huấn luyện trên dữ liệu quy mô rất lớn, đủ tổng quát để dùng lại cho nhiều bài toán khác nhau mà không huấn luyện lại từ đầu.
- **VLM (vision-language model)** — mô hình nhận cả ảnh lẫn văn bản làm đầu vào và sinh ra văn bản, thường bằng cách nối một bộ mã hoá ảnh vào một mô hình ngôn ngữ lớn.
- **ngân sách compute** — tổng số phép tính (FLOPs) bạn mua được cho một lần huấn luyện. Vì nó cố định, mọi quyết định về kích thước mô hình và lượng dữ liệu đều là quyết định phân bổ trong cùng một ngân sách.

## CÂU HỎI MỞ ĐẦU

- Q1. Với 500 ảnh nội soi, *linear probe* trên một *backbone* khổng lồ hay tinh chỉnh toàn bộ một ResNet nhỏ sẽ thắng? Và vì sao câu trả lời đảo chiều ở 50 000 ảnh?
- Q2. Nếu chỉ cần dạy mô hình rằng hai lát cắt của cùng một bức ảnh phải cho vector giống nhau, vì sao nó không học ngay nghiệm rẻ nhất là xuất một hằng số cho mọi ảnh?
- Q3. Vì sao một dòng chú thích cầu thả trên internet lại là tín hiệu giám sát ngữ nghĩa mạnh hơn một nhãn ImageNet do người gán cẩn thận?
- Q4. Gopher lớn gấp bốn lần Chinchilla và tốn cùng số FLOPs để huấn luyện, nhưng thua — cả ngành đã tối ưu nhằm đạt lượng nào suốt mấy năm?
- Q5. Vì sao một mô hình chưa bao giờ nhìn thấy ảnh vệ tinh lại cho đặc trưng ảnh vệ tinh tốt hơn một mạng huấn luyện riêng trên ảnh vệ tinh?

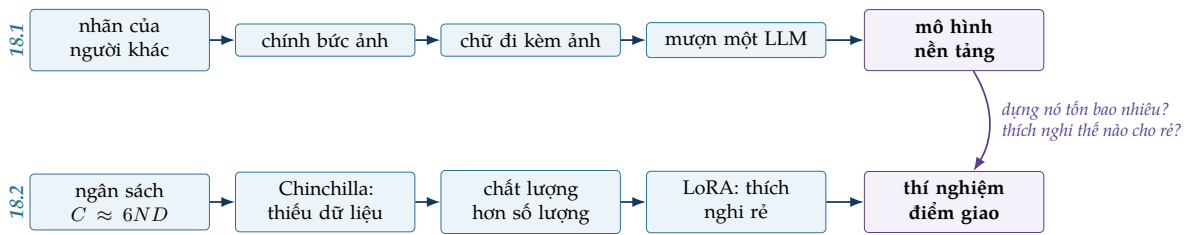
Câu hỏi dẫn dắt của cả chương chỉ có một dòng: *nhân thì đắt, dữ liệu không nhân thì vô hạn* — *tín hiệu giám sát miễn phí có thể lấy từ đâu?* Mọi kỹ thuật trong chương, từ học chuyển giao những năm 2014 tới VLM ngày nay, là một câu trả lời cho đúng câu hỏi đó, và các câu trả lời xếp thành một chuỗi ngày càng táo bạo: dùng nhãn của người khác, rồi dùng chính bức ảnh, rồi dùng dòng chữ đi kèm bức ảnh, rồi mượn luôn khả năng suy luận của một mô hình ngôn ngữ.

Chuỗi ấy là một mạch *định tính*: nó nói cho bạn biết có những nguồn giám sát nào và mỗi nguồn giả định gì. Nhưng nó không trả lời được câu hỏi mà bạn thực sự phải quyết định khi ngồi trước một cụm GPU có hạn: *tiêu ngân sách vào đâu, và bao nhiêu thì đủ?* Đó là một mạch khác, mang tính *định lượng* và kinh tế, với từ vựng riêng — *scaling laws*, tỉ lệ dữ liệu trên tham số, chi phí thích nghi. Hai mạch dùng chung đối tượng nhưng khác hẳn nhau về loại lập luận, và trộn chúng vào một mạch duy nhất là cách nhanh nhất để cả hai đều mờ đi. Vì vậy chương chia làm hai mục:

- **18.1 — Nguồn giám sát ngày càng rẻ.** Chuỗi bốn nguồn, dừng ở từng nguồn để hỏi nó chống lại nghiệm tầm thường bằng cách nào, nó ép vào biểu diễn những bất biến gì, và nó hỏng ở đâu.
- **18.2 — Kinh tế học của pretraining.** Lấy kết quả của mục trước làm cho sẵn rồi hỏi tiếp: với chừng này compute thì mô hình nên lớn cỡ nào, dữ liệu nên nhiều cỡ nào, và khi nào tinh chỉnh đáng giá hơn đóng băng.

Thứ tự đọc là bắt buộc theo chiều này: mục 18.2 nhắc tới *linear probe*, *fine-tune* và mô hình nền tảng như những khái niệm đã biết. Có một điểm nối giữa hai mục đáng nói trước: cả hai kết thúc ở cùng một kết luận về phương pháp luận. Mục 18.1 kết luận rằng bạn không thể biết một biểu diễn tự giám sát đã học gì nếu chỉ nhìn *benchmark* chung; mục 18.2 kết luận rằng bạn không thể biết chế độ thích nghi nào đúng nếu chỉ nghe lời khuyên chung. Cả hai đẩy bạn về cùng một hành động: đo trên chính dữ liệu của mình, với một tập đánh giá mà mô hình nền tảng chắc chắn chưa từng thấy.

Hình 7.1 vẽ chỗ nối giữa hai mục: mục 18.1 sản xuất ra một mô hình nền tảng, còn mục 18.2 hỏi cái mô hình ấy tốn bao nhiêu và thích nghi thế nào cho rẻ.



Hàng trên: giám sát rẻ dần, tín hiệu gián tiếp dần. Hàng dưới: cùng một ngân sách nhìn từ phía chi phí.

**Hình 7.1.** Bản đồ chương 18. Hai mục không nối tiếp nhau về thời gian mà nối nhau ở *đối tượng*: mục 18.1 kết thúc ở mô hình nền tảng, và mục 18.2 bắt đầu bằng câu hỏi mô hình đó tốn bao nhiêu. Mọi con số chung ở hàng dưới đều kết thúc ở cùng một chỗ — một thí nghiệm bạn phải tự chạy trên dữ liệu của mình.

## Bản đồ chương 18 — Pretraining, transfer và mô hình nền tảng

Chương này trả lời một câu hỏi duy nhất — *tín hiệu giám sát miễn phí lấy từ đâu, và mua bao nhiêu thì đủ* — nhưng câu hỏi đó có hai nửa dùng hai loại lập luận khác nhau, nên chương chia làm hai mục và thứ tự đọc là bắt buộc.

**18.1 — Nguồn giám sát ngày càng rẻ [T3] (cốt lõi).** Chuỗi bốn nguồn giám sát xếp theo chi phí giảm dần: nhãn của người khác (transfer learning và bảng bốn ô quyết định), chính bức ảnh (ba triết lý tự giám sát và ba cách chống sập khác nhau), văn bản đi kèm ảnh (CLIP và ngữ nghĩa mở), và cuối cùng là ghép thị giác vào một LLM (VLM). Đây là mục phải nắm chắc: nó là tiền đề trực tiếp của open-vocabulary detection ở chương 14 và của mọi quyết định chọn backbone về sau. Điểm cần mang theo là cột “hồng khi nào” của bảng ba họ tự giám sát, và ý thức rằng mỗi bất biến bạn ép vào augmentation là một loại thông tin bạn cố tình vứt đi.

**18.2 — Kinh tế học của pretraining [T2][T3] (cốt lõi phân nguyên lý, tham khảo phân con số).** Mặt định lượng: scaling laws cho phép ngoại suy thay vì thử mù; bài học Chinchilla cho thấy cả ngành đã thổi tham số mà quên dữ liệu; chất lượng corpus thắng số lượng; contamination phá hủy hiệu lực của mọi benchmark công khai; PEFT/LoRA đổi hoàn toàn kinh tế học của việc thích nghi; và quyết định cuối cùng luôn là thí nghiệm điểm giao trên dữ liệu của chính bạn. Nguyên lý ở đây thuộc tầng bền ([T2]); các hằng số cụ thể — tỉ lệ 20 token trên tham số, hạng LoRA hợp lý — thì không, và nên tra lại thay vì nhớ.

**Vì sao chia như vậy.** Mục 18.1 trả lời “có những gì và nó giả định gì”; mục 18.2 trả lời “tiêu bao nhiêu vào đâu”. Nếu gộp lại, phân định lượng sẽ nuốt mất phân định tính — người đọc nhớ con số mà quên cơ chế. Mục 18.2 dùng lại từ vựng của 18.1 (linear probe, fine-tune, mô hình nền tảng) như đã biết, nên đọc ngược sẽ vấp.

**Phụ thuộc lên trên.** B/06-chat-luong-du-lieu (vì sao nhãn đắt và bẩn), B/07-chia-du-lieu-va-danh-gia (vì sao contamination là vấn đề chết người), B/09-thiet-ke-ham-mat-mat (contrastive loss), C/13-kien-truc-backbone/03-vit-va-inductive-bias (vì sao ViT cần pretraining lớn).

**Toả xuống dưới.** C/14-bai-toan-cv-loi kế thừa trực tiếp không gian nhúng CLIP; D/20-toi-uu-va-nen-mo-hinh và D/21-inference-va-trien-khai nhận lấy mô hình nền tảng như một ràng buộc phải nén; E/24-thi-nghiem-va-ablation dạy cách chạy thí nghiệm điểm giao mà không tự lừa mình.

### 7.1 Nguồn giám sát ngày càng rẻ (Increasingly Cheap Supervision)

**PHỤ THUỘC** B/06-chat-luong-du-lieu (nhân đất và bắn tới mức nào); B/09-thiet-ke-ham-mat-mat (*contrastive loss*, cross-entropy); C/13-kien-truc-backbone/03-vit-va-inductive-bias (vì sao ViT cần nhiều dữ liệu hơn CNN); C/14-bai-toan-cv-loi (*open-vocabulary detection* là hệ quả trực tiếp của mục này).

### TÓM TẮT

Mục này xếp bốn nguồn giám sát theo chi phí giảm dần: nhãn của người khác, chính bức ảnh, dòng chữ đi kèm ảnh, và khả năng suy luận mượn từ một mô hình ngôn ngữ. Với mỗi nguồn, mục nói rõ nó chống nghiệm tầm thường bằng cơ chế nào và hỏng ở đâu. Đọc xong, trước một bài toán có vài trăm ảnh nhãn, bạn quyết được ngay ba việc: đóng băng hay tinh chỉnh, chọn backbone tự giám sát nào, và công thức augmentation nào sẽ phá hỏng đúng tín hiệu bạn cần. Đoán mò ba việc đó thì mỗi cái tốn vài ngày GPU mới biết là sai. Nếu chỉ nhớ một điều: mỗi bất biến bạn ép vào augmentation chính là một loại thông tin bạn cố tình vứt đi, nên hãy liệt kê các bất biến đó trước khi sao chép công thức của bất kỳ paper nào.

### KIẾN THỨC CẦN CÓ TRƯỚC

- **nhãn (label) và tín hiệu giám sát** — nhãn là đáp án đúng gắn với một mẫu dữ liệu; tín hiệu giám sát là bất cứ thứ gì đóng vai trò đáp án để tính hàm mất mát. Mọi ý tưởng trong mục này là cách tạo tín hiệu giám sát mà không phải trả tiền cho nhãn.
- **đặc trưng (feature) và embedding** — đặc trưng là vector số mà mạng sinh ra để mô tả một bức ảnh; *embedding* là cách gọi khác khi ta quan tâm tới hình học của không gian đó — ảnh giống nhau thì vector gần nhau.
- **đóng băng (frozen) và linear probe** — đóng băng là giữ nguyên trọng số backbone, không cho gradient cập nhật. *Linear probe* là huấn luyện đúng một tầng tuyến tính trên đặc trưng đóng băng — phép thử rẻ nhất để xem đặc trưng có chứa thông tin cần thiết hay không.
- **fine-tune (tinh chỉnh)** — huấn luyện tiếp một mô hình đã có trọng số, thường với *learning rate* nhỏ hơn nhiều lần để không xoá mất thứ nó đã học.
- **augmentation** — biến đổi ngẫu nhiên áp lên ảnh lúc huấn luyện (cắt, lật, đổi màu, làm mờ) nhằm dạy mô hình rằng những biến đổi đó không làm đổi đáp án. Chính vì thế nó cũng *xoá* thông tin: bất biến nào bạn dạy thì thông tin đó mất.
- **contrastive (tương phản)** — kiểu học kéo các cặp “giống nhau” lại gần và đẩy các cặp “khác nhau” ra xa trong không gian đặc trưng. Cặp giống nhau gọi là *positive*, cặp khác nhau gọi là *negative*.
- **masked modeling** — che một phần đầu vào rồi bắt mô hình khôi phục phần bị che. Vì đáp án chính là dữ liệu gốc, không cần nhãn nào cả.
- **zero-shot** — dùng mô hình cho một lớp hoặc một tác vụ mà nó chưa từng được huấn luyện trên đó, chỉ bằng cách mô tả lớp ấy bằng ngôn ngữ.

### A. Đặt vấn đề

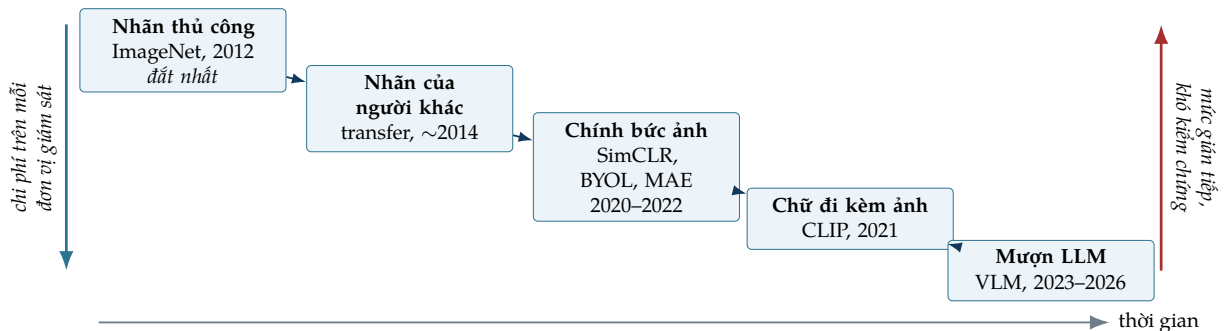
Nhãn đất, dữ liệu không nhãn thì gần như vô hạn. Một bộ ảnh nội soi có nhãn tổn thương cần bác sĩ ngồi vẽ *mask*; một bộ ảnh vệ tinh có nhãn loại đất cần chuyên gia đối chiếu thực địa. Trong khi đó, ổ cứng của cùng bệnh viện hay cùng cơ quan trắc địa chứa hàng triệu ảnh chưa ai chạm vào. Toàn bộ mục này là chuỗi câu trả lời ngày càng táo bạo cho một câu hỏi duy nhất: *tín hiệu giám sát miễn phí có thể lấy từ đâu?*

Câu hỏi nghe như một mẹo tiết kiệm chi phí, nhưng nó thực ra là câu hỏi về bản chất của việc học. Nhãn do người viết ra không phải thứ duy nhất mang thông tin về thế giới. Thông tin còn nằm ở ba chỗ khác. Hai bức ảnh chụp cùng một vật phải cho đặc trưng giống nhau. Phần bị che của một bức ảnh đoán được từ phần còn lại. Và dòng chú thích dưới một bức ảnh trên internet nói về đúng bức ảnh đó. Mỗi lần ngành khai thác được một nguồn như vậy, chi phí trên mỗi đơn

vị giám sát giảm một bậc. Cái giá cũng tăng một bậc: tín hiệu gián tiếp hơn, nhiều hơn, và khó truy ra mô hình đã học gì hơn.

**Học xong mục này thì làm được gì mà trước đó không làm được?** Bạn nhìn một bài toán có 300 ảnh nhân và quyết định ngay: đóng băng hay tinh chỉnh, chọn *backbone* tự giám sát nào, và công thức *augmentation* nào sẽ phá hỏng chính tín hiệu bạn cần. Đoán mò ba quyết định đó thì mỗi cái tốn vài ngày GPU mới lộ ra là sai.

Hình 7.2 đặt cả chuỗi lên một trục thời gian trước khi ta đi vào từng bước. Điều đáng nhìn không phải các mốc năm mà là *hai mũi tên ngược chiều nhau* ở hai bên: chi phí đi xuống trong khi mức gián tiếp đi lên. Toàn bộ chương là câu chuyện về việc đánh đổi giữa hai trục ấy.



**Hình 7.2.** Bốn nguồn giám sát trên một trục thời gian. Mỗi bước đi xuống một bậc chi phí và đi lên một bậc gián tiếp; không bước nào miễn phí thật sự, chúng chỉ dời hoá đơn từ “thuê người gắn nhãn” sang “không biết mô hình đã học gì”.

### TRỰC GIÁC

Nghĩ về nó như bài toán trắc địa. Để dựng một bản đồ, bạn không cần toạ độ tuyệt đối của từng cột mốc — một việc đắt như thuê máy đo GNSS cho mỗi điểm. Bạn chỉ cần đủ nhiều *ràng buộc tương đối* giữa các cột mốc (khoảng cách, góc), rồi giải hệ; hình dạng mạng lưới hiện ra chính xác, chỉ còn thiếu một phép biến đổi toàn cục — *gauge* — mà một vài điểm chuẩn rẻ tiền là ghim lại được.

Học tự giám sát làm đúng như vậy trong không gian đặc trưng. Mỗi cặp *augmentation* của cùng một ảnh là một ràng buộc “hai điểm này phải gần nhau”; mỗi cặp ảnh khác nhau là ràng buộc “hai điểm này phải xa nhau”. Giải cả hệ ra được một không gian đặc trưng có hình dạng đúng nhưng chưa gắn nhãn — tức là xác định tới một *gauge*. Vài trăm ảnh có nhãn sau đó đóng vai trò các điểm chuẩn: *linear probe* chính là bước ghim *gauge*, và nó rẻ vì nó chỉ phải xác định một phép biến đổi chứ không phải toàn bộ hình dạng.

### Mạch chính — bốn nguồn giám sát, theo thứ tự rẻ dần

- 1 Vấn đề** gắn nhãn cho bài toán của mình quá đắt, nhưng người khác đã gắn nhãn cho bài toán của họ.

**Giải pháp** **học chuyển giao** (*transfer learning*): lấy trọng số đã học trên ImageNet [128] làm điểm khởi đầu thay vì khởi tạo ngẫu nhiên.

**Giả định** các tầng đầu của mạng học những thứ chung cho mọi ảnh tự nhiên — cạnh, kết cấu, gradient màu — nên chúng không phụ thuộc vào việc nhãn đầu ra là gì.

**Giá phải trả** bạn thừa hưởng cả những thiên lệch của nguồn, kể cả những thiên lệch bạn không muốn.
- 2 Vấn đề** ngay cả nhãn của người khác cũng có giới hạn, và miền của bạn có thể chẳng ai gắn nhãn.



|                     |                                                                                                                                                                                                                                                                                                                                                                               |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Giải pháp</b>    | <b>học tự giám sát</b> ( <i>self-supervised learning</i> ): tạo bài toán phụ mà đáp án suy ra được từ chính bức ảnh.                                                                                                                                                                                                                                                          |
| <b>Giả định</b>     | cấu trúc thống kê của ảnh đủ giàu để ép mô hình học biểu diễn có nghĩa khi ta buộc nó giải bài toán phụ đó.                                                                                                                                                                                                                                                                   |
| <b>Vấn đề mới</b>   | bài toán phụ có nghiệm tầm thường — mô hình xuất ra một hằng số là thoả mọi ràng buộc “giống nhau”. Chống lại sụp biểu diễn ( <i>collapse</i> : mọi ảnh cho cùng một vector) này là toàn bộ nội dung kỹ thuật của mảng này.                                                                                                                                                   |
| <b>3 Vấn đề</b>     | ảnh tự nó không nói cho ta biết ngữ nghĩa; nó chỉ nói về tương quan pixel.                                                                                                                                                                                                                                                                                                    |
| <b>Giải pháp</b>    | CLIP và tiền huấn luyện <b>thị giác-ngôn ngữ</b> ( <i>vision-language</i> ) [129]: dùng dòng chú thích đi kèm ảnh trên internet làm giám sát.                                                                                                                                                                                                                                 |
| <b>Hệ quả</b>       | ngôn ngữ trở thành nguồn giám sát ngữ nghĩa rẻ nhất từng có, và nó mang theo một tập nhãn <i>mở</i> thay vì 1000 lớp cố định.                                                                                                                                                                                                                                                 |
| <b>Giá phải trả</b> | dữ liệu web bẩn, thiên lệch, và không kiểm toán được.                                                                                                                                                                                                                                                                                                                         |
| <b>4 Vấn đề</b>     | mô hình vẫn chỉ nhận dạng, chưa suy luận.                                                                                                                                                                                                                                                                                                                                     |
| <b>Giải pháp</b>    | <b>mô hình thị giác-ngôn ngữ</b> ( <i>vision-language model, VLM</i> ) — nối một <i>vision encoder</i> vào một LLM đã biết suy luận qua một tầng chiếu hoặc <i>cross-attention</i> [130], [131].                                                                                                                                                                              |
| <b>Giả định</b>     | biểu diễn thị giác có thể được “phiên dịch” thành thứ mà không gian <i>token</i> của LLM hiểu được, và phần suy luận không cần học lại.                                                                                                                                                                                                                                       |
| <b>Giá phải trả</b> | đắt, khó kiểm soát, và <b>ảo giác</b> ( <i>hallucination</i> ) của phần ngôn ngữ lan sang phần thị giác.                                                                                                                                                                                                                                                                      |
| <b>Hệ quả</b>       | tích lũy của cả bốn bước: <b>mô hình nền tảng</b> ( <i>foundation model</i> ) trở thành một <i>thành phần dựng sẵn</i> , giống thư viện BLAS chứ không giống một mô hình. Nhiều khi dùng đặc trưng đã tiền huấn luyện tốt hơn hẳn tự train, <i>kể cả trong lĩnh vực hẹp</i> như ảnh y tế hay ảnh vệ tinh. Điều này trái trực giác, nhưng đã lặp lại đủ nhiều lần để tin [47]. |

## B. Nguồn 1 — nhãn của người khác (Transfer Learning)

Học chuyển giao là kỹ thuật cũ nhất và vẫn là kỹ thuật đáng thử đầu tiên. Điều đáng nói không phải “nó hoạt động”, mà là *quyết định thực tế gói gọn trong một bảng bốn ô*. Học thuộc bảng này tiết kiệm rất nhiều thí nghiệm vô ích.

**Bảng 7.1.** Bốn ô quyết định của học chuyển giao. Trục dọc là khoảng cách miền, trục ngang là lượng dữ liệu có nhãn của bạn.

|                         | Dữ liệu ít                                               | Dữ liệu nhiều                                   |
|-------------------------|----------------------------------------------------------|-------------------------------------------------|
| <b>Miền giống nguồn</b> | <i>Linear probe</i> trên đặc trưng đóng băng             | <i>Fine-tune</i> toàn bộ                        |
| <b>Miền khác nguồn</b>  | <i>Fine-tune</i> vài tầng cuối, <i>augmentation</i> mạnh | <i>Fine-tune</i> toàn bộ; cân nhắc train từ đầu |

**Nói gọn thì** logic bên dưới Bảng 7.1 chỉ có một câu: *số tham số bạn cho phép thay đổi phải tương xứng với lượng thông tin mới mà dữ liệu của bạn mang tới*. Đây chính là trục đánh đổi giả định–dữ liệu đã gặp ở B/04-nen-tang-hoc-tu-du-lieu, lần này xuất hiện dưới dạng “mở bao nhiêu tầng”. Ba hệ quả cụ thể:

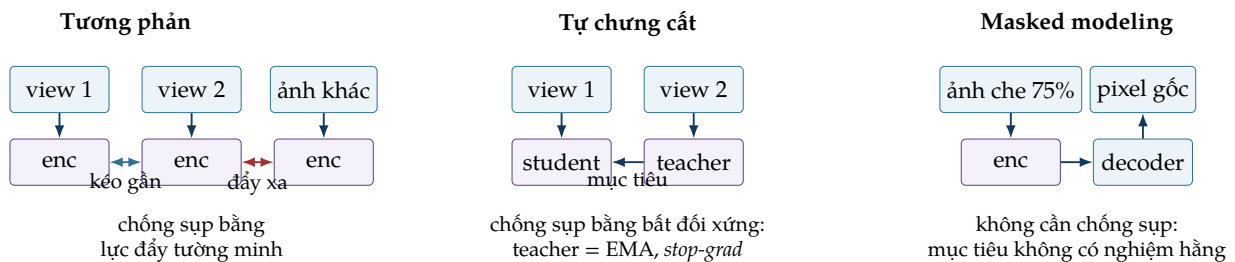
↪ **cùng nguyên lý với** B/04 — đây chính là trục đánh đổi giả định–dữ liệu, lần này xuất hiện dưới dạng “mở bao nhiêu tầng thì vừa với lượng nhãn đang có.

- **Dữ liệu ít**  $\Rightarrow$  **mở ít**. Mở toàn bộ mạng là mời gọi *overfitting* và **quên thảm khốc** (*catastrophic forgetting*) — mạng quên đặc trưng tổng quát để chạy theo vài trăm mẫu.
- **Miền xa**  $\Rightarrow$  **thay tầng cuối**. Các tầng cuối chứa thông tin chuyên biệt của nguồn nên phải thay; các tầng đầu vẫn dùng được vì cạnh vẫn là cạnh.
- **Cả hai điều kiện thuận**  $\Rightarrow$  **tiền huấn luyện chỉ còn là khởi tạo tốt**. Khi bạn có nhiều dữ liệu và miền xa, lợi ích của tiền huấn luyện thu hẹp dần và đôi khi biến mất.

Một chi tiết dễ bị bỏ qua: chất lượng mô hình nguồn trên ImageNet có tương quan với chất lượng chuyển giao, nhưng tương quan đó *yếu đi rõ rệt* khi miền đích xa [132]. Nói cách khác, đừng chọn *backbone* chỉ vì nó đứng cao trên bảng ImageNet; với ảnh nội soi hay ảnh radar, thứ hạng đó gần như không mang thông tin.

### C. Nguồn 2 – chính bức ảnh (Self-Supervised Learning)

Học tự giám sát tách thành ba triết lý khác nhau về cùng một câu hỏi: làm sao tạo tín hiệu từ hư không mà không rơi vào nghiệm tầm thường. Hình 7.3 cho thấy ba *cấu trúc đồ thị tính toán* khác nhau, và chính sự khác nhau về cấu trúc — chứ không phải về **hàm mất mát** (*loss function*) — mới là điều đáng nhớ.



**Hình 7.3.** Ba triết lý tự giám sát khác nhau ở *cấu trúc đồ thị*, không ở hàm mất mát. Tương phản chống sụp bằng lực đẩy từ *negative*; tự chưng cất chống sụp bằng bất đối xứng kiến trúc; *masked modeling* không cần chống sụp vì khôi phục pixel không có nghiệm hằng số.

**Bảng 7.2.** Ba họ tự giám sát. Cột cuối là cột đáng tiền nhất.

| Triết lý                                                            | Cơ chế chống sụp                                                                                                                   | Giả định ngầm                                               | Hỏng khi nào                                                                                                 |
|---------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| <b>Tương phản</b><br>( <i>contrastive</i> )<br>(SimCLR, MoCo)       | Lực đẩy tường minh từ rất nhiều <i>negative</i>                                                                                    | Hai ảnh khác nhau trong <i>batch</i> thì thực sự khác lớp   | Dữ liệu ít lớp hoặc lặp nhiều: <i>negative</i> hoá ra là <i>positive</i> ; <i>batch</i> nhỏ làm tín hiệu yếu |
| <b>Tự chưng cất</b><br>( <i>self-distillation</i> )<br>(BYOL, DINO) | Bất đối xứng kiến trúc: teacher <i>EMA</i> (trung bình trượt, cập nhật chậm theo student), <i>stop-gradient</i> , <i>centering</i> | Điểm cố định của động lực học huấn luyện không phải hằng số | Cơ chế mong manh: đổi tốc độ <i>EMA</i> hay bỏ <i>predictor</i> là sụp; rất nhạy siêu tham số                |
| <b>Masked modeling</b><br>(MAE)                                     | Không cần: khôi phục pixel không có nghiệm tầm thường                                                                              | Ngữ nghĩa nằm trong khả năng nội suy phần bị che            | Đặc trưng thô, <i>linear probe</i> kém; phải <i>fine-tune</i> mới mạnh — tức đắt hơn ở phía sau              |

Bảng 7.2 nói rằng ba họ không cạnh tranh trên cùng một trục. Tương phản [133], [134] và tự chưng cất [135], [136] cho đặc trưng dùng được ngay với *linear probe*, hợp khi bạn định đóng băng *backbone*. Masked modeling [137] cho đặc trưng cần tinh chỉnh, hợp khi bạn có đủ dữ liệu đích và muốn tận dụng khả năng mở rộng của ViT.

*Ví dụ nhỏ nhất: nhiệt độ quyết định gradient nằm ở đâu*

Lấy một *batch* bốn ảnh. Đặc trưng đã chuẩn hoá, **độ tương đồng cosine** (*cosine similarity*) của cặp dương là 0,9 và của ba cặp âm đều là 0,1. Hàm InfoNCE [138] cho một mẫu là

$$L = -\log \frac{e^{s^+/\tau}}{e^{s^+/\tau} + \sum_j e^{s_j^-/\tau}}.$$

Đọc bằng lời: đây là cross-entropy của một bài toán phân loại mà “lớp đúng” là bạn đồng hành của chính mình, còn  $\tau$  quyết định phân phối đó nhọn tới mức nào. Thay số cho ba giá trị **nhiệt độ** (*temperature*):

- $\tau = 1$ :  $L = -\log(2,460/5,775) = 0,853$
- $\tau = 0,5$ :  $L = -\log(6,050/9,714) = 0,474$
- $\tau = 0,1$ :  $L = -\log(8103,1/8111,2) = 0,0010$

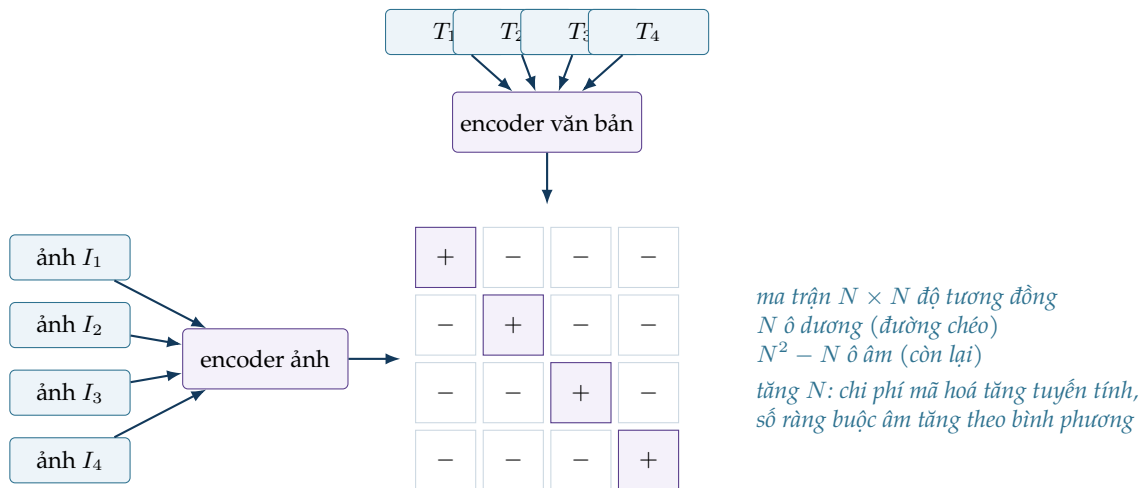
Ba con số nói rõ một điều mà công thức tổng quát giấu đi. Ở nhiệt độ thấp, một cặp đã tách được ở mức tầm thường (0,9 so với 0,1) đóng góp gần như *không* gradient; toàn bộ gradient dồn vào những *negative* có độ tương đồng gần với *positive*. Nhiệt độ vì thế không phải siêu tham số trang trí — nó là núm vặn quyết định hàm mất mát chú ý tới *negative* khó tới mức nào. Vặn quá thấp thì mô hình chỉ học từ vài cặp khó nhất và trở nên nhạy với nhiễu ngẫu nhiên; vặn quá cao thì mọi cặp đóng góp như nhau và biểu diễn không tách được các lớp gần nhau. Đây đúng là cơ chế đã gặp ở B/09-thiet-ke-ham-mat-mat dưới tên *hard negative mining*, chỉ khác là ở đây nó được điều khiển bằng một số thực thay vì bằng một bộ lọc.

↔ **cùng nguyên lý với** B/09 — nhiệt độ dồn gradient về các cặp khó chính là hard negative mining, chỉ khác là ở đây nó được điều khiển bằng một số thực thay vì bằng một bộ lọc.

#### D. Nguồn 3 — văn bản đi kèm ảnh (Vision-Language Pretraining)

CLIP giữ nguyên ý tưởng tương phản nhưng đổi định nghĩa cặp dương: thay vì hai *augmentation* của cùng một ảnh, cặp dương là (ảnh, dòng chú thích của chính nó), và cặp âm là mọi tổ hợp chéo còn lại trong *batch*. Với *batch* kích thước  $N$ , một lượt tính cho ma trận  $N \times N$  độ tương đồng và  $2N$  bài toán phân loại  $N$  lớp — đó là lý do CLIP thêm *batch* lớn tới mức bất thường.

Hình 7.4 cho thấy vì sao một *batch* lớn lại quan trọng đến thế: mỗi lượt chỉ có  $N$  ô dương trên đường chéo nhưng  $N^2 - N$  ô âm, nên tăng  $N$  làm số ràng buộc âm tăng theo bình phương trong khi chi phí mã hoá chỉ tăng tuyến tính.



**Hình 7.4.** CLIP đối định nghĩa cặp dương: thay vì hai *augmentation* của cùng một ảnh, cặp dương là (ảnh, chú thích của chính nó). Một lượt cho  $N$  bài toán phân loại theo hàng và  $N$  bài theo cột trên cùng một ma trận — đó là toàn bộ lý do CLIP thêm *batch* lớn bất thường.

Nhận thức then chốt không nằm ở kiến trúc mà ở tính chất của tín hiệu:

- **Nhãn ImageNet** là một hàm từ ảnh vào một tập 1000 phần tử *cố định*.
- **Chú thích văn bản** là một hàm vào một không gian ngữ nghĩa *mở*.

Hệ quả trực tiếp là phân loại *zero-shot*: muốn thêm một lớp mới, bạn viết thêm một câu chứ không gán thêm nhãn. Hệ quả xa hơn là toàn bộ dòng **phát hiện từ vựng mở** (*open-vocabulary detection*) và phân vùng ở chương 14. Chúng chỉ ghép bộ đề xuất vùng với không gian *embedding* văn bản này, nên mọi điểm yếu của CLIP di truyền nguyên vẹn sang đó.

↪ **hệ quả của C/14** — phát hiện từ vựng mở là bộ đề xuất vùng cắm vào chính không gian embedding này, nên nó thừa hưởng nguyên vẹn điểm yếu của CLIP về quan hệ không gian và đếm.

Cái giá thì cụ thể và đã được đo nhiều lần. Mô hình học từ chú thích web hành xử gần với một túi từ hơn là một bộ phân tích cú pháp: nó phân biệt tốt “có con chó” với “có con mèo”, nhưng yếu ở quan hệ không gian (“con mèo bên trái con chó”), ở đếm số lượng, và ở phủ định. Kết quả *zero-shot* còn nhạy với cách viết *prompt* tới mức khó chịu. Chênh lệch giữa “a photo of a {}” và “{}” có thể tới vài điểm phần trăm. Hãy đọc con số đó như dấu hiệu mô hình chưa ổn định, không phải như một mẹo tối ưu.

#### E. Nguồn 4 — ghép thị giác vào một LLM (Multimodal LLM)

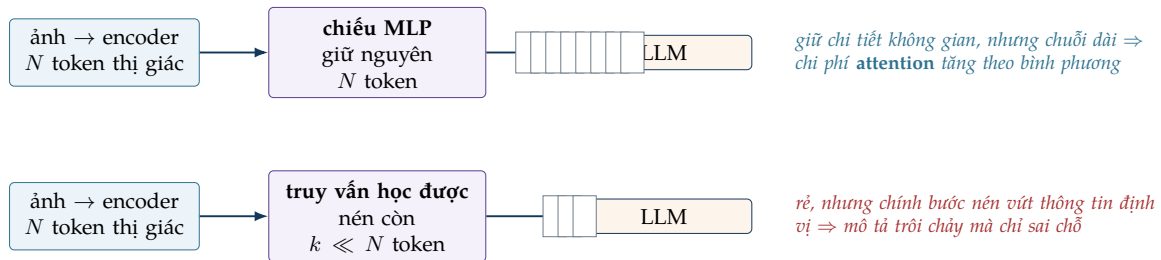
Bước tiếp theo hỏi: nếu đã có một mô hình ngôn ngữ biết suy luận, tại sao phải dạy lại phần suy luận cho thị giác? Kiến trúc VLM vì thế rất đơn giản về ý tưởng và gồm đúng ba khối:

1. **Vision encoder** — thường là CLIP hoặc SigLIP — biến ảnh thành một chuỗi *token* thị giác.
2. **Khối kết nối** — một MLP chiếu, hoặc một tập truy vấn học được kiểu Q-Former [130] — đưa chuỗi đó về cùng không gian với *token* văn bản.
3. **LLM** — xử lý chuỗi hỗn hợp hệt như xử lý văn bản thuần [131].

Trọng tâm thiết kế nằm ở khối thứ hai, và nó là một cuộc đánh đổi kinh điển. Chiếu MLP thẳng giữ nguyên số *token* thị giác nên giữ được chi tiết không gian, nhưng làm chuỗi dài ra và chi phí *attention* tăng theo bình phương. Nén xuống một số ít *token* truy vấn thì rẻ, nhưng chính bước nén đó vứt đi thông tin định vị — và đó là lý do nhiều VLM mô tả ảnh trôi chảy nhưng chỉ sai vị trí. Cái được là VQA, *grounding*, *captioning* và điều khiển giao diện bằng một mô hình duy

nhất; cái mất là chi phí, khả năng kiểm soát, và một dạng lỗi mới: mô hình bịa ra chi tiết không có trong ảnh với đúng giọng tự tin mà phần ngôn ngữ đã học được.

Hình 7.5 đặt hai biến thể của khối kết nối cạnh nhau, vì chính chỗ này quyết định VLM của bạn định vị được hay không.



**Hình 7.5.** Hai cách nối bộ mã hoá thị giác vào một LLM. Số ô nhỏ giữa khối kết nối và LLM là số **token** thị giác thực sự đi vào chuỗi; đó là đại lượng duy nhất quyết định cả chi phí lẫn khả năng định vị, nên đây là lựa chọn thiết kế quan trọng nhất của một VLM.

## F. Kết quả tích lũy – mô hình nền tảng như một thành phần

Điều đáng ngạc nhiên nhất của cả mục không phải là mô hình nền tảng mạnh, mà là nó mạnh ở *những nơi lẽ ra nó không nên mạnh*. Lấy đặc trưng DINOv2 hoặc DINOv3 đóng băng rồi gắn thêm một đầu tuyến tính, bạn thường thắng một mạng train từ đầu ngay trên dữ liệu chuyên ngành [47]. Điều đó đúng với ảnh nội soi, ảnh vệ tinh và ảnh vi thể, ở mức vài trăm tới vài nghìn nhãn. Trực giác thông thường nói dữ liệu trong miền phải thắng dữ liệu ngoài miền; thực nghiệm nói vài trăm mẫu không đủ để học lại những thống kê ảnh cơ bản mà 140 triệu ảnh đã học hộ.

Điều này thay đổi vị trí của bạn trong hệ thống: bạn không còn là người huấn luyện mô hình mà là người *tích hợp* nó — giống hệt việc dùng một solver phi tuyến có sẵn thay vì tự viết Levenberg–Marquardt. Và cũng giống hệt như vậy, trách nhiệm chuyển từ “mô hình có đúng không” sang “tôi có biết mô hình này giả định gì và hỏng ở đâu không”.

## G. Giới hạn và trường hợp hỏng

- **“Mở trọng số” không đồng nghĩa với “mở”.** Một số *backbone* tự giám sát mạnh nhất phát hành kèm giấy phép riêng hạn chế dùng thương mại, và dữ liệu huấn luyện thì hầu như không bao giờ được công bố. Với sản phẩm, đây là ràng buộc cứng cần kiểm tra *trước* khi chạy thí nghiệm.
- **Chuyển giao thất bại có hệ thống khi vật lý tạo ảnh khác nguồn** — ảnh nhiệt, radar, ảnh sự kiện, đa phổ. Ở đó thống kê tần số, phân bố nhiễu và cả số kênh đều khác, nên giả định “cạnh vẫn là cạnh” không còn đúng (xem A/02-vat-ly-tao-anh/04-cam-bien-khac-va-domain-gap).
- **Tự giám sát không miễn nhiễm với thiên lệch dữ liệu; nó chỉ làm thiên lệch khó nhìn hơn**, vì không còn bảng phân bố nhãn để kiểm tra. Nếu 90% ảnh không nhãn của bạn chụp ban ngày, biểu diễn học ra sẽ tốt ban ngày và không ai cảnh báo bạn.
- **Đánh giá bằng *linear probe* che giấu rất nhiều.** Một biểu diễn có thể tuyến tính phân tách tốt các lớp phổ biến mà vẫn vô dụng cho định vị chính xác — đúng loại thất bại quan trọng nhất nếu đầu ra của bạn là hộp, *mask* hay tương ứng điểm.

### CẠM BẤY

Bất biến mà bạn ép vào *augmentation* chính là thông tin bạn cố tình rút đi. *Color jitter* mạnh dạy mô hình rằng màu không quan trọng — một thảm họa nếu tác vụ đích là phát

hiện gỉ sét, phân loại nhuộm mô bệnh học, hay đọc đèn tín hiệu. *Random resized crop* dạy mô hình rằng tỉ lệ không quan trọng — một thảm họa nếu bạn cần ước lượng kích thước thật. Trước khi sao chép một công thức *augmentation* từ paper tự giám sát, hãy liệt kê các bất biến nó áp đặt và đối chiếu từng cái với thứ tác vụ của bạn cần *phân biệt*.

#### CẦN NHỚ

Mỗi bước trong chuỗi này đòi *nhân đất và chính xác lấy tín hiệu rẻ và gián tiếp*. Nó gần như luôn có lỗi, nhưng cái bạn mất là khả năng biết mô hình đã học gì — và bạn chỉ phát hiện ra điều đó ở đúng những trường hợp biên mà bạn quan tâm nhất. Hệ quả hành động: với mọi mô hình nền tảng, hãy tự thu một tập đánh giá nhỏ trên đúng phân phối triển khai trước khi tin bất kỳ con số nào. [T2]

## H. Kết nối

Mục này là tiền đề trực tiếp của [C/14-bai-toan-cv-loi](#): phát hiện từ vụng mở chỉ là bộ đề xuất vùng cấm vào không gian *embedding* CLIP. Nó đối lập với [B/06-chat-luong-du-lieu](#). Chương kia lập luận rằng chất lượng nhãn quyết định trần hiệu năng; mục này lập luận rằng phần lớn tín hiệu lấy được mà không cần nhãn. Cả hai đều đúng, và ranh giới nằm ở chỗ bạn cần *ngữ nghĩa* hay cần *biểu diễn*. Về phía sau, [C/18/02-kinh-te-hoc-pretraining](#) trả lời câu hỏi định lượng: đầu tư compute vào đâu, và khi nào tinh chỉnh đáng giá hơn đóng băng.

### Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao BYOL không sụp về nghiệm hằng số dù không có một *negative* nào, và bỏ thành phần nào thì nó sụp?
2. Với 500 ảnh nội soi, *linear probe* trên DINOv3 hay *fine-tune* toàn bộ ResNet ImageNet sẽ thắng, và lập luận đó dựa trên đại lượng nào — “số tham số được phép thay đổi so với lượng thông tin mới” nói gì ở đây?
3. Hạ nhiệt độ  $\tau$  trong InfoNCE làm gradient dịch chuyển đi đâu, và tác dụng phụ nào xuất hiện khi tập dữ liệu có nhiều ảnh gần trùng nhau?
4. MAE cho *linear probe* kém hơn DINO nhưng tinh chỉnh lại tốt hơn. Điều đó nói gì về loại thông tin mà hai mục tiêu huấn luyện giữ lại?
5. Vì sao “mở trọng số” không đồng nghĩa với “mở”, và điều đó ảnh hưởng gì tới lựa chọn *backbone* cho một sản phẩm thương mại?
6. Tác vụ nào sẽ bị công thức *augmentation* chuẩn của SimCLR làm hỏng biểu diễn, và bất biến nào bị ép sai ở đó?



- [1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [2] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [3] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *AISTATS*, 2010.
- [4] K. He, X. Zhang, S. Ren, et al., “Delving Deep into Rectifiers,” in *ICCV*, 2015.
- [5] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [6] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [7] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [8] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [9] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [10] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [11] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” in *ICLR*, 2017.
- [12] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [13] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [14] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [15] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [16] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [17] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [18] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [19] G. Huang, Y. Sun, Z. Liu, et al., “Deep Networks with Stochastic Depth,” in *ECCV*, 2016.
- [20] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [21] H. Zhang, M. Cisse, Y. N. Dauphin, et al., “mixup: Beyond Empirical Risk Minimization,” in *ICLR*, 2018.
- [22] S. Yun, D. Han, S. J. Oh, et al., “CutMix: Regularization Strategy to Train Strong Classifiers,” in *ICCV*, 2019.
- [23] E. D. Cubuk, B. Zoph, J. Shlens, et al., “RandAugment: Practical Automated Data Augmentation,” in *CVPR Workshops*, 2020.

- [24] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [25] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [26] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *NeurIPS*, 2012.
- [27] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in *ICLR*, 2015.
- [28] C. Szegedy, W. Liu, et al., “Going Deeper with Convolutions,” in *CVPR*, 2015.
- [29] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [30] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [31] G. Huang, Z. Liu, L. van der Maaten, et al., “Densely Connected Convolutional Networks,” in *CVPR*, 2017.
- [32] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [33] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [34] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [35] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [36] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [37] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.
- [38] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [39] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [40] J. Su, M. Ahmed, Y. Lu, et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, vol. 568, 2024.
- [41] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” *arXiv:2312.00752*, 2023.
- [42] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [43] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.
- [44] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [45] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [46] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [47] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [48] R. Girshick, J. Donahue, T. Darrell, et al., “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation,” in *CVPR*, 2014.

- [49] R. Girshick, "Fast R-CNN," in *ICCV*, 2015.
- [50] S. Ren, K. He, R. Girshick, et al., "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in *NeurIPS*, 2015.
- [51] T.-Y. Lin, P. Dollár, R. Girshick, et al., "Feature Pyramid Networks for Object Detection," in *CVPR*, 2017.
- [52] K. He, G. Gkioxari, P. Dollár, et al., "Mask R-CNN," in *ICCV*, 2017.
- [53] D. Bolya, S. Foley, J. Hays, et al., "TIDE: A General Toolbox for Identifying Object Detection Errors," in *ECCV*, 2020.
- [54] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [55] J. Redmon, S. Divvala, R. Girshick, et al., "You Only Look Once: Unified, Real-Time Object Detection," in *CVPR*, 2016.
- [56] W. Liu, D. Anguelov, D. Erhan, et al., "SSD: Single Shot MultiBox Detector," in *ECCV*, 2016.
- [57] T.-Y. Lin, P. Goyal, R. Girshick, et al., "Focal Loss for Dense Object Detection," in *ICCV*, 2017.
- [58] Z. Tian, C. Shen, H. Chen, et al., "FCOS: Fully Convolutional One-Stage Object Detection," in *ICCV*, 2019.
- [59] X. Zhou, D. Wang, and P. Krähenbühl, "Objects as Points," *arXiv:1904.07850*, 2019.
- [60] S. Zhang, C. Chi, Y. Yao, et al., "Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection," in *CVPR*, 2020.
- [61] Z. Ge, S. Liu, Z. Li, et al., "OTA: Optimal Transport Assignment for Object Detection," in *CVPR*, 2021.
- [62] N. Bodla, B. Singh, R. Chellappa, et al., "Soft-NMS: Improving Object Detection with One Line of Code," in *ICCV*, 2017.
- [63] X. Zhu, W. Su, L. Lu, et al., "Deformable DETR: Deformable Transformers for End-to-End Object Detection," in *ICLR*, 2021.
- [64] H. Zhang, F. Li, S. Liu, et al., "DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection," in *ICLR*, 2023.
- [65] J. Long, E. Shelhamer, and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," in *CVPR*, 2015.
- [66] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *MICCAI*, 2015.
- [67] L.-C. Chen, G. Papandreou, I. Kokkinos, et al., "DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs," *IEEE TPAMI*, vol. 40, no. 4, pp. 834–848, 2018.
- [68] E. Xie, W. Wang, Z. Yu, et al., "SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers," in *NeurIPS*, 2021.
- [69] B. Cheng, I. Misra, A. G. Schwing, et al., "Masked-Attention Mask Transformer for Universal Image Segmentation," in *CVPR*, 2022.
- [70] A. Kirillov, K. He, R. Girshick, et al., "Panoptic Segmentation," in *CVPR*, 2019.
- [71] A. Kirillov, E. Mintun, N. Ravi, et al., "Segment Anything," in *ICCV*, 2023.
- [72] K. Sun, B. Xiao, D. Liu, et al., "Deep High-Resolution Representation Learning for Human Pose Estimation," in *CVPR*, 2019.

- [73] Z. Cao, T. Simon, S.-E. Wei, et al., “Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields,” in *CVPR*, 2017.
- [74] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [75] J. Deng, J. Guo, N. Xue, et al., “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *CVPR*, 2019.
- [76] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [77] K. Xu, J. Ba, R. Kiros, et al., “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention,” in *ICML*, 2015.
- [78] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *ICLR*, 2015.
- [79] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [80] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [81] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [82] K. Simonyan and A. Zisserman, “Two-Stream Convolutional Networks for Action Recognition in Videos,” in *NeurIPS*, 2014.
- [83] J. Carreira and A. Zisserman, “Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset,” in *CVPR*, 2017.
- [84] D. Tran, H. Wang, L. Torresani, et al., “A Closer Look at Spatiotemporal Convolutions for Action Recognition,” in *CVPR*, 2018.
- [85] C. Feichtenhofer, H. Fan, J. Malik, et al., “SlowFast Networks for Video Recognition,” in *ICCV*, 2019.
- [86] A. Dosovitskiy, P. Fischer, E. Ilg, et al., “FlowNet: Learning Optical Flow with Convolutional Networks,” in *ICCV*, 2015.
- [87] D. Sun, X. Yang, M.-Y. Liu, et al., “PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume,” in *CVPR*, 2018.
- [88] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.
- [89] A. Bewley, Z. Ge, L. Ott, et al., “Simple Online and Realtime Tracking,” in *ICIP*, 2016.
- [90] Y. Zhang, P. Sun, Y. Jiang, et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *ECCV*, 2022.
- [91] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [92] K. Bernardin and R. Stiefelhagen, “Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics,” *EURASIP J. Image and Video Processing*, 2008.
- [93] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [94] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [95] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.

- [96] S. Wang, V. Leroy, Y. Cabon, et al., "DUST3R: Geometric 3D Vision Made Easy," in *CVPR*, 2024.
- [97] D. Eigen, C. Puhrsch, and R. Fergus, "Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network," in *NeurIPS*, 2014.
- [98] R. Ranftl, K. Lasinger, D. Hafner, et al., "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer," *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [99] J. Sturm, N. Engelhard, F. Endres, et al., "A Benchmark for the Evaluation of RGB-D SLAM Systems," in *IROS*, 2012.
- [100] C. R. Qi, H. Su, K. Mo, et al., "PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation," in *CVPR*, 2017.
- [101] C. R. Qi, L. Yi, H. Su, et al., "PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space," in *NeurIPS*, 2017.
- [102] A. H. Lang, S. Vora, H. Caesar, et al., "PointPillars: Fast Encoders for Object Detection from Point Clouds," in *CVPR*, 2019.
- [103] T. Müller, A. Evans, C. Schied, et al., "Instant Neural Graphics Primitives with a Multiresolution Hash Encoding," *ACM TOG*, vol. 41, no. 4, 2022.
- [104] B. Kerbl, G. Kopanas, T. Leimkühler, et al., "3D Gaussian Splatting for Real-Time Radiance Field Rendering," *ACM TOG*, vol. 42, no. 4, 2023.
- [105] R. Zhang, P. Isola, A. A. Efros, et al., "The Unreasonable Effectiveness of Deep Features as a Perceptual Metric," in *CVPR*, 2018.
- [106] D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," in *ICLR*, 2014.
- [107] I. Goodfellow, J. Pouget-Abadie, M. Mirza, et al., "Generative Adversarial Nets," in *NeurIPS*, 2014.
- [108] T. Karras, S. Laine, and T. Aila, "A Style-Based Generator Architecture for Generative Adversarial Networks," in *CVPR*, 2019.
- [109] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," in *NeurIPS*, 2020.
- [110] Y. Song, J. Sohl-Dickstein, D. P. Kingma, et al., "Score-Based Generative Modeling Through Stochastic Differential Equations," in *ICLR*, 2021.
- [111] J. Johnson, A. Alahi, and L. Fei-Fei, "Perceptual Losses for Real-Time Style Transfer and Super-Resolution," in *ECCV*, 2016.
- [112] P. Isola, J.-Y. Zhu, T. Zhou, et al., "Image-to-Image Translation with Conditional Adversarial Networks," in *CVPR*, 2017.
- [113] L. A. Gatys, A. S. Ecker, and M. Bethge, "Image Style Transfer Using Convolutional Neural Networks," in *CVPR*, 2016.
- [114] X. Huang and S. Belongie, "Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization," in *ICCV*, 2017.
- [115] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," in *NeurIPS Deep Learning Workshop*, 2015.
- [116] J. Song, C. Meng, and S. Ermon, "Denoising Diffusion Implicit Models," in *ICLR*, 2021.
- [117] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, et al., "Flow Matching for Generative Modeling," in *ICLR*, 2023.
- [118] J. Ho and T. Salimans, "Classifier-Free Diffusion Guidance," *arXiv:2207.12598*, 2022.
- [119] R. Rombach, A. Blattmann, D. Lorenz, et al., "High-Resolution Image Synthesis with Latent Diffusion Models," in *CVPR*, 2022.
- [120] W. Peebles and S. Xie, "Scalable Diffusion Models with Transformers," in *ICCV*, 2023.



- [121] L. Zhang, A. Rao, and M. Agrawala, "Adding Conditional Control to Text-to-Image Diffusion Models," in *ICCV*, 2023.
- [122] Y. Blau and T. Michaeli, "The Perception-Distortion Tradeoff," in *CVPR*, 2018.
- [123] C. Ledig, L. Theis, F. Huszár, et al., "Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network," in *CVPR*, 2017.
- [124] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., "Image Quality Assessment: From Error Visibility to Structural Similarity," *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [125] M. Heusel, H. Ramsauer, T. Unterthiner, et al., "GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium," in *NeurIPS*, 2017.
- [126] T. Brooks, B. Mildenhall, T. Xue, et al., "Unprocessing Images for Learned Raw Denoising," in *CVPR*, 2019.
- [127] A. Abdelhamed, S. Lin, and M. S. Brown, "A High-Quality Denoising Dataset for Smartphone Cameras," in *CVPR*, 2018.
- [128] J. Deng, W. Dong, R. Socher, et al., "ImageNet: A Large-Scale Hierarchical Image Database," in *CVPR*, 2009.
- [129] A. Radford, J. W. Kim, C. Hallacy, et al., "Learning Transferable Visual Models from Natural Language Supervision," in *ICML*, 2021.
- [130] J. Li, D. Li, S. Savarese, et al., "BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models," in *ICML*, 2023.
- [131] H. Liu, C. Li, Q. Wu, et al., "Visual Instruction Tuning," in *NeurIPS*, 2023.
- [132] S. Kornblith, J. Shlens, and Q. V. Le, "Do Better ImageNet Models Transfer Better?" In *CVPR*, 2019.
- [133] T. Chen, S. Kornblith, M. Norouzi, et al., "A Simple Framework for Contrastive Learning of Visual Representations," in *ICML*, 2020.
- [134] K. He, H. Fan, Y. Wu, et al., "Momentum Contrast for Unsupervised Visual Representation Learning," in *CVPR*, 2020.
- [135] J.-B. Grill, F. Strub, F. Altché, et al., "Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning," in *NeurIPS*, 2020.
- [136] M. Caron, H. Touvron, I. Misra, et al., "Emerging Properties in Self-Supervised Vision Transformers," in *ICCV*, 2021.
- [137] K. He, X. Chen, S. Xie, et al., "Masked Autoencoders Are Scalable Vision Learners," in *CVPR*, 2022.
- [138] A. van den Oord, Y. Li, and O. Vinyals, "Representation Learning with Contrastive Predictive Coding," in *arXiv:1807.03748*, 2018.
- [139] J. Kaplan, S. McCandlish, T. Henighan, et al., "Scaling Laws for Neural Language Models," *arXiv:2001.08361*, 2020.
- [140] X. Zhai, A. Kolesnikov, N. Houlsby, et al., "Scaling Vision Transformers," in *CVPR*, 2022.
- [141] J. Hoffmann, S. Borgeaud, A. Mensch, et al., "Training Compute-Optimal Large Language Models," in *NeurIPS*, 2022.
- [142] C. G. Northcutt, A. Athalye, and J. Mueller, "Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks," in *NeurIPS Datasets and Benchmarks*, 2021.
- [143] B. Recht, R. Roelofs, L. Schmidt, et al., "Do ImageNet Classifiers Generalize to ImageNet?" In *ICML*, 2019.
- [144] N. Houlsby, A. Giurgiu, S. Jastrzebski, et al., "Parameter-Efficient Transfer Learning for NLP," in *ICML*, 2019.



- [145] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [146] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *RecSys*, 2019.

## 7.2 Kinh tế học của pretraining (The Economics of Pretraining)

**PHỤ THUỘC** C/18/01-nguon-giam-sat (mục ngay trước, bắt buộc); B/11-gioi-han-tinh-toan (compute, bằng thông, FLOPs); B/07-chia-du-lieu-va-danh-gia (vì sao nhiệm bản tập kiểm tra phá huỷ mọi kết luận); E/24-thi-nghiem-va-ablation (cách chạy thí nghiệm điểm giao cho từ tế).

### TÓM TẮT

Mục này là mặt định lượng của việc tiền huấn luyện: có một ngân sách compute cố định thì chia cho tham số và dữ liệu theo tỉ lệ nào, và khi nào tính chính đáng giá hơn đồng bằng. Đọc xong bạn ước lượng được chi phí một lần tiền huấn luyện bằng một phép nhân trên giấy. Bạn đọc được một scaling law mà không bị nó lừa. Và bạn thiết kế được thí nghiệm điểm giao cho chính dữ liệu của mình, thay vì tin lời khuyên chung. Nếu chỉ nhớ một điều: compute, tham số và dữ liệu là ba mặt của cùng một ngân sách — bài học Chinchilla là ví dụ đắt nhất lịch sử ngành về việc tối ưu một mặt mà quên hai mặt kia.

### KIẾN THỨC CẦN CÓ TRƯỚC

- **FLOPs và ngân sách compute** — FLOPs là số phép tính dấu phẩy động. Ngân sách compute  $C$  là tổng FLOPs bạn mua được cho một lần huấn luyện; nó cố định, nên chọn mô hình to hơn đồng nghĩa với chọn dữ liệu ít hơn.
- **token** — đơn vị nhỏ nhất của dữ liệu huấn luyện mà mô hình xử lý — một mẫu từ với văn bản, một mảnh ảnh (*patch*) với ViT. Lượng dữ liệu  $D$  luôn được đếm bằng token, không phải bằng số file.
- **hàm mất mát (loss)** — con số đo mức sai của mô hình trên dữ liệu; huấn luyện là quá trình giảm nó. Mọi “scaling law” trong mục này là công thức dự đoán con số đó sẽ giảm tới đâu.
- **scaling law** — quan hệ định lượng khớp từ thực nghiệm giữa loss, số tham số, lượng dữ liệu và compute. Giá trị của nó là cho phép *ngoại suy*: chạy vài mô hình nhỏ rồi dự đoán mô hình lớn trước khi tiêu tiền.
- **tập kiểm tra (test set) và tập validation** — tập validation dùng để chọn siêu tham số; tập test dùng đúng một lần để báo cáo. Trộn hai vai trò này là cách phổ biến nhất để tự lừa mình.
- **siêu tham số (hyperparameter)** — tham số bạn chọn chứ không phải mô hình học, ví dụ *learning rate* (độ dài mỗi bước cập nhật trọng số), kích thước *batch*, số *epoch*.
- **checkpoint** — ảnh chụp toàn bộ trọng số mô hình (và thường cả trạng thái optimizer) tại một thời điểm huấn luyện, lưu ra đĩa để chạy tiếp hoặc để triển khai.
- **seed** — số khởi tạo bộ sinh ngẫu nhiên. Đổi seed là đổi khởi tạo trọng số, thứ tự dữ liệu và augmentation — nên hai lần chạy khác seed cho hai kết quả khác nhau ngay cả khi mã hoàn toàn giống nhau.

### A. Đặt vấn đề

Mục trước trả lời câu hỏi *lấy giám sát ở đâu*. Mục này trả lời câu hỏi đắt tiền hơn: *có một ngân sách compute cố định thì tiêu vào đâu?* Đây không phải câu hỏi triết học — nó là câu hỏi phân bổ nguồn lực, và trong nhiều năm cả ngành đã trả lời sai nó theo cùng một hướng.

Vấn đề khó ở chỗ ba đại lượng — số tham số  $N$ , lượng dữ liệu  $D$ , và compute  $C$  — ràng buộc nhau. Thêm tham số mà không thêm dữ liệu thì mô hình có thêm chỗ chứa mà không có gì để chứa. Thêm dữ liệu mà không thêm tham số thì mô hình bão hoà. Và cả hai đều bị chặn bởi tổng số phép tính bạn mua được. Nếu chỉ có trực giác, cách duy nhất tìm điểm cân bằng là thử; nhưng mỗi lần thử ở quy mô nghìn tỉ tham số tốn hàng triệu đô la, nên thử mà không chấp nhận được. Điều ngành cần là một *luật cho phép ngoại suy*: chạy vài mô hình nhỏ, khớp một đường cong, rồi dự đoán mô hình lớn trước khi tiêu tiền.

**Học xong mục này thì làm được gì?** Ba việc mà trước đó bạn chỉ làm được bằng cách hỏi người khác. Một, tính chi phí huấn luyện của một mô hình bất kỳ bằng một phép nhân trên giấy. Hai, phát hiện khi một bảng kết quả đang so hai mô hình có ngân sách khác nhau. Ba, thiết kế thí nghiệm điểm giao trả lời đúng câu hỏi bạn đang có.

#### TRỰC GIÁC

Hãy mượn ngôn ngữ của **hiệu chỉnh chùm tia** (*bundle adjustment*), với ý thức rằng đây là phép loại suy chứ không phải định lý. Trong BA, số ảnh là toạ độ điểm và tư thế camera; số phương trình là số quan sát. Nếu bạn thêm điểm 3D mà không thêm quan sát, hệ trở nên thiếu ràng buộc: nghiệm vẫn tồn tại, sai số tái chiếu vẫn nhỏ, nhưng cấu trúc thu được là ảo — nó khớp quan sát mà không tương ứng với thế giới.

Bài học Chinchilla nói rằng cả ngành đã chạy đúng tình huống đó suốt vài năm: mô hình được thổi phồng số tham số trong khi số “quan sát” — *token* dữ liệu — gần như đứng yên. Loss huấn luyện vẫn giảm, bảng kết quả vẫn đẹp, nên không ai thấy có gì sai. Chỗ mà phép loại suy gây cũng đáng nhớ. Mạng nơ-ron quá tham số hoá vẫn tổng quát được, nên “thiếu ràng buộc” ở đây không sinh nghiệm vô nghĩa như trong BA. Nó chỉ sinh ra *lãng phí*: cùng số FLOPs ấy lẽ ra mua được mô hình tốt hơn.

#### B. Scaling laws — từ thử mù sang dự đoán (Scaling Laws)

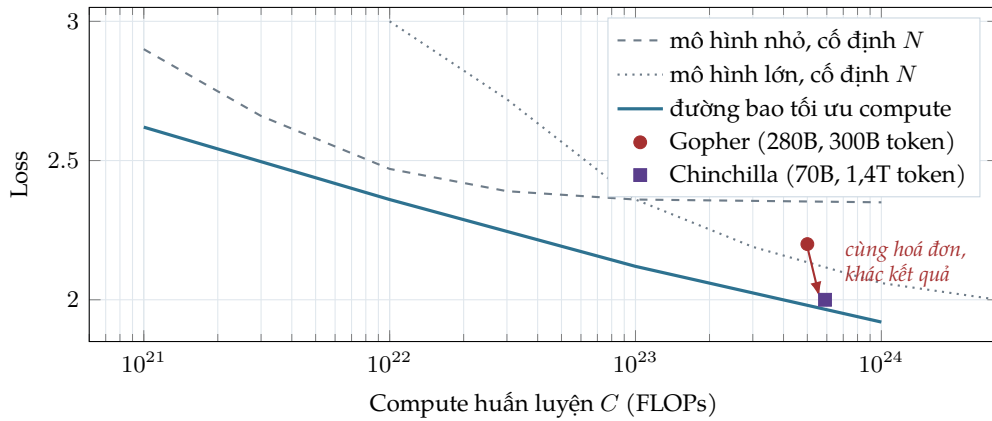
Quan sát thực nghiệm nền tảng là: khi tăng  $N$ ,  $D$  và  $C$ , loss giảm theo quan hệ lũy thừa (*power law*) chứ không theo bậc thang [139]. Dạng thường dùng là

$$L(N, D) = L_{\infty} + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}}.$$

*Công thức này thực sự nói điều gì?* Rằng loss của bạn là tổng của ba thứ độc lập. Thứ nhất là phần sàn không bao giờ giảm được,  $L_{\infty}$ : entropy thật của dữ liệu, cộng mọi thứ mà kiến trúc này về nguyên tắc không biểu diễn nổi. Thứ hai là phần phạt vì mô hình quá nhỏ. Thứ ba là phần phạt vì dữ liệu quá ít. Muốn giảm loss, bạn phải biết mình đang bị phạt ở số hạng nào — và đó chính là điều mà đường cong cho bạn biết.

Điều làm công thức có giá trị thực dụng không phải hình dạng của nó, mà là việc nó *khớp được từ các mô hình nhỏ và ngoại suy đúng lên vài bậc độ lớn*. Đó là lần đầu tiên trong lịch sử ngành, một quyết định chi tiêu chín chữ số được đưa ra dựa trên đường cong khớp từ thí nghiệm rẻ tiền. Hiện tượng tương tự đã được xác nhận cho thị giác với ViT [140], dù các hằng số hoàn toàn khác.

Hình 7.6 cho thấy vì sao lập luận này không nhìn ra được nếu chỉ theo dõi một mô hình: mỗi kích thước mô hình có đường loss riêng, và điều đáng quan tâm là *đường bao* của cả họ chứ không phải bất kỳ đường nào trong đó.



**Hình 7.6.** Đường bao tối ưu compute và hai mô hình cùng ngân sách (*trục và hình dạng minh hoạ; chỉ vị trí tương đối của Gopher với Chinchilla là dựa trên kết quả đã công bố*). Một mô hình cố định kích thước sẽ rời khỏi đường bao khi ta đổ thêm dữ liệu vào nó: loss chững lại vì nó đã hết chỗ chứa. Gopher nằm phía trên đường bao ở đúng mức compute của Chinchilla — đó là hình ảnh của việc thừa tham số và thiếu dữ liệu.

↔ **đối lập với** B/11 — chương kia đo giới hạn vật lý của phần cứng, mục này đo giới hạn kinh tế của việc dùng phần cứng đó — cùng một ngân sách nhìn từ hai phía.

Hai hệ quả cần đọc kỹ:

- **Lợi ích giảm dần nhưng không bao giờ dừng.** Luật lũy thừa không có ngưỡng mà thêm dữ liệu trở nên vô ích — chỉ có ngưỡng mà nó không còn đáng tiền. Câu hỏi đúng luôn là “đáng bao nhiêu”, không phải “còn tác dụng không”.
- **Scaling không sửa được lỗi bài toán.** Số hạng  $L_\infty$  nói rằng nếu nhân bản hoặc bài toán bị đặt sai, mọi đường cong đều chạm sàn ở một chỗ cao và không lượng compute nào cứu được.

### C. Bài học Chinchilla (The Chinchilla Lesson)

Câu hỏi tiếp theo: với  $C$  cho trước, chọn  $N$  và  $D$  thế nào? Xấp xỉ chuẩn cho **transformer** là

$$C \approx 6ND \quad (\text{FLOPs cho toàn bộ quá trình huấn luyện}),$$

với hệ số 6 đến từ hai FLOPs cho mỗi phép nhân–cộng, nhân với ba lượt (một **lượt xuôi** (*forward pass*), hai **lượt ngược** (*backward pass*)). Diễn giải bằng lời: chi phí huấn luyện tỉ lệ thuận với tích của kích thước mô hình và lượng dữ liệu — nên với ngân sách cố định, hai đại lượng ấy nằm trên một đường hyperbol và bạn chỉ được chọn một điểm trên đó. Đáp số thực nghiệm là  $N$  và  $D$  nên tăng **cùng tốc độ**, mỗi cái tỉ lệ với  $\sqrt{C}$  [141].

**Ví dụ nhỏ nhất, dùng hai mô hình đã công bố.**

- **Gopher:**  $N = 280$  tỉ tham số,  $D = 300$  tỉ **token**  $\Rightarrow C = 6 \times 2,8 \times 10^{11} \times 3 \times 10^{11} \approx 5,0 \times 10^{23}$  FLOPs. Tỉ lệ  $D/N = 1,07$ .
- **Chinchilla:**  $N = 70$  tỉ,  $D = 1400$  tỉ **token**  $\Rightarrow C = 6 \times 7 \times 10^{10} \times 1,4 \times 10^{12} \approx 5,9 \times 10^{23}$  FLOPs. Tỉ lệ  $D/N = 20$ .

Cùng bậc compute, tức cùng hoá đơn. Mô hình nhỏ hơn bốn lần nhưng ăn nhiều dữ liệu hơn gần năm lần thẳng rõ rệt trên hầu hết bài kiểm tra. Con số 20 **token** cho mỗi tham số chính là quy tắc ngón tay cái người ta nhớ.

Cơ chế của cái sai đáng học hơn con số 20. Số tham số là đại lượng dễ công bố, dễ so sánh và dễ gây ấn tượng; lượng **token** thì không ai khoe. Khi một đại lượng dễ đo trở thành đại lượng được tối ưu, nó thôi là đại lượng đúng — cùng một cơ chế đã làm hỏng nhiều bảng xếp hạng ở

E/24-thi-nghiem-va-ablation. Lưu ý thêm: con số 20 chỉ tối ưu cho *chi phí huấn luyện*. Mô hình phục vụ hàng tỉ lượt **suy luận** (*inference*) thì hoá đơn nằm ở phía chạy chứ không phải phía huấn luyện. Tối ưu tổng chi phí vòng đời vì thế đẩy bạn về mô hình *nhỏ hơn nữa*, huấn luyện lâu hơn nữa. Đó là lý do các mô hình mở gần đây thường vượt xa tỉ lệ 20.

#### D. Chất lượng hơn số lượng (Data Quality over Quantity)

Bước tiếp theo của cùng lập luận: nếu  $D$  quan trọng đến thế thì  $D$  là gì càng quan trọng hơn. Một dữ liệu bị lặp mười lần không mang thêm thông tin nào nhưng vẫn ngốn đúng số FLOPs như dữ liệu mới. Với thị giác, hiệu ứng còn mạnh hơn vì ảnh web trùng lặp ở mức độ khủng khiếp: cùng một logo, cùng một ảnh *stock*, cùng một khung hình phim xuất hiện hàng nghìn lần.

Điều này gắn thẳng vào B/06-chat-luong-du-lieu: các tập dữ liệu chuẩn của ngành có tỉ lệ nhãn sai không nhỏ, đủ để đảo thứ hạng giữa các mô hình gần nhau [142]. Khi bạn tự tuyển *corpus*, một giờ viết bộ lọc trùng lặp thường có lãi cao hơn một giờ GPU.

#### E. Nhiễm bẩn tập kiểm tra — vấn đề do chính pretraining sinh ra

Tiền huấn luyện quy mô web tạo ra một vấn đề mà các phương pháp cũ không có: **nhiễm bẩn tập kiểm tra** (*test set contamination*). Nếu ảnh hoặc câu hỏi trong tập test đã nằm đâu đó trong *corpus* tiền huấn luyện, con số đánh giá không đo khả năng tổng quát hoá mà đo khả năng ghi nhớ. Cái làm nó nguy hiểm không phải bản thân hiện tượng — ai cũng biết là xấu — mà là việc nó *không kiểm chứng được*: *corpus* thường không công bố, và ngay cả khi công bố thì đối sánh hàng tỉ ảnh với tập test cũng không rẻ.

Có một dạng tinh vi hơn và phổ biến hơn: nhiễm bẩn qua cộng đồng. Một tập test công khai được cả ngành dùng mười năm sẽ bị việc chọn siêu tham số bào mòn dần, ngay cả khi không ai nhìn trực tiếp vào nó. Khi người ta dựng lại một tập test mới theo đúng quy trình cũ, hiệu năng mọi mô hình đều tụt, và tụt gần như song song [143]. Tính song song đó là tin tốt — thứ hạng tương đối còn dùng được — nhưng giá trị tuyệt đối thì không.

Kết luận thực dụng: với bất kỳ mô hình nền tảng nào, con số *zero-shot* trên *benchmark* công khai nên coi là chặn trên lạc quan. Bằng chứng duy nhất đáng tin là tập đánh giá do chính bạn thu, sau ngày mô hình được huấn luyện, trên đúng phân phối bạn sẽ triển khai.

→ **tiền đề** B/07 — quy trình chia dữ liệu sạch là điều kiện để con số đánh giá có nghĩa; nhiễm bẩn chỉ là rò rỉ dữ liệu ở quy mô web, nơi bạn không kiểm toán được nguồn.

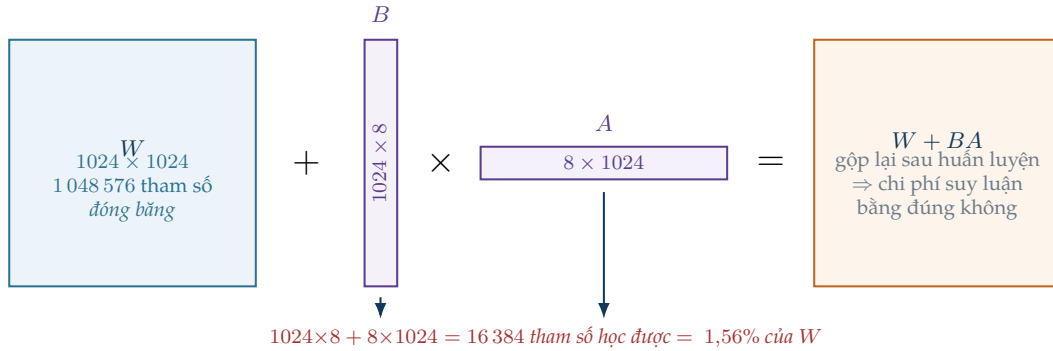
#### F. PEFT — đổi lại kinh tế học của việc thích nghi

Vấn đề rất cụ thể: tinh chỉnh toàn bộ một mô hình lớn nghĩa là lưu một bản sao toàn bộ trọng số cho mỗi khách hàng, mỗi tác vụ, mỗi phiên bản — và trạng thái *optimizer* của Adam còn tốn gấp đôi số đó. **PEFT** (tinh chỉnh chỉ một phần rất nhỏ tham số) giải bài toán này bằng cách đóng băng trọng số gốc và chỉ học một phần rất nhỏ tham số thêm vào:

- **Adapter** — chèn các khối nhỏ giữa các tầng, chỉ chúng được học [144].
- **LoRA** — phổ biến hơn: viết cập nhật dưới dạng tích hai ma trận hạng thấp,  $\Delta W = BA$  với  $B \in \mathbb{R}^{d \times r}$ ,  $A \in \mathbb{R}^{r \times k}$ ,  $r \ll \min(d, k)$  [145]. *Ý nghĩa*: ta giả định rằng “khoảng cách” giữa mô hình gốc và mô hình bạn cần nằm trong một không gian con rất thấp chiều, nên chỉ cần học toạ độ trong không gian con đó.
- **Prompt tuning** — không đụng trọng số nào, chỉ học một chuỗi *token* đầu vào. Rẻ nhất, cũng yếu nhất.

**Con số làm rõ mọi thứ.** Một ma trận trọng số  $1024 \times 1024$  có 1 048 576 tham số. LoRA hạng  $r = 8$  thay nó bằng  $1024 \times 8 + 8 \times 1024 = 16\,384$  tham số, tức 1,56%. Nhân lên toàn mô hình: một *adapter* vài chục megabyte thay cho một *checkpoint* vài chục gigabyte — và vì  $\Delta W$  cộng thẳng vào  $W$  được sau khi huấn luyện, chi phí suy luận bằng đúng không.

Hình 7.7 vẽ cùng phép tính đó dưới dạng diện tích, vì đó là cách nhanh nhất để thấy vì sao con số 1,56% không phải một thủ thuật mà là hệ quả hình học của việc chọn  $r \ll d$ .



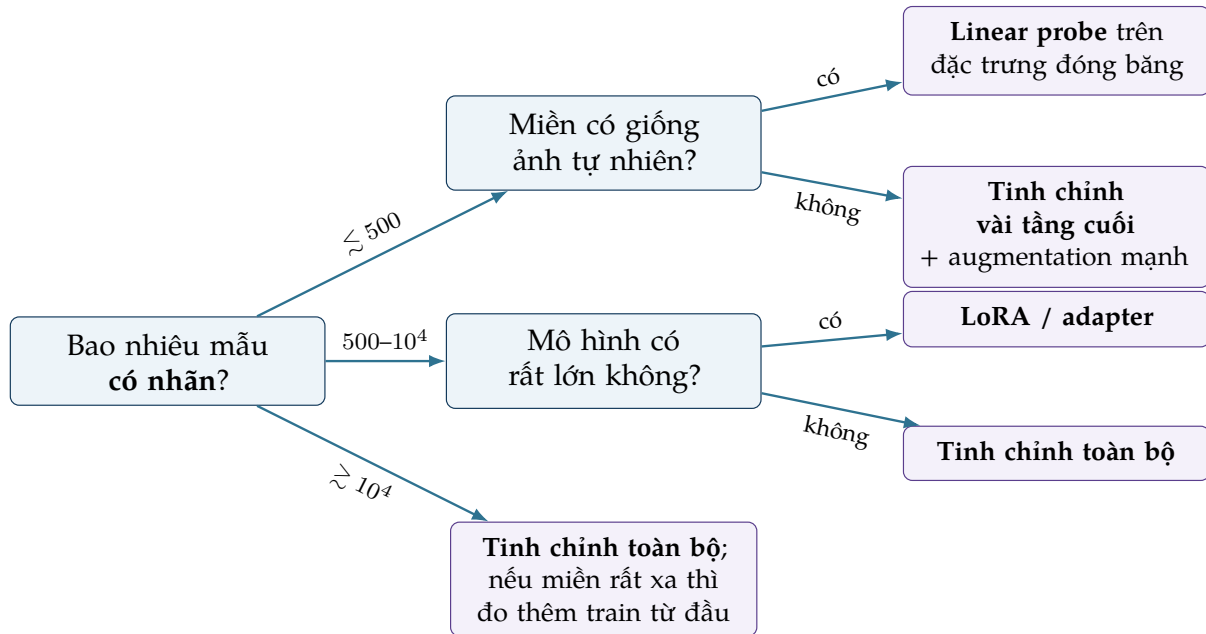
**Hình 7.7.** LoRA thay cập nhật đầy đủ  $\Delta W$  bằng tích hai ma trận hạng thấp. Diện tích hai dải mỏng so với hình vuông lớn chính là tỉ lệ tham số phải học và phải lưu; và vì  $BA$  cộng thẳng vào  $W$  sau khi huấn luyện nên phần tiết kiệm này không phải trả lại ở lúc suy luận. Giả định ngầm là cập nhật cần thiết thật sự nằm trong một không gian con  $r$  chiều.

Giả định ngầm cần gọi tên: LoRA tin rằng *cập nhật cần thiết để thích nghi có hạng thấp*. Điều này hợp lý khi tác vụ đích là biến thể của thứ mô hình đã biết. Nó sai khi bạn đòi mô hình học một năng lực thực sự mới; khi đó hạng thấp thành cổ chai, và triệu chứng là loss huấn luyện dừng ở mức cao hơn hẳn tinh chỉnh toàn bộ bất kể học bao lâu. Phản ứng đúng lúc đó là tăng  $r$  hoặc mở thêm module, không phải tăng *learning rate*.

↔ cùng nguyên lý với  $D/20$  — nén mô hình cũng giả định cái cần giữ nằm trong một không gian con nhỏ; LoRA là phiên bản của giả định đó ở phía thích nghi thay vì ở phía suy luận.

### G. Quyết định cuối cùng vẫn là của bạn: thí nghiệm điểm giao

Mọi thứ ở trên là kiến thức chung. Hình 7.8 nén nó thành một cây quyết định dùng được trong ba mươi giây, và Bảng 7.3 nói rõ từng nhánh hổng ở đâu. Cả hai chỉ thu hẹp không gian tìm kiếm — chúng không kết luận thay bạn.



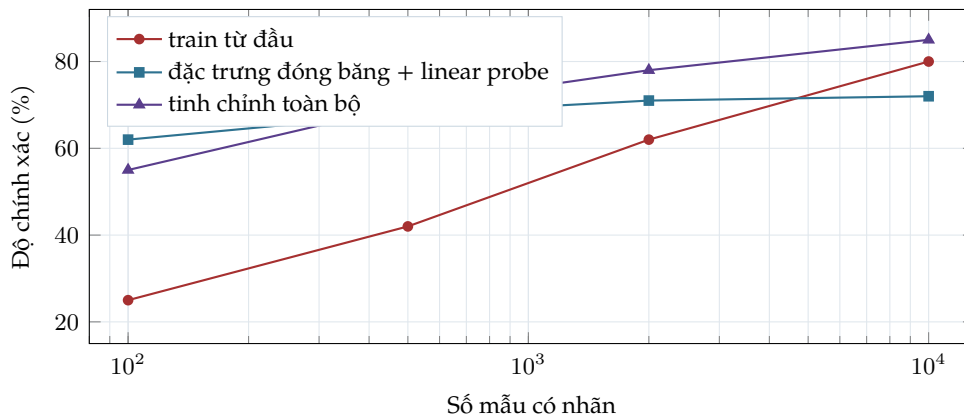
**Hình 7.8.** Cây quyết định chọn chế độ thích nghi. Các ngưỡng là điểm khởi đầu chứ không phải hằng số phổ quát — thí nghiệm điểm giao ở Hình 7.9 là cách xác định ngưỡng thật cho bài toán của bạn.

**Bảng 7.3.** Năm chế độ thích nghi, xếp theo số tham số được phép thay đổi.

| Chế độ                   | Cơ chế                                          | Giả định                                     | Hỏng khi nào                                                          |
|--------------------------|-------------------------------------------------|----------------------------------------------|-----------------------------------------------------------------------|
| Train từ đầu             | Học mọi thứ từ dữ liệu của bạn                  | Dữ liệu đủ để học lại cả thống kê ảnh cơ bản | Dưới vài chục nghìn mẫu: gần như luôn thua                            |
| Linear probe             | Chỉ học một siêu phẳng trên đặc trưng đóng băng | Thông tin cần thiết đã tuyến tính tách được  | Tác vụ cần định vị chính xác, hoặc đặc trưng không có sẵn trong nguồn |
| Tinh chỉnh vài tầng cuối | Mở phần chuyên biệt của nguồn                   | Khác biệt miền nằm ở tầng cao                | Khác biệt nằm ở tầng thấp (ảnh nhiệt, radar, đa phổ)                  |
| LoRA / adapter           | Học cập nhật hạng thấp, gốc đóng băng           | Cập nhật cần thiết có hạng thấp              | Cần năng lực thực sự mới: loss chứng ở mức cao                        |
| Tinh chỉnh toàn bộ       | Mở toàn bộ                                      | Đủ dữ liệu để không quên đặc trưng gốc       | Dữ liệu ít: overfit và quên thảm khốc                                 |

Thí nghiệm đáng làm nhất của cả chương là thí nghiệm điểm giao: chạy ba chế độ — train từ đầu, đóng băng, tinh chỉnh toàn bộ — ở các mức 100, 500, 2000, 10 000 mẫu, mỗi mức lặp ba *seed*, rồi vẽ ba đường cong trên một hình. Hình 7.9 minh họa hình dạng thường thấy, với cảnh báo rõ ràng rằng các con số là *minh họa*, không phải kết quả đo của một tập dữ liệu cụ thể.





**Hình 7.9.** Hình dạng điển hình của thí nghiệm điểm giao (số liệu minh họa, không phải kết quả đo). Đường đồng bằng bão hoà sớm nhưng thắng ở vùng dữ liệu rất ít; tinh chỉnh toàn bộ vượt lên sau điểm giao thứ nhất; train từ đầu chỉ cạnh tranh ở vùng dữ liệu lớn. Vị trí hai điểm giao khác nhau cho từng bài toán — đó chính là thứ bạn phải đo.

Hai điểm giao trên hình là hai con số đáng giá hơn mọi lời khuyên chung, vì chúng trả lời đúng câu hỏi bạn thực sự có: *với lượng nhãn tôi đang có, và với lượng nhãn tôi mua thêm được trong quý này, chế độ nào đúng?* Và vì trục hoành là log, chỉ bốn điểm đo đã đủ để nhìn ra hình dạng.

#### H. Giới hạn và trường hợp hỏng

- **Scaling law là quan hệ khớp thực nghiệm, không phải định luật.** Nó chỉ đúng *trong công thức huấn luyện đã dùng để khớp nó*: đổi kiến trúc, đổi lịch *learning rate*, đổi phân bố dữ liệu là các hằng số thay đổi và đường ngoại suy trượt. Đã có ít nhất một lần cả ngành ngoại suy sai vì công thức khớp có một chi tiết lịch học không nhất quán — chính là câu chuyện Chinchilla.
- **Scaling law mô tả loss, không mô tả năng lực.** Loss giảm mượt trong khi các năng lực rời rạc (đếm, suy luận nhiều bước) xuất hiện không mượt, nên đường cong đẹp không cho phép hứa hẹn gì về hành vi ở tác vụ đích.
- **LoRA dễ bị dùng để né việc chẩn đoán.** Khi kết quả kém, người ta tăng hạng thay vì hỏi liệu bài toán có thực sự là bài toán thích nghi hay không.
- **So sánh chỉ có nghĩa khi công sức tune ngang nhau.** So một *baseline* chưa tune với phương pháp đã tune là dạng thí nghiệm phổ biến nhất và vô giá trị nhất trong ngành [146].

#### CẠM BẤY

Ba lỗi hay đi cùng nhau khi chạy thí nghiệm điểm giao. **Một:** dùng cùng một tập *validation* để chọn siêu tham số cho cả ba chế độ rồi báo cáo trên chính tập đó — điểm giao dịch chuyển theo hướng có lợi cho chế độ được tune nhiều nhất. **Hai:** chạy một *seed* mỗi điểm; với 100 mẫu, độ lệch giữa các *seed* thường lớn hơn khoảng cách giữa hai chế độ, nên đường cong bạn vẽ là nhiễu. **Ba:** quên rằng *backbone* đã tiền huấn luyện có thể đã nhìn thấy dữ liệu đích của bạn — nếu ảnh của bạn từng ở trên internet, “*zero-shot*” không còn là *zero-shot*.

#### CẦN NHỚ

Compute, tham số và dữ liệu là ba mặt của cùng một ngân sách; tối ưu một mặt mà không nhìn hai mặt kia là cách sai tiền có hệ thống. Nhớ kèm hai công thức:  $C \approx 6ND$  để ước lượng hoá đơn, và tỉ lệ  $D/N \approx 20$  như điểm khởi đầu (không phải chân lý). Và mọi quy

tắc chung chỉ là điểm khởi đầu — con số duy nhất ràng buộc quyết định của bạn là điểm giao đo trên chính dữ liệu của bạn. [T2]

## I. Kết nối

Mục này là mặt định lượng của *C/18/01-nguồn-giam-sat*: mục kia nói giám sát lấy ở đâu, mục này nói mua bao nhiêu thì đủ. Nó là tiền đề của *D/20-toi-uu-va-nen-mo-hinh*, vì phần lớn kỹ thuật nén chỉ là cách mua lại compute suy luận bằng compute huấn luyện — cùng một logic ngân sách, chỉ khác cột chi. Nó phụ thuộc rất chặt vào *B/07-chia-du-lieu-va-danh-gia*: không có quy trình chia dữ liệu sạch thì nhiệm bản biến mọi con số ở đây thành vô nghĩa. Và nó đối lập có ích với *B/11-gioi-han-tinh-toan*: chương kia đo giới hạn vật lý của phần cứng, mục này đo giới hạn kinh tế của việc dùng phần cứng đó.

### I Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao Gopher và Chinchilla tốn xấp xỉ cùng số FLOPs, và điều gì trong cách ngành đo lường tiến bộ đã khiến sai lầm này sống sót nhiều năm?
2. Tỷ lệ 20 *token* trên tham số tối ưu cho chi phí gì? Nếu mô hình sẽ phục vụ một tỉ lượt suy luận thì tỉ lệ đúng dịch về phía nào, và vì sao?
3. Số hạng  $L_\infty$  nói gì về việc dùng compute để sửa một bài toán bị đặt sai?
4. LoRA hạng 8 trên ma trận  $1024 \times 1024$  tiết kiệm bao nhiêu phần trăm tham số, và triệu chứng nào cho biết giả định hạng thấp đã bị vi phạm?
5. Vì sao khi dựng lại một tập test theo đúng quy trình cũ thì mọi mô hình đều tụt điểm, và vì sao việc chúng tụt gần như song song lại là tin tốt?
6. Ba cách phổ biến làm cho thí nghiệm điểm giao ra kết quả sai là gì, và sửa từng cái thế nào?

- [1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [2] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [3] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *AISTATS*, 2010.
- [4] K. He, X. Zhang, S. Ren, et al., “Delving Deep into Rectifiers,” in *ICCV*, 2015.
- [5] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [6] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [7] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [8] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [9] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [10] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [11] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” in *ICLR*, 2017.
- [12] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [13] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [14] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [15] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [16] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [17] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [18] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [19] G. Huang, Y. Sun, Z. Liu, et al., “Deep Networks with Stochastic Depth,” in *ECCV*, 2016.
- [20] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [21] H. Zhang, M. Cisse, Y. N. Dauphin, et al., “mixup: Beyond Empirical Risk Minimization,” in *ICLR*, 2018.
- [22] S. Yun, D. Han, S. J. Oh, et al., “CutMix: Regularization Strategy to Train Strong Classifiers,” in *ICCV*, 2019.
- [23] E. D. Cubuk, B. Zoph, J. Shlens, et al., “RandAugment: Practical Automated Data Augmentation,” in *CVPR Workshops*, 2020.

- [24] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [25] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [26] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *NeurIPS*, 2012.
- [27] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in *ICLR*, 2015.
- [28] C. Szegedy, W. Liu, et al., “Going Deeper with Convolutions,” in *CVPR*, 2015.
- [29] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [30] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [31] G. Huang, Z. Liu, L. van der Maaten, et al., “Densely Connected Convolutional Networks,” in *CVPR*, 2017.
- [32] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [33] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [34] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [35] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [36] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [37] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.
- [38] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [39] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [40] J. Su, M. Ahmed, Y. Lu, et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, vol. 568, 2024.
- [41] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” *arXiv:2312.00752*, 2023.
- [42] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [43] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.
- [44] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [45] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [46] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [47] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [48] R. Girshick, J. Donahue, T. Darrell, et al., “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation,” in *CVPR*, 2014.

- [49] R. Girshick, "Fast R-CNN," in *ICCV*, 2015.
- [50] S. Ren, K. He, R. Girshick, et al., "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in *NeurIPS*, 2015.
- [51] T.-Y. Lin, P. Dollár, R. Girshick, et al., "Feature Pyramid Networks for Object Detection," in *CVPR*, 2017.
- [52] K. He, G. Gkioxari, P. Dollár, et al., "Mask R-CNN," in *ICCV*, 2017.
- [53] D. Bolya, S. Foley, J. Hays, et al., "TIDE: A General Toolbox for Identifying Object Detection Errors," in *ECCV*, 2020.
- [54] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [55] J. Redmon, S. Divvala, R. Girshick, et al., "You Only Look Once: Unified, Real-Time Object Detection," in *CVPR*, 2016.
- [56] W. Liu, D. Anguelov, D. Erhan, et al., "SSD: Single Shot MultiBox Detector," in *ECCV*, 2016.
- [57] T.-Y. Lin, P. Goyal, R. Girshick, et al., "Focal Loss for Dense Object Detection," in *ICCV*, 2017.
- [58] Z. Tian, C. Shen, H. Chen, et al., "FCOS: Fully Convolutional One-Stage Object Detection," in *ICCV*, 2019.
- [59] X. Zhou, D. Wang, and P. Krähenbühl, "Objects as Points," *arXiv:1904.07850*, 2019.
- [60] S. Zhang, C. Chi, Y. Yao, et al., "Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection," in *CVPR*, 2020.
- [61] Z. Ge, S. Liu, Z. Li, et al., "OTA: Optimal Transport Assignment for Object Detection," in *CVPR*, 2021.
- [62] N. Bodla, B. Singh, R. Chellappa, et al., "Soft-NMS: Improving Object Detection with One Line of Code," in *ICCV*, 2017.
- [63] X. Zhu, W. Su, L. Lu, et al., "Deformable DETR: Deformable Transformers for End-to-End Object Detection," in *ICLR*, 2021.
- [64] H. Zhang, F. Li, S. Liu, et al., "DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection," in *ICLR*, 2023.
- [65] J. Long, E. Shelhamer, and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," in *CVPR*, 2015.
- [66] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *MICCAI*, 2015.
- [67] L.-C. Chen, G. Papandreou, I. Kokkinos, et al., "DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs," *IEEE TPAMI*, vol. 40, no. 4, pp. 834–848, 2018.
- [68] E. Xie, W. Wang, Z. Yu, et al., "SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers," in *NeurIPS*, 2021.
- [69] B. Cheng, I. Misra, A. G. Schwing, et al., "Masked-Attention Mask Transformer for Universal Image Segmentation," in *CVPR*, 2022.
- [70] A. Kirillov, K. He, R. Girshick, et al., "Panoptic Segmentation," in *CVPR*, 2019.
- [71] A. Kirillov, E. Mintun, N. Ravi, et al., "Segment Anything," in *ICCV*, 2023.
- [72] K. Sun, B. Xiao, D. Liu, et al., "Deep High-Resolution Representation Learning for Human Pose Estimation," in *CVPR*, 2019.

- [73] Z. Cao, T. Simon, S.-E. Wei, et al., “Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields,” in *CVPR*, 2017.
- [74] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [75] J. Deng, J. Guo, N. Xue, et al., “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *CVPR*, 2019.
- [76] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [77] K. Xu, J. Ba, R. Kiros, et al., “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention,” in *ICML*, 2015.
- [78] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *ICLR*, 2015.
- [79] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [80] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [81] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [82] K. Simonyan and A. Zisserman, “Two-Stream Convolutional Networks for Action Recognition in Videos,” in *NeurIPS*, 2014.
- [83] J. Carreira and A. Zisserman, “Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset,” in *CVPR*, 2017.
- [84] D. Tran, H. Wang, L. Torresani, et al., “A Closer Look at Spatiotemporal Convolutions for Action Recognition,” in *CVPR*, 2018.
- [85] C. Feichtenhofer, H. Fan, J. Malik, et al., “SlowFast Networks for Video Recognition,” in *ICCV*, 2019.
- [86] A. Dosovitskiy, P. Fischer, E. Ilg, et al., “FlowNet: Learning Optical Flow with Convolutional Networks,” in *ICCV*, 2015.
- [87] D. Sun, X. Yang, M.-Y. Liu, et al., “PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume,” in *CVPR*, 2018.
- [88] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.
- [89] A. Bewley, Z. Ge, L. Ott, et al., “Simple Online and Realtime Tracking,” in *ICIP*, 2016.
- [90] Y. Zhang, P. Sun, Y. Jiang, et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *ECCV*, 2022.
- [91] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [92] K. Bernardin and R. Stiefelhagen, “Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics,” *EURASIP J. Image and Video Processing*, 2008.
- [93] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [94] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [95] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.



- [96] S. Wang, V. Leroy, Y. Cabon, et al., "DUST3R: Geometric 3D Vision Made Easy," in *CVPR*, 2024.
- [97] D. Eigen, C. Puhrsch, and R. Fergus, "Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network," in *NeurIPS*, 2014.
- [98] R. Ranftl, K. Lasinger, D. Hafner, et al., "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer," *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [99] J. Sturm, N. Engelhard, F. Endres, et al., "A Benchmark for the Evaluation of RGB-D SLAM Systems," in *IROS*, 2012.
- [100] C. R. Qi, H. Su, K. Mo, et al., "PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation," in *CVPR*, 2017.
- [101] C. R. Qi, L. Yi, H. Su, et al., "PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space," in *NeurIPS*, 2017.
- [102] A. H. Lang, S. Vora, H. Caesar, et al., "PointPillars: Fast Encoders for Object Detection from Point Clouds," in *CVPR*, 2019.
- [103] T. Müller, A. Evans, C. Schied, et al., "Instant Neural Graphics Primitives with a Multiresolution Hash Encoding," *ACM TOG*, vol. 41, no. 4, 2022.
- [104] B. Kerbl, G. Kopanas, T. Leimkühler, et al., "3D Gaussian Splatting for Real-Time Radiance Field Rendering," *ACM TOG*, vol. 42, no. 4, 2023.
- [105] R. Zhang, P. Isola, A. A. Efros, et al., "The Unreasonable Effectiveness of Deep Features as a Perceptual Metric," in *CVPR*, 2018.
- [106] D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," in *ICLR*, 2014.
- [107] I. Goodfellow, J. Pouget-Abadie, M. Mirza, et al., "Generative Adversarial Nets," in *NeurIPS*, 2014.
- [108] T. Karras, S. Laine, and T. Aila, "A Style-Based Generator Architecture for Generative Adversarial Networks," in *CVPR*, 2019.
- [109] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," in *NeurIPS*, 2020.
- [110] Y. Song, J. Sohl-Dickstein, D. P. Kingma, et al., "Score-Based Generative Modeling Through Stochastic Differential Equations," in *ICLR*, 2021.
- [111] J. Johnson, A. Alahi, and L. Fei-Fei, "Perceptual Losses for Real-Time Style Transfer and Super-Resolution," in *ECCV*, 2016.
- [112] P. Isola, J.-Y. Zhu, T. Zhou, et al., "Image-to-Image Translation with Conditional Adversarial Networks," in *CVPR*, 2017.
- [113] L. A. Gatys, A. S. Ecker, and M. Bethge, "Image Style Transfer Using Convolutional Neural Networks," in *CVPR*, 2016.
- [114] X. Huang and S. Belongie, "Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization," in *ICCV*, 2017.
- [115] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," in *NeurIPS Deep Learning Workshop*, 2015.
- [116] J. Song, C. Meng, and S. Ermon, "Denoising Diffusion Implicit Models," in *ICLR*, 2021.
- [117] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, et al., "Flow Matching for Generative Modeling," in *ICLR*, 2023.
- [118] J. Ho and T. Salimans, "Classifier-Free Diffusion Guidance," *arXiv:2207.12598*, 2022.
- [119] R. Rombach, A. Blattmann, D. Lorenz, et al., "High-Resolution Image Synthesis with Latent Diffusion Models," in *CVPR*, 2022.
- [120] W. Peebles and S. Xie, "Scalable Diffusion Models with Transformers," in *ICCV*, 2023.

- [121] L. Zhang, A. Rao, and M. Agrawala, “Adding Conditional Control to Text-to-Image Diffusion Models,” in *ICCV*, 2023.
- [122] Y. Blau and T. Michaeli, “The Perception-Distortion Tradeoff,” in *CVPR*, 2018.
- [123] C. Ledig, L. Theis, F. Huszár, et al., “Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network,” in *CVPR*, 2017.
- [124] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., “Image Quality Assessment: From Error Visibility to Structural Similarity,” *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [125] M. Heusel, H. Ramsauer, T. Unterthiner, et al., “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium,” in *NeurIPS*, 2017.
- [126] T. Brooks, B. Mildenhall, T. Xue, et al., “Unprocessing Images for Learned Raw Denoising,” in *CVPR*, 2019.
- [127] A. Abdelhamed, S. Lin, and M. S. Brown, “A High-Quality Denoising Dataset for Smartphone Cameras,” in *CVPR*, 2018.
- [128] J. Deng, W. Dong, R. Socher, et al., “ImageNet: A Large-Scale Hierarchical Image Database,” in *CVPR*, 2009.
- [129] A. Radford, J. W. Kim, C. Hallacy, et al., “Learning Transferable Visual Models from Natural Language Supervision,” in *ICML*, 2021.
- [130] J. Li, D. Li, S. Savarese, et al., “BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models,” in *ICML*, 2023.
- [131] H. Liu, C. Li, Q. Wu, et al., “Visual Instruction Tuning,” in *NeurIPS*, 2023.
- [132] S. Kornblith, J. Shlens, and Q. V. Le, “Do Better ImageNet Models Transfer Better?” In *CVPR*, 2019.
- [133] T. Chen, S. Kornblith, M. Norouzi, et al., “A Simple Framework for Contrastive Learning of Visual Representations,” in *ICML*, 2020.
- [134] K. He, H. Fan, Y. Wu, et al., “Momentum Contrast for Unsupervised Visual Representation Learning,” in *CVPR*, 2020.
- [135] J.-B. Grill, F. Strub, F. Altché, et al., “Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning,” in *NeurIPS*, 2020.
- [136] M. Caron, H. Touvron, I. Misra, et al., “Emerging Properties in Self-Supervised Vision Transformers,” in *ICCV*, 2021.
- [137] K. He, X. Chen, S. Xie, et al., “Masked Autoencoders Are Scalable Vision Learners,” in *CVPR*, 2022.
- [138] A. van den Oord, Y. Li, and O. Vinyals, “Representation Learning with Contrastive Predictive Coding,” in *arXiv:1807.03748*, 2018.
- [139] J. Kaplan, S. McCandlish, T. Henighan, et al., “Scaling Laws for Neural Language Models,” *arXiv:2001.08361*, 2020.
- [140] X. Zhai, A. Kolesnikov, N. Houlsby, et al., “Scaling Vision Transformers,” in *CVPR*, 2022.
- [141] J. Hoffmann, S. Borgeaud, A. Mensch, et al., “Training Compute-Optimal Large Language Models,” in *NeurIPS*, 2022.
- [142] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [143] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [144] N. Houlsby, A. Giurgiu, S. Jastrzebski, et al., “Parameter-Efficient Transfer Learning for NLP,” in *ICML*, 2019.

- [145] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [146] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *RecSys*, 2019.

### 7.3 Thực hành

Phần thực hành của chương này có một đặc điểm khác các chương trước: bạn gần như không huấn luyện gì từ đầu. Công việc chủ yếu là chọn đúng thành phần dựng sẵn, đo xem nó hợp với dữ liệu của mình tới đâu, và biết dừng lại ở chế độ thích nghi rẻ nhất còn đủ tốt.

**Repo và tài nguyên.** *Backbone tự giám sát:* [facebookresearch/dinov3](https://github.com/facebookresearch/dinov3) (có cả bản cho ảnh vệ tinh; giấy phép riêng — dùng DINOv2 nếu bắt buộc Apache 2.0), [facebookresearch/mae](https://github.com/facebookresearch/mae), [lightly-ai/lightly](https://github.com/lightly-ai/lightly) và [lightly-train](https://github.com/lightly-ai/lightly-train) khi muốn tự tiền huấn luyện trên dữ liệu của chính mình. *Thị giác-ngôn ngữ:* [mlfoundations/open\\_clip](https://github.com/mlfoundations/open_clip), SigLIP. *VLM mở đánh thử tại thời điểm viết:* Qwen3-VL (mạnh toàn diện, OCR và video tốt), InternVL3 (giấy phép MIT, thoáng nhất nhóm dẫn đầu), Molmo2 của AI2 (mở cả dữ liệu huấn luyện — lựa chọn đúng khi cần tái lập), Pixtral (Apache 2.0). *Thích nghi:* [huggingface/peft](https://github.com/huggingface/peft). Danh sách tên mô hình này chốt tại tháng 9/2026 và sẽ cũ trong 6–12 tháng, trong khi các repo hạ tầng như [peft](https://github.com/huggingface/peft) hay [timm](https://github.com/huggingface/timm) bền hơn nhiều — hãy nhớ tên hạ tầng, tra lại tên mô hình.

**Câu hỏi kiểm tra hiểu.** Với 500 ảnh trong một lĩnh vực rất hẹp, chẳng hạn ảnh nội soi, thì *linear probe* trên DINOv3 hay tinh chỉnh toàn bộ một ResNet ImageNet sẽ thắng — và lập luận của bạn dựa trên đại lượng nào? Vì sao “mở trọng số” không đồng nghĩa với “mở”, và điều đó ràng buộc thế nào tới việc đưa mô hình vào sản phẩm? Nếu phải giải thích cho một đồng nghiệp trong ba câu vì sao BYOL không sụp dù không có *negative*, ba câu đó là gì?

**Mini project (vài giờ đến hai ngày).** Chạy thí nghiệm điểm giao trên một bài toán phân loại nhỏ của riêng bạn: ba chế độ train từ đầu, đóng băng và tinh chỉnh toàn bộ, ở bốn mức 100, 500, 2000 và 10 000 mẫu, mỗi ô ba *seed*. Vẽ ba đường cong trên một hình kèm dải phương sai giữa các *seed*. Sản phẩm cuối không phải cái hình, mà là hai con số: hai hoành độ điểm giao — ngưỡng quyết định cho chính bài toán của bạn.

#### Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

| Thuật ngữ / câu hỏi                                | Trả lời                                                                                                                                                                             |
|----------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ●●● Học chuyển giao                                | Lấy trọng số đã học trên một tập lớn làm điểm khởi đầu thay vì khởi tạo ngẫu nhiên. Đánh thử đầu tiên; hòng khi vật lý tạo ảnh của miền đích khác nguồn, ví dụ ảnh nhiệt hay radar. |
| ●●● Linear probe                                   | Đóng băng backbone, chỉ huấn luyện một tầng tuyến tính trên đặc trưng. Phép thử rẻ nhất xem đặc trưng có chứa thông tin cần thiết; che giấu việc mô hình định vị kém.               |
| ●● Vì sao mở nhiều tham số lại hại khi ít dữ liệu? | Số tham số được phép thay đổi phải tương xứng với lượng thông tin mới. Vài trăm mẫu không trả nổi cho cả mạng, nên bạn nhận overfitting và quên thảm khốc.                          |
| ●●● Học tự giám sát                                | Tạo bài toán phụ mà đáp án suy ra từ chính bức ảnh, nên không cần nhãn người. Cái giá là bài toán phụ có nghiệm tầm thường phải chống.                                              |
| ●●● Sụp biểu diễn                                  | Mô hình xuất ra cùng một vector cho mọi ảnh, thoả mọi ràng buộc “giống nhau” mà không học gì. Chống nó là toàn bộ nội dung kỹ thuật của tự giám sát.                                |

| Thuật ngữ / câu hỏi                                                 | Trả lời (tiếp)                                                                                                                                                                       |
|---------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ●● Vì sao contrastive cần rất nhiều negative?                       | Lực đẩy từ negative chính là cơ chế chống sụp duy nhất của nó. Batch nhỏ thì tín hiệu đẩy yếu; dữ liệu ít lớp thì negative hoá ra là positive.                                       |
| ●● Masked modeling                                                  | Che một phần ảnh rồi bắt mô hình khôi phục. Không cần chống sụp, nhưng đặc trưng thô nên linear probe kém — phải fine-tune mới mạnh.                                                 |
| ●● Hạ nhiệt độ trong InfoNCE dồn gradient đi đâu?                   | Về những cặp âm có độ tương đồng gần với cặp dương. Vặn quá thấp thì chỉ học từ vài cặp khó nhất và rất nhạy với ảnh gần trùng nhau.                                                 |
| ●●● CLIP                                                            | Tương phản giữa ảnh và chú thích của chính nó, trên ma trận $N \times N$ trong một batch. Cho tập nhãn mở và zero-shot; yếu ở quan hệ không gian, đếm và phủ định.                   |
| ●● Vì sao VLM mô tả ảnh trôi chảy nhưng sai vị trí?                 | Vì khối kết nối nén nhiều token thì giác xuống ít token truy vấn. Bước nén đó rẻ nhưng vứt mất thông tin định vị.                                                                    |
| ●●● Mô hình nền tảng                                                | Mô hình lớn tiền huấn luyện trên dữ liệu quy mô rất lớn, dùng lại được cho nhiều bài toán. Thường thắng cả mạng train từ đầu ngay trong lĩnh vực hẹp.                                |
| ●● Vì sao chi phí huấn luyện xấp xỉ $6ND$ ?                         | Hai FLOPs cho mỗi phép nhân-cộng, nhân ba lượt: một lượt xuôi và hai lượt ngược. Hệ số 6 chỉ đúng cho transformer huấn luyện chuẩn.                                                  |
| ●●● Scaling law                                                     | Quan hệ lũy thừa khớp từ thực nghiệm giữa loss, tham số, dữ liệu và compute. Cho phép ngoại suy từ thí nghiệm rẻ; trượt ngay khi bạn đổi kiến trúc hoặc lịch learning rate.          |
| ●●● Chinchilla                                                      | Kết quả cho thấy phần lớn mô hình lớn thiếu dữ liệu chứ không thiếu tham số; quy tắc ngón tay cái là 20 token cho mỗi tham số. Chỉ tối ưu cho chi phí huấn luyện.                    |
| ●●● Contamination                                                   | Dữ liệu test lọt vào corpus tiền huấn luyện, nên điểm số đo trí nhớ chứ không đo tổng quát hoá. Không kiểm chứng được, vì corpus thường không công bố.                               |
| ●●● LoRA                                                            | Học cập nhật hạng thấp $\Delta W = BA$ , gốc đóng băng; hạng 8 trên ma trận $1024 \times 1024$ chỉ tốn 1,56% tham số. Cộng thẳng vào $W$ sau huấn luyện nên suy luận không đắt thêm. |
| ●● Loss huấn luyện chứng cao hơn hẳn full fine-tune dù học rất lâu? | Giả định hạng thấp của LoRA đã bị vi phạm: bạn đang đòi một năng lực mới chứ không phải thích nghi. Tăng $r$ hoặc mở thêm module, đừng tăng learning rate.                           |
| ●● Zero-shot đẹp trên benchmark nhưng tệ trên dữ liệu của bạn?      | Nghi contamination trước tiên, rồi nghi khoảng cách miền. Cách duy nhất kết luận được là một tập đánh giá bạn tự thu sau ngày mô hình ra đời.                                        |

●●● phải thuộc    ●● nên nắm    ● biết là có

## I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

| Công trình / hệ thống            | Dùng ý tưởng nào của chương                                                                                                     | Ghi chú                                                                                     |
|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| DINOv2 / DINOv3                  | Tự chưng cất không cần negative, dùng đúng cơ chế bất đối xứng teacher–student ở mục 18.1; đặc trưng đồng bằng cho linear probe | Hạ tầng đặc trưng phổ biến nhất cho thị giác 2024–2026; DINOv3 có bản riêng cho ảnh vệ tinh |
| OpenCLIP / SigLIP                | Tương phản chéo phương thức ảnh–văn bản, đúng ý tưởng “ngôn ngữ là giám sát ngữ nghĩa rẻ nhất”                                  | Bộ mã hoá được dùng lại trong gần như mọi VLM mở hiện nay                                   |
| LLaVA, Qwen-VL, InternVL         | Khối kết nối nối bộ mã hoá thị giác vào LLM — chính cuộc đánh đổi số token và thông tin định vị ở mục 18.1                      | Kiến trúc ba khối này ổn định từ 2023 tới nay dù tên mô hình đổi liên tục                   |
| <a href="#">huggingface/peft</a> | LoRA và adapter, tức giả định cập nhật thích nghi có hạng thấp                                                                  | Hạ tầng; là cách mặc định để tinh chỉnh mô hình lớn từ 2023                                 |

| Công trình / hệ thống | Dùng ý tưởng nào của chương                                                                                                   | Ghi chú                                                                             |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Mô hình mở gần đây    | Tỉ lệ dữ liệu trên tham số vượt xa 20, đúng như lập luận tối ưu <i>tổng chi phí vòng đời</i> chứ không chỉ chi phí huấn luyện | Được cho là động cơ chính; tôi không kiểm chứng được con số cụ thể của từng mô hình |

#### MINI PROJECT SLAM · Điểm giao trên mảnh ảnh quanh đặc trưng EuRoC · 1–2 ngày

|          |                                                                                                                                                                                                                                                                                                       |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Mục tiêu | trả lời bằng số, trên chính dữ liệu của bạn, câu hỏi “bao nhiêu mẫu thì tinh chỉnh vượt đặc trưng đóng băng”. Bảng bốn ô chỉ thu hẹp không gian; con số này mới quyết định.                                                                                                                           |
| Dữ liệu  | EuRoC MH_01 và V1_02 để dựng tập, V2_03 giữ riêng để đánh giá. Cắt mảnh $32 \times 32$ quanh các đặc trưng ORB do chính frontend của bạn phát hiện; nhãn “cùng điểm bản đồ / khác điểm” lấy miễn phí từ liên kết dữ liệu trong bản đồ sau khi chạy hệ đầy đủ, không cần ai gán tay.                   |
| Các bước | dựng tập ở bốn mức 100, 500, 2000 và 10 000 cặp. Chạy ba chế độ trên cùng bốn mức: một CNN nhỏ huấn luyện từ đầu; <i>linear probe</i> trên DINOv2 đóng băng; tinh chỉnh toàn bộ chính <i>backbone</i> đó. Ba <i>seed</i> mỗi ô, tổng 36 lần chạy ngắn. Đo độ chính xác ghép cặp trên chuỗi giữ riêng. |
| Xong khi | bạn vẽ được ba đường trên một hình kèm dải phương sai giữa các <i>seed</i> , và ghi ra được <i>hai con số</i> : hai hoành độ nơi thứ hạng đảo chiều. Nếu dải phương sai chồng lên nhau tại điểm giao, kết luận đúng là “chưa phân giải được” và bạn nói được cần bao nhiêu <i>seed</i> nữa.           |
| Nối với  | C/18 cho bảng bốn ô và hình điểm giao; B/07 cho lý do phải giữ riêng cả một chuỗi chứ không chỉ giữ riêng vài khung hình. Tập mảnh ảnh dựng ở đây được dùng lại nguyên vẹn ở mini project chương 19 và 20.                                                                                            |

#### TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Với 500 ảnh, *linear probe* trên đặc trưng nền tảng thường thắng, vì mở toàn bộ tham số của một ResNet với vài trăm mẫu là mời gọi *overfitting* và quên thảm khốc. Ở 50 000 mẫu chiều đảo lại: lúc đó dữ liệu đủ để *trả tiền* cho số tham số được mở, nên tinh chỉnh toàn bộ vượt lên — đó chính là điểm giao thứ nhất ở Hình 7.9.
- A2. Vì nghiệm hằng số bị chặn bằng *cấu trúc*, không bằng hàm mất mát: tương phản thêm lực đẩy từ negative, tự chưng cất dựng bất đối xứng teacher EMA cộng stop-gradient, còn masked modeling đặt mục tiêu khôi phục pixel vốn không có nghiệm hằng. Bỏ đúng thành phần chống sụp của mỗi họ là mô hình sụp thật.
- A3. Vì nhãn ImageNet là hàm vào một tập 1000 phần tử cố định, còn chú thích là hàm vào một không gian ngữ nghĩa mở. Chú thích nhiều hơn theo từng mẫu, nhưng nó phủ được vô hạn khái niệm và đi kèm quan hệ giữa các khái niệm — thứ mà một tập nhãn đóng không bao giờ có.
- A4. Số tham số. Đó là đại lượng dễ công bố và dễ gây ấn tượng, nên nó bị tối ưu thay cho đại lượng đúng; lượng token thì không ai khoe. Với cùng  $C \approx 6ND$ , Chinchilla chọn  $D/N = 20$  thay vì 1,07 và thắng rõ rệt.
- A5. Vì vài trăm tới vài nghìn ảnh chuyên ngành không đủ để học lại những thống kê ảnh cơ bản mà 140 triệu ảnh đã học hộ. Đặc trưng nền tảng cung cấp phần chung, còn phần chuyên ngành chỉ cần một đầu tuyến tính — trừ khi vật lý tạo ảnh khác hẳn, lúc đó lập luận này sụp.

#### NÓI LẠI THẬT ĐƠN GIẢN

Bạn mở một xưởng và cần một người phân loại ốc vít. Cách cũ là thuê một người chưa biết gì rồi ngồi cạnh, cầm từng con ốc lên và nói tên nó, hàng nghìn lần. Rất tốn công, và bạn phải trả tiền cho từng lần nói tên.

Mẹo của mười năm qua là: đừng tuyển người chưa biết gì. Hãy nhận một người đã nhìn hàng trăm triệu bức ảnh đủ loại trong đời — xe cộ, cây cối, mặt người, nhà cửa — dù chưa từng thấy con ốc



nào của bạn. Người đó đã có sẵn con mắt: biết thế nào là một cạnh, một vết xước, hai vật giống nhau. Bạn chỉ còn phải dạy tên gọi, và việc đó xong trong nửa buổi thay vì ba tháng.

Cái giá không nằm ở công sức của bạn mà nằm ở chỗ khác. Con mắt ấy được rèn bằng hàng trăm triệu bức ảnh và một hoá đơn tiền điện khổng lồ; chỉ là người khác đã trả hộ, và bạn không biết họ đã cho nó xem những gì. Nên khi ảnh của bạn chụp bằng máy hồng ngoại chứ không phải máy ảnh thường, kinh nghiệm kia bỗng vô dụng và không ai báo trước cho bạn.

Nửa sau của chương là một bài học về tiền: cả ngành từng tin máy càng to càng giỏi, rồi phát hiện phần lớn máy to bị bỏ đói — cùng số tiền điện, một cái máy nhỏ hơn nhưng được xem nhiều ảnh hơn thì giỏi hơn hẳn.

**Chỗ không rút gọn được:** vì sao con mắt rèn trên ảnh đời thường lại dùng tốt cho ảnh nội soi hay ảnh vệ tinh thì tới nay chưa ai giải thích gọn mà không sai; mọi cách nói đơn giản đều bỏ quên phần ví dụ là ảnh radar, nơi nó gần như không chuyển được.

**Một câu:** *Đừng dạy máy nhìn từ đâu — hãy mượn một con mắt đã được rèn bằng dữ liệu và tiền điện của người khác, rồi chỉ dạy nó tên gọi trong nghề của bạn.*

## Tổng kết chương

Chương này kể một chuyện động lịch sử. Giám sát từng là thứ đắt nhất trong hệ thống; ngành đã tìm ra bốn cách lấy nó gần như miễn phí. Cái giá mỗi lần là tín hiệu gián tiếp hơn và khó kiểm chứng hơn. Kết quả tích lũy là mô hình nền tảng. Nó đổi vai trò của bạn từ người huấn luyện sang người tích hợp. Câu hỏi trung tâm đổi theo: không còn là “mô hình này có đúng không”, mà là “tôi có biết nó giả định gì và hỏng ở đâu không”.

Mặt định lượng gọn hơn nhưng tổn kém hơn nếu làm sai: compute, tham số và dữ liệu là ba mặt của một ngân sách; bài học Chinchilla là ví dụ đắt tiền nhất trong lịch sử ngành về việc tối ưu một mặt mà quên hai mặt kia; và mọi con số chung, kể cả tỉ lệ 20 *token* trên tham số, chỉ là điểm khởi đầu cho phép đo của riêng bạn.

Hai chỗ chương này để ngỏ và được nhận tiếp ở nơi khác. Thứ nhất, làm sao *đánh giá* một mô hình nền tảng cho tử tế khi bạn không biết nó đã thấy gì — đó là E/24-thi-nghiem-va-ablation và E/25-do-tin-cay-va-van-hanh. Thứ hai, một khi đã chọn được mô hình thì làm sao ép nó vào ngân sách phần cứng thật. Đó là toàn bộ Phần D, bắt đầu ở D/20-toi-uu-va-nen-mo-hinh. Ở đó cùng một logic ngân sách xuất hiện lại, chỉ khác là cột chi đổi từ compute huấn luyện sang compute **suy luận** (*inference*).

## ▮ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Mục 1 xếp các nguồn giám sát theo độ rẻ, mục 2 xếp chi phí theo compute và dữ liệu. Ghép hai trục lại: nguồn giám sát rẻ nhất có phải nguồn có tổng chi phí thấp nhất không, và khoản chi bị bỏ sót nằm ở đâu?
2. Augmentation của học đối lập ép một tập bất biến vào biểu diễn, còn định luật scaling nói cứ thêm dữ liệu là loss giảm. Với một tác vụ mà bất biến bị ép sai, thêm dữ liệu pretraining có cứu được không, và số hạng nào trong công thức scaling mô tả đúng chỗ không cứu được?
3. Bạn nhận một backbone mở trọng số và định dùng cho ảnh y tế. Ba câu hỏi phải trả lời trước — một về nguồn giám sát, một về nhiệm vụ dữ liệu, một về giấy phép — là gì, và câu nào bạn thật sự kiểm chứng được bằng thí nghiệm?
4. Vì sao thí nghiệm tìm điểm giao giữa linear probe và fine-tune lại dễ ra kết quả sai, và trong các cách làm sai thường gặp, cách nào không lộ ra nếu chỉ nhìn bảng kết quả cuối?
5. LoRA giả định ma trận cập nhật có hạng thấp, còn mục 1 lập luận theo lượng thông tin mới so với số tham số được phép thay đổi. Hai lập luận đó gặp nhau ở đâu, và triệu chứng nào cho biết hạng bạn chọn quá nhỏ so với lượng thông tin mới?



6. Chương nói bạn đã đổi vai từ người huấn luyện thành người tích hợp. Việc đổi vai đó làm loại lỗi nào khó phát hiện hơn hẳn so với thời tự huấn luyện, và bạn bù lại bằng phép đo nào?

- [1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [2] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [3] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *AISTATS*, 2010.
- [4] K. He, X. Zhang, S. Ren, et al., “Delving Deep into Rectifiers,” in *ICCV*, 2015.
- [5] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [6] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [7] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [8] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [9] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [10] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [11] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” in *ICLR*, 2017.
- [12] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [13] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [14] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [15] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [16] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [17] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [18] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [19] G. Huang, Y. Sun, Z. Liu, et al., “Deep Networks with Stochastic Depth,” in *ECCV*, 2016.
- [20] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [21] H. Zhang, M. Cisse, Y. N. Dauphin, et al., “mixup: Beyond Empirical Risk Minimization,” in *ICLR*, 2018.
- [22] S. Yun, D. Han, S. J. Oh, et al., “CutMix: Regularization Strategy to Train Strong Classifiers,” in *ICCV*, 2019.
- [23] E. D. Cubuk, B. Zoph, J. Shlens, et al., “RandAugment: Practical Automated Data Augmentation,” in *CVPR Workshops*, 2020.

- [24] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [25] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [26] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *NeurIPS*, 2012.
- [27] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in *ICLR*, 2015.
- [28] C. Szegedy, W. Liu, et al., “Going Deeper with Convolutions,” in *CVPR*, 2015.
- [29] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [30] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [31] G. Huang, Z. Liu, L. van der Maaten, et al., “Densely Connected Convolutional Networks,” in *CVPR*, 2017.
- [32] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [33] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [34] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [35] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [36] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [37] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.
- [38] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [39] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [40] J. Su, M. Ahmed, Y. Lu, et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, vol. 568, 2024.
- [41] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” *arXiv:2312.00752*, 2023.
- [42] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [43] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.
- [44] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [45] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [46] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [47] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [48] R. Girshick, J. Donahue, T. Darrell, et al., “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation,” in *CVPR*, 2014.

- [49] R. Girshick, "Fast R-CNN," in *ICCV*, 2015.
- [50] S. Ren, K. He, R. Girshick, et al., "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in *NeurIPS*, 2015.
- [51] T.-Y. Lin, P. Dollár, R. Girshick, et al., "Feature Pyramid Networks for Object Detection," in *CVPR*, 2017.
- [52] K. He, G. Gkioxari, P. Dollár, et al., "Mask R-CNN," in *ICCV*, 2017.
- [53] D. Bolya, S. Foley, J. Hays, et al., "TIDE: A General Toolbox for Identifying Object Detection Errors," in *ECCV*, 2020.
- [54] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [55] J. Redmon, S. Divvala, R. Girshick, et al., "You Only Look Once: Unified, Real-Time Object Detection," in *CVPR*, 2016.
- [56] W. Liu, D. Anguelov, D. Erhan, et al., "SSD: Single Shot MultiBox Detector," in *ECCV*, 2016.
- [57] T.-Y. Lin, P. Goyal, R. Girshick, et al., "Focal Loss for Dense Object Detection," in *ICCV*, 2017.
- [58] Z. Tian, C. Shen, H. Chen, et al., "FCOS: Fully Convolutional One-Stage Object Detection," in *ICCV*, 2019.
- [59] X. Zhou, D. Wang, and P. Krähenbühl, "Objects as Points," *arXiv:1904.07850*, 2019.
- [60] S. Zhang, C. Chi, Y. Yao, et al., "Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection," in *CVPR*, 2020.
- [61] Z. Ge, S. Liu, Z. Li, et al., "OTA: Optimal Transport Assignment for Object Detection," in *CVPR*, 2021.
- [62] N. Bodla, B. Singh, R. Chellappa, et al., "Soft-NMS: Improving Object Detection with One Line of Code," in *ICCV*, 2017.
- [63] X. Zhu, W. Su, L. Lu, et al., "Deformable DETR: Deformable Transformers for End-to-End Object Detection," in *ICLR*, 2021.
- [64] H. Zhang, F. Li, S. Liu, et al., "DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection," in *ICLR*, 2023.
- [65] J. Long, E. Shelhamer, and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," in *CVPR*, 2015.
- [66] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *MICCAI*, 2015.
- [67] L.-C. Chen, G. Papandreou, I. Kokkinos, et al., "DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs," *IEEE TPAMI*, vol. 40, no. 4, pp. 834–848, 2018.
- [68] E. Xie, W. Wang, Z. Yu, et al., "SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers," in *NeurIPS*, 2021.
- [69] B. Cheng, I. Misra, A. G. Schwing, et al., "Masked-Attention Mask Transformer for Universal Image Segmentation," in *CVPR*, 2022.
- [70] A. Kirillov, K. He, R. Girshick, et al., "Panoptic Segmentation," in *CVPR*, 2019.
- [71] A. Kirillov, E. Mintun, N. Ravi, et al., "Segment Anything," in *ICCV*, 2023.
- [72] K. Sun, B. Xiao, D. Liu, et al., "Deep High-Resolution Representation Learning for Human Pose Estimation," in *CVPR*, 2019.

- [73] Z. Cao, T. Simon, S.-E. Wei, et al., “Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields,” in *CVPR*, 2017.
- [74] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [75] J. Deng, J. Guo, N. Xue, et al., “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *CVPR*, 2019.
- [76] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [77] K. Xu, J. Ba, R. Kiros, et al., “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention,” in *ICML*, 2015.
- [78] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *ICLR*, 2015.
- [79] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [80] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [81] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [82] K. Simonyan and A. Zisserman, “Two-Stream Convolutional Networks for Action Recognition in Videos,” in *NeurIPS*, 2014.
- [83] J. Carreira and A. Zisserman, “Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset,” in *CVPR*, 2017.
- [84] D. Tran, H. Wang, L. Torresani, et al., “A Closer Look at Spatiotemporal Convolutions for Action Recognition,” in *CVPR*, 2018.
- [85] C. Feichtenhofer, H. Fan, J. Malik, et al., “SlowFast Networks for Video Recognition,” in *ICCV*, 2019.
- [86] A. Dosovitskiy, P. Fischer, E. Ilg, et al., “FlowNet: Learning Optical Flow with Convolutional Networks,” in *ICCV*, 2015.
- [87] D. Sun, X. Yang, M.-Y. Liu, et al., “PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume,” in *CVPR*, 2018.
- [88] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.
- [89] A. Bewley, Z. Ge, L. Ott, et al., “Simple Online and Realtime Tracking,” in *ICIP*, 2016.
- [90] Y. Zhang, P. Sun, Y. Jiang, et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *ECCV*, 2022.
- [91] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [92] K. Bernardin and R. Stiefelhagen, “Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics,” *EURASIP J. Image and Video Processing*, 2008.
- [93] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [94] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [95] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.

- [96] S. Wang, V. Leroy, Y. Cabon, et al., "DUST3R: Geometric 3D Vision Made Easy," in *CVPR*, 2024.
- [97] D. Eigen, C. Puhrsch, and R. Fergus, "Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network," in *NeurIPS*, 2014.
- [98] R. Ranftl, K. Lasinger, D. Hafner, et al., "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer," *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [99] J. Sturm, N. Engelhard, F. Endres, et al., "A Benchmark for the Evaluation of RGB-D SLAM Systems," in *IROS*, 2012.
- [100] C. R. Qi, H. Su, K. Mo, et al., "PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation," in *CVPR*, 2017.
- [101] C. R. Qi, L. Yi, H. Su, et al., "PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space," in *NeurIPS*, 2017.
- [102] A. H. Lang, S. Vora, H. Caesar, et al., "PointPillars: Fast Encoders for Object Detection from Point Clouds," in *CVPR*, 2019.
- [103] T. Müller, A. Evans, C. Schied, et al., "Instant Neural Graphics Primitives with a Multiresolution Hash Encoding," *ACM TOG*, vol. 41, no. 4, 2022.
- [104] B. Kerbl, G. Kopanas, T. Leimkühler, et al., "3D Gaussian Splatting for Real-Time Radiance Field Rendering," *ACM TOG*, vol. 42, no. 4, 2023.
- [105] R. Zhang, P. Isola, A. A. Efros, et al., "The Unreasonable Effectiveness of Deep Features as a Perceptual Metric," in *CVPR*, 2018.
- [106] D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," in *ICLR*, 2014.
- [107] I. Goodfellow, J. Pouget-Abadie, M. Mirza, et al., "Generative Adversarial Nets," in *NeurIPS*, 2014.
- [108] T. Karras, S. Laine, and T. Aila, "A Style-Based Generator Architecture for Generative Adversarial Networks," in *CVPR*, 2019.
- [109] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," in *NeurIPS*, 2020.
- [110] Y. Song, J. Sohl-Dickstein, D. P. Kingma, et al., "Score-Based Generative Modeling Through Stochastic Differential Equations," in *ICLR*, 2021.
- [111] J. Johnson, A. Alahi, and L. Fei-Fei, "Perceptual Losses for Real-Time Style Transfer and Super-Resolution," in *ECCV*, 2016.
- [112] P. Isola, J.-Y. Zhu, T. Zhou, et al., "Image-to-Image Translation with Conditional Adversarial Networks," in *CVPR*, 2017.
- [113] L. A. Gatys, A. S. Ecker, and M. Bethge, "Image Style Transfer Using Convolutional Neural Networks," in *CVPR*, 2016.
- [114] X. Huang and S. Belongie, "Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization," in *ICCV*, 2017.
- [115] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," in *NeurIPS Deep Learning Workshop*, 2015.
- [116] J. Song, C. Meng, and S. Ermon, "Denoising Diffusion Implicit Models," in *ICLR*, 2021.
- [117] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, et al., "Flow Matching for Generative Modeling," in *ICLR*, 2023.
- [118] J. Ho and T. Salimans, "Classifier-Free Diffusion Guidance," *arXiv:2207.12598*, 2022.
- [119] R. Rombach, A. Blattmann, D. Lorenz, et al., "High-Resolution Image Synthesis with Latent Diffusion Models," in *CVPR*, 2022.
- [120] W. Peebles and S. Xie, "Scalable Diffusion Models with Transformers," in *ICCV*, 2023.



- [121] L. Zhang, A. Rao, and M. Agrawala, “Adding Conditional Control to Text-to-Image Diffusion Models,” in *ICCV*, 2023.
- [122] Y. Blau and T. Michaeli, “The Perception-Distortion Tradeoff,” in *CVPR*, 2018.
- [123] C. Ledig, L. Theis, F. Huszár, et al., “Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network,” in *CVPR*, 2017.
- [124] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., “Image Quality Assessment: From Error Visibility to Structural Similarity,” *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [125] M. Heusel, H. Ramsauer, T. Unterthiner, et al., “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium,” in *NeurIPS*, 2017.
- [126] T. Brooks, B. Mildenhall, T. Xue, et al., “Unprocessing Images for Learned Raw Denoising,” in *CVPR*, 2019.
- [127] A. Abdelhamed, S. Lin, and M. S. Brown, “A High-Quality Denoising Dataset for Smartphone Cameras,” in *CVPR*, 2018.
- [128] J. Deng, W. Dong, R. Socher, et al., “ImageNet: A Large-Scale Hierarchical Image Database,” in *CVPR*, 2009.
- [129] A. Radford, J. W. Kim, C. Hallacy, et al., “Learning Transferable Visual Models from Natural Language Supervision,” in *ICML*, 2021.
- [130] J. Li, D. Li, S. Savarese, et al., “BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models,” in *ICML*, 2023.
- [131] H. Liu, C. Li, Q. Wu, et al., “Visual Instruction Tuning,” in *NeurIPS*, 2023.
- [132] S. Kornblith, J. Shlens, and Q. V. Le, “Do Better ImageNet Models Transfer Better?” In *CVPR*, 2019.
- [133] T. Chen, S. Kornblith, M. Norouzi, et al., “A Simple Framework for Contrastive Learning of Visual Representations,” in *ICML*, 2020.
- [134] K. He, H. Fan, Y. Wu, et al., “Momentum Contrast for Unsupervised Visual Representation Learning,” in *CVPR*, 2020.
- [135] J.-B. Grill, F. Strub, F. Altché, et al., “Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning,” in *NeurIPS*, 2020.
- [136] M. Caron, H. Touvron, I. Misra, et al., “Emerging Properties in Self-Supervised Vision Transformers,” in *ICCV*, 2021.
- [137] K. He, X. Chen, S. Xie, et al., “Masked Autoencoders Are Scalable Vision Learners,” in *CVPR*, 2022.
- [138] A. van den Oord, Y. Li, and O. Vinyals, “Representation Learning with Contrastive Predictive Coding,” in *arXiv:1807.03748*, 2018.
- [139] J. Kaplan, S. McCandlish, T. Henighan, et al., “Scaling Laws for Neural Language Models,” *arXiv:2001.08361*, 2020.
- [140] X. Zhai, A. Kolesnikov, N. Houlsby, et al., “Scaling Vision Transformers,” in *CVPR*, 2022.
- [141] J. Hoffmann, S. Borgeaud, A. Mensch, et al., “Training Compute-Optimal Large Language Models,” in *NeurIPS*, 2022.
- [142] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [143] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [144] N. Houlsby, A. Giurgiu, S. Jastrzebski, et al., “Parameter-Efficient Transfer Learning for NLP,” in *ICML*, 2019.

- [145] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [146] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *RecSys*, 2019.

- [1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [2] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [3] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *AISTATS*, 2010.
- [4] K. He, X. Zhang, S. Ren, et al., “Delving Deep into Rectifiers,” in *ICCV*, 2015.
- [5] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [6] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [7] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [8] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [9] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [10] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [11] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” in *ICLR*, 2017.
- [12] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [13] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [14] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [15] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [16] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [17] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [18] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [19] G. Huang, Y. Sun, Z. Liu, et al., “Deep Networks with Stochastic Depth,” in *ECCV*, 2016.
- [20] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [21] H. Zhang, M. Cisse, Y. N. Dauphin, et al., “mixup: Beyond Empirical Risk Minimization,” in *ICLR*, 2018.
- [22] S. Yun, D. Han, S. J. Oh, et al., “CutMix: Regularization Strategy to Train Strong Classifiers,” in *ICCV*, 2019.
- [23] E. D. Cubuk, B. Zoph, J. Shlens, et al., “RandAugment: Practical Automated Data Augmentation,” in *CVPR Workshops*, 2020.

- [24] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [25] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [26] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *NeurIPS*, 2012.
- [27] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in *ICLR*, 2015.
- [28] C. Szegedy, W. Liu, et al., “Going Deeper with Convolutions,” in *CVPR*, 2015.
- [29] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [30] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [31] G. Huang, Z. Liu, L. van der Maaten, et al., “Densely Connected Convolutional Networks,” in *CVPR*, 2017.
- [32] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [33] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [34] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [35] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [36] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [37] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.
- [38] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [39] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [40] J. Su, M. Ahmed, Y. Lu, et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, vol. 568, 2024.
- [41] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” *arXiv:2312.00752*, 2023.
- [42] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [43] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.
- [44] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [45] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [46] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [47] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [48] R. Girshick, J. Donahue, T. Darrell, et al., “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation,” in *CVPR*, 2014.

- [49] R. Girshick, "Fast R-CNN," in *ICCV*, 2015.
- [50] S. Ren, K. He, R. Girshick, et al., "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in *NeurIPS*, 2015.
- [51] T.-Y. Lin, P. Dollár, R. Girshick, et al., "Feature Pyramid Networks for Object Detection," in *CVPR*, 2017.
- [52] K. He, G. Gkioxari, P. Dollár, et al., "Mask R-CNN," in *ICCV*, 2017.
- [53] D. Bolya, S. Foley, J. Hays, et al., "TIDE: A General Toolbox for Identifying Object Detection Errors," in *ECCV*, 2020.
- [54] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [55] J. Redmon, S. Divvala, R. Girshick, et al., "You Only Look Once: Unified, Real-Time Object Detection," in *CVPR*, 2016.
- [56] W. Liu, D. Anguelov, D. Erhan, et al., "SSD: Single Shot MultiBox Detector," in *ECCV*, 2016.
- [57] T.-Y. Lin, P. Goyal, R. Girshick, et al., "Focal Loss for Dense Object Detection," in *ICCV*, 2017.
- [58] Z. Tian, C. Shen, H. Chen, et al., "FCOS: Fully Convolutional One-Stage Object Detection," in *ICCV*, 2019.
- [59] X. Zhou, D. Wang, and P. Krähenbühl, "Objects as Points," *arXiv:1904.07850*, 2019.
- [60] S. Zhang, C. Chi, Y. Yao, et al., "Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection," in *CVPR*, 2020.
- [61] Z. Ge, S. Liu, Z. Li, et al., "OTA: Optimal Transport Assignment for Object Detection," in *CVPR*, 2021.
- [62] N. Bodla, B. Singh, R. Chellappa, et al., "Soft-NMS: Improving Object Detection with One Line of Code," in *ICCV*, 2017.
- [63] X. Zhu, W. Su, L. Lu, et al., "Deformable DETR: Deformable Transformers for End-to-End Object Detection," in *ICLR*, 2021.
- [64] H. Zhang, F. Li, S. Liu, et al., "DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection," in *ICLR*, 2023.
- [65] J. Long, E. Shelhamer, and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," in *CVPR*, 2015.
- [66] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *MICCAI*, 2015.
- [67] L.-C. Chen, G. Papandreou, I. Kokkinos, et al., "DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs," *IEEE TPAMI*, vol. 40, no. 4, pp. 834–848, 2018.
- [68] E. Xie, W. Wang, Z. Yu, et al., "SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers," in *NeurIPS*, 2021.
- [69] B. Cheng, I. Misra, A. G. Schwing, et al., "Masked-Attention Mask Transformer for Universal Image Segmentation," in *CVPR*, 2022.
- [70] A. Kirillov, K. He, R. Girshick, et al., "Panoptic Segmentation," in *CVPR*, 2019.
- [71] A. Kirillov, E. Mintun, N. Ravi, et al., "Segment Anything," in *ICCV*, 2023.
- [72] K. Sun, B. Xiao, D. Liu, et al., "Deep High-Resolution Representation Learning for Human Pose Estimation," in *CVPR*, 2019.

- [73] Z. Cao, T. Simon, S.-E. Wei, et al., “Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields,” in *CVPR*, 2017.
- [74] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [75] J. Deng, J. Guo, N. Xue, et al., “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *CVPR*, 2019.
- [76] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [77] K. Xu, J. Ba, R. Kiros, et al., “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention,” in *ICML*, 2015.
- [78] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *ICLR*, 2015.
- [79] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [80] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [81] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [82] K. Simonyan and A. Zisserman, “Two-Stream Convolutional Networks for Action Recognition in Videos,” in *NeurIPS*, 2014.
- [83] J. Carreira and A. Zisserman, “Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset,” in *CVPR*, 2017.
- [84] D. Tran, H. Wang, L. Torresani, et al., “A Closer Look at Spatiotemporal Convolutions for Action Recognition,” in *CVPR*, 2018.
- [85] C. Feichtenhofer, H. Fan, J. Malik, et al., “SlowFast Networks for Video Recognition,” in *ICCV*, 2019.
- [86] A. Dosovitskiy, P. Fischer, E. Ilg, et al., “FlowNet: Learning Optical Flow with Convolutional Networks,” in *ICCV*, 2015.
- [87] D. Sun, X. Yang, M.-Y. Liu, et al., “PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume,” in *CVPR*, 2018.
- [88] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.
- [89] A. Bewley, Z. Ge, L. Ott, et al., “Simple Online and Realtime Tracking,” in *ICIP*, 2016.
- [90] Y. Zhang, P. Sun, Y. Jiang, et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *ECCV*, 2022.
- [91] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [92] K. Bernardin and R. Stiefelhagen, “Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics,” *EURASIP J. Image and Video Processing*, 2008.
- [93] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [94] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [95] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.



- [96] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *CVPR*, 2024.
- [97] D. Eigen, C. Puhrsch, and R. Fergus, “Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network,” in *NeurIPS*, 2014.
- [98] R. Ranftl, K. Lasinger, D. Hafner, et al., “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer,” *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [99] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IROS*, 2012.
- [100] C. R. Qi, H. Su, K. Mo, et al., “PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation,” in *CVPR*, 2017.
- [101] C. R. Qi, L. Yi, H. Su, et al., “PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space,” in *NeurIPS*, 2017.
- [102] A. H. Lang, S. Vora, H. Caesar, et al., “PointPillars: Fast Encoders for Object Detection from Point Clouds,” in *CVPR*, 2019.
- [103] T. Müller, A. Evans, C. Schied, et al., “Instant Neural Graphics Primitives with a Multiresolution Hash Encoding,” *ACM TOG*, vol. 41, no. 4, 2022.
- [104] B. Kerbl, G. Kopanas, T. Leimkühler, et al., “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *ACM TOG*, vol. 42, no. 4, 2023.
- [105] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.
- [106] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” in *ICLR*, 2014.
- [107] I. Goodfellow, J. Pouget-Abadie, M. Mirza, et al., “Generative Adversarial Nets,” in *NeurIPS*, 2014.
- [108] T. Karras, S. Laine, and T. Aila, “A Style-Based Generator Architecture for Generative Adversarial Networks,” in *CVPR*, 2019.
- [109] J. Ho, A. Jain, and P. Abbeel, “Denoising Diffusion Probabilistic Models,” in *NeurIPS*, 2020.
- [110] Y. Song, J. Sohl-Dickstein, D. P. Kingma, et al., “Score-Based Generative Modeling Through Stochastic Differential Equations,” in *ICLR*, 2021.
- [111] J. Johnson, A. Alahi, and L. Fei-Fei, “Perceptual Losses for Real-Time Style Transfer and Super-Resolution,” in *ECCV*, 2016.
- [112] P. Isola, J.-Y. Zhu, T. Zhou, et al., “Image-to-Image Translation with Conditional Adversarial Networks,” in *CVPR*, 2017.
- [113] L. A. Gatys, A. S. Ecker, and M. Bethge, “Image Style Transfer Using Convolutional Neural Networks,” in *CVPR*, 2016.
- [114] X. Huang and S. Belongie, “Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization,” in *ICCV*, 2017.
- [115] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.
- [116] J. Song, C. Meng, and S. Ermon, “Denoising Diffusion Implicit Models,” in *ICLR*, 2021.
- [117] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, et al., “Flow Matching for Generative Modeling,” in *ICLR*, 2023.
- [118] J. Ho and T. Salimans, “Classifier-Free Diffusion Guidance,” *arXiv:2207.12598*, 2022.
- [119] R. Rombach, A. Blattmann, D. Lorenz, et al., “High-Resolution Image Synthesis with Latent Diffusion Models,” in *CVPR*, 2022.
- [120] W. Peebles and S. Xie, “Scalable Diffusion Models with Transformers,” in *ICCV*, 2023.

- [121] L. Zhang, A. Rao, and M. Agrawala, "Adding Conditional Control to Text-to-Image Diffusion Models," in *ICCV*, 2023.
- [122] Y. Blau and T. Michaeli, "The Perception-Distortion Tradeoff," in *CVPR*, 2018.
- [123] C. Ledig, L. Theis, F. Huszár, et al., "Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network," in *CVPR*, 2017.
- [124] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., "Image Quality Assessment: From Error Visibility to Structural Similarity," *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [125] M. Heusel, H. Ramsauer, T. Unterthiner, et al., "GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium," in *NeurIPS*, 2017.
- [126] T. Brooks, B. Mildenhall, T. Xue, et al., "Unprocessing Images for Learned Raw Denoising," in *CVPR*, 2019.
- [127] A. Abdelhamed, S. Lin, and M. S. Brown, "A High-Quality Denoising Dataset for Smartphone Cameras," in *CVPR*, 2018.
- [128] J. Deng, W. Dong, R. Socher, et al., "ImageNet: A Large-Scale Hierarchical Image Database," in *CVPR*, 2009.
- [129] A. Radford, J. W. Kim, C. Hallacy, et al., "Learning Transferable Visual Models from Natural Language Supervision," in *ICML*, 2021.
- [130] J. Li, D. Li, S. Savarese, et al., "BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models," in *ICML*, 2023.
- [131] H. Liu, C. Li, Q. Wu, et al., "Visual Instruction Tuning," in *NeurIPS*, 2023.
- [132] S. Kornblith, J. Shlens, and Q. V. Le, "Do Better ImageNet Models Transfer Better?" In *CVPR*, 2019.
- [133] T. Chen, S. Kornblith, M. Norouzi, et al., "A Simple Framework for Contrastive Learning of Visual Representations," in *ICML*, 2020.
- [134] K. He, H. Fan, Y. Wu, et al., "Momentum Contrast for Unsupervised Visual Representation Learning," in *CVPR*, 2020.
- [135] J.-B. Grill, F. Strub, F. Altché, et al., "Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning," in *NeurIPS*, 2020.
- [136] M. Caron, H. Touvron, I. Misra, et al., "Emerging Properties in Self-Supervised Vision Transformers," in *ICCV*, 2021.
- [137] K. He, X. Chen, S. Xie, et al., "Masked Autoencoders Are Scalable Vision Learners," in *CVPR*, 2022.
- [138] A. van den Oord, Y. Li, and O. Vinyals, "Representation Learning with Contrastive Predictive Coding," in *arXiv:1807.03748*, 2018.
- [139] J. Kaplan, S. McCandlish, T. Henighan, et al., "Scaling Laws for Neural Language Models," *arXiv:2001.08361*, 2020.
- [140] X. Zhai, A. Kolesnikov, N. Houlsby, et al., "Scaling Vision Transformers," in *CVPR*, 2022.
- [141] J. Hoffmann, S. Borgeaud, A. Mensch, et al., "Training Compute-Optimal Large Language Models," in *NeurIPS*, 2022.
- [142] C. G. Northcutt, A. Athalye, and J. Mueller, "Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks," in *NeurIPS Datasets and Benchmarks*, 2021.
- [143] B. Recht, R. Roelofs, L. Schmidt, et al., "Do ImageNet Classifiers Generalize to ImageNet?" In *ICML*, 2019.
- [144] N. Houlsby, A. Giurgiu, S. Jastrzebski, et al., "Parameter-Efficient Transfer Learning for NLP," in *ICML*, 2019.

- [145] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [146] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *RecSys*, 2019.